EXPERT INSIGHT

Early Access

Architecting ASP.NET Core Applications

An atypical design patterns guide for .NET 8, C# 12, and beyond



Third Edition

Carl-Hugo Marcotte

packt

Architecting ASP.NET Core Applications

Copyright © 2023 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing, and its dealers and distributors will be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Early Access Publication: Architecting ASP.NET Core Applications

Early Access Production Reference: B19826

Published by Packt Publishing Ltd.

Livery Place

35 Livery Street

Birmingham

B3 2PB, UK

ISBN: 978-1-80512-338-5

www.packt.com

Table of Contents

1. Architecting ASP.NET Core Applications, Third Edition: An atypical design patterns guide for .NET 8, C# 12, and beyond
2. 1 Introduction
    I. Before you begin: Join our book community on Discord
    II. What is a design pattern?
    III. Anti-patterns and code smells
        i. Anti-patterns
        ii. Code smells
    IV. Understanding the web – request/response
    V. Getting started with .NET
        i. .NET SDK versus runtime
        ii. .NET 5+ versus .NET Standard
        iii. Visual Studio Code versus Visual Studio versus the command-line interface
        iv. Technical requirements
    VI. Summary
    VII. Questions
    VIII. Further reading
3. 2 Automated Testing
    I. Before you begin: Join our book community on Discord
    II. Introduction to automated testing
        i. Unit testing
        ii. Integration testing
        iii. End-to-end testing
        iv. Other types of tests
        v. Picking the right test style
    III. Testing approaches
        i. TDD
        ii. ATDD
        iii. BDD
        iv. Refactoring
        v. Technical debt
    IV. Testing techniques

    i. White-box testing
    ii. Black-box testing
    iii. Grey-box testing
    iv. White-box vs. Black-box vs. Grey-box testing
    v. Conclusion
V. Test case creation
    i. Equivalence Partitioning
    ii. Boundary Value Analysis
    iii. Decision Table Testing
    iv. State Transition Testing
    v. Use Case Testing
VI. How to create an xUnit test project
VII. Key xUnit features
    i. Facts
    ii. Assertions
    iii. Theories
    iv. Closing words
VIII. Arrange, Act, Assert
IX. Organizing your tests
    i. Unit tests
    ii. Integration tests
X. Writing ASP.NET Core integration tests
    i. Classic web application
    ii. Minimal hosting
XI. Important testing principles
XII. Summary
XIII. Questions
XIV. Further reading
4. 3 Architectural Principles
I. Before you begin: Join our book community on Discord
II. Separation of concerns (SoC)
III. Don’t repeat yourself (DRY)
IV. Keep it simple, stupid (KISS)
V. The SOLID principles
    i. Single responsibility principle (SRP)
    ii. Open/Closed principle (OCP)
    iii. Liskov substitution principle (LSP)

    iv. Interface segregation principle (ISP)
    v. Dependency inversion principle (DIP)
VI. Summary
VII. Questions
VIII. Further reading
5. 4 REST APIs
I. Before you begin: Join our book community on Discord
II. REST & HTTP
    i. HTTP methods
    ii. HTTP Status code
    iii. HTTP headers
    iv. Versioning
    v. Wrapping up
III. Data Transfer Object (DTO)
    i. Goal
    ii. Design
    iii. Conceptual examples
    iv. Conclusion
IV. API contracts
    i. Code-first API Contract
    ii. Wrapping up
V. Summary
VI. Questions
VII. Further reading
VIII. Answers
6. 5 Minimal API
I. Before you begin: Join our book community on Discord
II. Top-level statements
III. Minimal Hosting
IV. Minimal APIs
    i. Map route-to-delegate
    ii. Configuring endpoints
    iii. Leveraging endpoint filters
    iv. Leveraging the endpoint filter factory
    v. Organizing endpoints
V. Using Minimal APIs with Data Transfer Objects
    i. Goal

    ii. Design
    iii. Project – Minimal API
    iv. Conclusion
  VI. Summary
  VII. Questions
  VIII. Further reading
  IX. Answers
7. 6 Model-View-Controller
  I. Before you begin: Join our book community on Discord
  II. The Model View Controller design pattern
    i. Goal
    ii. Design
    iii. Anatomy of ASP.NET Core web APIs
    iv. Conclusion
  III. Using MVC with DTOs
    i. Goal
    ii. Design
    iii. Project – MVC API
    iv. Conclusion
  IV. Summary
  V. Questions
  VI. Further reading
  VII. Answers
8. 7 Strategy, Abstract Factory, and Singleton Design Patterns
  I. Before you begin: Join our book community on Discord
  II. The Strategy design pattern
    i. Goal
    ii. Design
    iii. Project – Strategy
    iv. Conclusion
  III. The Abstract Factory design pattern
    i. Goal
    ii. Design
    iii. Project – Abstract Factory
    iv. Project – The mid-range vehicle factory
    v. Impacts of the Abstract Factory
    vi. Conclusion

IV. The Singleton design pattern
    i. Goal
    ii. Design
    iii. An alternate (better) way
    iv. Code smell – Ambient Context
    v. Conclusion
V. Summary
VI. Questions
VII. Answers
9. 8 Dependency Injection
I. Before you begin: Join our book community on Discord
II. What is dependency injection?
    i. The composition root
    ii. Striving for adaptability
    iii. Understanding the use of the IoC container
    iv. The role of an IoC container
    v. Code smell – Control Freak
    vi. Object lifetime
    vii. Registering our dependencies
    viii. Registering your features elegantly
    ix. Using external IoC containers
III. Revisiting the Strategy pattern
    i. Constructor injection
    ii. Property injection
    iii. Method injection
    iv. Project – Strategy
    v. Conclusion
IV. Understanding guard clauses
V. Revisiting the Singleton pattern
    i. Project – Application state
    ii. Project – Wishlist
    iii. Conclusion
VI. Understanding the Service Locator pattern
    i. Project – ServiceLocator
    ii. Conclusion
VII. Revisiting the Factory pattern
    i. Project – Factory

VIII. Summary
IX. Questions
X. Further reading
XI. Answers
10. 9 Options, Settings, and Configuration
I. Before you begin: Join our book community on Discord
II. Loading the configuration
III. Learning the building blocks
i. IOptionsMonitor<TOptions>
ii. IOptionsFactory<TOptions>
iii. IOptionsSnapshot<TOptions>
iv. IOptions<TOptions>
IV. Project – CommonScenarios
i. Manual configuration
ii. Using the settings file
iii. Injecting options
iv. Named options
v. Reloading options at runtime
V. Project – OptionsConfiguration
i. Creating the program
ii. Configuring the options
iii. Implementing a configurator object
iv. Adding post-configuration
v. Using multiple configurator objects
vi. Exploring other configuration possibilities
VI. Project – OptionsValidation
i. Eager validation
ii. Data annotations
iii. Validation types
VII. Project – OptionsValidationFluentValidation
VIII. Workaround – Injecting options directly
IX. Project – Centralizing the configuration
X. Using the configuration-binding source generator
i. Project – ConfigurationGenerators: Part 1
XI. Using the options validation source generator
i. Project – ConfigurationGenerators: Part 2
XII. Using the ValidateOptionsResultBuilder class

        i. Project - ValidateOptionsResultBuilder
    XIII. Summary
    XIV. Questions
    XV. Further reading
    XVI. Answers
11. 10 Logging patterns
    I. Before you begin: Join our book community on Discord
    II. About logging
    III. Writing logs
    IV. Log levels
    V. Logging providers
    VI. Configuring logging
    VII. Structured logging
    VIII. Summary
    IX. Questions
    X. Further reading
    XI. Answers
12. 11 Structural Patterns
    I. Before you begin: Join our book community on Discord
    II. Implementing the Decorator design pattern
        i. Goal
        ii. Design
        iii. Project – Adding behaviors
        iv. Project – Decorator using Scrutor
        v. Conclusion
    III. Implementing the Composite design pattern
        i. Goal
        ii. Design
        iii. Project – BookStore
        iv. Conclusion
    IV. Implementing the Adapter design pattern
        i. Goal
        ii. Design
        iii. Project – Greeter
        iv. Conclusion
    V. Implementing the Façade design pattern
        i. Goal

    ii. Design
    iii. Project – The façades
    iv. Conclusion
VI. Summary
VII. Questions
VIII. Further reading
IX. Answers
13. 12 Behavioral Patterns
I. Before you begin: Join our book community on Discord
II. Implementing the Template Method pattern
    i. Goal
    ii. Design
    iii. Project – Building a search machine
    iv. Conclusion
III. Implementing the Chain of Responsibility pattern
    i. Goal
    ii. Design
    iii. Project – Message interpreter
    iv. Conclusion
IV. Mixing the Template Method and Chain of Responsibility patterns
    i. Project – Improved message interpreter
    ii. Project – A final, finer-grained design
    iii. Conclusion
V. Summary
VI. Questions
VII. Answers
14. 13 Understanding the Operation Result Design Pattern
I. Before you begin: Join our book community on Discord
II. The Operation Result pattern
    i. Goal
    ii. Design
    iii. Project – Implementing different Operation Result patterns
    iv. Advantages and disadvantages
III. Summary
IV. Questions
V. Further reading

VI. Answers
15. 14 Layering and Clean Architecture
    I. Before you begin: Join our book community on Discord
    II. Introducing layering
        i. Classic layering model
        ii. Splitting the layers
        iii. Layers versus tiers versus assemblies
    III. Responsibilities of the common layers
        i. Presentation
        ii. Domain
        iii. Data
    IV. Abstract layers
    V. Sharing the model
    VI. Clean Architecture
    VII. Implementing layering in real life
        i. To be or not to be a purist?
        ii. Building a façade over a database
    VIII. Summary
    IX. Questions
    X. Further reading
    XI. Answers
16. 15 Object Mappers, Aggregate Services, and Façade
    I. Before you begin: Join our book community on Discord
    II. Object mapper
        i. Goal
        ii. Design
        iii. Project – Mapper
        iv. Conclusion
    III. Code smell – Too many dependencies
    IV. Pattern – Aggregate Services
    V. Pattern – Mapping Façade
    VI. Project – Mapping service
    VII. Project – AutoMapper
    VIII. Project – Mapperly
    IX. Summary
    X. Questions
    XI. Further reading

XII. Answers
17. 16 Mediator and CQRS Design Patterns
    I. Before you begin: Join our book community on Discord
    II. A high-level overview of Vertical Slice Architecture
    III. Implementing the Mediator pattern
        i. Goal
        ii. Design
        iii. Project – Mediator (IMediator)
        iv. Project – Mediator (IChatRoom)
        v. Conclusion
    IV. Implementing the CQS pattern
        i. Goal
        ii. Design
        iii. Project – CQS
        iv. Conclusion
    V. Code smell – Marker Interfaces
        i. Metadata
        ii. Dependency identifier
    VI. Using MediatR as a mediator
        i. Project – Clean Architecture with MediatR
        ii. Conclusion
    VII. Summary
    VIII. Questions
    IX. Further reading
    X. Answers
18. 17 Getting Started with Vertical Slice Architecture
    I. Before you begin: Join our book community on Discord
    II. Anti-pattern – Big Ball of Mud
    III. Vertical Slice Architecture
        i. What are the advantages and disadvantages?
        ii. Project – Vertical Slice Architecture
        iii. Conclusion
    IV. Continuing your journey: A few tips and tricks
    V. Summary
    VI. Questions
    VII. Further reading
    VIII. Answers

19. 18 Request-EndPoint-Response (REPR) and Minimal APIs
    I. Before you begin: Join our book community on Discord
    II. Request-EndPoint-Response (REPR) pattern
        i. Goal
        ii. Design
        iii. Project – SimpleEndpoint
        iv. Conclusion
    III. Project – REPR—A slice of the real-world
        i. Assembling our stack
        ii. Dissecting the code structure
        iii. Exploring the shopping basket
        iv. Managing exception handling
        v. Grey-box testing
    IV. Summary
    V. Questions
    VI. Further reading
    VII. Answers
20. 19 Introduction to Microservices Architecture
    I. Before you begin: Join our book community on Discord
    II. What are microservices?
        i. Cohesive unit of business
        ii. Ownership of data
        iii. Microservice independence
    III. An introduction to event-driven architecture
        i. Domain events
        ii. Integration events
        iii. Application events
        iv. Enterprise events
        v. Conclusion
    IV. Getting started with message queues
        i. Conclusion
    V. Implementing the Publish-Subscribe pattern
        i. Message brokers
        ii. The event sourcing pattern
        iii. Example
        iv. Conclusion
    VI. Introducing Gateway patterns

    i. Gateway Routing pattern
    ii. Gateway Aggregation pattern
    iii. Backend for Frontend pattern
    iv. Mixing and matching gateways
    v. Conclusion
VII. Project – REPR.BFF
    i. Layering APIs
    ii. Running the microservices
    iii. Creating typed HTTP clients using Refit
    iv. Creating a service that serves the current customer
    v. Features
    vi. Conclusion
VIII. Revisiting the CQRS pattern
    i. Advantages and potential risks
    ii. Conclusion
IX. Exploring the Microservice Adapter pattern
    i. Adapting an existing system to another
    ii. Decommissioning a legacy application
    iii. Adapting an event broker to another
    iv. Conclusion
X. Summary
XI. Questions
XII. Further reading
XIII. Answers
21. 20 Modular Monolith
I. Before you begin: Join our book community on Discord
II. What is a Modular Monolith?
    i. What are traditional Monoliths?
    ii. What are microservices?
III. Advantages of Modular Monoliths
IV. Key Components of a Modular Monolith
V. Implementing a Modular Monolith
    i. Planning the project
    ii. Defining our stack
VI. Project—Modular Monolith
    i. Sending events from the catalog module
    ii. Consuming the events from the basket module

    iii. Inside the aggregator
    iv. Exploring the REST API HttpClient
    v. Sending HTTP requests to the API
    vi. Validating the existence of a product
VII. Transitioning to Microservices
VIII. Challenges and Pitfalls
IX. Conclusion
X. Questions
XI. Further reading
XII. An end is simply a new beginning
XIII. Answers

Architecting ASP.NET Core Applications, Third Edition: An atypical design patterns guide for .NET 8, C# 12, and beyond

Welcome to Packt Early Access. We're giving you an exclusive preview of this book before it goes on sale. It can take many months to write a book, but our authors have cutting-edge information to share with you today. Early Access gives you an insight into the latest developments by making chapter drafts available. The chapters may be a little rough around the edges right now, but our authors will update them over time.You can dip in and out of this book or follow along from start to finish; Early Access is designed to be flexible. We hope you enjoy getting to know more about the process of writing a Packt book.

1. Chapter 1: Introduction
2. Chapter 2: Automated Testing
3. Chapter 3: Architectural Principles
4. Chapter 4: REST APIs
5. Chapter 5: Minimal API
6. Chapter 6: Model-View-Controller
7. Chapter 7: Strategy, Abstract Factory, and Singleton Design Patterns
8. Chapter 8: Dependency Injection
9. Chapter 9: Options, Settings, and Configuration
10. Chapter 10: Logging patterns
11. Chapter 11: Structural Patterns
12. Chapter 12: Behavioral Patterns
13. Chapter 13: Understanding the Operation Result Design Pattern
14. Chapter 14: Layering and Clean Architecture
15. Chapter 15: Object Mappers, Aggregate Services, and Façade
16. Chapter 16: Mediator and CQRS Design Patterns
17. Chapter 17: Getting Started with Vertical Slice Architecture

18. Chapter 18: Request-EndPoint-Response (REPR) and Minimal APIs
19. Chapter19: Introduction to Microservices Architecture
20. Chapter20: Modular Monolith

1 Introduction

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



The goal of this book is not to create yet another design pattern book; instead, the chapters are organized according to scale and topic, allowing you to start small with a solid foundation and build slowly upon it, just like you would build a program.Instead of a guide covering a few ways of applying a design pattern, we will explore the thought processes behind the systems we are designing from a software engineer's point of view.This is not a magic recipe book; from experience, there is no magical recipe when designing software; there are only your logic, knowledge, experience, and analytical skills. Let's define "experience" as *your past successes and failures*. And don't worry, you will fail during your career, but don't get discouraged by it. The faster you fail, the faster you can recover and learn, leading to successful products. Many techniques covered in this book should help you achieve success. Everyone has failed and made mistakes; you aren't the first and certainly won't be the last. To paraphrase a well-known saying by Roosevelt: *the people that never fail are the ones who never do anything*.At a high level:

- This book explores basic patterns, unit testing, architectural principles, and some ASP.NET Core mechanisms.
- Then, we move up to the component scale, exploring patterns oriented toward small chunks of software and individual units.
- After that, we move to application-scale patterns and techniques, exploring ways to structure an application.
- Some subjects covered throughout the book could have a book of their own, so after this book, you should have plenty of ideas about where to continue your journey into software architecture.

Here are a few pointers about this book that are worth mentioning:

- The chapters are organized to start with small-scale patterns and then progress to higher-level ones, making the learning curve easier.
- Instead of giving you a recipe, the book focuses on the thinking behind things and shows the evolution of some techniques to help you understand why the shift happened.
- Many use cases combine more than one design pattern to illustrate alternate usage so you can understand and use the patterns efficiently. This also shows that design patterns are not beasts to tame but tools to use, manipulate, and bend to your will.
- As in real life, no textbook solution can solve all our problems; real problems are always more complicated than what's explained in textbooks. In this book, I aim to show you how to mix and match patterns to think "architecture" instead of giving you step-by-step instructions to reproduce.

The rest of the introduction chapter introduces the concepts we explore throughout the book, including refreshers on a few notions. We also touch on .NET, its tooling, and some technical requirements.In this chapter, we cover the following topics:

- What is a design pattern?
- Anti-patterns and code smell.
- Understanding the web – request/response.
- Getting started with .NET.

What is a design pattern?

Since you just purchased a book about design patterns, I guess you have some idea of what design patterns are, but let's make sure that we are on the same page.**Abstract definition**: A design pattern is a proven technique that we can use to solve a specific problem.In this book, we apply different patterns to solve various problems and leverage some open-source tools to go further, faster! Abstract definitions make people sound smart, but understanding concepts requires more practice, and there is no better way to learn than by experimenting with something, and design patterns are no different.If that definition does not make sense to you yet, don't worry. You should have enough information by the end of the book to correlate the multiple practical examples and explanations with that definition, making it crystal clear.I like to compare programming to playing with LEGO® because what you have to do is very similar: put small pieces together to create something bigger. Therefore, if you lack imagination or skills, possibly because you are too young, your castle might not look as good as someone with more experience. With that analogy in mind, a design pattern is a plan to assemble a solution that fits one or more scenarios, like the tower of a castle. Once you designed a single tower, you can build multiple by following the same steps. Design patterns act as that tower plan and give you the tools to assemble reliable pieces to improve your masterpiece (program).However, instead of snapping LEGO® blocks together, you nest code blocks and interweave objects in a virtual environment!Before going into more detail, well-thought-out applications of design patterns should improve your application designs. That is true whether designing a small component or a whole system. However, be careful: throwing patterns into the mix just to use them can lead to the opposite result: over-engineering. Instead, aim to write the least amount of readable code that solves your issue or automates your process.As we have briefly mentioned, design patterns apply to different software engineering levels, and in this book, we start small and grow to a cloud-scale! We follow a smooth learning curve, starting with simpler patterns and code samples that bend good practices to focus on the patterns—finally ending with more advanced topics and good practices.Of course, some subjects are overviews more than deep dives, like automated testing, because no one can fit it all in a single book. Nonetheless, I've done my best to give you as much information about architecture-related subjects as possible to ensure the proper foundations are in place for you to get as much as possible out of the more advanced topics, and I sincerely hope you'll find this book a helpful and enjoyable read.Let's start with the opposite of design patterns because it is essential to identify wrong ways of doing things to avoid making those mistakes or to correct them when you see them. Of course, knowing the right way to overcome specific problems using design patterns is also crucial.

Anti-patterns and code smells

Anti-patterns and code smells are bad architectural practices or tips about possible bad design. Learning about best practices is as important as learning about bad ones, which is where we start. The book highlights multiple anti-patterns and code smells to help you get started. Next, we briefly explore the first few.

Anti-patterns

An **anti-pattern** is the opposite of a design pattern: it is a proven flawed technique that will most likely cause you trouble and cost you time and money (and probably give you headaches).An anti-pattern is a pattern that seems a good idea and seems to be the solution you were looking for, but it causes more harm than good. Some anti-patterns started as legitimate design patterns and were labelled anti-patterns later. Sometimes, it is a matter of opinion, and sometimes the classification can be influenced by the programming language or technologies.Let's look at an example next. We will explore some other anti-patterns throughout the book.

Anti-pattern – God Class

A **God class** is a class that handles too many things. Typically, this class serves as a central entity which many other classes inherit or use within the application it is the class that knows and manages everything in the system; it is *the* class. On the other hand, it is also the class that nobody wants to update, which breaks the application every time somebody touches it: **it is an evil class**!The best way to fix this is to segregate responsibilities and allocate them to multiple classes rather than concentrating them in a single class. We look at how to split responsibilities throughout the book, which helps create more robust software.If you have a personal project with a *God class* at its core, start by reading the book and then try to apply the principles and patterns you learn to divide that class into multiple smaller classes that interact together. Try to organize those new classes into cohesive units, modules, or assemblies.To help fix God classes, we dive into architectural principles in *Chapter 3*, *Architectural Principles*, opening the way to concepts such as responsibility segregation.

Code smells

A **code smell** is an indicator of a possible problem. It points to areas of your design that could benefit from a redesign. By “code smell,” we mean “code that stinks” or “code that does not smell right.”It is important to note that a code smell only indicates the possibility of a problem; it does not mean a problem exists. Code smells are usually good indicators, so it is worth analyzing your software’s “smelly” parts.An excellent example is when a method requires many comments to explain its logic. That often means that the code could be split into smaller methods with proper names, leading to more readable code and allowing you to get rid of those pesky comments.Another note about comments is that they don’t evolve, so what often happens is that the code described by a comment changes, but the comment remains the same. That leaves a false or obsolete description of a block of code that can lead a developer astray.The same is also true with method names. Sometimes, the method’s name and body tell a different story, leading to the same issues. Nevertheless, this happens less often than orphan or obsolete comments since programmers tend to read and write code better than spoken language comments. Nonetheless, keep that in mind when reading, writing, or reviewing code.

Code smell – Control Freak

An excellent example of a code smell is using the `new` keyword. This indicates a hardcoded dependency where the creator controls the new object and its lifetime. This is also known as the **Control Freak anti-pattern**, but I prefer to box it as a code smell instead of an anti-pattern since the `new` keyword is not intrinsically wrong.At this point, you may be wondering how it is possible not to use the `new` keyword in object-oriented programming, but rest assured, we will cover that and expand on the control freak code smell in *Chapter 7*, *Deep Dive into Dependency Injection*.

Code smell – Long Methods

The **long methods** code smell is when a method extends to more than 10 to 15 lines of code. That is a good indicator that you should think about that method differently. Having comments that separate multiple code blocks is a good indicator of a method that may be too long.Here are a few examples of what the case might be:

- The method contains complex logic intertwined in multiple conditional statements.
- The method contains a big `switch` block.
- The method does too many things.
- The method contains duplications of code.

To fix this, you could do the following:

- Extract one or more private methods.
- Extract some code to new classes.
- Reuse the code from external classes.

- If you have a lot of conditional statements or a huge `switch` block, you could leverage a design pattern such as the Chain of Responsibility, or CQRS, which you will learn about in *Chapter 10*, *Behavioral Patterns*, and *Chapter 14*, *Mediator and CQRS Design Patterns*.

Usually, each problem has one or more solutions; you need to spot the problem and then find, choose, and implement one of the solutions. Let's be clear: a method containing 16 lines does not necessarily need refactoring; it could be OK. Remember that a code smell indicates that there *might* be a problem, not that there necessarily *is* one—apply common sense.

Understanding the web – request/response

Before going any further, it is imperative to understand the basic concept of the web. The idea behind HTTP 1.X is that a client sends an HTTP request to a server, and then the server responds to that client. That can sound trivial if you have web development experience. However, it is one of the most important web programming concepts, irrespective of whether you are building web APIs, websites, or complex cloud applications.Let's reduce an HTTP request lifetime to the following:

1. The communication starts.
2. The client sends a request to the server.
3. The server receives the request.
4. The server does something with the request, like executing code/logic.
5. The server responds to the client.
6. The communication ends.

After that cycle, the server is no longer aware of the client. Moreover, if the client sends another request, the server is unaware that it responded to a request earlier for that same client because **HTTP is stateless**.There are mechanisms for creating a sense of persistence between requests for the server to be "aware" of its clients. The most well-known of these is cookies.If we dig deeper, an HTTP request comprises a header and an optional body. Then, requests are sent using a specific method. The most common HTTP methods are `GET` and `POST`. On top of those, extensively used by web APIs, we can add `PUT`, `DELETE`, and `PATCH` to that list.Although not every HTTP method accepts a body, can respond with a body, or should be idempotent, here is a quick reference table:

Method	Request has body	Response has body	Idempotent
`GET`	No*	Yes	Yes
`POST`	Yes	Yes	No
`PUT`	Yes	No	Yes
`PATCH`	Yes	Yes	No
`DELETE`	May	May	Yes

* Sending a body with a `GET` request is not forbidden by the HTTP specifications, but the semantics of such a request are not defined either. It is best to avoid sending `GET` requests with a body.

An **idempotent** request is a request that always yields the same result, whether it is sent once or multiple times. For example, sending the same `POST` request multiple times should create multiple similar entities, while sending the same `DELETE` request multiple times should delete a single entity. The status code of an idempotent request may vary, but the server state should remain the same. We explore those concepts in more depth in *Chapter 4*, *Model-View-Controller*.Here is an example of a `GET` request:

```
GET http: //www.forevolve.com/ HTTP/1.1
Host: www.forevolve.com
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
```

```
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/70.0.3538.110 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9,fr-CA;q=0.8,fr;q=0.7
Cookie: ...
```

The HTTP header comprises a list of key/value pairs representing metadata that a client wants to send to the server. In this case, I queried my blog using the `GET` method and Google Chrome attached some additional information to the request. I replaced the `Cookie` header's value with `...` because it can be pretty large and that information is irrelevant to this sample. Nonetheless, cookies are passed back and forth like any other HTTP header.

Important note about cookies

> The client sends cookies, and the server returns them for every request-response cycle. This could kill your bandwidth or slow down your application if you pass too much information back and forth (cookies or otherwise). One good example would be a serialized identity cookie that is very large.
>
> Another example, unrelated to cookies but that created such a back-and-forth, was the good old Web Forms `ViewState`. This was a hidden field sent with every request. That field could become very large when left unchecked.
>
> Nowadays, with high-speed internet, it is easy to forget about those issues, but they can significantly impact the user experience of someone on a slow network.

When the server decides to respond to the request, it returns a header and an optional body, following the same principles as the request. The first line indicates the request's status: whether it was successful. In our case, the status code was `200`, which indicates success. Each server can add more or less information to its response. You can also customize the response with code.Here is the response to the previous request:

```
HTTP/1.1 200 OK
Server: GitHub.com
Content-Type: text/html; charset=utf-8
Last-Modified: Wed, 03 Oct 2018 21:35:40 GMT
ETag: W/"5bb5362c-f677"
Access-Control-Allow-Origin: *
Expires: Fri, 07 Dec 2018 02:11:07 GMT
Cache-Control: max-age=600
Content-Encoding: gzip
X-GitHub-Request-Id: 32CE:1953:F1022C:1350142:5C09D460
Content-Length: 10055
Accept-Ranges: bytes
Date: Fri, 07 Dec 2018 02:42:05 GMT
Via: 1.1 varnish
Age: 35
Connection: keep-alive
X-Served-By: cache-ord1737-ORD
X-Cache: HIT
X-Cache-Hits: 2
X-Timer: S1544150525.288285,VS0,VE0
Vary: Accept-Encoding
X-Fastly-Request-ID: 98a36fb1b5642c8041b88ceace73f25caaf07746
<Response body truncated for brevity>
```

Now that the browser has received the server's response, it renders the HTML webpage. Then, for each resource, it sends another HTTP call to its URI and loads it. A resource is an external asset, such as an image, a JavaScript file, a CSS file, or a font.After the response, the server is no longer aware of the client; the communication has ended. It is essential to understand that to create a pseudo-state between each request, we need to use an external mechanism. That mechanism could be the *session-state* leveraging cookies, simply using cookies, or some other ASP.NET Core mechanisms, or we could create a stateless application. I recommend going stateless whenever possible. We write primarily stateless applications in the book.

Note

If you want to learn more about session and state management, I left a link in the *Further reading* section at the end of the chapter.

As you can imagine, the backbone of the internet is its networking stack. The **Hypertext Transfer Protocol** (**HTTP**) is the highest layer of that stack (layer 7). HTTP is an application layer built on the **Transmission Control Protocol** (**TCP**). TCP (layer 4) is the transport layer, which defines how data is moved over the network (for instance, the transmission of data, the amount of transmitted data, and error checking). TCP uses the **Internet Protocol** (**IP**) layer to reach the computer it tries to talk to. IP (layer 3) represents the network layer, which handles packet IP addressing.A packet is a chunk of data that is transmitted over the wire. We could send a large file directly from a source to a destination machine, but that is not practical, so the network stack breaks down large items into smaller packets. For example, the source machine breaks a file into multiple packets, sends them to the target machine, and then the target reassembles them back into the source file. This process allows numerous senders to use the same wire instead of waiting for the first transmission to be done. If a packet gets lost in transit, the source machine can also send only that packet back to the target machine.Rest assured, you don't need to understand every detail behind networking to program web applications, but it is always good to know that HTTP uses TCP/IP and chunks big payloads into smaller packets. Moreover, HTTP/1 limits the number of parallel requests a browser can open simultaneously. This knowledge can help you optimize your apps. For example, a high number of assets to load, their size, and the order in which they are sent to the browser can increase the page load time, the perceived page load time, or the paint time.To conclude this subject and not dig too deep into networking, HTTP/1 is older but foundational. HTTP/2 is more efficient and supports streaming multiple assets using the same TCP connection. It also allows the server to send assets to the client before it requests the resources, called a server push.If you find HTTP interesting, HTTP/2 is an excellent place to start digging deeper, as well as the HTTP/3 proposed standard that uses the QUIC transport protocol instead of HTTP (RFC 9114). ASP.NET Core 7.0+ supports HTTP/3, which is enabled by default in ASP.NET Core 8.0.Next, let's quickly explore .NET.

Getting started with .NET

A bit of history: .NET Framework 1.0 was first released in 2002. .NET is a managed framework that compiles your code into an **Intermediate Language** (**IL**) named **Microsoft Intermediate Language** (**MSIL**). That IL code is then compiled into native code and executed by the **Common Language Runtime** (**CLR**). The CLR is now known simply as the **.NET runtime**. After releasing several versions of .NET Framework, Microsoft never delivered on the promise of an interoperable stack. Moreover, many flaws were built into the core of .NET Framework, tying it to Windows.Mono, an open-source project, was developed by the community to enable .NET code to run on non-Windows OSes. Mono was used and supported by Xamarin, acquired by Microsoft in 2016. Mono enabled .NET code to run on other OSes like Android and iOS. Later, Microsoft started to develop an official cross-platform .NET SDK and runtime they named .NET Core.The .NET team did a magnificent job building ASP.NET Core from the ground up, cutting out compatibility with the older .NET Framework versions. That brought its share of problems at first, but .NET Standard alleviated the interoperability issues between the old .NET and the new .NET.After years of improvements and two major versions in parallel (Core and Framework), Microsoft reunified most .NET technologies into .NET 5+ and the promise of a shared **Base Class Library** (**BCL**). With .NET 5, .NET Core simply became .NET while ASP.NET Core remained ASP.NET Core. There is no .NET "Core" 4, to avoid any potential confusion with .NET Framework 4.X.New major versions of .NET release every year now. Even-number releases are **Long-Term Support** (**LTS**) releases with free support for 3 years, and odd-number releases (Current) have free support for only 18 months.The good thing behind this book is that the architectural principles and design patterns covered should remain relevant in the future and are not tightly coupled with the versions of .NET you are using. Minor changes to the code samples should be enough to migrate your knowledge and code to new versions.Next, let's cover some key information about the .NET ecosystem.

.NET SDK versus runtime

You can install different binaries grouped under SDKs and runtimes. The SDK allows you to build and run .NET programs, while the runtime only allows you to run .NET programs.As a developer, you want to install the SDK on your deployment environment. On the server, you want to install the runtime. The runtime is lighter, while the SDK contains more tools, including the runtime.

.NET 5+ versus .NET Standard

When building .NET projects, there are multiple types of projects, but basically, we can separate them into two categories:

- Applications
- Libraries

Applications target a version of .NET, such as `net5.0` and `net6.0`. Examples of that would be an ASP.NET application or a console application.Libraries are bundles of code compiled together, often distributed as a NuGet package. .NET Standard class library projects allow sharing code between .NET 5+, and .NET Framework projects. .NET Standard came into play to bridge the compatibility gap between .NET Core and .NET Framework, which eased the transition. Things were not easy when .NET Core 1.0 first came out.With .NET 5 unifying all the platforms and becoming the future of the unified .NET ecosystem, .NET Standard is no longer needed. Moreover, app and library authors should target the base **Target Framework Moniker** (**TFM**), for example, `net8.0`. You can also target `netstandard2.0` or `netstandard2.1` when needed, for example, to share code with .NET Framework. Microsoft also introduced OS-specific TFMs with .NET 5+, allowing code to use OS-specific APIs like `net8.0-android` and `net8.0-tvos`. You can also target multiple TFMs when needed.

> **Note**
>
> I'm sure we will see .NET Standard libraries stick around for a while. All projects will not just migrate from .NET Framework to .NET 5+ magically, and people will want to continue sharing code between the two.

The next versions of .NET are built over .NET 5+, while .NET Framework 4.X will stay where it is today, receiving only security patches and minor updates. For example, .NET 8 is built over .NET 7, iterating over .NET 6 and 5.Next, let's look at some tools and code editors.

Visual Studio Code versus Visual Studio versus the command-line interface

How can one of these projects be created? .NET Core comes with the `dotnet` **command-line interface** (**CLI**), which exposes multiple commands, including `new`. Running the `dotnet new` command in a terminal generates a new project.To create an empty class library, we can run the following commands:

```
md MyProject
cd MyProject
dotnet new classlib
```

That would generate an empty class library in the newly created `MyProject` directory.The `-h` option helps discover available commands and their options. For example, you can use `dotnet -h` to find the available SDK commands or `dotnet new -h` to find out about options and available templates.It is fantastic that .NET now has the `dotnet` CLI. The CLI enables us to automate our workflows in **continuous integration** (**CI**) pipelines while developing locally or through any other process.The CLI also makes it easier to write documentation that anyone can follow; writing a few commands in a terminal is way easier and faster than installing programs like Visual Studio and emulators.**Visual Studio Code** is my favourite text editor. I don't use it much for .NET coding, but I still do to reorganize projects, when it's CLI time, or for any other task that is easier to complete using a text editor, such as writing documentation using Markdown, writing JavaScript or TypeScript, or managing JSON, YAML, or XML files. To create a C# project, a Visual Studio solution, or to

add a NuGet package using Visual Studio Code, open a terminal and use the CLI.As for **Visual Studio**, my favourite C# IDE, it uses the CLI under the hood to create the same projects, making it consistent between tools and just adding a user interface on top of the `dotnet new` CLI command.You can create and install additional `dotnet new` project templates in the CLI or even create global tools. You can also use another code editor or IDE if you prefer. Those topics are beyond the scope of this book.

An overview of project templates

Here is an example of the templates that are installed (`dotnet new --list`):

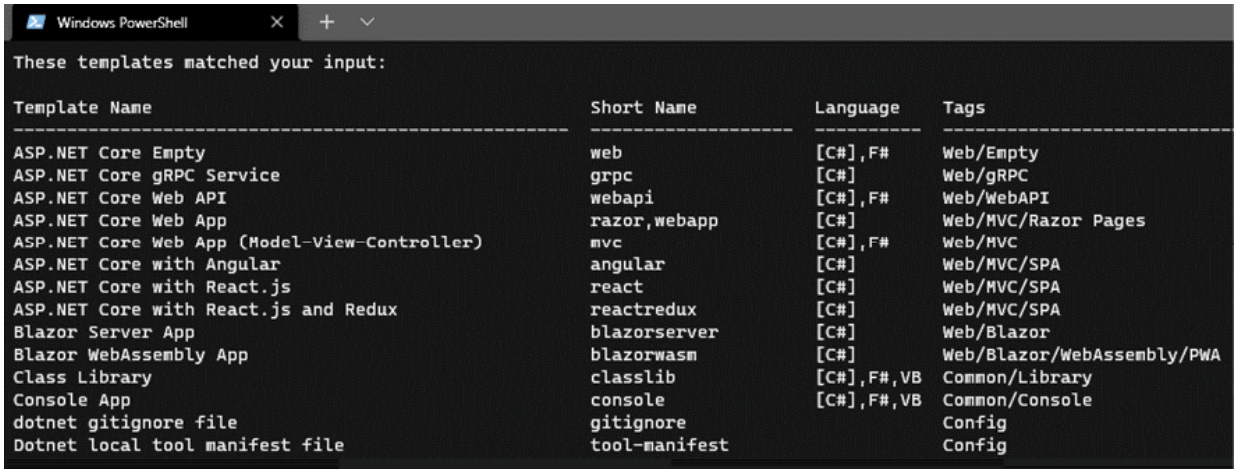


Figure 1.1: Project templates

A study of all the templates is beyond the scope of this book, but I'd like to visit the few that are worth mentioning, some of which we will use later:

- `dotnet new console` creates a console application
- `dotnet new classlib` creates a class library
- `dotnet new xunit` creates an xUnit test project
- `dotnet new web` creates an empty web project
- `dotnet new mvc` scaffolds an MVC application
- `dotnet new webapi` scaffolds a web API application

Running and building your program

If you are using Visual Studio, you can always hit the play button, or *F5*, and run your app. If you are using the CLI, you can use one of the following commands (and more). Each of them also offers different options to control their behaviour. Add the `-h` flag with any command to get help on that command, such as `dotnet build -h`:

Command	**Description**
`dotnet restore`	Restore the dependencies (a.k.a. NuGet packages) based on the `.csproj` or `.sln` file present in the current dictionary.
`dotnet build`	Build the application based on the `.csproj` or `.sln` file present in the current dictionary. It implicitly runs the `restore` command first.
`dotnet run`	Run the current application based on the `.csproj` file present in the current dictionary. It implicitly runs the `build` and `restore` commands first.
`dotnet watch run`	Watch for file changes. When a file has changed, the CLI updates the code from that file

	using the hot-reload feature. When that is impossible, it rebuilds the application and then reruns it (equivalent to executing the `run` command again). If it is a web application, the page should refresh automatically.
`dotnet test`	Run the tests based on the `.csproj` or `.sln` file present in the current directory. It implicitly runs the `build` and `restore` commands first. We cover testing in the next chapter.
`dotnet watch test`	Watch for file changes. When a file has changed, the CLI reruns the tests (equivalent to executing the `test` command again).
`dotnet publish`	Publish the current application, based on the `.csproj` or `.sln` file present in the current directory, to a directory or remote location, such as a hosting provider. It implicitly runs the `build` and `restore` commands first.
`dotnet pack`	Create a NuGet package based on the `.csproj` or `.sln` file present in the current directory. It implicitly runs the `build` and `restore` commands first. You don't need a `.nuspec` file.
`dotnet clean`	Clean the build(s) output of a project or solution based on the `.csproj` or `.sln` file present in the current directory.

Technical requirements

Throughout the book, we will explore and write code. I recommend installing Visual Studio, Visual Studio Code, or both to help with that. I use Visual Studio and Visual Studio Code. Other alternatives are Visual Studio for Mac, Riders, or any other text editor you choose.Unless you install Visual Studio, which comes with the .NET SDK, you may need to install it. The SDK comes with the CLI we explored earlier and the build tools for running and testing your programs. Look at the `README.md` file in the GitHub repository for more information and links to those resources.The source code of all chapters is available for download on GitHub at the following address: https://adpg.link/net6.

Summary

This chapter looked at design patterns, anti-patterns, and code smells. We also explored a few of them. We then moved on to a recap of a typical web application's request/response cycle.We continued by exploring .NET essentials, such as SDK versus runtime and app targets versus .NET Standard. We then dug a little more into the .NET CLI, where I laid down a list of essential commands, including `dotnet build` and `dotnet watch run`. We also covered how to create new projects. This has set us up to explore the different possibilities we have when building our .NET applications.In the next two chapters, we explore automated testing and architectural principles. These are foundational chapters for building robust, flexible, and maintainable applications.

Questions

Let's take a look at a few practice questions:

1. Can we add a body to a `GET` request?
2. Why are long methods a code smell?
3. Is it true that .NET Standard should be your default target when creating libraries?
4. What is a code smell?

Further reading

Here are some links to consolidate what has been learned in the chapter:

- Overview of how .NET is versioned: https://adpg.link/n52L

- .NET CLI overview: https://adpg.link/Lzx3
- Custom templates for `dotnet new`: https://adpg.link/74i2
- Session and state management in ASP.NET Core: https://adpg.link/Xzgf

2 Automated Testing

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter focuses on automated testing and how helpful it can be for crafting better software. It also covers a few different types of tests and the foundation of **test-driven development** (**TDD**). We also outline how testable ASP.NET Core is and how much easier it is to test ASP.NET Core applications than old ASP.NET MVC applications. This chapter overviews automated testing, its principles, xUnit, ways to sample test values, and more. While other books cover this topic more in-depth, this chapter covers the foundational aspects of automated testing. We are using parts of this throughout the book, and this chapter ensures you have a strong enough base to understand the samples.In this chapter, we cover the following topics:

- An overview of automated testing
- Testing .NET applications
- Important testing principles

Introduction to automated testing

Testing is an integral part of the development process, and automated testing becomes crucial in the long run. You can always run your ASP.NET Core website, open a browser, and click everywhere to test your features. That's a legitimate approach, but it is harder to test individual rules or more complex algorithms that way. Another downside is the lack of automation; when you first start with a small app containing a few pages, endpoints, or features, it may be fast to perform those tests manually. However, as your app grows, it becomes more tedious, takes longer, and increases the likelihood of making a mistake. Of course, you will always need real users to test your applications, but you want those tests to focus on the UX, the content, or some experimental features you are building instead of bug reports that automated tests could have caught early on.There are multiple types of tests and techniques in the testing space. Here is a list of three broad categories that represent how we can divide automated testing from a code correctness standpoint:

- Unit tests
- Integration tests
- End-to-end (E2E) tests

Usually, you want a mix of those tests, so you have fast unit tests testing your algorithms, slower tests that ensure the integrations between components are correct, and slow E2E tests that ensure the correctness of the system as a whole.The test pyramid is a good way of explaining a few concepts around automated testing. You want different granularity of tests and a different number of tests depending on their complexity and speed of execution. The following test pyramid shows the three types of tests stated above. However, we could add other types of tests in there as well. Moreover, that's just an abstract guideline to give you an idea. The most important aspect is the **return on investment** (**ROI**) and execution speed. If you can write one integration test that covers a large surface and is fast enough, this might be worth doing instead of multiple unit tests.

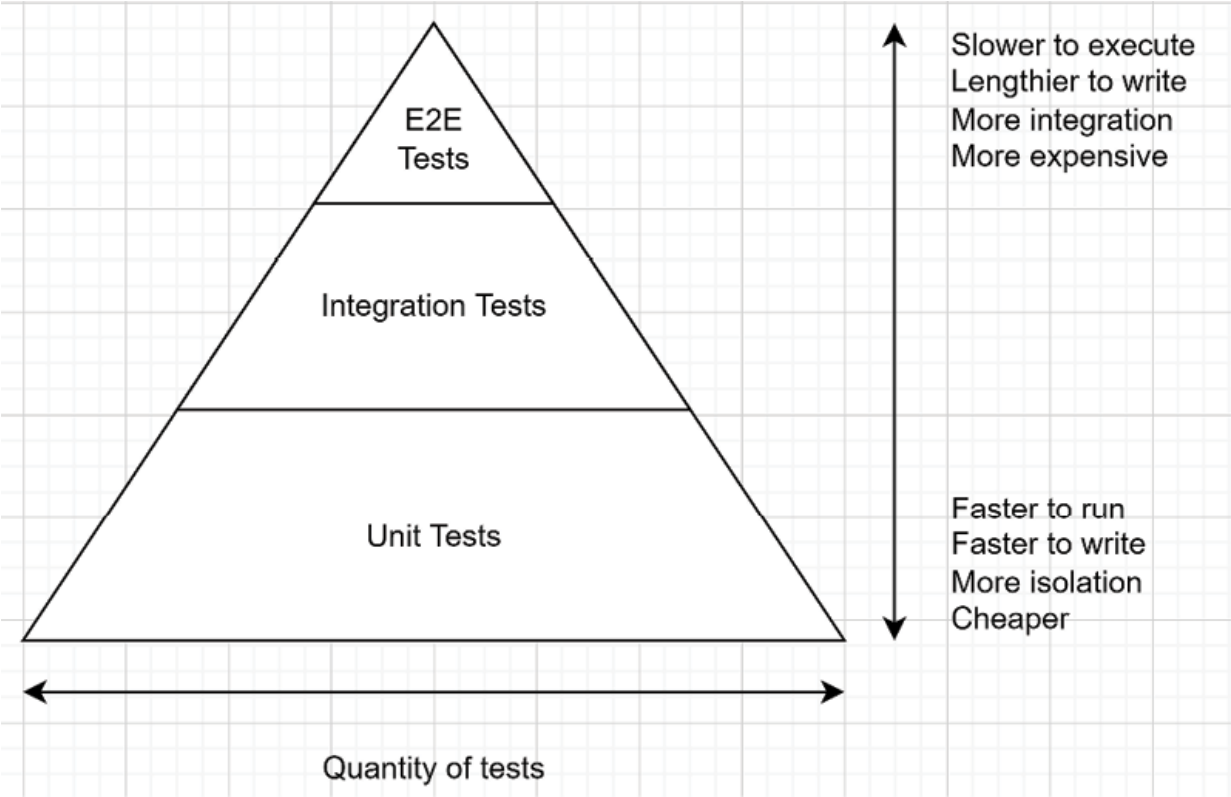


Figure 2.1: The test pyramid

> I cannot stress this enough; the execution speed of your tests is essential to receive fast feedback and know immediately that you have broken something with your code changes. Layering different types of tests allows you to execute only the fastest subset often, the not-so-fast occasionally, and the very slow tests infrequently. If your test suite is fast-enough, you don't even have to worry about it. However, if you have a lot of manual or E2E UI tests that take hours to run, that's another story (that can cost a lot of money).

Finally, on top of running your tests using a test runner, like in Visual Studio, VS Code, or the CLI, a great way to ensure code quality and leverage your automated tests is to run them in a CI pipeline, validating code changes for issues.Tech-wise, back when .NET Core was in pre-release, I discovered that the .NET team was using xUnit to test their code and that it was the only testing framework available. xUnit has become my favorite testing framework since, and we use it throughout the book. Moreover, the ASP.NET Core team made our life easier by designing ASP.NET Core for testability; testing is easier than before.Why are we talking about tests in an architectural book? Because testability is a sign of a good design. It also allows us to use tests instead of words to prove some concepts. In many code samples, the test cases are the consumers, making the program lighter without building an entire user interface and focusing on the patterns we are exploring instead of getting our focus scattered over some boilerplate UI code.

> To ensure we do not deviate from the matter at hand, we use automated testing moderately in the book, but I strongly recommend that you continue to study it, as it will help improve your code and design

Now that we have covered all that, let's explore those three types of tests, starting with unit testing.

Unit testing

Unit tests focus on individual units, like testing the outcome of a method. Unit tests should be fast and not rely on any infrastructure, such as a database. Those are the kinds of tests you want the most because they run fast, and each one tests a precise code path. They should also help you design your application better because you use your

code in the tests, so you become its first consumer, leading to you finding some design flaws and making your code better. If you don't like using your code in your tests, that is a good indicator that nobody else will. Unit tests should focus on testing algorithms (the ins and outs) and domain logic, not the code itself; how you wrote the code should have no impact on the intent of the test. For example, you are testing that a `Purchase` method executes the logic required to purchase one or more items, not that you created the variable `X`, `Y`, or `Z` inside that method.

> Don't discourage yourself if you find it challenging; writing a good test suite is not as easy as it sounds.

Integration testing

Integration tests focus on the interaction between components, such as what happens when a component queries the database or what happens when two components interact with each other.Integration tests often require some infrastructure to interact with, which makes them slower to run. By following the classic testing model, you want integration tests, but you want fewer of them than unit tests. An integration test can be very close to an E2E test but without using a production-like environment.

> We will break the test pyramid rule later, so always be critical of rules and principles; sometimes, breaking or bending them can be better. For example, having one good integration test can be better than *N* unit tests; don't discard that fact when writing your tests. See also Grey-box testing.

End-to-end testing

End-to-end tests focus on application-wide behaviors, such as what happens when a user clicks on a specific button, navigates to a particular page, posts a form, or sends a `PUT` request to some web API endpoint. E2E tests are usually run on infrastructure to test your application and deployment.

Other types of tests

There are other types of automated tests. For example, we could do load testing, performance testing, regression testing, contract testing, penetration testing, functional testing, smoke testing, and more. You can automate tests for anything you want to validate, but some tests are more challenging to automate or more fragile than others, such as UI tests.

> If you can automate a test in a reasonable timeframe, think ROI: do it! In the long run, it should pay off.

One more thing; don't blindly rely on metrics such as code coverage. Those metrics make for cute badges in your GitHub project's `readme.md` file but can lead you off track, resulting in you writing useless tests. Don't get me wrong, code coverage is a great metric when used correctly, but remember that one good test can be better than a lousy test suite covering 100% of your codebase. If you are using code coverage, ensure you and your team are not gaming the system.Writing good tests is not easy and comes with practice.

> One piece of advice: keep your test suite healthy by adding missing test cases and removing obsolete or useless tests. Think about use case coverage, not how many lines of code are covered by your tests.

Before moving forward to testing styles, let's inspect a hypothetical system and explore a more efficient way to test it.

Picking the right test style

Next is a dependency map of a hypothetical system. We use that diagram to pick the most meaningful type of test possible for each piece of the program. In real life, that diagram will most likely be in your head, but I drew it out in this case. Let's inspect that diagram before I explain its content:

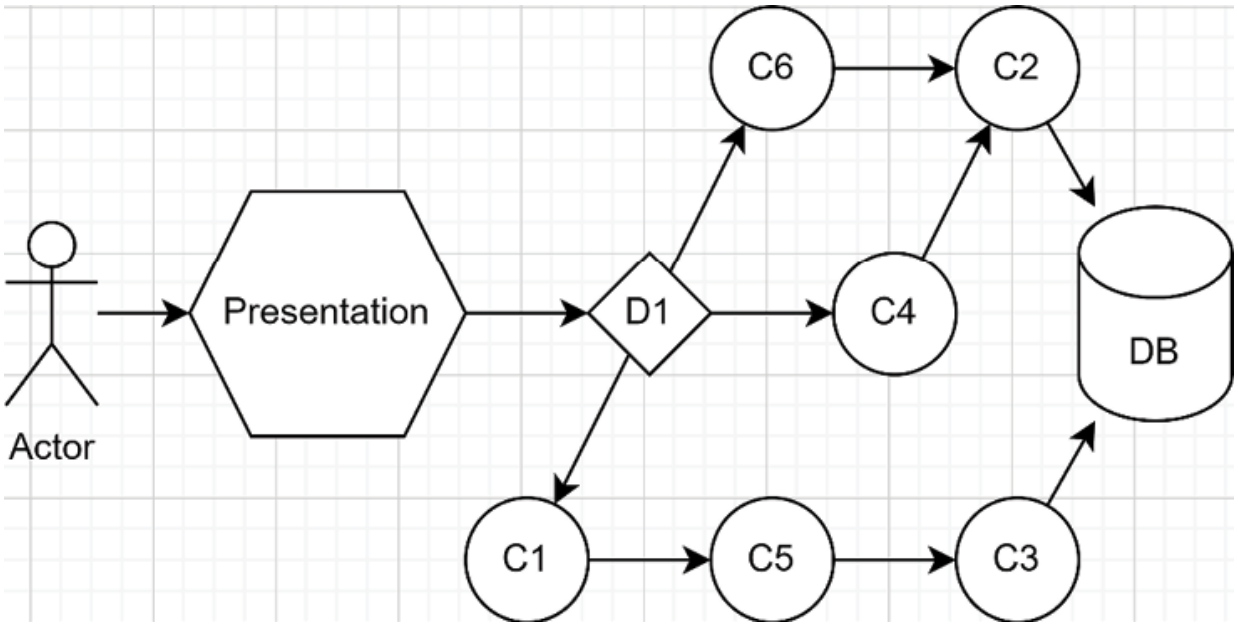


Figure 2.2: Dependency map of a hypothetical system

In the diagram, the **Actor** can be anything from a user to another system. **Presentation** is the piece of the system that the **Actor** interacts with and forwards the request to the system itself (this could be a user interface). **D1** is a component that has to decide what to do next based on the user input. **C1** to **C6** are other components of the system (could be classes, for example). **DB** is a database.D1 must choose between three code paths: interact with the components C1, C4, or C6. This type of logic is usually a good subject for unit tests, ensuring the algorithm yields the correct result based on the input parameter. Why pick a unit test? We can quickly test multiple scenarios, edge cases, out-of-bound data cases, and more. We usually mock the dependencies away in this type of test and assert that the subject under test made the expected call on the desired component.Then, if we look at the other code paths, we could write one or more integration tests for component C1, testing the whole chain in one go (C1, C5, and C3) instead of writing multiple mock-heavy unit tests for each component. If there is any logic that we need to test in components C1, C5, or C3, we can always add a few unit tests; that's what they are for.Finally, C4 and C6 are both using C2. Depending on the code (that we don't have here), we could write integration tests for C4 and C6, testing C2 simultaneously. Another way would be to unit test C4 and C6, and then write integration tests between C2 and the DB. If C2 has no logic, the latter could be the best and the fastest, while the former will most likely yield results that give you more confidence in your test suite in a continuous delivery model.When it is an option, I recommend evaluating the possibility of writing fewer meaningful integration tests that assert the correctness of a use case over a suite of mock-heavy unit tests. Remember always to keep the execution speed in mind.That may seem to go "against" the test pyramid, but does it? If you spend less time (thus lower costs) testing more use cases (adding more value), that sounds like a win to me. Moreover, we must not forget that mocking dependencies tends to make you waste time fighting the framework or other libraries instead of testing something meaningful and can add up to a high maintenance cost over time.Now that we have explored the fundamentals of automated testing, it is time to explore testing approaches and TDD, which is a way to apply those testing concepts.

Testing approaches

There are various approaches to testing, such as **behavior-driven development** (**BDD**), **acceptance test-driven development** (**ATDD**), and **test-driven development** (**TDD)**. The DevOps culture brings a mindset that embraces automated testing in line with its **continuous integration** (**CI**) and **continuous deployment** (**CD**) ideals. We can enable CD with a robust and healthy suite of tests that gives a high degree of confidence in our code, high enough to deploy the program when all tests pass without fear of introducing a bug.

TDD

TDD is a software development method that states that you should write one or more tests before writing the actual code. In a nutshell, you invert your development flow by following the **Red-Green-Refactor** technique, which goes like this:

1. You write a failing test (red).
2. You write just enough code to make your test pass (green).
3. You refactor that code to improve the design by ensuring all the tests pass.

We explore the meaning of **refactoring** next.

ATDD

ATDD is similar to TDD but focuses on acceptance (or functional) tests instead of software units and involves multiple parties like customers, developers, and testers.

BDD

BDD is another complementary technique originating from TDD and ATDD. BDD focuses on formulating test cases around application behaviors using spoken language and involves multiple parties like customers, developers, and testers. Moreover, practitioners of BDD often leverage the *given–when–then* grammar to formalize their test cases. Because of that, BDD output is in a human-readable format allowing stakeholders to consult such artifacts.The given–when–then template defines the way to describe the behavior of a user story or acceptance test, like this:

- *Given* one or more preconditions (context)
- *When* something happens (behavior)
- *Then* one or more observable changes are expected (measurable side effects)

ATDD and BDD are great areas to dig deeper into and can help design better apps; defining precise user-centric specifications can help build only what is needed, prioritize better, and improve communication between parties. For the sake of simplicity, we stick to unit testing, integration testing, and a tad of TDD in the book. Nonetheless, let’s go back to the main track and define refactoring.

Refactoring

Refactoring is about (continually) improving the code without changing its behavior.An automated test suite should help you achieve that goal and should help you discover when you break something. No matter whether you do TDD or not, I do recommend refactoring as often as possible; this helps clean your codebase, and it should also help you get rid of some technical debt at the same time.Okay, but what is **technical debt**?

Technical debt

Technical debt represents the corners you cut short while developing a feature or a system. That happens no matter how hard you try because life is life, and there are delays, deadlines, budgets, and people, including developers (yes, that’s you and me).The most crucial point is understanding that you cannot avoid technical debt altogether, so it’s better to embrace that fact and learn to live with it instead of fighting it. From that point forward, you can only try to limit the amount of technical debt you, or someone else, generate and ensure to always refactor some of it over time each sprint (or the unit of time that fits your projects/team/process).One way to limit the piling up of technical debt is to refactor the code often. So, factor the refactoring time into your time estimates. Another way is to improve collaboration between all the parties involved. Everyone must work toward the same goal if you want your projects to succeed.You will sometimes cut the usage of best practices short due to external forces like people or time constraints. The key is coming back at it as soon as possible to repay that technical debt, and automated tests are there to help you refactor that code and eliminate that debt elegantly. Depending on the size of your workplace, there will be more or less people between you and that decision.

> Some of these things might be out of your control, so you may have to live with more technical debt than you had hoped. However, even when things are out of your control, nothing stops you from becoming a pioneer and working toward improving the enterprise's culture. Don't be afraid to become an agent of change and lead the charge.

Nevertheless, don't let the technical debt pile up too high, or you may not be able to pay it back, and at some point, that's where a project begins to break and fail. Don't be mistaken; a project in production can be a failure. Delivering a product does not guarantee success, and I'm talking about the quality of the code here, not the amount of generated revenue (I'll leave that to other people to evaluate).Next, we look at different ways to write tests, requiring more or less knowledge of the inner working of the code.

Testing techniques

Here we look at different ways to approach our tests. Should we know the code? Should we test user inputs and compare them against the system results? How to identify a proper value sample? Let's start with white-box testing.

White-box testing

White-box testing is a software testing technique that uses knowledge of the internal structure of the software to design tests. We can use white-box testing to find defects in the software's logic, data structures, and algorithms.

> This type of testing is also known as clear-box testing, open-box testing, transparent-box testing, glass-box testing, and code-based testing.

Another benefit of white-box testing is that it can help optimize the code. By reviewing the code to write tests, developers can identify and improve inefficient code structures, improving overall software performance. The developer can also improve the application design by finding architectural issues while testing the code.

> White-box testing encompasses most unit and integration tests.

Next, we look at black-box testing, the opposite of white-box testing.

Black-box testing

Black-box testing is a software testing method where a tester examines an application's functionality without knowing the internal structure or implementation details. This form of testing focuses solely on the inputs and outputs of the system under test, treating the software as a "black box" that we can't see into.The main goal of black-box testing is to evaluate the system's behavior against expected results based on requirements or user stories. Developers writing the tests do not need to know the codebase or the technology stack used to build the software.We can use black-box testing to assess the correctness of several types of requirements, like:

1. **Functional testing**: This type of testing is related to the software's functional requirements, emphasizing what the system does, a.k.a. behavior verification.
2. **Non-functional testing**: This type of testing is related to non-functional requirements such as performance, usability, reliability, and security, a.k.a. performance evaluation.
3. **Regression testing**: This type of testing ensures the new code does not break existing functionalities, a.k.a. change impact.

Next, let's explore a hybrid between white-box and black-box testing.

Grey-box testing

Grey-box testing is a blend between white-box and black-box testing. Testers need only partial knowledge of the application's internal workings and use a combination of the software's internal structure and external behavior to

craft their tests.We implement grey-box testing use cases in *Chapter 16, Request-Endpoint-Response (REPR)*. Meanwhile, let's compare the three techniques.

White-box vs. Black-box vs. Grey-box testing

To start with a concise comparison, here's a table that compares the three broad techniques:

Feature	**Whitebox Testing**	**Blackbox Testing**	**Gray-box Testing**
Definition	Testing based on the internal design of the software	Testing based on the behavior and functionality of the software	Testing that combines the internal design and behavior of the software
Knowledge of code required	Yes	No	Yes
Types of defects found	Logic, data structure, architecture, and performance issues	Functionality, usability, performance, and security issues	Most types of issues
Coverage per test	Small; targeted on a unit	Large; targeted on a use case	Up to large; can vary in scope
Testers	Usually performed by developers.	Testers can write the tests without specific technical knowledge of the application's internal structure.	Developers can write the tests, while testers also can with some knowledge of the code.
When to use each style?	Write unit tests to validate complex algorithms or code that yields multiple results based on many inputs. These tests are usually high-speed so you can have many of them.	Write if you have specific scenarios you want to test, like UI tests, or if testers and developers are two distinct roles in your organization. These usually run the slowest and require you to deploy the application to test it. You want as few as possible to improve the feedback time.	Write to avoid writing black-box or white-box tests. Layer the tests to cover as much as possible with as few tests as possible. Depending on the application's architecture, this type of test can yield optimal results for many scenarios.

Let's conclude next and explore a few advantages and disadvantages of each technique.

Conclusion

White-box testing includes unit and integration tests. Those tests run fast, and developers use them to improve the code and test complex algorithms. However, writing a large quantity of those tests takes time. Writing brittle tests that are tightly coupled with the code itself is easier due to the proximity to the code, increasing the maintenance cost of such test suites. It also makes it prone to overengineering your application in the name of testability.Black-box testing encompasses different types of tests that tend towards end-to-end testing. Since the tests target the external surface of the system, they are less likely to break when the system changes. Moreover, they are excellent at testing behaviors, and since each test tests an end-to-end use case, we need fewer of them, leading to a decrease in writing time and maintenance costs. Testing the whole system has drawbacks, including the slowness of executing each test, so combining black-box testing with other types of tests is very important to find the right balance between the number of tests, test case coverage, and speed of execution of the tests.Grey-box testing is a fantastic mix between the two others; you can treat any part of the software as a black box, leverage your inner-working knowledge to mock or stub parts of the test case (like to assert if the system persisted a record in the database), and test end-to-end scenarios more efficiently. It brings the best of both worlds, significantly reducing the number of tests while increasing the test surface considerably for each test case. However, doing grey-box testing on smaller units or heavily mocking the system may yield the same drawbacks as white-box testing. Integration tests or almost-E2E tests are good candidates for grey-box testing. We implement grey-box testing use cases in *Chapter 16, Request-Endpoint-Response (REPR)*. Meanwhile, let's explore a few techniques to help optimize our test case creation by applying different techniques, like testing a small subset of values to assert the correctness of our programs by writing an optimal number of tests.

Test case creation

Multiple ways exist to break down and create test cases to help find software defects with a minimal test count. Here are some techniques to help minimize the number of tests while maximizing the test coverage:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Use Case Testing

I present the techniques theoretically. They apply to all sorts of tests and should help you write better test suites. Let’s have a quick look at each.

Equivalence Partitioning

This technique divides the input data of the software into different equivalence data classes and then tests these classes rather than individual inputs. An equivalence data class means that all values in that partition set should lead to the same outcome or yield the same result. Doing this allows for limiting the number of tests considerably.For example, consider an application that accepts an integer value between 1 and 100 (inclusive). Using equivalence partitioning, we can divide the input data into two equivalence classes:

- Valid
- Invalid

To be more precise, we could further divide it into three equivalence classes:

- Class 1: Less than 1 (Invalid)
- Class 2: Between 1 and 100 (Valid)
- Class 3: Greater than 100 (Invalid)

Then we can write three tests, picking one representative from each class (e.g., 0, 50, and 101) to create our test cases. Doing so ensures a broad coverage with minimal test cases, making our testing process more efficient.

Boundary Value Analysis

This technique focuses on the values at the boundary of the input domain rather than the center. This technique is based on the principle that errors are most likely to occur at the boundaries of the input domain.The **input domain** represents the set of all possible inputs for a system. The **boundaries** are the edges of the input domain, representing minimum and maximum values.For example, if we expect a function to accept an integer between 1 and 100 (inclusive), the boundary values would be 1 and 100. With Boundary Value Analysis, we would create test cases for these values, values just outside the boundaries (like 0 and 101), and values just inside the boundaries (like 2 and 99).Boundary Value Analysis is a very efficient testing technique that provides good coverage with a relatively small number of test cases. However, it’s unsuitable for finding errors within the boundaries or for complex logic errors. Boundary Value Analysis should be used on top of other testing methods, such as equivalence partitioning and decision table testing, to ensure the software is as defect-free as possible.

Decision Table Testing

This technique uses a decision table to design test cases. A decision table is a table that shows all possible combinations of input values and their corresponding outputs.It’s handy for complex business rules that can be expressed in a table format, enabling testers to identify missing and extraneous test cases.For example, our system only allows access to a user with a valid username and password. Moreover, the system denies access to users when it is under maintenance. The decision table would have three conditions (username, password, and maintenance) and one action (allow access). The table would list all possible combinations of these conditions and the expected action for each combination. Here is an example:

Valid Username	Valid Password	System under Maintenance	Allow Access
True	True	False	Yes
True	True	True	No
True	False	False	No
True	False	True	No
False	True	False	No
False	True	True	No
False	False	False	No
False	False	True	No

The main advantage of Decision Table Testing is that it ensures we test all possible input combinations. However, it can become complex and challenging to manage when systems have many input conditions, as the number of rules (and therefore test cases) increases exponentially with the number of conditions.

State Transition Testing

We usually use State Transition Testing to test software with a state machine since it tests the different system states and their transitions. It's handy for systems where the system behavior can change based on its current state. For example, a program with states like "logged in" or "logged out".To perform State Transition Testing, we need to identify the states of the system and then the possible transitions between the states. For each transition, we need to create a test case. The test case should test the software with the specified input values and verify that the software transitions to the correct state. For example, a user with the state "logged in" must transition to the state "logged out" after signing out.The main advantage of State Transition Testing is that it tests sequences of events, not just individual events, which could reveal defects not found by testing each event in isolation. However, State Transition Testing can become complex and time-consuming for systems with many states and transitions.

Use Case Testing

This technique validates that the system behaves as expected when used in a particular way by a user. Use cases could have formal descriptions, be user stories, or take any other form that fits your needs.A use case involves one or more actors executing steps or taking actions that should yield a particular result. A use case can include inputs and expected outputs. For example, when a user (actor) that is "signed in" (precondition) clicks the "sign out" button (action), then navigates to the profile page (action), the system denies access to the page and redirects the users to the sign in page, displaying an error message (expected behaviors).Use case testing is a systematic and structured approach to testing that helps identify defects in the software's functionality. It is very user-centric, ensuring the software meets the users' needs. However, creating test cases for complex use cases can be difficult. In the case of a user interface, the time to execute end-to-end tests of use cases can take a long time, especially as the number of tests grows.

> It is an excellent approach to think of your test cases in terms of functionality to test, whether using a formal use case or just a line written on a napkin. The key is to test behaviors, not code.

Now that we have explored these techniques, it is time to introduce the xUnit library, ways to write tests, and how tests are written in the book. Let's start by creating a test project.

How to create an xUnit test project

To create a new xUnit test project, you can run the `dotnet new xunit` command, and the CLI does the job for you by creating a project containing a `UnitTest1` class. That command does the same as creating a new xUnit project from Visual Studio.For unit testing projects, name the project the same as the project you want to test and append `.Tests` to it. For example, `MyProject` would have a `MyProject.Tests` project associated with it. We explore more details in the *Organizing your tests* section below.The template already defines all the required NuGet packages, so you can start testing immediately after adding a reference to your project under test.

You can also add project references using the CLI with the `dotnet add reference` command. Assuming we are in the `./test/MyProject.Tests` directory and the project file we want to reference is in the `./src/MyProject` directory; we can execute the following command to add a reference:

```
dotnet add reference ../../src/MyProject.csproj.
```

Next, we explore some xUnit features that will allow us to write test cases.

Key xUnit features

In xUnit, the `[Fact]` attribute is the way to create unique test cases, while the `[Theory]` attribute is the way to make data-driven test cases. Let's start with facts, the simplest way to write a test case.

Facts

Any method with no parameter can become a test method by decorating it with a `[Fact]` attribute, like this:

```
public class FactTest
{
    [Fact]
    public void Should_be_equal()
    {
        var expectedValue = 2;
        var actualValue = 2;
        Assert.Equal(expectedValue, actualValue);
    }
}
```

You can also decorate asynchronous methods with the fact attribute when the code under test needs it:

```
public class AsyncFactTest
{
    [Fact]
    public async Task Should_be_equal()
    {
        var expectedValue = 2;
        var actualValue = 2;
        await Task.Yield();
        Assert.Equal(expectedValue, actualValue);
    }
}
```

In the preceding code, the highlighted line conceptually represents an asynchronous operation and does nothing more than allow using the `async` / `await` keywords.When we run the tests from Visual Studio's Test Explorer, the test run result looks like this:

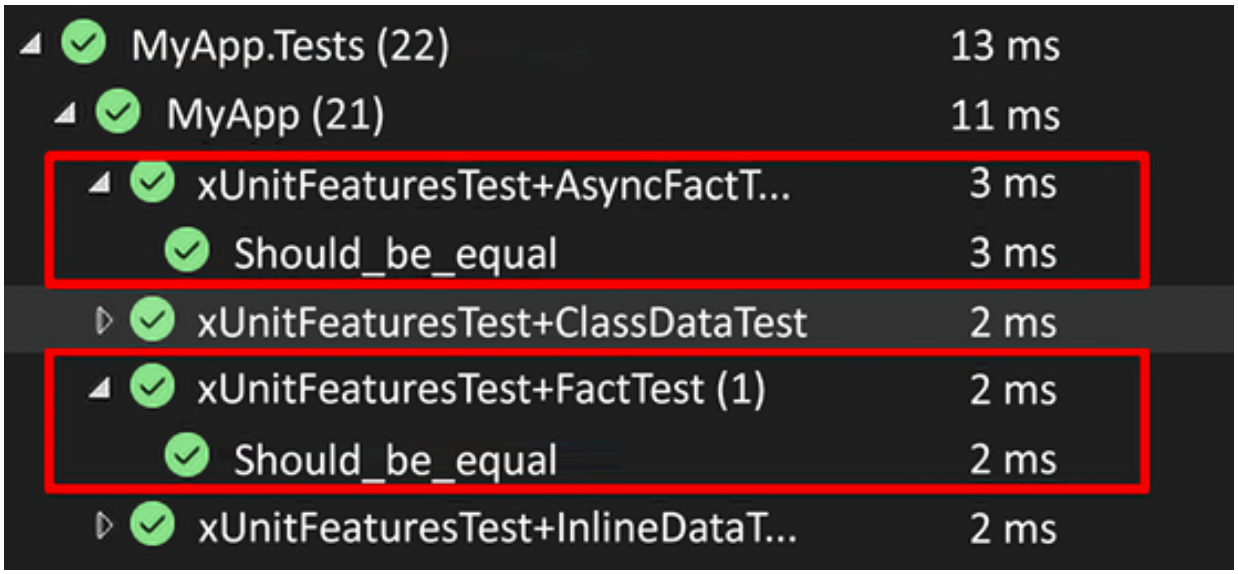


Figure 2.3: Test results in Visual Studio

You may have noticed from the screenshot that the test classes are nested in the `xUnitFeaturesTest` class, part of the `MyApp` namespace, and under the `MyApp.Tests` project. We explore those details later in the chapter.Running the `dotnet test` CLI command should yield a result similar to the following:

```
Passed!  - Failed:     0, Passed:    23, Skipped:     0, Total:    23, Duration: 22 ms - MyApp.Te
```

As we can read from the preceding output, all tests are passing, none have failed, and none were skipped. It is as simple as that to create test cases using xUnit.

> Learning the CLI can be very helpful in creating and debugging CI/CD pipelines, and you can use them, like the `dotnet test` command, in any script (like bash and PowerShell).

Have you noticed the `Assert` keyword in the test code? If you are not familiar with it, we will explore assertions next.

Assertions

An assertion is a statement that checks whether a particular condition is `true` or `false`. If the condition is `true`, the test passes. If the condition is `false`, the test fails, indicating a problem with the subject under test.Let's visit a few ways to assert correctness. We use barebone xUnit functionality in this section, but you can bring in the assertion library of your choice if you have one.

> In xUnit, the assertion throws an exception when it fails, but you may never even realize that. You do not have to handle those; that's the mechanism to propagate the failure result to the test runner.

We won't explore all possibilities, but let's start with the following shared pieces:

```
public class AssertionTest
{
    [Fact]
    public void Exploring_xUnit_assertions()
    {
        object obj1 = new MyClass { Name = "Object 1" };
        object obj2 = new MyClass { Name = "Object 1" };
        object obj3 = obj1;
        object? obj4 = default(MyClass);
        //
        // Omitted assertions
```

```
        //
        static void OperationThatThrows(string name)
        {
            throw new SomeCustomException { Name = name };
        }
    }
    private record class MyClass
    {
        public string? Name { get; set; }
    }
    private class SomeCustomException : Exception
    {
        public string? Name { get; set; }
    }
}
```

The two preceding record classes, the `OperationThatThrows` method, and the variables are utilities used in the test to help us play with xUnit assertions. The variables are of type `object` for exploration purposes, but you can use any type in your test cases. I omitted the assertion code that we are about to see to keep the code leaner.The following two assertions are very explicit:

```
Assert.Equal(expected: 2, actual: 2);
Assert.NotEqual(expected: 2, actual: 1);
```

The first compares whether the actual value equals the expected value, while the second compares if the two values are different. `Assert.Equal` is probably the most commonly used assertion method.

> As a rule of thumb, it is better to assert equality (`Equal`) than assert that the values are different (`NotEqual`). Except in a few rare cases, asserting equality will yield more consistent results and close the door to missing defects.

The next two assertions are very similar to the equality ones but assert that the objects are the same instance or not (the same instance means the same reference):

```
Assert.Same(obj1, obj3);
Assert.NotSame(obj2, obj3);
```

The next one validates that the two objects are equal. Since we are using record classes, it makes it super easy for us; `obj1` and `obj2` are not the same (two instances) but are equal (see *Appendix A* for more information on record classes):

```
Assert.Equal(obj1, obj2);
```

The next two are very similar and assert that the value is `null` or not:

```
Assert.Null(obj4);
Assert.NotNull(obj3);
```

The next line asserts that `obj1` is of the `MyClass` type and then returns the argument (`obj1`) converted to the asserted type (`MyClass`). If the type is incorrect, the `IsType` method will throw an exception:

```
var instanceOfMyClass = Assert.IsType<MyClass>(obj1);
```

Then we reuse the `Assert.Equal` method to validate that the value of the `Name` property is what we expect:

```
Assert.Equal(expected: "Object 1", actual: instanceOfMyClass.Name);
```

The following code block asserts that the `testCode` argument throws an exception of the `SomeCustomException` type:

```
var exception = Assert.Throws<SomeCustomException>(
    testCode: () => OperationThatThrows("Toto")
);
```

The `testCode` argument executes the `OperationThatThrows` inline function we saw initially. The `Throws` method allows us to test some exception properties by returning the exception in the specified type. The same behavior as the `IsType` method happens here; if the exception is of the wrong type or no exception is thrown, the `Throws` method will fail the test.

> It is a good idea to ensure that not only the proper exception type is thrown, but the exception carries the correct values as well.

The following line asserts that the value of the `Name` property is what we expect it to be, ensuring our program would propagate the proper exception:

```
Assert.Equal(expected: "Toto", actual: exception.Name);
```

We covered a few assertion methods, but many others are part of xUnit, like the `Collection`, `Contains`, `False`, and `True` methods. We use many assertions throughout the book, so if these are still unclear, you will learn more about them.Next, let's look at data-driven test cases using theories.

Theories

For more complex test cases, we can use theories. A theory contains two parts:

- A `[Theory]` attribute that marks the method as a theory.
- At least one data attribute that allows passing data to the test method: `[InlineData]`, `[MemberData]`, or `[ClassData]`.

When writing a theory, your primary constraint is ensuring that the number of values matches the parameters defined in the test method. For example, a theory with one parameter must be fed one value. We look at some examples next.

> You are not limited to only one type of data attribute; you can use as many as you need to suit your needs and feed a theory with the appropriate data.

The `[InlineData]` attribute is the most suitable for constant values or smaller sets of values. Inline data is the most straightforward way of the three because of the proximity of the test values and the test method.Here is an example of a theory using inline data:

```
public class InlineDataTest
{
    [Theory]
    [InlineData(1, 1)]
    [InlineData(2, 2)]
    [InlineData(5, 5)]
    public void Should_be_equal(int value1, int value2)
    {
        Assert.Equal(value1, value2);
    }
}
```

That test method yields three test cases in the Test Explorer, where each can pass or fail individually. Of course, since 1 equals 1, 2 equals 2, and 5 equals 5, all three test cases are passing, as shown here:

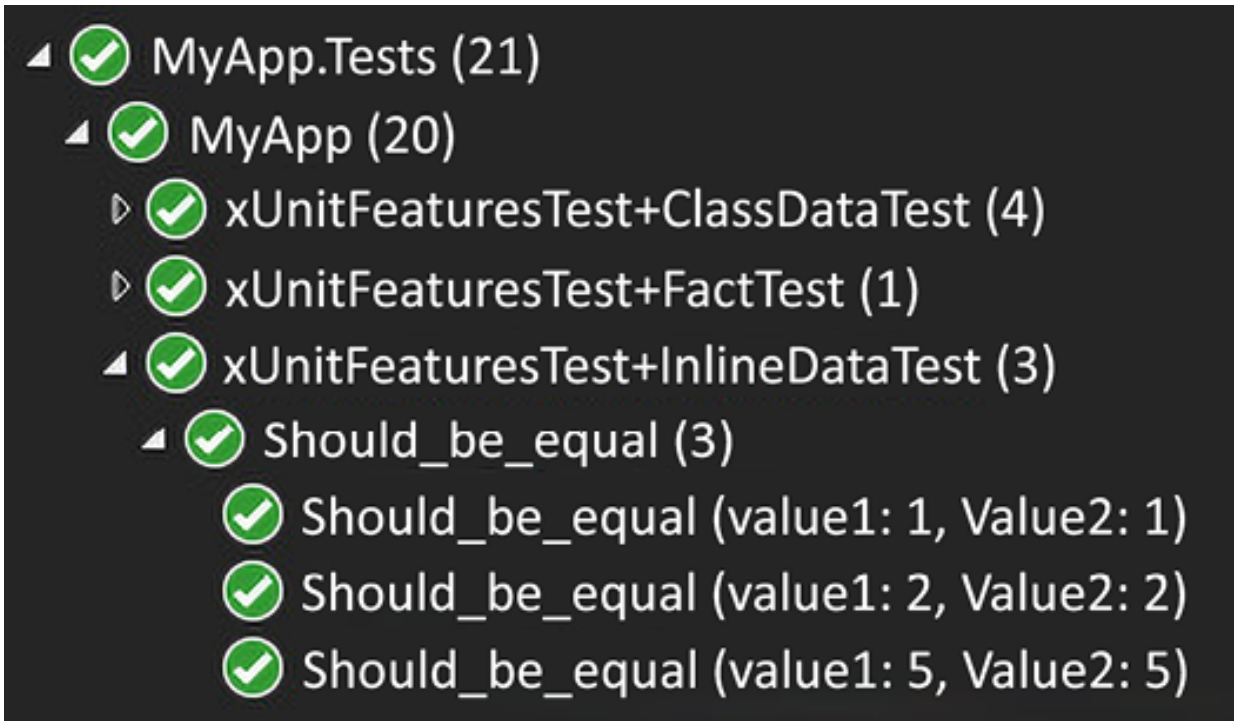


Figure 2.4: Inline data theory test results

We can also use the `[MemberData]` and `[ClassData]` attributes to simplify the test method's declaration when we have a large set of data to tests. We can also do that when it is impossible to instantiate the data in the attribute. We can also reuse the data in multiple test methods or encapsulate the data away from the test class.Here is a medley of examples of the `[MemberData]` attribute usage:

```
public class MemberDataTest
{
    public static IEnumerable<object[]> Data => new[]
    {
        new object[] { 1, 2, false },
        new object[] { 2, 2, true },
        new object[] { 3, 3, true },
    };
    public static TheoryData<int, int, bool> TypedData =>new TheoryData<int, int, bool>
    {
        { 3, 2, false },
        { 2, 3, false },
        { 5, 5, true },
    };
    [Theory]
    [MemberData(nameof(Data))]
    [MemberData(nameof(TypedData))]
    [MemberData(nameof(ExternalData.GetData), 10, MemberType = typeof(ExternalData))]
    [MemberData(nameof(ExternalData.TypedData), MemberType = typeof(ExternalData))]
    public void Should_be_equal(int value1, int value2, bool shouldBeEqual)
    {
        if (shouldBeEqual)
        {
            Assert.Equal(value1, value2);
        }
        else
        {
            Assert.NotEqual(value1, value2);
       }
    }
    public class ExternalData
    {
```

```
        public static IEnumerable<object[]> GetData(int start) => new[]
        {
            new object[] { start, start, true },
            new object[] { start, start + 1, false },
            new object[] { start + 1, start + 1, true },
        };
        public static TheoryData<int, int, bool> TypedData => new TheoryData<int, int, bool>
        {
            { 20, 30, false },
            { 40, 50, false },
            { 50, 50, true },
        };
    }
}
```

The preceding test case yields 12 results. If we break it down, the code starts by loading three sets of data from the `Data` property by decorating the test method with the `[MemberData(nameof(Data))]` attribute. This is how to load data from a member of the class the test method is declared in.Then, the second property is very similar to the `Data` property but replaces `IEnumerable<object[]>` with a `TheoryData<…>` class, making it more readable and type-safe. Like with the first attribute, we feed those three sets of data to the test method by decorating it with the `[MemberData(nameof(TypedData))]` attribute. Once again, it is part of the test class.

I strongly recommend using `TheoryData<…>` by default.

The third data feeds three more sets of data to the test method. However, that data originates from the `GetData` method of the `ExternalData` class, sending `10` as an argument during the execution (the `start` parameter). To do that, we must specify the `MemberType` instance where the method is located so xUnit knows where to look. In this case, we pass the argument `10` as the second parameter of the `MemberData` constructor. However, in other cases, you can pass zero or more arguments there.Finally, we are doing the same for the `ExternalData.TypedData` property, which is represented by the `[MemberData(nameof(ExternalData.TypedData), MemberType = typeof(ExternalData))]` attribute. Once again, the only difference is that the property is defined using `TheoryData` instead of `IEnumerable<object[]>`, which makes its intent clearer.When running the tests, the data provided by the `[MemberData]` attributes are combined, yielding the following result in the Test Explorer:

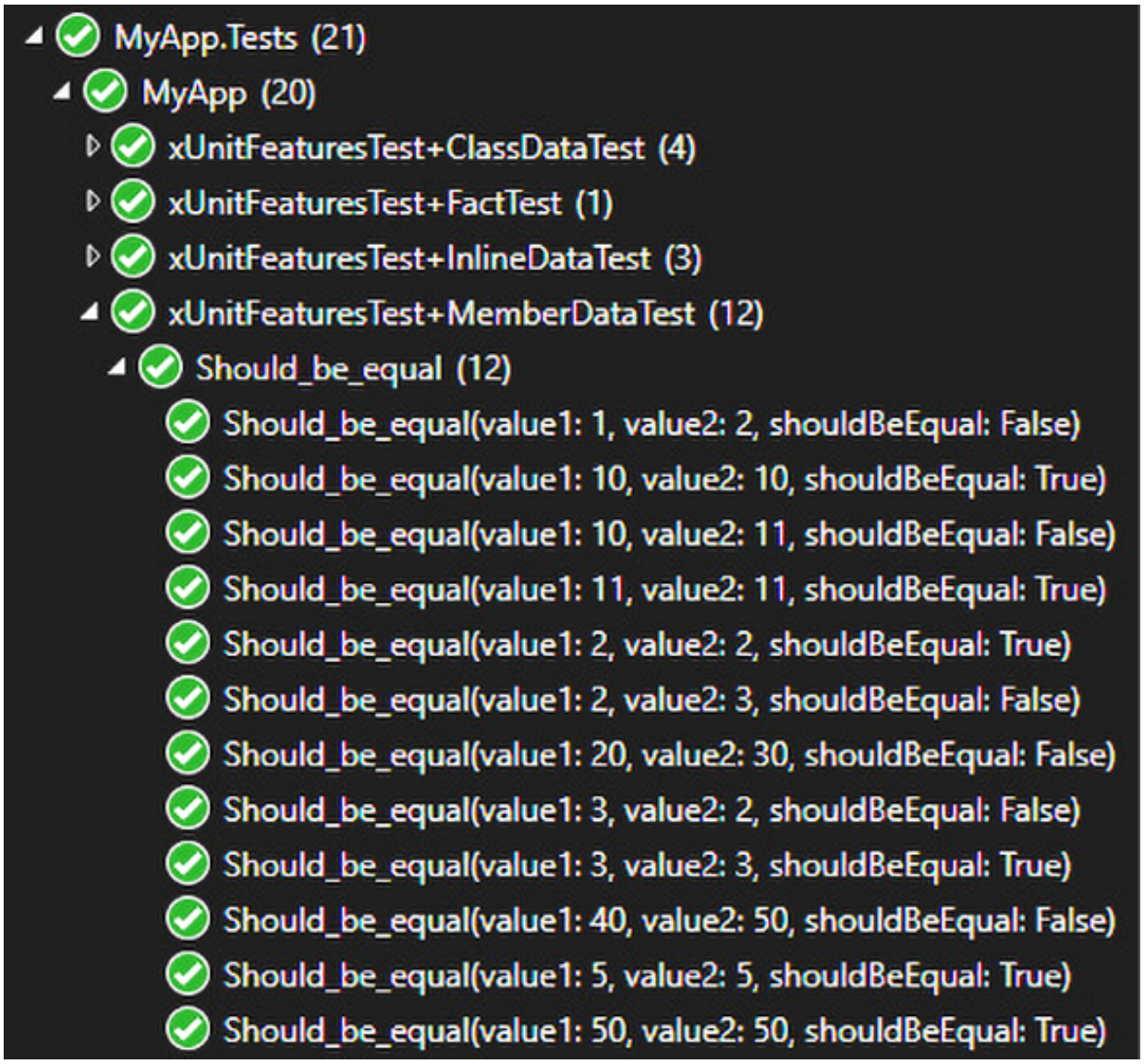


Figure 2.5: Member data theory test results

These are only a few examples of what we can do with the `[MemberData]` attribute.

> I understand that's a lot of condensed information, but the goal is to cover just enough to get you started. I don't expect you to become an expert in xUnit by reading this chapter.

Last but not least, the `[ClassData]` attribute gets its data from a class implementing `IEnumerable<object[]>` or inheriting from `TheoryData<…>`. The concept is the same as the other two. Here is an example:

```
public class ClassDataTest
{
    [Theory]
    [ClassData(typeof(TheoryDataClass))]
    [ClassData(typeof(TheoryTypedDataClass))]
    public void Should_be_equal(int value1, int value2, bool shouldBeEqual)
    {
        if (shouldBeEqual)
        {
            Assert.Equal(value1, value2);
        }
```

```
        else
        {
            Assert.NotEqual(value1, value2);
        }
    }
    public class TheoryDataClass : IEnumerable<object[]>
    {
        public IEnumerator<object[]> GetEnumerator()
        {
            yield return new object[] { 1, 2, false };
            yield return new object[] { 2, 2, true };
            yield return new object[] { 3, 3, true };
        }
        IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
    }
    public class TheoryTypedDataClass : TheoryData<int, int, bool>
    {
        public TheoryTypedDataClass()
        {
            Add(102, 104, false);
        }
    }
}
```

These are very similar to `[MemberData]`, but we point to a type instead of pointing to a member.In `TheoryDataClass`, implementing the `IEnumerable<object[]>` interface makes it easy to `yield return` the results. On the other hand, in the `TheoryTypedDataClass` class, by inheriting `TheoryData`, we can leverage a list-like `Add` method. Once again, I find inheriting from `TheoryData` more explicit, but either way works with xUnit. You have many options, so choose the best one for your use case.Here is the result in the Test Explorer, which is very similar to the other attributes:

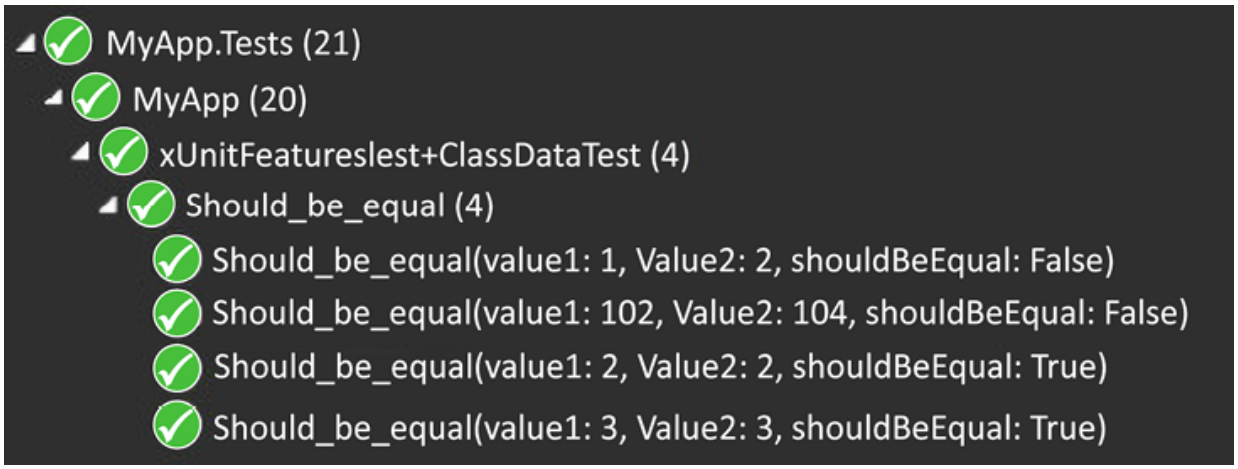


Figure 2.6: Test Explorer

That's it for the theories—next, a few last words before organizing our tests.

Closing words

Now that facts, theories, and assertions are out of the way, xUnit offers other mechanics to allow developers to inject dependencies into their test classes. These are named fixtures. Fixtures allow dependencies to be reused by all test methods of a test class by implementing the `IClassFixture<T>` interface. Fixtures are very helpful for costly dependencies, like creating an in-memory database. With fixtures, you can create the dependency once and use it multiple times. The `ValuesControllerTest` class in the `MyApp.IntegrationTests` project shows that in action.It is important to note that xUnit creates an instance of the test class for every test run, so your dependencies are recreated every time if you are not using the fixtures.You can also share the dependency provided by the fixture between multiple test classes by using `ICollectionFixture<T>`, `[Collection]`, and `[CollectionDefinition]`

instead. We won’t get into the details here, but at least you know it’s possible and know what types to look for when you need something similar.Finally, if you have worked with other testing frameworks, you might have encountered **setup** and **teardown** methods. In xUnit, there are no particular attributes or mechanisms for handling setup and teardown code. Instead, xUnit uses existing OOP concepts:

- To set up your tests, use the class constructor.
- To tear down (clean up) your tests, implement `IDisposable` or `IAsyncDisposable` and dispose of your resources there.

That’s it, xUnit is very simple and powerful, which is why I adopted it as my main testing framework several years ago and chose it for this book.Next, we learn to write readable test methods.

Arrange, Act, Assert

Arrange, Act, Assert (AAA or 3A) is a well-known method for writing readable tests. This technique allows you to clearly define your setup (arrange), the operation under test (act), and your assertions (assert). One efficient way to use this technique is to start by writing the 3A as comments in your test case and then write the test code in between. Here is an example:

```
[Fact]
public void Should_be_equals()
{
    // Arrange
    var a = 1;
    var b = 2;
    var expectedResult = 3;
    // Act
    var result = a + b;
    // Assert
    Assert.Equal(expectedResult, result);
}
```

Of course, that test case cannot fail, but the three blocks are easily identifiable with the 3A comments.In general, **you want the Act block of your unit tests to be a single line**, making the test focus clear. If you need more than one line, the chances are that something is wrong in the test or the design.

> When the tests are very small (only a few lines), removing the comments might help readability. Furthermore, when you have nothing to set up in your test case, delete the Arrange comment to improve its readability further.

Next, we learn how to organize tests into projects, directories, and files.

Organizing your tests

There are many ways of organizing test projects inside a solution, and I tend to create a unit test project for each project in the solution and one or more integration test projects.A unit test is directly related to a single unit of code, whether it’s a method or a class. It is straightforward to associate a unit test project with its respective code project (assembly), leading to a one-on-one relationship. One unit test project per assembly makes them portable, easier to navigate, and even more so when the solution grows.

> If you have a preferred way to organize yours that differs from what we are doing in the book, by all means, use that approach instead.

Integration tests, on the other hand, can span multiple projects, so having a single rule that fits all scenarios is challenging. One integration test project per solution is often enough. Sometimes we can need more than one, depending on the context.

> I recommend starting with one integration test project and adding more as needed during development instead of overthinking it before getting started. Trust your judgment; you can always change the structure as your

project evolves.

Folder-wise, at the solution level, creating the application and its related libraries in an `src` directory helps isolate the actual solution code from the test projects created under a `test` directory, like this:

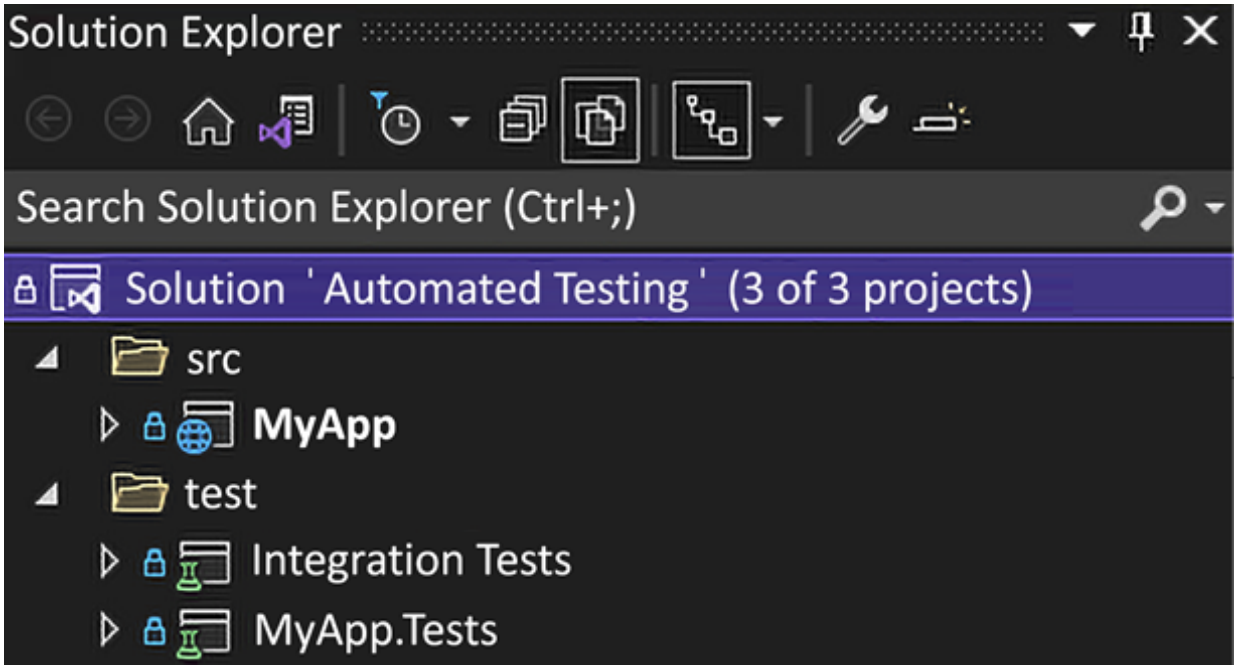


Figure 2.7: The Automated Testing Solution Explorer, displaying how the projects are organized

That's a well-known and effective way of organizing a solution in the .NET world.

> Sometimes, it is not possible or unwanted to do that. One such use case would be multiple microservices written under a single solution. In that case, you might want the tests to live closer to your microservices and not split them between `src` and `test` folders. So you could organize your solution by microservice instead, like one directory per microservice that contains all the projects, including tests.

Let's now dig deeper into organizing unit tests.

Unit tests

How you organize your test projects may make a big difference between searching for your tests or making it easy to find them. Let's look at the different aspects, from the `namespace` to the test code itself.

Namespace

I find it convenient to create unit tests in the same namespace as the subject under test when creating unit tests. That helps get tests and code aligned without adding any additional using statements. To make it easier when creating files, you can change the default namespace used by Visual Studio when creating a new class in your test project by adding `<RootNamespace>[Project under test namespace]</RootNamespace>` to a `PropertyGroup` of the test project file (`*.csproj`), like this:

```
<PropertyGroup>
  ...
  <RootNamespace>MyApp</RootNamespace>
</PropertyGroup>
```

Test class name

By convention, I name test classes `[class under test]Test.cs` and create them in the same directory as in the original project. Finding tests is easy when following that simple rule since the test code is in the same location of the file tree as the code under test but in two distinct projects.

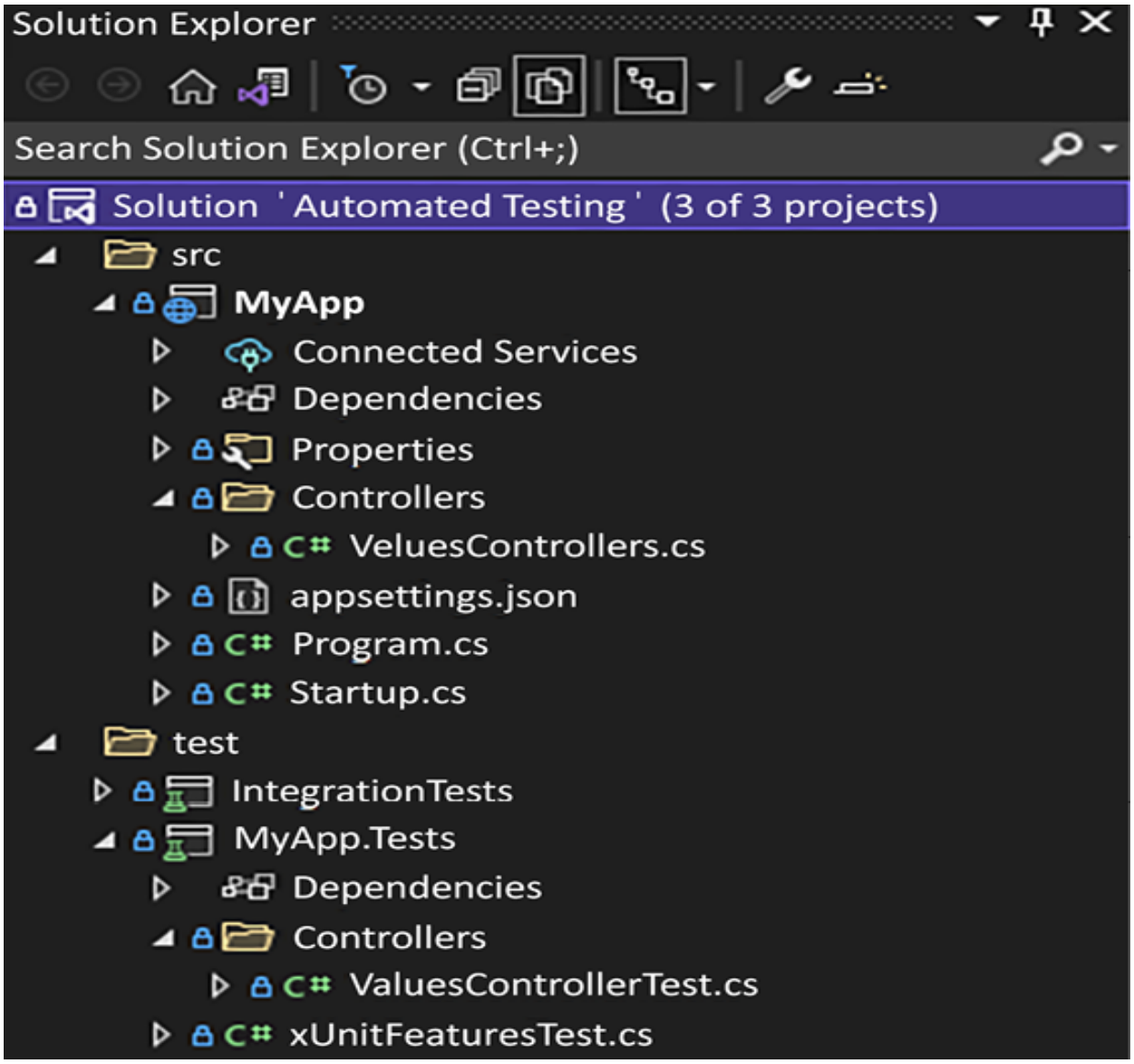


Figure 2.8: The Automated Testing Solution Explorer, displaying how tests are organized

Test code inside the test class

For the test code itself, I follow a multi-level structure similar to the following:

- One test class is named the same as the class under test.
- One nested test class per method to test from the class under test.
- One test method per test case of the method under test.

This technique helps organize tests by test case while keeping a clear hierarchy, leading to the following hierarchy:

- Class under test
- Method under test
- Test case using that method

In code, that translates to the following:

```
namespace MyApp.IntegrationTests.Controllers;
public class ValuesControllerTest
{
    public class Get : ValuesControllerTest
    {
        [Fact]
        public void Should_return_the_expected_strings()
        {
            // Arrange
            var sut = new ValuesController();
            // Act
            var result = sut.Get();
            // Assert
            Assert.Collection(result.Value,
                x => Assert.Equal("value1", x),
                x => Assert.Equal("value2", x)
            );
        }
    }
}
```

This convention allows you to set up tests step by step. For example, by inheriting the outer class (the `ValuesControllerTest` class here) from the inner class (the `Get` nested class), you can create top-level private mocks or classes shared by all nested classes and test methods. Then, for each method to test, you can modify the setup or create other private test elements in the nested classes. Finally, you can do more configuration per test case inside the test method (the `Should_return_the_expected_strings` method here).

> Don’t go too hard on reusability inside your test classes, as it can make tests harder to read from an external eye, such as a reviewer or another developer that needs to play there. Unit tests should remain focused, small, and easy to read: a unit of code testing another unit of code. Too much reusability may lead to a brittle test suite.

Now that we have explored organizing unit tests, let’s look at integration tests.

Integration tests

Integration tests are harder to organize because they depend on multiple units, can cross project boundaries, and interact with various dependencies.We can create one integration test project for most simple solutions or many for more complex scenarios.When creating one, you can name the project `IntegrationTests` or start with the entry point of your tests, like a REST API project, and name the project `[Name of the API project].IntegrationTests`. At this point, how to name the integration test project depends on your solution structure and intent.When you need multiple integration projects, you can follow a convention similar to unit tests and associate your integration projects one-to-one: `[Project under test].IntegrationTests`.Inside those projects, it depends on how you want to attack the problem and the structure of the solution itself. Start by identifying the features under test. Name the test classes in a way that mimics your requirements, organize those into sub-folders (maybe a category or group of requirements), and code test cases as methods. You can also leverage nested classes, as we did with unit tests.

> We write tests throughout the book, so you will have plenty of examples to make sense of all this if it’s not clear now.

Next, we implement an integration test by leveraging ASP.NET Core features.

Writing ASP.NET Core integration tests

When Microsoft built ASP.NET Core from the ground up, they fixed and improved so many things that I cannot enumerate them all here, including testability.Nowadays, there are two ways to structure a .NET program:

- The classic ASP.NET Core `Program` and the `Startup` classes. This model might be found in existing projects (created before .NET 6).
- The minimal hosting model introduced in .NET 6. This may look familiar to you if you know Node.js, as this model encourages you to write the start-up code in the Program.cs file by leveraging top-level statements. You will most likely find this model in new projects (created after the release of .NET 6).

No matter how you write your program, that's the place to define how the application's composition and how it boots. Moreover, we can leverage the same testing tools more or less seamlessly.In the case of a web application, the scope of our integration tests is often to call the endpoint of a controller over HTTP and assert the response. Luckily, in .NET Core 2.1, the .NET team added the `WebApplicationFactory<TEntry>` class to make the integration testing of web applications easier. With that class, we can boot up an ASP.NET Core application in memory and query it using the supplied `HttpClient` in a few lines of code. The test classes also provide extension points to configure the server, such as replacing implementations with mocks, stubs, or other test-specific elements.Let's start by booting up a classic web application test.

Classic web application

In a classic ASP.NET Core application, the `TEntry` generic parameter of the `WebApplicationFactory<TEntry>` class is usually the `Startup` or `Program` class of your project under test.

The test cases are in the `Automated Testing` solution under the `MyApp.IntegrationTests` project.

Let's start by looking at the test code structure before breaking it down:

```
namespace MyApp.IntegrationTests.Controllers;
public class ValuesControllerTest : IClassFixture<WebApplicationFactory<Startup>>
{
    private readonly HttpClient _httpClient;
    public ValuesControllerTest(
        WebApplicationFactory<Startup> webApplicationFactory)
    {
        _httpClient = webApplicationFactory.CreateClient();
    }
    public class Get : ValuesControllerTest
    {
        public Get(WebApplicationFactory<Startup> webApplicationFactory)
            : base(webApplicationFactory) { }
        [Fact]
        public async Task Should_respond_a_status_200_OK()
        {
            // Omitted Test Case 1
        }
        [Fact]
        public async Task Should_respond_the_expected_strings()
        {
            // Omitted Test Case 2
        }
    }
}
```

The first piece of the preceding code that is relevant to us is how we get an instance of the `WebApplicationFactory<Startup>` class. We inject a `WebApplicationFactory<Startup>` object into the constructor by implementing the `IClassFixture<T>` interface (a xUnit feature). We can also use the factory to configure the test server, but we don't need to here, so we can only keep a reference on the `HttpClient`, preconfigured to connect to the in-memory test server.Then, we may have noticed we have the nested `Get` class that inherits the `ValuesControllerTest` class. The `Get` class contains the test cases. By inheriting the `ValuesControllerTest` class, we can leverage the `_httpClient` field from the test cases we are about to see.In the first test case, we use `HttpClient` to query the `http://localhost/api/values` URI, accessible through the in-memory server. Then, we assert that the status code of the HTTP response was a success (`200 OK`):

```
[Fact]
public async Task Should_respond_a_status_200_OK()
{
    // Act
    var result = await _httpClient
        .GetAsync("/api/values");
    // Assert
    Assert.Equal(HttpStatusCode.OK, result.StatusCode);
}
```

The second test case also sends an HTTP request to the in-memory server but deserializes the body's content as a string[] to ensure the values are the same as expected instead of validating the status code:

```
[Fact]
public async Task Should_respond_the_expected_strings()
{
    // Act
    var result = await _httpClient
        .GetFromJsonAsync<string[]>("/api/values");
    // Assert
    Assert.Collection(result,
        x => Assert.Equal("value1", x),
        x => Assert.Equal("value2", x)
    );
}
```

> As you may have noticed from the test cases, the `WebApplicationFactory` preconfigured the `BaseAddress` property for us, so we don't need to prefix our requests with `http://localhost`.

When running those tests, an in-memory web server starts. Then, HTTP requests are sent to that server, testing the complete application. The tests are simple in this case, but you can create more complex test cases in more complex programs.Next, we explore how to do the same for minimal APIs.

Minimal hosting

Unfortunately, we must use a workaround to make the `Program` class discoverable when using minimal hosting. Let's explore a few workarounds that leverage minimal APIs, allowing you to pick the one you prefer.

First workaround

The **first workaround** is to use any other class in the assembly as the `TEntryPoint` of `WebApplicationFactory<TEntryPoint>` instead of the `Program` or `Startup` class. This makes what `WebApplicationFactory` does a little less explicit, but that's all. Since I tend to prefer readable code, I do not recommend this.

Second workaround

The **second workaround** is to add a line at the bottom of the `Program.cs` file (or anywhere else in the project) to change the autogenerated `Program` class visibility from `internal` to `public`. Here is the complete `Program.cs` file with that added line (highlighted):

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
app.MapGet("/", () => "Hello World!");
app.Run();
public partial class Program { }
```

Then, the test cases are very similar to the ones of the classic web application explored previously. The only difference is the program itself, both programs don't do the same thing.

```
namespace MyMinimalApiApp;
public class ProgramTest : IClassFixture<WebApplicationFactory<Program>>
```

```
{
    private readonly HttpClient _httpClient;
    public ProgramTest(
        WebApplicationFactory<Program> webApplicationFactory)
    {
        _httpClient = webApplicationFactory.CreateClient();
    }
    public class Get : ProgramTest
    {
        public Get(WebApplicationFactory<Program> webApplicationFactory)
            : base(webApplicationFactory) { }
        [Fact]
        public async Task Should_respond_a_status_200_OK()
        {
            // Act
            var result = await _httpClient.GetAsync("/");
            // Assert
            Assert.Equal(HttpStatusCode.OK, result.StatusCode);
        }
        [Fact]
        public async Task Should_respond_hello_world()
        {
            // Act
            var result = await _httpClient.GetAsync("/");
            // Assert
            var contentText = await result.Content.ReadAsStringAsync();
            Assert.Equal("Hello World!", contentText);
        }
    }
}
```

The only change is the expected result as the endpoint returns the `text/plain` string `Hello World!` instead of a collection of strings serialized as JSON. The test cases would be identical if the two endpoints produced the same result.

Third workaround

The **third workaround** is to instantiate `WebApplicationFactory` manually instead of leveraging a fixture. We can use the `Program` class, which requires changing its visibility by adding the following line to the `Program.cs` file:

```
public partial class Program { }
```

However, instead of injecting the instance using the `IClassFixture` interface, we instantiate the factory manually. To ensure we dispose the `WebApplicationFactory` instance, we also implement the `IAsyncDisposable` interface.Here's the complete example, which is very similar to the previous workaround:

```
namespace MyMinimalApiApp;
public class ProgramTestWithoutFixture : IAsyncDisposable
{
    private readonly WebApplicationFactory<Program> _webApplicationFactory;
    private readonly HttpClient _httpClient;
    public ProgramTestWithoutFixture()
    {
        _webApplicationFactory = new WebApplicationFactory<Program>();
        _httpClient = _webApplicationFactory.CreateClient();
    }
    public ValueTask DisposeAsync()
    {
        return ((IAsyncDisposable)_webApplicationFactory)
            .DisposeAsync();
    }
    // Omitted nested Get class
}
```

I omitted the test cases in the preceding code block because they are the same as the previous workarounds. The full source code is available on GitHub: https://adpg.link/vzkr.

Using class fixtures is more performant since the factory and the server get created only once per test run instead of recreated for every test method.

Creating a test application

Finally, we can create a dedicated class that instantiates `WebApplicationFactory` manually. It leverages the other workarounds but makes the test cases more readable. By encapsulating the setup of the test application in a class, you will improve the reusability and maintenance cost in most cases.First, we need to change the `Program` class visibility by adding the following line to the `Project.cs` file:

```
public partial class Program { }
```

Now that we can access the Program class without the need to allow internal visibility to our test project, we can create our test application like this:

```
namespace MyMinimalApiApp;
public class MyTestApplication : WebApplicationFactory<Program> {}
```

Finally, we can reuse the same code to test our program but instantiate `MyTestApplication` instead of `WebApplicationFactory<Program>`, highlighted in the following code:

```
namespace MyMinimalApiApp;
public class MyTestApplicationTest
{
    public class Get : ProgramTestWithoutFixture
    {
        [Fact]
        public async Task Should_respond_a_status_200_OK()
        {
            // Arrange
            await using var app = new MyTestApplication();
            var httpClient = app.CreateClient();
            // Act
            var result = await httpClient.GetAsync("/");
            // Assert
            Assert.Equal(HttpStatusCode.OK, result.StatusCode);
        }
    }
}
```

You can also leverage fixtures, but for the sake of simplicity, I decided to show you how to instantiate our new test application manually.And that’s it. We have covered multiple ways to work around integration testing minimal APIs simplistically and elegantly. Next, we explore a few testing principles before moving to architectural principles in the next chapter.

Important testing principles

One essential thing to remember when writing tests is to test use cases, not the code itself; we are testing features’ correctness, not code correctness. Of course, if the expected outcome of a feature is correct, that also means the codebase is correct. However, it is not always true the other way around; correct code may yield an incorrect outcome. Also, remember that code costs money to write, while features deliver value.To help with that, test requirements should revolve around **inputs and outputs**. When specific values go into your subject under test, you expect particular values to come out. Whether you are testing a simple `Add` method where the ins are two or more numbers, and the out is the sum of those numbers, or a more complex feature where the ins come from a form, and the out is the record getting persisted in a database, most of the time, we are testing that inputs produced an output or an outcome.Another concept is to divide those units as a query or a command. No matter how you organize your code, from a simple single-file application to a microservices architecture-base Netflix clone, all simple or compounded operations are queries or commands. Thinking about a system this way should help you test the ins and outs. We discuss queries and commands in several chapters, so keep reading to learn more.Now that we have laid this out, what if a unit must perform multiple operations, such as reading from a database, and then send

multiple commands? You can create and test multiple smaller units (individual operations) and another unit that orchestrates those building blocks, allowing you to test each piece in isolation. We explore how to achieve this throughout the book.In a nutshell, when writing automated tests:

- In case of a query, we assert the output of the unit undergoing testing based on its input parameters.
- In case of a command, we assert the outcome of the unit undergoing testing based on its input parameters.

We explore numerous techniques throughout the book to help you achieve that level of separation, starting with architectural principles in the next chapter.

Summary

This chapter covered automated testing, such as unit and integration tests. We also briefly covered end-to-end tests, but covering that in only a few pages is impossible. Nonetheless, how to write integration tests can also be used for end-to-end testing, especially in the REST API space.We explored different testing approaches from a bird's eye view, tackled technical debt, and explored multiple testing techniques like black-box, white-box, and grey-box testing. We also peaked at a few formal ways to choose the values to test, like equivalence partitioning and boundary value analysis.We then looked at xUnit, the testing framework used throughout the book, and a way of organizing tests. We explored ways to pick the correct type of test and some guidelines about choosing the right quantity for each kind of test. Then we saw how easy it is to test our ASP.NET Core web applications by running it in memory. Finally, we explored high-level concepts that should guide you in writing testable, flexible, and reliable programs.Now that we have talked about testing, we are ready to explore a few architectural principles to help us increase programs' testability. Those are a crucial part of modern software engineering and go hand in hand with automated testing.

Questions

Let's take a look at a few practice questions:

1. Is it true that in TDD, you write tests before the code to be tested?
2. What is the role of unit tests?
3. How big can a unit test be?
4. What type of test is usually used when the subject under test has to access a database?
5. Is doing TDD required?
6. Do you need to know the inner working of the application to do black-box testing?

Further reading

Here are some links to build upon what we have learned in the chapter:

- xUnit: https://xunit.net/
- If you use Visual Studio, I have a few handy snippets to help improve productivity. They are available on GitHub: https://adpg.link/5TbY

3 Architectural Principles

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter delves into fundamental architectural principles: pillars of contemporary software development practices. These principles help us create flexible, resilient, testable, and maintainable code.We can use these principles to stimulate critical thinking, fostering our ability to evaluate trade-offs, anticipate potential issues, and create solutions that stand the test of time by influencing our decision-making process and helping our design choices.As we embark on this journey, we constantly refer to those principles throughout the book, particularly the SOLID principles, which improve our ability to build flexible and robust software systems.In this chapter, we cover the following topics:

- The separation of concerns (SoC) principle
- The **DRY** principle
- The **KISS** principle
- The **SOLID** principles

We also revise the following notions:

- Covariance
- Contravariance
- Interfaces

Separation of concerns (SoC)

As its name implies, the idea is to separate our software into logical blocks, each representing a concern. A “concern” refers to a specific aspect of a program. It’s a particular interest or focus within a system that serves a distinct purpose. Concerns could be as broad as data management, as specific as user authentication, or even more specific, like copying an object into another. The Separation of Concerns principle suggests that each concern should be isolated and managed separately to improve the system’s maintainability, modularity, and understandability.

> The Separation of Concerns principle applies to all programming paradigms. In a nutshell, this principle means factoring a program into the correct pieces. For example, modules, subsystems, and microservices are macro-pieces, while classes and methods are smaller pieces.

By correctly separating concerns, we can prevent changes in one area from affecting others, allow for more efficient code reuse, and make it easier to understand and manage different parts of a system independently.Here are a few examples:

- Security and logging are cross-cutting concerns.
- Rendering a user interface is a concern.

- Handling an HTTP request is a concern.
- Copying an object into another is a concern.
- Orchestrating a distributed workflow is a concern.

Before moving to the DRY principle, it is imperative to consider concerns when dividing software into pieces to create cohesive units. A good separation of concerns helps create modular designs and face design dilemmas more effectively, leading to a maintainable application.

Don’t repeat yourself (DRY)

The DRY principle advocates the separation of concerns principle and aims to eliminate redundancy in code as well. It promotes the idea that each piece of knowledge or logic should have a single, unambiguous representation within a system.So, when you have duplicated logic in your system, encapsulate it and reuse that new encapsulation in multiple places instead. If you find yourself writing the same or similar code in multiple places, refactor that code into a reusable component instead. Leverage functions, classes, modules, or other abstractions to refactor the code.Adhering to the DRY principle makes your code more maintainable, less error-prone, and easier to modify because a change in logic or bug fix needs to be made in only one place, reducing the likelihood of introducing errors or inconsistencies.However, it is imperative to regroup duplicated logic by concern, not only by the similarities of the code itself. Let’s look at those two classes:

```
public class AdminApp
{
    public async Task DisplayListAsync(
        IBookService bookService,
        IBookPresenter presenter)
    {
        var books = await bookService.FindAllAsync();
        foreach (var book in books)
        {
            await presenter.DisplayAsync(book);
        }
    }
}
public class PublicApp
{
    public async Task DisplayListAsync(
        IBookService bookService,
        IBookPresenter presenter)
    {
        var books = await bookService.FindAllAsync();
        foreach (var book in books)
        {
            await presenter.DisplayAsync(book);
        }
    }
}
```

The code is very similar, but encapsulating a single class or method could very well be a mistake. Why? Keeping two separate classes is more logical because the admin program can have different reasons for modification compared to the public program.However, encapsulating the list logic into the `IBookPresenter` interface could make sense. It would allow us to react differently to both types of users if needed, like filtering the admin panel list but doing something different in the public section. One way to do this is by replacing the `foreach` loop with a `presenter DisplayListAsync(books)` call, like the following highlighted code:

```
public class AdminApp
{
    public async Task DisplayListAsync(
        IBookService bookService,
        IBookPresenter presenter)
    {
        var books = await bookService.FindAllAsync();
        // We could filter the list here
        await presenter.DisplayListAsync(books);
    }
```

```
}
public class PublicApp
{
    public async Task DisplayListAsync(
        IBookService bookService,
        IBookPresenter presenter)
    {
        var books = await bookService.FindAllAsync();
        await presenter.DisplayListAsync(books);
    }
}
```

There is more to those simple implementations to discuss, like the possibility of supporting multiple implementations of the interfaces for added flexibility, but let's keep some subjects for further down the book.

> When you don't know how to name a class or a method, you may have identified a problem with your separation of concerns. This is a good indicator that you should go back to the drawing board. Nevertheless, naming is hard, so sometimes, that's just it.

Keeping our code DRY while following the separation of concerns principles is imperative. Otherwise, what may seem like a good move could become a nightmare.

Keep it simple, stupid (KISS)

This is another straightforward principle, yet one of the most important. Like in the real world, the more moving pieces, the more chances something breaks. This principle is a design philosophy that advocates for simplicity in design. It emphasizes the idea that systems work best when they are kept simple rather than made complex.Striving for simplicity might involve writing shorter methods or functions, minimizing the number of parameters, avoiding over-architecting, and choosing the simplest solution to solve a problem.Adding interfaces, abstraction layers, and complex object hierarchy adds complexity, but are the added benefits better than the underlying complexity? If so, they are worth it; otherwise, they are not.

> As a guiding principle, when you can write the same program with less complexity, do it. This is also why predicting future requirements can often prove detrimental, as it may inadvertently inject unnecessary complexity into your codebase for features that might never materialize.

We study design patterns in the book and design systems using them. We learn how to apply a high degree of engineering to our code, which can lead to over-engineering if done in the wrong context. Towards the end of the book, we circle back on the KISS principle when exploring the vertical slice architecture and request-endpoint-response (REPR) patterns.Next, we delve into the SOLID principles, which are the key to flexible software design.

The SOLID principles

SOLID is an acronym representing five principles that extend the basic OOP concepts of **Abstraction**, **Encapsulation**, **Inheritance**, and **Polymorphism**. They add more details about what to do and how to do it, guiding developers toward more robust and flexible designs.It is crucial to remember that these are just guiding principles, not rules that you must follow, no matter what. Think about what makes sense for your specific project. If you're building a small tool, it might be acceptable not to follow these principles as strictly as you would for a crucial business application. In the case of business-critical applications, it might be a good idea to stick to them more closely. Still, it's usually a smart move to follow them, no matter the size of your app. That's why we're discussing them before diving into design patterns.The SOLID acronym represents the following:

- **S**ingle responsibility principle
- **O**pen/Closed principle
- **L**iskov substitution principle
- **I**nterface segregation principle
- **D**ependency inversion principle

By following these principles, your systems should become easier to test and maintain.

Single responsibility principle (SRP)

Essentially, the SRP means that a single class should hold one, and only one, responsibility, leading me to the following quote:

> *"There should never be more than one reason for a class to change."— Robert C. Martin, originator of the single responsibility principle*

OK, but why? Before answering, take a moment to remember a project you've worked on where someone changed one or more requirements along the way. I recall several projects that would have benefited from this principle. Now, imagine how much simpler it would have been if each part of your system had just one job: one reason to change.

> Software maintainability problems can be due to both tech and non-tech people. Nothing is purely black or white—most things are a shade of gray. The same applies to software design: always do your best, learn from your mistakes, and stay humble (a.k.a. continuous improvement).

By understanding that applications are born to change, you will feel better when that happens, while the SRP helps mitigate the impact of changes. For example, it helps make our classes more readable and reusable and to create more flexible and maintainable systems. Moreover, when a class does only one thing, it's easier to see how changes will affect the system, which is more challenging with complex classes since one change might break other parts. Furthermore, fewer responsibilities mean less code. Less code is easier to understand, helping you grasp that part of the software more quickly.Let's try this out in action.

Project – Single Responsibility

First, we look at the `Product` class used in both code samples. That class represents a simple fictive product:

```
public record class Product(int Id, string Name);
```

> The code sample has no implementation because it is irrelevant to understanding the SRP. We focus on the class API instead. Please assume we implemented the data-access logic using your favorite database.

The following class breaks the SRP:

```
namespace BeforeSRP;
public class ProductRepository
{
    public ValueTask<Product> GetOnePublicProductAsync(int productId)
        => throw new NotImplementedException();
    public ValueTask<Product> GetOnePrivateProductAsync(int productId)
        => throw new NotImplementedException();
    public ValueTask<IEnumerable<Product>> GetAllPublicProductsAsync()
        => throw new NotImplementedException();
    public ValueTask<IEnumerable<Product>> GetAllPrivateProductsAsync()
        => throw new NotImplementedException();
    public ValueTask CreateAsync(Product product)
        => throw new NotImplementedException();
    public ValueTask UpdateAsync(Product product)
        => throw new NotImplementedException();
    public ValueTask DeleteAsync(Product product)
        => throw new NotImplementedException();
}
```

What does not conform to the SRP in the preceding class? By reading the name of the methods, we can extract two responsibilities:

- Handling public products (highlighted code).
- Handling private products.

The `ProductRepository` class mixes public and private product logic. From that API alone, there are many possibilities where an error could lead to leaking restricted data to public users. That is also true because the class exposes the private logic to the public-facing consumers; someone else could make a mistake.We are ready to rethink the class now that we identified the responsibilities. We know it has two responsibilities, so breaking the class into two sounds like an excellent first step. Let's start with extracting a public API:

```
namespace AfterSRP;
public class PublicProductReader
{
    public ValueTask<IEnumerable<Product>> GetAllAsync()
        => throw new NotImplementedException();
    public ValueTask<Product> GetOneAsync(int productId)
        => throw new NotImplementedException();
}
```

The `PublicProductReader` class now contains only two methods: `GetAllAsync` and `GetOneAsync`. When reading the name of the class and its methods, it is clear that the class handles only public product data. By lowering the complexity of the class, we made it easier to understandNext, let's do the same for the private products:

```
namespace AfterSRP;
public class PrivateProductRepository
{
    public ValueTask<IEnumerable<Product>> GetAllAsync()
        => throw new NotImplementedException();
    public ValueTask<Product> GetOneAsync(int productId)
        => throw new NotImplementedException();
    public ValueTask CreateAsync(Product product)
        => throw new NotImplementedException();
    public ValueTask DeleteAsync(Product product)
        => throw new NotImplementedException();
    public ValueTask UpdateAsync(Product product)
        => throw new NotImplementedException();
}
```

The `PrivateProductRepository` class follows the same pattern. It includes the read methods, named the same as the `PublicProductReader` class, and the mutation methods only users with private access can use.We improved our code's readability, flexibility, and security by splitting the initial class into two. However, one thing to be careful about with the SRP is not to over-separate classes. The more classes in a system, the more complex assembling the system can become, and the harder it can be to debug and follow the execution paths. On the other hand, many well-separated responsibilities should lead to a better, more testable system.It is tough to define one hard rule that defines "one reason" or "a single responsibility". However, as a rule of thumb, aim at packing a cohesive set of functionalities in a single class that revolves around its responsibility. You should strip out any excess logic and add missing pieces.A good indicator of the SRP violation is when you don't know how to name an element, which points towards the fact that the element should not reside there, that you should extract it, or that you should split it into multiple smaller pieces.

> Using precise names for variables, methods, classes, and other elements is very important and should not be overlooked.

Another good indicator is when a method becomes too big, maybe containing many `if` statements or loops. In that case, you can split that method into multiple smaller methods, classes, or any other construct that suits your requirements. That should make the code easier to read and make the initial method's body cleaner. It often also helps you get rid of useless comments and improve testability. Next, we explore how to change behaviors without modifying code, but before that, let's look at interfaces.

Open/Closed principle (OCP)

Let's start this section with a quote from Bertrand Meyer, the person who first wrote the term open/closed principle in 1988:

> *“Software entities (classes, modules, functions, and so on) should be open for extension but closed for modification.”*

OK, but what does that mean? It means you should be able to change the class behaviors from the outside without altering the code.As a bit of history, the first appearance of the OCP in 1988 referred to inheritance, and OOP has evolved a lot since then. Inheritance is still useful, but you should be careful as it is easily misused. Inheritance creates direct coupling between classes. You should, most of the time, opt for composition over inheritance.

> “Composition over inheritance” is a principle that suggests it’s better to build objects by combining simple, flexible parts (composition) rather than by inheriting properties from a larger, more complex object (inheritance).
>
> > Think of it like building with LEGO® blocks. It’s easier to build and adjust your creation if you put together small blocks (composition) rather than trying to alter a big, single block that already has a fixed shape (inheritance).

Meanwhile, we explore three versions of a business process to illustrate the OCP.

Project – Open Close

First, we look at the `Entity` and `EntityRepository` classes used in the code samples:

```
public record class Entity();
public class EntityRepository
{
    public virtual Task CreateAsync(Entity entity)
        => throw new NotImplementedException();
}
```

The `Entity` class represents a simple fictive entity with no properties; consider it anything you’d like. The `EntityRepository` class has a single `CreateAsync` method that inserts an instance of an `Entity` in a database (if it was implemented).

> The code sample has few implementation details because it is irrelevant to understanding the OCP. Please assume we implemented the `CreateAsync` logic using your favorite database.

For the rest of the sample, we refactor the `EntityService` class, beginning with a version that inherits the `EntityRepository` class, breaking the OCP:

```
namespace OCP.NoComposability;
public class EntityService : EntityRepository
{
    public async Task ComplexBusinessProcessAsync(Entity entity)
    {
        // Do some complex things here
        await CreateAsync(entity);
        // Do more complex things here
    }
}
```

As the namespace implies, the preceding `EntityService` class offers no composability. Moreover, we tightly coupled it with the `EntityRepository` class. Since we just covered the *composition over inheritance* principle, we can quickly isolate the problem: **inheritance**.As the next step to fix this mess, let’s extract a private `_repository` field to hold an `EntityRepository` instance instead:

```
namespace OCP.Composability;
public class EntityService
{
    private readonly EntityRepository _repository
        = new EntityRepository();
    public async Task ComplexBusinessProcessAsync(Entity entity)
    {
```

```
        // Do some complex things here
        await _repository.CreateAsync(entity);
        // Do more complex things here
    }
}
```

Now the `EntityService` is composed of an `EntityRepository` instance, and there is no more inheritance. However, we still tightly coupled both classes, and it is impossible to change the behavior of the `EntityService` this way without changing its code.To fix our last issues, we can inject an `EntityRepository` instance into the class constructor where we set our private field like this:

```
namespace OCP.DependencyInjection;
public class EntityService
{
    private readonly EntityRepository _repository;
    public EntityService(EntityRepository repository)
    {
        _repository = repository;
    }
    public async Task ComplexBusinessProcessAsync(Entity entity)
    {
        // Do some complex things here
        await _repository.CreateAsync(entity);
        // Do more complex things here
    }
}
```

With the preceding change, we broke the tight coupling between the `EntityService` and the `EntityRepository` classes. We can also control the behavior of the `EntityService` class from the outside by deciding what instance of the `EntityRepository` class we inject into the `EntityService` constructor. We could even go further by leveraging an abstraction instead of a concrete class and explore this subsequently while covering the DIP.As we just explored, the OCP is a super powerful principle, yet simple, that allows controlling an object from the outside. For example, we could create two instances of the `EntityService` class with different `EntityRepository` instances that connect to different databases. Here's a rough example:

```
using OCP;
using OCP.DependencyInjection;
// Create the entity in database 1
var repository1 = new EntityRepository(/* connection string 1 */);
var service1 = new EntityService(repository1);
// Create the entity in database 2
var repository2 = new EntityRepository(/* connection string 2 */);
var service2 = new EntityService(repository2);
// Save an entity in two different databases
var entity = new Entity();
await service1.ComplexBusinessProcessAsync(entity);
await service2.ComplexBusinessProcessAsync(entity);
```

In the preceding code, assuming we implemented the `EntityRepository` class and configured `repository1` and `repository2` differently, the result of executing the `ComplexBusinessProcessAsync` method on `service1` and `service2` would create the entity in two different databases. The behavior change between the two instances happened without changing the code of the `EntityService` class; composition: 1, inheritance: 0.

> We explore the **Strategy pattern**—the best way of implementing the OCP—in *Chapter 5*, *Strategy, Abstract Factory, and Singleton*. We revisit that pattern and also learn to assemble our program's well-designed pieces and sew them together using dependency injection in *Chapter 6*, *Dependency Injection*.

Next, we explore the principle we can perceive as the most complex of the five, yet the one we will use the less.

Liskov substitution principle (LSP)

The Liskov Substitution Principle (LSP) states that in a program, if we replace an instance of a superclass (supertype) with an instance of a subclass (subtype), the program should not break or behave unexpectedly.Imagine we have a base class called `Bird` with a function called `Fly`, and we add the `Eagle` and

`Penguin` subclasses. Since a penguin can't fly, replacing an instance of the `Bird` class with an instance of the `Penguin` subclass might cause problems because the program expects all birds to be able to fly.So, according to the LSP, our subclasses should behave so the program can still work correctly, even if it doesn't know which subclass it's using, preserving system stability.Before moving on with the LSP, let's look at covariance and contravariance.

Covariance and contravariance

We won't go too deep into this, so we don't move too far away from the LSP, but since the formal definition mentions them, we must understand these at least a minimum.Covariance and contravariance represent specific polymorphic scenarios. They allow reference types to be converted into other types implicitly. They apply to generic type arguments, delegates, and array types. Chances are, you will never need to remember this, as most of it is implicit, yet, here's an overview:

- **Covariance (** `out` **)** enables us to use a more derived type (a subtype) instead of the supertype. Covariance is usually applicable to method return types. For instance, if a base class method returns an instance of a class, the equivalent method of a derived class can return an instance of a subclass.
- **Contravariance (** `in` **)** is the reverse situation. It allows a less derived type (a supertype) to be used instead of the subtype. Contravariance is usually applicable to method argument types. If a method of a base class accepts a parameter of a particular class, the equivalent method of a derived class can accept a parameter of a superclass.

Let's use some code to understand this more, starting with the model we are using:

```
public record class Weapon { }
public record class Sword : Weapon { }
public record class TwoHandedSword : Sword { }
```

Simple class hierarchy, we have a `TwoHandedSword` class that inherits from the `Sword` class and the `Sword` class that inherits from the `Weapon` class.

Covariance

To demo covariance, we leverage the following generic interface:

```
public interface ICovariant<out T>
{
    T Get();
}
```

In C#, the `out` modifier, the highlighted code, explicitly specifies that the generic parameter `T` is covariant. Covariance applies to return types, hence the `Get` method that returns the generic type `T` .Before testing this out, we need an implementation. Here's a barebone one:

```
public class SwordGetter : ICovariant<Sword>
{
    private static readonly Sword _instance = new();
    public Sword Get() => _instance;
}
```

The highlighted code, which represents the `T` parameter, is of type `Sword` , a subclass of `Weapon` . Since covariance means you can **return (output) the instance of a subtype as its supertype**, using the `Sword` subtype allows exploring this with the `Weapon` supertype. Here's the xUnit fact that demonstrates covariance:

```
[Fact]
public void Generic_Covariance_tests()
{
    ICovariant<Sword> swordGetter = new SwordGetter();
    ICovariant<Weapon> weaponGetter = swordGetter;
    Assert.Same(swordGetter, weaponGetter);
    Sword sword = swordGetter.Get();
```

```
    Weapon weapon = weaponGetter.Get();
    var isSwordASword = Assert.IsType<Sword>(sword);
    var isWeaponASword = Assert.IsType<Sword>(weapon);
    Assert.NotNull(isSwordASword);
    Assert.NotNull(isWeaponASword);
}
```

The highlighted line represents covariance, showing that we can implicitly convert the `ICovariant<Sword>` subtype to the `ICovariant<Weapon>` supertype.The code after that showcases what happens with that polymorphic change. For example, the `Get` method of the `weaponGetter` object returns a `Weapon` type, not a `Sword`, even if the underlying instance is a `SwordGetter` object. However, that `Weapon` is, in fact, a `Sword`, as the assertions demonstrate.Next, let's explore contravariance.

Contravariance

To demo covariance, we leverage the following generic interface:

```
public interface IContravariant<in T>
{
    void Set(T value);
}
```

In C#, the `in` modifier, the highlighted code, explicitly specifies that the generic parameter `T` is contravariant. Contravariance applies to input types, hence the `Set` method that takes the generic type `T` as a parameter.Before testing this out, we need an implementation. Here's a barebone one:

```
public class WeaponSetter : IContravariant<Weapon>
{
    private Weapon? _weapon;
    public void Set(Weapon value)
        => _weapon = value;
}
```

The highlighted code, which represents the `T` parameter, is of type `Weapon`, the topmost class in our model; other classes derive from it. Since contravariance means you can **input the instance of a subtype as its supertype**, using the `Weapon` supertype allows exploring this with the `Sword` and `TwoHandedSword` subtypes. Here's the xUnit fact that demonstrates contravariance:

```
[Fact]
public void Generic_Contravariance_tests()
{
    IContravariant<Weapon> weaponSetter = new WeaponSetter();
    IContravariant<Sword> swordSetter = weaponSetter;
    Assert.Same(swordSetter, weaponSetter);
    // Contravariance: Weapon > Sword > TwoHandedSword
    weaponSetter.Set(new Weapon());
    weaponSetter.Set(new Sword());
    weaponSetter.Set(new TwoHandedSword());
    // Contravariance: Sword > TwoHandedSword
    swordSetter.Set(new Sword());
    swordSetter.Set(new TwoHandedSword());
}
```

The highlighted line represents contravariance. We can implicitly convert the `IContravariant<Weapon>` supertype to the `IContravariant<Sword>` subtype.The code after that showcases what happens with that polymorphic change. For example, the `Set` method of the `weaponSetter` object can take a `Weapon`, a **Sword**, or a `TwoHandedSword` instance because they are all subtypes of the `Weapon` type (or is the `Weapon` type itself).The same happens with the `swordSetter` instance, but it only takes a `Sword` or a `TwoHandedSword` instance, starting at the `Sword` type in the inheritance hierarchy because the compiler considers the `swordSetter` instance to be an `IContravariant<Sword>`, even if the underlying implementation is of the `WeaponSetter` type.Writing the following yields a compiler error:

```
swordSetter.Set(new Weapon());
```

The error is:

```
Cannot convert from Variance.Weapon to Variance.Sword.
```

That means that for the compiler, `swordSetter` is of type `IContravariant<Sword>`, not `IContravariant<Weapon>`.

Note

I left a link in the *Further reading* section that explains covariance and contravariance if you want to know more since we just covered the basics here.

Now that we grazed covariance and contravariance, we are ready to explore the formal version of the LSP.

The LSP explained

The LSP came from Barbara Liskov at the end of the ‘80s and was revisited during the ‘90s by both Liskov and Jeannette Wing to create the principle that we know and use today. It is also similar to *Design by contract*, by Bertrand Meyer.Next, let’s look at the formal subtype requirement definition:

Let



be a property provable about objects x of type T. Then,



should be true for objects y of type S, where S is a subtype of T.

In simpler words, if `S` is a subtype of `T`, we can replace objects of type `T` with objects of type `S` without changing any of the expected behaviors of the program (correctness).The LSP adds the following signature requirements:

- The parameters of methods in subtypes must be contravariant.
- The return type of methods in subtypes must be covariant.
- You can't throw a new type of exception in subtypes.

The first two rules are tough to violate without effort in C#.

> Throwing a new type of exception in subtypes is also considered changing behaviors. You can, however, throw subtyped exceptions in subtypes because the existing consumers can handle them.

The LSP also adds the following behavioral conditions:

Conditions	**Examples**
Any precondition implemented in a supertype should yield the same outcome in its subtypes, but subtypes can be less strict about it, never more.	If a supertype validates that an argument cannot be `null`, the subtype could remove that validation but not add stricter validation rules.
Any postcondition implemented in a supertype should yield the same outcome in its subtypes, but subtypes can be more strict about it, never less.	If the supertype never returns `null`, the subtype should not return `null` either or risk breaking the consumers of the object that are not testing for `null`. If the supertype does not guarantee the returned value cannot be `null`, then a subtype could decide never to return `null`, making both instances interchangeable.
Subtypes must preserve the invariance of the supertype.	A subtype must pass all the tests written for the

	supertype, so there is no variance between them (they don’t vary/they react the same).
The history constraint dictates that what happens in the supertype must still occur in the subtype, and you can’t change this.	A subtype can add new properties (state) and methods (behaviors).
	A subtype must not modify the supertype state in any new way.

Table 3.1: LSP behavioral conditions

OK, at this point, you are right to feel that this is rather complex. Yet, rest assured that this is the less important of those principles because we are moving as far as we can from inheritance, so the LSP should not apply often.

We can summarize the LSP to:

In your subtypes, add new behaviors and states; don’t change existing ones.

In a nutshell, applying the LSP allows us to swap an instance of a class for one of its subclasses without breaking anything.To make a LEGO® analogy: LSP is like swapping a 4x2 block with a 4x2 block with a sticker on it: neither the structure's structural integrity nor the block's role changed; the new block only has a new sticker state.

Tip

An excellent way of enforcing those behavioral constraints is automated testing. You can write a test suite and run it against all subclasses of a specific supertype to enforce the preservation of behaviors.

Let’s jump into some code to visualize that in practice.

Project – Liskov Substitution

To demonstrate the LSP, we will explore some scenarios. Each scenario is a test class that follows the same structure:

```
namespace LiskovSubstitution;
public class TestClassName
{
    public static TheoryData<SuperClass> InstancesThatThrowsSuperExceptions = new TheoryData<Supe
    {
        new SuperClass(),
        new SubClassOk(),
        new SubClassBreak(),
    };
    [Theory]
    [MemberData(nameof(InstancesThatThrowsSuperExceptions))]
    public void Test_method_name(SuperClass sut)
    {
        // Scenario
    }
    // Other classes, like SuperClass, SubClassOk,
    // and SubClassBreak
}
```

In the preceding code structure, the highlighted code changes for every test. The setup is simple; I use the test method to simulate code that a program could execute, and just by running the same code three times on different classes, each theory fails once:

- The initial test passes
- The test of a subtype respecting the LSP passes
- The test of a subtype violating the LSP fails.

The parameter `sut` is the subject under test, a well-known acronym.

Of course, we can't explore all scenarios, so I picked three; let's check the first one.

Scenario 1: ExceptionTest

This scenario explores what can happen when a subtype throws a new exception type.The following code is the consumer of the subject under test:

```
try
{
    sut.Do();
}
catch (SuperException ex)
{
    // Some code
}
```

The preceding code is very standard. We wrapped the execution of some code (the `Do` method) in a try-catch block to handle a specific exception.The initial subject under test (the `SuperClass`) simulates that at some point during the execution of the `Do` method, it throws an exception of type `SuperException` . When we execute the code, the try-catch block catches the `SuperException` , and everything goes as planned. Here's the code:

```
public class SuperClass
{
    public virtual void Do()
        => throw new SuperException();
}
public class SuperException : Exception { }
```

Next, the `SubClassOk` class simulates that the execution changed, and it throws a `SubException` that inherits the `SuperException` class. When we execute the code, the try-catch block catches the `SubException` , because it's a subtype of `SuperException` , and everything goes as planned. Here's the code:

```
public class SubClassOk : SuperClass
{
    public override void Do()
        => throw new SubException();
}
public class SubException : SuperException { }
```

Finally, the `SubClassBreak` class simulates that it is throwing `AnotherException` , a new type of exception. When we execute the code, the program stops unexpectedly because we did not design the try-catch block for that. Here's the code:

```
public class SubClassBreak : SuperClass
{
    public override void Do()
        => throw new AnotherException();
}
public class AnotherException : Exception { }
```

So as trivial as it may sound, throwing that exception breaks the program and go against the LSP.

Scenario 2: PreconditionTest

This scenario explores that *any precondition implemented in a supertype should yield the same outcome in its subtypes, but subtypes can be less strict about it, never more*.The following code is the consumer of the subject under test:

```
var value = 5;
var result = sut.IsValid(value);
Console.WriteLine($"Do something with {result}");
```

The preceding code is very standard. We have the `value` variable that could come from anywhere. Then we pass it to the `IsValid` method. Finally, we do something with the `result` ; in this case, we write a line to the console.The initial subject under test (the `SuperClass`) simulates that a precondition exists that enforces that the value must be positive. Here's the code:

```
public class SuperClass
{
    public virtual bool IsValid(int value)
    {
        if (value < 0)
        {
            throw new ArgumentException(
                "Value must be positive.",
                nameof(value)
            );
        }
        return true;
    }
}
```

Next, the `SubClassOk` class simulates that the execution changed and tolerates negative values up to -10. Everything is fine when executing the code because the precondition is less strict. Here's the code:

```
public class SubClassOk : SuperClass
{
    public override bool IsValid(int value)
    {
        if (value < -10)
        {
            throw new ArgumentException(
                "Value must be greater or equal to -10.",
                nameof(value)
            );
        }
        return true;
    }
}
```

Finally, the `SubClassBreak` class simulates that the execution changed and restricts using values under 10. When executing the code, it breaks because we were not expecting that error; the precondition was more strict than the `SuperClass` . Here's the code:

```
public class SubClassBreak : SuperClass
{
    public override bool IsValid(int value)
    {
        if (value < 10) // Break LSP
        {
            throw new ArgumentException(
                "Value must be greater than 10.",
                nameof(value)
            );
        }
        return true;
    }
}
```

Yet another example of how a simple change can break its consumers and the LSP. Of course, this is an overly simplified example focusing only on the precondition, but the same applies to more complex scenarios. Coding is like playing with blocks.

Scenario 3: PostconditionTest

This scenario explores that *postconditions implemented in a supertype should yield the same outcome in its subtypes, but subtypes can be more strict about it, never less*.The following code is the consumer of the subject under test:

```
var value = 5;
var result = sut.Do(value);
Console.WriteLine($"Do something with {result.Value}");
```

The preceding code is very standard and very similar to the second scenario. We have the `value` variable that could come from anywhere. Then we pass it to the `Do` method. Finally, we do something with the `result`; in this case, we write a line to the console. The `Do` method returns an instance of a `Model` class, which has only a `Value` property. Here's the code:

```
public record class Model(int Value);
```

The initial subject under test (the `SuperClass`) simulates that at some point during the execution, it returns a `Model` instance and sets the value of the `Value` property to the value of the `value` parameter. Here's the code:

```
public class SuperClass
{
    public virtual Model Do(int value)
    {
        return new(value);
    }
}
```

Next, the `SubClassOk` class simulates that the execution changed and returns a `SubModel` instance instead. The `SubModel` class inherits the `Model` class and adds a `DoCount` property. When executing the code, everything is fine because the output is invariant (a `SubModel` is a `Model` and behaves the same). Here's the code:

```
public class SubClassOk : SuperClass
{
    private int _doCount = 0;
    public override Model Do(int value)
    {
        var baseModel = base.Do(value);
        return new SubModel(baseModel.Value, ++_doCount);
    }
}
public record class SubModel(int Value, int DoCount) : Model(Value);
```

Finally, the `SubClassBreak` class simulates that the execution changed and returns `null` when the value of the `value` parameter is 5. When executing the code, it breaks at runtime with a `NullReferenceException` when accessing the `Value` property during the interpolation that happens in the `Console.WriteLine` call. Here's the code:

```
public class SubClassBreak : SuperClass
{
    public override Model Do(int value)
    {
        if (value == 5)
        {
            return null;
        }
        return base.Do(value);
    }
}
```

This last scenario shows once again how a simple change can break our program. Of course, this is also an overly simplified example focusing only on the postcondition and history constraint, but the same applies to more complex scenarios.What about the history constraint? We added a new state element to the `SubClassOk` class by creating the `_doCount` field. Moreover, by adding the `SubModel` class, we added the `DoCount` property to the return type. That field and property were nonexistent in the supertype, and they did not alter its behaviors: LSP followed!

Conclusion

The key idea of the LSP is that the consumer of a supertype should remain unaware of whether it's interacting with an instance of a supertype or an instance of a subtype.We could also name this principle the backward-compatibility principle because everything that worked in a way before must still work at least the same after the substitution, which is why this principle is essential.Once again, this is only a principle, not a law. You can also see a violation of the LSP as a *code smell*. From there, analyze whether you have a design problem and its impact. Use your analytical skills on a case-by-case basis and conclude whether or not it would be acceptable to break the LSP in that specific case. Sometimes you want to change the program's behavior and break the LSP, but beware that you might break certain execution paths you did not account for and introduce defects.The more we progress, the more we move away from inheritance, and the less we need to worry about this principle. However, if you use inheritance and want to ensure your subtypes don't break the program: apply the LSP, and you will be rewarded by improving your chances of producing defect-free, backward-compatible changes.Let's look at the ISP next.

Interface segregation principle (ISP)

Let's start with another famous quote by Robert C. Martin:

> *"Many client-specific interfaces are better than one general-purpose interface."*

What does that mean? It means the following:

- You should create interfaces.
- You should value small interfaces more.
- You should not create multipurpose interfaces.

> You can see a multipurpose interface as "an interface to rule them all" or a God class, introduced in *Chapter 1*, *Introduction*.

An interface could refer to a class interface (the public members of a class) or a C# interface. We focus on C# interfaces in the book, as we use them extensively. Moreover, C# interfaces are very powerful.Speaking of interfaces, let's quickly look at them before digging into some code.

What is an interface?

Interfaces are among the most valuable tools in the C# toolbox for creating flexible and maintainable software. It can be tough to understand and grasp the power of interfaces at first, especially from an explanation, so don't worry if you don't; you will see plenty in action throughout the book.

> You can see an interface as allowing a class to impersonate different things (APIs), bringing polymorphism to the next level.

Next are some more details that overview interfaces:

- The role of an interface is to define a cohesive contract (public methods, properties, and events). In its theoretical form, an interface contains no code; it is only a contract. In practice, since C# 8, we can create default implementation in interfaces, which could be helpful to limit breaking changes in a library (such as adding a method to an interface without breaking any class implementing that interface).
- An interface should be small (ISP), and its members should align toward a common goal (cohesion) and share a single responsibility (SRP).
- In C#, a class can implement multiple interfaces, exposing multiples of those public contracts or, more accurately, be any and all of them. By leveraging polymorphism, we can consume a class as if it was any of the interfaces it implements or its supertype if it inherits another class.

> A class does not inherit from an interface; it implements an interface. However, an interface can inherit from another interface.

Let's explore the ISP example now that we refreshed our memory.

Project – Interface Segregation

In this project, we start with the same class as the SRP example but extract an interface from the `ProductRepository` class. Let's start by looking at the `Product` class as a reminder, which represents a simple fictive product:

```
public record class Product(int Id, string Name);
```

> The code sample has no implementation because it is irrelevant to understanding the ISP. We focus on the interfaces instead. Please assume we implemented the data-access logic using your favorite database.

Now, let's look at the interface extracted from the `ProductRepository` class:

```
namespace InterfaceSegregation.Before;
public interface IProductRepository
{
    public ValueTask<IEnumerable<Product>> GetAllPublicProductAsync();
    public ValueTask<IEnumerable<Product>> GetAllPrivateProductAsync();
    public ValueTask<Product> GetOnePublicProductAsync(int productId);
    public ValueTask<Product> GetOnePrivateProductAsync(int productId);
    public ValueTask CreateAsync(Product product);
    public ValueTask UpdateAsync(Product product);
    public ValueTask DeleteAsync(Product product);
}
```

At this point, the `IProductRepository` interface breaks the SRP and the ISP the same way the `ProductRepository` class did before. We already identified the SRP issues earlier but did not reach the point of extracting interfaces.

> The `ProductRepository` class implements the `IProductRepository` interface and is the same as the SRP example (all methods `throw new NotImplementedException()`).

In the SRP example, we identified the following responsibilities:

- Handling public products.
- Handling private products.

Based on our previous analysis, we have two functional requirements (public and private access). By digging deeper, we can also identify five different database operations. Here's the result in a grid:

	Public	**Private**
Read one product	Yes	Yes
Read all products	Yes	Yes
Create a product	No	Yes
Update a product	No	Yes
Delete a product	No	Yes

Table 3.3: a grid that shows what the software needs to do (functional requirements) and what needs to happen in the database (database operation requirements).

We can extract the following families of database operations from Table 3.3:

- Read products (read one, read all).
- Write or alter products (create, update, delete).

Based on that more thorough analysis, we can extract the `IProductReader` and `IProductWriter` interfaces representing the database operation. Then we can create the `PublicProductReader` and `PrivateProductRepository` classes to implement our functional requirements.Let's start with the `IProductReader` interface:

```
namespace InterfaceSegregation.After;
public interface IProductReader
{
    public ValueTask<IEnumerable<Product>> GetAllAsync();
    public ValueTask<Product> GetOneAsync(int productId);
}
```

With this interface, we cover the *read one product* and *read all products* use cases. Next, the `IProductWriter` interface covers the other three database operations:

```
namespace InterfaceSegregation.After;
public interface IProductWriter
{
    public ValueTask CreateAsync(Product product);
    public ValueTask UpdateAsync(Product product);
    public ValueTask DeleteAsync(Product product);
}
```

We can cover all the database use cases with the preceding interfaces. Next, let's create the `PublicProductReader` class:

```
namespace InterfaceSegregation.After;
public class PublicProductReader : IProductReader
{
    public ValueTask<IEnumerable<Product>> GetAllAsync()
        => throw new NotImplementedException();
    public ValueTask<Product> GetOneAsync(int productId)
        => throw new NotImplementedException();
}
```

In the preceding code, the `PublicProductReader` only implements the `IProductReader` interface, covering the identified scenarios. We do the `PrivateProductRepository` class next before exploring the advantages of the ISP:

```
namespace InterfaceSegregation.After;
public class PrivateProductRepository : IProductReader, IProductWriter
{
    public ValueTask<IEnumerable<Product>> GetAllAsync()
        => throw new NotImplementedException();
    public ValueTask<Product> GetOneAsync(int productId)
        => throw new NotImplementedException();
    public ValueTask CreateAsync(Product product)
        => throw new NotImplementedException();
    public ValueTask DeleteAsync(Product product)
        => throw new NotImplementedException();
    public ValueTask UpdateAsync(Product product)
        => throw new NotImplementedException();
}
```

In the preceding code, the `PrivateProductRepository` class implements the `IProductReader` and `IProductWriter` interfaces, covering all the database needs. Now that we have covered the building blocks, let's explore what this can do. Here's the `Program.cs` file:

```
using InterfaceSegregation.After;
var publicProductReader = new PublicProductReader();
var privateProductRepository = new PrivateProductRepository();
ReadProducts(publicProductReader);
ReadProducts(privateProductRepository);
// Error: Cannot convert from PublicProductReader to IProductWriter
// ModifyProducts(publicProductReader); // Invalid
WriteProducts(privateProductRepository);
ReadAndWriteProducts(privateProductRepository, privateProductRepository);
ReadAndWriteProducts(publicProductReader, privateProductRepository);
void ReadProducts(IProductReader productReader)
{
    Console.WriteLine(
        "Reading from {0}.",
        productReader.GetType().Name
```

```
    );
}
void WriteProducts(IProductWriter productWriter)
{
    Console.WriteLine(
        "Writing to {0}.",
        productWriter.GetType().Name
    );
}
void ReadAndWriteProducts(IProductReader productReader, IProductWriter productWriter)
{
    Console.WriteLine(
        "Reading from {0} and writing to {1}.",
        productReader.GetType().Name,
        productWriter.GetType().Name
    );
}
```

From the preceding code, the `ReadProducts`, `ModifyProducts`, and `ReadAndUpdateProducts` methods write messages in the console to demonstrate the advantages of applying the ISP.The `publicProductReader` (instance of `PublicProductReader`) and `privateProductRepository` (instance of `PrivateProductRepository`) variables are passed to the methods to show what we can and cannot do with the current design.Before getting into the weed, when we execute the program, we obtain the following output:

```
Reading from PublicProductReader.
Reading from PrivateProductRepository.
Writing to PrivateProductRepository.
Reading from PrivateProductRepository and writing to PrivateProductRepository.
Reading from PublicProductReader and writing to PrivateProductRepository.
```

First operation

The following code represents the first operation:

```
ReadProducts(publicProductReader);
ReadProducts(privateProductRepository);
```

Since the `PublicProductReader` and `PrivateProductRepository` classes implement the `IProductReader` interface, the `ReadProducts` method accepts them, leading to the following output:

```
Reading from PublicProductReader.
Reading from PrivateProductRepository.
```

That means we can centralize some code that reads from both implementations without changing them.

Second operation

The following code represents the second operation:

```
WriteProducts(privateProductRepository);
```

Since only the `PrivateProductRepository` class implements the `IProductWriter` interface, the `WriteProducts` method accepts only the `privateProductRepository` variable and outputs the following:

```
Writing to PrivateProductRepository.
```

This is one advantage of well-segregated interfaces and responsibilities; if we try to execute the following line, the compiler yields the error saying that we “cannot convert from PublicProductReader to IProductWriter”:

```
ModifyProducts(publicProductReader);
```

That error makes sense because PublicProductReader does not implement the `IProductWriter` interface.

Third operation

The following code represents the third operation:

```
ReadAndWriteProducts(
    privateProductRepository,
    privateProductRepository
);
ReadAndWriteProducts(
    publicProductReader,
    privateProductRepository
);
```

Let's analyze the two calls to the `ReadAndWriteProducts` method individually, but before that, let's look at the console output:

```
Reading from PrivateProductRepository and writing to PrivateProductRepository.
Reading from PublicProductReader and writing to PrivateProductRepository.
```

The first execution reads and writes to the `PrivateProductRepository` instance, which is possible because it implements both the `IProductReader` and `IProductWriter` interfaces.The second call, however, reads from the public reader but writes using the private writer. The last example shows the power of the ISP, especially when mixed with the SRP, and how easy it is to swap one piece for another when segregating our interfaces correctly and designing our code for the program's use cases.

> You should not divide all your repositories into readers and writers; this sample only demonstrates some possibilities. Always design your programs for the specifications that you have.

Conclusion

To summarize the idea behind the ISP, if you have multiple smaller interfaces, it is easier to reuse them and expose only the features you need instead of exposing APIs that part of your program doesn't need. Furthermore, it is easier to compose bigger pieces using multiple specialized interfaces by implementing them as needed than remove methods from a big interface if we don't need them in one of its implementations.

> The main takeaway is to **only depend on the interfaces that you consume**.

If you don't see all of the benefits yet, don't worry. All the pieces should come together as we move on to the last SOLID principle, to dependency injection, the rest of the book, and as you practice applying the SOLID principles.

> Like the SRP, be careful not to overuse the ISP mindlessly. Think about cohesion and what you are trying to achieve, not how granular an interface can become. The finer-grained your interfaces, the more flexible your system will be but remember that flexibility has a cost, which can become very high very quickly. For example, your highly-flexible system may be very hard to navigate and understand, increasing the cognitive load required to work on the project.

Next, we explore the last of the SOLID principles.

Dependency inversion principle (DIP)

The DIP provides flexibility, testability, and modularity, by reducing tight coupling between classes or modules.Let's continue with another quote from Robert C. Martin (including the implied context from Wikipedia):

> *One should "depend upon abstractions, [not] concretions."*

In the previous section, we explored interfaces (abstractions), one of the pivotal elements of our SOLID arsenal, and using interfaces is the best way to approach the DIP.

> Are you wondering why not use abstract classes? While helpful at providing default behaviors over inheritance, they're not fully abstract. If one is, it's better to use an interface instead.

> Interfaces are more flexible and powerful, acting as contracts between parts of a system. They also allow a class to implement multiple interfaces, boosting flexibility. However, don’t discard abstract classes mindlessly. Actually, don’t discard anything mindlessly.

Exposing interfaces can save countless hours of struggling to find complex workaround when writing unit tests. That is even more true when building a framework or library that others use. In that case, please pay even more attention to providing your consumers with interfaces to mock if necessary.All that talk about interfaces again is great, but how can we invert the flow of dependencies? Spoiler alert: interfaces!Let’s compare a direct dependency and an inverted dependency first.

Direct dependency

A direct dependency occurs when a particular piece of code (like a class or a module) relies directly on another. For example, if Class A uses a method from Class B, then Class A directly depends on Class B, which is a typical scenario in traditional programming.Say we have a `SomeService` class that uses the `SqlDataPersistence` class for production but the `LocalDataPersistence` class during development and testing. Without inverting the dependency flow, we end up with the following UML dependency graph:

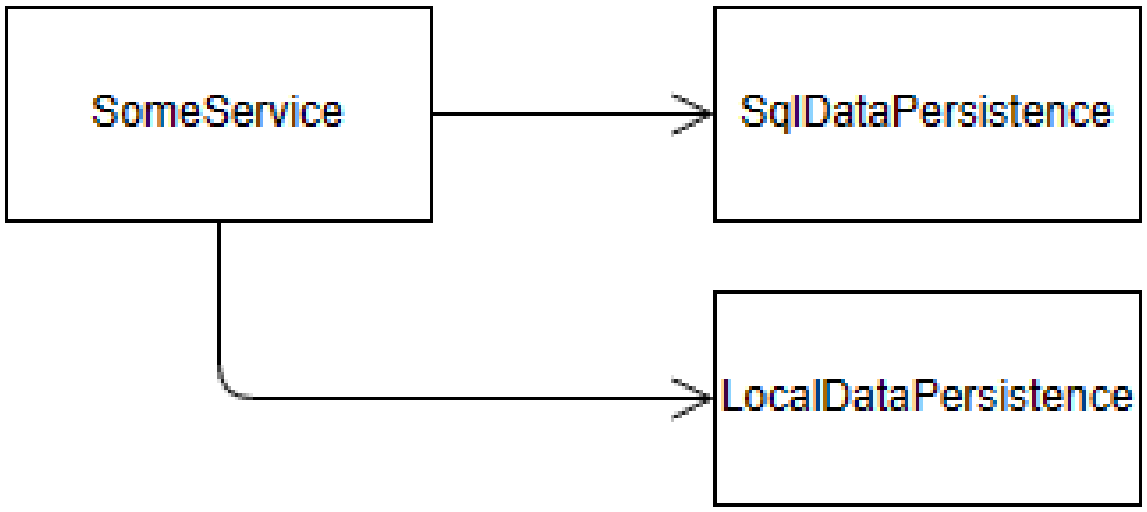


Figure 3.2: Direct dependency graph schema

With the preceding system, we could not change the `SqlDataPersistence` or `LocalDataPersistence` classes by the `CosmosDbDataPersistence` class (not in the diagram) without impacting the `SomeService` class.We call direct dependencies like these **tight coupling**.

Inverted dependency

An inverted dependency occurs when high-level modules (which provide complex logic) are independent of low-level modules (which provide basic, foundational operations). We can achieve this by introducing an abstraction (like an interface) between the modules. This means that instead of Class A depending directly on Class B, Class A would rely on an abstraction that Class B implements.Here is the updated schema that improves the direct dependency example:

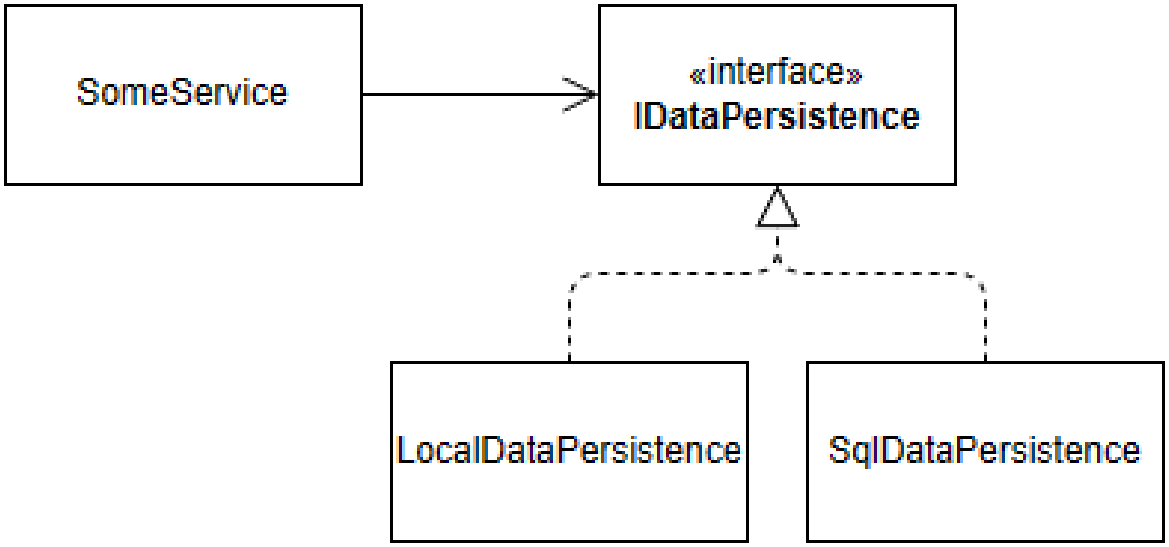


Figure 3.3: Indirect dependency graph schema

In the preceding diagram, we successfully inverted the dependency flow by ensuring the `SomeService` class depends only on an `IDataPersistance` interface (abstraction) that the `SqlDataPersistence` and `LocalDataPersistence` classes implement. We could then use the `CosmosDbDataPersistence` class (not in the diagram) without impacting the `SomeService` class.We call inverted dependencies like these **loose coupling**.Now that we covered how to invert the dependency flow of classes, we look at inverting subsystems.

Direct subsystems dependency

The preceding direct dependency example divided into packages, which have the same issue, would look like the following:

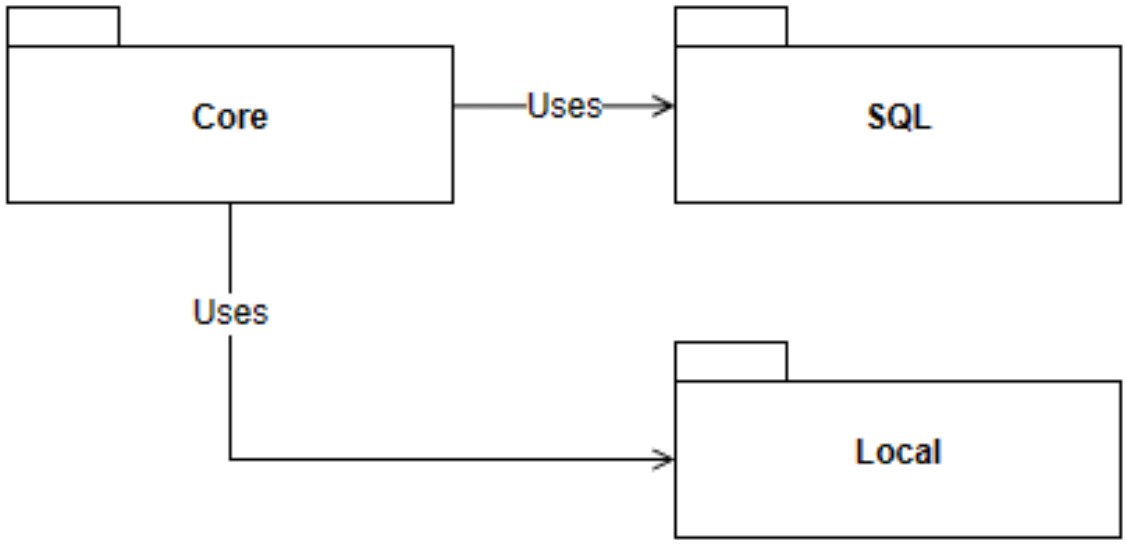


Figure 3.3: direct dependency graph divided into packages

The `Core` package depends on the `SQL` and `Local` packages leading to tight coupling.

Packages usually represent assemblies or namespaces. However, dividing responsibilities around assemblies allows loading only the implementations that the program need. For example, one program could load the `Local` assembly, another could load the `SQL` assembly, and a third could load both.

Enough said; let's invert the dependency flow of those subsystems.

Inverted subsystems dependency

We discussed modules and packages, yet the example diagram of inverted dependency illustrated classes. Using a similar approach, we can reduce dependencies between subsystems and create more flexible programs by arranging our code in separate assemblies. This way, we can achieve loose coupling and improved modularity in our software. To continue the inverted dependency example, we can do the following:

1. Create an abstraction assembly containing only interfaces.
2. Create other assemblies that contain the implementation of the contracts from that first assembly.
3. Create assemblies that consume the code through the abstraction assembly.

There are multiple examples of this in .NET, such as the `Microsoft.Extensions.DependencyInjection.Abstractions` and `Microsoft.Extensions.DependencyInjection` assemblies. We explore this concept further in *Chapter 12, Layering and Clean Architecture*.

Then, if we divide the inverted dependency examples into multiple packages, it would look like the following:

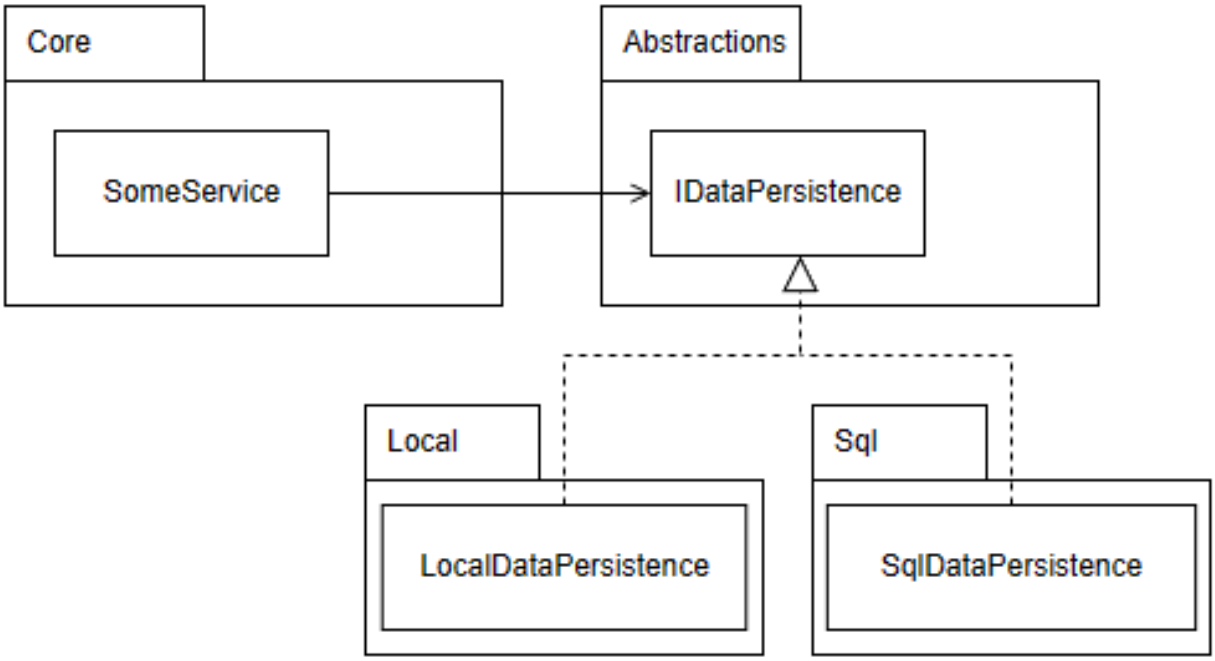


Figure 3.4: inverted dependency examples divided into multiple packages

In the diagram, the `Core` package directly depends on the `Abstractions` package, while two implementations are available: `Local` and `Sql`. Since we only rely on abstractions, we can swap one implementation for the other without impacting `Core`, and the program will run just fine unless something is wrong with the implementation itself (but that has nothing to do with the DIP).We could also create a new `CosmosDb` package and a `CosmosDbDataPersistence` class that implements the `IDataPersistence` interface, then use it in the `Core` without breaking anything. Why? Because we are only directly depending on abstractions, leading to a loosely coupled system.Next, we dig into some code.

Project – Dependency inversion

In this section, we translate the preceding iteration of the inverted dependency example in code. We create the following assemblies to align with the preceding diagram:

- `App` is a console application that references all projects to showcase different use cases.
- `Core` is a class library that depends on the `Abstractions` package.
- `Abstractions` is a class library that contains the `IDataPersistence` interface.
- `Sql` and `Local` are class libraries that reference the `Abstractions` project and implement the `IDataPersistence` interface.

The code sample has few implementation details because it is irrelevant to understanding the DIP. Please assume we implemented the `Persist` methods logic using your favorite in-memory and SQL databases.

Visually, the relationships between the packages look like the following:

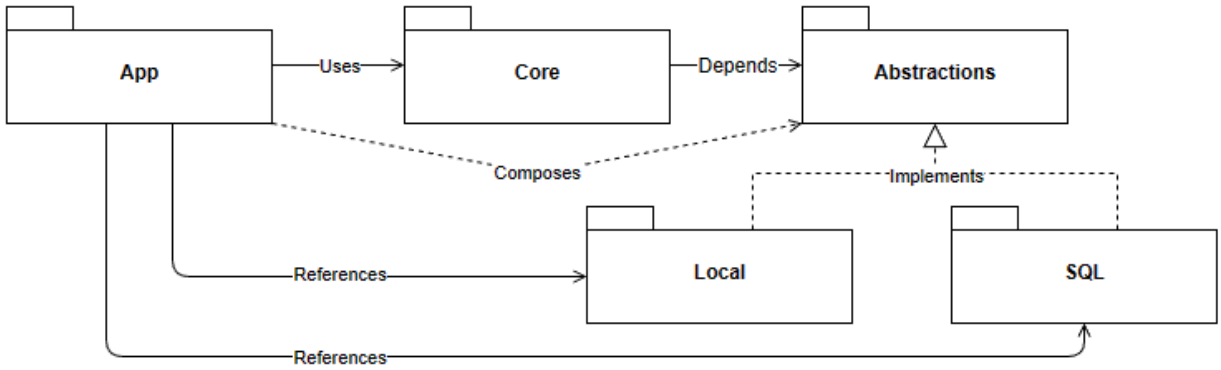


Figure 3.5: the visual representation of the packages and their relationships

Code-wise, our abstraction contains a `Persist` method that we use to showcase the DIP:

```
namespace Abstractions;
public interface IDataPersistence
{
    void Persist();
}
```

Next, the `LocalDataPersistence` class depends on the `Abstractions` package and outputs a line to the console, allowing us to trace what happens in the system:

```
using Abstractions;
namespace Local;
public class LocalDataPersistence : IDataPersistence
{
    public void Persist()
    {
        Console.WriteLine("Data persisted by LocalDataPersistence.");
    }
}
```

Next, the `SqlDataPersistence` class is very similar to the `LocalDataPersistence` class; it depends on the `Abstractions` package and outputs a line in the console, allowing us to trace what happens in the system:

```
using Abstractions;
namespace Sql;
public class SqlDataPersistence : IDataPersistence
{
    public void Persist()
    {
```

```
        Console.WriteLine("Data persisted by SqlDataPersistence.");
    }
}
```

Before we get to the program flow, we still have the `SomeService` class to look at, which depends on the `Abstractions` package:

```
using Abstractions;
namespace App;
public class SomeService
{
    public void Operation(IDataPersistence someDataPersistence)
    {
        Console.WriteLine("Beginning SomeService.Operation.");
        someDataPersistence.Persist();
        Console.WriteLine("SomeService.Operation has ended.");
    }
}
```

The highlighted code shows that the `SomeService` class calls the `Persist` method of the provided `IDataPersistence` interface implementation. The `SomeService` class is not aware of where the data go. In the case of full implementation, the `someDataPersistence` instance is responsible for where the data would be persisted. Other than that, the `Operation` method writes lines to the console so we can trace what happens. Now from the `App` package, the `Program.cs` file contains the following code:

```
using Core;
using Local;
using Sql;
var sqlDataPersistence = new SqlDataPersistence();
var localDataPersistence = new LocalDataPersistence();
var service = new SomeService();
service.Operation(localDataPersistence);
service.Operation(sqlDataPersistence);
```

In the preceding code, we create a `SqlDataPersistence` and a `LocalDataPersistence` instance. Doing that forced us to depend on both packages, but we could have chosen otherwise.Then we create an instance of the `SomeService` class. We then pass both `IDataPersistence` implementations to the `Operation` method one after the other.When we execute the program we get the following output:

```
Beginning SomeService.Operation.
Data persisted by LocalDataPersistence.
SomeService.Operation has ended.
Beginning SomeService.Operation.
Data persisted by SqlDataPersistence.
SomeService.Operation has ended.
```

The first half of the preceding terminal output represents the first call to the `Operation` method, where we passed the `LocalDataPersistence` instance. The second half represents the second call, where we passed the `SqlDataPersistence` instance.The highlighted lines show that depending on an interface allowed us to change this behavior (OCP). Moreover, we could create a `CosmosDb` package, reference it from the `App` package, then pass an instance of a `CosmosDbDataPersistence` class to the `Operation` method, and the `Core` package would not know about it. Why? Because we inverted the dependency flow, creating a loosely coupled system. We even did some *dependency injection*.

> **Dependency injection**, or **Inversion of Control** (**IoC**), is a design principle that is a first-class citizen of ASP.NET Core. It allows us to map abstractions to implementations, and when we need a new type, the whole object tree gets created automatically based on our configuration. We start that journey in *Chapter 7*, *Dependency Injection*.

Conclusion

The core idea is to depend on abstractions. Interfaces are pure contracts, which makes them more flexible than abstract classes. Abstract classes are still helpful, and we explore ways to leverage them in the book.Depending on

implementations (classes) creates tight coupling between classes, which leads to a system that can be harder to maintain. The cohesion between your dependencies is essential in whether the coupling will help or hurt you in the long run. Don’t discard concrete types everywhere mindlessly.

Summary

In this chapter, we covered many architectural principles. We began by exploring DRY, KISS, and separation of concerns principles before learning about the SOLID principles and their importance in modern software engineering. By following those principles, you should be able to build better, more maintainable software.As we also covered, principles are only principles, not laws. You must always be careful not to abuse them so they remain helpful instead of harmful. The context is always essential; internal tools and critical business applications require different levels of tinkering. The key takeaways from this chapter are:

- Don’t over-engineer your solutions (KISS).
- Encapsulate and reuse business logic (DRY).
- Organize elements around concerns and responsibilities (SoC/SRP).
- Aim at composability (OCP).
- Support backward compatibility (LSP).
- Write granular interfaces/contracts (ISP).
- Depend on abstractions and invert the dependency flow (DIP).

With all those principles in our toolbox, we are ready to jump into design patterns and get our design level one step further, with the next chapter covering the MVC pattern in the context of ASP.NET Core REST APIs.Afterward, in the following few chapters, we explore how to implement some of the most frequently used Gang of Four (GoF) patterns and then how to apply them at another level using dependency injection.

Questions

Let’s take a look at a few practice questions:

1. How many principles are represented by the SOLID acronym?
2. Is it true that when following the SOLID principles, the idea is to create bigger components that can each manage more elements of a program by creating God-sized classes?
3. By following the DRY principle, you want to remove all code duplication from everywhere, irrespective of the source, and encapsulate that code into a reusable component. Is this affirmation correct?
4. Is it true that the ISP tells us that creating multiple smaller interfaces is better than creating one large one?
5. What principle tells us that creating multiple smaller classes that handle a single responsibility is better than one class handling multiple responsibilities?

Further reading

- Covariance and contravariance (C#): https://adpg.link/BxBG

4 REST APIs

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter delves into the heart of web application communication–REST APIs. In today's connected digital world, effective communication between different applications is paramount, and RESTful APIs play a pivotal role in facilitating this interaction.We start by exploring the basic fabric of the web: the HTTP protocol. We touch on the core HTTP methods such as GET, POST, PUT, and DELETE to see how they carry out CRUD (Create, Read, Update, Delete) operations in a RESTful context. We then turn our attention to HTTP status codes–the system's way of informing clients about the status of their requests–and HTTP headers.Since APIs evolve and managing these changes without disrupting existing clients is a significant challenge, we look at different strategies for API versioning and the trade-offs involved with each.Then we learn about the Data-Transfer Object (DTO) pattern. Packaging data into DTOs can provide many benefits, from reducing the number of calls to better encapsulation and improved performance when sending data over the network.Finally, we also explore the importance of defining clear and robust API contracts, which ensures API stability. We discuss techniques for designing and documenting these contracts, ensuring

they serve as practical guides for API consumers.By the end of this chapter, you'll know how REST APIs work and will be ready to start building some using ASP.NET Core as we move further into our architectural journey in the next few chapters.In this chapter, we cover the following topics:

- REST & HTTP
- Data Transfer Object (DTO)
- API contracts

Let’s begin with REST.

REST & HTTP

REST, or **Representational State Transfer**, is a way to create internet-based services, known as web services, web APIs, REST APIs, or RESTful APIs. Those services commonly use HTTP as their transport protocol. REST reuses well-known HTTP specifications instead of recreating new ways of exchanging data. For example, returning an HTTP status code `200 OK` indicates success, while `400 Bad Request` indicates failure.Here are some defining characteristics:

- **Statelessness:** In a RESTful system, every client-to-server request should contain all the details necessary for the server to comprehend and execute it. The server retains no information about the client's most recent HTTP request. This enhances both reliability and scalability.
- **Caching capabilities:** Clients should be able to cache responses to enhance performance.
- **Simplicity and lose coupling:** REST uses HTTP to ensure a simplified, decoupled architecture. This makes the development, maintenance, and scaling of REST APIs easier and facilitates their usage.
- **Resource identifiability:** Each REST API endpoint is a distinct resource, enabling us to secure each piece of the system separately.
- **Interface as a contract:** The REST API layer serves as an exchange contract or an abstraction. It effectively conceals the backend system's underlying implementation, fostering streamlined interactions.

While we could delve much deeper into the intricacies of REST APIs, the preceding characteristics serve as foundational knowledge, providing good enough knowledge to get started with RESTful services. Having navigated through these essentials, let's shift our focus toward understanding how REST APIs harness the power of HTTP.

HTTP methods

HTTP methods, also known as verbs, define the type of action a client can perform on a resource in a RESTful API. Each method represents a specific operation that defines the endpoint's intent on a resource. Here is a list of the most frequently used methods, what they are for, and their expected success status code:

Method	Typical role	Success status code
`GET`	Retrieve a resource (read data).	200 OK
`POST`	Create a new resource.	201 CREATED
`PUT`	Replace a resource.	200 OK or 204 No Content
`DELETE`	Delete a resource.	200 OK or 204 No Content
`PATCH`	Partially update a resource.	200 OK

Next, we explore the commonly used status codes.

HTTP Status code

HTTP status codes are part of the HTTP response and provide the client with information about the success or failure of their request; the status of the request.Status codes touching similar subjects are grouped under the same broad “hundredth” categories:

- `1XX` (informational) codes indicate that the request was received and the process is continuing, such as **100 Continue** and **101 Switching Protocols**.
- `2XX` (successful) codes indicate that the request was received successfully.

- `3XX` (redirection) codes indicate that the client must take further action to complete the redirection request.
- `4XX` (client error) codes indicate an error on the client's part, such as validation errors. The client sent an empty required field, for example.
- `5XX` (server error) codes indicate that the server failed to fulfill an apparently valid request and that the client cannot do anything about it (retrying the request is not an option).

The following table explains some of the most common ones:

Status code	**Role**
200 OK	Indicates the request has succeeded. It usually includes data related to the resource in the response body.
201 CREATED	Indicates the has succeeded and the system created a resource. It should also include a `Location` HTTP header pointing to the newly created resource and can include the new entity in the response body.
202 ACCEPTED	Indicates the request has been accepted for processing but is not processed yet. We use this code for asynchronous operations. In an event-driven system (see *Chapter 17*, *Introduction to Microservices Architecture*), this could mean that an event has been published, the current resource has completed its job (published the event), but to know more, the client needs to contact another resource, wait for a notification, just wait, or can’t know.

204 NO CONTENT	Indicates the request has succeeded with no content in the response body.
302 FOUND	Indicates that the requested resource resides temporarily under a different URL specified in the `Location` header. We commonly use this status code for redirection.
400 BAD REQUEST	Indicates that the server could not understand or process the request. This usually relates to a validation error like a bad input or a missing field.
401 UNAUTHORIZED	Indicates that the request requires user authentication to access the resource.
403 FORBIDDEN	Indicates that the server understood the request but refused to authorize it. This usually means the client lacks the access rights for the resource (authorization).
404 NOT FOUND	Indicates the resource does not exist or was not found. REST APIs often return this from valid endpoints.
409 CONFLICT	Indicates that the server cannot complete the request due to a conflict with the current state of the resource. A typical scenario would be that the entity has changed between its read operation (`GET`) and the current update (`PUT`) operation.
500 INTERNAL SERVER ERROR	Indicates that an unhandled error occurred on the server side and prevented it from fulfilling the request.

Now that we covered the HTTP methods and status codes, we look at how to pass more metadata between the client and the server.

HTTP headers

REST APIs leverage HTTP headers to transmit clients' information and describe their options and capabilities. Headers are part of both the request and the response.One well-known header is the `Location` header, that we use for different purposes. For example:

- After creating an entity (`201 Created`), the `Location` header should point to the `GET` endpoint where the client can access that new entity.
- After starting an asynchronous operation (`202 Accepted`), the `Location` header could point to the status endpoint where you can poll for the state of the operation (has it completed, failed, or is it still ongoing).
- When a server wants to instruct a client to load another page (a redirection), the `Location` header contains the destination URL. The following status codes are the most common for redirections: `301 Moved Permanently` , `302 Found` , `303 See Other` , `307 Temporary Redirect` , and `308 Permanent Redirect` .

The `Retry-After` header can also come in handy when mixed with `202 Accepted` , `301 Moved Permanently` , `429 Too Many Requests` , or `503 Service Unavailable` . The `ETag` header identifies the version of the entity and can be used in conjunction with `If-Match` to avoid *mid-air collisions*. The `ETag` and `If-Match` headers form a sort of *optimistic concurrency* method that prevents *request two* from overwriting changes made by *request one* when changes are happening simultaneously or not in the expected order; a.k.a. a way to manage conflicts. We can also add the following to the mix as an example of HTTP headers that describe a REST endpoint: `Allow` , `Authorization` , and `Cache-Control` . The list is very long, and it would help no one to enumerate all HTTP headers here.

This information should be enough theory to get you started with HTTP and REST. In case you want to know more, I left links to the

MDN web docs about HTTP in the Further Reading section at the end of the chapter.

Next, we look at versioning because nothing stays the same forever; business needs change, and APIs must evolve with them.

Versioning

Versioning is a crucial aspect of a REST API. Whether the version of the API is long-lived or transitory (during the decommissioning cycle of an old endpoint, for example), both ends of the pipe must know what to expect; what API contract to respect. Unless you are your only consumer, you'll need a way for the API clients to query specific API versions when the contract changes.This section explores a few ways to think about our versioning strategy.

Default versioning strategy

The default strategy is the first thing to consider when versioning an API. What happens when no version is specified? Will the endpoint return an error, the first or the latest version?If the API returns an error, you should implement that versioning strategy from day one so clients know a version is required. In this case, there is no real drawback. On the other hand, putting this strategy in place after the fact will break all clients that do not specify a version number, which might not be the best way to keep your consumers happy.The second way is always to return the first version. This method is an excellent way to preserve backward compatibility. You can add more endpoint versions without breaking your consumers.The opposite way is always to return the latest version. For consumers, this means specifying a version to consume or be up to date or break, and this might not be the best user experience to provide to your consumers. Nonetheless, many have opted for this default strategy.Another way to go is to pick any version as the default baseline for the API (like version 3.2, for example) or even choose a different version per endpoint. Say you default to 3.2, then deploy 4.0. Since the clients must opt-in to access the new API, they won't break automatically and will have the time to update from 3.2 to 4.0 following their own roadmap. This is a good strategy to default to a well-

known and stable API version before moving forward with breaking changes.

> No matter what you choose, always think it through by weighing the pros and cons.

Next, we explore ways to define those versions.

Versioning strategy

Of course, there are multiple ways to think this through. You can leverage URL patterns to define and include the API version, like `https://localhost/v2/some-entities`. This strategy is easier to query from a browser, making it simple to know the version at a glance, but the endpoint is not pointing to a unique resource anymore (a key principle of REST), as each resource has one endpoint for each version. Nonetheless, this way of versioning an API is used extensively and is one of the most popular, if not *the* most popular way of doing REST versioning, even if it violates one of its core principles (debatably).The other way is to use HTTP headers. You can use a custom header like `api-version` or `Accept-version`, for example, or the standard `Accept` header. This way allows resources to have unique endpoints (URI) while enabling multiple versions of each entity (multiple versions of each API contract describing the same entity).For example, a client could specify an HTTP header while calling the endpoint like this (custom header):

```
GET https://localhost/some-entities
Accept-version: v2
```

Or like the following, by leveraging the `Accept` header for *content negotiation*:

```
GET https://localhost/some-entities
Accept: application/vnd.api.v2+json
```

> Different people are using different values for the `Accept` headers, for example:
>
> - `application/vnd.myapplication.v2+json`

- `application/vnd.myapplication.entity.v2+json`
- `application/vnd.myapplication.api+json; version=2`
- `application/json; version=2`

Whether you are using one way or another, you'll most likely need to version your APIs at some point. Some people are strong advocates of one way or the other, but ultimately, you should decide on a case-by-case basis what best covers your needs and capacities: simplicity, formality, or a mix of both.

Wrapping up

With a method (verb), the client (and the endpoint) can express the intent to create, update, read, or delete an entity. With a status code, the endpoint can tell the client the state of the operation. Adding HTTP headers, clients, and servers can add more metadata to the request or response. Finally, by adding versioning, the REST API can evolve without breaking existing clients while giving them options to consume specific versions.With what we just covered, you should have more than what's needed to follow along with the examples in this book and build a few REST APIs along the way. Next, we explore how those HTTP pieces create API contracts.

Data Transfer Object (DTO)

The Data Transfer Object (DTO) design pattern is a robust approach to managing and transferring data in a service-oriented architecture like REST APIs. The DTO pattern is about organizing the data to deliver it to API clients optimally. DTOs are an integral part of the API contract, that we explore next.

Goal

A DTO's objective is to *control an endpoint's inputs and outputs* by loosely coupling the exposed API surface from the application's inner workings. DTOs empower us to craft our web services the way we want the consumers to interact with them. So, no matter the underlying system, we

can use DTOs to design endpoints that are easier to consume, maintain, and evolve.

Design

Each DTO represents an entity with all the necessary properties. That entity is either an input or an output and allows crafting the interaction between the clients and the API.DTOs serve to loosely couple our domain from the data exposed over the API by adding a level of abstraction. This allows us to change the underlying domain model without affecting the data exposed to the API consumers and vice versa.Another way to use a DTO is by packaging related pieces of information together, allowing a client to make a single call to fetch all necessary data, thereby eliminating the need for multiple requests.Based on REST and HTTP, the flow of a request goes like the following: an HTTP request comes in, some code is executed (domain logic), and an HTTP response goes back to the client. The following diagram represents this flow:

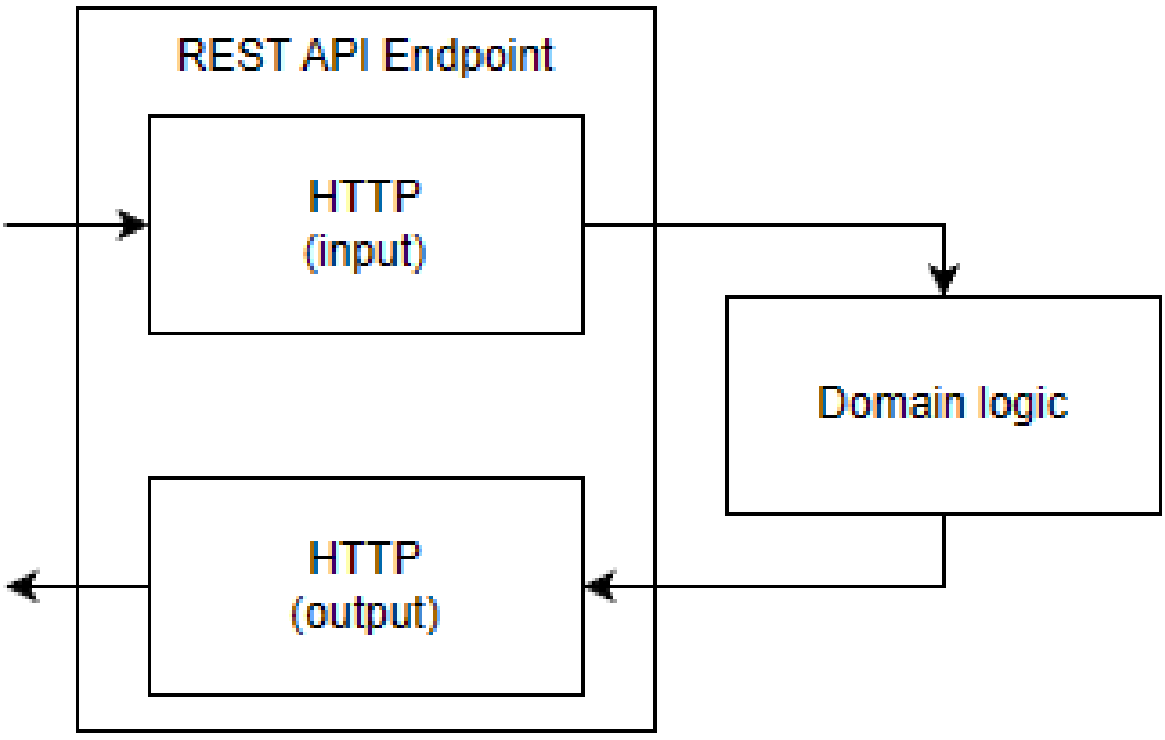


Figure 4.1: An HTTP request getting in and out of a REST API endpoint.

Now, if we take that flow and change HTTP with DTO, we can see that a DTO can be part of the data contract as an input or an output:

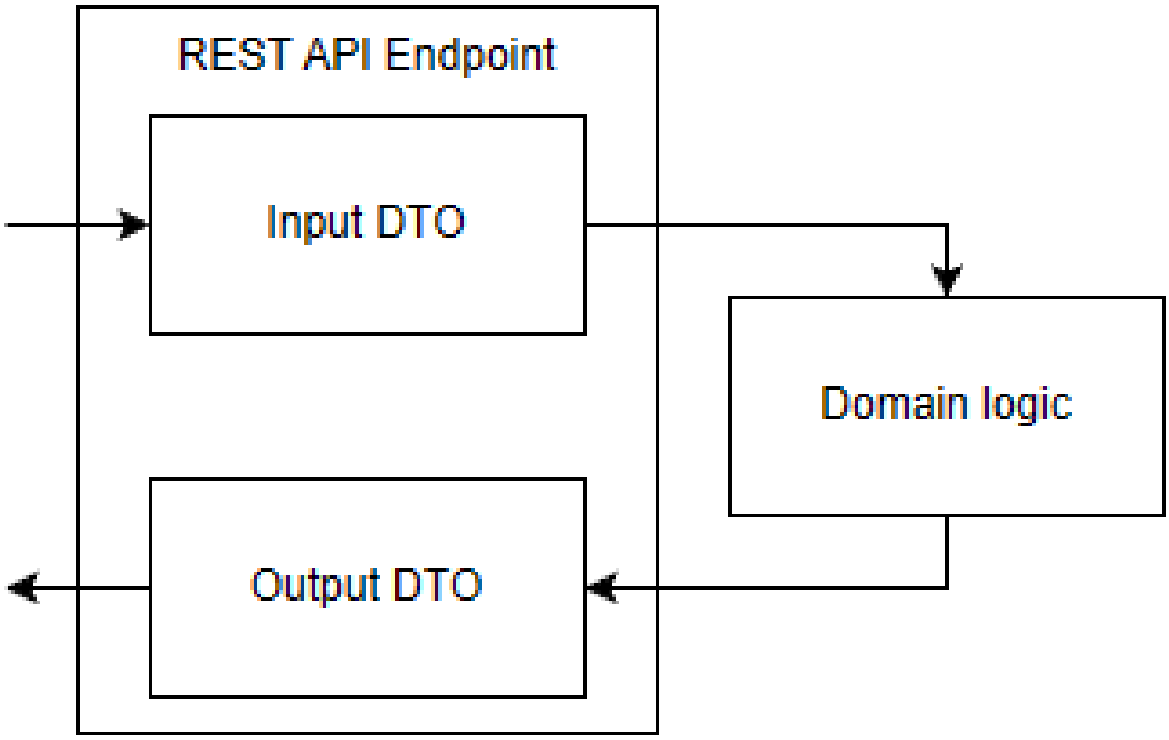


Figure 4.2: An input DTO hitting some domain logic, then the endpoint returning an output DTO

How can the HTTP request become an object? Most of the time:

- We use deserialization or data binding for inputs.
- We use serialization for outputs.

Let’s look at a few examples.

Conceptual examples

Conceptually, say that we are building a web application allowing people to register for events. We explore two use cases next.

Registering for an activity

The first scenario we are exploring is a user registering for an activity. An activity is a sort of event in the system. We use an external payment gateway, so our application never handles financial data. Nevertheless, we must send transaction data to our backend to associate and complete the payment. The following diagram depicts the workflow:

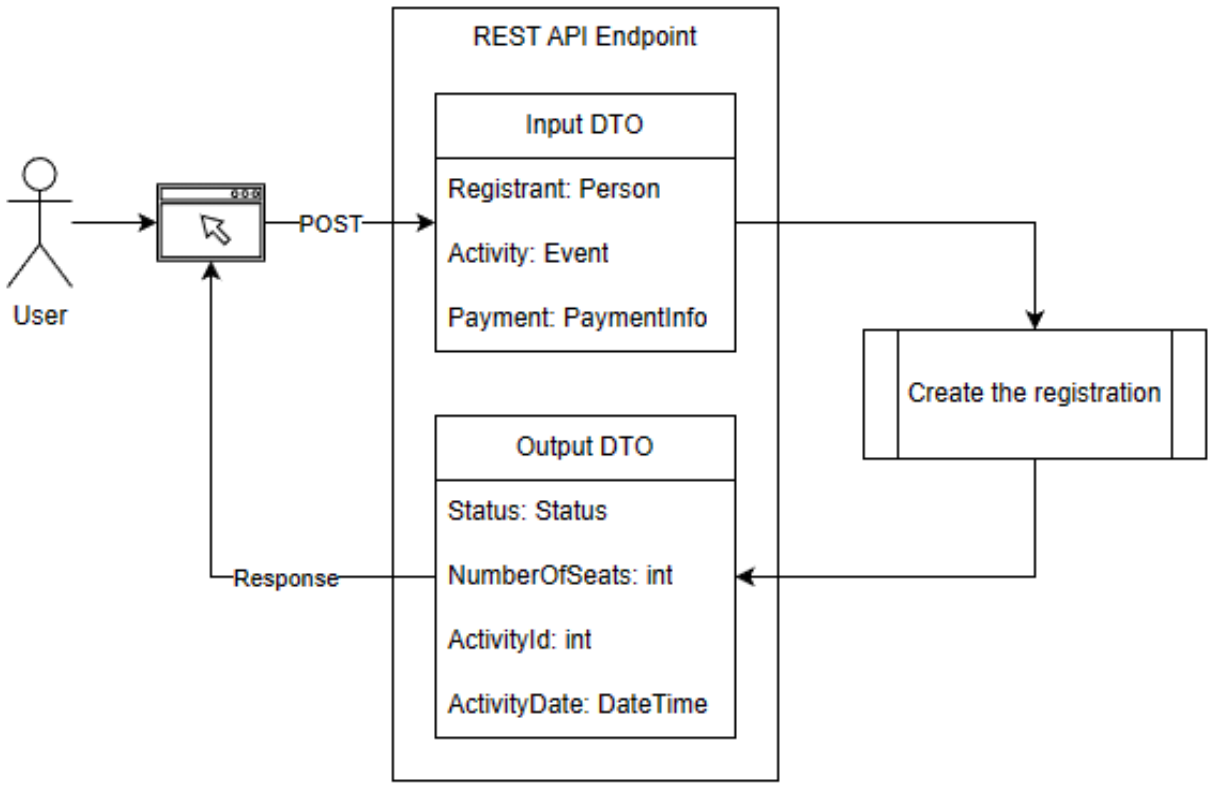


Figure 4.3: The DTOs implicated in an activity registration flow.

The body of the request could look like the following JSON snippet:

```
{
    "registrant": {
        "firstname": "John",
        "lastname": "Doe"
    },
    "activity": {
        "id": 123,
        "seats": 2
    },
    "payment": {
        "nonce": "abc123"
```

```
    }
}
```

Next, the following JSON snippet could represent the body of the response:

```
{
    "status": "Success",
    "numberOfSeats": 2,
    "activityId": 123,
    "activityDate": "2023-06-03T20:00:00"
}
```

Of course, this is a very lightweight version of a registration system. The objective is to show that:

1. Three entities came in as an HTTP POST request (a registrant, an activity, and payment information).
2. The system executed some business logic to register the person to the activity and to complete the financial transaction.
3. The API returned mixed information to the client.

This pattern is handy to input and output only what you need. If you are designing the user interface that consumes the API, outputting a well-thought DTO can ensure that the UI renders the next screen just by reading the response from the server, saving your UI to fetch more data, speeding up the process, and improving the user experience.

We explore fetching information about an activity registration next.

Fetching activity registration details

In the same system, the user wants to review the details of an activity he registered using the preceding process. In this case, the flow goes like this:

1. The client sends the registration identifier over a `GET` request.
2. The system fetches the registrant information, the activity information, and the number of seats the user reserved for that activity.
3. The server returns the data to the client.

The following diagram visually represents the use case:

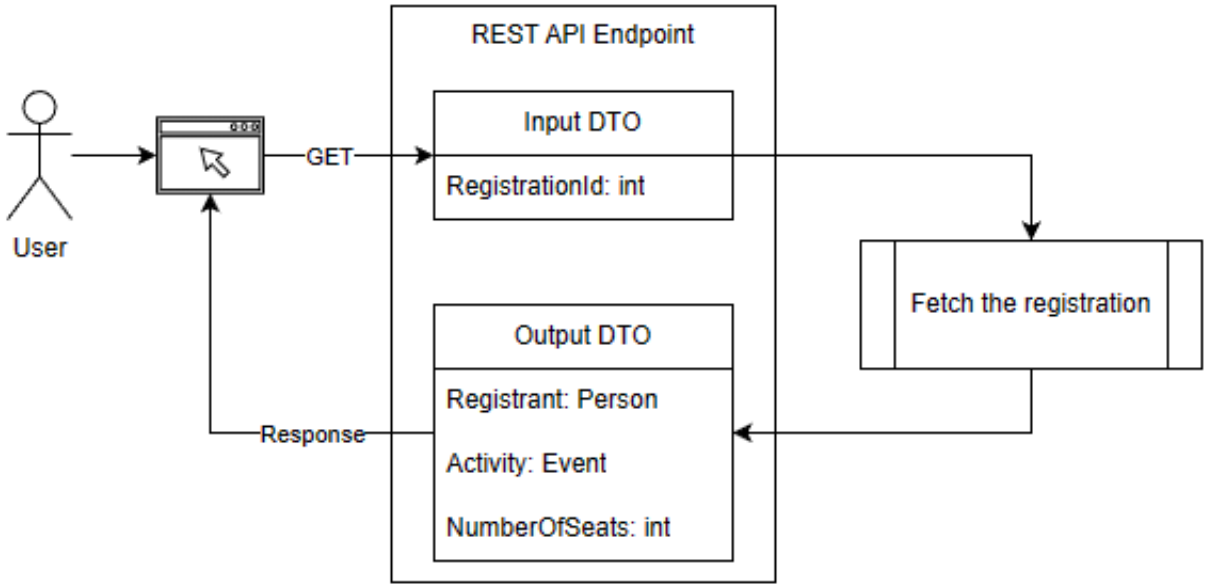


Figure 4.4: The DTOs implicated in fetching the info related to a registered activity.

In this case, the input would be part of the URL, like `/registrations/123`. The output would be part of the response body, and could look like the following:

```
{
    "registrant": {
        "firstname": "John",
        "lastname": "Doe"
    },
    "activity": {
        "id": 123,
        "name": "Super Cool Show",
        "date": "2023-06-03T20:00:00"
    },
    "numberOfSeats": 2
}
```

By creating that endpoint using a well-crafted output DTO, we condensed three HTTP requests into one: the registrant, the activity, and the registration (number of seats). This powerful technique applies to any technology, not just ASP.NET Core, and allows us to design APIs without connecting clients directly to our data (loose coupling).

Conclusion

A data transfer object (DTO) allows us to design an API endpoint with specialized input and output instead of exposing the domain or data model. Those DTOs shield our internal business logic, which improves our ability to design our APIs and also helps us make them more secure.

> By defining DTOs, we can avoid a malicious actor trying to bind data that he should not have access to. For example, when using an input "Login DTO" that only contains a `username` and `password` properties, a malicious user could not try to bind the `IsAdmin` field available in our domain and database. There are other ways to mitigate this, but they are out of the scope of this chapter, yet, a DTO is a great candidate to mitigate this attack vector.

This separation between the presentation and the domain is a crucial element that leads to having multiple independent components instead of a bigger, more fragile one or leaking the internal data structure to the clients consuming the API.We explore building APIs in the next few chapters and explore some topics more in-depth in *Section 4, Designing at Application Scale*.Using the DTO pattern helps us follow the SOLID principles in the following ways:

- **S**: A DTO adds clear boundaries between the domain logic or the data and the API contract, dividing one model into several distinct responsibilities to help keep things isolated.
- **O**: N/A
- **L**: N/A
- **I**: A DTO is a smaller, specifically crafted model that serves a clear purpose. With a DTO, we now have two models (domain and API contract) and several classes (input DTO, output DTO, and domain or data entities) instead of a generic one (only the domain or data entity).
- **D**: N/A

Next, we look at how we can glue the pieces that we explored so far into API contracts.

API contracts

API Contracts serve as an essential blueprint, outlining the rules of engagement between your API and its consumers. This includes available endpoints, HTTP methods they support, expected request formats, and potential response structures, including HTTP status codes.These contracts provide clarity, robustness, consistency, and interoperability, facilitating seamless system interactions, no matter the language they are built with. Moreover, well-documented API contracts are a reliable reference guide, helping developers understand and utilize your API effectively. Thus, designing comprehensive and clear API contracts is critical in building high-quality, maintainable, and user-friendly APIs.An API contract describes a REST API, so a consumer should know how to call an endpoint and what to expect from it in return. What an endpoint does or the capability it provides should be clear just by reading its contract.Each endpoint in a REST API should provide at least the following signature:

- A Uniform Resource Identifier (URI) that indicates where to access it.
- An HTTP method that describes the type of operation it does.
- An input that defines what is needed for the operation to happen. For example, the input can be the HTTP body, URL parameters, query parameters, HTTP headers, or even a combination.
- An output that defines what the client should expect. A client should expect multiple output definitions since an endpoint will not return the same information if the request succeeds or fails.

  The input and output of an endpoint are often DTOs, making DTOs even more important.

There are multiple ways to define API contracts. For example, to define an API contract, we could do the following:

- Open any text editor, such as MS Word or Notepad, and start writing a document describing our web APIs; this is probably the most tedious and least flexible way. I do not recommend this for many reasons.
- Writing specifications in Markdown files and saving those files within your project Git repository for easy discoverability. Very similar to MS

Word, but more accessible for all team members to consume. This approach is better than Word, yet not optimal since you need to manually update those files when the API changes.
- Use an existing standard, such as the OpenAPI specification (formerly Swagger). This technique implies a learning curve, but the result should be easier to consume. Moreover, many tools allow us to create automation using the OpenAPI specs. This approach is starting to remove the need for manual intervention.
- Use a code-first approach and ASP.NET Core tooling to extract the OpenAPI specs from your code.
- Use any other tools that fit our requirements.

Tip

Postman is a fantastic tool for building web APIs documentation, test suites, and experimenting with your APIs. It supports OpenAPI specifications, allows you to create mock servers, supports environments, and more.

No matter the tools, there are two major trends in how to design the API contract of a REST API:

- Design the contract first, then build the API (contract-first).
- Build the API, then extract the contract for the code (code-first).

To design the contract-first, one must adopt a tool to write the specifications, then code the API according to the specs.

I left a link in the *Further Reading* section below about Open API.

On the other hand, to use a code-first approach and automatically extract the OpenAPI specifications from the API, we must ensure our endpoints are discoverable by the .NET `ApiExplorer` .No matter how you do it, in ASP.NET Core, we use classes and struct to represent the data contract of our REST APIs; whether it happens before or after you write the API contract does not matter. Since I prefer to write C# to YAML or JSON, we explore how to leverage Swagger to generate a data contract in a code-first manner next.

Code-first API Contract

In this example, we have a tiny API with two endpoints:

- Read the specified entity.
- Create a new entity.

The code doesn't do much and returns fake data, but it is enough to explore its data contract.

> Remember that code is like playing LEGO® blocks, but we connect many tiny patterns used together to create our software and create value. Understanding and learning that skill will lead you beyond just being able to use some canned magic recipe, which limits you to what people share with you.

In this sample, we use the OpenAPI specification to describe our API. To save ourselves from writing JSON and go code-first instead, we leverage the SwagerGen package.

> To use SwaggerGen, we must install the `Swashbuckle.AspNetCore.SwaggerGen` NuGet package.

Here's the `Program.cs` file, without the endpoints, showing how to leverage SwaggerGen:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
var app = builder.Build();
app.UseSwagger();
// Omitted endpoints
app.Run();
```

The highlighted lines are the only things we must do to use SwaggerGen in a project, which will generate the API contract in the OpenAPI specification for us. The JSON file is very long (113 lines), so I only pasted some snippets in the book for clarity. However, you can navigate to the

`/swagger/v1/swagger.json` URL to access the complete JSON code or open the `swagger.json` file in the project.

> I created the `swagger.json` file in the project for convenience. The tool does not generate a physical file.

Let's have a look at those endpoints.

The first endpoint

The code of the first endpoint that allows a client to read an entity looks like this:

```
app.MapGet(
    "/{id:int}",
    (int id) => new ReadOneDto(
        id,
        "John Doe"
    )
);
public record class ReadOneDto(int Id, string Name);
```

Here's the API contract we can extract from the preceding code:

Contract segment	Value
HTTP Method	GET
URI	`/{id}` (for example, `/123`)
Input	The `id` parameter
Output	An instance of the `ReadOneDto` class.

Sending the following HTTP request (you can use the `ReadOneEntity.http` file) results in the output that follows:

```
GET /123 HTTP/1.1
Accept: application/json
Host: localhost:7000
Connection: keep-alive
```

The trimmed-down response is:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Sat, 03 Jun 2023 17:41:42 GMT
Server: Kestrel
Alt-Svc: h3=":7000"; ma=86400
Transfer-Encoding: chunked

{"id":123,"name":"John Doe"}
```

As we can see, when we query the API for the entity `id=123`, the endpoint returns that entity with a `200 OK` status code, and the response body is a serialized instance of the `ReadOneDto` class.

> The `.http` files are new to VS 2022 and allow us to write and execute HTTP requests from VS itself. I left a link in the *Further Reading* section if you want to know more.

SwaggerGen generated the following OpenAPI specs for the first endpoint:

```
"/{id}": {
  "get": {
    "parameters": [
      {
        "name": "id",
        "in": "path",
        "required": true,
        "schema": {
          "type": "integer",
          "format": "int32"
        }
      }
    ],
    "responses": {
      "200": {
        "description": "Success",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/ReadOneDto"
            }
          }
        }
      }
    }
```

```
  }
},
```

That snippet describes the endpoint and references our output model (highlighted line). The schemas are at the bottom of the JSON file. Here's the schema that represents the `ReadOneDto`:

```
"ReadOneDto": {
  "type": "object",
  "properties": {
    "id": {
      "type": "integer",
      "format": "int32"
    },
    "name": {
      "type": "string",
      "nullable": true
    }
  },
  "additionalProperties": false
}
```

As we can see from the highlighted lines, that schema has a property `name` of type `string` and a property `id` of type `integer`, the same as our `ReadOneDto` class. Fortunately, we don't need to write that JSON since the tool generates it based on our code. Next, we look at the second endpoint.

The second endpoint

The code of the second endpoint that allows a client to create an entity looks like this:

```
app.MapPost(
    "/",
    (CreateDto input) => new CreatedDto(
        Random.Shared.Next(int.MaxValue),
        input.Name
    )
);
public record class CreateDto(string Name);
public record class CreatedDto(int Id, string Name);
```

Here's the API contract we can extract from the preceding code:

Contract segment	Value
HTTP Method	POST
URI	`/`
Input	An instance of the `CreateDto` class.
Output	An instance of the `CreatedDto` class.

Sending the following HTTP request (you can use the `CreateEntity.http` file) results in the output that follows:

```
POST / HTTP/1.1
Content-Type: application/json
Host: localhost:7000
Accept: application/json
Connection: keep-alive
Content-Length: 28

{
    "name": "Jane Doe"
}
```

The trimmed-down response is:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Sat, 03 Jun 2023 17:59:25 GMT
Server: Kestrel
Alt-Svc: h3=":7000"; ma=86400
Transfer-Encoding: chunked

{"id":1624444431,"name":"Jane Doe"}
```

As we can see from the preceding request, the client sent a serialized instance of the `CreateDto` class, set the name to Jane Doe, and received that same entity back but with a numeric `id` property (an instance of the `CreatedDto` class).The OpenAPI specs of our endpoint look like the following:

```
"/": {
  "post": {
    "requestBody": {
      "content": {
        "application/json": {
```

```
          "schema": {
            "$ref": "#/components/schemas/CreateDto"
          }
        }
      },
      "required": true
    },
    "responses": {
      "200": {
        "description": "Success",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/CreatedDto"
            }
          }
        }
      }
    }
  }
}
```

The input and output schemas are:

```
"CreateDto": {
  "type": "object",
  "properties": {
    "name": {
      "type": "string",
      "nullable": true
    }
  },
  "additionalProperties": false
},
"CreatedDto": {
  "type": "object",
  "properties": {
    "id": {
      "type": "integer",
      "format": "int32"
    },
    "name": {
      "type": "string",
      "nullable": true
    }
  },
```

```
  "additionalProperties": false
},
```

Similar to the first endpoint, SwaggerGen translates our C# classes into OpenAPI specs. Let’s wrap this up.

Wrapping up

Some ASP.NET Core templates come with SwaggerGen preconfigured. It also comes with the Swagger UI that lets you visually explore the API contract from your application and even query it. NSwag is another tool that offers similar features. Plenty of online documentation shows how to take advantage of those tools.Besides exploring tooling, we defined that an API contract is fundamental and promotes robustness and reliability. Each endpoint has the following pieces as part of the overall API contract:

- The URI it is accessible from.
- The HTTP method that best defines the operation.
- An input.
- One or more outputs.

A single URI can lead to multiple endpoints by combining different HTTP methods and inputs. For example, `GET /api/entities` may return a list of entities, while `POST /api/entities` may create a new entity. Using the entity's name in its plural form is a convention used by many.

We explore data transfer objects next to add more clarity to that pattern.

Summary

REST APIs facilitate communication between applications in today's interconnected digital world. We explored the HTTP protocol, HTTP methods, HTTP status codes, and HTTP headers. We then explored API versioning, the Data Transfer Objects (DTOs), and the importance of API contracts. Here are a few Key Takeaways:

- **REST & HTTP**: REST APIs are integral to web application communication. They use HTTP as their transport protocol, leveraging its methods, status codes, and headers to facilitate interaction between different applications.
- **HTTP Methods**: HTTP methods or verbs (GET, POST, PUT, DELETE, PATCH) define the type of action a client can perform on a resource in a RESTful API. Understanding these methods is crucial for carrying out CRUD operations.
- **HTTP Status Codes and Headers**: HTTP status codes inform clients about the success or failure of their requests. HTTP headers transmit additional information and describe clients' options and capabilities. Both are essential components of HTTP communication.
- **Versioning**: Managing changes in APIs without disrupting existing clients is a significant challenge. Different strategies for API versioning can help address this issue, but each comes with its own trade-offs.
- **Data Transfer Object (DTO)**: DTOs package data into a format that can provide many benefits, including reducing the number of HTTP calls, improving encapsulation, and enhancing performance when sending data over the network.
- **API Contracts**: Clear and robust API contracts ensure API stability. They serve as a blueprint for interaction between an API and its consumers, outlining available endpoints, supported HTTP methods, expected request formats, and potential response structures.
- **Practical Application**: Understanding these concepts is not only theoretically important but also practically helpful in building and working with REST APIs using ASP.NET Core or any other similar technology.

By now, you should have a solid understanding of REST APIs and be ready to explore how to implement one using ASP.NET Core. ASP.NET Core makes writing REST APIs using MVC or minimal APIs a breeze. MVC is a well-used pattern that is almost impossible to avoid. However, the new minimal API model makes the process leaner. Moreover, with application patterns like Request-EndPoint-Response (REPR) or Vertical Slice Architecture, we can organize our API per feature instead of by layer, leading to an improved organization. We cover those topics in *Section 4*:

Application patterns.Next, we explore designing with ASP.NET Core, starting with Minimal APIs.

Questions

Let's look at a few practice questions:

1. What is the most common status code sent in a REST API after creating an entity?
2. If you introduce a default strategy that returns the lowest possible version when no version is specified, would it break existing clients?
3. If you want to read data from the server, what HTTP method would you use?
4. Can DTOs add flexibility and robustness to a system?
5. Are DTOs part of an API contract?

Further reading

Here are some links to build on what we have learned in the chapter:

- HTTP request methods (MDN): https://adpg.link/MFWb
- HTTP response status codes (MDN): https://adpg.link/34Jq
- HTTP headers (MDN): https://adpg.link/Hx55
- Use .http files in Visual Studio 2022: https://adpg.link/cbhv
- OpenAPI specification: https://adpg.link/M4Uz

Answers

1. An API usually returns the status code `201 Created` after creating a new entity.
2. No, it will not break clients because they will either be using the lowest API version or have already specified a specific version.
3. We typically use the HTTP GET method to read data from a REST API.
4. Yes, Data Transfer Objects (DTOs) can add flexibility and robustness to a system. They allow you to control exactly what data you expose to

the client and can reduce the amount of unnecessary data that needs to be sent over the network.

5. Yes, DTOs are part of an API contract. They define the data format exchanged between the client and server, ensuring both sides understand the data being sent and received.

5 Minimal API

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter covers Minimal APIs, a simplified way of setting up and running .NET applications. We explore what makes Minimal Hosting and Minimal APIs a pivotal update in ASP.NET Core as we unravel the simplicity of creating APIs with less ceremony. We cover many possibilities that ASP.NET Core Minimal API brings, like how to configure, customize, and organize those endpoints.We also explore using Minimal APIs with Data Transfer Objects (DTOs), combining simplicity with effective data management to structure API contracts effectively.Inspired by other technologies, these topics bring a fresh perspective to the .NET world, allowing us to build lean and performant APIs without compromising resiliency.In this chapter, we cover the following topics:

- Top-level statements
- Minimal Hosting
- Minimal APIs
- Using Minimal APIs with Data Transfer Objects

Let’s begin with Top-level statements.

Top-level statements

The .NET team introduced top-level statements to the language in .NET 5 and C# 9. From that point, writing statements before declaring namespaces and other members is possible. Under the hood, the compiler emits those statements into a `Program.Main` method.With top-level statements, a minimal .NET “Hello World” console program looked like this (`Program.cs`):

```
using System;
Console.WriteLine("Hello world!");
```

Unfortunately, we still need a project to run it, so we have to create a `.csproj` file with the following content:

```
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net5.0</TargetFramework>
        <OutputType>Exe</OutputType>
    </PropertyGroup>
</Project>
```

From there, we can use the .NET CLI to `dotnet run` the application, and it will output the following in the console before the program terminates:

```
Hello world!
```

On top of such statements, we can also declare other members, like classes, and use them in our application. However, we must declare classes at the end of the top-level code.

> Be aware that the top-level statement code is not part of any namespace, and creating classes in a namespace is recommended, so you should limit the number of declarations done in the `Program.cs` file to what is internal to its inner workings, if anything.

Top-level statements are great for getting started with C#, writing code samples, and cutting out boilerplate code.

> The highlighted line of the preceding C# code (`using System;`) is unnecessary when the *implicit usings* feature is enabled, which is the default in .NET 6+ projects. The templates add the following line to the `.csproj` file:

```
<ImplicitUsings>enable</ImplicitUsings>
```

Next, we explore the minimal hosting model built using top-level statements.

Minimal Hosting

.NET 6 introduced the minimal hosting model. It combines the `Startup` and `Program` classes into a single `Program.cs` file. It leverages top-level statements to minimize the boilerplate code necessary to bootstrap the application. It also uses *global using directives* and the *implicit usings* feature to reduce the amount of boilerplate code further. This model only requires one file with the following three lines of code to create a web application:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
app.Run();
```

Let’s call that way leaner than before. Of course, the preceding code starts an app that does nothing, but doing the same before would have required tens of lines of code.The minimal hosting code is divided into two pieces:

- The *web application builder* we use to configure the application, register services, settings, environment, logging, and more (the highlighted code).
- The *web application* we use to configure the HTTP pipeline and routes (the non-highlighted lines).

That simplified model led to minimal APIs that we explore next.

Minimal APIs

ASP.NET Core’s Minimal APIs are built on the minimal hosting model and bring a lean approach to constructing web applications. Highly inspired by Node.js, they facilitate the development of APIs by reducing the boilerplate code. By emphasizing simplicity and performance, they enhance readability and maintainability. They are an excellent fit for microservices architecture and applications that aim to remain lean.

> You can also build large applications using Minimal APIs; the word minimal refers to their lean approach, not the type of application you can make with them.

This minimalist approach does compromise a little on functionalities but improves flexibility and speed, ensuring you have complete control over your API’s behavior while keeping your project lean and efficient. Minimal APIs include the necessary features we need for most applications, like model binding, dependency injection, filters, and a route-to-delegate model. If you need all the features from MVC, you can still opt to use MVC. You can even use both; this is not one or the other.

> We explore the Model-View-Controller (MVC) pattern in *Chapter 6*, *MVC*.

Let’s have a look at how to map routes next.

Map route-to-delegate

How does it work? Minimal APIs bring multiple extension methods to configure the HTTP pipeline and configure endpoints. We can use those methods to map a route (a URL pattern) to a `RequestDelegate` delegate.We can use the following methods to map different HTTP methods:

Method	Description
`MapGet`	Maps a `GET` request to a `RequestDelegate` .
`MapPost`	Maps a `POST` request to a `RequestDelegate` .
`MapPut`	Maps a `PUT` request to a `RequestDelegate` .
`MapDelete`	Maps a `DELETE` request to a `RequestDelegate` .
`MapMethods`	Maps a route pattern and multiple HTTP methods to a `RequestDelegate` .
`Map`	Maps a route pattern to a `RequestDelegate` .
`MapFallback`	Maps a fallback `RequestDelegate` which runs when no other routes match.
`MapGroup`	Allows configuring a route pattern and properties that apply to all endpoints defined under that group.

Table 5.1: Map route-to-delegate extension methods.

Here's a minimal GET example:

```
app.MapGet("minimal-endpoint-inline", () => "GET!");
```

When executing the program, navigating to the `/minimal-endpoint-inline` URI routes the request to the registered `RequestDelegate` (highlighted code), which outputs the following string:

```
GET!
```

As simple as that, we can route requests to delegates and create endpoints.

> On top of registering endpoints, we can also register middleware like any other ASP.NET Core application. Moreover, the built-in middlewares, like authentication and CORS, work the same with Minimal APIs.

Next, we explore ways to configure endpoints so we can create better APIs than an endpoint returning a literal string.

Configuring endpoints

Now that we know that with minimal APIs, we map routes to delegates and that we have learned of a few methods to do that, let's explore how to register the delegates:

- Inline, as with the preceding example.
- Using a method.

To declare the delegate inline, we can do the following:

```
app.MapGet("minimal-endpoint-inline", () => "GET!");
```

To use a method, we can do the following:

```
app.MapGet("minimal-endpoint-method", MyMethod);
void MyMethod() { }
```

> When enabled, ASP.NET Core registers the class name that contains the method with the `ApiExplorer` as a tag. We dig deeper into metadata further in the chapter.

All the concepts we explore in this chapter apply to both ways of registering delegates. Let's start by studying how to input data in our endpoints.

Inputs

An endpoint rarely has no parameter (no input value). Minimal APIs, like MVC, support a wide variety of binding sources. A binding source represents the conversion from the HTTP request into a strongly typed C# object, inputted as a parameter. Most of the parameter binding happens implicitly, but in case you need to bind a parameter explicitly, here are the supported binding sources:

Source	Attribute	Description
`Route`	`[FromRoute]`	Binds the route value that matches the name of the parameter.
`Query`	`[FromQuery]`	Binds the query string value that matches the name of the parameter.
`Header`	`[FromHeader]`	Binds the HTTP header value that matches the name of the parameter.
`Body`	`[FromBody]`	Binds the JSON body of the request to the parameter's type.
`Form`	`[FromForm]`	Binds the form value that matches the name of the parameter.
`Services`	`[FromServices]`	Inject the service from the ASP.NET Core dependency container.
`Custom`	`[AsParameters]`	Binds the form values to a type. The matches happen between the form keys and the properties' names.

Table 5.2: supported binding sources

Next is a demo where we implicitly bind the `id` parameter from the route (highlighted code) to a parameter in the delegate:

```
app.MapGet(
    "minimal-endpoint-input-route-implicit/{id}",
    (int id) => $"The id was {id}."
);
```

In most cases, the bindings work implicitly. However, you can explicitly bind the delegate's parameters like this:

```
app.MapGet(
    "minimal-endpoint-input-route-explicit/{id}",
    ([FromRoute] int id) => $"The id was {id}."
);
```

We can also implicitly inject dependencies into our delegates and even mix that with a route parameter like this:

```
app.MapGet(
    "minimal-endpoint-input-service/{value}",
    (string value, SomeInternalService service)
        => service.Respond(value)
);
public class SomeInternalService {
    public string Respond(string value)
        => $"The value was {value}";
}
```

Following this pattern opens endless possibilities to input data into our endpoints.

> If you are unfamiliar with Dependency Injection (DI), we explore DI more in-depth in *Chapter 8*, *Dependency Injection*. Meanwhile, remember that we can bind objects to parameters, whether they are a DTO or a service.

On top of that, ASP.NET Core provides us with a few special types, which we explore next.

Special types

We can inject the following objects into our delegates as parameters, and ASP.NET Core manages them for us:

Class	Description
`HttpContext`	The `HttpContext` encompasses all the current HTTP request and response details.

	The `HttpContext` exposes all the other special types we are exploring here, so if you need more than one, you can inject the `HttpContext` directly to reduce the number of parameters.
`HttpRequest`	We can use the `HttpRequest` to do basic HTTP operations on the current request, like query the parameters manually and bypass the ASP.NET Core data-binding mechanism. Same as the `HttpContext.Request` property.
`HttpResponse`	Like the `HttpRequest` , we can leverage the `HttpResponse` object to execute manual operations on the HTTP response, like writing directly to the response stream, managing HTTP headers manually, etc. Same as the `HttpContext.Response` property.
`CancellationToken`	Passing a cancellation token to an asynchronous operation is a recommended practice. In this case, it allows canceling the operation when the request is canceled. Same as the `HttpContext.RequestAborted` property.
`ClaimsPrincipal`	To access the current user, we can inject a `ClaimsPrincipal` instance. Same as the `HttpContext.User` property.

Tabel 5.3: special HTTP types

Here's an example where two endpoints write to the response stream, one using the `HttpContext` and the other the `HttpResponse` object:

```
app.MapGet(
    "minimal-endpoint-input-HttpContext/",
    (HttpContext context)
        => context.Response.WriteAsync("HttpContext!")
);
app.MapGet(
    "minimal-endpoint-input-HttpResponse/",
    (HttpResponse response)
        => response.WriteAsync("HttpResponse!")
);
```

We can treat those special types like any other bindings and seamlessly integrate them with other types, such as route values and services.We cover one last piece of data-binding next.

Custom binding

We can manually bind data from the request to an instance of a custom class. We can achieve this in the following ways:

- Create a static `TryParse` method that parses a string from a route, query, or header value.
- Create a static `BindAsync` method that directly controls the binding process using the `HttpContext` .

We must write those static methods in the class we intend to create using the HTTP request's data. We explore those two scenarios next.

Manual parsing

The `TryParse` method takes a string and an `out` parameter of the type itself. The framework uses that method to parse a value into the desired type.The parsing API supports the implementation of one of the following methods:

```
public static bool TryParse(string value, TSelf out result);
public static bool TryParse(string value, IFormatProvider provider, TSelf out result);
```

Implementing the `IParsable<TSelf>` interface provides the appropriate `TryParse` method.

Here is an example that parses latitude and longitude coordinates:

```
app.MapGet(
    "minimal-endpoint-input-Coordinate/",
    (Coordinate coordinate) => coordinate
);
public class Coordinate : IParsable<Coordinate>
{
    public double Latitude { get; set; }
    public double Longitude { get; set; }
    public static Coordinate Parse(
        string value,
        IFormatProvider? provider)
    {
        if (TryParse(value, provider, out var result))
        {
            return result;
        }
        throw new ArgumentException(
            "Cannot parse the value into a Coordinate.",
            nameof(value)
        );
    }
    public static bool TryParse(
        [NotNullWhen(true)] string? s,
        IFormatProvider? provider,
        [MaybeNullWhen(false)] out Coordinate result)
    {
        var segments = s?.Split(
            ',',
            StringSplitOptions.TrimEntries |
            StringSplitOptions.RemoveEmptyEntries
        );
        if (segments?.Length == 2)
        {
            var latitudeIsValid = double.TryParse(
                segments[0],
                out var latitude
            );
            var longitudeIsValid = double.TryParse(
                segments[1],
                out var longitude
            );
            if (latitudeIsValid && longitudeIsValid)
            {
                result = new() {
                    Latitude = latitude,
                    Longitude = longitude
                };
                return true;
            }
        }
        result = null;
        return false;
    }
}
```

In the preceding code, the endpoint returns a JSON representation of the `Coordinate` class, while the `TryParse` method parses the input string into a `Coordinate` object.

The `Parse` method of the `Coordinate` class comes from the `IParsable<TSelf>` interface and is not needed for model binding.

For example, if we request the following URI:

```
/minimal-endpoint-input-Coordinate?coordinate=45.501690%2C%20-73.567253
```

The endpoint returns:

```
{
  "latitude": 45.50169,
  "longitude": -73.567253
}
```

Parsing a string into an object is a viable choice for simple scenarios. However, more complex scenarios require another technique that we explore next.

Manual binding

The `BindAsync` method takes an `HttpContext` and a `ParameterInfo` parameter and returns a `ValueTask<TSelf>` where `TSelf` is the type we are writing data binding for. The `HttpContext` represents the source of the data (the HTTP request), and the `ParameterInfo` represents the delegate's parameter, from which we could want to know something, like its name.The data-binding API supports the implementation of one of the following methods:

```
public static ValueTask<TSelf?> BindAsync(HttpContext context, ParameterInfo parameter);
public static ValueTask<TSelf?> BindAsync(HttpContext context);
```

> Implementing the `IBindableFromHttpContext<TSelf>` interface provides the appropriate `BindAsync` method.

Here is an example that binds a `Person` from the HTTP request's query parameters:

```
app.MapGet(
    "minimal-endpoint-input-Person/",
    (Person person) => person
);
public class Person : IBindableFromHttpContext<Person>
{
    public required string Name { get; set; }
    public required DateOnly Birthday { get; set; }
    public static ValueTask<Person?> BindAsync(
        HttpContext context,
        ParameterInfo parameter)
    {
        var name = context.Request.Query["name"].Single();
        var birthdayIsValid = DateOnly.TryParse(
            context.Request.Query["birthday"],
            out var birthday
        );
        if (name is not null && birthdayIsValid) {
            var person = new Person() {
                Name = name,
                Birthday = birthday
            };
            return ValueTask.FromResult(person)!;
        }
        return ValueTask.FromResult(default(Person));
    }
}
```

The preceding code returns a JSON representation of the person. For example, if we request the following URI:

```
/minimal-endpoint-input-Person?name=John%20Doe&birthday=2023-06-14
```

The endpoint returns:

```
{
  "name": "John Doe",
  "birthday": "2023-06-14"
}
```

As we can see, the `BindAsync` method is way more powerful than the `TryParse` method because we can access a broader range of options using the `HttpContext`, allowing us to cover complex use cases.However, in this case, we could have leveraged the `[AsParameters]` attribute to achieve the same result and get the data from the query without needing to write the data-binding code manually. What a great opportunity to explore this attribute; here's the updated version of the same code:

```
app.MapGet(
    "minimal-endpoint-input-Person2/",
    ([AsParameters] Person2 person) => person
);
public class Person2
{
    public required string Name { get; set; }
    public required DateOnly Birthday { get; set; }
}
```

That's it; the `AsParameters` attribute did the work for us!Now that we covered reading the input values from different places in the HTTP request, it is time to explore how to output results.

Outputs

There are several ways to output data from our delegates:

- Return a serializable object.
- Return an `IResult` implementation.
- Return a `Results<TResult1, TResult2, …, TResultN>` where the `TResult` generic parameters represent the different `IResult` implementation the endpoint can return.

We explore those possibilities next.

Serializable object

The first is to return a serializable object, as we did in the previous section about inputs. ASP.NET Core serializes the object into a JSON string and sets the `Content-Type` header to `application/json`. This is the easiest way to do it but also the less flexible.For example, the following code:

```
app.MapGet(
    "minimal-endpoint-output-coordinate/",
    () => new Coordinate {
        Latitude = 43.653225,
        Longitude = -79.383186
    }
);
```

Outputs the following JSON string:

```
{
  "latitude": 43.653225,
  "longitude": -79.383186
}
```

The problem with this approach is that we don't control the status code, nor can we return multiple different results from the endpoint. For example, if the endpoint returns `200 OK` in one case and `404 Not Found` in another. To help us with this, we explore the `IResult` abstraction next.

IResult

The next option is to return the `IResult` interface. We can leverage the `Results` or `TypedResults` classes from the `Microsoft.AspNetCore.Http` namespace to do that.

I recommend defaulting to using `TypedResults`, which .NET 7 introduced.

The main difference between the two is that the methods in the `Results` class return `IResult`, while those in `TypedResults` return a typed implementation of the `IResult` interface. This difference may sound insignificant, but it changes everything regarding discoverability by the API Explorer. The API Explorer can't automatically discover the API contract of the former, while it can for the latter. This is possible because the compiler can infer the return type, but it creates challenges when returning more than one result type.

> This choice impacts the amount of work you'll have to put into getting well-crafted OpenAPI specifications automatically (or not so automatically).

The following two endpoints explicitly state the result is 200 OK, one with each class:

```
app.MapGet(
    "minimal-endpoint-output-coordinate-ok1/",
    () => Results.Ok(new Coordinate {
        Latitude = 43.653225,
        Longitude = -79.383186
    })
);
app.MapGet(
    "minimal-endpoint-output-coordinate-ok2/",
    () => TypedResults.Ok(new Coordinate {
        Latitude = 43.653225,
        Longitude = -79.383186
    })
);
```

When looking at the generated OpenAPI specifications, the first endpoint has no return value, while the other has a `Coordinate` definition mimicking our C# class. Next, we dig deeper into the `TypedResults` class.

Typed results

We can use the methods of the `TypedResults` class to generate strongly-typed outputs. They allow us to control the output while informing ASP.NET Core of the specifics, like the status code and return type.

> Please note that for the sake of simplicity, I've omitted variants and overloads, focusing solely on the fundamental of each method in the tables.

Let's start with the successful status code, where the `200 OK` status code is most likely the most common:

Method	Description
`Accepted`	Produces a `202 Accepted` response, indicating the beginning of an asynchronous process.
`Created`	Produces a `201 Created` response, indicating the system created the entity, the location of the entity, and the entity itself.
`Ok`	Produces a `200 OK` response, indicating the operation was successful.

Table 5.4: TypedResults successful status code methods.

On top of the successes, we must know how to tell clients about the errors. For example, the `400 Bad Request` and `404 Not Found` are very common to point out the issues with the request. The following table contains methods to assist in indicating such issues to the clients:

Method	Description
`BadRequest`	Produces a `400 Bad Request` response, indicating an issue with the client request, often a validation error.
`Conflict`	Produces a `409 Conflict` response, indicating a conflict occurred when processing the request, often a concurrency error.
`NotFound`	Produces a `404 Not Found` response, indicating the resource was not found.
`Problem`	Produces a response adhering to the *Problem Details* structure defined by **RFC7807**, providing a standardized encapsulation of the error. We can modify the status code, which defaults to a `500 Internal Server Error .`

`UnprocessableEntity`	Produces a `422 Unprocessable Content` response, indicating that while the server comprehends the request's content type and the syntax is correct, it cannot process the instructions or the entity.
`ValidationProblem`	Produces a `400 Bad Request` response adhering to the *Problem Details* structure defined by **RFC7807** . We can use this method to communicate input validation problems to the client.

Table 5.5: TypedResults problematic status code.

> Leveraging the *Problem Details* structure improves the interoperability of our API by choosing a standard instead of crafting a custom way of returning our API errors.

It is rarer in APIs than with conventional web applications to send redirections to clients, yet, we can redirect the clients to another URL with one of the following methods when needed:

Method	**Description**
`LocalRedirect`	Produces a `301 Moved Permanently` , `302 Found` , `307 Temporary Redirect` , or `308 Permanent Redirect` based on the specified arguments. This method throws an exception at runtime if the URL is not local, which is an excellent option to ensure dynamically generated URLs are not sending users away. For example, when the URL is composed using user inputs.
`Redirect`	Produces a `301 Moved Permanently` , `302 Found` , `307 Temporary Redirect` , or `308 Permanent Redirect` based on the specified arguments.

Table 5.6: TypedResults redirection status code.

Sending files to the client is another helpful feature; for example, the API could protect the files using authorization. The following table showcases a few helper methods to send files to the client:

Method	**Description**
`File`	Writes the file content to the response stream. The `File` methods are aliases for the `Bytes` and `Stream` methods. We look at those soon.
`PhysicalFile`	Writes the content of a physical file to the response using an absolute or relative path. Caution: Do not expose this method to raw user inputs because it can read files outside the web content root. So a malicious actor could craft a request to access restricted files.
`VirtualFile`	Writes the content of a physical file to the response using an absolute or relative path.

	This method limits the file's location to the web content root and is safer when dealing with user inputs.

Table 5.7: TypedResults methods for downloading files.

On top of the methods we have explored so far, the following table lists ways to handle the content directly in its raw format. These content-handling methods can become handy when you need more control over what is happening:

Method	**Description**
`Bytes`	Writes the byte-array or `ReadOnlyMemory<byte>` content directly to the response. It defaults to sending an `application/octet-stream` MIME Type to the client. This default behavior can be customized as needed.
`Content`	Writes the specified content `string` to the response stream. It defaults to sending a `text/plain` MIME Type to the client. This default behavior can be customized as needed.
`Json`	Serializes the specified object to JSON. It defaults to sending an `application/json` MIME Type to the client with a `200 OK` status code. These default behaviors can be customized as needed. Compared to the other methods, like the `Ok` method, the primary advantage is that it allows us to use a non-default instance of the `JsonSerializerOptions` class to configure the serialization of the response.
`NoContent`	Produces an empty `204 No Content` response.
`StatusCode`	Produces an empty response with the specified status code.
`Stream`	Allows writing directly to the response stream from another `Stream`. It defaults to sending an `application/octet-stream` MIME Type to the client. This default behavior can be customized as needed. This method is highly customizable, returns a `200 OK` status code by default, and supports range requests that produce a status code `206 Partial Content` or `416 Range Not Satisfiable`.
`Text`	Writes the content string to the HTTP response. It defaults to sending a `text/plain` MIME Type to the client. This default behavior can be customized as needed, as well as the text encoding.

Table 5.8: TypedResults raw content handling methods.

> The `application/octet-stream` MIME Type suggests that the response body is a file without specifying its type, which typically prompts the browser to download the file.

Finally, we can leverage the following methods of the `TypedResults` class to create security flows. Most of these methods rely on the current implementation of the `IAuthenticationService` interface, which dictates their behavior:

Method	Description
`Challenge`	Initiates a challenge for authentication when an unauthenticated user requests an endpoint that necessitates authentication.
`Forbid`	Invoke when an authenticated user tries to access a resource for which they do not have the necessary permissions. By default, it produces a `403 Forbidden` response, although the behavior may vary depending on the specific authentication scheme.
`SignIn`	Commences the sign-in process for a user, based on the specified authentication scheme.
`SignOut`	Initiates the sign-out process for the given authentication scheme.
`Unauthorized`	Produces a `401 Unauthorized` response.

Table 5.9: TypedResults security-related methods.

Now that we have covered the `TypedResults` possibilities, we explore how to return those typed results next.

Returning multiple typed results

While returning a single typed result is helpful, the capability to produce multiple results is better aligned with real-life scenarios. As we previously examined, it's possible to return multiple `IResult` objects, but we're restricted to a single typed result. This limitation arises from the compiler's inability to identify a shared interface and deduce an `IResult` return type from the typed results. Even if the compiler could, that wouldn't enhance discoverability.To overcome this limitation, .NET 7 introduced the `Results<T1, TN>` types, allowing us to return up to six different typed results.Here’s an example that returns `200 OK` when the random number is even and `209 Conflict` when it is odd:

```
app.MapGet(
    "multiple-TypedResults/",
    Results<Ok, Conflict> ()
        => Random.Shared.Next(0, 100)  %  2  ==  0
            ? TypedResults.Ok()
            : TypedResults.Conflict()
);
```

This also works with methods like this:

```
app.MapGet(
    "multiple-TypedResults-delegate/",
    MultipleResultsDelegate
);
Results<Ok, Conflict> MultipleResultsDelegate()
{
    return Random.Shared.Next(0, 100) % 2 == 0
        ? TypedResults.Ok()
        : TypedResults.Conflict();
}
```

Adopting this approach enhances the API Explorer's comprehension of the API, thereby allowing libraries like Swagger and Swagger UI to automatically generate more accurate and detailed API documentation.Next, we explore adding more metadata to endpoints.

Metadata

Sometimes, relying solely on automatic metadata is not enough. That's why ASP.NET Core offers different helper methods to fine-tune the metadata of our endpoints. We can use most helper methods with groups and routes. In the case of a group, the metadata cascades to its children, whether it is another group or a route. However, a child can override the inherited values by changing the metadata.Here's a partial list of helpers and their usage:

Method	Description
`Accepts`	Specifies the supported request content types. *Only applicable to routes.*
`AllowAnonymous`	Specifies that anonymous users can access the endpoint(s).
`CacheOutput`	Adds an output caching policy to the endpoint(s).
`DisableRateLimiting`	Turns off the rate-limiting feature on the endpoint(s).
`ExcludeFromDescription`	Excludes the item from the API Explorer data.

`Produces`
 	
`ProducesProblem`
 	
`ProducesValidationProblem`
	Describes a response, including its type, content type, and status code.
	
	Only applicable to routes.

`RequireAuthorization`	Specifies that only authorized users can access the endpoint(s). We can be more granular by using one of the overloads. For example, we can specify a policy name or an `AuthorizationPolicy` instance. *You must configure authorization for this to work*.
`RequireCors`	Specifies that the endpoint(s) must follow a CORS policy. *You must configure CORS for this to work*.
`RequireRateLimiting`	Adds a rate-limiting policy to the endpoint(s). *You must configure rate-limiting for this to work*.
`WithDescription`	Describes the route. When used on a group, the description cascades to all routes within that group.
`WithName`	Attributes a name to the route. We can use this name to identify the route, which must be unique. For example, we can use the route name with the `LinkGenerator` class to generate the URL of that endpoint.
`WithOpenApi`	

	Ensure the builder adds the compatible metadata about the endpoint so tools like Swagger can generate the Open API specifications. We can also use this method to configure the operation and parameters instead of the other extension methods. *You must call this method for many others to work.*
`WithSummary`	Add a summary to the route. When used on a group, the summary cascades to all routes within that group.
`WithTags`	Add tags to the route. When used on a group, the tags cascade to all routes within that group.

Table 5.10: Metadata helper methods.

Let's look at an example that creates a group, tags it, then ensures that all routes under that group have their metadata harvestable by SwaggerGen (the API Explorer):

```
var metadataGroup = app
    .MapGroup("minimal-endpoint-metadata")
    .WithTags("Metadata Endpoints")
    .WithOpenApi()
;
```

We can now define endpoints on that group using the `metadataGroup` as if it was the `app` variable. Next, we create an endpoint that we name `"Named Endpoint"` and describe it using the `WithOpenApi` method, including deprecating it:

```
const string NamedEndpointName = "Named Endpoint";
metadataGroup
    .MapGet(
        "with-name",
        () => $"Endpoint with name '{NamedEndpointName}'."
    )
    .WithName(NamedEndpointName)
    .WithOpenApi(operation => {
        operation.Description = "An endpoint that returns its name.";
        operation.Summary = $"Endpoint named '{NamedEndpointName}'.";
        operation.Deprecated = true;
        return operation;
    })
;
```

Next, we generate a URL based on the preceding named endpoint, we describe the endpoint using the `WithDescription` method and add metadata to the `endpointName` parameter, including an example. Once again we leverage the `WithOpenApi` method:

```
metadataGroup
    .MapGet(
        "url-of-named-endpoint/{endpointName?}",
        (string? endpointName, LinkGenerator linker) => {
            var name = endpointName ?? NamedEndpointName;
            return new {
                name,
                uri = linker.GetPathByName(name)
```

```
            };
        }
    )
    .WithDescription("Return the URL of the specified named endpoint.")
    .WithOpenApi(operation => {
        var endpointName = operation.Parameters[0];
        endpointName.Description = "The name of the endpoint to get the URL for.";
        endpointName.AllowEmptyValue = true;
        endpointName.Example = new OpenApiString(NamedEndpointName);
        return operation;
    })
;
```

When requesting the preceding endpoint, we get the URL of the specified route. By default, we get the URL of the `"Named Endpoint"` route—our only named route—in the following JSON format:

```
{
  "name": "Named Endpoint",
  "uri": "/minimal-endpoint-metadata/with-name"
}
```

As a last example, we can exclude a route from the metadata with the `ExcludeFromDescription` method:

```
metadataGroup
    .MapGet("excluded-from-open-api", () => { })
    .ExcludeFromDescription()
;
```

When looking at the Swagger UI, we can see the following section representing the group we just defined:

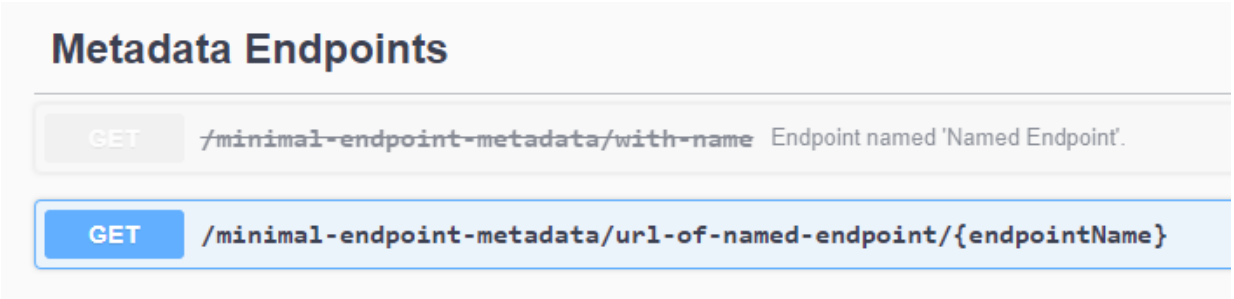


Figure 5.1: a screenshot of the Swagger UI "Metadata Endpoints" route group.

We can see two endpoints in the preceding screenshot and, as expected, the third endpoint was excluded.The first route is marked as deprecated and shows a summary. When we open it, we see a warning, a description, no parameters, and one 200 OK response with a mime-type `text/plain`.

> I omitted to add a screenshot of the Swagger UI for the first endpoint since it does not add much and would be hard to read. It is better to run the `Minimal.API` program and navigate to the `/swagger/index.html` URL instead.

The second route does not have a summary, but when we open it, we have a description. The metadata we added for the `endpointName` parameter is there, and most interestingly, the example became the default value:

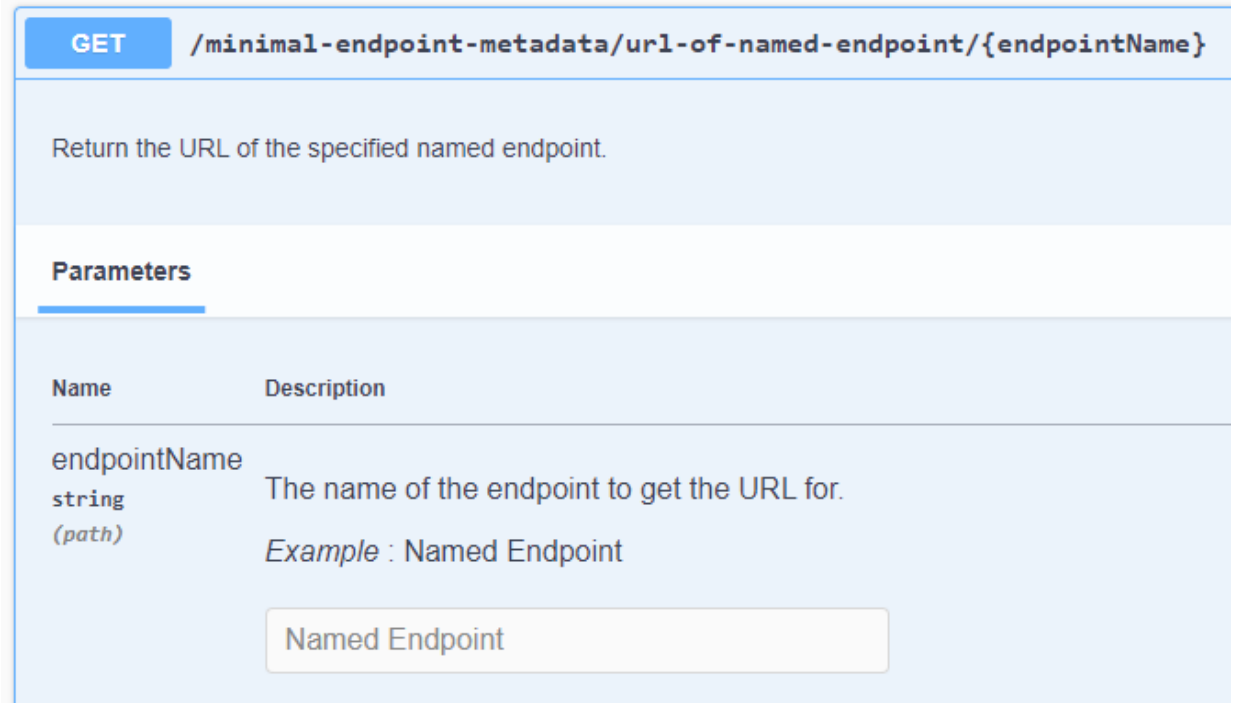


Figure 5.2: a screenshot of the Swagger UI showcasing the metadata of the `endpointName` parameter.

Swagger UI can become very handy for manually calling our API during development or leveraging other Open API compatible tools. For example, we could generate code based on the Open API specs, like a TypeScript client.Next, we explore how to configure the Minimal API JSON serializer.

Configuring JSON serialization

In ASP.NET Core, we can customize the JSON serializer globally or create a new one for a specific scenario.To change the default serializer behaviors, we can invoke the `ConfigureHttpJsonOptions` method, which configures the `JsonOptions` object. From there, we can change the options like this:

```
builder.Services.ConfigureHttpJsonOptions(options => {
    options.SerializerOptions.PropertyNamingPolicy = JsonNamingPolicy.KebabCaseLower;
});
```

With the preceding code in the `Program.cs` file, we tell the serializer to serialize the property name following a `lower-case-kebab` naming convention. Here is an example:

Endpoint	**Response**
jsonGroup.MapGet(	{
"kebab-person/",	"first-name": "John",
() => new {	"last-name": "Doe"
FirstName = "John",	}
LastName = "Doe"	
}	

);

Table 5.11: Showcasing the output of the JsonNamingPolicy.KebabCaseLower policy.

We can achieve the same for specific endpoints by using the `TypedResults.Json` method and specifying an instance of `JsonSerializerOptions`. The following code accomplishes the same outcome while preserving the default serialization options:

```
var kebabSerializer = new JsonSerializerOptions(JsonSerializerDefaults.Web)
{
    PropertyNamingPolicy = JsonNamingPolicy.KebabCaseLower
};
jsonGroup.MapGet(
    "kebab-person/",
    () => TypedResults.Json(new {
        FirstName = "John",
        LastName = "Doe"
    }, kebabSerializer)
);
```

I highlighted the changes between this code and the previous example. First, we create an instance of the `JsonSerializerOptions` class. To simplify the configuration, we start with the default web serialization values by passing the `JsonSerializerDefaults.Web` argument to the constructor. Then, in the object initializer, we set the value of the `PropertyNamingPolicy` property to `JsonNamingPolicy.KebabCaseLower`, which end up with the same result as before. Finally, to use our options, we pass the `kebabSerializer` variable as the second argument of the `TypedResults.Json` method.

Serializing enums as string

I often find myself changing a configuration to output the string representation of an `enum` value. To do so, we must register an instance of the `JsonStringEnumConverter` class. Afterward, enums will be serialized as strings. Here is an example:

```
var enumSerializer = new JsonSerializerOptions(JsonSerializerDefaults.Web);
enumSerializer.Converters.Add(new JsonStringEnumConverter());
jsonGroup.MapGet(
    "enum-as-string/",
    () => TypedResults.Json(new {
        FirstName = "John",
        LastName = "Doe",
        Rating = Rating.Good,
    }, enumSerializer)
);
```

When executing the preceding code, we obtain the following result:

```
{
  "firstName": "John",
  "lastName": "Doe",
  "rating": "Good"
}
```

Using the default options yields the following instead:

```
{
  "firstName": "John",
  "lastName": "Doe",
  "rating": 2
}
```

I usually change this option globally since having a human-readable value instead of a number is more explicit, easier to understand for a human, and easier to leverage for a client (machine).

Many other options exist to tweak the serializer, but we can't explore them all here. Next, we look at endpoint filters.

Leveraging endpoint filters

ASP.NET Core 7.0 added the possibility to register endpoint filters. This way, we can encapsulate and reuse cross-cutting concerns and logic across endpoints.

> An example of reusability is that I prefer FluentValidation to .NET attributes, so I created an open-source project implementing a filter that ties Minimal APIs with FluentValidation. I can then reuse that filter across projects by referencing a NuGet package. We explore FluentValidation in *Section 4: Application Patterns*, and I left a link to that project in the *Further Reading* section.

How does it work?

- We can register endpoint filters inline or by creating a class.
- We can add filters to an endpoint or a group using the `AddEndpointFilter` method.
- When adding a filter to a group, it applies to all its children.
- We can add multiple filters per endpoint or group.
- ASP.NET Core executes the filters in the order they are registered.

Let's look at a simple inline filter:

```
inlineGroup
    .MapGet("basic", () => { })
    .AddEndpointFilter((context, next) =>
    {
        return next(context);
    });
```

The highlighted code represents the filter in the form of a delegate. The filter does nothing but executes the next delegate in the chain, in this case, the endpoint delegate itself.In the following example, we use the `Rating` enum and only accept positive ratings in the endpoint. To achieve this, we add a filter that validates the input value before reaching the endpoint:

```
public enum Rating
{
    Bad = 0,
    Ok,
    Good,
    Amazing
}
// ...
inlineGroup
    .MapGet("good-rating/{rating}", (Rating rating)
        => TypedResults.Ok(new { Rating = rating }))
    .AddEndpointFilter(async (context, next) =>
    {
        var rating = context.GetArgument<Rating>(0);
        if (rating == Rating.Bad)
        {
            return TypedResults.Problem(
                detail: "This endpoint is biased and only accepts positive ratings.",
                statusCode: StatusCodes.Status400BadRequest
            );
        }
        return await next(context);
    });
```

From the filter, we leveraged the `EndpointFilterFactoryContext` to access the rating argument. The code then validates that the rating is not `Bad`. If the rating is `Bad`, the filter immediately returns a *problem details* with a `400 Bad Request` status code before reaching out to the endpoint delegate. Otherwise, the endpoint code is executed.You probably wonder how useful writing code like this can be; well, this case is purely educational and

not that useful in a real-world scenario. We could have validated the parameter in the endpoint delegate directly and saved ourselves the trouble of accessing it through its index. Nonetheless, it shows how filters work so that you can build helpful real-life filters with this knowledge; remember that coding is like playing LEGO® blocks.To improve on this foundation and make the previous example reusable, we can extract the filter logic into a class and apply it to multiple endpoints. We could also move the inline implementation to a group so it affects all its children. Let's have a look at making our inline filter a class.A filter class must implement the `IEndpointFilter` interface. Here's the reimplementation of the previous logic in the `GoodRatingFilter` class, and two endpoints using it:

```
filterGroup
    .MapGet("good-rating/{rating}", (Rating rating)
        => TypedResults.Ok(new { Rating = rating }))
    .AddEndpointFilter<GoodRatingFilter>();
;
filterGroup
    .MapPut("good-rating/{rating}", (Rating rating)
        => TypedResults.Ok(new { Rating = rating }))
    .AddEndpointFilter<GoodRatingFilter>();
;
public class GoodRatingFilter : IEndpointFilter
{
    public async ValueTask<object?> InvokeAsync(
        EndpointFilterInvocationContext context,
        EndpointFilterDelegate next)
    {
        var rating = context.GetArgument<Rating>(0);
        if (rating == Rating.Bad)
        {
            return TypedResults.Problem(
                detail: "This endpoint is biased and only accepts positive ratings.",
                statusCode: StatusCodes.Status400BadRequest
            );
        }
        return await next(context);
    }
}
```

The `GoodRatingFilter` class `InvokeAsync` method code is the same as the inline version. However, we used it twice, keeping our code DRY.Encapsulating pieces of logic in filters can be very beneficial, whether it is input validation, logging, exception handling, or another scenario.And this is not it; there is one more thing about filters we must explore.

Leveraging the endpoint filter factory

We can use an endpoint filter factory to run code when ASP.NET Core builds the endpoint (makes the `RequestDelegate`) before declaring the filter. Then, from the factory, we control the creation of the filter itself.

> We explore the factory pattern in *Chapter 7*.

The following code registers an endpoint filter factory:

```
inlineGroup
    .MapGet("endpoint-filter-factory", () => "RAW")
    .AddEndpointFilterFactory((filterFactoryContext, next) =>
    {
        // Building RequestDelegate code here.
        var logger = filterFactoryContext.ApplicationServices
            .GetRequiredService<ILoggerFactory>()
            .CreateLogger("endpoint-filter-factory");
        logger.LogInformation("Code that runs when ASP.NET Core builds the RequestDelegate");
        // Returns the EndpointFilterDelegate ASP.NET Core executes as part of the pipeline.
        return async invocationContext =>
        {
            logger.LogInformation("Code that ASP.NET Core executes as part of the pipeline");
            // Filter code here
            return await next(invocationContext);
```

```
        };
    });
```

The preceding code adds an endpoint filter factory that logs some information to the console, which allows us to track what is happening. The highlighted code represents the filter itself. For example, we could write the same code as the `GoodRatingFilter` class there.Next, let's look at what happens when we execute the program and load the endpoint five times:

```
[11:22:56.673] info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7298
[11:22:56.698] info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5085
[11:22:56.702] info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
[11:22:56.705] info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
[11:22:56.708] info: Microsoft.Hosting.Lifetime[0]
      Content root path: .../C05/Minimal.API
[11:23:28.349] info: endpoint-filter-factory[0]
      Code that runs when ASP.NET Core builds the RequestDelegate
[11:23:45.181] info: endpoint-filter-factory[0]
      Code that runs when ASP.NET Core builds the RequestDelegate
[11:24:56.043] info: endpoint-filter-factory[0]
      Code that ASP.NET Core executes as part of the pipeline
[11:24:57.439] info: endpoint-filter-factory[0]
      Code that ASP.NET Core executes as part of the pipeline
[11:24:58.443] info: endpoint-filter-factory[0]
      Code that ASP.NET Core executes as part of the pipeline
[11:24:59.262] info: endpoint-filter-factory[0]
      Code that ASP.NET Core executes as part of the pipeline
[11:25:00.154] info: endpoint-filter-factory[0]
      Code that ASP.NET Core executes as part of the pipeline
```

Here's what happened from the preceding output:

1. The API starts (the first ten lines).
2. ASP.NET Core executes the factory code when building the `RequestDelegate` from the `EndpointRoutingMiddleware` (the next two lines).
3. `SwaggerGen`, using the `ApiExplorer`, also does the same from the `SwaggerMiddleware`, hence the second factory call (the next two lines).
4. Afterward, ASP.NET Core only executes the filters during requests—in this case, five times (the last 10 lines).

Now that we've seen how it runs, it is time to learn how it works.

> Don't worry if you don't understand how the `GetRequiredService` method or the `ILoggerFactory` interface work; we explore those topics in *Chapter 8*, *Dependency Injection*, and *Chapter 10*, *Logging*.

We start by registering the endpoint filter factory using the `AddEndpointFilterFactory` method, which applies to groups and individual routes (we dig deeper into groups next). The factory delegate is of type `Func<EndpointFilterFactoryContext, EndpointFilterDelegate, EndpointFilterDelegate>`.Inside the delegate, using the `EndpointFilterFactoryContext` parameter named `filterFactoryContext`, we have access to the following objects:

- The `ApplicationServices` property provides access to an `IServiceProvider` interface, allowing us to extract services from the container, as demonstrated with the `ILoggerFactory` interface.
- The `MethodInfo` property offers a `MethodInfo` object granting access to the caller to which we add the endpoint filter factory. This object encapsulates the reflection data, including types, generic parameters, attributes, and more.

Finally, the factory delegate returns the filter ASP.NET Core executes when a request hits the endpoint(s). In this case, the filter is the following:

```
async invocationContext =>
{
    logger.LogInformation("Code that ASP.NET Core executes as part of the pipeline");
    return await next(invocationContext);
};
```

The `next` parameter (highlighted code) represents the next filter in line or the endpoint itself—it works the same here as with any endpoint filter. Not calling the `next` parameter means ASP.NET Core will never execute the endpoint code, which is a way to control the flow of the application.

> To make an endpoint filter factory more reusable, we could create a class, an extension method, or return an existing filter class. We can also combine those ways to craft a more testable and DRY implementation of an endpoint filter factory. While we won't delve into these specific approaches in this context, by the end of the book, you should have acquired enough knowledge to achieve these tasks by yourself.

Next, we look at organizing our endpoints.

Organizing endpoints

Grouping endpoints using the `MapGroup` method is an effective organizational strategy. However, defining all routes directly within the `Program.cs` file can result in a long and challenging-to-navigate file. To mitigate this, we can arrange these groups of endpoints in separate classes and create an extension method to add these endpoints to the `IEndpointRouteBuilder`. We can also encapsulate the groups, or even multiple groups, within another assembly, which we can load from the API.

> We explore ways to design applications in *Section 4: Application Patterns*, including in *Chapter 18*, *Request-EndPoint-Response (REPR)*, and *Chapter 20*, *Modular Monolith*.

Let’s start with simple groups.

MapGroup

Creating groups is the first tool to organize the routes of our APIs. It comes with the following advantages:

- We can create a shared URL prefix for the group’s children.
- We can add metadata that applies to the group’s children.
- We can add endpoint filters that apply to the group’s children.

Here is an example of a group that configures those three items:

```
// Create a reusable logger
var loggerFactory = app.ServiceProvider
    .GetRequiredService<ILoggerFactory>();
var groupLogger = loggerFactory
    .CreateLogger("organizing-endpoints");
// Create the group
var group = app
    .MapGroup("organizing-endpoints")
    .WithTags("Organizing Endpoints")
    .AddEndpointFilter(async (context, next) => {
        groupLogger.LogTrace("Entering organizing-endpoints");
        // Omited argument logging
        var result = await next(context);
        groupLogger.LogTrace("Exiting organizing-endpoints");
        return result;
    })
;
// Map endpoints in the group
group.MapGet("demo/", ()
    => "GET endpoint from the organizing-endpoints group.");
group.MapGet("demo/{id}", (int id)
    => $"GET {id} endpoint from the organizing-endpoints group.");
```

The highlighted code does the following:

- Configures the `organizing-endpoints` URL prefix.
- Add the `Organizing Endpoints` tag (metadata).
- Add an inline filter that logs information about the requests.

We can reach the endpoints at the following URL:

- `/organizing-endpoints/demo`
- `/organizing-endpoints/demo/123`

As the following Swagger UI screenshot shows, the two endpoints are tagged correctly:

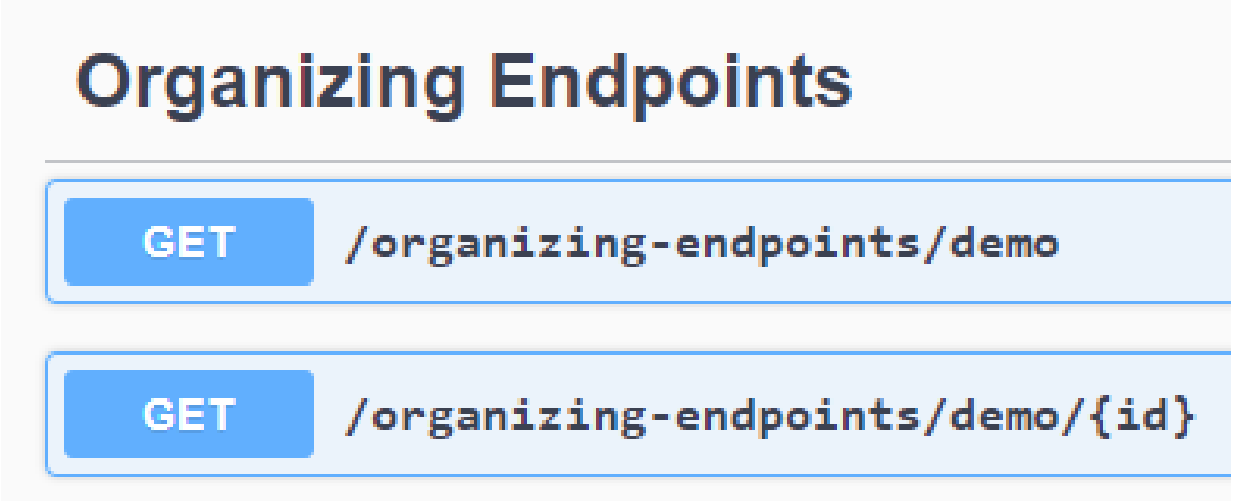


Figure 5.3: The two endpoints under the Organizing Endpoints tag

Then, after requesting the two endpoints, we end up with the following logs excerpt:

```
[23:55:01.516] trce: organizing-endpoints[0]
      Entering organizing-endpoints
[23:55:01.516] trce: organizing-endpoints[0]
      Exiting organizing-endpoints
[23:55:06.028] trce: organizing-endpoints[0]
      Entering organizing-endpoints
[23:55:06.028] dbug: organizing-endpoints[0]
      Argument 1: Int32 = 123
[23:55:06.028] trce: organizing-endpoints[0]
      Exiting organizing-endpoints
```

As demonstrated, leveraging groups for shared configuration streamlines the process of setting up aspects like authorization rules, caching, and more. By adopting this approach, we uphold the DRY (Don't Repeat Yourself) principle, improving the maintainability of our code.Next, we encapsulate mapping endpoints into classes.

Create a custom Map extension method

Now that we explored how to create groups, it is time to move the endpoints out of the `Program.cs` file. One way is to create an extension method that registers the route. To achieve this, we must extend the `IEndpointRouteBuilder` interface as follows:

```
namespace Minimal.API;
public static class OrganizingEndpoints
{
    public static void MapOrganizingEndpoints(
        this IEndpointRouteBuilder app)
```

```
    {
        // Map endpoints and groups here
    }
}
```

Then we must call our extension method in the `Program.cs` file, as follows:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
app.MapOrganizingEndpoints();
app.Run();
```

And with that, we've seen how a simple technique allows us to group our routes together, offering an organized way to structure our APIs.We can improve on this technique by returning the `IEndpointRouteBuilder` instead of `void` in our extension method, which makes our extension “fluent”, as demonstrated below:

```
namespace Minimal.API;
public static class OrganizingEndpoints
{
    public static IEndpointRouteBuilder MapOrganizingEndpointsFluently(
        this IEndpointRouteBuilder app)
    {
        var group = app
            .MapGroup("organizing-endpoints-fluently")
            .WithTags("Organizing Fluent Endpoints")
        ;
        // Map endpoints and groups here
        return app;
    }
}
```

Then from the `Program.cs` file, we can call multiple maps in a “single line of code” like the following:

```
app
    .MapOrganizingEndpointsFluently()
    .MapOrganizingEndpoints()
;
```

Creating fluent APIs is very convenient, especially in such cases.

This technique allows you to create fluent APIs for anything, not just registering routes.

Another variation of this pattern exists that's worth noting. Rather than returning the `IEndpointRouteBuilder` , the extension method can return the `RouteGroupBuilder` instead, granting the caller access to the group itself. Here is an example:

```
namespace Minimal.API;
public static class OrganizingEndpoints
{
    public static RouteGroupBuilder MapOrganizingEndpointsComposable(
        this IEndpointRouteBuilder app)
    {
        var group = app
            .MapGroup("organizing-endpoints-composable")
            .WithTags("Organizing Composable Endpoints")
        ;
        // Map endpoints and groups here
        return group;
    }
}
```

We can use such methods to create a complex hierarchy of routes and groups by splitting the registration into multiple files. The second version is the most common way. It does not expose the group to the outside (encapsulation) and allows fluent chaining of other method calls.And voilà! We now know how basic extension methods can help us organize our endpoints. Next, we explore how to move those extension methods into class libraries.

Class libraries

This last technique allows us to create and register routes from class libraries using the previously explored techniques.First, we must create a class library project, which we can do using the `dotnet new classlib` CLI command.Unfortunately, a class library project cannot access everything we need, like the `IEndpointRouteBuilder` interface. The good news is that it is trivial to change this fact. All we have to do is add a `FrameworkReference` element in an `ItemGroup` element in the `csproj` file, as follows:

```
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net8.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <FrameworkReference Include="Microsoft.AspNetCore.App" />
    </ItemGroup>
</Project>
```

That minor addition equips us with everything necessary to create an ASP.NET Core-enabled library, including mapping endpoints! Transferring the preceding C# code into this class library project should yield the same functional results as in a web application.

> I used this technique in the `Shared` project of the solution we explore in this chapter and the next. If you are curious, the complete source code is available on GitHub.

Next, we mix Minimal APIs and DTOs.

Using Minimal APIs with Data Transfer Objects

This section explores leveraging the **Data Transfer Object** (**DTO**) pattern with minimal APIs.

> This section is the same as we explore in *Chapter 6*, *MVC*, but in the context of Minimal APIs. Moreover, the two code projects are part of the same Visual Studio solution for convenience, allowing you to compare the two implementations.

Goal

As a reminder, DTOs aim to *control the inputs and outputs of an endpoint* by decoupling the API contract from the application's inner workings. DTOs empower us to define our APIs without thinking about the underlying data structures, leaving us to craft our REST APIs how we want.

> We discuss REST APIs and DTOs more in-depth in *Chapter 4*, *REST APIs*.

Other possible objectives are to save bandwidth by limiting the amount of information the API transmits, flattening the data structure, or adding API features that cross multiple entities.

Design

Let's start by analyzing a diagram that shows how minimal APIs work with DTOs:

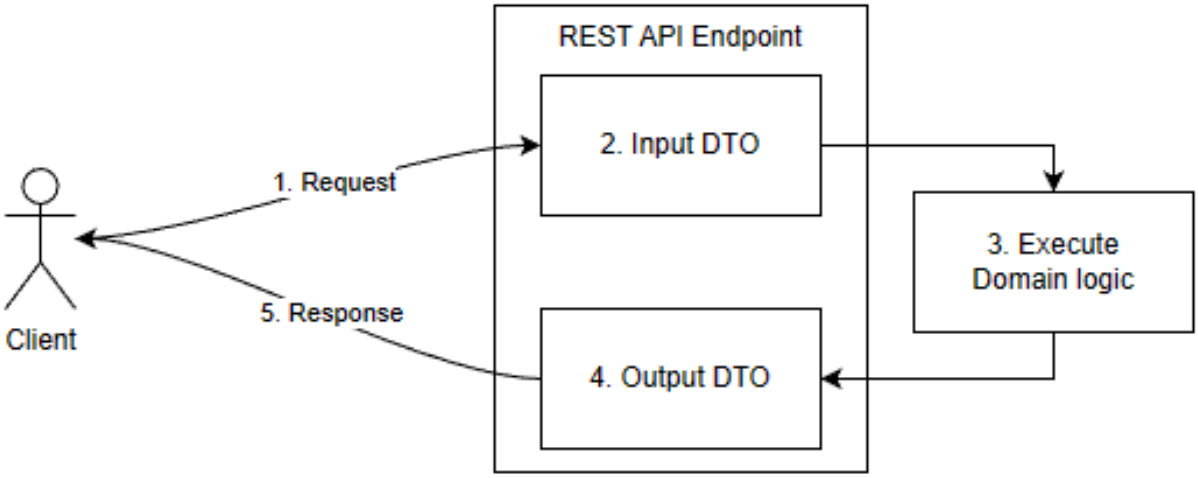


Figure 5.4: An input DTO hitting some domain logic, then the endpoint returning an output DTO

DTOs allow the decoupling of the domain (3) from the request (1) and the response (5). This model empowers us to manage the inputs and outputs of our REST APIs independently from the domain. Here's the flow:

1. The client sends a request to the server.
2. ASP.NET Core leverages its data binding and parsing mechanism to convert the information of the HTTP request to C# (input DTO).
3. The endpoint does what it is supposed to do.
4. ASP.NET Core serializes the output DTO to the HTTP response.
5. The client receives and handles the response.

Let's explore some code to understand the concept better.

Project – Minimal API

This code sample is the same as the next chapter but uses Minimal APIs instead of the MVC framework.**Context**: we must build an application to manage customers and contracts. We must track the state of each contract and have a primary contact in case the business needs to contact the customer. Finally, we must display the number of contracts and the number of opened contracts for each customer on a dashboard.The model is the following:

```
namespace Shared.Models;
public record class Customer(
    int Id,
    string Name,
    List<Contract> Contracts
);
public record class Contract(
    int Id,
    string Name,
    string Description,
    WorkStatus Status,
    ContactInformation PrimaryContact
);
public record class WorkStatus(int TotalWork, int WorkDone)
{
    public WorkState State =>
        WorkDone == 0 ? WorkState.New :
        WorkDone == TotalWork ? WorkState.Completed :
        WorkState.InProgress;
}
public record class ContactInformation(
    string FirstName,
    string LastName,
```

```
    string Email
);
public enum WorkState
{
    New,
    InProgress,
    Completed
}
```

The preceding code is straightforward. The only piece of logic is the `WorkStatus.State` property that returns `WorkState.New` when the work has not yet started on that contract, `WorkState.Completed` when all the work is completed, or `WorkState.InProgress` otherwise.The endpoints (`CustomersEndpoints.cs`) leverage the `ICustomerRepository` interface to simulate database operations. The implementation is unimportant. It uses a `List<Customer>` as the database. Here's the interface that allows querying and updating the data:

```
using Shared.Models;
namespace Shared.Data;
public interface ICustomerRepository
{
    Task<IEnumerable<Customer>> AllAsync(
        CancellationToken cancellationToken);
    Task<Customer> CreateAsync(
        Customer customer,
        CancellationToken cancellationToken);
    Task<Customer?> DeleteAsync(
        int customerId,
        CancellationToken cancellationToken);
    Task<Customer?> FindAsync(
        int customerId,
        CancellationToken cancellationToken);
    Task<Customer?> UpdateAsync(
        Customer customer,
        CancellationToken cancellationToken);
}
```

Now that we know about the underlying foundation, we explore CRUD endpoints that do not leverage DTOs.

Raw CRUD endpoints

Many issues can arise if we create CRUD endpoints to manage the customers directly (see `CustomersEndpoints.cs`). First, a little mistake from the client could erase several data points. For example, if the client forgets to send the contracts during a `PUT` operation, that would delete all the contracts associated with that customer. Here's the controller code:

```
// PUT raw/customers/1
group.MapPut("/{customerId}", async (int customerId, Customer input, ICustomerRepository customer
{
    var updatedCustomer = await customerRepository.UpdateAsync(
        input,
        cancellationToken
    );
    if (updatedCustomer == null)
    {
        return Results.NotFound();
    }
    return Results.Ok(updatedCustomer);
});
```

The highlighted code represents the customer update. So to mistakenly remove all contracts, a client could send the following HTTP request (from the `Minimal.API.http` file):

```
PUT {{Minimal.API.BaseAddress}}/customers/1
Content-Type: application/json
{
  "id": 1,
  "name": "Some new name",
```

```
  "contracts": []
}
```

That request would result in the following response entity:

```
{
  "id": 1,
  "name": "Some new name",
  "contracts": []
}
```

Previously, however, that customer had contracts (seeded when we started the application). Here's the original data:

```
{
  "id": 1,
  "name": "Jonny Boy Inc.",
  "contracts": [
    {
      "id": 1,
      "name": "First contract",
      "description": "This is the first contract.",
      "status": {
        "totalWork": 100,
        "workDone": 100,
        "state": "Completed"
      },
      "primaryContact": {
        "firstName": "John",
        "lastName": "Doe",
        "email": "john.doe@jonnyboy.com"
      }
    },
    {
      "id": 2,
      "name": "Some other contract",
      "description": "This is another contract.",
      "status": {
        "totalWork": 100,
        "workDone": 25,
        "state": "InProgress"
      },
      "primaryContact": {
        "firstName": "Jane",
        "lastName": "Doe",
        "email": "jane.doe@jonnyboy.com"
      }
    }
  ]
}
```

As we can see, by exposing our entities directly, we are giving a lot of power to the consumers of our API. Another issue with this design is the dashboard. The user interface would have to calculate the statistics about the contracts. Moreover, if we implement paging the contracts over time, the user interface could become increasingly complex and even overquery the database, hindering our performance.

I implemented the entire API, which is available on GitHub but without UI.

Next, we explore how we can fix those two use cases using DTOs.

DTO-enabled endpoints

To solve our problems, we reimplement the endpoints using DTOs. These endpoints use methods instead of inline delegates and returns `Results<T1, T2, …>` instead `IResult`. So, let's start with the declaration of the endpoints:

```
var group = routes
    .MapGroup("/dto/customers")
    .WithTags("Customer DTO")
    .WithOpenApi()
;
group.MapGet("/", GetCustomersSummaryAsync)
    .WithName("GetAllCustomersSummary");
group.MapGet("/{customerId}", GetCustomerDetailsAsync)
    .WithName("GetCustomerDetailsById");
group.MapPut("/{customerId}", UpdateCustomerAsync)
    .WithName("UpdateCustomerWithDto");
group.MapPost("/", CreateCustomerAsync)
    .WithName("CreateCustomerWithDto");
group.MapDelete("/{customerId}", DeleteCustomerAsync)
    .WithName("DeleteCustomerWithDto");
```

Next, to make it easier to follow along, here are all the DTOs as a reference:

```
namespace Shared.DTO;
public record class ContractDetails(
    int Id,
    string Name,
    string Description,
    int StatusTotalWork,
    int StatusWorkDone,
    string StatusWorkState,
    string PrimaryContactFirstName,
    string PrimaryContactLastName,
    string PrimaryContactEmail
);
public record class CustomerDetails(
    int Id,
    string Name,
    IEnumerable<ContractDetails> Contracts
);
public record class CustomerSummary(
    int Id,
    string Name,
    int TotalNumberOfContracts,
    int NumberOfOpenContracts
);
public record class CreateCustomer(string Name);
public record class UpdateCustomer(string Name);
```

First, let’s fix our update problem, starting with the reimplementation of the update endpoint leveraging DTOs (see the `DTOEndpoints.cs` file):

```
// PUT dto/customers/1
private static async Task<Results<
    Ok<CustomerDetails>,
    NotFound,
    Conflict
>> UpdateCustomerAsync(
        int customerId,
        UpdateCustomer input,
        ICustomerRepository customerRepository,
        CancellationToken cancellationToken)
{
    // Get the customer
    var customer = await customerRepository.FindAsync(
        customerId,
        cancellationToken
    );
    if (customer == null)
    {
        return TypedResults.NotFound();
    }
    // Update the customer's name using the UpdateCustomer DTO
    var updatedCustomer = await customerRepository.UpdateAsync(
        customer with { Name = input.Name },
```

```
        cancellationToken
    );
    if (updatedCustomer == null)
    {
        return TypedResults.Conflict();
    }
    // Map the updated customer to a CustomerDetails DTO
    var dto = MapCustomerToCustomerDetails(updatedCustomer);
    // Return the DTO
    return TypedResults.Ok(dto);
}
```

In the preceding code, the main differences are (highlighted):

- The request body is now bound to the `UpdateCustomer` class instead of the `Customer` itself.
- The action method returns an instance of the `CustomerDetails` class instead of the `Customer` itself when the operation succeeds.

However, we can see more code in our endpoint than before. That's because it now handles the changes instead of the clients. The action now does:

1. Load the data from the database.
2. Ensure the entity exists.
3. Use the input DTO to update the data, limiting the clients to a subset of properties.
4. Proceed with the update.
5. Ensure the entity still exists (handles conflicts).
6. Copy the Customer into the output DTO and return it.

By doing this, we now control what the clients can do when they send a `PUT` request through the input DTO (`UpdateCustomer`). Moreover, we encapsulated the logic to calculate the statistics on the server. We hid the computation behind the output DTO (`CustomerDetails`), which lowers the complexity of our user interface and allows us to improve the performance without impacting any of our clients (loose coupling).Furthermore, we now use the `customerId` parameter.If we send the same HTTP request as before, which sends more data than we accept, only the customer's name will change. On top of that, we get all the data we need to display the customer's statistics. Here's a response example:


```
{
  "id": 1,
  "name": "Some new name",
  "contracts": [
    {
      "id": 1,
      "name": "First contract",
      "description": "This is the first contract.",
      "statusTotalWork": 100,
      "statusWorkDone": 100,
      "statusWorkState": "Completed",
      "primaryContactFirstName": "John",
      "primaryContactLastName": "Doe",
      "primaryContactEmail": "john.doe@jonnyboy.com"
    },
    {
      "id": 2,
      "name": "Some other contract",
      "description": "This is another contract.",
      "statusTotalWork": 100,
      "statusWorkDone": 25,
      "statusWorkState": "InProgress",
      "primaryContactFirstName": "Jane",
      "primaryContactLastName": "Doe",
      "primaryContactEmail": "jane.doe@jonnyboy.com"
    }
  ]
}
```

As we can see from the preceding response, only the customer's name changed, but we now received the `statusWorkDone` and `statusTotalWork` fields. Lastly, we flattened the data structure.

> DTOs are a great resource to flatten data structures, but you don't have to. You must always design your systems, including DTOs and data contracts, for specific use cases.

As for the dashboard, the "get all customers" endpoint achieves this by doing something similar. It outputs a collection of `CustomerSummary` objects instead of the customers themselves. In this case, the endpoint executes the calculations and copies the entity's relevant properties to the DTO. Here's the code:

```
// GET: dto/customers
private static async Task<Ok<IEnumerable<CustomerSummary>>> GetCustomersSummaryAsync(
    ICustomerRepository customerRepository,
    CancellationToken cancellationToken)
{
    // Get all customers
    var customers = await customerRepository
        .AllAsync(cancellationToken);
    // Map customers to CustomerSummary DTOs
    var customersSummary = customers.Select(customer => new CustomerSummary(
        Id: customer.Id,
        Name: customer.Name,
        TotalNumberOfContracts: customer.Contracts.Count,
        NumberOfOpenContracts: customer.Contracts
            .Count(x => x.Status.State != WorkState.Completed)
    ));
    // Return the DTOs
    return TypedResults.Ok(customersSummary);
}
```

In the preceding code, the action method:

1. Read the entities
2. Create the DTOs and calculate the number of open contracts.
3. Return the DTOs.

As simple as that, we now encapsulated the computation on the server.

> You should optimize such code based on your real-life data source. In this case, a `static List<T>` is low latency. However, querying the whole database to get a count can become a bottleneck.

Calling the endpoint results in the following:

```
[
  {
    "id": 1,
    "name": "Some new name",
    "totalNumberOfContracts": 2,
    "numberOfOpenContracts": 1
  },
  {
    "id": 2,
    "name": "Some mega-corporation",
    "totalNumberOfContracts": 1,
    "numberOfOpenContracts": 1
  }
]
```

It is now super easy to build our dashboard. We can query that endpoint once and display the data in the UI. The UI offloaded the calculation to the backend.

> User interfaces tend to be more complex than APIs because they are stateful. As such, offloading as much complexity to the backend helps. You can use a Backend-for-frontend (BFF) to help with this task. We

explore ways to layer APIs, including the BFF pattern in *Chapter 19*, *Introduction to Microservices Architecture*.

Lastly, you can play with the API using the HTTP requests in the `MVC.API.DTO.http` file. I implemented all the endpoints using a similar technique. If your endpoints become too complex, it is good practice to encapsulate them into other classes. We explore many techniques to organize application code in *Section 4*: *Applications patterns*.

Conclusion

A data transfer object allows us to design an API endpoint with a specific data contract (input and output) instead of exposing the domain or data model. This separation between the presentation and the domain is a crucial element that leads to having multiple independent components instead of a bigger, more fragile one.We use DTOs to control the endpoints' inputs and outputs, giving us more control over what the clients can do or receive.Using the data transfer object pattern helps us follow the SOLID principles in the following ways:

- **S**: A DTO adds clear boundaries between the domain or data model and the API contract. Moreover, having an input and an output DTO help further separate the responsibilities.
- **O**: N/A
- **L**: N/A
- **I**: A DTO is a small, specifically crafted data contract (abstraction) with a clear purpose in the API contract.
- **D**: Due to those smaller interfaces (ISP), DTOs allow changing the implementation details of the endpoint without affecting the clients because they depend only on the API contract (the abstraction).

You should now understand the added value of DTOs and what part in an API contract they play. Finally, you should have a strong base of Minimal APIs possibilities.

Summary

Throughout the chapter, we explored ASP.NET Core Minimal APIs and their integration with the DTO pattern. Minimal APIs simplify web application development by reducing boilerplate code. The DTO pattern helps us decouple the API contract from the application's inner workings, allowing flexibility in crafting REST APIs. DTOs can also save bandwidth and flatten or change data structures. Endpoints exposing their domain or data entities directly can lead to issues, while DTO-enabled endpoints offer better control over data exchanges. We also discussed numerous Minimal APIs aspects, including input binding, outputting data, metadata, JSON serialization, endpoint filters, and endpoint organization. With this foundational knowledge, we can begin to design ASP.NET Core minimal APIs.

For more information about Minimal APIs and what they have to offer, you can visit the *Minimal APIs quick reference* page of the official documentation: https://adpg.link/S47i

In the next chapter, we revisit the same notions in an ASP.NET Core MVC context.

Questions

Let’s look at a few practice questions:

1. How to map different HTTP requests to delegates with Minimal APIs?
2. Can we use middleware with Minimal APIs?
3. Can you name at least two binding sources that minimal APIs supports?
4. What is the difference between using the `Results` and `TypedResults` classes?
5. What is the purpose of endpoint filters?

Further reading

Here are some links to build on what we have learned in the chapter:

- Minimal APIs quick reference: https://adpg.link/S47i
- Problem Details for HTTP APIs (RFC7807): https://adpg.link/1hpM
- FluentValidation.AspNetCore.Http: https://adpg.link/sRtU

Answers

1. Minimal APIs provide extension methods such as `MapGet`, `MapPost`, `MapPut`, and `MapDelete` to configure the HTTP pipeline and map specific HTTP requests to delegates.
2. Yes, we can use middleware with Minimal APIs, just like any other ASP.NET Core application.
3. Minimal APIs support various binding sources, including `Route`, `Query`, `Header`, `Body`, `Form`, and `Services`.
4. The methods in the `Results` class return `IResult`, while those in `TypedResults` return a typed implementation of the `IResult` interface. This difference is significant because the API Explorer can automatically discover the API contract from the typed results (`TypedResults` methods) but not from the generic `IResult` interface (`Results` methods).
5. Endpoint filters allow encapsulation and reuse of cross-cutting logic across endpoints. For example, they're helpful for input validation, logging, exception handling, and promoting code reusability.

6 Model-View-Controller

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter delves into the Model-View-Controller (MVC) design pattern, a cornerstone of modern software architecture that intuitively structures your code around entities. MVC is perfect for CRUD operations or to tap into the advanced features unavailable in Minimal APIs. The MVC pattern partitions your application into three interrelated parts: Models, Views, and Controllers.

- **Models**, which represent our data and business logic.
- **Views**, which are the user-facing components.
- **Controllers**, that act as intermediaries, mediating the interaction between Models and Views.

With its emphasis on the separation of concerns, the MVC pattern is a proven pattern for creating scalable and robust web applications. In the context of ASP.NET Core, MVC has provided a practical approach to building applications efficiently for years. While we discussed REST APIs in *Chapter 4*, this chapter provides insight into how to use MVC to create REST APIs. We also address using Data Transfer Objects (DTOs) within this framework.In this chapter, we cover the following topics:

- The Model-View-Controller design pattern
- Using MVC with DTOs

Our ultimate goal is clean, maintainable, and scalable code; the ASP.NET Core MVC framework is a favored tool for achieving this. Let's dive in!

The Model View Controller design pattern

Now that we have explored the basics of REST and Minimal APIs, it is time to explore the MVC pattern to build ASP.NET Core REST APIs.Model-View-Controller (MVC) is a design pattern commonly used in web development. It has a long history of building REST APIs in ASP.NET and is widely used and praised by many.This pattern divides an application into three interconnected components: the Model, the View, and the Controller. A View in MVC formerly represented a user interface. However, in our case, the View is a data contract that reflects the REST API's data-oriented nature.

> Dividing responsibilities this way aligns with the **Single Responsibility Principle** (**SRP**) explored in *Chapter 3, Architectural Principles*. However, this is not the only way to build REST APIs with ASP.NET Core, as we saw in *Chapter 5*, *Minimal APIs*.
>
> > The new minimal API model mixed with the Request-EndPoint-Response (REPR) pattern can make building REST APIs leaner. We cover that pattern in *Chapter 18, Request-EndPoint-Response (REPR)*. We could see REPR as what ASP.NET Core Razor Pages are to page-oriented web applications, but for REST APIs.

We often design MVC applications around entities, and each entity has a controller that orchestrates its endpoints. We called those CRUD controllers. However, you can design your controller to fit your needs.In the past few decades, the number of REST APIs just exploded to a gazillion; everybody builds APIs nowadays, not because people follow the trend but based on good reasons. REST APIs have fundamentally transformed how systems communicate, offering various benefits that make them indispensable in modern software architecture. Here are a few key factors that contribute to their widespread appeal:

- **Data Efficiency**: REST APIs promote efficient data sharing across different systems, fostering seamless interconnectivity.
- **Universal Communication**: REST APIs leverage universally recognized data formats like JSON or XML, ensuring broad compatibility and interoperability.
- **Backend Centralization**: REST APIs enable the backend to serve as a centralized hub, supporting multiple frontend platforms, including mobile, desktop, and web applications.
- **Layered Backends**: REST APIs facilitate the stratification of backends, allowing for the creation of foundational, low-level APIs that provide basic functionalities. These, in turn, can be consumed by higher-level, product-centric APIs that offer specialized capabilities, thus promoting a flexible and modular backend architecture.
- **Security Measures**: REST APIs can function as gateways, providing security measures to protect downstream systems and ensuring data access is appropriately regulated—a good example of layering APIs.
- **Encapsulation**: REST APIs allow for the encapsulation of specific units of logic into reusable, independent modules, often leading to cleaner, more maintainable code.
- **Scalability**: due to their stateless nature, REST APIs are easier to scale up to accommodate increasing loads.

These advantages greatly facilitate the reuse of backend systems across various user interfaces or even other backend services. Consider, for instance, a typical mobile application that needs to support iOS, Android, and web platforms. By utilizing a shared backend through REST APIs, development teams can streamline their efforts, saving significant time and cost. This shared backend approach ensures consistency across platforms while reducing the complexity of maintaining multiple codebases.

> We explore different such patterns in *Chapter 19*, *Introduction to Microservices Architecture*.

Goal

In the context of REST APIs, the MVC pattern aims to streamline the process of managing an entity by breaking it down into three separate, interacting components. Rather than struggling with large, bloated blocks of code that are hard to test, developers work with smaller units that enhance maintainability and promote efficient testing. This compartmentalization results in small, manageable pieces of functionality that are simpler to maintain and test.

Design

MVC divides the application into three distinct parts, where each has a single responsibility:

- **Model**: The model represents the data and business logic we are modeling.
- **View**: The view represents what the user sees. In the context of REST APIs, that usually is a serialized data structure.
- **Controller**: The controller represents a key component of MVC. It orchestrates the flow between the client request and the server response. The primary role of the controller is to act as an HTTP bridge. Essentially, the controller facilitates the communication in and out of the system.

> The code of a controller should remain minimalistic and not contain complex logic, serving as a thin layer between the clients and the domain.
>
> We explore alternative points of view in *Chapter 14*, *Layering and Clean Architecture*.

Here is a diagram that represents the MVC flow of a REST API:

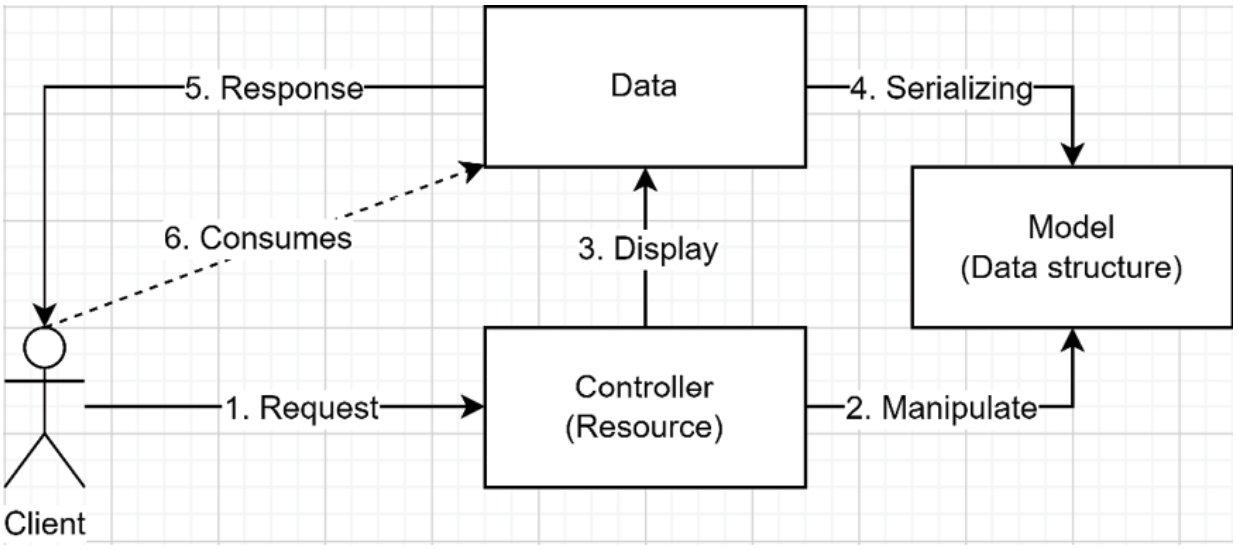


Figure 6.1: Workflow of a REST API using MVC

In the preceding diagram, we send the model directly to the client. In most scenarios, this is not ideal. We generally prefer sending only the necessary data portion, formatted according to our requirements. We can design robust API contracts by leveraging the Data Transfer Object (DTO) pattern to achieve that. But before we delve into that, let's first explore the basics of ASP.NET Core MVC.

Anatomy of ASP.NET Core web APIs

There are many ways to create a REST API project in .NET, including the `dotnet new webapi` CLI command, also available from Visual Studio's UI. Next, we explore a few pieces of the MVC framework, starting with the entry point.

The entry point

The first piece is the entry point: the `Program.cs` file. Since .NET 6, there is no more `Startup` class by default, and the compiler autogenerates the `Program` class. As explored in the previous chapter, using the minimal hosting model leads to a simplified `Program.cs` file with less boilerplate code.Here is an example:

```
using Shared;
using System.Text.Json.Serialization;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddCustomerRepository();
builder.Services
    .AddControllers()
    .AddJsonOptions(options => options
        .JsonSerializerOptions
        .Converters
        .Add(new JsonStringEnumConverter())
    )
;
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
var app = builder.Build();
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseDarkSwaggerUI();
}
app.MapControllers();
app.InitializeSharedDataStore();
app.Run();
```

In the preceding `Program.cs` file, the highlighted lines identify the minimum code required to enable ASP.NET Core MVC. The rest is very similar to the Minimal APIs code.

Directory structure

The default directory structure contains a `Controllers` folder to host the controllers. On top of that, we can create a `Models` folder to store your model classes or use any other structure.

> While controllers are typically housed in the `Controllers` directory for organizational purposes, this convention is more for the benefit of developers than a strict requirement. ASP.NET Core is indifferent to the file's location, offering us the flexibility to structure our project as we see fit.
>
> *Section 4*, *Applications Patterns*, explores many ways of designing applications.

Next, we look at the central part of this pattern—the controllers.

Controller

The easiest way to create a controller is to create a class inheriting from `ControllerBase`. However, while `ControllerBase` adds many utility methods, the only requirement is to decorate the controller class with the `[ApiController]` attribute.

> By convention, we write the controller's name in its plural form and suffix it with `Controller`. For example, if the controller relates to the `Employee` entity, we'd name it `EmployeesController`, which, by default, leads to an excellent URL pattern that is easy to understand:

- Get all employees: `/employees`
- Get a specific employee: `/employees/{id}`
- And so on.

Once we have a controller class, we must add actions. Actions are public methods that represent the operations that a client can perform. Each action represents an HTTP endpoint.More precisely, the following defines a controller:

- A controller exposes one or more actions.
- An action can take zero or more input parameters.
- An action can return zero or one output value.
- The action is what handles the HTTP request.

> We should group cohesive actions under the same controller, thus creating a loosely coupled unit.

For example, the following represents the `SomeController` class containing a single `Get` action:

```
[Route("api/[controller]")]
[ApiController]
public class SomeController : ControllerBase
{
    [HttpGet]
    public IActionResult Get() => Ok();
}
```

The preceding `Get` method (action) returns an empty `200 OK` response to the client. We can reach the endpoint at the `/api/some` URI. From there, we can add more actions.

> The `ControllerBase` class gives us access to most of the same utility methods as we had with the Minimal APIs `TypedResults` class.

Next, we look at returning value.

Returning values

Building a REST API aims to return data to clients and execute remote operations. Most of the plumbing is done for us by the ASP.NET Core code, including serialization.

> Most of the ASP.NET Core pipeline is customizable, which is out of the scope of this chapter.

Before returning values, let's look at a few valuable helper methods provided by the `ControllerBase` class:

Method	Description
`StatusCode`	Produces an empty response with the specified status code. We can optionally include a second argument to serialize in the response body.
`Ok`	Produces a `200 OK` response, indicating the operation was successful. We can optionally include a second argument to serialize in the response body.
`Created`	Produces a `201 Created` response, indicating the system created the entity. We can optionally specify the location where to read the entity and the entity itself as arguments. The `CreatedAtAction` and `CreatedAtRoute` methods give us options to compose the location value.
`NoContent`	Produces an empty `204 No Content` response.
`NotFound`	Produces a `404 Not Found` response, indicating the resource was not found.
`BadRequest`	Produces a `400 Bad Request` response, indicating an issue with the client request, often a validation error.
`Redirect`	Produces a `302 Found` response, accepting the `Location` URL as an argument. Different `Redirect*` methods produce `301 Moved Permanently`, `307 Temporary Redirect`, and `308 Permanent Redirect` responses instead.
`Accepted`	Produces a `202 Accepted` response, indicating the beginning of an asynchronous process. We can optionally specify the location the client can query to learn about the status of the asynchronous operation. We can also optionally specify an object to serialize in the response body.

	The `AcceptedAtAction` and `AcceptedAtRoute` methods give us options to compose the location value.
`Conflict`	Produces a `409 Conflict` response, indicating a conflict occurred when processing the request, often a concurrency error.

Table 6.1: a subset of the ControllerBase methods producing an IActionResult.

> Other methods in the `ControllerBase` class are self-discoverable using IntelliSense (code completion) or in the official documentation. Most, if not all, of what we covered in *Chapter 5*, *Minimal APIs*, is also available to controllers.

The advantage of using a helper method is leveraging the ASP.NET Core MVC mechanism, making our life easier. However, you could manually manage the HTTP response using lower-level APIs like `HttpContext` or create custom classes that implement the `IActionResult` interface to hook your custom response classes into the MVC pipeline.Now let's look at the multiple ways we can use to return data to the client:

Return type	Description
`void`	We can return `void` and manually manage the HTTP response using the `HttpContext` class. This is the most low-level and complex way.
`TModel`	We can directly return the model, which ASP.NET Core will serialize. The problem with this approach is that we don't control the status code, nor can we return multiple different results from the action.

`ActionResult`
 	
`IActionResult`
	We can return one of those two abstractions. The concrete result can take many forms depending on the implementation that the action method returns.
	
	However, doing this makes our API less auto-discoverable by tools like SwaggerGen.

`ActionResult<TModel>`	We can return the `TModel` directly and other results like a `NotFoundResult` or a `BadRequestResult`. This is the most flexible way that makes the API the most discoverable by the ApiExplorer.

Table 6.2: multiple ways to return data

We start with an example where the actions return an instance of the `Model` class by leveraging the `Ok` method (highlighted code):

```
using Microsoft.AspNetCore.Mvc;
namespace MVC.API.Controllers;
[Route("api/[controller]")]
[ApiController]
public class ValuesController : ControllerBase
{
    [HttpGet("IActionResult")]
    public IActionResult InterfaceAction()
        => Ok(new Model(nameof(InterfaceAction)));
    [HttpGet("ActionResult")]
    public ActionResult ClassAction()
```

```
        => Ok(new Model(nameof(ClassAction)));
    // ...
    public record class Model(string Name);
}
```

The problem with the preceding code is API discoverability. The `ApiExplorer` can't know what the endpoints return. The `ApiExplorer` describes the actions as returning `200 OK` but doesn't know about the `Model` class.To overcome this limitation, we can decorate our actions with the `ProducesResponseType` attribute, effectively circumventing the limitation as shown below:

```
[ProducesResponseType(typeof(Model), StatusCodes.Status200OK)]
public IActionResult InterfaceAction() { ... }
```

In the preceding code, we specify the return type as the first argument and the status code as the second. Using the constants of the `StatusCodes` class is a convenient way to reference standard status codes. We can decorate each action with multiple `ProducesResponseType` attributes to define alternate states, such as `404` and `400`.

> With ASP.NET Core MVC, we can also define conventions that apply broad rules to our controllers, allowing us to define those conventions once and reuse them throughout our application. I left a link in the *Further reading* section.

Next, We explore how we can return a `Model` instance directly. The `ApiExplorer` can discover the return value of the method this way, so we do not need to use the `ProducesResponseType` attribute:

```
[HttpGet("DirectModel")]
public Model DirectModel()
    => new Model(nameof(DirectModel));
```

Next, thanks to **class conversion operators** (see *Appendix A* for more info), we can do the same with `ActionResult<T>`, like this:

```
[HttpGet("ActionResultT")]
public ActionResult<Model> ActionResultT()
    => new Model(nameof(ActionResultT));
```

The main benefit of using `ActionResult<T>` is to return other types of results. Here is an example showing this where the method returns either `Ok` or `NotFound`:

```
[HttpGet("MultipleResults")]
public ActionResult<Model> MultipleResults()
{
    var condition = Random.Shared
        .GetItems(new[] { true, false }, 1)
        .First();
    return condition
        ? Ok(new Model(nameof(MultipleResults)))
        : NotFound();
}
```

However, the `ApiExplorer` does not know about the `404 Not Found`, so we must document it using the `ProducesResponseType` attribute.

> We can return a `Task<T>` or a `ValueTask<T>` from the action method when the method body is asynchronous. Doing so lets you write the async/await code from the controller.
>
> > I highly recommend returning a `Task<T>` or a `ValueTask<T>` whenever possible because it allows your REST API to handle more requests using the same resources without effort. Nowadays, non-Task-based methods in libraries are infrequent, so you will most likely have little choice.

We learned multiple ways to return values from an action. The `ActionResult<T>` class is the most flexible regarding feature support. On the other hand, `IActionResult` is the most abstract one.Next, we look at routing requests to those action methods.

Attribute routing

Attribute routing maps an HTTP request to a controller action. Those attributes decorate the controllers and the actions to create the complete routes. We already used some of those attributes. Nonetheless, let's visit those attributes:

```
namespace MVC.API.Controllers.Empty;
[Route("empty/[controller]")]
[ApiController]
public class CustomersController : ControllerBase
{
    [HttpGet]
    public Task<IEnumerable<Customer>> GetAllAsync(
        ICustomerRepository customerRepository)
        => throw new NotImplementedException();
    [HttpGet("{id}")]
    public Task<ActionResult<Customer>> GetOneAsync(
        int id, ICustomerRepository customerRepository)
        => throw new NotImplementedException();
    [HttpPost]
    public Task<ActionResult> PostAsync(
        [FromBody] Customer value, ICustomerRepository customerRepository)
        => throw new NotImplementedException();
    [HttpPut("{id}")]
    public Task<ActionResult<Customer>> PutAsync(
        int id, [FromBody] Customer value,
        ICustomerRepository customerRepository)
        => throw new NotImplementedException();
    [HttpDelete("{id}")]
    public Task<ActionResult<Customer>> DeleteAsync(
        int id, ICustomerRepository customerRepository)
        => throw new NotImplementedException();
}
```

The `Route` attributes and `Http[Method]` attributes define what a user should query to reach a specific resource. The `Route` attribute allows us to define a routing pattern that applies to all HTTP methods under the decorated controller. The `Http[Method]` attributes determine the HTTP method used to reach that action method. They also offer the possibility to set an optional and additive route pattern to handle more complex routes, including specifying route parameters. Those attributes are beneficial in crafting concise and clear URLs while keeping the routing system close to the controller. All routes must be unique.Based on the code, `[Route("empty/[controller]")]` means that the actions of this controller are reachable through `empty/customers` (MVC ignores the `Controller` suffix). Then, the other attributes tell ASP.NET to map specific requests to specific methods:

Routing Attribute	HTTP Method	URL
`HttpGet`	`GET`	`empty/customers`
`HttpGet("{id}")`	`GET`	`empty/customers/{id}`
`HttpPost`	`POST`	`empty/customers`
`HttpPut("{id}")`	`PUT`	`empty/customers/{id}`
`HttpDelete("{id}")`	`DELETE`	`empty/customers/{id}`

Table 6.3: routing attributes of the example controller and their final URL

As we can see from the preceding table, we can even use the same attribute for multiple actions as long as the URL is unique. In this case, the `id` parameter is the `GET` discriminator.Next, we can use the `FromBody` attribute to tell the model binder to use the HTTP request body to get the value of that parameter. There are many of those attributes; here's a list:

Attribute	Description
`FromBody`	Binds the JSON body of the request to the parameter's type.
`FromForm`	Binds the form value that matches the name of the parameter.

`FromHeader`	Binds the HTTP header value that matches the name of the parameter.
`FromQuery`	Binds the query string value that matches the name of the parameter.
`FromRoute`	Binds the route value that matches the name of the parameter.
`FromServices`	Inject the service from the ASP.NET Core dependency container.

Table 6.4: MVC binding sources

> ASP.NET Core MVC does many implicit binding, so you don't always need to decorate all parameters with an attribute. For example, .NET injects the services we needed in the code samples, and we never used the `FromServices` attribute. Same with the `FromRoute` attribute.

Now, if we look back at `CustomersController` , the route map looks like the following (I excluded non-route-related code to improve readability):

URL	**Action/Method**
`GET empty/customers`	`GetAllAsync()`
`GET empty/customers/{id}`	`GetOneAsync(int id)`
`POST empty/customers`	`PostAsync([FromBody] Customer value)`
`PUT empty/customers/{id}`	`PutAsync(int id, [FromBody] Customer value)`
`DELETE empty/customers/{id}`	`DeleteAsync(int id)`

Table 6.5: the map between the URLs and their respective action methods

When designing a REST API, the URL leading to our endpoints should be clear and concise, making it easy for consumers to discover and learn. Hierarchically grouping our resources by responsibility (concern) and creating a cohesive URL space help achieve that goal. Consumers (a.k.a. other developers) should understand the logic behind the endpoints easily. Think about your endpoints as if you were the consumer of the REST API. I would even extend that suggestion to any API; always consider the consumers of your code to create the best possible APIs.

Conclusion

This section explored the MVC pattern, how to create controllers and action methods, and how to route requests to those actions.We could talk about MVC for the remainder of the book, but we would be missing the point. The subset of features we covered here should be enough theory to fill the gap you might have had and allow you to understand the code samples that leverage ASP.NET Core MVC.Using the MVC pattern helps us follow the SOLID principles in the following ways:

- **S**: The MVC pattern divides the rendering of a data structure into three different roles. The framework handles most of the serialization portion (the View), leaving us only two pieces to manage: the Model and the Controller.
- **O**: N/A
- **L**: N/A
- **I**: Each controller handles a subset of features and represents a smaller interface into the system. MVC makes the system easier to manage than having a single entry point for all routes, like a single controller.
- **D**: N/A

Next, we explore the **Data Transfer Object** pattern to isolate the API's model from the domain.

Using MVC with DTOs

This section explores leveraging the **Data Transfer Object** (**DTO**) pattern with the MVC framework.

> This section is the same as we explore in *Chapter 5*, *Minimal APIs*, but in the context of MVC. Moreover, the two code projects are part of the same Visual Studio solution for convenience, allowing you to compare the two implementations.

Goal

As a reminder, DTOs aim to *control the inputs and outputs of an endpoint* by decoupling the API contract from the application's inner workings. DTOs empower us to define our APIs without thinking about the underlying data structures, leaving us to craft our REST APIs how we want.

We discuss REST APIs and DTOs more in-depth in *Chapter 4, REST APIs*.

Other possible objectives are to save bandwidth by limiting the amount of information the API transmits, flattening the data structure, or adding API features that cross multiple entities.

Design

Let's start by analyzing a diagram that expands MVC to work with DTOs:

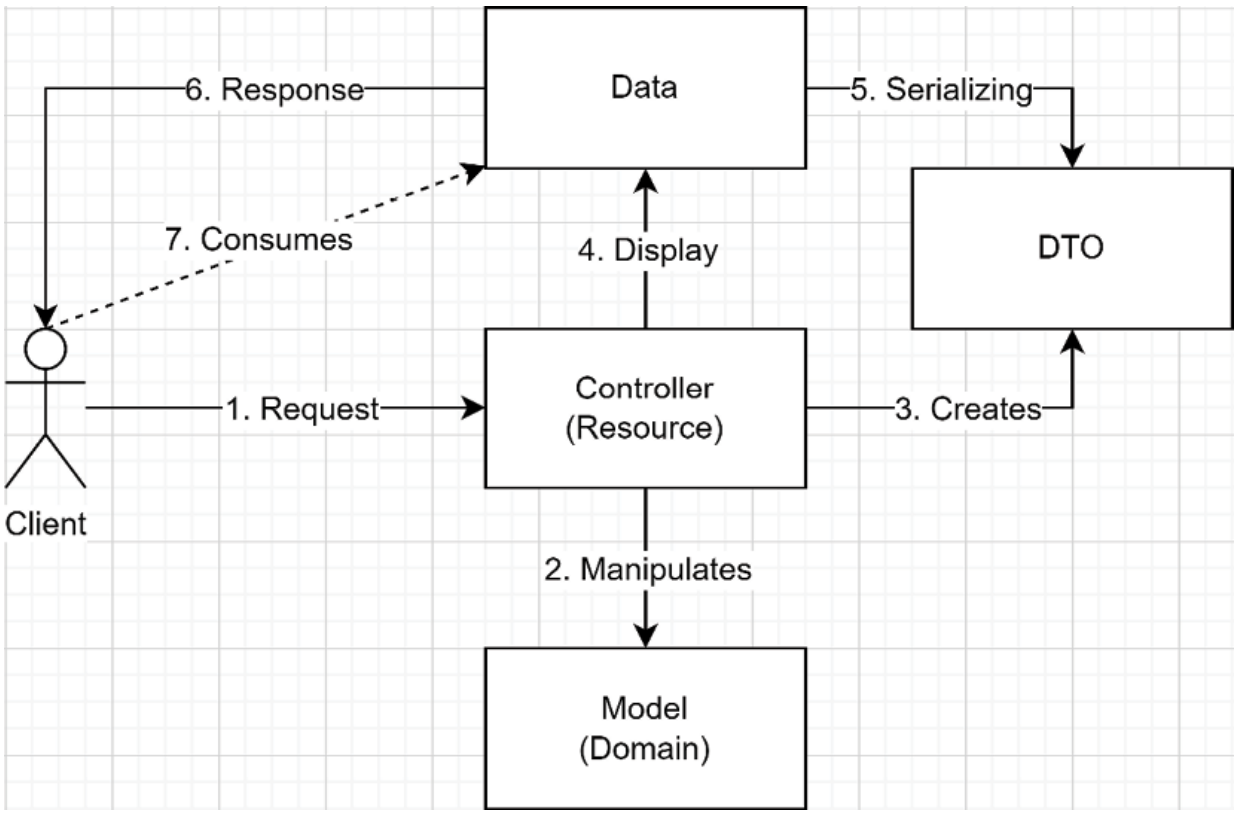


Figure 6.2: MVC workflow with a DTO

DTOs allow the decoupling of the domain from the view (data) and empower us to manage the inputs and outputs of our REST APIs independently from the domain. The controller still manipulates the domain model but returns a serialized DTO instead.

Project – MVC API

This code sample is the same as in the previous chapter but uses the MVC framework instead of Minimal APIs.**Context**: we must build an application to manage customers and contracts. We must track the state of each contract and have a primary contact in case the business needs to contact the customer. Finally, we must display the number of contracts and the number of opened contracts for each customer on a dashboard.As a reminder, the model is the following:

```
namespace Shared.Models;
public record class Customer(
    int Id,
    string Name,
    List<Contract> Contracts
);
public record class Contract(
    int Id,
    string Name,
    string Description,
    WorkStatus Status,
    ContactInformation PrimaryContact
);
public record class WorkStatus(int TotalWork, int WorkDone)
{
    public WorkState State =>
        WorkDone == 0 ? WorkState.New :
        WorkDone == TotalWork ? WorkState.Completed :
        WorkState.InProgress;
}
public record class ContactInformation(
    string FirstName,
    string LastName,
    string Email
);
public enum WorkState
{
    New,
    InProgress,
    Completed
}
```

The preceding code is straightforward. The only piece of logic is the `WorkStatus.State` property that returns `WorkState.New` when the work has not yet started on that contract, `WorkState.Completed` when all the work is completed, or `WorkState.InProgress` otherwise.The controllers leverage the `ICustomerRepository` interface to simulate database operations. The implementation is unimportant. It uses a `List<Customer>` as the database. Here's the interface that allows querying and updating the data:

```
using Shared.Models;
namespace Shared.Data;
public interface ICustomerRepository
{
    Task<IEnumerable<Customer>> AllAsync(
        CancellationToken cancellationToken);
    Task<Customer> CreateAsync(
        Customer customer,
        CancellationToken cancellationToken);
    Task<Customer?> DeleteAsync(
        int customerId,
        CancellationToken cancellationToken);
    Task<Customer?> FindAsync(
        int customerId,
        CancellationToken cancellationToken);
    Task<Customer?> UpdateAsync(
        Customer customer,
        CancellationToken cancellationToken);
}
```

Now that we know about the underlying foundation, we explore a CRUD controller that does not leverage DTOs.

Raw CRUD Controller

Many issues can arise if we create a CRUD controller to manage the customers directly (see `RawCustomersController.cs`). First, a little mistake from the client could erase several data points. For example, if the client forgets to send the contracts during a `PUT` operation, that would delete all the contracts associated with that customer. Here's the controller code:

```
// PUT raw/customers/1
[HttpPut("{id}")]
public async Task<ActionResult<Customer>> PutAsync(
    int id,
    [FromBody] Customer value,
    ICustomerRepository customerRepository)
{
    var customer = await customerRepository.UpdateAsync(
        value,
        HttpContext.RequestAborted);
    if (customer == null)
    {
        return NotFound();
    }
    return customer;
}
```

The highlighted code represents the customer update. So to mistakenly remove all contracts, a client could send the following HTTP request (from the `MVC.API.http` file):

```
PUT {{MVC.API.BaseAddress}}/customers/1
Content-Type: application/json
{
  "id": 1,
  "name": "Some new name",
  "contracts": []
}
```

That request would result in the following response entity:

```
{
  "id": 1,
  "name": "Some new name",
  "contracts": []
}
```

Previously, however, that customer had contracts (seeded when we started the application). Here's the original data:

```
{
  "id": 1,
  "name": "Jonny Boy Inc.",
  "contracts": [
    {
      "id": 1,
      "name": "First contract",
      "description": "This is the first contract.",
      "status": {
        "totalWork": 100,
        "workDone": 100,
        "state": "Completed"
      },
      "primaryContact": {
        "firstName": "John",
        "lastName": "Doe",
        "email": "john.doe@jonnyboy.com"
      }
    },
    {
      "id": 2,
      "name": "Some other contract",
      "description": "This is another contract.",
      "status": {
        "totalWork": 100,
        "workDone": 25,
        "state": "InProgress"
      },
      "primaryContact": {
        "firstName": "Jane",
```

```
        "lastName": "Doe",
        "email": "jane.doe@jonnyboy.com"
      }
    }
  ]
}
```

As we can see, by exposing our entities directly, we are giving a lot of power to the consumers of our API. Another issue with this design is the dashboard. The user interface would have to calculate the statistics about the contracts. Moreover, if we implement paging the contracts over time, the user interface could become increasingly complex and even overquery the database, hindering our performance.

I implemented the entire API, which is available on GitHub but without UI.

Next, we explore how we can fix those two use cases using DTOs.

DTO controller

To solve our problems, we reimplement the controller using DTOs. To make it easier to follow along, here are all the DTOs as a reference:

```
namespace Shared.DTO;
public record class ContractDetails(
    int Id,
    string Name,
    string Description,
    int StatusTotalWork,
    int StatusWorkDone,
    string StatusWorkState,
    string PrimaryContactFirstName,
    string PrimaryContactLastName,
    string PrimaryContactEmail
);
public record class CustomerDetails(
    int Id,
    string Name,
    IEnumerable<ContractDetails> Contracts
);
public record class CustomerSummary(
    int Id,
    string Name,
    int TotalNumberOfContracts,
    int NumberOfOpenContracts
);
public record class CreateCustomer(string Name);
public record class UpdateCustomer(string Name);
```

First, let’s fix our update problem, starting with the reimplementation of the update endpoint leveraging DTOs (see the `DTOCustomersController.cs` file):

```
// PUT dto/customers/1
[HttpPut("{customerId}")]
public async Task<ActionResult<CustomerDetails>> PutAsync(
        int customerId,
        [FromBody] UpdateCustomer input,
        ICustomerRepository customerRepository)
{
    // Get the customer
    var customer = await customerRepository.FindAsync(
        customerId,
        HttpContext.RequestAborted
    );
    if (customer == null)
    {
        return NotFound();
    }
    // Update the customer's name using the UpdateCustomer DTO
```

```
    var updatedCustomer = await customerRepository.UpdateAsync(
        customer with { Name = input.Name },
        HttpContext.RequestAborted
    );
    if (updatedCustomer == null)
    {
        return Conflict();
    }
    // Map the updated customer to a CustomerDetails DTO
    var dto = MapCustomerToCustomerDetails(updatedCustomer);
    // Return the DTO
    return dto;
}
```

In the preceding code, the main differences are (highlighted):

- The request body is now bound to the `UpdateCustomer` class instead of the `Customer` itself.
- The action method returns an instance of the `CustomerDetails` class instead of the `Customer` itself.

However, we can see more code in our controller action than before. That's because the controller now handles the data changes instead of the clients. The action now does:

1. Load the data from the database.
2. Ensure the entity exists.
3. Use the input DTO to update the data, limiting the clients to a subset of properties.
4. Proceed with the update.
5. Ensure the entity still exists (handles conflicts).
6. Copy the Customer into the output DTO and return it.

By doing this, we now control what the clients can do when they send a `PUT` request through the input DTO (`UpdateCustomer`). Moreover, we encapsulated the logic to calculate the statistics on the server. We hid the computation behind the output DTO (`CustomerDetails`), which lowers the complexity of our user interface and allows us to improve the performance without impacting any of our clients (loose coupling).Furthermore, we now use the `customerId` parameter.If we send the same HTTP request as before, which sends more data than we accept, only the customer's name will change. On top of that, we get all the data we need to display the customer's statistics. Here's a response example:

```
{
  "id": 1,
  "name": "Some new name",
  "contracts": [
    {
      "id": 1,
      "name": "First contract",
      "description": "This is the first contract.",
      "statusTotalWork": 100,
      "statusWorkDone": 100,
      "statusWorkState": "Completed",
      "primaryContactFirstName": "John",
      "primaryContactLastName": "Doe",
      "primaryContactEmail": "john.doe@jonnyboy.com"
    },
    {
      "id": 2,
      "name": "Some other contract",
      "description": "This is another contract.",
      "statusTotalWork": 100,
      "statusWorkDone": 25,
      "statusWorkState": "InProgress",
      "primaryContactFirstName": "Jane",
      "primaryContactLastName": "Doe",
      "primaryContactEmail": "jane.doe@jonnyboy.com"
    }
  ]
}
```

As we can see from the preceding response, only the customer's name changed, but we now received the `statusWorkDone` and `statusTotalWork` fields. Lastly, we flattened the data structure.

> DTOs are a great resource to flatten data structures, but you don't have to. You must always design your systems, including DTOs and data contracts, for specific use cases.

As for the dashboard, the "get all customers" endpoint achieves this by doing something similar. It outputs a collection of `CustomerSummary` objects instead of the customers themselves. In this case, the controller executes the calculations and copies the entity's relevant properties to the DTO. Here's the code:

```
// GET: dto/customers
[HttpGet]
public async Task<IEnumerable<CustomerSummary>> GetAllAsync(
    ICustomerRepository customerRepository)
{
    // Get all customers
    var customers = await customerRepository.AllAsync(
        HttpContext.RequestAborted
    );
    // Map customers to CustomerSummary DTOs
    var customersSummary = customers
        .Select(customer => new CustomerSummary(
            Id: customer.Id,
            Name: customer.Name,
            TotalNumberOfContracts: customer.Contracts.Count,
            NumberOfOpenContracts: customer.Contracts.Count(x => x.Status.State != WorkState.Com
        ))
    ;
    // Return the DTOs
    return customersSummary;
}
```

In the preceding code, the action method:

1. Read the entities
2. Create the DTOs and calculate the number of open contracts.
3. Return the DTOs.

As simple as that, we now encapsulated the computation on the server.

> You should optimize such code based on your real-life data source. In this case, a `static List<T>` is low latency. However, querying the whole database to get a count can become a bottleneck.

Calling the endpoint results in the following:

```
[
  {
    "id": 1,
    "name": "Some new name",
    "totalNumberOfContracts": 2,
    "numberOfOpenContracts": 1
  },
  {
    "id": 2,
    "name": "Some mega-corporation",
    "totalNumberOfContracts": 1,
    "numberOfOpenContracts": 1
  }
]
```

It is now super easy to build our dashboard. We can query that endpoint once and display the data in the UI. The UI offloaded the calculation to the backend.

> User interfaces tend to be more complex than APIs because they are stateful. As such, offloading as much complexity to the backend helps. You can use a Backend-for-frontend (BFF) to help with this task. We

explore ways to layer APIs, including the BFF pattern in *Chapter 19*, *Introduction to Microservices Architecture*.

Lastly, you can play with the API using the HTTP requests in the `MVC.API.DTO.http` file. I implemented all the endpoints using a similar technique. If your controller logic becomes too complex, it is good practice to encapsulate it into other classes. We explore many techniques to organize application code in *Section 4*: *Applications patterns*.

Conclusion

A data transfer object allows us to design an API endpoint with a specific data contract (input and output) instead of exposing the domain or data model. This separation between the presentation and the domain is a crucial element that leads to having multiple independent components instead of a bigger, more fragile one. Using DTOs to control the inputs and outputs gives us more control over what the clients can do or receive.Using the data transfer object pattern helps us follow the SOLID principles in the following ways:

- **S**: A DTO adds clear boundaries between the domain model and the API contract. Moreover, having an input and an output DTO help further separate the responsibilities.
- **O**: N/A
- **L**: N/A
- **I**: A DTO is a small, specifically crafted data contract (abstraction) with a clear purpose in the API contract.
- **D**: Due to those smaller interfaces (ISP), DTOs allow changing the implementation details of the endpoint without affecting the clients because they depend only on the API contract (an abstraction).

You have learned DTOs' added value, their role in an API contract, and the ASP.NET Core MVC framework.

Summary

This chapter explored the Model-View-Controller (MVC) design pattern, a well-established framework in the ASP.NET ecosystem that offers more advanced features than its newer Minimal APIs counterpart. Minimal APIs are not competing against MVC; we can use them together. The MVC pattern emphasizes the separation of concerns, making it a proven pattern for creating maintainable, scalable, and robust web applications. We broke down the MVC pattern into its three core components: Models, Views, and Controllers. Models represent data and business logic, Views are user-facing components (serialized data structures), and Controllers act as intermediaries, mediating the interaction between Models and Views. We also discussed using Data Transfer Objects (DTOs) to package data in the format we need, providing many benefits, including flexibility, efficiency, encapsulation, and improved performance. DTOs are a crucial part of the API contract.Now that we have explored principles and methodologies, it is time to continue our learning and tackle more design patterns and features. The following two chapters explore our first Gang of Four (GoF) design patterns and deep dive into ASP.NET Core dependency injection (DI). All of this will help us to continue on the path we started: to learn the tools to design better software.

Questions

Let’s look at a few practice questions:

1. What are the three components of the MVC design pattern?
2. What is the role of a Controller in the MVC pattern?
3. What are Data Transfer Objects (DTOs), and why are they important?
4. How does the MVC pattern contribute to the maintainability of an application?
5. How does attribute routing work in MVC?

Further reading

Here are some links to build on what we have learned in the chapter:

- Using web API conventions: https://adpg.link/ioKV
- Getting started with Swashbuckle and ASP.NET Core: https://adpg.link/ETja

Answers

1. The three components of the MVC design pattern are Models, Views, and Controllers.
2. In the MVC pattern, a Controller acts as an intermediary, mediating the interaction between Models and Views.
3. We use Data Transfer Objects (DTOs) to package data into a format that provides many benefits, including efficient data sharing, encapsulation, and improved maintainability.
4. The MVC pattern contributes to the maintainability of an application by separating concerns. Each component (Model, View, Controller) has a specific role and responsibility, making the code easier to manage, test, and extend. This separation allows changes to one component to have minimal impact on the others.
5. Attribute routing in MVC maps an HTTP request to a controller action. These attributes decorate the controllers and the actions to create the complete routes.

7 Strategy, Abstract Factory, and Singleton Design Patterns

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter explores object creation using a few classic, simple, and yet powerful design patterns from the **Gang of Four** (**GoF**). These patterns allow developers to encapsulate and reuse behaviors, centralize object creation, add flexibility to our designs, or control object lifetime. Moreover, you will most likely use some of them in all software you build directly or indirectly in the future.

GoF

> Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides are the authors of *Design Patterns: Elements of Reusable Object-Oriented Software* (1994), known as the **Gang of Four** (**GoF**). In that book, they introduce 23 design patterns, some of which we revisit in this book.

Why are they that important? Because they are the building blocks of robust object composition and help create flexibility and reliability. Moreover, in *Chapter 8*, *Dependency Injection*, we make those patterns even more powerful!But first things first. In this chapter, we cover the following topics:

- The Strategy design pattern
- The Abstract Factory design pattern
- The Singleton design pattern

The Strategy design pattern

The Strategy pattern is a behavioral design pattern that allows us to change object behaviors at runtime. We can also use this pattern to compose complex object trees and rely on it to follow the **Open/Closed Principle** (**OCP**) without much effort. Moreover, it plays a significant role in the *composition over inheritance* way of thinking. In this chapter, we focus on the behavioral part of the Strategy pattern. The next chapter covers how to use the Strategy pattern to compose systems dynamically.

Goal

The Strategy pattern aims to extract an algorithm (a strategy) from the host class needing it (the context or consumer). That allows the consumer to decide on the strategy (algorithm) to use at runtime.For example, we could design a system that fetches data from two different types of databases. Then we could apply the same logic to that data and use the same user interface to display it. To achieve this, we could use the Strategy pattern to create two strategies, one named `FetchDataFromSql` and the other `FetchDataFromCosmosDb`. Then we could plug the strategy we need at runtime in the `context` class. That way, when the consumer calls the `context`, the `context` does not need to know where the data comes from, how it is fetched, or what strategy is in use; it only gets what it needs to work, delegating the fetching responsibility to an abstracted strategy (an interface).

Design

Before any further explanation, let's take a look at the following class diagram:

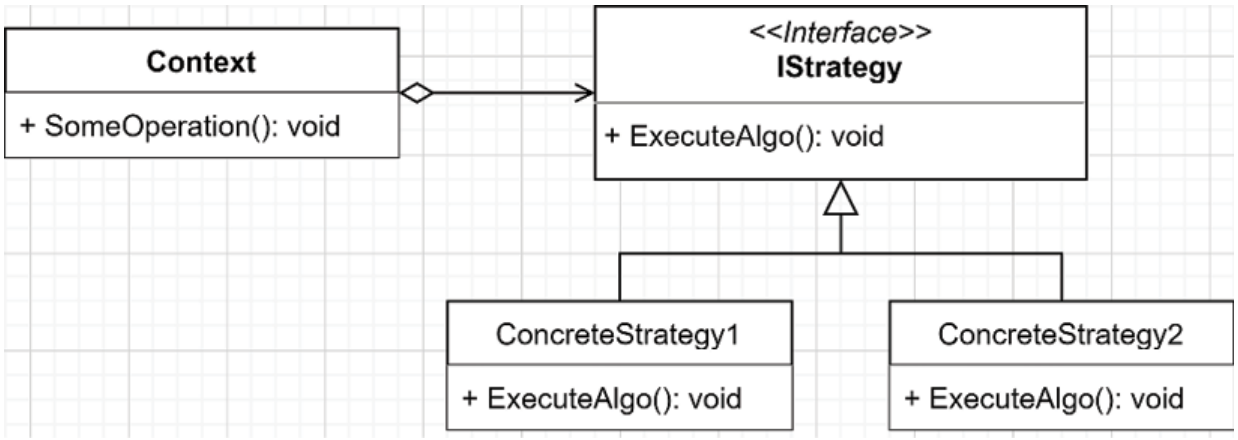


Figure 7.1: Strategy pattern class diagram

Based on the preceding diagram, the building blocks of the Strategy pattern are the following:

- `Context` is a class that depends on the `IStrategy` interface and leverages an implementation of the `IStrategy` interface to execute the `ExecuteAlgo` method.
- `IStrategy` is an interface defining the strategy API.
- `ConcreteStrategy1` and `ConcreteStrategy2` represent one or more different concrete implementations of the `IStrategy` interface.

In the following diagram, we explore what happens at runtime. The actor represents any code consuming the `Context` object.

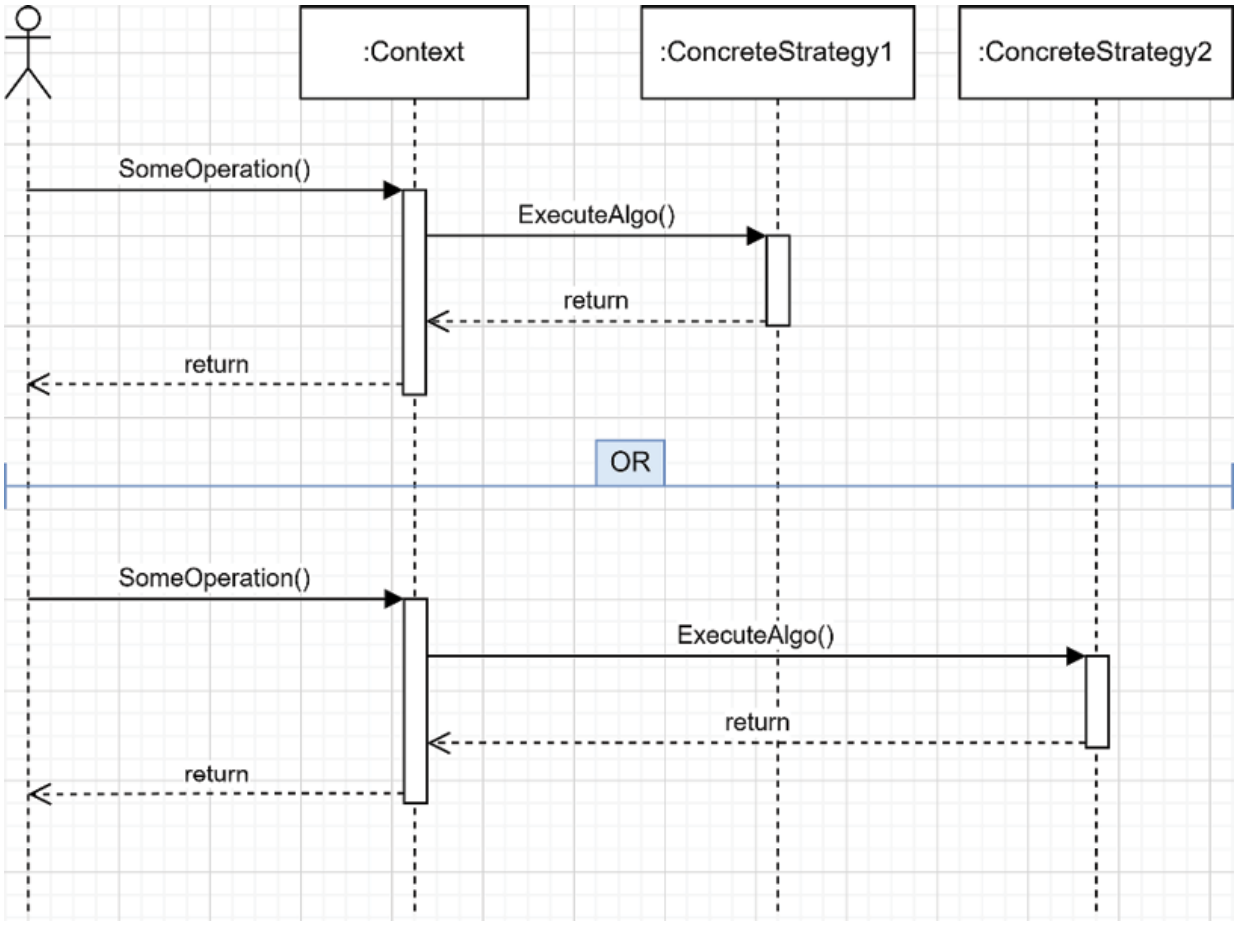


Figure 7.2: Strategy pattern sequence diagram

When the consumer calls the `Context.SomeOperation()` method, it does not know which implementation is executed, which is an essential part of this pattern. The `Context` class should not be aware of the strategy it uses either. It should run the strategy through the interface, unaware of the implementation. That is the strength of the Strategy pattern: it abstracts the implementation away from both the `Context` class and its consumers. Because of that, we can change the strategy during either the object creation or at runtime without the object knowing, changing its behavior on the fly.

Note

> We could even generalize that last sentence and extend it to any interface. Depending on an interface breaks the ties between the consumer and the implementation by relying on that abstraction instead.

Project – Strategy

Context: We want to sort a collection differently, eventually even using different sort algorithms (out of the scope of the example but possible). Initially, we want to support sorting the elements of any collection in ascending or descending order.To achieve this, we need to implement the following building blocks:

- The `Context` is the `SortableCollection` class.
- The `IStrategy` is the `ISortStrategy` interface.
- The concrete strategies are:

1. `SortAscendingStrategy`

2. `SortDescendingStrategy`

The consumer is a small REST API that allows the user to change the strategy, sort the collection, and display the items. Let's start with the `ISortStrategy` interface:

```
public interface ISortStrategy
{
    IOrderedEnumerable<string> Sort(IEnumerable<string> input);
}
```

That interface contains only one method that expects a string collection as input and returns an ordered string collection. Now let's inspect the two implementations:

```
public class SortAscendingStrategy : ISortStrategy
{
    public IOrderedEnumerable<string> Sort(IEnumerable<string> input)
        => input.OrderBy(x => x);
}
public class SortDescendingStrategy : ISortStrategy
{
    public IOrderedEnumerable<string> Sort(IEnumerable<string> input)
        => input.OrderByDescending(x => x);
}
```

Both implementations are super simple, using **Language Integrated Query** (**LINQ**) to sort the input and return the result directly.

Tip

When using expression-bodied methods, please ensure you do not make the method harder to read for your colleagues (or future you) by creating very complex one-liners. Writing multiple lines often makes the code easier to read.

The next building block to inspect is the `SortableCollection` class. It is composed of multiple string items (the `Items` property) and can sort them using an `ISortStrategy`. On top of that, it implements the `IEnumerable<string>` interface through its `Items` property, making it iterable. Here's the class:

```
using System.Collections;
using System.Collections.Immutable;
namespace MySortingMachine;
public sealed class SortableCollection : IEnumerable<string>
{
    private ISortStrategy _sortStrategy;
    private ImmutableArray<string> _items;
    public IEnumerable<string> Items => _items;
    public SortableCollection(IEnumerable<string> items)
    {
        _items = items.ToImmutableArray();
        _sortStrategy = new SortAscendingStrategy();
    }
    public void SetSortStrategy(ISortStrategy strategy)
        => _sortStrategy = strategy;
    public void Sort()
    {
        _items = _sortStrategy
            .Sort(Items)
            .ToImmutableArray()
        ;
    }
    public IEnumerator<string> GetEnumerator()
        => Items.GetEnumerator();
    IEnumerator IEnumerable.GetEnumerator()
        => ((IEnumerable)Items).GetEnumerator();
}
```

The `SortableCollection` class is the most complex one so far, so let's take a more in-depth look:

- The `_sortStrategy` field references the algorithm: an `ISortStrategy` implementation.
- The `_items` field references the strings themselves.
- The `Items` property exposes the strings to the consumers of the class.
- The constructor initializes the `Items` property using the `items` parameter and sets the default sorting strategy.
- The `SetSortStrategy` method allows consumers to change the strategy at runtime.
- The `Sort` method uses the `_sortStrategy` field to sort the items.
- The two `GetEnumerator` methods represent the implementation of the `IEnumerable<string>` interface and make the class enumerable through the `Items` property.

With that code, we can see the Strategy pattern in action. The `_sortStrategy` field represents the current algorithm, respecting an `ISortStrategy` contract, which is updatable at runtime using the `SetSortStrategy` method. The `Sort` method delegates the work to the `ISortStrategy` implementation (the concrete strategy). Therefore, changing the value of the `_sortStrategy` field leads to a change of behavior of the `Sort` method, making this pattern very powerful yet simple. The highlighted code represents this pattern.

> The `_items` field is an `ImmutableArray<string>`, which makes changing the list impossible from the outside. For example, a consumer cannot pass a `List<string>` to the constructor, then change it later. Immutability has many advantages.

Let's experiment with this by looking at the `Consumer.API` project a REST API application that uses the previous code. Next is a breakdown of the `Program.cs` file:

```
using MySortingMachine;
SortableCollection data = new(new[] {
    "Lorem", "ipsum", "dolor", "sit", "amet." });
```

The `data` member is the context, our sortable collection of items. Next, we look at some boilerplate code to create the application and serialize `enum` values as strings:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.ConfigureHttpJsonOptions(options => {
    options.SerializerOptions.Converters
        .Add(new JsonStringEnumConverter());
});
var app = builder.Build();
```

Finally, the last part represents the consumer of the context:

```
app.MapGet("/", () => data);
app.MapPut("/", (ReplaceSortStrategy sortStrategy) =>
{
    ISortStrategy strategy = sortStrategy.SortOrder == SortOrder.Ascending
        ? new SortAscendingStrategy()
        : new SortDescendingStrategy();
    data.SetSortStrategy(strategy);
    data.Sort();
    return data;
});
app.Run();
public enum SortOrder
{
    Ascending,
    Descending
}
public record class ReplaceSortStrategy(SortOrder SortOrder);
```

In the preceding code, we declared the following endpoints:

- The first endpoint returns the `data` object when a client sends a `GET` request.
- The second endpoint allows changing the sort strategy based on the `SortOrder` enum when a client sends a **PUT** request. Once the strategy is modified, it sorts the collection and returns the sorted data.

The highlighted code represents the consumption of this implementation of the strategy pattern.

The `ReplaceSortStrategy` class is an input DTO. Combined with the `SortOrder` enum, they represent the data contract of the second endpoint.

When we run the API and request the first endpoint, it responds with the following JSON body:

```
[
  "Lorem",
  "ipsum",
  "dolor",
  "sit",
  "amet."
]
```

As we can see, the items are in the order we set them because the code never called the `Sort` method. Next, let's send the following HTTP request to the API to change the sort strategy to "descending":

```
PUT https://localhost:7280/
Content-Type: application/json
{
    "sortOrder": "Descending"
}
```

After the execution, the endpoint responds with the following JSON data:

```
[
  "sit",
  "Lorem",
  "ipsum",
  "dolor",
  "amet."
]
```

As we can see from the content, the sorting algorithm worked. Afterward, the list will remain in the same order if we query the GET endpoint. Next, let's look at this use case using a sequence diagram:

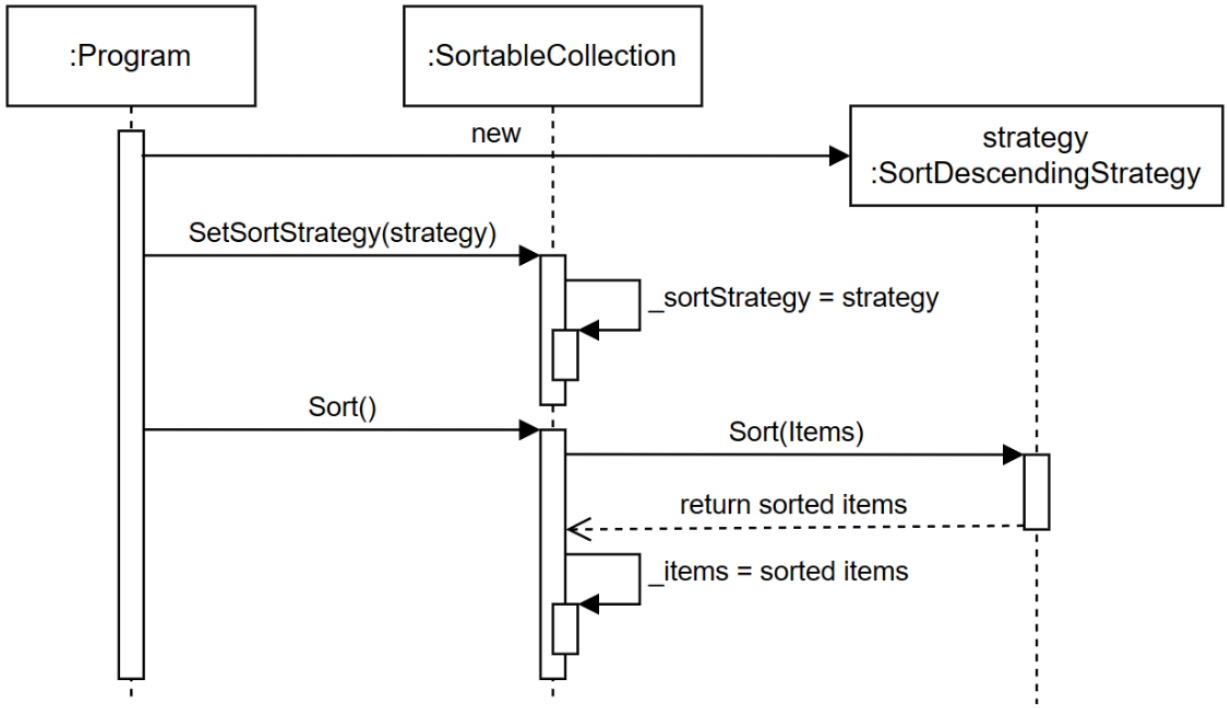


Figure 7.3: Sequence diagram sorting the items using the "sort descending strategy"

The preceding diagram shows the `Program` creating a strategy and assigning it to `SortableCollection` using its `SetSortStrategy` method. Then, when the `Program` calls the `Sort()` method, the `SortableCollection` instance delegates the sorting computation to the underlying implementation of the `ISortStrategy` interface. That implementation is the `SortDescendingStrategy` class (the **strategy**) which was set by the `Program` at the beginning.

Sending another `PUT` request but specifying the `Ascending` sort order end up in a similar result, but the items would be sorted alphabetically.

The HTTP requests are available in the `Consumer.API.http` file.

From a strategy pattern perspective, the `SortableCollection` class (the **context**) is responsible for referencing and using the current strategy.

Conclusion

The Strategy design pattern is very effective at delegating responsibilities to other objects, allowing you to hand over the responsibility of an algorithm to other objects while keeping its usage trivial. It also allows having a rich interface (context) with behaviors that can change at runtime.As we can see, the Strategy pattern is excellent at helping us follow the **SOLID** principles:

- **S**: It helps extract responsibilities from external classes and use them interchangeably.
- **O**: It allows extending classes without updating its code by changing the current strategy at runtime, which is pretty much the actual definition of the OCP.
- **L**: It does not rely on inheritance. Moreover, it plays a large role in the *composition over inheritance principle*, helping us avoid inheritance altogether and the LSP.
- **I**: By creating smaller strategies based on lean and focused interfaces, the Strategy pattern is an excellent enabler of the ISP.
- **D**: The creation of dependencies is moved from the class using the strategy (the context) to the class's consumer. That makes the context depend on abstraction instead of implementation, inverting the flow of control.

C# Features

If you noticed C# features you are less familiar with, *Appendix A* explains many of them briefly.

Next, let's explore the Abstract Factory pattern.

The Abstract Factory design pattern

The Abstract Factory design pattern is a creational design pattern from the GoF. We use creational patterns to create other objects, and factories are a very popular way of doing that.The Strategy pattern is the backbone of dependency injection, enabling the composition of complex object trees, while factories are used to create some of those complex objects that can't be assembled automatically by a dependency injection library. More on that in the next chapter.

Goal

The Abstract Factory pattern is used to abstract the creation of a family of objects. It usually implies the creation of multiple object types within that family. A family is a group of related or dependent objects (classes).Let's think about creating automotive vehicles. There are multiple vehicle types, and there are multiple models and makes for each type. We can use the Abstract Factory pattern to model this sort of scenario.

Note

The *Factory Method* pattern also focuses on creating a single type of object instead of a family. We only cover Abstract Factory here, but we use other types of factories later in the book.

Design

With Abstract Factory, the consumer asks for an abstract object and gets one. The factory is an abstraction, and the resulting objects are also abstractions, decoupling the creation of an object from its consumers.That allows adding or removing families of objects produced together without impacting the consumers (all actors communicate through abstractions).In our case, the family (the set of objects the factory can produce) is composed of a car and a bike, and each factory (family) must produce both objects.If we think about vehicles, we could have the ability to create low- and high-end models of each vehicle type. Here is a diagram representing how to achieve that using the Abstract Factory pattern:

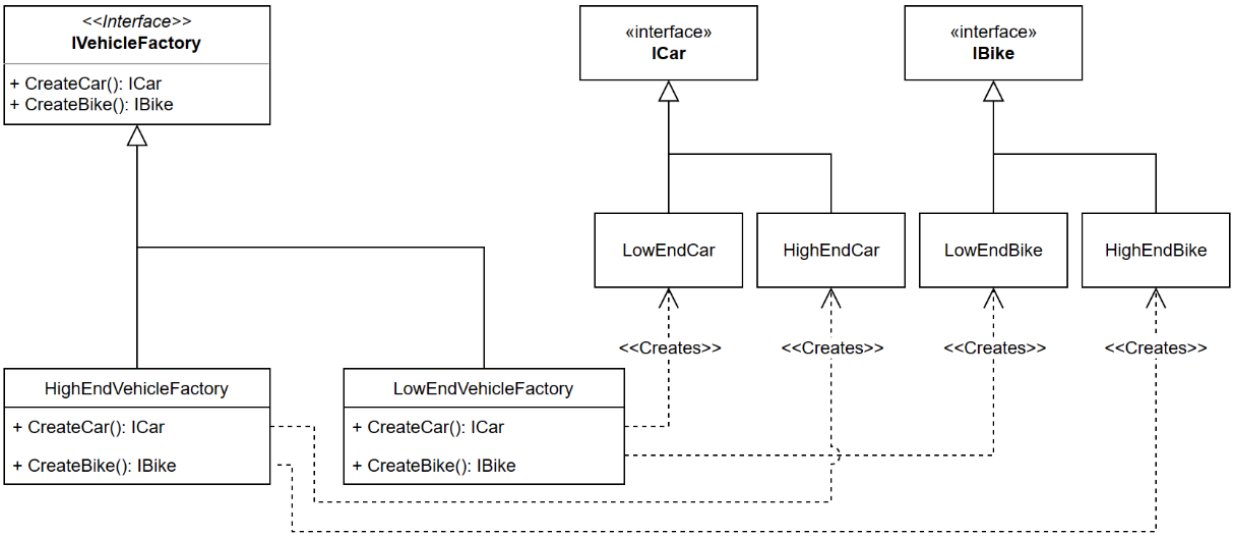


Figure 7.4: Abstract Factory class diagram

In the diagram, we have the following elements:

- The `IVehicleFactory` interface represents the Abstract Factory. It defines two methods: one that creates cars of type `ICar` and another that creates bikes of type `IBike`.
- The `HighEndVehicleFactory` class is a concrete factory implementing the `IVehicleFactory` interface. It handles high-end vehicle model creation, and its methods return `HighEndCar` or `HighEndBike` instances.
- The `LowEndVehicleFactory` is a second concrete factory implementing the `IVehicleFactory` interface. It handles low-end vehicle model creation, and its methods return `LowEndCar` or `LowEndBike` instances.
- `LowEndCar` and `HighEndCar` are two implementations of `ICar`.
- `LowEndBike` and `HighEndBike` are two implementations of `IBike`.

Based on that diagram, consumers use the concrete factories through the `IVehicleFactory` interface and should not be aware of the implementation used underneath. Applying this pattern abstracts away the vehicle creation process.

Project – Abstract Factory

Context: We need to support the creation of multiple models of vehicles. We also need to be able to add new models as they become available without impacting the system. To begin with, we only support high-end and low-end models, but we know this will change sooner rather than later. The program must only support the creation of cars and bikes.For the sake of our demo, the vehicles are just empty classes and interfaces because learning how to model vehicles is not necessary to understand the pattern; that would be noise. The following code represents those entities:

```
public interface ICar { }
public interface IBike { }
public class LowEndCar : ICar { }
public class LowEndBike : IBike { }
public class HighEndCar : ICar { }
public class HighEndBike : IBike { }
```

Next, we look at the part that we want to study—the factories:

```
public interface IVehicleFactory
{
    ICar CreateCar();
    IBike CreateBike();
}
public class LowEndVehicleFactory : IVehicleFactory
{
    public IBike CreateBike() => new LowEndBike();
    public ICar CreateCar() => new LowEndCar();
}
public class HighEndVehicleFactory : IVehicleFactory
{
    public IBike CreateBike() => new HighEndBike();
    public ICar CreateCar() => new HighEndCar();
}
```

The factories are simple implementations that describe the pattern well:

- `LowEndVehicleFactory` creates low-end models.
- `HighEndVehicleFactory` creates high-end models.

The consumer of this code is an xUnit test project. Unit tests are often your first consumers, especially if you are doing **test-driven development** (**TDD**).To make the tests easier, I created the following base test class:

```
using Xunit;
namespace Vehicles;
public abstract class BaseAbstractFactoryTest<TConcreteFactory, TExpectedCar, TExpectedBike>
    where TConcreteFactory : IVehicleFactory, new()
{
    // Test methods here
}
```

The key to that class is the following generic parameters:

- The `TConcreteFactory` parameter represents the type of concrete factory we want to test. Its generic constraint specifies that it must implement the `IVehicleFactory` interface and have a parameterless constructor.
- The `TExpectedCar` parameter represents the type of `ICar` we expect from the `CreateCar` method.
- The `TExpectedBike` parameter represents the type of `IBike` we expect from the `CreateBike` method.

The first test method contained by that class is the following:

```
[Fact]
public void Should_create_a_ICar_of_type_TExpectedCar()
{
    // Arrange
    IVehicleFactory vehicleFactory = new TConcreteFactory();
    var expectedCarType = typeof(TExpectedCar);
    // Act
    ICar result = vehicleFactory.CreateCar();
    // Assert
    Assert.IsType(expectedCarType, result);
}
```

The preceding test method creates a vehicle factory using the TConcreteFactory generic parameter, then creates a car using that factory. Finally, it asserts ICar instance is of the expected type.The second test method contains by that class is the following:

```
[Fact]
public void Should_create_a_IBike_of_type_TExpectedBike()
{
    // Arrange
    IVehicleFactory vehicleFactory = new TConcreteFactory();
    var expectedBikeType = typeof(TExpectedBike);
    // Act
    IBike result = vehicleFactory.CreateBike();
    // Assert
    Assert.IsType(expectedBikeType, result);
}
```

The preceding test method is very similar and creates a vehicle factory using the `TConcreteFactory` generic parameter but then creates a bike instead of a car using that factory. Finally, it asserts `IBike` instance is of the expected type.

> I used the `ICar` and `IBike` interfaces to type the variables instead of `var`, to clarify the `result` variable type. In another context, I would have used `var` instead. The same applies to the `IVehicleFactory` interface.

Now, to test the low-end factory, we declare the following test class:

```
namespace Vehicles.LowEnd;
public class LowEndVehicleFactoryTest : BaseAbstractFactoryTest<LowEndVehicleFactory, LowEndCar,
{
}
```

That class solely depends on the `BaseAbstractFactoryTest` class and specifies the types to test for (highlighted).Next, to test the high-end factory, we declare the following test class:

```
namespace Vehicles.HighEnd;
public class HighEndVehicleFactoryTest : BaseAbstractFactoryTest<HighEndVehicleFactory, HighEndCa
{
}
```

Like the low-end factory, that class depends on the `BaseAbstractFactoryTest` class and specifies the types to test for (highlighted).

> In a more complex scenario where we can't use the `new()` generic constraint, we can leverage an IoC container to create the instance of `TConcreteFactory` and optionally mock its dependencies.

With that test code, we created the following two sets of two tests:

- A `LowEndVehicleFactory` class that should create a `LowEndCar` instance.
- A `LowEndVehicleFactory` class that should create a `LowEndBike` instance.
- A `HighEndVehicleFactory` class that should create a `HighEndCar` instance.
- A `HighEndVehicleFactory` class that should create a `HighEndBike` instance.

We now have four tests: two for bikes and two for cars.If we review the tests' execution, both test methods are unaware of types. They use the Abstract Factory (`IVehicleFactory`) and test the `result` against the expected type without knowing what they are testing but the abstraction. That shows how loosely coupled the consumers (tests) and the factories are.

> We would use the `ICar` or the `IBike` instances in a real-world program to do something relevant based on the specifications. That could be a racing game or a rich person's garage management system; who knows!

The important part of this project is **the abstraction of the object creation process**. The test code (consumer) is not aware of the implementations.Next, we extend our implementation.

Project – The mid-range vehicle factory

To prove the flexibility of our design based on the Abstract Factory pattern, let's add a new concrete factory named `MidRangeVehicleFactory`. That factory should return a `MidRangeCar` or a `MidRangeBike` instance. Once again, the car and bike are just empty classes (of course, in your programs, they will do something):

```
public class MiddleGradeCar : ICar { }
public class MiddleGradeBike : IBike { }
```

The new `MidRangeVehicleFactory` looks pretty much the same as the other two:

```
public class MidRangeVehicleFactory : IVehicleFactory
{
    public IBike CreateBike() => new MiddleGradeBike();
    public ICar CreateCar() => new MiddleGradeCar();
}
```

Now, to test the mid-range factory, we declare the following test class:

```
namespace Vehicles.MidRange;
public class MidRangeVehicleFactoryTest : BaseAbstractFactoryTest<MidRangeVehicleFactory, MidRang
{
}
```

Like the low-end and high-end factories, the mid-range test class depends on the `BaseAbstractFactoryTest` class and specifies the types to test for (highlighted).If we run the tests, we now have the following six passing tests:

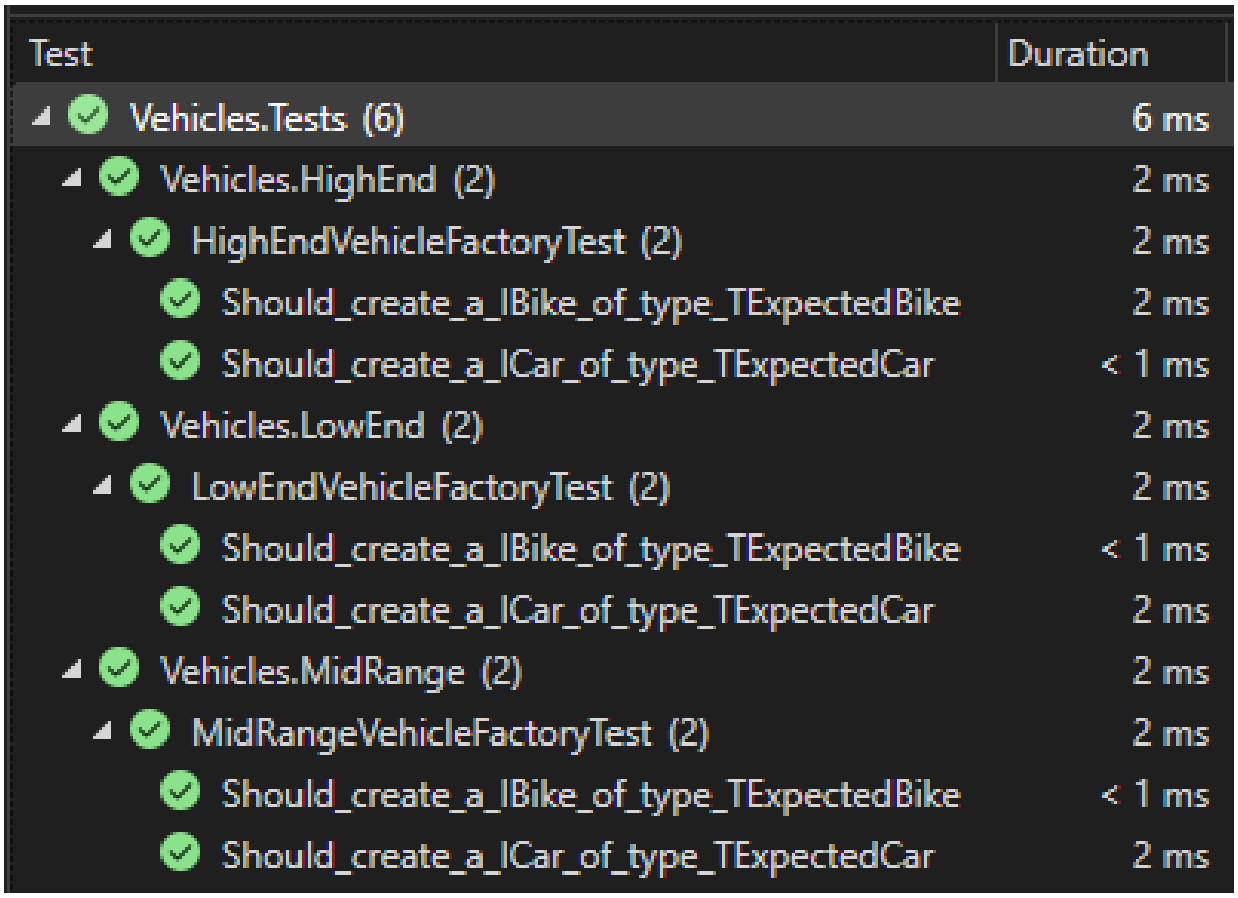


Figure 7.5: Visual Studio Test Explorer showcasing the six passing tests.

So, without updating the consumer (the `AbstractFactoryTest` class), we added a new family of vehicles, the middle-end cars and bikes; kudos to the Abstract Factory pattern for that wonderfulness!

Impacts of the Abstract Factory

Before concluding, what would happen if we packed everything in a large interface instead of using an Abstract Factory (breaking the ISP along the way)? We could have created something like the following interface:

```
public interface ILargeVehicleFactory
{
    HighEndBike CreateHighEndBike();
    HighEndCar CreateHighEndCar();
    LowEndBike CreateLowEndBike();
    LowEndCar CreateLowEndCar();
}
```

As we can see, the preceding interface contains four specific methods and seems docile. However, the consumers of that code would be tightly coupled with those specific methods. For example, to change a consumer's behavior, we'd need to update its code, like changing the call from `CreateHighEndBike` to `CreateLowEndBike`, which breaks the OCP. On the other hand, with the factory method, we can set a different factory for the consumers to spit out different results, which moves the flexibility out of the object itself and becomes a matter of composing the object graph instead (more on that in the next chapter).Moreover, when we want to add mid-range vehicles, we must update the `ILargeVehicleFactory` interface, which becomes a breaking change (the implementation(s) of the `ILargeVehicleFactory` must be updated). Here's an example of the two new methods:

```
public interface ILargeVehicleFactory
{
    HighEndBike CreateHighEndBike();
    HighEndCar CreateHighEndCar();
    LowEndBike CreateLowEndBike();
    LowEndCar CreateLowEndCar();
    MidRangeBike CreateMidRangeBike();
    MidRangeCar CreateMidRangeCar();
}
```

From there, once the implementation(s) are updated, if we want to consume the new mid-range vehicles, we need to open each consumer class and apply the changes there, which once again breaks the OCP.

> The most crucial part is understanding and seeing the coupling and its impacts. Sometimes, it's okay to tightly couple one or more classes together as we don't always need the added flexibility the SOLID principles and some design patterns can bring.

Now let's conclude before exploring the last design pattern of the chapter.

Conclusion

The Abstract Factory pattern is excellent for abstracting away the creation of object families, isolating each family and its concrete implementation, leaving the consumers unaware of the family created at runtime by the factory.We talk more about factories in the next chapter; meanwhile, let's see how the Abstract Factory pattern can help us follow the **SOLID** principles:

- **S**: Each concrete factory is solely responsible for creating a family of objects. You could combine Abstract Factory with other creational patterns, such as the **Prototype** and **Builder** patterns for more complex creational needs.
- **O**: We can create new families of objects, like the mid-range vehicles, without breaking existing client code.
- **L**: We aim at composition, so there's no need for any inheritance, implicitly discarding the need for the LSP. If you use abstract classes in your design, you must ensure you don't break the LSP when creating new abstract factories.
- **I**: Extracting a small abstraction with many implementations where each concrete factory focuses on one family makes that interface very focused on one task instead of having a large interface that exposes all types of products (like the `ILargeVehicleFactory` interface).
- **D**: By depending only on interfaces, the consumer is unaware of the concrete types it uses.

Next, we explore the last design pattern of the chapter.

The Singleton design pattern

The Singleton design pattern allows creating and reusing a single instance of a class. We could use a static class to achieve almost the same goal, but not everything is doable using static classes. For example, a static class can’t implement an interface. We can’t pass an instance of a static class as an argument because there is no instance. We can only use static classes directly, which leads to tight coupling every time.The Singleton pattern in C# is an anti-pattern, and we should rarely use it, if ever, and use dependency injection instead. That said, it is a classic design pattern worth learning to at least avoid implementing it. We explore a better alternative in the next chapter.Here are a few reasons why we are covering this pattern:

- It translates into a singleton scope in the next chapter.
- Without knowing about it, you cannot locate it, try to remove it, or avoid its usage.
- It is a simple pattern to explore.
- It leads to other patterns, such as the **Ambient Context** pattern.

Goal

The Singleton pattern limits the number of instances of a class to one. Then, the idea is to reuse the same instance subsequently. A singleton encapsulates both the object logic itself and its creational logic. For example, the Singleton pattern could lower the cost of instantiating an object with a large memory footprint since the program instantiates it only once.Can you think of a SOLID principle that gets broken right there?The Singleton pattern promotes that one object must have two responsibilities, breaking the **Single Responsibility Principle** (**SRP**). A singleton is the object itself and its own factory.

Design

This design pattern is straightforward and is limited to a single class. Let’s start with a class diagram:

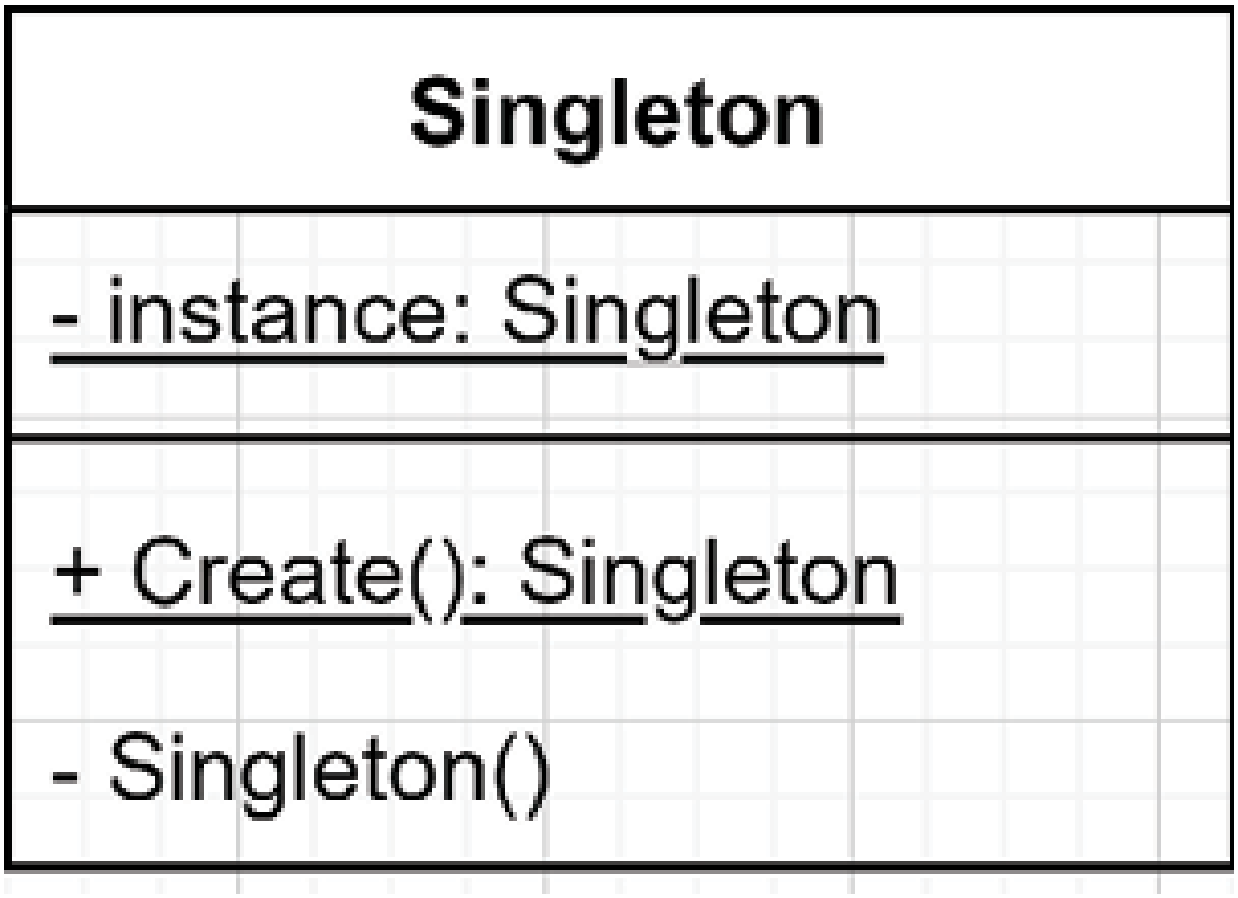


Figure 7.6: Singleton pattern class diagram

The `Singleton` class is composed of the following:

- A private static field that holds its unique instance.
- A public static `Create()` method that creates or returns the unique instance.
- A private constructor, so external code cannot instantiate it without passing by the `Create` method.

You can name the `Create()` method anything or even get rid of it, as we see in the next example. We could name it `GetInstance()`, or it could be a static property named `Instance` or bear any other relevant name.

We can translate the preceding diagram to the following code:

```
public class MySingleton
{
    private static MySingleton? _instance;
    private MySingleton() { }
    public static MySingleton Create()
    {
        _instance ??= new MySingleton();
        return _instance;
    }
}
```

The null-coalescing assignment operator `??=` assigns the new instance of `MySingleton` only if the `_instance` member is `null`. That line is equivalent to writing the following if statement:

```
if (_instance == null)
{
```

```
    _instance = new MySingleton();
}
```

Before discussing the code more, let’s explore our new class's behavior. We can see in the following unit test that `MySingleton.Create()` always returns the same instance as expected:

```
public class MySingletonTest
{
    [Fact]
    public void Create_should_always_return_the_same_instance()
    {
        var first = MySingleton.Create();
        var second = MySingleton.Create();
        Assert.Same(first, second);
    }
}
```

And voilà! We have a working Singleton pattern, which is extremely simple—probably the most simple design pattern that I can think of.Here is what is happening under the hood:

1. The first time that a consumer calls `MySingleton.Create()`, it creates the first instance of `MySingleton`. Since the constructor is `private`, it can only be created from the inside.
2. The `Create` method then persists that first instance to the `_instance` field for future use.
3. When a consumer calls `MySingleton.Create()` a second time, it returns the `_instance` field, reusing the class's previous (and only) instance.

Now that we understand the logic, there is a potential issue with that design: it is not thread-safe. If we want our singleton to be thread-safe, we can `lock` the instance creation like this:

```
public class MySingletonWithLock
{
    private static readonly object _myLock = new();
    private static MySingletonWithLock? _instance;
    private MySingletonWithLock() { }
    public static MySingletonWithLock Create()
    {
        lock (_myLock)
        {
            _instance ??= new MySingletonWithLock();
        }
        return _instance;
    }
}
```

In the preceding code, we ensure two threads are not attempting to access the `Create` method simultaneously to ensure they are not getting different instances. Next, we improve our thread-safe example by making it shorter.

An alternate (better) way

Previously, we used the “long way” of implementing the Singleton pattern and had to implement a thread-safe mechanism. Now that classic is behind us. We can shorten that code and even get rid of the `Create()` method like this:

```
public class MySimpleSingleton
{
    public static MySimpleSingleton Instance { get; } = new MySimpleSingleton();
    private MySimpleSingleton() { }
}
```

The preceding code relies on the static initializer to ensure that only one instance of the `MySimpleSingleton` class is created and assigned to the `Instance` property.

This simple technique should do the trick unless the singleton's constructor executes some heavy processing.

With the property instead of a method, we can use the singleton class like this:

```
MySimpleSingleton.Instance.SomeOperation();
```

We can prove the correctness of that claim by executing the following test method:

```
[Fact]
public void Create_should_always_return_the_same_instance()
{
    var first = MySimpleSingleton.Instance;
    var second = MySimpleSingleton.Instance;
    Assert.Same(first, second);
}
```

It is usually best to delegate responsibilities to the language or the framework whenever possible like we did here with the property initializer. Using a static constructor would also be a valid, thread-safe alternative, once again delegating the job to language features.

Beware of the arrow operator.

It may be tempting to use the arrow operator `=>` to initialize the `Instance` property like this: `public static MySimpleSingleton Instance => new MySimpleSingleton();`, but doing so would return a new instance every time. This would defeat the purpose of what we want to achieve. On the other hand, the property initializer runs only once.

The arrow operator makes the `Instance` property an expression-bodied member, equivalent to creating the following getter: `get { return new MySimpleSingleton(); }`. You can consult *Appendix A* for more information about expression-bodies statements.

Before we conclude the chapter, the Singleton (anti-)pattern also leads to a code smell.

Code smell – Ambient Context

That last implementation of the **Singleton** pattern led us to the **Ambient Context** pattern. We could even call the Ambient Context an anti-pattern, but let's just state that it is a consequential code smell.I do not recommend using ambient contexts for multiple reasons. First, I do my best to avoid anything global; an ambient context is a global state. Globals, like static members in C#, can look very convenient because they are easy to access and use. They are always there and accessible whenever needed: easy. However, they bring many drawbacks in terms of flexibility and testability.When using an ambient context, the following occurs:

- **Tight coupling**: global states lead to less flexible systems; consumers are tightly coupled with the ambient context.
- **Testing difficulty**: global objects are harder to replace, and we cannot easily swap them for other objects, like a mock.
- **Unforseen impacts**: if some part of your system messes up your global state, that may have unexpected consequences on other parts of your system, and you may have difficulty finding out the root cause of those errors.
- **Potential misuse**: developers could be tempted to add non-global concerns to the ambient context, leading to a bloated component.

Fun fact

Many years ago, before the JavaScript frameworks era, I fixed a bug in a system where some function was overriding the value of `undefined` due to a subtle error. This is an excellent example of how global variables could impact your whole system and make it more brittle. The same applies to the Ambient Context and Singleton patterns in C#; globals can be dangerous and annoying.

Rest assured that, nowadays, browsers won't let developers update the value of `undefined`, but it was possible back then.

Now that we've discussed global objects, an ambient context is a global instance, usually available through a static property. The Ambient Context pattern can bring good things, but it is a code smell that smells bad.

> There are a few examples in .NET Framework, such as `System.Threading.Thread.CurrentPrincipal` and `System.Threading.Thread.CurrentThread`, that are scoped to a thread instead of being purely global like most static members. An ambient context does not have to be a singleton, but that is what they are most of the time. Creating a non-global (scoped) ambient context is harder and requires more work.

Is the Ambient Context pattern good or bad? I'd go with both! It is useful primarily because of its convenience and ease of use. Most of the time, it could and should be designed differently to reduce its drawbacks.There are many ways of implementing an ambient context, but to keep it brief and straightforward, we are focusing only on the singleton version of the ambient context. The following code is a good example:

```
public class MyAmbientContext
{
    public static MyAmbientContext Current { get; } = new MyAmbientContext();
    private MyAmbientContext() { }
    public void WriteSomething(string something)
    {
        Console.WriteLine($"This is your something: {something}");
    }
}
```

That code is an exact copy of the `MySimpleSingleton` class, with a few subtle changes:

- `Instance` is named `Current`.
- The `WriteSomething` method is new but has nothing to do with the Ambient Context pattern itself; it is just to make the class do something.

If we take a look at the test method that follows, we can see that we use the ambient context by calling `MyAmbientContext.Current`, just like we did with the last singleton implementation:

```
[Fact]
public void Should_echo_the_inputted_text_to_the_console()
{
    // Arrange (make the console write to a StringBuilder
    // instead of the actual console)
    var expectedText = "This is your something: Hello World!" + Environment.NewLine;
    var sb = new StringBuilder();
    using (var writer = new StringWriter(sb))
    {
        Console.SetOut(writer);
        // Act
        MyAmbientContext.Current.WriteSomething("Hello World!");
    }
    // Assert
    var actualText = sb.ToString();
    Assert.Equal(expectedText, actualText);
}
```

The property could include a public setter or support more complex logic. Building the right classes and exposing the right behaviors is up to you and your specifications.To conclude this interlude, avoid ambient contexts and use instantiable classes instead. We see how to replace a singleton with a single instance of a class using dependency injection in the next chapter. That gives us a more flexible alternative to the Singleton pattern. We can also create a single instance per HTTP request, which saves us the trouble of coding it while eliminating the disadvantages.

Conclusion

The Singleton pattern allows the creation of a single instance of a class for the whole lifetime of the program. It leverages a `private static` field and a `private` constructor to achieve its goal, exposing the instantiation through a `public static` method or property. We can use a field initializer, the `Create` method itself, a static

constructor, or any other valid C# options to encapsulate the initialization logic.Now let’s see how the Singleton pattern can help us (not) follow the SOLID principles:

- **S**: The singleton violates this principle because it has two clear responsibilities:
  - It has the responsibility for which it has been created (not illustrated here), like any other class.
  - It has the responsibility of creating and managing itself (lifetime management).
- **O**: The Singleton pattern also violates this principle. It enforces a single static instance, locked in place by itself, which limits extensibility. It is impossible to extend the class without changing its code.
- **L**: There is no inheritance directly involved, which is the only good point.
- **I**: No C# interface is involved, which violates this principle. However, we can look at the class interface instead, so building a small targeted singleton instance would satisfy this principle.
- **D**: The singleton class has a rock-solid hold on itself. It also suggests using its static property (or method) directly without using an abstraction, breaking the DIP with a sledgehammer.

As you can see, the Singleton pattern violates all the SOLID principles but the LSP and should be used cautiously. Having only a single instance of a class and always using that same instance is a common concept. However, we explore more proper ways to do this in the next chapter, leading me to the following advice: do not use the Singleton pattern, and if you see it used somewhere, try refactoring it out.

> I suggest avoiding static members that create global states as a general good practice. They can make your system less flexible and more brittle. There are occasions where `static` members are worth using, but try keeping their number as low as possible. Ask yourself if you can replace that `static` member or class with something else before coding one.

Some may argue that the Singleton design pattern is a legitimate way of doing things. However, in ASP.NET Core, I am afraid I have to disagree: we have a powerful mechanism to do it differently, called dependency injection. When using other technologies, maybe, but not with modern .NET.

Summary

In this chapter, we explored our first GoF design patterns. These patterns expose some of the essential basics of software engineering, not necessarily the patterns themselves, but the concepts behind them:

- The Strategy pattern is a behavioral pattern that we use to compose most of our future classes. It allows swapping behavior at runtime by composing an object with small pieces and coding against interfaces, following the SOLID principles.
- The Abstract Factory pattern brings the idea of abstracting away object creation, leading to a better separation of concerns. More specifically, it aims to abstract the creation of object families and follow the SOLID principles.
- Even if we defined it as an anti-pattern, the Singleton pattern brings the application-level objects to the table. It allows the creation of a single instance of an object that lives for the whole lifetime of a program. The pattern violates most SOLID principles.

We also peeked at the Ambient Context code smell, which is used to create an omnipresent entity accessible from everywhere. It is often implemented as a singleton and brings a global state object to the program.The next chapter explores how dependency injection helps us compose complex yet maintainable systems. We also revisit the Strategy, the Factory, and the Singleton patterns to see how to use them in a dependency-injection-oriented context and how powerful they really are.

Questions

Let’s take a look at a few practice questions:

1. Why is the Strategy pattern a behavioral pattern?
2. How could we define the goal of the creational patterns?

3. If I write the code `public MyType MyProp => new MyType();`, and I call the property twice (`var v1 = MyProp; var v2 = MyProp;`), are `v1` and `v2` the same instance or two different instances?
4. Is it true that the Abstract Factory pattern allows us to add new families of elements without modifying the existing consuming code?
5. Why is the Singleton pattern an anti-pattern?

Answers

1. It helps manage behaviors at runtime, such as changing an algorithm in the middle of a running program.
2. The creational patterns are responsible for creating objects.
3. `v1` and `v2` are two different instances. The code on the right-hand side of the arrow operator is executed every time you call the property's getter.
4. Yes, it is true. That's the primary goal of the pattern, as we demonstrated in the `MidRangeVehicleFactory` code sample.
5. The Singleton pattern violates the SOLID principles and encourages using global (static) state objects. We can avoid this pattern most of the time.

8 Dependency Injection

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter explores the ASP.NET Core **Dependency Injection** (**DI**) system, how to leverage it efficiently, and its limits and capabilities.We learn to compose objects using DI and delve into the Inversion of Control (IoC) principle. As we traverse the landscape of the built-in DI container, we explore its features and potential uses.Beyond practical examples, we lay down the conceptual foundation of Dependency Injection to understand its purpose, its benefits, and the problems it solves and to lay down the ground for the rest of the book as we rely heavily on DI.We then return to the first three Gang of Four (GoF) design patterns we encountered, but this time, through the lens of Dependency Injection. By refactoring these patterns using DI, we gain a more holistic understanding of how this powerful design tool influences the structure and flexibility of our software.Dependency Injection is a cornerstone in your path toward mastering modern application design and its transformative role in developing efficient, adaptable, testable, and maintainable software.In this chapter, we cover the following topics:

- What is dependency injection?
- Revisiting the Strategy pattern
- Understanding guard clauses
- Revisiting the Singleton pattern
- Understanding the Service Locator pattern
- Revisiting the Factory pattern

What is dependency injection?

DI is a way to apply the **Inversion of Control** (**IoC**) principle. IoC is a broader version of the dependency inversion principle (the *D* in SOLID).The idea behind DI is to move the creation of dependencies from the objects themselves to the **composition root**. That way, we can delegate the management of dependencies to an **IoC container**, which does the heavy lifting.

> An IoC container and a **DI container** are the same thing—they're just different words people use. I use both interchangeably in real life, but I stick to IoC container in the book because it seems more accurate than DI container.
>
> > IoC is the concept (the principle), while DI is a way of inverting the flow of control (applying IoC). For example, you apply the IoC principle (inverting the flow) by injecting dependencies at runtime (doing DI) using a container. Feel free to use any or both.

Next, we define the composition root.

The composition root

A critical concept behind DI is the composition root. The composition root is where we tell the container about our dependencies and their expected lifetime: where we compose our dependency trees. The composition root should be as close to the program's starting point as possible, so from ASP.NET Core 6 onward, the composition root is in the `Program.cs` file. In the previous versions, it was in the `Program` or `Startup` classes.Next, we explore how to leverage DI to create highly adaptable systems.

Striving for adaptability

To achieve a high degree of flexibility with DI, we can apply the following formula, driven by the SOLID principles:Object `A` should not know about object `B` that it is using. Instead, `A` should use an interface, `I`, implemented by `B`, and `B` should be resolved and injected at runtime.Let’s decompose this:

- Object `A` should depend on interface `I` instead of concrete type `B`.
- Instance `B`, injected into `A`, should be resolved by the IoC container at runtime.
- `A` should not be aware of the existence of `B`.
- `A` should not control the lifetime of `B`.

We can also inject objects directly without passing by an interface. It all depends on what we inject, in what context, and our requirements. We tackle many use cases throughout the book to help you understand DI.

Next, we translate this equation into an analogy that helps explain the reasons to use a container.

Understanding the use of the IoC container

To better understand the use of the IoC container and to create an image around the previous adaptability concept, let’s start with a LEGO® analogy where IoC is the equivalent of drawing a plan to build a LEGO®castle:

1. We draw the plan
2. We gather the blocks
3. We press the start button on a hypothetical robot builder
4. The robot assembles the blocks by following our plan
5. The castle is complete

By following this logic, we can create a new 4x4 block with a unicorn painted on its side (concrete type), update the plan (composition root), and then press the restart button to rebuild the castle with that new block inserted into it, replacing the old one without affecting the structural integrity of the castle (program). As long as we respect the 4x4 block contract (interface), everything is updatable without impacting the rest of the castle, leading to great flexibility.Following that idea, if we need to manage every single LEGO® block one by one, it would quickly become incredibly complex! Therefore, managing all dependencies by hand in a project would be super tedious and error-prone, even in the smallest program. This situation is where an IoC container (the hypothetical robot builder) comes into play.

The role of an IoC container

An IoC container manages objects for us. We configure it, and then, when we ask for a service, the container resolves and injects it. On top of that, the container manages the lifetime of dependencies, leaving our classes to do only one thing, the job we designed them to do. No more need to think about their dependencies!The bottom line is that an IoC container is a DI framework that does the auto-wiring for us. We can conceptualize Dependency Injection as follows:

1. The *consumer* of a dependency states its needs about one or more dependencies (contracts).
2. The IoC container injects that dependency (implementation) upon creating the *consumer*, fulfilling its needs at runtime.

Next, we explore an code smell that applying Dependency Injection helps us avoid.

Code smell – Control Freak

Control freak is a code smell and even an anti-pattern that forbids us from using the `new` keyword. Yes, using the `new` keyword is the code smell! The following code is wrong and can't leverage DI:

```
namespace CompositionRoot.ControlFreak;
public class Consumer
{
    public void Do()
    {
        var dependency = new Dependency();
        dependency.Operation();
    }
}
public class Dependency
{
    public void Operation()
        => throw new NotImplementedException();
}
```

The highlighted line shows the anti-pattern in action. To enable the Consumer class to use dependency injection, we could update it like the following:

```
public class Consumer
{
    private readonly Dependency _dependency;
    public DIEnabledConsumer(Dependency dependency)
    {
        _dependency = dependency;
    }
    public void Do()
    {
        _dependency.Operation();
    }
}
```

The preceding code removes the new keyword and is now open for modification. The highlighted lines represent the constructor injection pattern we explore subsequently in this chapter.Nevertheless, do not ban the `new` keyword just yet. Instead, every time you use it, ask yourself whether the object you instantiated using the `new` keyword is a dependency that could be managed by the container and injected instead.To help with that, I borrowed two terms from Mark Seemann's book *Dependency Injection in .NET*; the name *Control Freak* also comes from that book. He describes the following two categories of dependencies:

- Stable dependencies
- Volatile dependencies

Next is my take on defining them.

Stable dependencies

Stable dependencies should not break our application when a new version is released. They should use deterministic algorithms (input `X` should always produce output `Y`), and you should not expect to change them with something else in the future.

> Most data structures devoided of behaviors, like Data Transfer Objects (DTOs), fall into this category. You can also consider the .NET BCL as stable dependencies.

We can still instantiate objects using the `new` keyword when they fall into this category because the dependencies are stable and unlikely to break anything if they change.Next, we look at their counterpart.

Volatile dependencies

Volatile dependencies can change at runtime, like extendable elements with contextual behaviors. They may also be likely to change for various reasons like new features development.

Most classes we create, such as data access and business logic code, are volatile dependencies.

The primary way to break the tight coupling between classes is to rely on interfaces and DI and no longer instantiate those volatile dependencies using the `new` keyword. Volatile dependencies are why dependency injection is key to building flexible, testable, and maintainable software.

Conclusion

To conclude this interlude: don't be a control freak anymore; those days are behind you!

When in doubt, inject the dependency instead of using the `new` keyword.

Next, we explore the available lifetimes we can attribute to our volatile dependencies.

Object lifetime

Now that we understand we should no longer use the `new` keyword, we need a way to create those classes. From now on, the IoC container will play that role and manage object instantiation and their lifetime for us.

What's an object's lifetime?

When we create an instance manually, using the `new` keyword, we create a hold on that object; we know when we create it and when its life ends. That's the lifetime of the object.Of course, using the `new` keyword leaves no chance to control these objects from the outside, enhance them, intercept them, or swap them for another implementation—as covered in the preceding *Code smell – Control Freak* section.

.NET object lifetime

With dependency injection, we need to forget about controlling objects and start to think about using dependencies, or more explicitly, depending on their interfaces. In ASP.NET Core, there are three possible lifetimes to choose from:

Lifetime	**Description**
Transient	The container creates a new instance every time.
Scoped	The container creates an instance per HTTP request and reuses it. In some rare cases, we can also create custom scopes.
Singleton	The container creates a single instance of that dependency and always reuses that unique object.

Table 8.1: objects lifetime description

We can now manage our volatile dependencies using one of those three scopes. Here are some questions to help you choose:

- Do I need a single instance of my dependency? Yes? Use the **singleton** lifetime.
- Do I need a single instance of my dependency shared over an HTTP request? Yes? Use the **scoped** lifetime.
- Do I need a new instance of my dependency every time? Yes? Use the **transient** lifetime.

A general approach to object lifetime is to design the components to be *singletons*. When impossible, we go for *scoped*. When *scoped* is also impossible, go for *transient*. This way, we maximize instance reuse, lower the overhead of creating objects, lower the memory cost of keeping those objects in memory, and lower the amount of garbage collection needed to remove unused instances.

For example, we can pick *singleton* mindlessly for stateless objects, which are the easiest to maintain and less likely to break.

For stateful objects, where multiple consumers use the same instance, we must ensure the object is thread-safe if the lifetime is *singleton* or *scoped* because multiple consumers could try to access it simultaneously.

One essential aspect to consider when choosing a lifetime is the consumers of stateful objects. For example, if we load data related to the current user, we must ensure that data do not leak to other users. To do so, we can define the lifetime of that object to *scoped*, which is limited to a single HTTP request. If we don't want to reuse that state between multiple consumers, we can choose a *transient* lifetime to ensure every consumer gets their own instance.

How does that translate into code? .NET offers multiple extension methods to help us configure the lifetimes of our objects, like `AddTransient`, `AddScoped`, and `AddSingleton`, which explicitly state their lifetimes.

We use the built-in container throughout the book with many of its registration methods, so you should grow familiar with it very quickly. It has good discoverability, so you can explore the possibilities using IntelliSense while writing code or reading the documentation.

Next, we use those methods and explore how to register dependencies with the container.

Registering our dependencies

In ASP.NET Core, we register our dependencies in the `Program.cs` file, which represents the composition root. Since the minimal hosting model, the `WebApplicationBuilder` exposes the `Services` property we use to add our dependencies to the container. Afterward, .NET creates the container when it builds the `WebApplication` instance.Next is a minimal `Program.cs` file depicting this concept:

```
var builder = WebApplication.CreateBuilder(args);
// Register dependencies
var app = builder.Build();
// The IoC container is now available
app.Run();
```

Then, we use the `builder.Services` property to register our dependencies in that `IServiceCollection` implementation. Here's an example of registering some dependencies:

```
builder.Services.AddSingleton<Dependency1>();
builder.Services.AddSingleton<Dependency2>();
builder.Services.AddSingleton<Dependency3>();
```

The preceding code registers the dependencies using the singleton lifetime, so we get the same instance each time we request one.

Remember to compose the program in the composition root. That removes the need for those pesky `new` keywords spread around your code base and all the tight coupling that come with them. Moreover, it centralizes the application's composition into that location, creating the plan to assemble the LEGO® blocks.

As you may be thinking right now, that can lead to a lot of registration statements in a single location, and you are correct; maintaining such a composition root would be a challenge in almost any application. To address this concern, we introduce an elegant way to encapsulate the registration code next, ensuring it remains manageable.

Registering your features elegantly

As we've just discovered, while we should register dependencies in the composition root, we can also arrange our registration code in a structured manner. For example, we can break down our application's composition into several methods or classes and invoke them from our composition root. Another strategy could be to use an auto-discovery system to automate the registration of certain services.

> The critical part is to centralize the program composition in one place.

A common pattern in ASP.NET Core is having special methods like `Add[Feature name]`. These methods register their dependencies, letting us add a group of dependencies with just one method call. This pattern is convenient for breaking down program composition into smaller, easier-to-handle parts, like individual features. This also makes the composition root more readable.

> A feature is the correct size as long as it stays cohesive. If your feature becomes too big, does too many things, or starts to share dependencies with other features, it may be time for a redesign before losing control over it. That's usually a good indicator of undesired coupling.

To implement this pattern, we use extension methods, making it trivial. Here's a guide:

1. Create a static class named `[subject]Extensions` in the `Microsoft.Extensions.DependencyInjection` namespace.
2. Create an extension method that returns the `IServiceCollection` interface, which allows method calls to be chained.

> According to Microsoft's recommendation, we should create the class in the same namespace as the element we extend. In our case, the `IServiceCollection` interface lives in the `Microsoft.Extensions.DependencyInjection` namespace.

Of course, this is not mandatory, and we can adapt this process to our needs. For example, we can define the class in another namespace if we want consumers to add a `using` statement implicitly. We can also return something else when the registration process can continue beyond that first method, like a builder interface.

> Builder interfaces are used to configure more complex features, like ASP.NET Core MVC. For example, the `AddControllers` extension method returns an `IMvcBuilder` interface that contains a `PartManager` property. Moreover, some extension methods target the `IMvcBuilder` interface, allowing further configuration of the feature by requiring its registration first; that is, you can't configure `IMvcBuilder` before calling `AddControllers`. You can also design your features to leverage that pattern when needed.

Let's explore a demo.

Project – Registering the Demo Feature

Let's explore registering the dependencies of the Demo Feature. That feature contains the following code:

```
namespace CompositionRoot.DemoFeature;
public class MyFeature
{
    private readonly IMyFeatureDependency _myFeatureDependency;
    public MyFeature(IMyFeatureDependency myFeatureDependency)
    {
        _myFeatureDependency = myFeatureDependency;
    }
    public void Operation()
    {
        // use _myFeatureDependency
    }
}
public interface IMyFeatureDependency { }
public class MyFeatureDependency : IMyFeatureDependency { }
```

As we can see, there is nothing complex but two empty classes and an interface. Remember that we are exploring the registration of dependencies, not what to do with them or what they can do—yet.Now, we want the container to serve an instance of the `MyFeatureDependency` class when a dependency requests the `IMyFeatureDependency` interface as the `MyFeature` class does. We want a singleton lifetime.To achieve this, in the `Program.cs` file, we can write the following code:

```
builder.Services.AddSingleton<MyFeature>();
builder.Services.AddSingleton<IMyFeatureDependency, MyFeatureDependency>();
```

We can also chain the two method calls instead:

```
builder.Services
    .AddSingleton<MyFeature>()
    .AddSingleton<IMyFeatureDependency, MyFeatureDependency>()
;
```

However, this is not yet elegant. What we want to achieve is this:

```
builder.Services.AddDemoFeature();
```

To build that registration method, we can write the following extension method:

```
using CompositionRoot.DemoFeature;
namespace Microsoft.Extensions.DependencyInjection;
public static class DemoFeatureExtensions
{
    public static IServiceCollection AddDemoFeature(this IServiceCollection services)
    {
        return services
            .AddSingleton<MyFeature>()
            .AddSingleton<IMyFeatureDependency, MyFeatureDependency>()
        ;
    }
}
```

As highlighted, the registration is the same but uses the `services` parameter, which is the extended type, instead of the `builder.Services` (`builder` does not exist in that class, yet the `services` parameter is the same object as the `builder.Services` property).If you are unfamiliar with extension methods, they come in handy for extending existing classes, like we just did. Besides having a static method inside a static class, the `this` keyword next to the first parameter determines whether it is an extension method.For example, we can build sophisticated libraries that are easy to use with a set of extension methods. Think `System.Linq` for such a system.Now that we learned the basics of dependency injection, there is one last thing to cover before revisiting the Strategy design pattern.

Using external IoC containers

ASP.NET Core provides an extensible built-in IoC container out of the box. It is not the most powerful IoC container because it lacks some advanced features, but it does the job for most applications.Rest assured; we can change it to another one if need be. You might also want to do that if you are used to another IoC container and want to stick to it.Here’s the strategy I recommend:

1. Use the built-in container, as per Microsoft's recommendation.
2. When you can’t achieve something with it, look at your design and see if you can redesign your feature to work with the built-in container and simplify your design.
3. If it is impossible to achieve your goal, see if extending the default container using an existing library or coding the feature yourself is possible.
4. If it is still impossible, explore swapping it for another IoC container.

Assuming the container supports it, it is super simple to swap. The third-party container must implement the `IServiceProviderFactory<TContainerBuilder>` interface. Then, in the `Program.cs` file, we must register that factory using the `UseServiceProviderFactory<TContainerBuilder>` method like this:

```
var builder = WebApplication.CreateBuilder(args);
builder.Host.UseServiceProviderFactory<ContainerBuilder>(new ContainerBuilderFactory());
```

In this case, the `ContainerBuilder` and `ContainerBuilderFactory` classes are just wrappers around ASP.NET Core, but your third-party container of choice should provide you with those types. I suggest you visit their documentation to know more.Once that factory is registered, we can configure the container using the `ConfigureContainer<TContainerBuilder>` method and register our dependencies as usual, like this:

```
builder.Host.ConfigureContainer<ContainerBuilder>((context, builder) =>
{
    builder.Services.AddSingleton<Dependency1>();
    builder.Services.AddSingleton<Dependency2>();
    builder.Services.AddSingleton<Dependency3>();
});
```

That's the only difference; the rest of the `Program.cs` file remains the same.As I sense you don't feel like implementing your own IoC container, multiple third-party integrations already exist. Here is a non-exhaustive list taken from the official documentation:

- Autofac
- DryIoc
- Grace
- LightInject
- Lamar
- Stashbox
- Simple Injector

On top of replacing the container entirely, some libraries extend the default container and add functionalities to it. We explore this option in *Chapter 11*, *Structural Patterns*.Now that we have covered most of the theory, we revisit the Strategy pattern as the primary tool to compose our applications and add flexibility to our systems.

Revisiting the Strategy pattern

In this section, we leverage the Strategy pattern to compose complex object trees and use DI to dynamically create those instances without using the `new` keyword, moving away from being control freaks and toward writing DI-ready code.The Strategy pattern is a behavioral design pattern we can use to compose object trees at runtime, allowing extra flexibility and control over objects' behavior. Composing our objects using the Strategy pattern makes our classes smaller, easier to test and maintain, and puts us on the SOLID path.From now on, we want to compose objects and lower the amount of inheritance to a minimum. We call that principle **composition over inheritance**. The goal is to inject dependencies (composition) into the current class instead of depending on base class features (inheritance). Additionally, this approach enables us to pull out behaviors and place them in separate classes, adhering to the Single Responsibility Principle (SRP) and Interface Segregation Principle (ISP). We can reuse these behaviors in one or more different classes through their interface, embodying the Dependency Inversion Principle (DIP). This strategy promotes code reuse and composition.The following list covers the most popular ways of injecting dependencies into objects, allowing us to control their behaviors from the outside by composing our objects:

- Constructor injection
- Property injection
- Method injection

We can also get dependencies directly from the container. This is known as the Service Locator (anti-)pattern. We explore the Service Locator pattern later in this chapter.

Let's look at some theory and then jump into the code to see DI in action.

Constructor injection

Constructor injection consists of injecting dependencies into the constructor as parameters. This is the most popular and preferred technique by far. Constructor injection is useful for injecting required dependencies; you can add null checks to ensure that, also known as the guard clause (see the *Adding a guard clause* section).

Property injection

The built-in IoC container does not support **property injection** out of the box. The concept is to inject **optional dependencies** into properties. Most of the time, you want to avoid doing this because property injection leads to optional dependencies, leading to nullable properties, more null checks, and often avoidable code complexity. So when we think about it, it is good that ASP.NET Core left this one out of the built-in container.You can usually remove the property injection requirements by reworking your design, leading to a better design. If you cannot avoid using property injection, use a third-party container or find a way to build the dependency tree yourself (maybe leveraging one of the Factory patterns).Nevertheless, from a high-level view, the container would do something like this:

1. Create a new instance of the class and inject all required dependencies into the constructor.
2. Find extension points by scanning properties (this could be attributes, contextual bindings, or something else).
3. For each extension point, inject (set) a dependency, leaving unconfigured properties untouched, hence its definition of an optional dependency.

There are a couple of exceptions to the previous statement regarding the lack of support:

- Razor components (Blazor) support property injection using the `[Inject]` attribute.
- Razor contains the `@inject` directive, which generates a property to hold a dependency (ASP.NET Core manages to inject it).

We can't call that property injection per se because they are not optional but required, and the `@inject` directive is more about generating code than doing DI. They are more about an internal workaround than "real" property injection. That is as close as .NET gets from property injection.

> I recommend aiming for constructor injection instead. Not having property injection should not cause you any problems. Often, our need for property injection stems from less-than-optimal design choices, whether from our design strategies or a framework we're utilizing.

Next, we look at method injection.

Method injection

ASP.NET Core supports method injection only at a few locations, such as in a controller's actions (methods), the `Startup` class (if you are using the pre-.NET 6 hosting model), and the middleware's `Invoke` or `InvokeAsync` methods. We cannot liberally use method injection in our classes without some work on our part.Method injection is also used to inject **optional dependencies** into classes. We can also validate those at runtime using null checks or any other required logic.

> **I recommend aiming for constructor injection whenever you can**. We should only resort to method injection when it's our sole option or when it brings added value to our design.
>
> > For example, in a controller, injecting a transient service in the only action that needs it instead of using constructor injection could save a lot of useless object instantiation and, by doing so, increase performance (less instantiation and less garbage collection). This can also lower the number of class-level dependencies a single class has.

Manually injecting a dependency in a method as an argument is valid. Here's an example, starting with the classes:

```
namespace CompositionRoot.ManualMethodInjection;
public class Subject
{
    public int Operation(Context context)
```

```
    {
        // ...
        return context.Number;
    }
}
public class Context
{
    public required int Number { get; init; }
}
```

The preceding code represents the `Subject` class that consumes an instance of the `Context` from its `Operation` method. It then returns the value of its `Number` property.

> This example follows a similar pattern to injecting an `HttpContext` into an endpoint delegate. In that case, the `HttpContext` represents the current HTTP request. In our case, the `Context` contains only an arbitrary number we use in the consuming code next.

To test that our code does as it should, we can write the following test:

```
[Fact]
public void Should_return_the_value_of_the_Context_Number_property()
{
    // Arrange
    var subject = new Subject();
    var context = new Context { Number = 44 };
    // Act
    var result = subject.Operation(context);
    // Assert
    Assert.Equal(44, result);
}
```

When we run the test, it works. We successfully injected the `context` into the `subject`. Now to simulate a more complex system, let's have a look at a theory that does the same more dynamically:

```
[Theory]
[MemberData(nameof(GetData))]
public void Showcase_manual_method_injection(
    Subject subject, Context context, int expectedNumber)
{
    // Manually injecting the context into the
    // Operation method of the subject.
    var number = subject.Operation(context);
    // Validate that we got the specified context.
    Assert.Equal(expectedNumber, number);
}
```

The preceding code showcases the same concept, but xUnit injects the dependencies into the method, which is closer to what would happen in a real program. Remember, we want to remove the `new` keywords from our life!

> The rest of the implementation is not important. I only pieced the simulation together to showcase this scenario. One interesting detail is that the `Subject` is always the same (singleton) while the `Context` is always different (transient), leading to a different outcome every time (`Context { Number = 0 }`, `Context { Number = 1 }`, and `Context { Number = 2 }`).

Having explored how to inject dependencies, we are ready to roll up our sleeves and dive into hands-on coding.

Project – Strategy

In the Strategy project, we delve into various methods of injecting dependencies, transitioning from the Control Freak approach to a SOLID one. Through this exploration, we evaluate the advantages and drawbacks of each technique.The project takes the form of a travel agency's location API, initially returning only hardcoded cities. We've implemented the same endpoint five times across different controllers to facilitate comparison and trace the progression. Each controller comes in pair except for one. The pairs comprise a base controller that uses an in-

memory service (dev) and an updated controller that simulates a SQL database (production). Here's the breakdown of each controller:

- The `ControlFreakLocationsController` instantiates the `InMemoryLocationService` class using the `new` keyword.
- The `ControlFreakUpdatedLocationsController` instantiates the `SqlLocationService` class and its dependency using the `new` keyword.
- The `InjectImplementationLocationsController` leverages constructor injection to get an instance of the `InMemoryLocationService` class from the container.
- The `InjectImplementationUpdatedLocationsController` leverages constructor injection to get an instance of the `SqlLocationService` class from the container.
- The `InjectAbstractionLocationsController` leverages dependency injection and interfaces to let its consumers change its behavior at runtime.

The controllers share the same building blocks; let's start there.

Shared building blocks

The `Location` data structure is the following:

```
namespace Strategy.Models;
public record class Location(int Id, string Name, string CountryCode);
```

The `LocationSummary` DTO returned by the controller is the following:

```
namespace Strategy.Controllers;
public record class LocationSummary(int Id, string Name);
```

The service interface is the following and has only one method that returns one or more `Location` objects:

```
using Strategy.Models;
namespace Strategy.Services;
public interface ILocationService
{
    Task<IEnumerable<Location>> FetchAllAsync(CancellationToken cancellationToken);
}
```

The two implementations of this interface are an in-memory version to use when developing and a SQL version to use when deploying (let's call this production to keep it simple).The in-memory service returns a predefined list of cities:

```
using Strategy.Models;
namespace Strategy.Services;
public class InMemoryLocationService : ILocationService
{
    public async Task<IEnumerable<Location>> FetchAllAsync(CancellationToken cancellationToken)
    {
        await Task.Delay(Random.Shared.Next(1, 100), cancellationToken);
        return new Location[] {
            new Location(1, "Paris", "FR"),
            new Location(2, "New York City", "US"),
            new Location(3, "Tokyo", "JP"),
            new Location(4, "Rome", "IT"),
            new Location(5, "Sydney", "AU"),
            new Location(6, "Cape Town", "ZA"),
            new Location(7, "Istanbul", "TR"),
            new Location(8, "Bangkok", "TH"),
            new Location(9, "Rio de Janeiro", "BR"),
            new Location(10, "Toronto", "CA"),
        };
    }
}
```

The SQL implementation uses an `IDatabase` interface to access the data:

```
using Strategy.Data;
using Strategy.Models;
namespace Strategy.Services;
public class SqlLocationService : ILocationService
{
    private readonly IDatabase _database;
    public SqlLocationService(IDatabase database) {
        _database = database;
    }
    public Task<IEnumerable<Location>> FetchAllAsync(CancellationToken cancellationToken) {
        return _database.ReadManyAsync<Location>(
            "SELECT * FROM Location",
            cancellationToken
        );
    }
}
```

That database access interface is simply the following:

```
namespace Strategy.Data;
public interface IDatabase
{
    Task<IEnumerable<T>> ReadManyAsync<T>(string sql, CancellationToken cancellationToken);
}
```

In the project itself, the `IDatabase` interface has only the `NotImplementedDatabase` implementation, which throws a `NotImplementedException` when its `ReadManyAsync` method is called:

```
namespace Strategy.Data;
public class NotImplementedDatabase : IDatabase
{
    public Task<IEnumerable<T>> ReadManyAsync<T>(string sql, CancellationToken cancellationToken)
        => throw new NotImplementedException();
}
```

> Since the goal is not learning database access, I mocked that part in a test case in a xUnit test using the controller and the `SqlLocationService` class.

With those shared pieces, we can start with the first two controllers.

Control Freak controllers

This first version of the code showcases the lack of flexibility that creating dependencies using the `new` keyword brings when the time to update the application arises. Here's the initial controller that leverages an in-memory collection:

```
using Microsoft.AspNetCore.Mvc;
using Strategy.Services;
namespace Strategy.Controllers;
[Route("travel/[controller]")]
[ApiController]
public class ControlFreakLocationsController : ControllerBase
{
    [HttpGet]
    public async Task<IEnumerable<LocationSummary>> GetAsync(CancellationToken cancellationToken)
    {
        var locationService = new InMemoryLocationService();
        var locations = await locationService
            .FetchAllAsync(cancellationToken);
        return locations
            .Select(l => new LocationSummary(l.Id, l.Name));
    }
}
```

Executing this code works and returns the `LocationSummary` equivalent of the `Location` objects returned by the `FetchAllAsync` method of the `InMemoryLocationService` class. However, changing the

`InMemoryLocationService` to a `SqlLocationService` is impossible without changing the code like this:

```
public class ControlFreakUpdatedLocationsController : ControllerBase
{
    [HttpGet]
    public async Task<IEnumerable<LocationSummary>> GetAsync(CancellationToken cancellationToken)
    {
        var database = new NotImplementedDatabase();
        var locationService = new SqlLocationService(database);
        var locations = await locationService.FetchAllAsync(cancellationToken);
        return locations.Select(l => new LocationSummary(l.Id, l.Name));
    }
}
```

The changes are highlighted in the two code blocks. We could also create an if statement to load one or the other conditionally, but exporting this to a whole system makes a lot of duplication.**Advantages:**

- It is easy to understand the code and what objects the controller uses.

Disadvantages:

- The controller is tightly coupled with its dependencies, leading to a lack of flexibility.
- Going from `InMemoryLocationService` to `SqlLocationService` requires updating the code.

Let’s improve on that design next with the next controller pair.

Injecting an implementation in the controllers

This second version of the codebase improves flexibility by leveraging dependency injection. In the following controller, we inject the `InMemoryLocationService` class in its constructor:

```
using Microsoft.AspNetCore.Mvc;
using Strategy.Services;
namespace Strategy.Controllers;
[Route("travel/[controller]")]
[ApiController]
public class InjectImplementationLocationsController : ControllerBase
{
    private readonly InMemoryLocationService _locationService;
    public InjectImplementationLocationsController(
        InMemoryLocationService locationService)
    {
        _locationService = locationService;
    }
    [HttpGet]
    public async Task<IEnumerable<LocationSummary>> GetAsync(CancellationToken cancellationToken)
    {
        var locations = await _locationService.FetchAllAsync(cancellationToken);
        return locations.Select(l => new LocationSummary(l.Id, l.Name));
    }
}
```

Assuming the **InMemoryLocationService** class is registered with the container, running this code would yield the same result as the Control Freak version and return the in-memory cities.

To register a class with the container, we can do the following:

```
builder.Services.AddSingleton<InMemoryLocationService>();
```

Unfortunately, to change that service for the `SqlLocationService`, we need to change the code again. This time, however, we must only change the constructor injection code like this:

```
public class InjectImplementationUpdatedLocationsController : ControllerBase
{
    private readonly SqlLocationService _locationService;
```

```
    public InjectImplementationUpdatedLocationsController(SqlLocationService locationService)
    {
        _locationService = locationService;
    }
    // ...
}
```

This is yet another not ideal outcome.**Advantages:**

- It is easy to understand the code and what objects the controller uses.
- Using constructor injection allows changing the dependency in one place, and all the methods get it (assuming we have more than one method).
- We can inject subclasses without changing the code.

Disadvantages:

- The controller is tightly coupled with its dependencies, leading to a lack of flexibility.
- Going from `InMemoryLocationService` to `SqlLocationService` requires updating the code.

We are getting there but still have a last step to make that controller flexible.

Injecting an abstraction in the controller

In this last controller, we leverage the SOLID principles, constructor injection, and, inherently, the Strategy pattern to build a controller that we can change from the outside. All we have to do to make the code flexible is inject the interface instead of its implementation, like this:

```
using Microsoft.AspNetCore.Mvc;
using Strategy.Services;
namespace Strategy.Controllers;
[Route("travel/[controller]")]
[ApiController]
public class InjectAbstractionLocationsController : ControllerBase
{
    private readonly ILocationService _locationService;
    public InjectAbstractionLocationsController(ILocationService locationService)
    {
        _locationService = locationService;
    }
    [HttpGet]
    public async Task<IEnumerable<LocationSummary>> GetAsync(CancellationToken cancellationToken)
    {
        var locations = await _locationService.FetchAllAsync(cancellationToken);
        return locations.Select(l => new LocationSummary(l.Id, l.Name));
    }
}
```

The highlighted lines showcase the changes. Injecting the `ILocationService` interface lets us control if we inject an instance of the `InMemoryLocationService` class, the `SqlLocationService` class, or any other implementation we'd like.This is the most flexible possibility we can get.**Advantages:**

- Using constructor injection allows changing the dependency in one place, and all the methods get it (assuming we have more than one method).
- Injecting the `ILocationService` interface allows us to inject any of its implementations without changing the code.
- Because of the `ILocationService` interface, the controller is loosely coupled with its dependencies.

Disadvantages:

- Understanding what objects the controller uses is harder since the dependencies are resolved at runtime. However, this forces us to program against an interface instead (a good thing).

Let’s have a look at this flexibility in action.

Constructing the InjectAbstractionLocationsController

I created a few xUnit tests to explore the possibilities, making it easy to create classes manually.

> I used Moq to mock implementations. If you are unfamiliar with Moq and want to learn more, I left a link in the *Further Reading* section.

Two of the tests refers to the following member, a static `Location` object:

```
public static Location ExpectedLocation { get; }
    = new Location(11, "Montréal", "CA");
```

The test cases are not to assess the correctness of our code but to explore how easy it is to compose the controller differently. Let’s explore the first test case.

Mock_the_IDatabase

The first is an integration test that injects an instance of the `SqlLocationService` class into the controller and mocks the database. The fake database returns a collection of one item. That item is the `Location` instance referenced by the `ExpectedLocation` property. Here’s that code:

```
var databaseMock = new Mock<IDatabase>();
databaseMock.Setup(x => x.ReadManyAsync<Location>(It.IsAny<string>(), It.IsAny<CancellationToken>
    .ReturnsAsync(() => new Location[] { ExpectedLocation })
;
var sqlLocationService = new SqlLocationService(
    databaseMock.Object);
var sqlController = new InjectAbstractionLocationsController(
    sqlLocationService);
```

The preceding code shows how we can control the dependency we inject into the classes because of how the `InjectAbstractionLocationsController` was designed. We can’t say the same about the four other controller versions.Next, we call the GetAsync method to verify that everything works as expected:

```
var result = await sqlController.GetAsync(CancellationToken.None);
```

Finally, let’s verify we received that collection of one object:

```
Assert.Collection(result,
    location =>
    {
        Assert.Equal(ExpectedLocation.Id, location.Id);
        Assert.Equal(ExpectedLocation.Name, location.Name);
    }
);
```

> Optionally, or instead, we could validate the service called the database mock, like this:

```
databaseMock.Verify(x => x
    .ReadManyAsync<Location>(
        It.IsAny<string>(),
        It.IsAny<CancellationToken>()
    ),
    Times.Once()
);
```

> There are a lot of useful features in the Moq library.

Validating that the code was correct is not important for this example. The key is to understand the composition of the controller, which the following diagram represents:

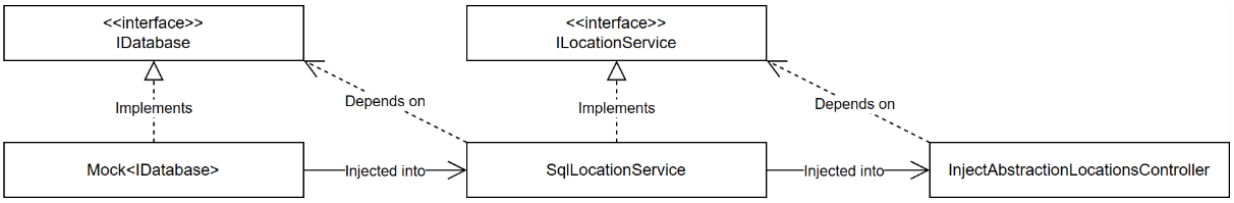


Figure 8.1: Composition of the controller in a test that mocks the IDatabase interface

As we can see from the diagram, the classes depend on interfaces, and we inject implementations when building them. The next two tests are simpler than this, only depending on the `ILocationService`. Let's explore the second one.

Use_the_InMemoryLocationService

Next, we use the in-memory location service to compose the controller like this:

```
var inMemoryLocationService = new InMemoryLocationService();
var devController = new InjectAbstractionLocationsController(
    inMemoryLocationService);
```

As we can see from the preceding code, we injected a different service into the controller, changing its behavior. This time, after calling the `GetAsync` method, the controller returned the ten `Location` objects from the `InMemoryLocationService`.The visual representation of our object tree is as follows:

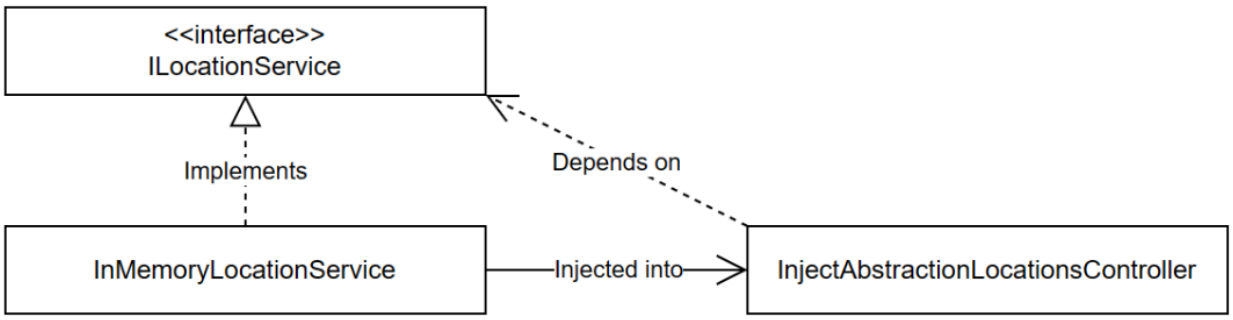


Figure 8.2: Composition of the controller in a test that injects an InMemoryLocationService instance.

It is harder to write assertions for the preceding test because we inject an instance of the `InMemoryLocationService` class, which ties the result to its implementation. For this reason, we won't look at that code here. Nonetheless, we succeeded at composing the controller differently. Let's have a look at the last test case.

Mock_the_ILocationService

The last unit test mocks the ILocationService directly. The mock service returns a collection of one item. That item is the `Location` instance referenced by the `ExpectedLocation` property. Here's that code:

```
var locationServiceMock = new Mock<ILocationService>();
locationServiceMock.Setup(x => x.FetchAllAsync(It.IsAny<CancellationToken>())).ReturnsAsync(() =>
var testController = new InjectAbstractionLocationsController(
    locationServiceMock.Object);
```

When executing the `GetAsync` method, we get the same result as in the first test case: a collection of a single test `Location` object. We can assert the correctness of the method by comparing values like this:

```
Assert.Collection(result,
    location =>
```

```
    {
        Assert.Equal(ExpectedLocation.Id, location.Id);
        Assert.Equal(ExpectedLocation.Name, location.Name);
    }
);
```

We can also leverage Moq to verify that the controller called the `FetchAllAsync` method using the following code:

```
locationServiceMock.Verify(x => x
    .FetchAllAsync(It.IsAny<CancellationToken>()),
    Times.Once()
);
```

The object tree of is very similar to the previous diagram but we faked the service implementation, making this a real unit test:

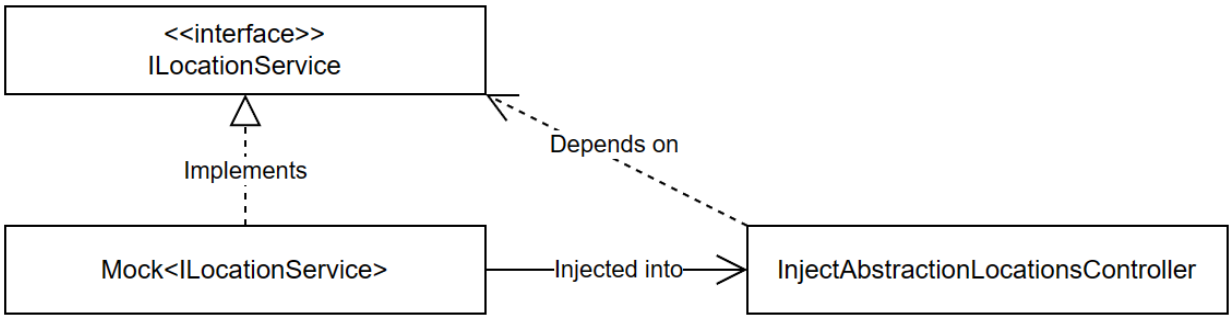


Figure 8.3: Composition of the controller in a test that mocks the ILocationService interface.

As we explored in this project, with the right design and dependency injection, we can easily compose different object trees using the same building blocks. However, with a bad design, it is hard to impossible to do so without altering the code.

> As you may have noticed, we used the `new` keyword in the controller to instantiate the DTO. DTOs are stable dependencies. We also explore object mappers in *Chapter 15*, *Object mappers, Aggregate Services, and Façade*, which is a way to encapsulate the logic of copying an object into another.

Let’s conclude before our next subject.

Conclusion

In this section, we saw that the strategy pattern went from a simple behavioral GoF pattern to the cornerstone of dependency injection. We explored different ways of injecting dependencies with a strong focus on constructor injection.Constructor injection is the most commonly used approach as it injects required dependencies, which we want the most. Method injection allows injecting algorithms, shared states, or contexts in a method that could not otherwise access that information. We can use property injection to inject optional dependencies, which should rarely happen.You can see optional dependencies as code smells because if the class has an optional role to play, it also has a primary role resulting in dual responsibilities. Moreover, if a role is optional, it could be better to move it to another class or rethink the system's design in that specific area.To practice what you just learned, you could connect the code sample to a real database, an Azure Table, Redis, a JSON file, or any other data source—tip: code classes that implement the `ILocationService` interface.

> As we covered, we can inject classes into other classes directly. There is nothing wrong with that. However, I suggest injecting interfaces as your initial approach until you are confident that you have mastered the different architectural principles and patterns covered in this book.

Next, we explore guard clauses.

Understanding guard clauses

A guard clause represents a condition the code must meet before executing a method. Essentially, it's a type of code that "guards" against continuing the execution of the method if certain conditions aren't met.In most cases, guard clauses are implemented at the beginning of a method to throw an exception early when the conditions necessary for the method's execution are not satisfied. Throwing an exception allows callers to catch the error without the need to implement a more complex communication mechanism.We already stated that we use constructor injection to inject the required dependencies reliably. However, nothing fully guarantees us that the dependencies are not `null`. Ensuring a dependency is not `null` is one of the most common guard clauses, which is trivial to implement. For example, we could check for nulls in the controller by replacing the following:

```
_locationService = locationService;
```

With the following:

```
_locationService = locationService ?? throw new ArgumentNullException(nameof(locationService));
```

The preceding code uses a `throw` expression from C# 7 (See *Appendix A* for more information). The `ArgumentNullException` type makes it evident that the `locationService` parameter is `null`. So if the `locationService` parameter is `null`, an `ArgumentNullException` is thrown; otherwise, the `locationService` parameter is assigned to the `_locationService` member.Of course, with the introduction of the nullable reference types (see *Appendix A*), receiving a `null` argument is less likely yet still possible.

> A built-in container will automatically throw an exception if it can’t fulfill all dependencies during the instantiation of a class (such as a controller). That does not mean that all third-party containers act the same.
>
> Moreover, that does not protect you from passing `null` to a class you manually instantiates, nor that a method will not receive a `null` value. I recommend adding guards even since they are less mandatory now. The tooling can handle most of the work for us, leading to only a minor time overhead.
>
> Furthermore, suppose you are writing code consumed by other projects, like a library. In that case, adding guards is more important since nothing guarantees that the consumers of that code have nullable reference type checks enabled.

When we need to validate a parameter and don't need an assignment, like with most parameters of a constructor, we can use the following helper, and the BCL handles the check for us:

```
ArgumentNullException.ThrowIfNull(locationService);
```

When we need to validate a string and want to ensure it is not empty, we can use the following instead:ArgumentException.ThrowIfNullOrEmpty(name);Of course, we can always revert to `if` statements to validate parameters. When doing so, we must ensure we throw relevant exceptions. If no pertinent exceptions exist, we can create one. Creating custom exceptions is a great way to write manageable applications.Next, we revisit an (anti-)pattern while exploring the singleton lifetime replacing it.

Revisiting the Singleton pattern

The Singleton pattern is obsolete, goes against the SOLID principles, and we replace it with a lifetime, as we’ve already seen. This section explores that lifetime and recreates the good old application state, which is nothing more than a singleton-scoped dictionary.We explore two examples: one about the application state, in case you were wondering where that feature disappeared to. Then, the Wishlist project also uses the singleton lifetime to provide an application-level feature. There are also a few unit tests to play with testability and to allow safe refactoring.

Project – Application state

You might remember the application state if you programmed ASP.NET using .NET Framework or the “good” old classic ASP with VBScript. If you don’t, the application state was a key/value dictionary that allowed you to store

data globally in your application, shared between all sessions and requests. That is one of the things that ASP always had and other languages, such as PHP, did not (or do not easily allow).For example, I remember designing a generic reusable typed shopping cart system with classic ASP/VBScript. VBScript was not a strongly typed language and had limited object-oriented capabilities. The shopping cart fields and types were defined at the application level (once per application), and then each user had their own “instance” containing the products in their “private shopping cart” (created once per session).In ASP.NET Core, there is no more `Application` dictionary. To achieve the same goal, you could use a static class or static members, which is not the best approach; remember that global objects (`static`) make your application harder to test and less flexible. We could also use the Singleton pattern or create an ambient context, allowing us to create an application-level instance of an object. We could even mix that with a factory to create end-user shopping carts, but we won’t; these are not the best solution either. Another way could be to use one of the ASP.NET Core caching mechanisms, memory cache, or distributed cache, but this is a stretch.We could also save everything in a database to persist the shopping cart between visits, but that is not related to the application state and requires more work, potentially a user account, so we will not do that either.We could save the shopping cart on the client-side using cookies, local storage, or any other modern mechanism to save data on the user’s computer. However, we’d get even further from the application state than using a database.For most cases requiring an application state-like feature, the best approach would be to create a standard class and an interface and then register the binding with a singleton lifetime in the container. Finally, you inject it into the component that needs it, using constructor injection. Doing so allows the mocking of dependencies and changing the implementations without touching the code but the composition root.

> Sometimes, the best solution is not the technically complex ones or design pattern-oriented; the best solution is often the simplest. Less code means less maintenance and fewer tests, resulting in a simpler application.

Let’s implement a small program that simulates the application state. The API is a single interface with two implementations. The program also exposes part of the API over HTTP, allowing users to get or set a value associated with the specified key. We use the singleton lifetime to ensure the data is shared between all requests.The interface looks like the following:

```
public interface IApplicationState
{
    TItem? Get<TItem>(string key);
    bool Has<TItem>(string key);
    void Set<TItem>(string key, TItem value) where TItem : notnull;
}
```

We can get the value associated with a key, associate a value with a key (set), and validate whether a key exists.The `Program.cs` file contains the code responsible for handling HTTP requests. We can swap the implementations by commenting or uncommenting the first line of the `Program.cs` file, which is `#define USE_MEMORY_CACHE` . That changes the dependency registration, as highlighted in the following code:

```
var builder = WebApplication.CreateBuilder(args);
#if USE_MEMORY_CACHE
        builder.Services.AddMemoryCache();
        builder.Services.AddSingleton<IApplicationState, ApplicationMemoryCache>();
#else
        builder.Services.AddSingleton<IApplicationState,
ApplicationDictionary>();
#endif
var app = builder.Build();
app.MapGet("/", (IApplicationState myAppState, string key) =>
{
    var value = myAppState.Get<string>(key);
    return $"{key} = {value ?? "null"}";
});
app.MapPost("/", (IApplicationState myAppState, SetAppState dto) =>
{
    myAppState.Set(dto.Key, dto.Value);
    return $"{dto.Key} = {dto.Value}";
});
app.Run();
public record class SetAppState(string Key, string Value);
```

Let’s now explore the first implementation.

First implementation

The first implementation uses the memory cache system, and I thought it would be educational to show that to you. Caching data in memory is something you might need to do sooner rather than later. However, we are hiding the cache system behind our implementation, which is also educational.Here is the `ApplicationMemoryCache` class:

```
public class ApplicationMemoryCache : IApplicationState
{
    private readonly IMemoryCache _memoryCache;
    public ApplicationMemoryCache(IMemoryCache memoryCache)
    {
        _memoryCache = memoryCache ?? throw new ArgumentNullException(nameof(memoryCache));
    }
    public TItem Get<TItem>(string key)
    {
        return _memoryCache.Get<TItem>(key);
    }
    public bool Has<TItem>(string key)
    {
        return _memoryCache.TryGetValue<TItem>(key, out _);
    }
    public void Set<TItem>(string key, TItem value)
    {
        _memoryCache.Set(key, value);
    }
}
```

Note

The `ApplicationMemoryCache` class is a thin wrapper over `IMemoryCache`, hiding the implementation details. Such a wrapper is similar to the Façade and Adapter patterns we explore in *Chapter 11*, *Structural Patterns*.

This simple class and two lines in our composition root make it an application-wide key-value store; done already! Let's now explore the second implementation.

Second implementation

The second implementation uses `ConcurrentDictionary<string, object>` to store the application state data and ensure thread safety, as multiple users could use the application state simultaneously. The `ApplicationDictionary` class is almost as simple as `ApplicationMemoryCache`:

```
using System.Collections.Concurrent;
namespace ApplicationState;
public class ApplicationDictionary : IApplicationState
{
    private readonly ConcurrentDictionary<string, object> _memoryCache = new();
    public TItem? Get<TItem>(string key)
    {
        return _memoryCache.TryGetValue(key, out var item)
            ? (TItem)item
            : default;
    }
    public bool Has<TItem>(string key)
    {
        return _memoryCache.TryGetValue(key, out var item) && item is TItem;
    }
    public void Set<TItem>(string key, TItem value)
        where TItem : notnull
    {
        _memoryCache.AddOrUpdate(key, value, (k, v) => value);
    }
}
```

The preceding code leverages the `TryGetValue` and `AddOrUpdate` methods to ensure thread safety while keeping the logic to a minimum and ensuring we avoid coding mistakes.

> Can you spot the flaw that might cause some problems in this design? See the solution at the end of the project section.

Let's explore how to use the implementations.

Using the implementations

We can now use any of the two implementations without impacting the rest of the program. That demonstrates the strength of DI when it comes to dependency management. Moreover, we control the lifetime of the dependencies from the composition root.If we were to use the `IApplicationState` interface in another class, say `SomeConsumer`, its usage could look similar to the following:

```
namespace ApplicationState;
public class SomeConsumer
{
    private readonly IApplicationState _myApplicationWideService;
    public SomeConsumer(IapplicationState myApplicationWideService)
    {
        _myApplicationWideService = myApplicationWideService ?? throw new ArgumentNullException(n
    }
    public void Execute()
    {
        if (_myApplicationWideService.Has<string>("some-key"))
        {
            var someValue = _myApplicationWideService.Get<string>("some-key");
            // Do something with someValue
        }
        // Do something else like:
        _myApplicationWideService.Set("some-key", "some-value");
        // More logic here
    }
}
```

In that code, `SomeConsumer` depends only on the `IApplicationState` interface, not `ApplicationDictionary` or `ApplicationMemoryCache`, and even less on `IMemoryCache` or `ConcurrentDictionary<string, object>`. Using DI allows us to hide the implementation by inverting the flow of dependencies. It also breaks direct coupling between concrete implementations. This approach also promotes programming against interfaces, as recommended by the Dependency Inversion Principle (DIP), and facilitates the creation of open-closed classes, in accordance with the Open/Closed Principle (OCP).Here is a diagram illustrating our application state system, making it visually easier to notice how it breaks coupling:

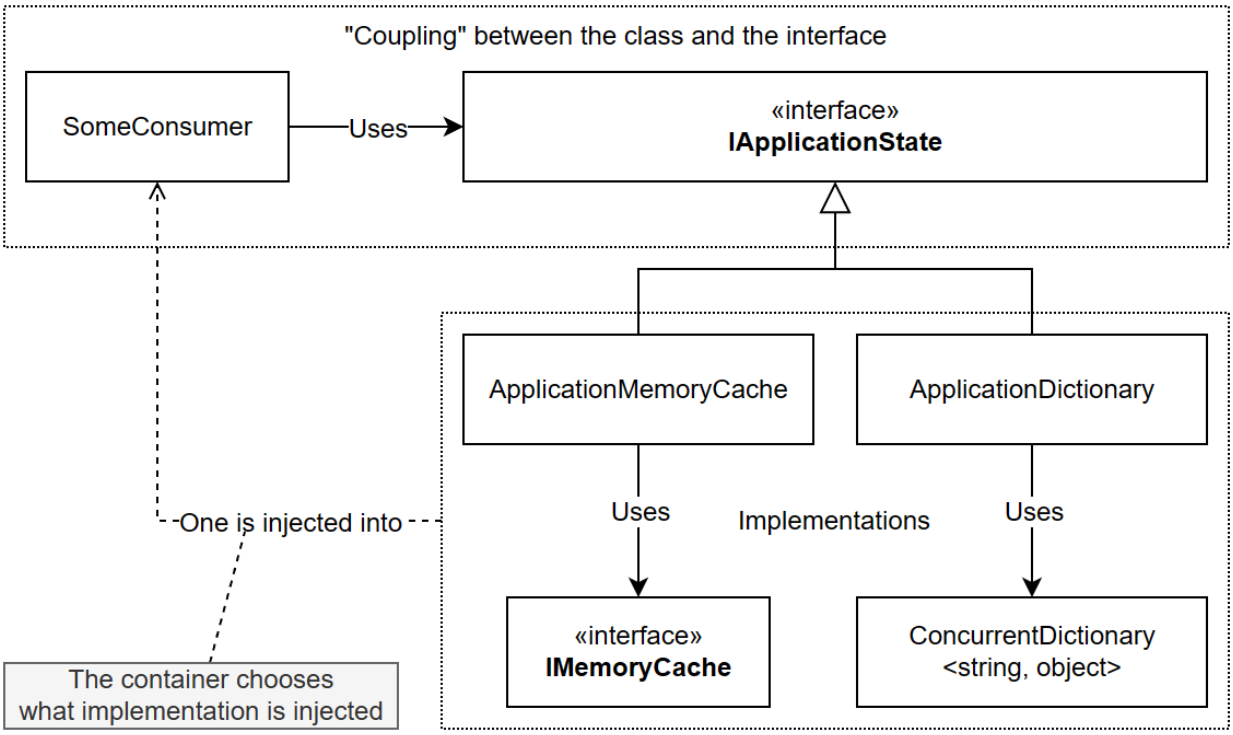


Figure 8.2: DI-oriented diagram representing the application state system

From this sample, let's remember that the singleton lifetime allows us to reuse objects between requests and share them application-wide. Moreover, hiding implementation details behind interfaces can improve the flexibility of our design.It is important to note that the singleton scope is only valid in a single process, so you can't rely purely on in-memory mechanisms for larger applications that span multiple servers. We could use the `IDistributedCache` interface to circumvent this limitation and persist our application state system to a persistent caching tool, like Redis.

The flaw: If we look closely at the `Has<TItem>` method, it returns true only when an entry is present for the specified key AND has the right type. So we could override an entry of a different type without knowing it exists.

For example, `ConsumerA` sets an item of type `A` for the key `K`. Elsewhere in the code, `ConsumerB` looks to see if an item of type `B` exists for the key `K`. The method returns `false` because it's a different type. `ConsumerB` overrides the value of the `K` with an object of type `B`. Here's the code representing this:

```
// Arrange
var sp = new ServiceCollection()
    .AddSingleton<IApplicationState, ApplicationDictionary>()
    .BuildServiceProvider()
;
// Step 1: Consumer A sets a string
var consumerA = sp.GetRequiredService<IApplicationState>();
consumerA.Set("K", "A");
Assert.True(consumerA.Has<string>("K")); // true
// Step 2: Consumer B overrides the value with an int
var consumerB = sp.GetRequiredService<IApplicationState>();
if (!consumerB.Has<int>("K")) // Oops, key K exists but it's of type string, not int
{
    consumerB.Set("K", 123);
}
Assert.True(consumerB.Has<int>("K")); // true
// Consumer A is broken!
Assert.False(consumerA.Has<string>("K")); // false
```

Improving the design to support such a scenario could be a good practice exercise. You could, for example, remove the `TItem` type from the `Has` method or, even better, allow storing multiple items under the same key, as long as their types are different.

Let's now explore the next project.

Project – Wishlist

Let's get into another sample to illustrate using the singleton lifetime and DI. Seeing DI in action should help understand it and then leverage it to create SOLID software.**Context**: The application is a site-wide wishlist where users can add items. Items expire every 30 seconds. When a user adds an existing item, the system must increment the count and reset the item's expiration time. That way, popular items stay on the list longer, making it to the top. When displayed, the system must sort the items by count (highest count first).

An expiration time of 30 seconds is very fast, but I'm sure you don't want to wait days before an item expires when running the app. It is a test config.

The program is a tiny web API that exposes two endpoints:

- Add an item to the wishlist (`POST`).
- Read the wishlist (`GET`).

The wishlist interface looks like this:

```
public interface IWishList
{
    Task<WishListItem> AddOrRefreshAsync(string itemName);
    Task<IEnumerable<WishListItem>> AllAsync();
}
public record class WishListItem(string Name, int Count, DateTimeOffset Expiration);
```

The two operations are there, and by making them async (returning a `Task<T>`), we could implement another version that relies on a remote system, such as a database, instead of an in-memory store. Then, the `WishListItem` record class is part of the `IWishList` contract; it is the model. To keep it simple, the wishlist only stores the names of items.

Note

Trying to foresee the future is not usually a good idea, but designing APIs to be awaitable is generally a safe bet. Other than this, I'd recommend you stick to the simplest code that satisfies the program's needs (KISS). When you try to solve problems that do not exist yet, you usually end up coding a lot of useless stuff, leading to additional unnecessary maintenance and testing time.

In the composition root, we must serve the `IWishList` implementation instance in a singleton scope (highlighted) so all requests share the same instance. Let's start with the first half of the `Program.cs` file:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services
    .ConfigureOptions<InMemoryWishListOptions>()
    .AddTransient<IValidateOptions<InMemoryWishListOptions>, InMemoryWishListOptions>()
    .AddSingleton(serviceProvider => serviceProvider.GetRequiredService<IOptions<InMemoryWishLis
    // The singleton registration
    .AddSingleton<IWishList, InMemoryWishList>()
;
```

If you are wondering where `IConfigureOptions` , `IValidateOptions` , and `IOptions` come from, we cover the ASP.NET Core Options pattern in *Chapter 9*, *Options*, *Settings*, *and Configuration*.

Let's now look at the second half of the `Program.cs` file that contains the minimal API code to handle the HTTP requests:

```
var app = builder.Build();
app.MapGet("/", async (IWishList wishList) =>
    await wishList.AllAsync());
app.MapPost("/", async (IWishList wishList, CreateItem? newItem) =>
{
    if (newItem?.Name == null)
    {
        return Results.BadRequest();
    }
    var item = await wishList.AddOrRefreshAsync(newItem.Name);
    return Results.Created("/", item);
});
app.Run();
public record class CreateItem(string? Name);
```

The `GET` endpoint delegates the logic to the injected `IWishList` implementation and returns the result, while the `POST` endpoint validates the `CreateItem` DTO before delegating the logic to the wishlist.To help us implement the `InMemoryWishList` class, we started by writing some tests to back our specifications up. Since static members are hard to configure in tests (remember globals?), I borrowed a concept from the ASP.NET Core memory cache and created an `ISystemClock` interface that abstracts away the static call to `DateTimeOffset.UtcNow` or `DateTime.UtcNow`.This way, we can program the value of `UtcNow` in our tests to create expired items. Here's the clock interface and implementation:

```
namespace Wishlist.Internal;
public interface ISystemClock
{
    DateTimeOffset UtcNow { get; }
}
public class SystemClock : ISystemClock
{
    public DateTimeOffset UtcNow => DateTimeOffset.UtcNow;
}
```

.NET 8 adds a new `TimeProvider` class to the `System` namespace, which does not help us much here. However, if we want to leverage that API, we could update the SystemClock to the following:

```
public class CustomClock : ISystemClock
{
    private readonly TimeProvider _timeProvider;
    public CustomClock(TimeProvider timeProvider)
    {
        _timeProvider = timeProvider ?? throw new ArgumentNullException(nameof(timeProvider));
    }
    public DateTimeOffset UtcNow => _timeProvider.GetUtcNow();
}
```

That code leverages the new API, but we'll stick to our simple implementation instead.

Let's look at the outline of the unit tests next because the whole code would take pages and be of low value:

```
namespace Wishlist;
public class InMemoryWishListTest
{
    // Constructor and private fields omitted
    public class AddOrRefreshAsync : InMemoryWishListTest
    {
        [Fact]
        public async Task Should_create_new_item();
        [Fact]
        public async Task Should_increment_Count_of_an_existing_item();
        [Fact]
        public async Task Should_set_the_new_Expiration_date_of_an_existing_item();
        [Fact]
        public async Task Should_set_the_Count_of_expired_items_to_1();
        [Fact]
        public async Task Should_remove_expired_items();
    }
    public class AllAsync : InMemoryWishListTest
```

```
    {
        [Fact]
        public async Task Should_return_items_ordered_by_Count_Descending();
        [Fact]
        public async Task Should_not_return_expired_items();
    }
    // Private helper methods omitted
}
```

The full source code is on GitHub: https://adpg.link/ywy8.

In the test class, we can mock the `ISystemClock` interface and program it to obtain the desired results based on each test case. We can also program some helper methods to make it easier to read the tests. Those helpers use tuples to return multiple values (see *Appendix A* for more information on language features). Here's the mock field:

```
private readonly Mock<ISystemClock> _systemClockMock = new();
```

Here's an example of such a helper method setting the clock to the present time and the `ExpectedExpiryTime` to a later time (`UtcNow + ExpirationInSeconds` later):

```
private (DateTimeOffset UtcNow, DateTimeOffset ExpectedExpiryTime) SetUtcNow()
{
    var utcNow = DateTimeOffset.UtcNow;
    _systemClockMock.Setup(x => x.UtcNow).Returns(utcNow);
    var expectedExpiryTime = utcNow.AddSeconds(_options.ExpirationInSeconds);
    return (utcNow, expectedExpiryTime);
}
```

Here is an example of another helper method setting the clock and the `ExpectedExpiryTime` to the past (two-time `ExpirationInSeconds` for the clock and once `ExpirationInSeconds` for the `ExpectedExpiryTime`):

```
private (DateTimeOffset UtcNow, DateTimeOffset ExpectedExpiryTime) SetUtcNowToExpired()
{
    var delay = -(_options.ExpirationInSeconds * 2);
    var utcNow = DateTimeOffset.UtcNow.AddSeconds(delay);
    _systemClockMock.Setup(x => x.UtcNow).Returns(utcNow);
    var expectedExpiryTime = utcNow.AddSeconds(_options.ExpirationInSeconds);
    return (utcNow, expectedExpiryTime);
}
```

We now have five tests covering the `AddOrRefreshAsync` method and two covering the `AllAsync` method. Now that we have those failing tests, here is the implementation of the `InMemoryWishList` class:

```
namespace Wishlist;
public class InMemoryWishList : IWishList
{
    private readonly InMemoryWishListOptions _options;
    private readonly ConcurrentDictionary<string, InternalItem> _items = new();
    public InMemoryWishList(InMemoryWishListOptions options)
    {
        _options = options ?? throw new ArgumentNullException(nameof(options));
    }
    public Task<WishListItem> AddOrRefreshAsync(string itemName)
    {
        var expirationTime = _options.SystemClock.UtcNow.AddSeconds(_options.ExpirationInSeconds
        _items
            .Where(x => x.Value.Expiration < _options.SystemClock.UtcNow)
            .Select(x => x.Key)
            .ToList()
            .ForEach(key => _items.Remove(key, out _))
        ;
        var item = _items.AddOrUpdate(
            itemName,
            new InternalItem(Count: 1, Expiration: expirationTime),
            (string key, InternalItem item) => item with {
                Count = item.Count + 1,
```

```
                Expiration = expirationTime
            }
        );
        var wishlistItem = new WishListItem(
            Name: itemName,
            Count: item.Count,
            Expiration: item.Expiration
        );
        return Task.FromResult(wishlistItem);
    }
    public Task<IEnumerable<WishListItem>> AllAsync()
    {
        var items = _items
            .Where(x => x.Value.Expiration >= _options.SystemClock.UtcNow)
            .Select(x => new WishListItem(
                Name: x.Key,
                Count: x.Value.Count,
                Expiration: x.Value.Expiration
            ))
            .OrderByDescending(x => x.Count)
            .AsEnumerable()
        ;
        return Task.FromResult(items);
    }
    private record class InternalItem(int Count, DateTimeOffset Expiration);
}
```

The `InMemoryWishList` class uses `ConcurrentDictionary<string, InternalItem>` internally to store the items and make the wishlist thread-safe. It also uses a `with` expression to manipulate and copy the `InternalItem` record class.The `AllAsync` method filters out expired items, while the `AddOrRefreshAsync` method removes expired items. This might not be the most advanced logic ever, but that does the trick.

> You might have noticed that the code is not the most elegant of all, and I left it this way on purpose. While using the test suite, I invite you to refactor the methods of the `InMemoryWishList` class to be more readable.
>
> > I took a few minutes to refactor it myself and saved it as `InMemoryWishListRefactored`. You can also uncomment the first line of `InMemoryWishListTest.cs` to test that class instead of the main one. My refactoring is a way to make the code cleaner, to give you ideas. It is not the only way, nor the best way, to write that class (the “best way” being subjective).
> >
> > Lastly, optimizing for readability and performance are often very different things.

Back to DI, the line that makes the wishlist shared between users is in the composition root we explored earlier. As a reference, here it is:

```
builder.Services.AddSingleton<IWishList, InMemoryWishList>();
```

Yes, only that line makes all the difference between creating multiple instances and a single shared instance. Setting the lifetime to Singleton allows you to open multiple browsers and share the wishlist.

> To `POST` to the API, I recommend using the `Wishlist.http` file in the project or the Postman collection (https://adpg.link/postman6) that comes with the book. The collection already contains multiple requests you can execute in batches or individually. You can also use the Swagger UI that I added to the project.

That’s it! All that code to demo what a single line of code in the composition root can do, and we have a working program, as tiny as it may be.

Conclusion

This section explored replacing the classic Singleton pattern with a standard instantiable class registered with a singleton lifetime. We looked at the old application state, learned that it was no more, and implemented two versions of it. We no longer need that, but it was a good way of learning about singletons.We then implemented a wishlist system as a second example. We concluded that the whole thing was working due to and managed by a

single line of the composition root: the call to the `AddSingleton` method. Changing that line could drastically change the system's behavior, making it unusable.From now on, you can see the Singleton pattern as an anti-pattern in .NET, and unless you find strong reasons to implement it, you should stick to normal classes and DI instead. Doing this moves the creation responsibility from the singleton class to the composition root, which is the composition root's responsibility, leaving the class only one responsibility.Next, we explore the Service Locator anti-pattern/code smell.

Understanding the Service Locator pattern

Service Locator is an anti-pattern that reverts the IoC principle to its Control Freak roots. The only difference is using the IoC container to build the dependency tree instead of the `new` keyword.There is some use of this pattern in ASP.NET, and we may argue that there are some reasons for using the Service Locator pattern, but it should happen rarely or never in most applications. For that reason, let's call the Service Locator pattern a **code smell** instead of an **anti-pattern**.My strong recommendation is *don't use the Service Locator pattern* unless you know you are not creating hidden coupling or have no other option. As a rule of thumb, you want to avoid injecting an `IServiceProvider` in your application's codebase. Doing so reverts to the classic flow of control and defeats the purpose of dependency injection.A good use of Service Locator could be to migrate a legacy system that is too big to rewrite. So you could build the new code using DI and update the legacy code using the Service Locator pattern, allowing both systems to live together or migrate one into the other, depending on your goal. Fetching dependencies dynamically is another potential use of the Service Locator pattern; we explore this in *Chapter 15*, *Object Mappers, Aggregate Services, and Façade*.Without further ado, let's jump into some more code.

Project – ServiceLocator

The best way to avoid something is to know about it, so let's see how to implement the Service Locator pattern using `IServiceProvider` to find a dependency.The service we want to use is an implementation of `IMyService`. Let's start with the interface:

```
namespace ServiceLocator;
public interface IMyService : IDisposable
{
    void Execute();
}
```

The interface inherits from the `IDisposable` interface and contains a single `Execute` method. Here is the implementation, which does nothing more than throw an exception if the instance has been disposed of (we'll leverage this later):

```
namespace ServiceLocator;
public class MyServiceImplementation : IMyService
{
    private bool _isDisposed = false;
    public void Dispose() => _isDisposed = true;
    public void Execute()
    {
        if (_isDisposed)
        {
            throw new NullReferenceException("Some dependencies have been disposed.");
        }
    }
}
```

Then, let's add a controller that implements the Service Locator pattern:

```
namespace ServiceLocator;
public class MyController : ControllerBase
{
    private readonly IServiceProvider _serviceProvider;
    public MyController(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider ?? throw new ArgumentNullException(nameof(serviceProv
    }
```

```
    [Route("/service-locator")]
    public IActionResult Get()
    {
        using var myService = _serviceProvider
            .GetRequiredService<IMyService>();
        myService.Execute();
        return Ok("Success!");
    }
}
```

In the preceding code, instead of injecting `IMyService` into the constructor, we are injecting `IServiceProvider`. Then, we use it (highlighted line) to locate the `IMyService` instance. Doing so shifts the responsibility for creating the object from the container to the consumer (`MyController` , in this case). `MyController` should not be aware of `IServiceProvider` and should let the container do its job without interference.What could go wrong? If we run the application and navigate to `/service-locator` , everything works as expected. However, if we reload the page, we get an error thrown by the `Execute()` method because we called `Dispose()` during the previous request. `MyController` should not control its injected dependencies, which is the point that I am trying to emphasize here: leave the container to control the lifetime of dependencies rather than trying to be a control freak. Using the Service Locator pattern opens pathways toward those wrong behaviors, which will likely cause more harm than good in the long run.Moreover, even though the ASP.NET Core container does not natively support this, we lose the ability to inject dependencies contextually when using the Service Locator pattern because the consumer controls its dependencies. What do I mean by contextually? Let's assume we have two classes, `A` and `B` , implementing interface `I` . We could inject an instance of `A` into `Consumer1` but an instance of `B` into `Consumer2` .Before exploring ways to fix this, here is the `Program.cs` code that powers this program:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services
    .AddSingleton<IMyService, MyServiceImplementation>()
    .AddControllers()
;
var app = builder.Build();
app.MapControllers();
app.Run();
```

The preceding code enables controller support and registers our service.To fix the controller, we must either remove the using statement or even better: move away from the Service Locator pattern and inject our dependencies instead. Of course, you are reading a dependency injection chapter, so I picked moving away from the Service Locator pattern. Here's what we are about to tackle:

- Method injection
- Constructor injection
- Minimal API

Let's start with method injection.

Implementing method injection

The following controller uses *method injection* instead of the Service Locator pattern. Here's the code that demonstrates this:

```
public class MethodInjectionController : ControllerBase
{
    [Route("/method-injection")]
    public IActionResult GetUsingMethodInjection([FromServices] IMyService myService)
    {
        ArgumentNullException.ThrowIfNull(myService, nameof(myService));
        myService.Execute();
        return Ok("Success!");
    }
}
```

Let's analyze the code:

- The `FromServicesAttribute` class tells the model binder about method injection.
- We added a guard clause to protect us from `null`.
- Finally, we kept the original code except for the `using` statement.

Method injection like this is convenient when a controller has multiple actions but only one uses the service.

Let’s reexplore constructor injection.

Implementing constructor injection

At this point, you should be familiar with constructor injection. Nonetheless, next is the controller's code after migrating the Service Locator pattern to constructor injection:

```
namespace ServiceLocator;
public class ConstructorInjectionController : ControllerBase
{
    private readonly IMyService _myService;
    public ConstructorInjectionController(IMyService myService)
    {
        _myService = myService ?? throw new ArgumentNullException(nameof(myService));
    }
    [Route("/constructor-injection")]
    public IActionResult GetUsingConstructorInjection()
    {
        _myService.Execute();
        return Ok("Success!");
    }
}
```

When using constructor injection, we ensure that `IMyService` is not `null` upon class instantiation. Since it is a class member, it is even less tempting to call its `Dispose()` method in an action method, leaving that responsibility to the container (as it should be).Let’s analyze the code before moving to the next possibility:

- We implemented the strategy pattern with constructor injection.
- We added a guard clause to ensure no `null` value could get in at runtime.
- We simplified the action to the bare minimum.

Both techniques are an acceptable replacement for the Service Locator pattern.

Implementing a minimal API

Of course, we can do the same with a minimal API. Here is the code of that endpoint:

```
app.MapGet("/minimal-api", (IMyService myService) =>
{
    myService.Execute();
    return "Success!";
});
```

That code does the same as the method injection sample without the guard clause that I omitted because no external consumer will likely inject nulls into it: the endpoint is a delegate passed directly to the `MapGet` method.Refactoring out the Service Locator pattern is often as trivial as this.

Conclusion

Most of the time, by following the Service Locator anti-pattern, we only hide that we are taking control of objects instead of decoupling our components. The code sample demonstrated a problem when disposing of an object, which could also happen using constructor injection. However, when thinking about it, it is more tempting to dispose of an object that we create than one we inject.Moreover, the service locator takes control away from the container and moves it into the consumer, against the **Open-Closed Principle**. You should be able to update the

consumer by updating the composition root's bindings.In the case of the sample code, we could change the binding, and it would work. In a more advanced case, binding two implementations to the same interface would be tough when contextual injection is required.

> The IoC container is responsible for weaving the program's thread, connecting its pieces together where each independent piece should be as clueless as possible about the others.

On top of that, the Service Locator pattern complicates testing. When unit testing your class, you must mock a container that returns a mocked service instead of mocking only the service.One place where I can see its usage justified is in the composition root, where bindings are defined, and sometimes, especially when using the built-in container, we can't avoid it to compensate for the lack of advanced features. Another good place would be a library that adds functionalities to the container. Other than that, try to stay away!

> **Beware**
>
> Moving the service locator elsewhere does not make it disappear; it only moves it around, like any dependency. However, moving it to the composition root can improve the maintainability of that code and remove the tight coupling.

Next, we revisit our third and final pattern of this chapter.

Revisiting the Factory pattern

A factory creates other objects; it is like a literal real-world factory. We explored in the previous chapter how to leverage the Abstract Factory pattern to create families of objects. A factory can be as simple as an interface with one or more `Create[Object]` methods or, even more, a simple delegate. We explore a DI-oriented simple factory in this section. We are building on top of the Strategy pattern example.In that example, we coded two classes implementing the `ILocationService` interface. The composition root used the `#define` preprocessor directive to tell the compiler what bindings to compile. In this version, we want to choose the implementation at runtime.

> Not compiling the code we don't need is good for many reasons, including security (lowering the attack surface). In this case, we are simply using an alternative strategy useful for many scenarios.

To achieve our new goal, we can extract the construction logic of the `ILocationService` interface into a factory.

Project – Factory

In the project, a copy from the Strategy project, we start by renaming the `InjectAbstractionLocationsController` class to `LocationsController`. We can then delete the other controllers.Now, we want to change the `ILocationService` bindings to reflect the following logic:

- When developing the application, we use the `InMemoryLocationService` class.
- When deploying to any environment, we must use the `SqlLocationService` class.

To achieve this, we use the `Environment` property of the `WebApplicationBuilder` object. That property of type `IWebHostEnvironment` contains some useful properties like the `EnvironmentName`, and .NET adds extension methods, like the `IsDevelopment` method that returns true when the `EnvironmentName` equals `Development`. Here's the `Program.cs` file code:

```
using Factory.Data;
using Factory.Services;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();
builder.Services.AddSingleton<ILocationService>(sp =>
{
    if (builder.Environment.IsDevelopment())
    {
        return new InMemoryLocationService();
    }
```

```
    return new SqlLocationService(new NotImplementedDatabase());
});
var app = builder.Build();
app.MapControllers();
app.Run();
```

The preceding code is fairly straightforward; it registers a delegate to act as a factory, which builds the appropriate service based on the ASP.NET Core `Environment`.

> We are using the `new` keyword here, but is this wrong? The composition root is where we should create or configure elements, so instantiating objects there is correct, as it is to use the Service Locator pattern. It is best to avoid the `new` keyword and the Service Locator pattern whenever possible, but using the default container makes it harder than with a full-featured third-party one. Nevertheless, we can avoid doing that in many cases, and even if we must use the `new` keyword and the Service Locator pattern, we often don't need a third-party container.

When we run the program, the right instance is injected into the controller based on the logic we added to the factory. The flow is similar to the following:

1. The application starts.
2. A client sends an HTTP request to the controller (`GET /travel/locations`).
3. ASP.NET Core creates the controller and leverages the IoC container to inject the `ILocationService` dependency.
4. Our factory creates the correct instance based on the current environment.
5. The action method runs, and the client receives the response.

We could also create a factory class and an interface, as explored in the previous chapter. However, in this case, it would likely just create noise.

> An essential thing to remember is that *moving code around your codebase does not make that code, logic, dependencies, or coupling disappear*. Coding a factory doesn't make all your design issues disappear. Moreover, adding more complexity adds a cost to your project, so factory or not, each time you try to break tight coupling or remove a dependency, ensure that you are not just moving the responsibility elsewhere or overengineering your solution.

Of course, to keep our composition root clean, we could create an extension method that does the registration, like an `AddLocationService` method. I'll leave you to try this one out, find other ways to improve the project, or even improve one of your own projects.The possibilities are almost endless when you think about the Factory patterns. Now that you've seen a few in action, you may find other uses for a factory when injecting some classes with complex instantiation logic into other objects.

Summary

This chapter delved into Dependency Injection, understanding its crucial role in crafting adaptable systems. We learned how DI applies the Inversion of Control principle, shifting dependency creation from the objects to the composition root. We explored the IoC container's role in object management, service resolution and injection, and dependency lifetime management. We tackled the Control Freak anti-pattern, advocating for dependency injection over using the `new` keyword.We revisited the Strategy pattern and explored how to use it with Dependency Injection to compose complex object trees. We learned about the principle of composition over inheritance, which encourages us to inject dependencies into the classes instead of relying on base class features and inheritance. We explored different ways of injecting dependencies into objects, including constructor injection, property injection, and method injection.We learned that a guard clause is a condition that must be met before a method is executed, often used to prevent null dependencies. We explored how to implement guard clauses. We also discussed the importance of adding guard clauses, as nullable reference type checks offer no guarantee at runtime.We revisited the Singleton pattern and how to replace it with a lifetime. We explored two examples utilizing the singleton lifetime to provide application-level features.We delved into the Service Locator pattern, often considered an anti-pattern, as it can create hidden coupling and revert the Inversion of Control principle. We learned that avoiding

using the Service Locator pattern is generally best. We explored how to implement the Service Locator pattern and discussed the potential issues that could arise. We revisited the Factory pattern and learned how to build a simple, DI-oriented factory that replaces the object creation logic of the IoC container.**Here are the key takeaways from this substantial chapter**:

- Dependency Injection is a technique applying the Inversion of Control principle for effective dependency management and lifetime control.
- An IoC container resolves and manages dependencies, offering varying control over object behavior.
- We can categorize dependencies into stable and volatile, the latter justifying DI.
- The lifetime of a service is Transient, Scoped, or Singleton.
- Dependency injection allows us to avoid the Control Freak anti-pattern and stop creating objects with the `new` keyword, improving flexibility and testability.
- The Service Locator pattern often creates hidden coupling and should be avoided but in the composition root.
- The composition root is where we register our service bindings with the IoC container; in the `Program.cs` file.
- Composing objects using the Strategy pattern alongside constructor injection facilitates handling complex object trees, emphasizing the principle of composition over inheritance.
- On top of constructor injection, there's also method injection and property injection, which are less supported. It is best to prioritize constructor injection over the others.
- Guard clauses safeguard method execution from unmet conditions.
- It is better to avoid the Singleton pattern in favor of binding a class and an interface with a singleton lifetime in the container.
- The Factory pattern is ideal for creating objects with complex instantiation logic.
- Moving code around doesn't eliminate dependencies or coupling; it's important not to overengineer solutions.

In subsequent sections, we explore tools that add functionalities to the default built-in container. Meanwhile, we explore options, settings, and configurations in the next chapter. These ASP.NET Core patterns aim to make our lives easier when managing such common problems.

Questions

Let's take a look at a few practice questions:

1. What are the three DI lifetimes that we can assign to objects in ASP.NET Core?
2. What is the composition root for?
3. Is it true that we should avoid the `new` keyword when instantiating volatile dependencies?
4. What is the pattern that we revisited in this chapter that helps compose objects to eliminate inheritance?
5. Is the Service Locator pattern a design pattern, a code smell, or an anti-pattern?
6. What is the principle of composition over inheritance?

Further reading

Here are some links to build upon what we have learned in the chapter:

- Moq: https://adpg.link/XZv8
- If you need more options, such as contextual injections, you can check out an open-source library I built. It adds support for new scenarios: https://adpg.link/S3aT
- Official documentation, Default service container replacement: https://adpg.link/5ZoG

Answers

1. Transient, Scoped, Singleton.
2. The composition root holds the code that describes how to compose the program's object graph—the types bindings.
3. Yes, it is true. Volatile dependencies should be injected instead of instantiated.

4. The Strategy pattern.
5. The Service Locator pattern is all three. It is a design pattern used by DI libraries internally but becomes a code smell in application code. If misused, it is an anti-pattern with the same drawbacks as using the `new` keyword directly.
6. The principle of composition over inheritance encourages us to inject dependencies into classes and use them instead of relying on base class features and inheritance. This approach promotes flexibility and code reuse. It also negates the need for the LSP.

9 Options, Settings, and Configuration

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter covers the .NET Options pattern, a building block of any application. .NET Core introduced new predefined mechanisms to enhance the usage of application settings available to ASP.NET Core applications. These allow us to divide our configuration into multiple smaller objects, configure them during various stages of the startup flow, validate them, and even watch for runtime changes with minimal effort.

> The new options system repurposed the `ConfigurationManager` class as an internal piece. We no longer can use it as the old .NET Framework-era static methods are gone. The new patterns and mechanisms help avoid useless coupling, add flexibility to our designs, and are DI-native. The system is also simpler to extend.

The Options pattern goal is to use settings at runtime, allowing changes to the application to happen without changing the code. The settings could be as simple as a `string`, a `bool`, a database connection string, or a complex object that holds an entire subsystem's configuration.This chapter delves into various tools and methodologies we can use for managing, injecting, and loading configurations and options into our ASP.NET Core applications. Our journey spans a broad spectrum of scenarios, covering everything from commonly encountered to more complex use cases.At the end of the chapter, you will know how to leverage the .NET options and settings infrastructure.In this chapter, we cover the following topics:

- Loading the configuration
- Learning the building blocks
- Exploring common usage scenarios
- Learning options configuration
- Validating our options objects
- Validating options using FluentValidation
- Injecting options objects directly—a workaround
- Centralizing the configuration for easier management
- Using the configuration-binding source generator
- Using the options validation source generator
- Using the options validation source generator

Let's get started!

Loading the configuration

ASP.NET Core allows us to load settings from multiple sources seamlessly. We can customize these sources from the `WebApplicationBuilder`, or use the defaults set by calling the `WebApplication.CreateBuilder(args)` method.The default sources, in order, are as follows:

1. `appsettings.json`

2. `appsettings.{Environment}.json`
3. User secrets; these are only loaded when the environment is `Development`
4. Environment variables
5. Command-line arguments

The order is essential, as the last to be loaded overrides previous values. For example, you can set a value in `appsettings.json` and override it in `appsettings.Staging.json` by redefining the value in that file, user secrets, an environment variable or by passing it as a command-line argument when you run your application.

> You can name your environments as you want, but by default, ASP.NET Core has built-in helper methods for `Development`, `Staging`, and `Production`.

On top of the default providers, we can register other configuration sources out of the box, like `AddIniFile`, `AddInMemoryCollection`, and `AddXmlFile`, for example. We can also load NuGet packages to install custom providers, like Azure KeyVault and Azure App Configuration, to centralize secrets and configuration management into the Azure cloud. The most interesting part of those configuration providers is that no matter the sources, it does not affect the consumption of the settings, only the composition root. This means we can start loading settings one way, then change our mind later or have different strategies for dev and prod, and none of that affects the codebase but the composition root.We explore a few building blocks next.

Learning the building blocks

There are four main interfaces to use settings: `IOptionsMonitor<TOptions>`, `IOptionsFactory<TOptions>`, `IOptionsSnapshot<TOptions>`, and `IOptions<TOptions>`. We must inject that dependency into a class to use the available settings. `TOptions` is the type that represents the settings that we want to access.The framework returns an empty instance of your options class if you don't configure it. We learn how to configure options properly in the next subsection; meanwhile, remember that using property initializers inside your options class can also be a great way to ensure certain defaults are used. You can also use constants to centralize those defaults somewhere in your codebase (making them easier to maintain). Nevertheless, proper configuration and validation are always preferred, but both combined can add a safety net. Don't use initializers or constants for default values that change based on the environment (dev, staging, or production) or for secrets such as connection strings and passwords.

> You should always keep secrets out of your Git history, whether it's out of the C# code or out of setting files. Use ASP.NET Core secrets locally and a secret store like Azure KeyVault for Staging and Production environments.

If we create the following class, since the default value of an `int` is `0`, the default number of items to display per page would be 0, leading to an empty list.

```
public class MyListOption
{
    public int ItemsPerPage { get; set; }
}
```

However, we can configure this using a property initializer, as next:

```
public class MyListOption
{
    public int ItemsPerPage { get; set; } = 20;
}
```

The default number of items to display per page is now 20.

> In the source code for this chapter, I've included a few tests in the `CommonScenarios.Tests` project that assert the lifetime of the different options interfaces. I haven't included this code here for brevity, but it describes the behavior of the different options via unit tests. See https://adpg.link/AXa5 for more information.

The options served by each interface have different DI lifetimes and other features. The following table exposes some of those features:

Interface	Lifetime	Support named options	Support change notification
`IOptionsMonitor<TOptions>`	Singleton	Yes	Yes
`IOptionsFactory<TOptions>`	Transient	Yes	No
`IOptionsSnapshot<TOptions>`	Scoped	Yes	No
`IOptions<TOptions>`	Singleton	No	No

Table 9.1: The different options interfaces, their DI lifetime, and support for other features.

Next, we explore those interfaces more in-depth.

IOptionsMonitor<TOptions>

This interface is the most versatile of them all:

- It supports receiving notifications about reloading the configuration (like when the setting file changed).
- It supports caching.
- It supports named configuration (identifying multiple different `TOptions` with a name).
- The injected `IOptionsMonitor<TOptions>` instance is always the same (**singleton lifetime**).
- It supports unnamed default settings through its `Value` property.

> If we only configure named options or no instance at all, the consumer will receive an empty `TOptions` instance (`new TOptions()`).

IOptionsFactory<TOptions>

This interface is a factory, as we saw in *Chapter 7*, *Strategy, Abstract Factory, and Singleton*, and in *Chapter 8*, *Dependency Injection*, we use factories to create instances; this interface is no different.

> Unless necessary, I suggest sticking with `IOptionsMonitor<TOptions>` or `IOptionsSnapshot<TOptions>` instead.

How the factory works is simple: the container creates a new factory every time you ask for one (transient lifetime), and the factory creates a new options instance every time you call its `Create(name)` method (**transient lifetime**).To get the default instance (non-named options), you can use the `Options.DefaultName` field or pass an empty string; this is usually handled for you by the framework.

> If we only configure named options or no instance at all, the consumer will receive an empty `TOptions` instance (`new TOptions()`) after calling `factory.Create(Options.DefaultName)`.

IOptionsSnapshot<TOptions>

This interface is useful when you need a snapshot of the settings for the duration of an HTTP request.

- The container creates only one instance per request (**scoped lifetime**).
- It supports named configuration.
- It supports unnamed default settings through its `CurrentValue` property.

> If we only configure named options or no instance at all, the consumer will receive an empty `TOptions` instance (`new TOptions()`).

IOptions<TOptions>

This interface is the first that was added to ASP.NET Core.

- It does not support advanced scenarios such as what snapshots and monitors do.
- Whenever you request an `IOptions<TOptions>` instance, you get the same instance (**singleton lifetime**).

`IOptions<TOptions>` does not support named options, so you can only access the default instance.

Now that we looked at the building blocks, we dig into some code to explore leveraging those interfaces.

Project – CommonScenarios

This first example covers multiple basic use cases, such as injecting options, using named options, and storing options values in settings.Let's start with the shared building block.

Manual configuration

In the composition root, we can manually configure options, which is very useful for configuring ASP.NET Core MVC, the JSON serializer, other pieces of the framework, or our own handcrafted options.Here's the first options class we use in the code, which contains only a `Name` property:

```
namespace CommonScenarios;
public class MyOptions
{
    public string? Name { get; set; }
}
```

In the composition root, we can use the `Configure` extension method that extends the `IServiceCollection` interface to achieve this. Here's how we can set the default options of the `MyOptions` class:

```
builder.Services.Configure<MyOptions>(myOptions =>
{
    myOptions.Name = "Default Option";
});
```

With that code, if we inject that options instance into a class, the value of the `Name` property will be `Default Options`.We explore loading settings from a non-hardcoded configuration source next.

Using the settings file

Loading configurations from a file is often more convenient than hardcoding the values in C#. Moreover, the mechanism allows overriding the configurations using different sources, bringing even more advantages.To load `MyOptions` from the `appsettings.json` file, we must first get the configuration section, then configure the options, like the following:

```
var defaultOptionsSection = builder.Configuration
    .GetSection("defaultOptions");
builder.Services
    .Configure<MyOptions>(defaultOptionsSection);
```

The preceding code loads the following data from the appsettings.json file:

```
{
  "defaultOptions": {
    "name": "Default Options"
  }
}
```

The **defaultOptions** section maps to objects with the same key in the JSON file (highlighted code). The `name` property of the `defaultOptions` section translates to the `Name` property of the `MyOptions` class.That code does the same as the preceding hardcoded version. However, manually loading the section this way allows us to load a different section for different named options.Alternatively, we can also "bind" a configuration section to an existing object using the `Bind` method like this:

```
var options = new MyOptions();
builder.Configuration.GetSection("options1").Bind(options);
```

That code loads the settings and assigns them to the object's properties, matching the settings key to the properties name. However, this does not add the object to the IoC container.To overcome this, if we do not want to register the dependency manually and don't need the object, we can use the `Bind` or `BindConfiguration` methods from the `OptionsBuilder<TOptions>`. We create that object with the `AddOptions` method, like for `Bind`:

```
builder.Services.AddOptions<MyOptions>("Options3")
    .Bind(builder.Configuration.GetSection("options3"));
```

The preceding code loads the `options3` configuration section using the `GetSection` method (highlighted), then the `OptionsBuilder<TOptions>` binds that value to the name `Options3` through the `Bind` method. This registers a named instance of `MyOptions` with the container. We dig into named options later.Then again, we can skip the use of the `GetSection` method by using the `BindConfiguration` method instead, like this:

```
builder.Services.AddOptions<MyOptions>("Options4")
    .BindConfiguration("options4");
```

The preceding code loads the settings from the `options4` section, then registers that new setting with the IoC container.These are just a subset of the different ways we can leverage the ASP.NET Core Options pattern and configuration system. Now that we know how to configure the options, it is time to use them.

Injecting options

Let's start by learning how to leverage the `IOptions<TOptions>` interface, the first and simplest interface that came out of .NET Core.To try this out, let's create an endpoint and inject the `IOptions<MyOptions>` interface as a parameter:

```
app.MapGet(
    "/my-options/",
    (IOptions<MyOptions> options) => options.Value
);
```

In the preceding code, the `Value` property returns the configured value, which is the following, serialized as JSON:

```
{
  "name": "Default Options"
}
```

And voilà! We can also use constructor injection or any other method we know to use the value of our options object.Next, we explore configuring multiple instances of the same options class.

Named options

Now, let's explore named options by configuring two more instances of the `MyOptions` class. The concept is to associate a configuration of the options with a name. Once that is done, we can request the configuration we need.

> Unfortunately, the ways we explore named options and most online examples break the Inversion of Control principle.
>
> > Why? By injecting an interface that is directly tied to a lifetime, the consuming class controls that part of the dependency.
> >
> > Rest assured, we are revisiting this at the end of the chapter.

First, in the `appsettings.json` file, let's add the highlighted sections:

```
{
  "defaultOptions": {
```

```
    "name": "Default Options"
  },
  "options1": {
    "name": "Options 1"
  },
  "options2": {
    "name": "Options 2"
  }
}
```

Now that we have those configs, let's configure them in the `Program.cs` file by adding the following lines:

```
builder.Services.Configure<MyOptions>(
    "Options1",
    builder.Configuration.GetSection("options1")
);
builder.Services.Configure<MyOptions>(
    "Options2",
    builder.Configuration.GetSection("options2")
);
```

In the preceding code, the highlighted strings represent the names of the options we are configuring. We associate each configuration section with a named instance.Now to consume those named options, we have multiple choices. We can inject an `IOptionsFactory<MyOptions>`, `IOptionsMonitor<MyOptions>`, or an `IOptionsSnapshot<MyOptions>` interface. The final choice depends on the lifetime the consumer of the options needs. However, in our case, we use all of them to ensure we explore them all.

IOptionsFactory<MyOptions>

Let's start with creating an endpoint where we inject a factory:

```
app.MapGet(
    "/factory/{name}",
    (string name, IOptionsFactory<MyOptions> factory)
        => factory.Create(name)
);
```

The factory interface forces us to pass in a name that is convenient for us. When we execute the program, the endpoint serves us the options based on the specified name. For example, when we send the following request:

```
GET https://localhost:8001/factory/Options1
```

The endpoint returns the following JSON:

```
{
  "name": "Options 1"
}
```

If we pass Options2 instead, we get the following JSON:

```
{
  "name": "Options 2"
}
```

As simple as that, we can now choose between three different options. Of course, once again, we can leverage any other technique we know, like constructor injections.Let's explore the next interface.

IOptionsMonitor<MyOptions>

We use the `IOptionsMonitor` interface similarly to the `IOptionsFactory` interface when we need named options. So, let's start by creating a similar endpoint:

```
app.MapGet(
    "/monitor/{name}",
```

```
    (string name, IOptionsMonitor<MyOptions> monitor)
        => monitor.Get(name)
);
```

The preceding code is almost the same as the factory one, but the `IOptionsMonitor` interface exposes a `Get` method instead of a `Create` method. This semantically expresses that the code is getting an options instance (singleton) instead of creating a new one (transient).Again, similarly, if we send the following request:

```
GET https://localhost:8001/monitor/Options2
```

The server returns the following JSON:

```
{
  "name": "Options 2"
}
```

One difference is that we can access the default options as well; here's how:

```
app.MapGet(
    "/monitor",
    (IOptionsMonitor<MyOptions> monitor)
        => monitor.CurrentValue
);
```

In the preceding code, the `CurrentValue` property returns the default options. So, when calling this endpoint, we should receive the following JSON:

```
{
  "name": "Default Options"
}
```

As simple as that, we can either access the default value or a named value. We explore one other scenario that the `IOptionsMonitor` interface supports after we cover the `IOptionsSnapshot` interface next.

IOptionsSnapshot<MyOptions>

The `IOptionsSnapshot` interface inherits the `IOptions` interface, contributing its `Value` property, and also offers a `Get` method (scoped lifetime) that works like the `IOptionsMonitor` interface.Let's start with the first endpoint:

```
app.MapGet(
    "/snapshot",
    (IOptionsSnapshot<MyOptions> snapshot)
        => snapshot.Value
);
```

It should be no surprise that the preceding endpoint returns the following default options:

```
{
  "name": "Default Options"
}
```

Then the following parametrized endpoint returns the specified named options:

```
app.MapGet(
    "/snapshot/{name}",
    (string name, IOptionsSnapshot<MyOptions> snapshot)
        => snapshot.Get(name)
);
```

Say we are passing the name `Options1`, then the endpoint will return the following options:

```
{
  "name": "Options 1"
```

```
}
```

And we are done. It is quite simple to use the options as .NET does most of the work for us. The same goes for configuring options classes.But wait, our exploration isn't over yet! Up next, we delve into the process of reloading options at runtime.

Reloading options at runtime

A fascinating aspect of the ASP.NET Core options is that the system reloads the value of the options when someone updates a configuration file like `appsettings.json`. To try it out, you can:

1. Run the program.
2. Query an endpoint using the request available in the `CommonScenarios.http` file.
3. Change the value of that option in the `appsettings.json` file and save the file.
4. Query the same endpoint again, and you should see the updated value.

This is an out-of-the-box feature. However, the system rebuilds the options instance, which does not update the references on the previous instance. The good news is that we can hook into the system and react to the changes.For most scenarios, we don’t need to manually check for change since the value of the `CurrentValue` property gets updated. However, if you directly reference that value, this mechanism can be useful.In this scenario, we have a notification service that sends emails. The SMTP client’s configurations are settings. In this case, we only have the `SenderEmailAddress` since sending actual emails is unnecessary. We are logging the notification in the console instead, allowing us to see the configuration changes appear live.Let’s start with the `EmailOptions` class:

```
namespace CommonScenarios.Reload;
public class EmailOptions
{
    public string? SenderEmailAddress { get; set; }
}
```

Next, we have the `NotificationService` class itself. Let’s start with its first iteration:

```
namespace CommonScenarios.Reload;
public class NotificationService
{
    private EmailOptions _emailOptions;
    private readonly ILogger _logger;
    public NotificationService(IOptionsMonitor<EmailOptions> emailOptionsMonitor, ILogger<Notifi
    {
        _logger = logger ?? throw new ArgumentNullException(nameof(logger));
        ArgumentNullException.ThrowIfNull(emailOptionsMonitor);
        _emailOptions = emailOptionsMonitor.CurrentValue;
    }
    public Task NotifyAsync(string to)
    {
        _logger.LogInformation(
            "Notification sent by '{SenderEmailAddress}' to '{to}'.",
            _emailOptions.SenderEmailAddress,
            to
        );
        return Task.CompletedTask;
    }
}
```

In the preceding code, the class holds a reference on the `EmailOptions` class upon creation (highlighted lines). The `NotifyAsync` method writes an information message in the console and then returns.

We explore logging in the next chapter.

Because the `NotificationService` class has a singleton lifetime and references the options class itself, if we change the configuration, the value will not update since the system recreates a new instance with the updated configuration. Here’s the service registration method:

```
public static WebApplicationBuilder AddNotificationService(
    this WebApplicationBuilder builder)
{
    builder.Services.Configure<EmailOptions>(builder.Configuration
        .GetSection(nameof(EmailOptions)));
    builder.Services.AddSingleton<NotificationService>();
    return builder;
}
```

How to fix this? In this case, we could fix the issue by referencing the `IOptionsMonitor` interface instead of its `CurrentValue` property. However, if you face a scenario where it's impossible, we can tap into the `OnChange` method of the `IOptionsMonitor` interface. In the constructor, we could add the following code:

```
emailOptionsMonitor.OnChange((options) =>_emailOptions = options);
```

With that code, when the `appsettings.json` file changes, the code updates the `_emailOptions` field. As easy as this, we reactivated the reloading feature.

> One more thing, the `OnChange` method returns an `IDisposable` we can dispose of to stop listening for changes. I implemented two additional methods in the source code: `StartListeningForChanges` and `StopListeningForChanges` , and three endpoints, one to send notifications, one to stop listening for changes, and one to start listening for changes again.

Now that we know how to use the options, let's explore additional ways to configure them.

Project – OptionsConfiguration

Now that we have covered basic usage scenarios, let's attack some more advanced possibilities, such as creating types to configure, initialize, and validate our options.We start by configuring options which happen in two phases:

1. The configuration phase.
2. The post-configuration phase.

In a nutshell, the post-configuration phase happens later in the process. This is a good place to enforce that some values are configured a certain way or to override configuration, for example, in integration tests.To configure an options class, we have many options, starting with the following interfaces:

Interface	**Description**
`IConfigureOptions<TOptions>`	Configure the default `TOptions` type.
`IConfigureNamedOptions<TOptions>`	Configure the default and named `TOptions` type.
`IPostConfigureOptions<TOptions>`	Configure the default and named `TOptions` type during the post-configuration phase.

Table 9.2: interfaces to configure options classes.

> If a configuration class implements both `IConfigureOptions` and `IConfigureNamedOptions` interfaces, the `IConfigureNamedOptions` interface will take precedence, and the `Configure` method of the `IConfigureOptions` interface will not be executed.
>
> > You can configure the default instance using the `Configure` method of the `IConfigureNamedOptions` interface; the name of the options will be empty (equal to the member `Options.DefaultName`).

We can also leverage the following methods that extend the `IServiceCollection` interface:

Method	**Description**
`Configure<TOptions>`	Configure the default and named `TOptions` type inline or from a configuration section.
`ConfigureAll<TOptions>`	Configure all options of type `TOptions` inline.

`PostConfigure<TOptions>`	Configure the default and named `TOptions` type inline during the post-configuration phase.
`PostConfigureAll<TOptions>`	Configure all options of type `TOptions` inline during the post-configuration phase.

Table 9.3: configuration methods.

As we are about to see, the registration order is very important. The configurators are executed in order of registration. Each phase is independent of the other; thus, the sequence in which we arrange the configuration and post-configuration phases doesn't influence one another.First, we must lay out the groundwork for our little program.

Creating the program

After creating an empty web application, the first building block is to create the options class that we want to configure:

```
namespace OptionsConfiguration;
public class ConfigureMeOptions
{
    public string? Title { get; set; }
    public IEnumerable<string> Lines { get; set; } = Enumerable.Empty<string>();
}
```

We use the `Lines` property as a trace bucket. We add lines to it to visually confirm the order that the configurators are executed.Next, we define application settings in the `appsettings.json` file:

```
{
  "configureMe": {
    "title": "Configure Me!",
    "lines": [
      "appsettings.json"
    ]
  }
}
```

We use the configuration as a starting point. It defines the value of the `Title` property and adds a first line to the `Lines` property, allowing us to trace the order it is executed.Next, we need an endpoint to access the settings, serialize the result to a JSON string, and then write it to the response stream:

```
app.MapGet(
    "/configure-me",
    (IOptionsMonitor<ConfigureMeOptions> options) => new {
        DefaultInstance = options.CurrentValue,
        NamedInstance = options.Get(NamedInstance)
    }
);
```

By calling this endpoint, we can consult the values of the default and named instances we are about to create.

> ASP.NET Core configures the options when they are requested for the first time. In this case, both instances of the `ConfigureMeOptions` class are configured when calling the `/configure-me` endpoint for the first time.

If we run the program now, we end up with two empty instances, so before doing that, we need to tell ASP.NET about the `configureMe` configuration section we added to the `appsettings.json` file.

Configuring the options

We want two different options to test out many possibilities:

- A default options (unnamed)

- A named instance.

To achieve this, we must add the following lines in the `Program.cs` file:

```
const string NamedInstance = "MyNamedInstance";
builder.Services
    .Configure<ConfigureMeOptions>(builder.Configuration
        .GetSection("configureMe"))
    .Configure<ConfigureMeOptions>(NamedInstance, builder.Configuration
        .GetSection("configureMe"))
;
```

The preceding code registers a default instance (highlighted code) and a named instance. Both use the `configureMe` configuration sections and so start with the same initial values, as we can see when running the project:

```
{
  "defaultInstance": {
    "title": "Configure Me!",
    "lines": [
      "appsettings.json"
    ]
  },
  "namedInstance": {
    "title": "Configure Me!",
    "lines": [
      "appsettings.json"
    ]
  }
}
```

The `defaultInstance` and `namedInstance` properties are self-explanatory and relate to their respective options instance.Now that we completed our building blocks, we are ready to explore the `IConfigureOptions<TOptions>` interface.

Implementing a configurator object

We can encapsulate the configuration logic into classes to apply the single responsibility principle (SRP). To do so, we must implement an interface and create the binding with the IoC container.First, we must create a class that we name `ConfigureAllConfigureMeOptions` , which configures all `ConfigureMeOptions` instances; default and named:

```
namespace OptionsConfiguration;
public class ConfigureAllConfigureMeOptions : IConfigureNamedOptions<ConfigureMeOptions>
{
    public void Configure(string? name, ConfigureMeOptions options)
    {
        options.Lines = options.Lines.Append(
            $"ConfigureAll:Configure name: {name}");
        if (name != Options.DefaultName)
        {
            options.Lines = options.Lines.Append(
                $"ConfigureAll:Configure Not Default: {name}");
        }
    }
    public void Configure(ConfigureMeOptions options)
        => Configure(Options.DefaultName, options);
}
```

In the preceding code, we implement the interface (highlighted code), which contains two methods. The second `Configure` method should never be called, but just in case, we can simply redirect the call to the other method if it happens. The body of the first `Configure` method (highlighted) adds a line to all options and a second line when the options is not the default one.

Instead of testing if the options is not the default one (`name != Options.DefaultName`), you can check for the options name or use a `switch` to configure specific options by name.

We can tell the IoC container about this code, so ASP.NET Core executes it like this:

```
builder.Services.AddSingleton<IConfigureOptions<ConfigureMeOptions>, ConfigureAllConfigureMeOpti
```

Now with this binding in place, ASP.NET Core will run our code the first time we request our endpoint. Here's the result:

```
{
  "defaultInstance": {
    "title": "Configure Me!",
    "lines": [
      "appsettings.json",
      "ConfigureAll:Configure name: "
    ]
  },
  "namedInstance": {
    "title": "Configure Me!",
    "lines": [
      "appsettings.json",
      "ConfigureAll:Configure name: MyNamedInstance",
      "ConfigureAll:Configure Not Default: MyNamedInstance"
    ]
  }
}
```

As we can see from that JSON output, the configurator ran and added the expected lines to each instance.

It is important to note that you must bind the `IConfigureOptions<TOptions>` to your configuration class even if you implemented the `IConfigureNamedOptions<TOptions>` interface.

And voilà—we have a neat result that took almost no effort. This can lead to so many possibilities! Implementing `IConfigureOptions<TOptions>` is probably the best way to configure the default values of an options class.Next, we add post-configuration to the mix!

Adding post-configuration

We must take a similar path to add post-configuration values but implement the `IPostConfigureOptions<TOptions>` instead. To achieve this, we update the `ConfigureAllConfigureMeOptions` class to implement that interface:

```
namespace OptionsConfiguration;
public class ConfigureAllConfigureMeOptions :
    IPostConfigureOptions<ConfigureMeOptions>,
    IConfigureNamedOptions<ConfigureMeOptions>
{
    // Omitted previous code
    public void PostConfigure(string? name, ConfigureMeOptions options)
    {
        options.Lines = options.Lines.Append(
            $"ConfigureAll:PostConfigure name: {name}");
    }
}
```

In the preceding code, we implemented the interface (highlighted lines). The `PostConfigure` method simply adds a line to the `Lines` property. To register it with the IoC container, we must add the following line:

```
builder.Services.AddSingleton<IPostConfigureOptions<ConfigureMeOptions>, ConfigureAllConfigureMe(
```

The big difference is that this runs during the post-configuration phase, independent of the initial configuration phase. Executing the application now leads to the following result:

```
{
  "defaultInstance": {
    "title": "Configure Me!",
    "lines": [
      "appsettings.json",
      "ConfigureAll:Configure name: ",
      "ConfigureAll:PostConfigure name: "
    ]
  },
  "namedInstance": {
    "title": "Configure Me!",
    "lines": [
      "appsettings.json",
      "ConfigureAll:Configure name: MyNamedInstance",
      "ConfigureAll:Configure Not Default: MyNamedInstance",
      "ConfigureAll:PostConfigure name: MyNamedInstance"
    ]
  }
}
```

In the preceding JSON, the highlighted lines represent our post-configuration code that was added at the end. You might tell yourself, of course, it's the last line; it's the last code we registered, which is a legitimate assumption. However, here's the complete registration code, which clearly shows the `IPostConfigureOptions<TOptions>` interface was registered first (highlighted), proving the post-configuration code runs last:

```
builder.Services
    .AddSingleton<IPostConfigureOptions<ConfigureMeOptions>, ConfigureAllConfigureMeOptions>()
    .Configure<ConfigureMeOptions>(builder.Configuration
        .GetSection("configureMe"))
    .Configure<ConfigureMeOptions>(NamedInstance, builder.Configuration
        .GetSection("configureMe"))
    .AddSingleton<IConfigureOptions<ConfigureMeOptions>, ConfigureAllConfigureMeOptions>()
;
```

Next, we create a second configuration class.

Using multiple configurator objects

A very interesting concept with the ASP.NET Core options pattern is that we can register as many configuration classes as we want. This creates many possibilities, including code from one or more assemblies configuring the same options class.Now that we know how this works, let's add the `ConfigureMoreConfigureMeOptions` class, which also adds a line to the `Lines` property:

```
namespace OptionsConfiguration;
public class ConfigureMoreConfigureMeOptions : IConfigureOptions<ConfigureMeOptions>
{
    public void Configure(ConfigureMeOptions options)
    {
        options.Lines = options.Lines.Append("ConfigureMore:Configure");
    }
}
```

This time, we want that class only to augment the default instance, so it implements the **IConfigureOptions<TOptions>** interface (highlighted lines).Next, we must register the binding:

```
builder.Services.AddSingleton<IConfigureOptions<ConfigureMeOptions>, ConfigureMoreConfigureMeOpt
```

As we can see, it's the same binding but pointing to the `ConfigureMoreConfigureMeOptions` class instead of the `ConfigureAllConfigureMeOptions` class.Executing the application and querying the endpoint outputs the following JSON:

```
{
  "defaultInstance": {
    "title": "Configure Me!",
    "lines": [
      "appsettings.json",
```

```
      "ConfigureAll:Configure name: ",
      "ConfigureMore:Configure",
      "ConfigureAll:PostConfigure name: "
    ]
  },
  "namedInstance": {
    "title": "Configure Me!",
    "lines": [
      "appsettings.json",
      "ConfigureAll:Configure name: MyNamedInstance",
      "ConfigureAll:Configure Not Default: MyNamedInstance",
      "ConfigureAll:PostConfigure name: MyNamedInstance"
    ]
  }
}
```

The preceding JSON shows the line our new class added to only the default instance (highlighted) before the post-configure option.The possibilities are great, right? The code can contribute configuration objects and register them in one of the two phases to configure options objects. Next, we explore a few more possibilities.

Exploring other configuration possibilities

We can mix those configuration classes with extension methods. For example:

- We can call the `Configure` and `PostConfigure` methods multiple times.
- We can call the `ConfigureAll` and `PostConfigureAll` methods to configure all the options of a given `TOptions`.

Here, we use the `PostConfigure` method to demonstrate that. Let's add the following two lines of code (highlighted):

```
const string NamedInstance = "MyNamedInstance";
var builder = WebApplication.CreateBuilder(args);
builder.Services.PostConfigure<ConfigureMeOptions>(
    NamedInstance,
    x => x.Lines = x.Lines.Append("Inline PostConfigure Before")
);
builder.Services
    .AddSingleton<IPostConfigureOptions<ConfigureMeOptions>, ConfigureAllConfigureMeOptions>()
    .Configure<ConfigureMeOptions>(builder.Configuration
        .GetSection("configureMe"))
    .Configure<ConfigureMeOptions>(NamedInstance, builder.Configuration
        .GetSection("configureMe"))
    .AddSingleton<IConfigureOptions<ConfigureMeOptions>, ConfigureAllConfigureMeOptions>()
    //.AddSingleton<IConfigureNamedOptions<ConfigureMeOptions>, ConfigureAllConfigureMeOptions>(
    .AddSingleton<IConfigureOptions<ConfigureMeOptions>, ConfigureMoreConfigureMeOptions>()
;
builder.Services.PostConfigure<ConfigureMeOptions>(
    NamedInstance,
    x => x.Lines = x.Lines.Append("Inline PostConfigure After")
);
// ...
```

The preceding code registers two configuration delegates that target our named instance. They both run in the post-configuration phase. So running the app and accessing the endpoint shows the order in which all lines are added:

```
{
  "defaultInstance": {
    "title": "Configure Me!",
    "lines": [
      "appsettings.json",
      "ConfigureAll:Configure name: ",
      "ConfigureMore:Configure",
      "ConfigureAll:PostConfigure name: "
    ]
  },
  "namedInstance": {
```

```
    "title": "Configure Me!",
    "lines": [
      "appsettings.json",
      "ConfigureAll:Configure name: MyNamedInstance",
      "ConfigureAll:Configure Not Default: MyNamedInstance",
      "Inline PostConfigure Before",
      "ConfigureAll:PostConfigure name: MyNamedInstance",
      "Inline PostConfigure After"
    ]
  }
}
```

In the preceding JSON, we can see that the two highlighted lines are the ones we just added, loaded in order, and not applied to the default options.

There is one more possibility, which comes from the validation API. This is most likely an unintended side effect, but it works nonetheless.

The following code adds the `"Inline Validate"` line after the post-configuration phase:

```
builder.Services.AddOptions<ConfigureMeOptions>().Validate(options =>
{
    // Validate was not intended for this, but it works nonetheless...
    options.Lines = options.Lines.Append("Inline Validate");
    return true;
});
```

On the separation of concerns aspect, we should stay away from this. However, knowing this may help you work around a post-configuration order issue one day.

Now that we know the options interface types, their lifetimes, and many ways to configure their values, it is time to validate them and enforce a certain level of integrity in our programs.

Project – OptionsValidation

Another feature that comes out of the box is options validation, which allows us to run validation code when a `TOptions` object is created. The validation code is guaranteed to run the first time an option is created and does not account for subsequent options modifications. Depending on the lifetime of your options object, the validation may or may not run. For example:

Interface	Lifetime	Validation
`IOptionsMonitor<TOptions>`	Singleton	Validate the options once.
`IOptionsFactory<TOptions>`	Transient	Validate the options every time the code calls the `Create` method.
`IOptionsSnapshot<TOptions>`	Scoped	Validate the options once per HTTP request (per scope).
`IOptions<TOptions>`	Singleton	Validate the options once.

Table 9.4: the effect of validation on options lifetime.

I wrote three test cases in the `ValidateLifetime.cs` file if you are interested to see this in action.

We can create validation types to validate options classes. They must implement the `IValidateOptions<TOptions>` interface or use data annotations such as `[Required]`. Implementing the interface works very similarly to the options configuration.First, let’s see how to force the validation when the program starts.

Eager validation

Eager validation has been added to .NET 6 and allows catching incorrectly configured options at startup time in a fail-fast mindset.The `Microsoft.Extensions.Hosting` assembly adds the `ValidateOnStart` extension method

to the `OptionsBuilder<TOptions>` type.There are different ways of using this, including the following, which binds a configuration section to an options class:

```
services.AddOptions<Options>()
    .Configure(o => /* Omitted configuration code */)
    .ValidateOnStart()
;
```

The highlighted line is all we need to apply our validation rules during startup. I recommend using this as your new default so you know that options are misconfigured at startup time instead of later at runtime, limiting unexpected issues.Now that we know that, let's look at how to configure options validation.

Data annotations

Let's start by using `System.ComponentModel.DataAnnotations` types to decorate our options with validation attributes. We activate this feature with the `ValidateDataAnnotations` extension method. This also works with eager validation by chaining both methods.

> If you are unfamiliar with `DataAnnotations` , they are attributes used to validate EF Core and MVC model classes. Don't worry, they are very explicit, so you should understand the code.

To demonstrate this, let's look at the skeleton of two small tests:

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Options;
using System.ComponentModel.DataAnnotations;
using Xunit;
namespace OptionsValidation;
public class ValidateOptionsWithDataAnnotations
{
    [Fact]
    public void Should_pass_validation() { /*omitted*/ }
    [Fact]
    public void Should_fail_validation() { /*omitted*/ }
    private class Options
    {
        [Required]
        public string? MyImportantProperty { get; set; }
    }
}
```

The preceding code shows that the `MyImportantProperty` property of the `Options` class is required and cannot be `null` (highlighted line). Next, we look at the test cases.The first test is expecting the validation to pass:

```
[Fact]
public void Should_pass_validation()
{
    // Arrange
    var services = new ServiceCollection();
    services.AddOptions<Options>()
        .Configure(o => o.MyImportantProperty = "A value")
        .ValidateDataAnnotations()
        .ValidateOnStart() // eager validation
    ;
    var serviceProvider = services.BuildServiceProvider();
    var options = serviceProvider
        .GetRequiredService<IOptionsMonitor<Options>>();
    // Act & Assert
    Assert.Equal(
        "Some important value",
        options.CurrentValue.MyImportantProperty
    );
}
```

The test simulates the execution of a program where the IoC container creates the options class, and its consumer (the test) leverages it. The highlighted line sets the property to `"A value"`, making the validation pass. The code also enables eager validation (`ValidateOnStart`) on top of the validation of data annotations (`ValidateDataAnnotations`).The second test is expecting the validation to fail:

```
[Fact]
public void Should_fail_validation()
{
    // Arrange
    var services = new ServiceCollection();
    services.AddOptions<Options>()
        .ValidateDataAnnotations()
        .ValidateOnStart() // eager validation
    ;
    var serviceProvider = services.BuildServiceProvider();
        // Act & Assert
        var error = Assert.Throws<OptionsValidationException>(
            () => options.CurrentValue);
        Assert.Collection(error.Failures,
            f => Assert.Equal("DataAnnotation validation failed for 'Options' members: 'MyImporta
        );
    );
}
```

In the preceding code, the `MyImportantProperty` is never set (highlighted code), leading to the validation failing and throwing an `OptionsValidationException` . The test simulates catching that exception.

> The eager validation does not work in the tests because it is not an ASP.NET Core program but xUnit test cases (Fascts).

That's it—.NET does the job for us and validates our instance of the `Options` class using the data annotation like you can do when using EF Core or MVC model.Next, we explore how to create validation classes to validate our options objects manually.

Validation types

To implement options validation types or options validators, we can create a class that implements one or more `IValidateOptions<TOptions>` interfaces. One type can validate multiple options, and multiple types can validate the same options, so the possible combinations should cover all use cases.Using a custom class is no harder than using data annotations. However, it allows us to remove the validation concerns from the options class and code more complex validation logic. You should pick the way that makes the most sense for your project.

> On top of personal preferences, say you use a third-party library with options. You load that library into your application and expect the configuration to be a certain way. You could create a class to validate that the options class provided by the library is configured appropriately for your application and even validate this at startup time.
>
> > You can't use data annotations for that because you don't control the code. Moreover, it is not a general validation that should apply to all consumers but specific validation for that one app.

Let's start with the skeleton of the test class:

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Options;
using Xunit;
namespace OptionsValidation;
public class ValidateOptionsWithTypes
{
    [Fact]
    public void Should_pass_validation() {}
    [Fact]
    public void Should_fail_validation() {}
    private class Options
```

```
    {
        public string? MyImportantProperty { get; set; }
    }
    private class OptionsValidator : IValidateOptions<Options>
    {
        public ValidateOptionsResult Validate(
            string name, Options options)
        {
            if (string.IsNullOrEmpty(options.MyImportantProperty))
            {
                return ValidateOptionsResult.Fail(
                    "'MyImportantProperty' is required.");
            }
            return ValidateOptionsResult.Success;
        }
    }
}
```

In the preceding code, we have the `Options` class that is similar to the previous example but without the data annotation. The difference is that instead of using the `[Required]` attribute, we created the `OptionsValidator` class (highlighted) containing the validation logic. `OptionsValidator` implements `IValidateOptions<Options>`, which only contains a `Validate` method. This method allows named and default options to be validated. The `name` argument represents the options' names. In our case, we implemented the required logic for all options. The `ValidateOptionsResult` class exposes a few members to help us, such as the `Success` and `Skip` fields, and two `Fail()` methods. `ValidateOptionsResult.Success` indicates success. `ValidateOptionsResult.Skip` indicates that the validator did not validate the options, most likely because it only validates certain named options but not the given one.The `ValidateOptionsResult.Fail(message)` and `ValidateOptionsResult.Fail(messages)` methods take a single message or a collection of messages as an argument.To make this work, we must make the validator available to the IoC container, as we did with the options configuration. We explore the two test cases next, which are very similar to the data annotation example.Here's the first test case that passes the validation:

```
[Fact]
public void Should_pass_validation()
{
    // Arrange
    var services = new ServiceCollection();
    services.AddSingleton<IValidateOptions<Options>, OptionsValidator>();
    services.AddOptions<Options>()
        .Configure(o => o.MyImportantProperty = "A value")
        .ValidateOnStart()
    ;
    var serviceProvider = services.BuildServiceProvider();
    // Act & Assert
    var options = serviceProvider
        .GetRequiredService<IOptionsMonitor<Options>>();
    Assert.Equal(
        "A value",
        options.CurrentValue.MyImportantProperty
    );
}
```

The test case simulates an application that configures the `MyImportantProperty` correctly, which passes validation. The highlighted line shows how to register the validator class. The rest is done by the framework when using the options class.Next, we explore a test that fails the validation:

```
[Fact]
public void Should_fail_validation()
{
    // Arrange
    var services = new ServiceCollection();
    services.AddSingleton<IValidateOptions<Options>, OptionsValidator>();
    services.AddOptions<Options>().ValidateOnStart();
    var serviceProvider = services.BuildServiceProvider();
    // Act & Assert
    var options = serviceProvider
        .GetRequiredService<IOptionsMonitor<Options>>();
    var error = Assert.Throws<OptionsValidationException>(
```

```
        () => options.CurrentValue);
    Assert.Collection(error.Failures,
        f => Assert.Equal("'MyImportantProperty' is required.", f)
    );
}
```

The test simulates a program where the `Options` class is not configured appropriately. When accessing the options object, the framework builds the class and validates it, throwing an `OptionsValidationException` because of the validation rules (highlighted lines).Using types to validate options is handy when you don't want to use data annotations, can't use data annotations, or need to implement certain logic that is easier within a method than with attributes.Next, we glance at how to leverage options with FluentValidation.

Project – OptionsValidationFluentValidation

In this project, we validate options classes using FluentValidation. FluentValidation is a popular open-source library that provides a validation framework different from data annotations. We explore FluentValidation more in *Chapter 15*, *Getting Started with Vertical Slice Architecture*, but that should not hinder you from following this example.Here, I want to show you how to leverage a few patterns we've learned so far to implement this ourselves with only a few lines of code. In this micro-project, we leverage:

- Dependency injection
- The Strategy design pattern
- The Options pattern
- Options validation: validation types
- Options validation: eager validation

Let's start with the options class itself:

```
public class MyOptions
{
    public string? Name { get; set; }
}
```

The options class is very thin, containing only a nullable `Name` property. Next, let's look at the FluentValidation validator, which validates that the `Name` property is not empty:

```
public class MyOptionsValidator : AbstractValidator<MyOptions>
{
    public MyOptionsValidator()
    {
        RuleFor(x => x.Name).NotEmpty();
    }
}
```

If you have never used FluentValidation before, the `AbstractValidator<T>` class implements the `IValidator<T>` interface and adds utility methods like `RuleFor`. The `MyOptionsValidator` class contains the validation rules.To make ASP.NET Core validate `MyOptions` instances using FluentValidation, we implement an `IValidateOptions<TOptions>` interface as we did in the previous example, inject our validator in it, and then leverage it to ensure the validity of `MyOptions` objects. This implementation of the `IValidateOptions` interface creates a bridge between the FluentValidation features and the ASP.NET Core options validation.Here is a generic implementation of such a class that could be reused for any type of options:

```
public class FluentValidateOptions<TOptions> : IValidateOptions<TOptions>
    where TOptions : class
{
    private readonly IValidator<TOptions> _validator;
    public FluentValidateOptions(IValidator<TOptions> validator)
    {
        _validator = validator;
    }
    public ValidateOptionsResult Validate(string name, TOptions options)
    {
```

```
        var validationResult = _validator.Validate(options);
        if (validationResult.IsValid)
        {
            return ValidateOptionsResult.Success;
        }
        var errorMessages = validationResult.Errors.Select(x => x.ErrorMessage);
        return ValidateOptionsResult.Fail(errorMessages);
    }
}
```

In the preceding code, the `FluentValidateOptions<TOptions>` class adapts the `IValidateOptions<TOptions>` interface to the `IValidator<TOptions>` interface by leveraging FluentValidation in the `Validate` method. In a nutshell, we use the output of one system and make it an input of another system.

> This type of adaptation is known as the Adapter design pattern. We explore the Adapter pattern in the next chapter.

Now that we have all the building blocks, let's have a look at the composition root:

```
using FluentValidation;
using Microsoft.Extensions.Options;
var builder = WebApplication.CreateBuilder(args);
builder.Services
    .AddSingleton<IValidator<MyOptions>, MyOptionsValidator>()
    .AddSingleton<IValidateOptions<MyOptions>, FluentValidateOptions<MyOptions>>()
;
builder.Services
    .AddOptions<MyOptions>()
    .ValidateOnStart()
;
var app = builder.Build();
app.MapGet("/", () => "Hello World!");
app.Run();
```

The highlighted code is the key to this system:

- It registers the FluentValidation `MyOptionsValidator` that contains the validation rules.
- It registers the generic `FluentValidateOptions` instance, so .NET uses it to validate `MyOptions` class.
- Under the hood, The `FluentValidateOptions` class uses the `MyOptionsValidator` to validate the options internally.

When running the program, the console yields the following error, as expected:

```
Hosting failed to start
Unhandled exception. Microsoft.Extensions.Options.OptionsValidationException: 'Name' must not be
[...]
```

This may look like a lot of trouble for a simple required field; however, the `FluentValidateOptions<TOptions>` is reusable. We could also scan one or more assemblies to register the validator with the IoC container automatically.Now that we've explored many ways to configure and validate options objects, it is time to look at a way to inject options classes directly, either by choice or to work around a library capability issue.

Workaround – Injecting options directly

The only negative point about the .NET Options pattern is that we must tie our code to the framework's interfaces. We must inject an interface like `IOptionsMonitor<Options>` instead of the `Options` class itself. By letting the consumers choose the interface, we let them control the lifetime of the options, which breaks the inversion of control, dependency inversion, and open/closed principles. We should move that responsibility out of the consumer up to the composition root.

> As we explored at the beginning of this chapter, the `IOptions`, `IOptionsFactory`, `IOptionsMonitor`, and `IOptionsSnapshot` interfaces define the options object's lifetime.

In most cases, I prefer to inject `Options` directly, controlling its lifetime from the composition root, instead of letting the class itself control its dependencies. I'm a little *anti-control-freak*, I know. Moreover, writing tests using the `Options` class directly over mocking an interface like `IOptionsSnapshot` is easier.It just so happens that we can circumvent this easily with the following two parts trick:

1. Set up the options class normally, as explored in this chapter.
2. Create a dependency binding that instructs the container to inject the options class directly using the Options pattern.

The xUnit test of the `ByPassingInterfaces` class from the `OptionsValidation` project demonstrates this. Here's the skeleton of that test class:

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Options;
using Xunit;
namespace OptionsValidation;
public class ByPassingInterfaces
{
    [Fact]
    public void Should_support_any_scope() { /*...*/ }
    private class Options
    {
        public string? Name { get; set; }
    }
}
```

The preceding `Options` class has only a `Name` property. We are using it next to explore the workaround in the test case:

```
[Fact]
public void Should_support_any_scope()
{
    // Arrange
    var services = new ServiceCollection();
    services.AddOptions<Options>()
        .Configure(o => o.Name = "John Doe");
    services.AddScoped(serviceProvider => {
        var snapshot = serviceProvider
            .GetRequiredService<IOptionsSnapshot<Options>>();
        return snapshot.Value;
    });
    var serviceProvider = services.BuildServiceProvider();
    // Act & Assert
    using var scope1 = serviceProvider.CreateScope();
    var options1 = scope1.ServiceProvider.GetService<Options>();
    var options2 = scope1.ServiceProvider.GetService<Options>();
    Assert.Same(options1, options2);
    using var scope2 = serviceProvider.CreateScope();
    var options3 = scope2.ServiceProvider.GetService<Options>();
    Assert.NotSame(options2, options3);
}
```

In the preceding code block, we registered the `Options` class using a factory method. That way, we can inject the `Options` class directly (with a scoped lifetime). Moreover, the delegate now controls the `Options` class's creation and lifetime (highlighted code).And voilà, this workaround allows us to inject `Options` directly into our system without tying our classes with any .NET-specific options interface.

Consuming options through the `IOptionsSnapshot<TOptions>` interface results in a *scoped* lifetime.

The *Act & Assert* section of the test validates the correctness of the setup by creating two scopes and ensuring that each scope returns a different instance while returning the same instance within the scope. For example, both `options1` and `options2` come from `scope1`, so they should be the same. On the other hand, `options3` comes from `scope2`, so it should be different than `options1` and `options2`.This workaround also applies to existing systems that could benefit from the Options pattern without updating its code—assuming the system is dependency injection-ready. We can also use this trick to compile an assembly that does not depend on

`Microsoft.Extensions.Options`. By using this trick, we can set the lifetime of the options from the composition root, which is a more classic dependency injection-enabled flow. To change the lifetime, use a different interface, like `IOptionsMonitor` or `IOptionsFactory`.Next, we explore a way to organize all this code.

Project – Centralizing the configuration

Creating classes and classes is very object-oriented and follows the single-responsibility principle, among others. However, dividing responsibilities into programming concerns is not always what leads to the easiest code to understand because it creates a lot of classes and files, often spread across multiple layers and more.An alternative is to regroup the initialization and validation with the options class itself, shifting the multiple responsibilities to a single one: an end-to-end options class.In this example, we explore `ProxyOptions` class, which carries the name of the service and the time the proxy service should cache items in seconds. We want to set a default value for the `CacheTimeInSeconds` property and validate that the `Name` property is not empty.On the other hand, we don't want the consumer of that class to have access to any other methods, like `Configure` or `Validate`.To achieve this, we can implement the interfaces explicitly, hiding them from the `ProxyOptions` but showing them to the consumers of the interfaces. For example, binding the `ProxyOptions` class to the `IValidateOptions<ProxyOptions>` interface gives the consumer access to the `Validate` method through the `IValidateOptions<ProxyOptions>` interface. Explaining this should be simpler in code; here's the class:

```
using Microsoft.Extensions.Options;
namespace CentralizingConfiguration;
public class ProxyOptions : IConfigureOptions<ProxyOptions>, IValidateOptions<ProxyOptions>
{
    public static readonly int DefaultCacheTimeInSeconds = 60;
    public string? Name { get; set; }
    public int CacheTimeInSeconds { get; set; }
    void IConfigureOptions<ProxyOptions>.Configure(
        ProxyOptions options)
    {
        options.CacheTimeInSeconds = DefaultCacheTimeInSeconds;
    }
    ValidateOptionsResult IValidateOptions<ProxyOptions>.Validate(
        string? name, ProxyOptions options)
    {
        if (string.IsNullOrWhiteSpace(options.Name))
        {
            return ValidateOptionsResult.Fail(
                "The 'Name' property is required.");
        }
        return ValidateOptionsResult.Success;
    }
}
```

The preceding code implements both `IConfigureOptions<ProxyOptions>` and `IValidateOptions<ProxyOptions>` interfaces explicitly (highlighted) by omitting the visibility modifier and prefixing the name of the method with the name of the interface, like the following:

```
ValidateOptionsResult IValidateOptions<ProxyOptions>.Validate(...)
```

Now, to leverage it, we must register it with the IoC container like this:

```
builder.Services
    .AddSingleton<IConfigureOptions<ProxyOptions>, ProxyOptions>()
    .AddSingleton<IValidateOptions<ProxyOptions>, ProxyOptions>()
    .AddSingleton(sp => sp
        .GetRequiredService<IOptions<ProxyOptions>>()
        .Value
    )
    .Configure<ProxyOptions>(options
        => options.Name = "High-speed proxy")
    .AddOptions<ProxyOptions>()
    .ValidateOnStart()
;
```

In the preceding code, we combined many notions we explored, like:

- Registering the options class
- Using the workaround to access the `ProxyOptions` class directly
- Configuring the options inline and through a configurator class
- Leverage a validation class
- Enforcing the validation by eager loading our options during the startup.

If you comment out the highlighted line, the application will throw an exception on startup.

The only endpoint defined in the application is the following:

```
app.MapGet("/", (ProxyOptions options) => options);
```

When we run the application, we get the following output:

```
{
  "name": "High-speed proxy",
  "cacheTimeInSeconds": 60
}
```

As expected, the value of the `cacheTimeInSeconds` property equals the value of the `DefaultCacheTimeInSeconds` field, and the value of the `name` property to what we configured in the `Program.cs` file.When using the IntelliSense feature inside your favorite IDE, I'm using Visual Studio 2022 here, we can see only the properties, no method:

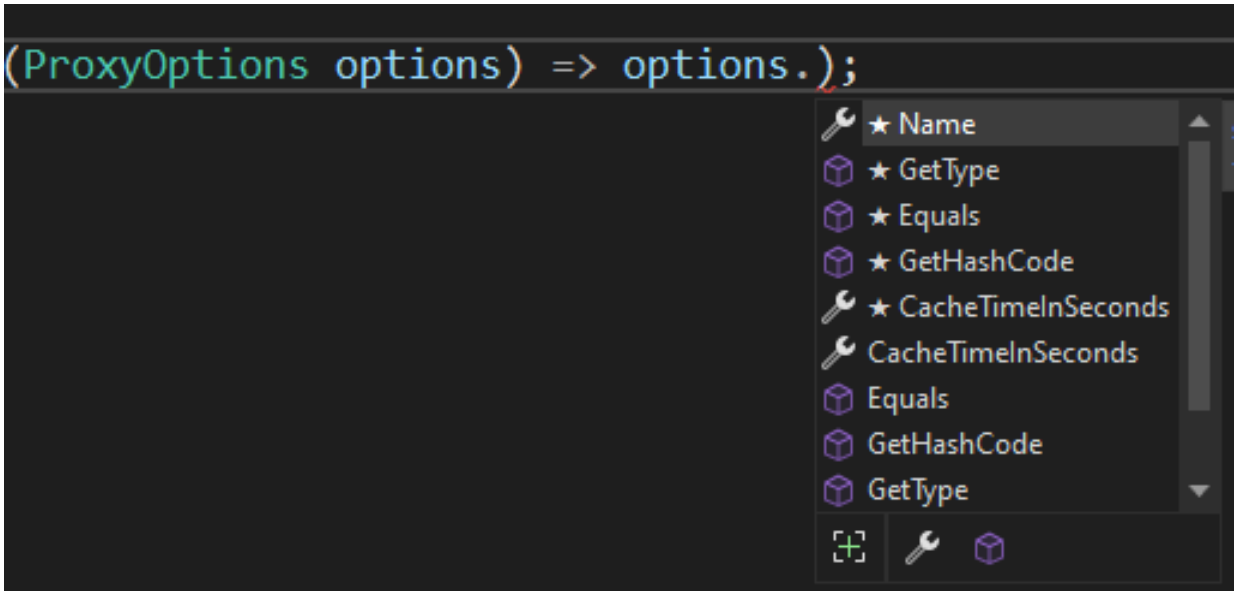


Figure 9.1: VS IntelliSense not showing explicitly implemented interfaces members.

That's it; we are done with this organizational technique.

To keep the composition cleaner, we could encapsulate the bindings in an extension method, and, even better, make that extension method register the whole proxy feature. For example, `services.AddProxyService()`.

I'll let you practice this one on your own as we already explored this.

Next, we explore code generators!

Using the configuration-binding source generator

.NET 8 introduces a **configuration-binding source generator** that provides an alternative to the default reflection-based implementation. In simple terms, the name of the options class properties and the settings keys are now hard-coded, accelerating the configuration retrieval.

> Beware, the settings keys are case-sensitive and map one-on-one with the C# class property name, unlike the non-generated code.

Web applications using Native AOT deployment (ahead-of-time compilation to native code) or trimming self-contained deployments to ship only the bits in use now leverage this option by default.

> The native AOT deployment model compiles the code to a single runtime environment like Windows x64. It does not need the just-in-time (JIT) compiler since the code is already compiled to the native version of the targetted environment. AOT deployments are self-contained and do not need the .NET runtime to work.

We can use the `EnableConfigurationBindingGenerator` property in your `csproj` file to manually activate or deactivate the generator:

```
<PropertyGroup>
  <EnableConfigurationBindingGenerator>true</EnableConfigurationBindingGenerator>
</PropertyGroup>
```

Now that the generator is enabled, let' see how this works. The generator looks for a few options, including the Configure and Bind methods. It then generates the binding code.

Project – ConfigurationGenerators: Part 1

In this first part of the project, we create an options class and register it with the IoC container to consume it through an API endpoint.We use the following options class:

```
namespace ConfigurationGenerators;
public class MyOptions
{
    public string? Name { get; set; }
}
```

In the `Program.cs` file, we can use the source generator like this:

```
builder.Services
    .AddOptions<MyOptions>()
    .BindConfiguration("MyOptions")
;
```

As you may have noticed, the preceding code is the same as we used before and does what you expect it to do, but the new source generator generates the code under the hood—no functional or usage changes.Let's explore another source generator next.

Using the options validation source generator

.NET 8 introduces the **options validation source generator**, which generates the validation code based on data annotations. The idea is similar to the configuration-binding source generator but for the validation code.To leverage the validation generator, we must add a reference on the `Microsoft.Extensions.Options.DataAnnotations` package.Afterward, we must:

1. Create an empty validator class.
2. Ensure the class is `partial`.
3. Implement the `IValidateOptions<TOptions>` interface (but not the methods).
4. Decorate the validator class with the `[OptionsValidator]` attribute.
5. Register the validator class with the container.

This procedure sounds complicated but is way simpler in code; let's look at that now.

Project – ConfigurationGenerators: Part 2

In this second part of the project, we continue to build on the previous pieces and add validation to our `MyOptions` class. Of course, we also want to test the new source generator.Here's the updated `MyOptions` class:

```
using System.ComponentModel.DataAnnotations;
namespace ConfigurationGenerators;
public class MyOptions
{
    [Required]
    public string? Name { get; set; }
}
```

The highlighted line represents the changes. We want to ensure the `Name` property is not empty.Now that we updated our options class, let's create the following validator class:

```
using Microsoft.Extensions.Options;
namespace ConfigurationGenerators;
[OptionsValidator]
public partial class MyOptionsValidator : IValidateOptions<MyOptions>
{
}
```

The preceding code is an empty shell that prepares the class for the code generator. The `[OptionsValidator]` attribute represents the generator hook (a.k.a. that’s the flag the generator is looking for). And with this code, we are done with steps 1 to 4; simpler than English, right?Now, for the last step, we register our validator like normal:

```
builder.Services.AddSingleton<IValidateOptions<MyOptions>, MyOptionsValidator>();
```

To test this out, let's add a `valid` named options instance bound to the following configuration section in the `appsettings.json` file:

```
{
  "MyOptions": {
    "Name": "Options name"
  }
}
```

Here's how we bind it in the `Program.cs` file:

```
builder.Services
    .AddOptions<MyOptions>("valid")
    .BindConfiguration("MyOptions")
    .ValidateOnStart()
;
```

The preceding code registers the `valid` named options, binds it to the configuration section `MyOptions`, and validates it when the application starts.

Other ways to register the named options also work. I used this one for convenience purposes only.

If we were to inspect the content of the options at runtime, it would be what we expect; nothing is different from what we explored throughout the chapter:

```
{
  "name": "Options name"
}
```

At this point, the program should start.Next, to test this out, let's add another named options class, but an invalid one this time. We won't change anything in the `appsettings.json` file, and add the following registration code:

```
builder.Services
    .AddOptions<MyOptions>("invalid")
    .BindConfiguration("MissingSection")
    .ValidateOnStart()
;
```

The preceding code binds a missing section to the `invalid` named options, making the `Name` property equal to `null`. That object will not pass our validation because the `Name` property is required.If we run the application now, we get the following message:

```
Hosting failed to start
Microsoft.Extensions.Options.OptionsValidationException: Name: The invalid.Name field is required
```

From that error, we know the validation works as expected. It is not every day that we are happy when our application doesn’t start but this is one of those time.That's it for the code generation, it behaves the same, but the code under the hood is different, enabling technologies like AOT and trimming that do not support reflection-based mechanisms well. Moreover, code generation should speed up the program execution because the behaviors are hard-coded instead of relying on a dynamic reflection-based approach.Next, let’s dig into another class introduced in .NET 8.

Using the ValidateOptionsResultBuilder class

The `ValidateOptionsResultBuilder` is a new type in .NET 8. It allows to dynamically accumulate validation errors and create a `ValidateOptionsResult` object representing its current state.Its basic usage is straightforward, as we are about to see.

Project - ValidateOptionsResultBuilder

In this project, we are validating the `MyOptions` object. The type has multiple validation rules, and we want to ensure we are not stopping after the first rule fails validation so a consumer would know all the errors in one go. To achieve this, we decided to use the `ValidateOptionsResultBuilder` class.Let's start with the options class:

```
namespace ValidateOptionsResultBuilder;
public class MyOptions
{
    public string? Prop1 { get; set; }
    public string? Prop2 { get; set; }
}
```

Next, let's implement a validator class that enforces both properties are not empty:

```
using Microsoft.Extensions.Options;
namespace ValidateOptionsResultBuilder;
public class SimpleMyOptionsValidator : IValidateOptions<MyOptions>
{
    public ValidateOptionsResult Validate(string? name, MyOptions options)
    {
        var builder = new Microsoft.Extensions.Options.ValidateOptionsResultBuilder();
        if (string.IsNullOrEmpty(options.Prop1))
        {
            builder.AddError(
                "The value cannot be empty.",
                nameof(options.Prop1)
            );
        }
        if (string.IsNullOrEmpty(options.Prop2))
        {
            builder.AddError(
                "The value cannot be empty.",
                nameof(options.Prop2)
            );
        }
        return builder.Build();
```

```
    }
}
```

In the preceding code, we create a `ValidateOptionsResultBuilder` object, add errors to it, then returns an instance of the `SimpleMyOptionsValidator` class by leveraging its `Build` method. The usage of the `ValidateOptionsResultBuilder` class is highlighted.Next, to test this out, we must register the options. Let's also create an endpoint. Here's the `Program.cs` file:

```
using ValidateOptionsResultBuilder;
using Microsoft.Extensions.Options;
var builder = WebApplication.CreateBuilder(args);
builder.Services
    .AddSingleton<IValidateOptions<MyOptions>, SimpleMyOptionsValidator>()
    .AddOptions<MyOptions>("simple")
    .BindConfiguration("SimpleMyOptions")
    .ValidateOnStart()
;
var app = builder.Build();
app.MapGet("/", (IOptionsFactory<MyOptions> factory) => new
{
    simple = factory.Create("simple")
});
app.Run();
```

The preceding code is as normal as it can get after a whole chapter on the Options pattern. We register our options class, the validator, and create an endpoint.When we call the endpoint, we get the following result:

```
Hosting failed to start
Microsoft.Extensions.Options.OptionsValidationException: Property Prop1: The value cannot be empt
```

As expected, the application failed to start because the validation of the `MyOptions` class failed. One difference is that we have two combined error messages instead of one.As a reference, a validator doing the same without using the `ValidateOptionsResultBuilder` type would look like this:

```
using Microsoft.Extensions.Options;
namespace ValidateOptionsResultBuilder;
public class ClassicMyOptionsValidator : IValidateOptions<MyOptions>
{
    public ValidateOptionsResult Validate(string? name, MyOptions options)
    {
        if (string.IsNullOrEmpty(options.Prop1))
        {
            return ValidateOptionsResult.Fail(
                $"Property {nameof(options.Prop1)}: The value cannot be empty."
            );
        }
        if (string.IsNullOrEmpty(options.Prop2))
        {
            return ValidateOptionsResult.Fail(
                $"Property {nameof(options.Prop2)}: The value cannot be empty."
            );
        }
        return ValidateOptionsResult.Success;
    }
}
```

The highlighted code represents the standard process which get replaced by the use of the `ValidateOptionsResultBuilder` type in the `SimpleMyOptionsValidator` class.This concludes our project. Nothing very complex, yet it is a nice addition to help accumulate multiple error messages. On top of that, the `ValidateOptionsResultBuilder` type can also accumulate `ValidationResult` and `ValidateOptionsResult` objects which can lead to more complex systems like collecting results from multiple validators. I'll let you fiddle with this one.Let's recap this chapter before jumping into ASP.NET Core logging.

Summary

This chapter explored the Options pattern, a powerful tool allowing us to configure our ASP.NET Core applications. It enables us to change the application without altering the code. The capability even allows the application to reload the options at runtime when a configuration file is updated without downtime. We learned to load settings from multiple sources, with the last loaded source overriding previous values. We discovered the following interfaces to access settings and learned that the choice of interface influences the lifetime of the options object:

- `IOptionsMonitor<TOptions>`
- `IOptionsFactory<TOptions>`
- `IOptionsSnapshot<TOptions>`
- `IOptions<TOptions>`

We delved into manually configuring options in the composition root and loading them from a settings file. We also learned how to inject options into a class and configure multiple instances of the same options type using named options. We explored encapsulating the configuration logic into classes to apply the single responsibility principle (SRP). We achieved this by implementing the following interfaces:

- `IConfigureOptions<TOptions>`
- `IConfigureNamedOptions<TOptions>`
- `IPostConfigureOptions<TOptions>`

We also learned that we could mix configuration classes with inline configurations using the `Configure` and `PostConfigure` methods and that the registration order of configurators is crucial as they are executed in order of registration.We also delved into options validation. We learned that the frequency at which options objects are validated depends on the lifetime of the options interface used. We also discovered the concept of eager validation, which allows us to catch incorrectly configured options classes at startup time. We learned to use data annotations to decorate our options with validation attributes such as `[Required]`. We can create validation classes to validate our options objects for more complex scenarios. Those validation classes must implement the `IValidateOptions<TOptions>` interface. We also learned how to bridge other validation frameworks like *FluentValidation* to complement the out-of-the-box functionalities or accommodate your taste for a different validation framework.We explored a workaround allowing us to inject options classes directly into their consumers. Doing this allows us to control their lifetime from the composition root instead of letting the types consuming them control their lifetime. This approach aligns better with dependency injection and Inversion of Control principles. That also makes testing the classes easier.Finally, we looked at the .NET 8 code generators that change how the options are handled but do not impact how we use the Options pattern. We also explored the `ValidateOptionsResultBuilder` type, also introduced in .NET 8.The Options pattern helps us adhere to the SOLID principles, as illustrated next:

- **S**: The Options pattern divides managing settings into multiple pieces where each has a single responsibility. Loading unmanaged settings into strongly typed classes is one responsibility, validating options using classes is another, and configuring options from multiple independent sources is one more.

  On the other hand, I find data annotations validation to mix two responsibilities in the options class, bending this principle. If you like data annotations, I don't want to stop you from using them.

  > Data annotations can seem to improve development speed but make testing validation rules harder. For example, testing a `Validate` method that returns a `ValidateOptionsResult` object is easier than attributes.

- **O**: The different `IOptions*<Toptions>` interfaces break this principle by forcing the consumer to decide what lifetime and capabilities the options should have. To change the lifetime of a dependency, we must update the consuming class when using those interfaces. On the other hand, we explored an easy and flexible workaround that allows us to bypass this issue for many scenarios and inject the options directly, inverting the dependency flow again, leading to open/closed consumers.
- **L**: N/A
- **I**: The `IValidateOptions<TOptions>` and `IConfigureOptions<TOptions>` interfaces are two good examples of segregating a system into smaller interfaces where each has a single purpose.

- **D**: The options framework is built around interfaces, allowing us to depend on abstractions.

  Again, the `IOptions*<Toptions>` interfaces are the exceptions to this. Even if they are interfaces, they tie us to implementation details like the options lifetime. In this case, I think it is more beneficial to inject the options object directly (a data contract) instead of those interfaces.

Next, we explore .NET logging, which is another very important aspect of building applications; good traceability can make all the difference when observing or debugging applications.

Questions

Let's take a look at a few practice questions:

1. Name one interface we can use to inject a settings class.
2. Name the two phases the ASP.NET Core uses when configuring options.
3. How significant is the order in which we register configuration objects and inline delegates?
4. Can we register multiple configuration classes?
5. What is eager validation, and why should you use it?
6. What interface must we implement to create a validator class?

Further reading

Here are some links to build upon what we learned in the chapter:

- Options pattern in ASP.NET Core (official docs): https://adpg.link/RTGc
- Quickstart: Create an ASP.NET Core app with Azure App Configuration: https://adpg.link/qhLV
- Secret storage in the Production environment with Azure Key Vault: https://adpg.link/Y5D7

Answers

1. We can use one of the following interfaces: `IOptionsMonitor<TOptions>`, `IOptionsFactory<TOptions>`, `IOptionsSnapshot<TOptions>`, or `IOptions<TOptions>`.
2. The configuration and the post-configuration phases.
3. Configurators are executed in the order of their registration, so their order is crucial.
4. Yes, we can register as many configuration classes as we want.
5. Eager validation allows catching incorrectly configured options at startup time, which can save you runtime issues.
6. We must implement the `IValidateOptions<TOptions>` interface.

10 Logging patterns

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter covers a .NET-specific feature and closes the *Designing for ASP.NET Core* section. The logging feature that comes with a few patterns is another building block that most applications need: built-in ASP.NET Core. We explore the system hands-on while not trying to master every aspect.Logging is a crucial aspect of application development and serves various purposes, such as debugging errors, tracing operations, analyzing usage, and more.The logging abstractions we explore here are another improvement of .NET Core over .NET Framework. Instead of relying on third-party libraries, the new, uniform system offers clean interfaces backed by a flexible and robust mechanism that helps implement logging into our applications.At the end of this chapter, you will understand what logging is and how to write application logs.In this chapter, we cover the following topics:

- About logging
- Writing logs
- Log levels
- Logging providers
- Configuring logging
- Structured logging

Let’s start by exploring what logging is.

About logging

Logging is the practice of writing messages into a log and cataloging information for later use. That information can be used to debug errors, trace operations, analyze usage, or any other reason we can come up with. Logging is a cross-cutting concern, meaning it applies to every piece of your application. We talk about layers in *Chapter 14*, *Layering and Clean*

Architecture, but until then, let's just say that a cross-cutting concern affects all layers and cannot be centralized in just one; it affects a bit of everything.A log is made up of log entries. We can view each log entry as an event that happened during the program's execution. Those events are then written to the log. This log can be a file, a remote system, `stdout`, or a combination of multiple destinations.When creating a log entry, we must also think about the severity of that log entry. In a way, this severity level represents the type of message or the level of importance that we want to log. We can also use it to filter those logs. `Trace`, `Error`, and `Debug` are examples of log entry levels. Those levels are defined in the `Microsoft.Extensions.Logging.LogLevel` enum.Another important aspect of a log entry is how it is structured. You can log a single string. Everyone on your team could log single strings in their own way. But what happens when someone searches for information? Chaos ensues! There's the stress of not finding what that person is looking for and the displeasure of the log's structure, as experienced by that same person. One way to fix this is by using structured logging. It is simple yet complex; you must create a structure the program follows for all log entries. That structure could be more or less complex or be serialized into JSON. The important part is that the log entries are structured. We won't get into this subject here, but if you must decide on a logging strategy, I recommend digging into structured logging first. If you are part of a team, then chances are someone else already did. If that's not the case, you can always bring it up. Continuous improvement is a key aspect of life.We could write a whole book on logging, best logging practices, structured logging, and distributed tracing, but this chapter aims to teach you how to use .NET logging abstractions.

Writing logs

First, the logging system is provider-based, meaning we must register one or more `ILoggerProvider` instances if we want our log entries to go somewhere. By default, when calling `WebApplication.CreateBuilder(args)`, it registers the Console, Debug, EventSource, and EventLog (Windows only) providers, but we can modify this list. You can add and remove providers if you need to. The required dependencies for using the logging system are also registered as part of this process.Before we look at the code, let's learn how to create log entries, which is the objective behind logging. To create an entry, we can use one of the following interfaces: `ILogger`, `ILogger<T>`, or `ILoggerFactory`. Let's take a look at them in more detail:

Interface	**Description**
`ILogger`	Base type that allows us to perform logging operations.
`ILogger<T>`	Base type that allows us to perform logging operations. Inherit from the ILogger interface. The system uses the generic parameter `T` as the log entry's *category*.
`ILoggerFactory`	A factory interface that allows creating `ILogger` objects and specifying the category name manually as a string.

Table 10.1: the logging interfaces.

The following code represents the most commonly used pattern, which consists of injecting an `ILogger<T>` interface and storing it in an `ILogger` field before using it, like this:

```
public class Service : IService
{
    private readonly ILogger _logger;
    public Service(ILogger<Service> logger)
    {
        _logger = logger;
    }
    public void Execute()
    {
        _logger.LogInformation("Service.Execute()");
    }
}
```

The preceding `Service` class has a private `_logger` field. It takes an `ILogger<Service>` logger as a parameter and stores it in that field. It uses that field in the `Execute` method to write an information-level message to the log.The `IService` interface is very simple and only exposes a single `Execute` method for testing purposes:

```
public interface IService
{
    void Execute();
}
```

I loaded a small library I created to test this out, providing additional logging providers for testing purposes. With that, we are creating a generic host (`IHost`) since we don't need a `WebApplication` in our tests, then we configure it:

```
namespace Logging;
public class BaseAbstractions
{
    [Fact]
    public void Should_log_the_Service_Execute_line()
    {
        // Arrange
        var lines = new List<string>();
        var host = Host.CreateDefaultBuilder()
            .ConfigureLogging(loggingBuilder =>
            {
                loggingBuilder.ClearProviders();
                loggingBuilder.AddAssertableLogger(lines);
            })
            .ConfigureServices(services =>
            {
                services.AddSingleton<Service>();
            })
            .Build();
        var service = host.Services.GetRequiredService<Service>();
        // Act
        service.Execute();
        // Assert
        Assert.Collection(lines,
```

```
            line => Assert.Equal("Service.Execute()", line)
        );
    }
    // Omitted other members
}
```

In the `Arrange` phase of the test, we create some variables, configure `IHost`, and get an instance of the `Service` class that we want to use to test the logging capabilities that we programmed.The highlighted code removes all providers using the `ClearProviders` method. Then it uses the `AddAssertableLogger` extension to add a new provider. The extension method comes from the library that we loaded. We could have added a new provider if we wanted, but I wanted to show you how to remove existing providers so we can start from a clean slate. That's something you might need someday.

> The library that I loaded is available on NuGet and is named `ForEvolve.Testing.Logging`, but you do not need to understand any of this to understand logging abstractions and examples.

In the `Act` phase, we call the `Execute` method of our service. This method logs a line to the `ILogger` implementation that is injected upon instantiation. Then, we assert that the line was written in the `lines` list (that's what `AssertableLogger` does; it writes to a `List<string>`). In an ASP.NET Core application, all that logging goes to the console by default. Logging is a great way to know what is happening in the background when running the application.The `Service` class is a simple consumer of an `ILogger<Service>`. You can do the same for any class you want to add logging support to. Change `Service` by that class name to have a logger configured for your class. That generic argument becomes the logger's category name when writing log entries.Since ASP.NET Core uses a `WebApplication` instead of a generic `IHost`, here is the same test code using that construct:

```
[Fact]
public void Should_log_the_Service_Execute_line_using_WebApplication()
{
    // Arrange
    var lines = new List<string>();
    var builder = WebApplication.CreateBuilder();
    builder.Logging.ClearProviders()
        .AddAssertableLogger(lines);
    builder.Services.AddSingleton<IService, Service>();
    var app = builder.Build();
    var service = app.Services.GetRequiredService<IService>();
    // Act
    service.Execute();
    // Assert
    Assert.Collection(lines,
        line => Assert.Equal("Service.Execute()", line)
    );
}
```

I highlighted the changes in the preceding code. In a nutshell, the extension methods used on the generic host have been replaced by `WebApplicationBuilder` properties like `Logging` and `Services`. Finally, the `Create` method creates a `WebApplication` instead of an `IHost`, exactly like in the `Program.cs` file.To wrap this up, these test cases allowed us to implement the most commonly used logging pattern in ASP.NET Core and add a custom provider to ensure we logged the correct information. Logging is essential and adds visibility to production systems. Without logs, you don't know what is happening in your system unless you are the only one using it, which is very unlikely. You can also log what is happening in your infrastructure and run real-time security analysis on those log streams to quickly identify security breaches, intrusion attempts, or system failures. These subjects are out of the scope of this book, but having strong logging capabilities at the application level can only help your overall logging strategy.Before moving on to the next subject, let's explore an example that leverages the `ILoggerFactory` interface. The code sets a custom category name and uses the created `ILogger` instance to log a message. This is very similar to the previous example. Here's the whole code:

```
namespace Logging;
public class LoggerFactoryExploration
{
    private readonly ITestOutputHelper _output;
    public LoggerFactoryExploration(ITestOutputHelper output)
    {
        _output = output ?? throw new ArgumentNullException(nameof(output));
    }
    [Fact]
    public void Create_a_ILoggerFactory()
    {
        // Arrange
        var lines = new List<string>();
        var host = Host.CreateDefaultBuilder()
            .ConfigureLogging(loggingBuilder => loggingBuilder
                .AddAssertableLogger(lines)
                .AddxUnitTestOutput(_output))
            .ConfigureServices(services => services.AddSingleton<Service>())
            .Build()
        ;
        var service = host.Services.GetRequiredService<Service>();
        // Act
        service.Execute();
        // Assert
        Assert.Collection(lines,
            line => Assert.Equal("LogInformation like any ILogger<T>.", line)
        );
    }
    public class Service
    {
        private readonly ILogger _logger;
        public Service(ILoggerFactory loggerFactory)
        {
            ArgumentNullException.ThrowIfNull(loggerFactory);
            _logger = loggerFactory.CreateLogger("My Service");
        }
        public void Execute()
```

```
        {
            _logger.LogInformation("LogInformation like any ILogger<T>.");
        }
    }
}
```

The preceding code should look very familiar. Let's focus on the highlighted lines, which relate to the current pattern:

1. We inject the `ILoggerFactory` interface into the `Service` class constructor (instead of an `ILogger<Service>`).
2. We create an `ILogger` instance with the `" My Service"` category name.
3. We assign the logger to the `_logger` field.
4. We then use that `ILogger` from the `Execute` method.

As a rule of thumb, I recommend using the `ILogger<T>` interface by default. If impossible, or if you need a more dynamic way of setting the category name for your log entries, leverage the `ILoggerFactory` instead. By default, when using `ILogger<T>`, the category name is the T parameter, which should be the name of the class creating log entries. The `ILoggerFactory` interface is more of an internal piece than something made for us to consume; nonetheless, it exists and satisfies some use cases.

Note

In the preceding example, the `ITestOutputHelper` interface is part of the `Xunit.Abstractions` assembly. It allows us to write lines as *standard output* to the test log. That output is available in the Visual Studio Test Explorer.

Now that we have covered how to write log entries, it's time to learn how to manage their severity.

Log levels

In the previous examples, we used the `LogInformation` method to log information messages, but there are other levels as well, shown in the following table:

Level	Method	Description	Production
Trace	`LogTrace`	This is used to capture detailed information about the program, instrument execution speed, and debugging. You can also log sensitive information when using traces.	Disabled.
Debug	`LogDebug`	This is used to log debugging and development information.	Disabled unless troubleshooting.
Information	`LogInformation`	This is used to track the flow of the application. Normal events that occur in	Enabled.

		the system are often information-level events, such as the system started, the system stopped, and a user has signed in.	
Warning	`LogWarning`	This is used to log abnormal behavior in the application flow that does not cause the program to stop, but that may need to be investigated; for example, handled exceptions, failed network calls, and accessing resources that do not exist.	Enabled.
Error	`LogError`	This is used to log errors in the application flow that do not cause the application to stop. Errors must usually be investigated. Examples include the failure of the current operation and an exception that cannot be handled.	Enabled.
Critical	`LogCritical`	This is used to log errors that require immediate attention and represent a catastrophic state. The program is most likely about to stop, and the integrity of the application might be compromised; for example, a hard drive is full, the server is out of memory, or the database is in a deadlocked state.	Enabled with some alerts that could be configured to trigger automatically.

Table 10.2: log entry levels

As described in the preceding table, each log level serves one or more purposes. Those log levels tell the logger what severity a log entry is. Then, we can configure the system to log only entries of at least a certain level so we don’t fill out production logs with traces and debug entries, for example. In a project I led, we benchmarked multiple ways to log simple and complex messages using ASP.NET Core to build clear and optimized guidelines around that. We could not reach a fair conclusion when the messages were logged due to a large time variance between benchmark runs. However, we observed a constant trend when messages were not logged (*trace* logs with the minimum logging level configured to *debug*, for example).Based on that conclusion, I recommend logging the `Trace` and `Debug` messages using the following construct instead of interpolation, `string.Format`, or other means. That may sound strange to optimize for *not logging something*, but if you think about it, those log entries will be skipped in production, so optimizing them will save your production app a few milliseconds of computing time here and there. Moreover, it's not harder or longer to do, so it's just a good habit.Let’s look at the fastest way to *not write log entries*:

```
_logger.LogTrace("Some: {variable}", variable);
// Or
_logger.LogTrace("Some: {0}", variable);
```

When the log level is disabled, such as in production, you only pay the price of a method call because no processing is done on your log entries. On the other hand, if we use interpolation, the processing is done, so that one argument is passed to the `Log[Level]` method, leading to a higher cost in processing power for each log entry.Here's an example of interpolation (a.k.a. what not to do):

```
_logger.LogTrace($"Some: {variable}");
```

For warning and higher levels, you can keep the good habit and use the same technique or other methods because we know those lines will be logged anyway. Therefore, using interpolation in the code or letting the logger do it later should yield a similar result.

> One last note. I suggest you don't try to over-optimize your code before there is a need for that. The action of investing a lot of effort in optimizing something that does not need optimizing is known as **premature optimization**. The idea is to optimize just enough upfront and fix the performance when you find real issues.

Now that we know the log levels that .NET offers us, let's look at the logging providers.

Logging providers

To give you an idea of the possible built-in logging providers, here is a list from the official documentation (see the *Further reading* section at the end of this chapter):

- Console
- Debug
- EventSource
- EventLog (Windows only)
- ApplicationInsights

The following is a list of third-party logging providers, also from the official documentation:

- elmah.io
- Gelf
- JSNLog
- KissLog.net
- Log4Net
- NLog
- PLogger
- Sentry
- Serilog
- Stackdriver

Now, if you need any of those or your favorite logging library is part of the preceding list, you know you can use it. If it is not, maybe it supports ASP.NET Core but was not part of the documentation when I consulted it.Next, let’s learn how to configure the logging system.

Configuring logging

As with most features of ASP.NET Core, we can configure logging. The default `WebApplicationBuilder` that do a lot for us, but in case we want to tweak the defaults, we can. On top of that, the system loads the `Logging` section of the configuration. That section is present, by default, in the `appsettings.json` file. Like all configurations, it is cumulative, so we can redefine part of it in another file or configuration provider.We won’t dig too deep into customization, but it is good to know that we can customize the minimum level of what we are logging. We can also use transformation files (such as `appsettings.Development.json`) to customize those levels per environment.For example, we can define our defaults in `appsettings.json`, then update a few for development purposes in `appsettings.Development.json`, change production settings in `appsettings.Production.json`, then change the staging settings in `appsettings.Staging.json`, and add some testing settings in `appsettings.Testing.json`.Before we move on, let’s take a peek at the default settings:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning"
    }
  }
}
```

We can define default levels (using `Logging:LogLevel:Default`) and a custom level for each category (such as `Logging:LogLevel:Microsoft`) representing base namespaces. For example, from that configuration file, the minimum level is `Information`, while every item part of the `Microsoft` or `Microsoft.*` namespaces have a minimum level of `Warning`. That allows for removing noise when running the application. We can also leverage these configurations to debug certain parts of the application by lowering the log level to `Debug` or `Trace` for only a subset of items (items from one or more namespaces, for example).We can also filter what we want to log on a provider basis, using configuration or code. In the configuration file, we can change the default level of the console provider to `Trace`, like this:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning"
    },
    "Console": {
```

```
      "LogLevel": {
        "Default": "Trace"
      }
    }
  }
}
```

We kept the same default values but added the `Logging:Console` section (see highlighted code) with a default `LogLevel` set to `Trace`. We can define as many settings as we need.Instead of configurations, we can use the `AddFilter` extension methods, as shown in the following experimental test code, or in conjunction with configurations.Here is the consumer class that logs data:

```
public class Service
{
    private readonly ILogger _logger;
    public Service(ILogger<Service> logger)
    {
        _logger = logger;
    }
    public void Execute()
    {
        _logger.LogInformation("[info] Service.Execute()");
        _logger.LogWarning("[warning] Service.Execute()");
    }
}
```

The preceding class is like other classes we used during the chapter but logs messages using two different levels: `Information` and `Warning`. Here is a test case in which we leverage the `AddFilter` method:

```
[Fact]
public void Should_filter_logs_by_provider()
{
    // Arrange
    var lines = new List<string>();
    var host = Host.CreateDefaultBuilder()
        .ConfigureLogging(loggingBuilder =>
        {
            loggingBuilder.ClearProviders();
            loggingBuilder.AddConsole();
            loggingBuilder.AddAssertableLogger(lines);
            loggingBuilder.AddxUnitTestOutput(_output);
            loggingBuilder
                .AddFilter<XunitTestOutputLoggerProvider>(
                    level => level >= LogLevel.Warning
                );
        })
        .ConfigureServices(services =>
        {
            services.AddSingleton<Service>();
        })
        .Build();
    var service = host.Services.GetRequiredService<Service>();
    // Act
```

```
    service.Execute();
    // Assert
    Assert.Collection(lines,
        line => Assert.Equal("[info] Service.Execute()", line),
        line => Assert.Equal("[warning] Service.Execute()", line)
    );
}
```

We created a generic host in the preceding test code and added three providers: the console and two test providers—one that logs to a list and another to the xUnit output. Then, we told the system to filter out everything that is not at least a `Warning` from `XunitTestOutputLoggerProvider` (see highlighted code); other providers are unaffected by that code.

In the code, the `_output` member is a field of type `ITestOutputHelper`.

We now know of two options to set the minimum logging levels:

- Code
- Configuration

We can tweak the way we configure our logging policies as needed. Code can be more testable, while configurations can be updated at runtime without redeploying. Moreover, with the cascading model, which allows us to override configuration, we can cover most use cases using configurations. The biggest downside of configuration is that writing strings in a JSON file is more error-prone than writing code (assuming you are not reverting to using strings there either).I usually stick with configurations to set those values, as they do not change often. If you prefer code, I'm unaware of any drawbacks, and it's just a matter of preference; the configuration becomes code at some point.Next, let's look at a brief example of structured logging.

Structured logging

As stated at the beginning, structured logging can become very important and open opportunities. Querying a data structure is always more versatile than querying a single line of text. That is even more true if there is no clear guideline around logging, whether a line of text or a JSON-formatted data structure.To keep it simple, we leverage a built-in formatter (highlighted line below) that serializes our log entries into JSON. Here is the `Program.cs` file:

```
var builder = WebApplication.CreateBuilder(args);
builder.Logging.AddJsonConsole();
var app = builder.Build();
app.MapGet("/", (ILoggerFactory loggerFactory) =>
{
    const string category = "root";
    var logger = loggerFactory.CreateLogger(category);
    logger.LogInformation("You hit the {category} URL!", category);
```

```
    return "Hello World!";
});
app.Run();
```

That transforms the console to logging JSON. For example, every time we hit the `/` endpoint, the console displays the following JSON:

```
{
  "EventId": 0,
  "LogLevel": "Information",
  "Category": "root",
  "Message": "You hit the root URL!",
  "State": {
    "Message": "You hit the root URL!",
    "category": "root",
    "{OriginalFormat}": "You hit the {category} URL!"
  }
}
```

Without that formatter, the usual output would have been:

```
info: root[0]
      You hit the root URL!
```

Based on that comparison, it is more versatile to query the JSON logs programmatically than the `stdout` line.The biggest benefit of structured logging is improved searchability. You can run more precise queries at scale with a predefined data structure.Of course, if you are setting up a production system, you would probably want more information attached to such log items like the correlation ID of the request, optionally some information about the current user, the server's name on which the code is running, and possibly more details depending on the application.You may need more than the out-of-the-box features to utilize structured logging fully. Some third-party libraries like Serilog offer those additional capabilities. However, defining the way to send plain text to the logger could be a start.Each project should dictate the needs and depth of each feature, including logging. Moreover, structured logging is a broader subject that merits studying independently. Nonetheless, I wanted to touch on this subject a bit, and hopefully, you learned enough about logging to get started.

Summary

In this chapter, we delved into the concept of logging. We learned that logging is the practice of recording messages into a log for later use, such as debugging errors, tracing operations, and analyzing usage. Logging is essential, and ASP.NET Core offers us various ways to log information independently of third-party libraries while allowing us to use our favorite logging framework. We can customize the way the logs are written and categorized. We can use zero or more logging providers. We can also create custom logging providers. Finally, we can use configurations or code to filter logs and much more.Here is the default logging pattern to remember:

1. Inject an `ILogger<T>`, where `T` is the type of the class into which the logger is injected. `T` becomes the category.
2. Save a reference of that logger into a `private readonly ILogger` field.
3. Use that logger in your methods to log messages using the appropriate log level.

The logging system is a great addition to .NET Core compared to .NET Framework. It allows us to standardize the logging mechanism, making our systems easier to maintain in the long run. For example, suppose you want to use a new third-party library or even a custom-made one. In that case, you can load the provider into your `Program`, and the entire system will adapt and start using it without any further changes as long as you depend only on the logging abstractions. This is a good example of what well-designed abstractions can bring to a system.**Here are a few key takeaways**:

- Logging is a cross-cutting concern, affecting all layers of an application.
- A log comprises many log entries representing an event that occurred at runtime during the program's execution.
- The severity of a log entry is important for filtering and prioritization.
- The severity levels are Trace, Debug, Information, Warning, Error, and Critical.
- We can configure the logging system to log only certain messages based on the severity level of each entry.
- Structured logging can help maintain consistency and ease of searching within logs.
- The logging system in .NET is provider-based, allowing us to customize the default providers.
- We can use interfaces like ILogger, ILogger<T>, or ILoggerFactory to create log entries.

This chapter closes the second section of this book with ASP.NET Core at its center. We explore design patterns to create flexible and robust components in the next few chapters.

Questions

Let’s take a look at a few practice questions:

1. Can we write log entries to the console and a file at the same time?
2. Is it true that we should log the trace- and debug-level log entries in a production environment?
3. What is the purpose of structured logging?
4. How can we create a log entry in .NET?

Further reading

Here is a link to build upon what we learned in the chapter:

- [Official docs] *Logging in .NET Core and ASP.NET Core*: https://adpg.link/MUVG

Answers

1. Yes, you can configure as many providers as you want. One could be for the console, and another could append entries to a file.
2. No, you should not log trace-level entries in production. You should only log debug-level entries when debugging an issue.
3. Structured logging maintains a consistent structure across all log entries, making searching and analyzing logs easier.
4. We can create a log entry using interfaces like `ILogger`, `ILogger<T>`, and `ILoggerFactory`.

11 Structural Patterns

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter explores four design patterns from the well-known Gang of Four (GoF). We use Structural patterns to build and organize complex object hierarchies in a maintainable fashion. They allow us to dynamically add behaviors to existing classes, whether we designed the initial system this way or as an afterthought that emerges out of necessity later in the program's lifecycle. Structural patterns promote reusability and enhance the overall flexibility of the system.In this chapter, we cover the following topics:

- Implementing the Decorator design pattern
- Implementing the Composite design pattern
- Implementing the Adapter design pattern
- Implementing the Façade design pattern

The first two patterns help us extend a class dynamically and efficiently manage a complex object structure. The last two help us adapt an interface to another or shield a complex system with a simple interface.Let's dive into unlocking the power of structural patterns!

Implementing the Decorator design pattern

The Decorator Pattern allows us to dynamically add new functionality to an object by wrapping it with one or more decorator objects. This pattern follows the Open-Closed principle, allowing us to add additional behaviors to an object at runtime without modifying its original code. This pattern enables us to separate responsibilities into multiple smaller pieces. It is a simple but powerful pattern. In this section, we explore how to implement this pattern in the traditional way and how to leverage an open-source tool named **Scrutor** to help us create powerful dependency injection-ready decorators.

Goal

The decorator aims to extend an existing object at runtime without changing its code. Moreover, the decorated object should remain unaware of the decoration process, making this approach an excellent fit for complex or long-lasting systems that necessitate evolution. This pattern fits systems of all sizes.

> I often use this pattern to add flexibility and create adaptability to a program for next to no cost. In addition, small classes are easier to test, so the Decorator pattern adds ease of testability into the mix, making it worth mastering.

The Decorator pattern makes it easier to encapsulate responsibilities into multiple classes, instead of packing multiple responsibilities inside a single class. Having multiple classes with a single responsibility makes the system easier to manage.

Design

A **decorator** class must implement and use the interface the **decorated** class implements. Let's see this step by step, starting with a non-decorated class design:

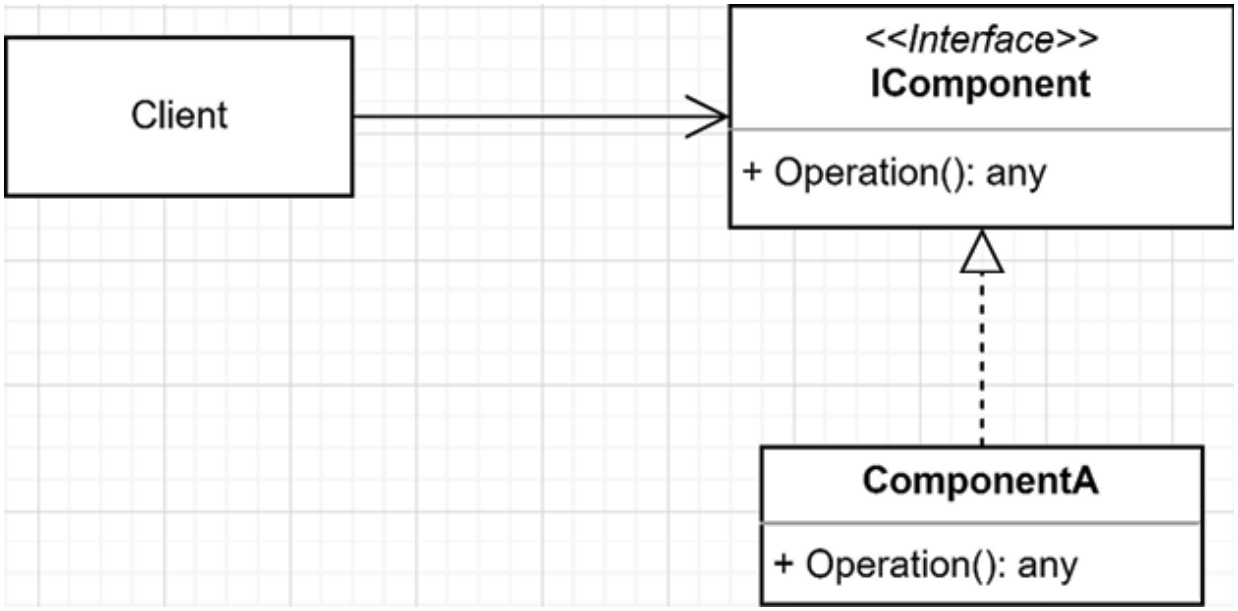


Figure 11.1: A class diagram representing the ComponentA class implementing the IComponent interface

In the preceding diagram, we have the following components:

- A client that calls the `Operation()` method of the `IComponent` interface.
- `ComponentA`, which implements the `IComponent` interface.

This translates into the following sequence diagram:

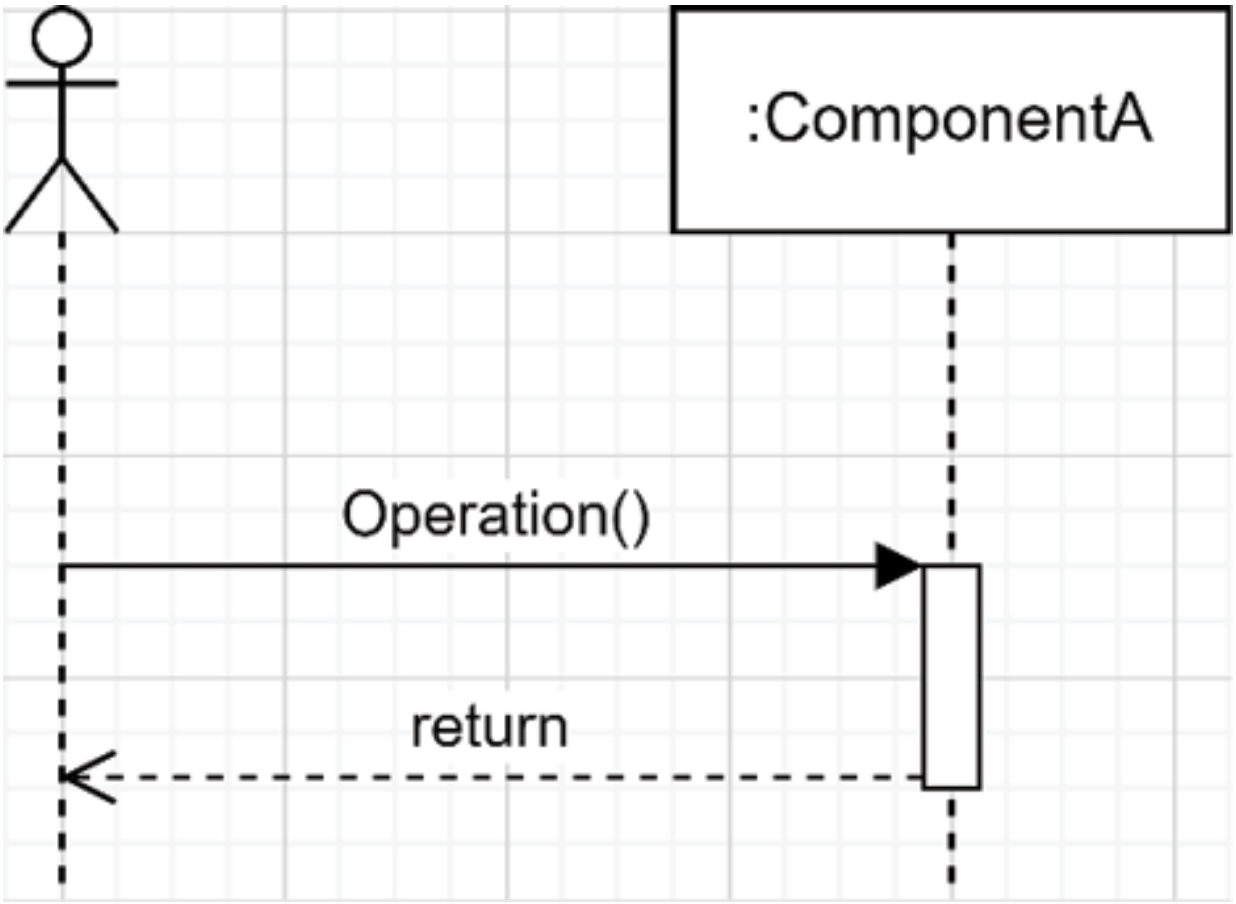


Figure 11.2: A sequence diagram showing a consumer calling the Operation method of the ComponentA class

Now, say that we want to add a behavior to `ComponentA`, but only in some cases. In other cases, we want to keep the initial behavior. To do so, we could choose the Decorator pattern and implement it as follows:

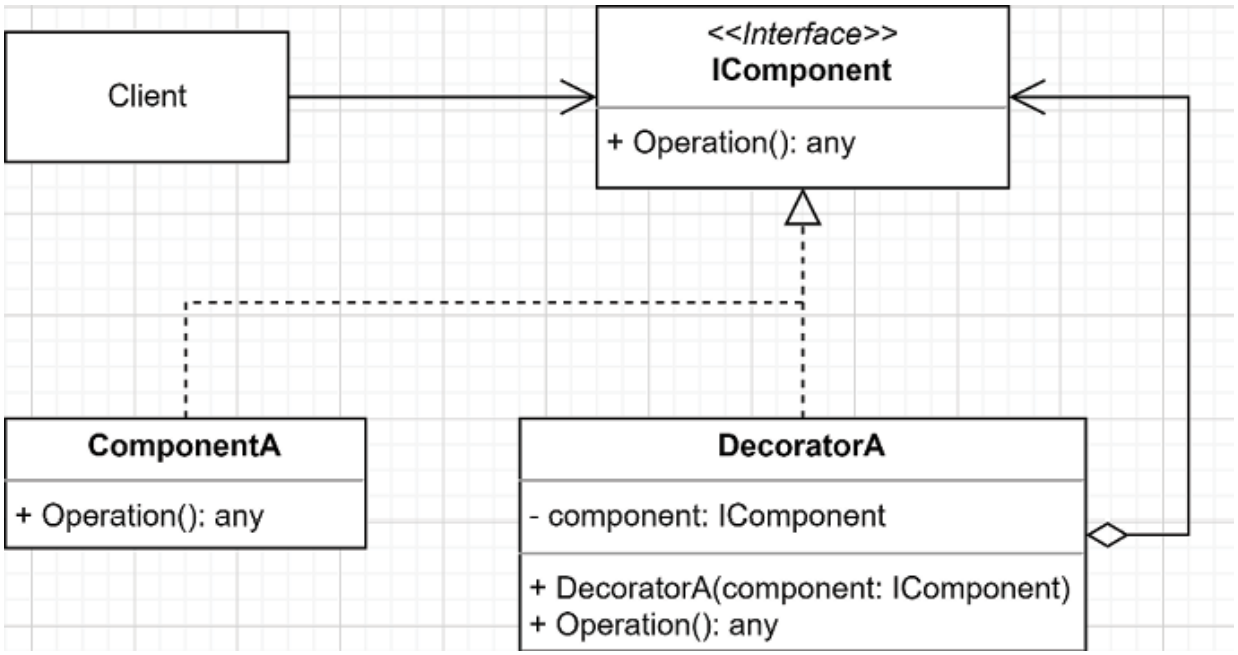


Figure 11.3: Decorator class diagram

Instead of modifying the `ComponentA` class, we created `DecoratorA`, which also implements the `IComponent` interface. This way, the `Client` object can use an instance of `DecoratorA` instead of `ComponentA` and leverage the new behavior without impacting the other consumers of `ComponentA`. Then, to avoid rewriting the whole component, an implementation of the `IComponent` interface (say `ComponentA`) is injected when creating a new `DecoratorA` instance (constructor injection). This new instance is stored in the `component` field and used by the `Operation()` method (implicitly using the **Strategy** pattern).We can translate the updated sequence like so:

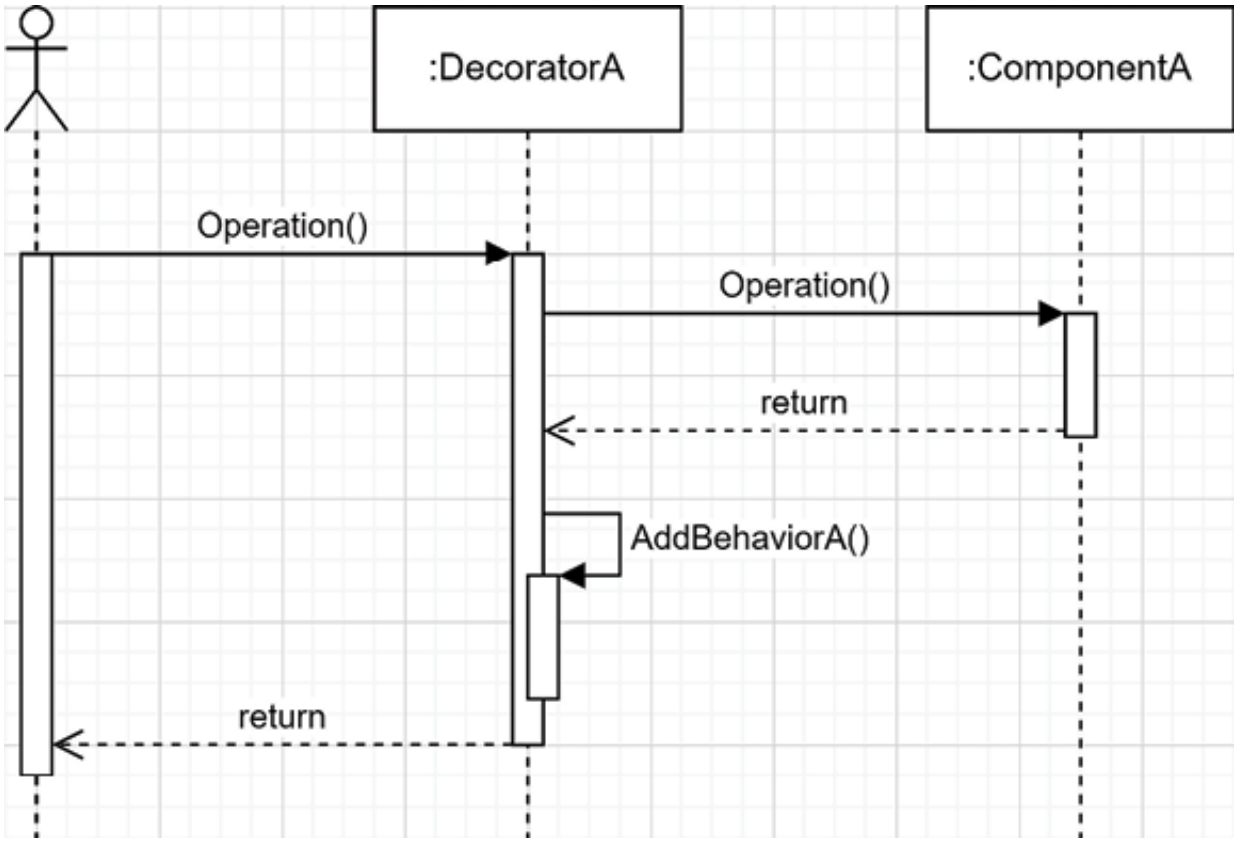


Figure 11.4: Decorator sequence diagram

In the preceding diagram, instead of calling `ComponentA` directly, `Client` calls `DecoratorA`, which in turn calls `ComponentA`. Finally, `DecoratorA` does some postprocessing by calling its private method, `AddBehaviorA()`.

> Nothing from the Decorator pattern limits us from doing preprocessing, postprocessing, wrapping the decorated class's call (the `Operation` method in this example) with some logic (like an `if` statement or a `try-catch`), or all of that combined. The use of adding a postprocessing behavior is only an example.

To show you how powerful the Decorator pattern is before we jump into the code, know this: we can chain decorators! Since our decorator depends on the interface (not the implementation), we could inject another decorator, let's call it `DecoratorB`, inside `DecoratorA` (or vice versa). We could then create a long chain of rules that decorate one another, leading to a very powerful yet simple design.Let's take a look at the following class diagram, which represents our chaining example:

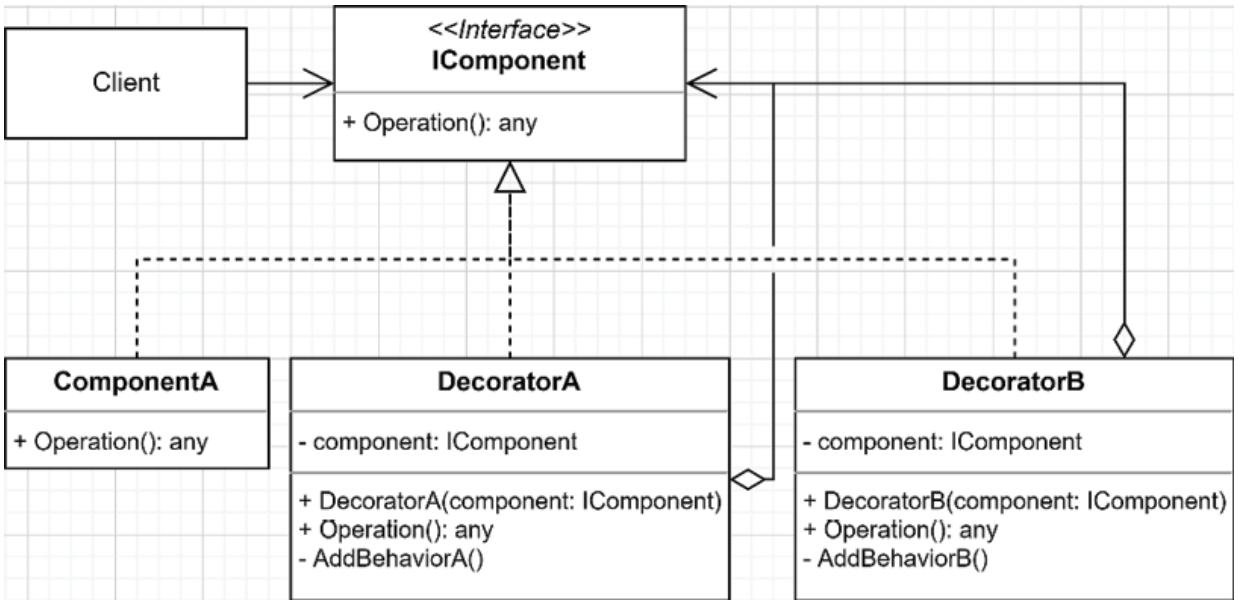


Figure 11.5: Decorator class diagram, including two decorators

Here, we created the `DecoratorB` class, which looks very similar to `DecoratorA` but has a private `AddBehaviorB()` method instead of `AddBehaviorA()`.

> How we implement the decorator logic is irrelevant to the pattern, so I excluded the `AddBehaviorA()` method from *Figure 9.3* to show you only the pattern. However, I added it to *Figure 9.5* to clarify the idea behind having a second decorator.

Let's take a look at the sequence diagram for this:

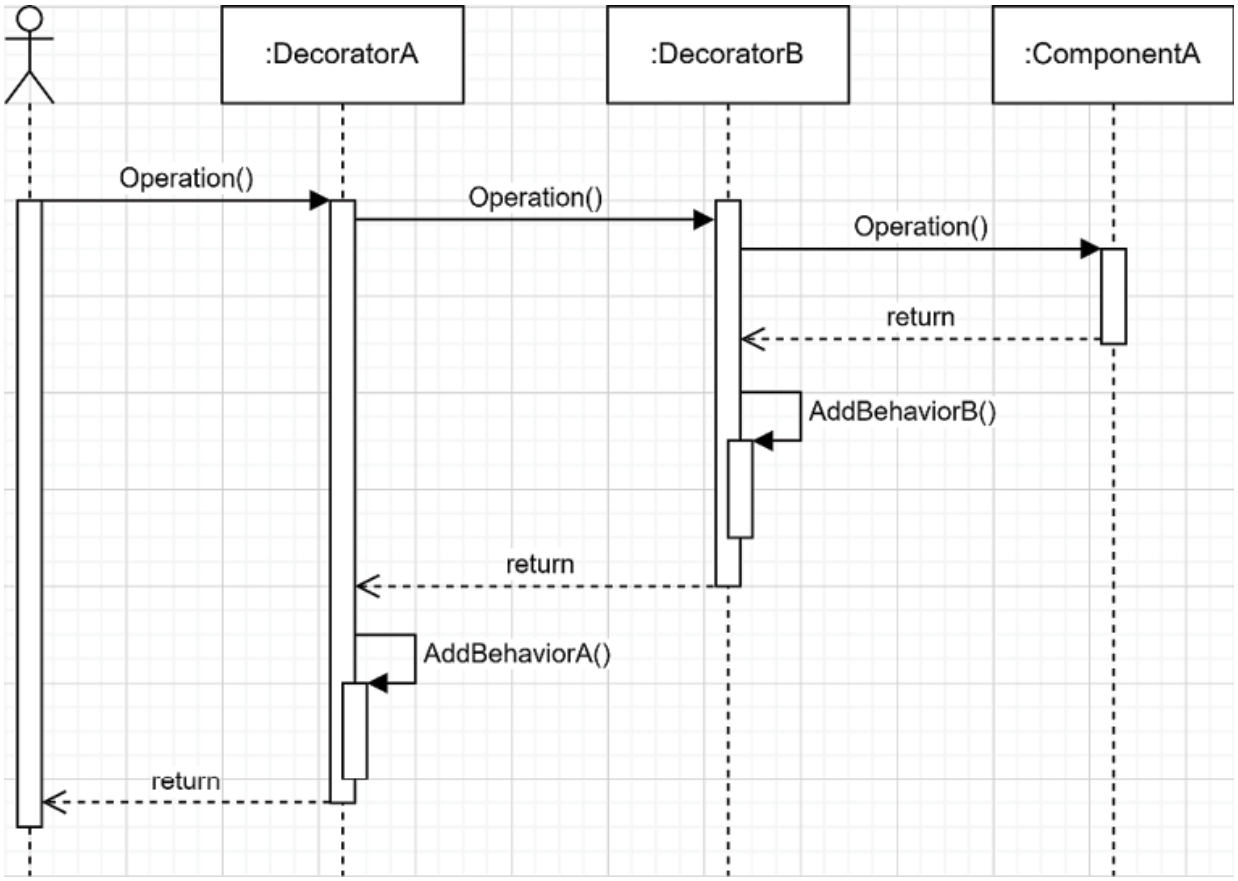


Figure 11.6: Sequence diagram of two nested decorators

With this, we are beginning to see the power of decorators. In the preceding diagram, we can assess that the behaviors of `ComponentA` have been changed twice without `Client` knowing about it. All those classes are unaware of the next `IComponent` in the chain. They don't even know that they are being decorated. They only play their role in the plan—that's all.It is also important to note that the decorator's power resides in its dependency on the interface, not on an implementation, making it reusable. Based on that fact, we could swap `DecoratorA` and `DecoratorB` to invert the order the new behaviors are applied without touching the code itself. We could also apply the same decorator (say `DecoratorC`) to multiple `IComponent` implementations, like decorating both `DecoratorA` and `DecoratorB`. A decorator could even decorate itself.Let's now dig into some code.

Project – Adding behaviors

Let's implement the previous example to help visualize the Decorator pattern, which adds some arbitrary behaviors. Each `Operation()` method returns a string that is then outputted to the response stream. It is not fancy but visually shows how the pattern works.First, let's look at the `IComponent` interface:

```
public interface IComponent
{
    string Operation();
}
```

The `IComponent` interface only states that an implementation should have an `Operation()` method that returns a `string`.Next, let's look at the `ComponentA` class:

```
public class ComponentA : IComponent
{
```

```
    public string Operation()
    {
        return "Hello from ComponentA";
    }
}
```

The `Operation()` method of the `ComponentA` class returns a literal string.Now that we described the first pieces, let's look at the consumer:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<IComponent, ComponentA>();
var app = builder.Build();
app.MapGet("/", (IComponent component) => component.Operation());
app.Run();
```

In the `Program.cs` file above, we register `ComponentA` as the implementation of `IComponent` , with a singleton lifetime. We then inject an `IComponent` implementation when an HTTP request hits the `/` endpoint. The delegate then calls the `Operation()` method and outputs the result to the response.At this point, running the application results in the following response:

```
Hello from ComponentA
```

So far, it's pretty simple; the client calls the endpoint, the container injects an instance of the `ComponentA` class into the endpoint delegate, then the endpoint returns the results of `Operation` method to the client.Next, we add the first decorator.

DecoratorA

Here, we want to modify the response without touching the code of the `ComponentA` class. To do so, we chose to create a decorator named `DecoratorA` that wraps the `Operation()` result into a `<DecoratorA>` tag:

```
public class DecoratorA : IComponent
{
    private readonly IComponent _component;
    public DecoratorA(IComponent component)
    {
        _component = component ?? throw new ArgumentNullException(nameof(component));
    }
    public string Operation()
    {
        var result = _component.Operation();
        return $"<DecoratorA>{result}</DecoratorA>";
    }
}
```

`DecoratorA` implements and depends on the `IComponent` interface. It uses the injected `IComponent` implementation in its `Operation()` method and wraps its result in an HTML-like (XML) tag.Now that we have a decorator, we need to tell the IoC container to send an instance of `DecoratorA` instead of `ComponentA` when injecting an `IComponent` interface. `DecoratorA` should decorate `ComponentA` . More precisely, the container should inject an instance of the `ComponentA` class into the `DecoratorA` class.To achieve this, we could register it as follows:

```
builder.Services.AddSingleton<IComponent>(serviceProvider => new DecoratorA(new ComponentA()));
```

Here, we are telling ASP.NET Core to inject an instance of `DecoratorA` that decorates an instance of `ComponentA` when injecting an `IComponent` interface. When we run the application, we should see the following result in the browser:

```
<DecoratorA>Hello from ComponentA</DecoratorA>
```

> You may have noticed a few `new` keywords there, but even though it is not very elegant, we can manually create new instances in the composition root without jeopardizing our application's health. We learn how to get rid of some of them later with the introduction of Scrutor.

Next, let’s create the second decorator.

DecoratorB

Now that we have a decorator, it is time to create a second decorator to demonstrate the power of chaining decorators.**Context**: we need another content wrapper but don’t want to modify existing classes. To achieve this, we concluded that creating a second decorator would be perfect, so we created the following `DecoratorB` class:

```
public class DecoratorB : IComponent
{
    private readonly IComponent _component;
    public DecoratorB(IComponent component)
    {
        _component = component ?? throw new ArgumentNullException(nameof(component));
    }
    public string Operation()
    {
        var result = _component.Operation();
        return $"<DecoratorB>{result}</DecoratorB>";
    }
}
```

The preceding code is similar to `DecoratorA`, but the XML tag is `DecoratorB` instead. The important part is that the decorator depends on and implements the `IComponent` interface and doesn’t depend on a concrete class. This is what gives us the flexibility of decorating any `IComponent`, and this is what enables us to chain decorators.To complete this example, we need to update our composition root like this:

```
builder.Services.AddSingleton<IComponent>(serviceProvider => new DecoratorB(new DecoratorA(new C
```

Now, `DecoratorB` decorates `DecoratorA`, which decorates `ComponentA`. When running the application, you see the following output:

```
<DecoratorB><DecoratorA>Hello from ComponentA</DecoratorA></DecoratorB>
```

And voilà! These decorators allowed us to modify the behavior of `ComponentA` without impacting the code. However, our composition root is beginning to get messy as we instantiate multiple dependencies inside each other, making our application harder to maintain. Moreover, the code is becoming harder to read. Furthermore, the code would be even harder to read if the decorators were also depending on other classes.

> We can use decorators to change the behavior or state of an object. We can be very creative with decorators; for example, you could create a class that queries remote resources over HTTP and then decorate that class with a small component that manages a memory cache of the results, limiting the round trip to the remote server. You could create another decorator that monitors the time needed to query those resources and then log that to Application Insights—so many possibilities.

Next, we eliminate the `new` keywords and clean up our composition root.

Project – Decorator using Scrutor

This update aims to simplify the composition of the system we just created. To achieve this, we use **Scrutor**, an open-source library that allows us to do just that, among other things.We first need to install the Scrutor NuGet package using Visual Studio or the CLI. When using the CLI, run the following command:

```
dotnet add package Scrutor
```

Once Scrutor is installed, we can use the `Decorate` extension method on the `IServiceCollection` to add decorators.By using Scrutor, we can update the following messy line:

```
builder.Services.AddSingleton<IComponent>(serviceProvider => new DecoratorB(new DecoratorA(new C
```

And convert it into these three more elegant lines:

```
builder.Services
    .AddSingleton<IComponent, ComponentA>()
    .Decorate<IComponent, DecoratorA>()
    .Decorate<IComponent, DecoratorB>()
;
```

In the preceding code, we registered `ComponentA` as the implementation of `IComponent`, with a singleton lifetime, just like the first time.Then, by using Scrutor, we told the IoC container to override that first binding and to decorate the already registered `IComponent` (`ComponentA`) with an instance of `DecoratorA` instead. Then, we overrode the second binding by telling the IoC container to return an instance of `DecoratorB` that decorates the last known binding of `IComponent` instead (`DecoratorA`).The result is the same as we did previously, but the code is now more elegant. On top of that improved readability, this lets the container create the instances instead of us using the `new` keyword, adding more flexibility and stability to our system.As a reminder, the IoC container serves the equivalent of the following `instance` when an `IComponent` interface is requested:

```
var instance = new DecoratorB(new DecoratorA(new ComponentA()));
```

Why am I talking about elegance and flexibility? This code is a simple example, but if we add other dependencies to those classes, it could quickly become a complex code block that could become a maintenance nightmare, very hard to read, and have manually managed lifetimes. Of course, if the system is simple, you can always instantiate the decorators manually without loading an external library.

> Whenever possible, keep your code simple. Using Scrutor is one way to achieve this. Code simplicity helps in the long run as it is easier to read and follow, even for someone else reading it. Consider that someone will most likely read your code one day.
>
> > Moreover, adding any external dependency to a project should be considered carefully. Remember that you must keep the dependency up to date, so having too many can take maintenance time. The library's author can also stop maintaining it, and the library will become outdated. The library may introduce breaking changes forcing you to update your code. And so on.
> >
> > Furthermore, there is the security aspect to consider. Supply chain attacks are not uncommon. If you work in a regulated place, you may have to go through a cybersecurity vetting process, etc.
> >
> > Besides those general tips, I've been using Scrutor for many years; I find it very stable and don't remember any breaking changes that caused me issues.

To ensure both programs behave the same, with or without Scrutor, let's explore the following integration test that runs for both projects, ensuring their correctness:

```
namespace Decorator.IntegrationTests;
//...
[Fact]
public async Task Should_return_a_double_decorated_string()
{
    // Arrange
    var client = _webApplicationFactory.CreateClient();
    // Act
    var response = await client.GetAsync("/");
    // Assert
    response.EnsureSuccessStatusCode();
    var body = await response.Content.ReadAsStringAsync();
    Assert.Equal(
        "Operation: <DecoratorB><DecoratorA>Hello from ComponentA</DecoratorA></DecoratorB>",
        body
    );
}
```

The preceding test sends an HTTP request to one of the applications running in memory and compares the server response to the expected value. Since both projects should have the same output, we reuse this test in both the `DecoratorPlainStartupTest` and `DecoratorScrutorStartupTest` classes. They are empty and only routes the test to the correct program. Here's an example of the Visual Studio Test Explorer:

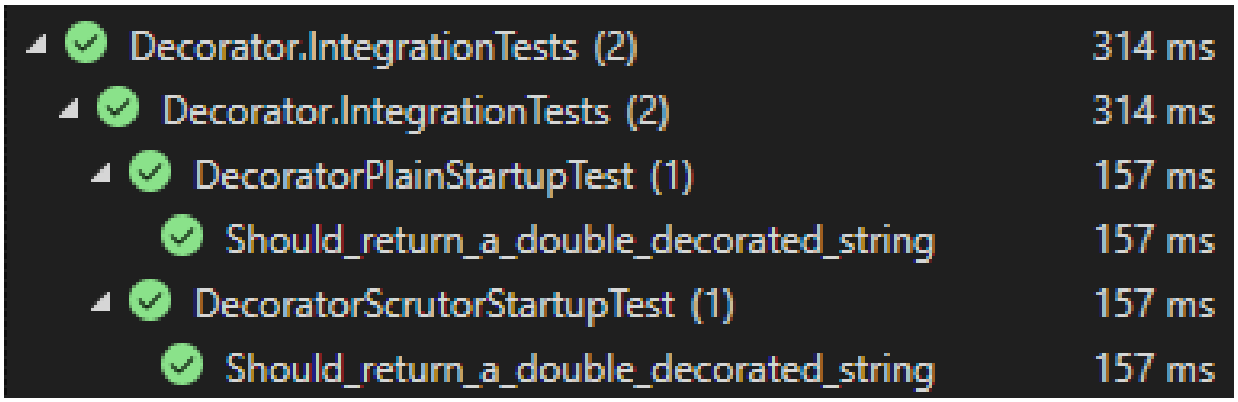


Figure 11.7: A Visual Studio Explorer screenshot displaying the Decorator integration tests.

> You can also do assembly scanning using Scrutor (https://adpg.link/xvfS), which allows you to perform automatic dependency registration. This is outside the scope of this chapter, but it is worth looking into. Scrutor allows you to use the built-in IoC container for more complex scenarios, postponing the need to replace it with a third-party one.

Conclusion

The Decorator pattern is one of our toolbox's simplest yet most powerful design patterns. It augments existing classes without modifying them. A decorator is an independent block of logic that we can use to create complex and granular object trees that fit our needs.We also explored the Scrutor open-source library to assist us in registering our decorator with the container.The Decorator pattern helps us stay in line with the **SOLID** principles (and vice versa), as follows:

- **S**: The Decorator pattern suggests creating small classes to add behaviors to other classes, segregating responsibilities, and fostering reuse.
- **O**: Decorators add behaviors to other classes without modifying them, which is literally the definition of the OCP.
- **L**: N/A
- **I**: By following the ISP, creating decorators for your specific needs should be easy. However, implementing the Decorator pattern may become difficult if your interfaces are too complex. Having a hard time creating a decorator is a good indicator that something is wrong with the design—a code smell. A well-segregated interface should be easy to decorate.
- **D**: Depending on abstractions is the key to the Decorator's power.

Next, we explore the Composite pattern, which helps us manage complex objects' structures differently than the decorator does.

Implementing the Composite design pattern

The Composite design pattern is another structural GoF pattern that helps us manage complex object structures.

Goal

The goal behind the Composite pattern is to create a hierarchical data structure where you don't need to differentiate groups from single components, making the traversal and manipulation of the hierarchy easy for its consumers.

> You could think of the Composite pattern as a way of building a graph or a tree with self-managing nodes.

Design

The design is straightforward; we have *components* and *composites*. Both implement a common interface that defines the shared operations. The *components* are single nodes, while the *composites* are collections of *components*. Let's take a look at a diagram:

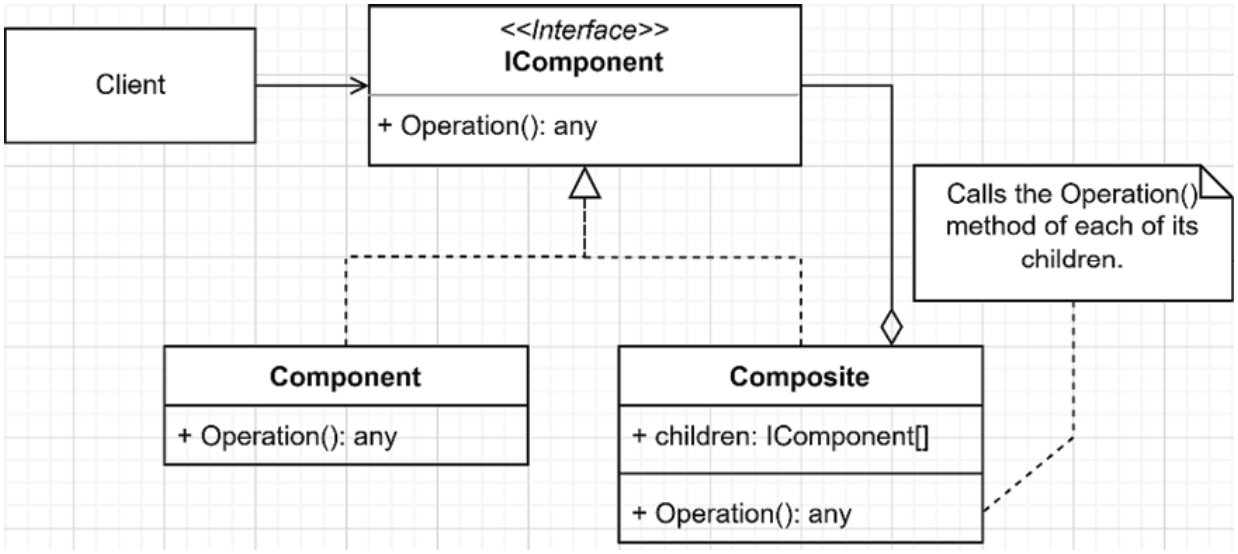


Figure 11.7: Composite class diagram

In the preceding diagram, `Client` depends on an `IComponent` interface and is unaware of the underlying implementation—it could be an instance of a `Component` or a `Composite`; it does not matter. Then, we have two implementations:

- `Component` represents a single element; a leaf.
- `Composite` represents a collection of `IComponent`. The `Composite` object uses its children to manage the hierarchy's complexity by delegating part of the process to them.

Those three pieces put together create the Composite design pattern. Considering that it is possible to add instances of the `Composite` and `Component` classes as children of other `Composite` objects, it is possible to create complex, non-linear, and self-managed data structures with next to no effort.

> You are not limited to one type of component and one type of composite; you can create as many implementations of the `IComponent` interface as you need. Then, you can even mix and match them to create a non-linear tree.

Project – BookStore

Context: We built a program in the past to support a bookstore. However, the store is going so well that our little program is not enough anymore. Our fictional company now owns multiple stores. They want to divide those stores into sections and manage book sets and single books. After a few minutes of gathering information and asking them questions, we realize they can have sets of sets, subsections, and think of creating sub-stores, so we need a flexible design.We have decided to use the Composite pattern to solve this problem. Here's our class hierarchy:

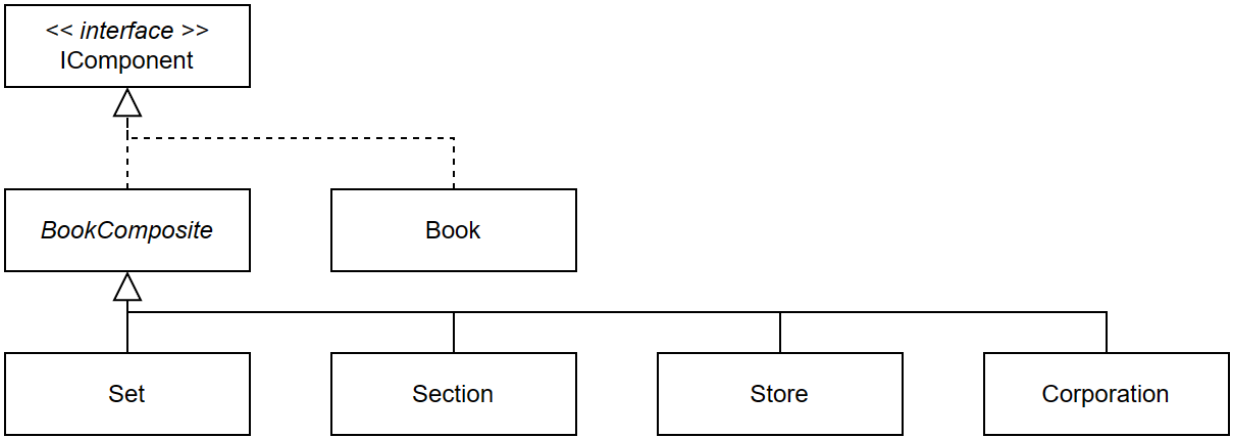


Figure 11.8: the BookStore project composite class hierarchy

Due to the complexity of our class hierarchy and the uncertainty of a project in an early stage, we decided that a factory would be adequate to create our class hierarchy, showcase our design, and validate it with the customer. Here's the high-level design:

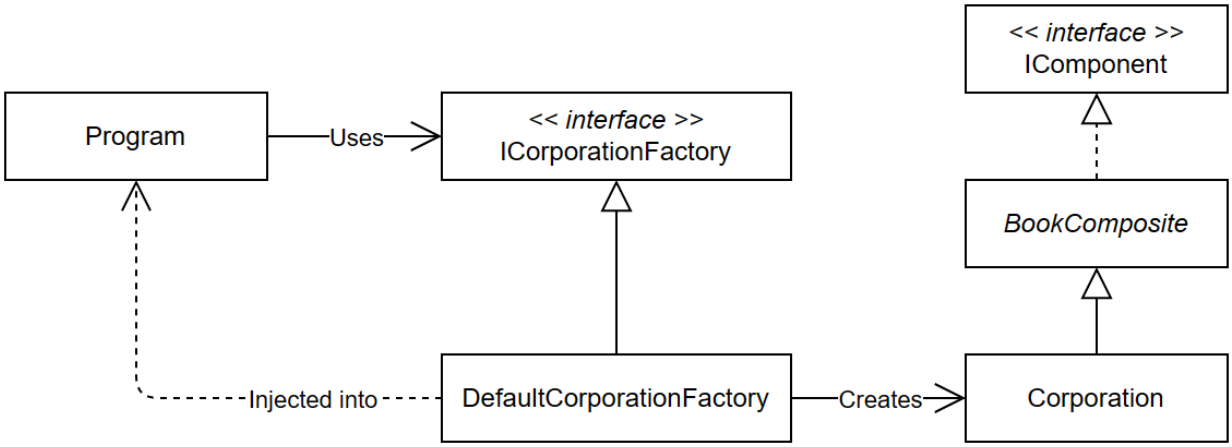


Figure 11.9: high-level design of the BookStore project

We decided to aim for the smallest possible interface to get the ball rolling. Since we want to know how many items are available in any part of the store and what type of component we are interacting with, we created the following interface:

```
namespace Composite.Models;
public interface IComponent
{
    int Count { get; }
    string Type { get; }
}
```

The `Count` property allows us to calculate how many items are available under the corporation, a store, a section, a set, or any other composite component we create in the future. The `Type` property forces each component to display its type linearly.

> We can create such a minimal interface because we are not executing any operations on the data structure but counting the elements, then serializing it to JSON. The serializer will take care of navigating the class hierarchy for us. In another context, the minimal subset of properties might be more than this. For example, in

this case, we could have added a `Name` property to the interface, but the book's name is its title, so I decided not to include it.

Next, let's create our composite structure, starting with the `Book` class (the *Component*):

```
namespace Composite.Models;
public class Book : IComponent
{
    public Book(string title)
    {
        Title = title ?? throw new ArgumentNullException(nameof(title));
    }
    public string Title { get; }
    public string Type => "Book";
    public int Count { get; } = 1;
}
```

The preceding `Book` class implements the interface by always returning a count of 1 because it is a single book, a leaf in the tree. The `Type` property is also hard-coded. As a book, the class requires a title upon construction that it stores in the `Title` property (not inherited and only available to `Book` instances).

In a real scenario, we'd have more properties, like the ISBN and author, but doing so here would just clutter the example. We are not designing a real bookstore but learning about the Composite pattern.

Next, let's create our composite component, the `BookComposite` class:

```
using System.Collections;
using System.Collections.ObjectModel;
namespace Composite.Models;
public abstract class BookComposite : IComponent
{
    protected readonly List<IComponent> children = new();
    public BookComposite(string name)
    {
        Name = name ?? throw new ArgumentNullException(nameof(name));
    }
    public string Name { get; }
    public virtual string Type => GetType().Name;
    public virtual int Count
        => children.Sum(child => child.Count);
    public virtual IEnumerable Children
        => new ReadOnlyCollection<IComponent>(children);
    public virtual void Add(IComponent bookComponent)
    {
        children.Add(bookComponent);
    }
    public virtual void Remove(IComponent bookComponent)
    {
        children.Remove(bookComponent);
    }
}
```

The `BookComposite` class implements the following shared features:

- Children management (highlighted in the code).
- Setting the `Name` property of the composite object and forcing the classes inheriting it to set a name upon construction.
- Automatically finds and sets the `Type` name of its derived class.
- Counting the number of children (and, implicitly, the children's children).
- Exposing the children through the `Children` property and ensuring consumers can't modify the collection from the outside by returning a `ReadOnlyCollection` object.

Using the LINQ `Sum()` extension method in the `children.Sum(child => child.Count());` expression allowed us to replace a more complex `for` loop and an accumulator variable.

Adding the `virtual` modifier to the `Type` property allows sub-types to override the property in case their type's name does not reflect the type that should be displayed in the program.

Now, we can start implementing the other classes of our complex composite hierarchy and assign a responsibility to each class, showing how flexible the Composite pattern is.The following classes inherit from the `BookComposite` class:

- The `Corporation` class represents the corporation that owns multiple stores. However, it is not limited to owning stores; a corporation could own other corporations, stores, or any other `IComponent` .
- The `Store` class represents a bookstore.
- The `Section` class represents a section of a bookstore, an aisle, or a category of books.
- The `Set` class represents a book set, such as a trilogy.

These can be composed of any `IComponent` , making this an ultra-flexible data structure. Let's look at the code for these `BookComposite` sub-types, starting with the `Corporation` class:

```
namespace Composite.Models;
public class Corporation : BookComposite
{
    public Corporation(string name, string ceo)
        : base(name)
    {
        CEO = ceo;
    }
    public string CEO { get; }
}
```

The corporation contributes a CEO to the model because someone has to manage the place.Next, we look at the `Store` class:

```
namespace Composite.Models;
public class Store : BookComposite
{
    public string Location { get; }
    public string Manager { get; }
    public Store(string name, string location, string manager)
        : base(name)
    {
        Location = location;
        Manager = manager;
    }
}
```

On top of the `BookComposite` members, a store has a manager and a location.Now, the `Section` class does not add anything, but we can use it as a flexible organizer:namespace Composite.Models;

```
public class Section : BookComposite
{
    public Section(string name) : base(name) { }
}
```

Finally, the `Set` class allows creating the book set upon construction through the books parameter:

```
namespace Composite.Models;
public class Set : BookComposite
{
    public Set(string name, params IComponent[] books)
        : base(name)
    {
        foreach (var book in books)
        {
            Add(book);
        }
    }
}
```

Composing a set of books upon creation of the instance will be convenient later when we assemble the tree.Next, let's explore the last part of the program that helps encapsulate the data structure's creation: the factory.

> The factory is not part of the Composite pattern, but now that we know what a factory is, we can use one to encapsulate the creation logic of our data structure and talk about it.

The factory interface looks like the following:

```
public interface ICorporationFactory
{
    Corporation Create();
}
```

The default concrete implementation of the `ICorporationFactory` interface is the `DefaultCorporationFactory` class. It creates a large non-linear data structure with sections, subsections, sets, and subsets. This whole structure is defined using our composite model in the `DefaultCorporationFactory` class. Due to its large size, let's start with the class's skeleton and its `Create` method:

```
using Composite.Models;
namespace Composite.Services;
public class DefaultCorporationFactory : ICorporationFactory
{
    public Corporation Create()
    {
        var corporation = new Corporation(
            "Boundless Shelves Corporation",
            "Bosmang Kapawu"
        );
        corporation.Add(CreateTaleTowersStore());
        corporation.Add(CreateEpicNexusStore());
        return corporation;
    }
    // ...
}
```

In the preceding `Create` method, we create the corporation, add two stores, then return the result.The `CreateTaleTowersStore` and `CreateEpicNexusStore` methods create a store, set their name, address, and manager, and create three sections each:

```
private IComponent CreateTaleTowersStore()
{
    var store = new Store(
        "Tale Towers",
        "125 Enchantment Street, Storyville, SV 72845",
        "Malcolm Reynolds"
    );
    store.Add(CreateFantasySection());
    store.Add(CreateAdventureSection());
    store.Add(CreateDramaSection());
    return store;
}
private IComponent CreateEpicNexusStore()
{
    var store = new Store(
        "Epic Nexus",
        "369 Parchment Plaza, Novelty, NV 68123",
        "Ellen Ripley"
    );
    store.Add(CreateFictionSection());
    store.Add(CreateFantasySection());
    store.Add(CreateAdventureSection());
    return store;
}
```

Both stores share two sections (have the same books; highlighted code), each with a unique section. If we look at the `CreateFictionSection` method, it adds an imaginary book and a subsection:

```
private IComponent CreateFictionSection()
{
    var section = new Section("Fiction");
    section.Add(new Book("Some alien cowboy"));
    section.Add(CreateScienceFictionSection());
    return section;
}
```

The `CreateScienceFictionSection` method adds an invented book and the Star Wars book set composed of three trilogies (a set of sets):

```
private IComponent CreateScienceFictionSection()
{
    var section = new Section("Science Fiction");
    section.Add(new Book("Some weird adventure in space"));
    section.Add(new Set(
        "Star Wars",
        new Set(
            "Prequel trilogy",
            new Book("Episode I: The Phantom Menace"),
            new Book("Episode II: Attack of the Clones"),
            new Book("Episode III: Revenge of the Sith")
        ),
        new Set(
            "Original trilogy",
            new Book("Episode IV: A New Hope"),
            new Book("Episode V: The Empire Strikes Back"),
            new Book("Episode VI: Return of the Jedi")
        ),
        new Set(
            "Sequel trilogy",
            new Book("Episode VII: The Force Awakens"),
            new Book("Episode VIII: The Last Jedi"),
            new Book("Episode IX: The Rise of Skywalker")
        )
    ));
    return section;
}
```

Now, if we look at this part of the data structure, we have the following:

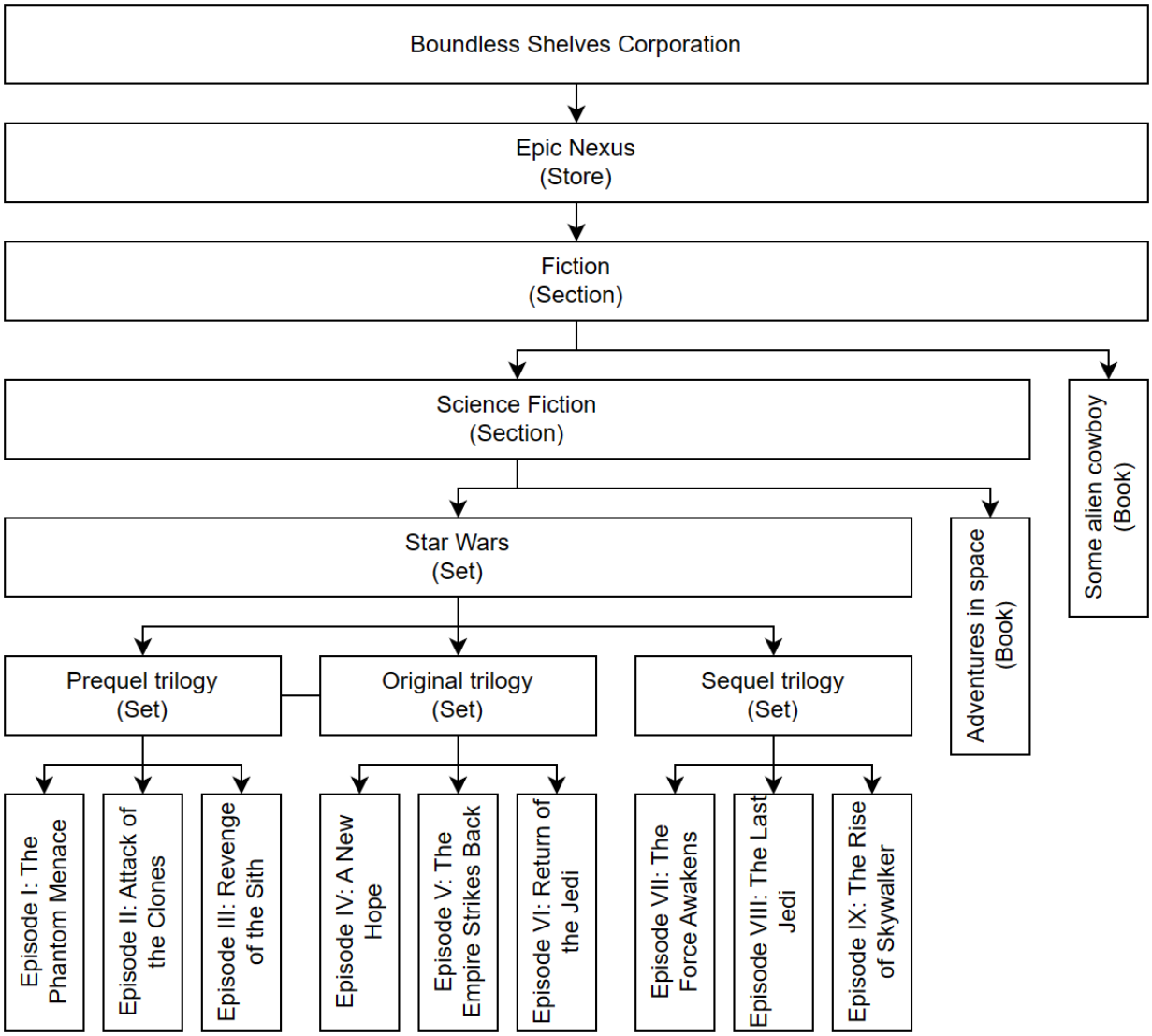


Figure 11.10: The Fiction section of the Epic Nexus store data

In the big scheme of things, the whole organizational structure, down to the section level (without the books and sets), looks like this:

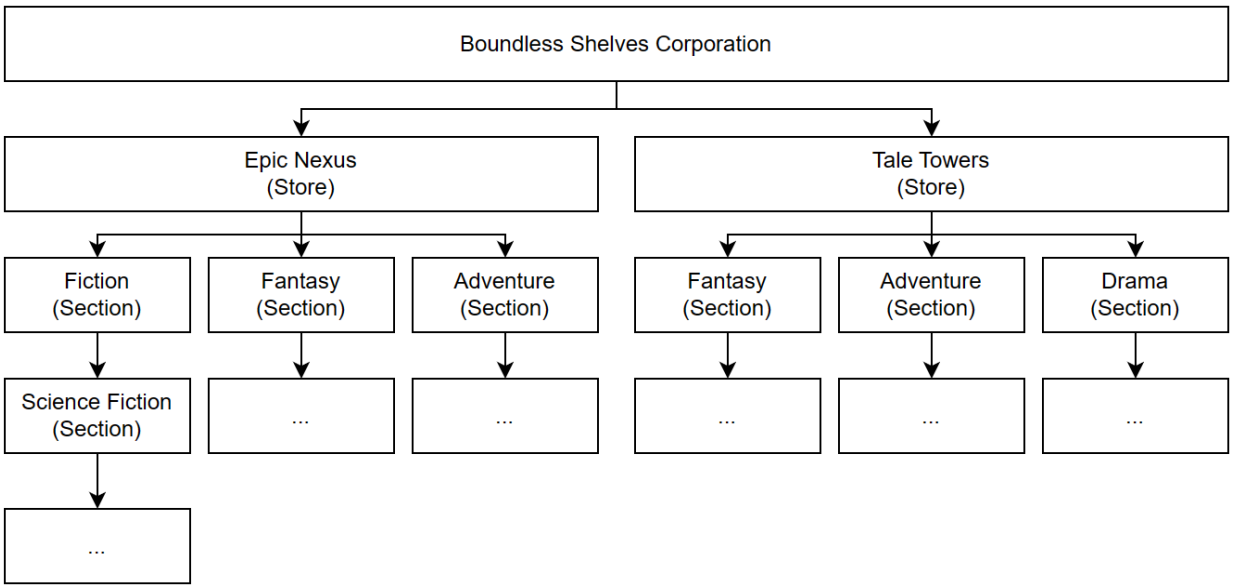


Figure 11.11: the composite hierarchy without the books and sets

I omitted to publish the whole data structure, including the books, as an image because it is too large and would be hard to read. Rest assured, the content itself is unimportant, and the section we are studying is enough to understand the flexibility the composite pattern brings to the design.

As we explore this, we can see how flexible the design is. We can create almost any organizational structure we want.Now, let's look at the `Program.cs` file and register our dependencies and an endpoint to query the data structure:

```
using Composite.Services;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<ICorporationFactory, DefaultCorporationFactory>();
var app = builder.Build();
app.MapGet(
    "/",
    (ICorporationFactory corporationFactory)
        => corporationFactory.Create()
);
app.Run();
```

The preceding code registers the factory that creates the corporation data structure with the container and an endpoint to serve it.When we execute the code, we get the full data structure or the corporation. For brevity reasons, the following JSON represents the fiction section, excluding the books:

```
{
  "ceo": "Bosmang Kapawu",
  "name": "Boundless Shelves Corporation",
  "type": "Corporation",
  "count": 43,
  "children": [
    {
      "location": "369 Parchment Plaza, Novelty, NV 68123",
      "manager": "Ellen Ripley",
      "name": "Epic Nexus",
      "type": "Store",
      "count": 25,
      "children": [
        {
          "name": "Fiction",
          "type": "Section",
          "count": 11,
          "children": [
```

```
            {
              "name": "Science Fiction",
              "type": "Section",
              "count": 10,
              "children": [
                {
                  "name": "Star Wars",
                  "type": "Set",
                  "count": 9,
                  "children": [
                    {
                      "name": "Prequel trilogy",
                      "type": "Set",
                      "count": 3,
                      "children": []
                    },
                    {
                      "name": "Original trilogy",
                      "type": "Set",
                      "count": 3,
                      "children": []
                    },
                    {
                      "name": "Sequel trilogy",
                      "type": "Set",
                      "count": 3,
                      "children": []
                    }
                  ]
                }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

> The value of the `count` fields reflect the total count. In this case, there is no book so the count should be 0. If you run the program and play with the preprocessor symbols define in the `DefaultCorporationFactory.cs` file (`ADD_BOOKS` , `ADD_SETS` , and `ONLY_FICTION`), you will end up with different number.

The Composite pattern allowed us to render a complex data structure in a small method call. Since each component autonomously handles itself, the Composite pattern removes the burden of managing this complexity from the consumer.I encourage you to play around with the existing data structure so that you understand the pattern. You could also try adding a `Movie` class to manage movies; a bookstore must diversify its activities. You could also differentiate movies from books so that customers are not confused. The bookstores could have physical and digital books as well.If you are still looking for more, try building a new application from scratch and use the Composite pattern to create, manage, and display a multi-level menu structure or a file system API.

Conclusion

The Composite pattern effectively builds, manages, and maintains complex non-linear data structures. Its power is primarily in its self-management capabilities. Each node, component, or composite is responsible for its own logic, leaving little to no work for the composite's consumers. Of course, a more complex scenario would have led to a more complex interface.Using the Composite pattern helps us follow the **SOLID** principles in the following ways:

- **S**: It helps divide multiple elements of a complex data structure into small classes to split responsibilities.
- **O**: By allowing us to "mix and match" different implementations of `IComponent` interface, the Composite pattern allows us to extend the data structure without impacting the other existing classes. For example, you could create a new class that implements `IComponent` and start using it immediately without modifying any other component classes.
- **L**: N/A

- **I**: The Composite pattern may violate the ISP when single items implement operations that only impact the collections, like the `Add` and `Remove` methods, but we have not done this here.
- **D**: The Composite pattern actors depend solely on `IComponent` which invert the dependency flow.

Next, we move to a different type of structural pattern that adapts one interface to another.

Implementing the Adapter design pattern

The Adapter pattern is another structural design pattern that allows two incompatible interfaces to work together without modifying their existing code. This pattern introduces a wrapper class called the *Adapter*, which bridges the gap between the interfaces.

Goal

The Adapter design pattern is applicable when we want to use an existing class, but its interface is incompatible with what we want to use it for. Instead of refactoring the class, which could introduce bugs or errors in the existing codebase or even cascade changes to other parts of the system, we can use an *Adapter* class to make the class's interface compatible with the *Target* interface. The Adapter pattern is handy when we cannot change the *Adaptee's* code or do not want to change it.

Design

You can think of the adapter as a power outlet's universal adapter; you can connect a North American device to a European outlet by connecting it to the adapter and then to the power outlet. The Adapter design pattern does precisely that but for APIs.Let's start by looking at the following diagram:

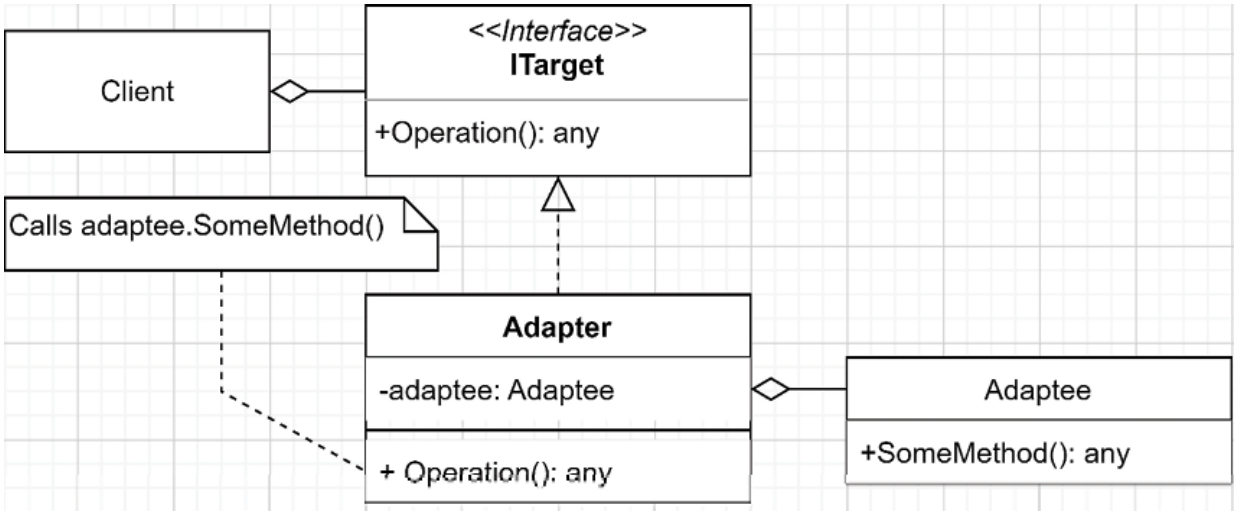


Figure 11.12: Adapter class diagram

In the preceding diagram, we have the following actors:

- The `ITarget` interface holds the contract we want (or have) to use.
- The `Adaptee` class represents the concrete component we want to use that does not conform to `ITarget`.
- The `Adapter` class adapts the `Adaptee` class to the `ITarget` interface.

There is a second way of implementing the Adapter pattern that implies inheritance. If you can go for composition, go for it, but if you need access to `protected` methods or other internal states of `Adaptee`, you can go for inheritance instead, like this:

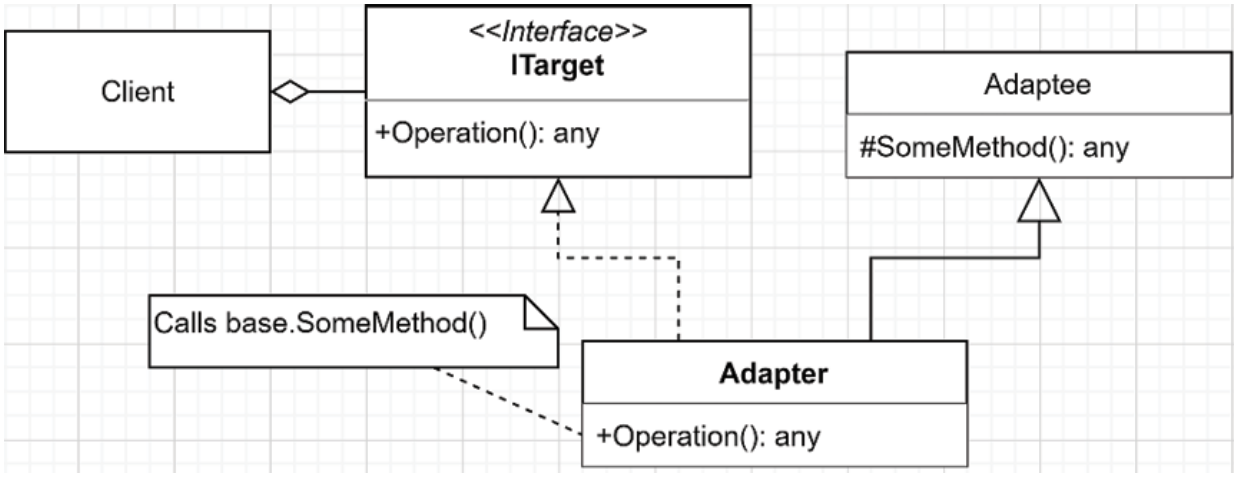


Figure 11.13: Adapter class diagram inheriting the Adaptee

The actors are the same, but instead of composing the `Adapter` class with the `Adaptee` class, the `Adapter` class inherits from the `Adaptee` class. This design makes the `Adapter` class become both an `Adaptee` and an `ITarget` .Let's explore how this looks in code.

Project – Greeter

Context: We've programmed a highly sophisticated greeting system that we want to reuse in a new program. However, its interface does not match the new design, and we cannot modify it because other systems use that greeting system.To fix this problem, we decided to apply the Adapter pattern. Here is the code of the external greeter (`ExternalGreeter`) and the new interface (`IGreeter`) used in the new system. This code must not directly modify the `ExternalGreeter` class to prevent any breaking changes from occurring in other systems:

```
public interface IGreeter
{
    string Greeting();
}
public class ExternalGreeter
{
    public string GreetByName(string name)
    {
        return $"Adaptee says: hi {name}!";
    }
}
```

Next is how the external greeter is adapted to meet the latest requirements:

```
public class ExternalGreeterAdapter : IGreeter
{
    private readonly ExternalGreeter _adaptee;
    public ExternalGreeterAdapter(ExternalGreeter adaptee)
    {
        _adaptee = adaptee ?? throw new ArgumentNullException(nameof(adaptee));
    }
    public string Greeting()
    {
        return _adaptee.GreetByName("ExternalGreeterAdapter");
    }
}
```

In the preceding code, the actors are as follows:

- The `IGreeter` interface represents the *Target* and is the interface that we must use.

- The `ExternalGreeter` class represents the *Adaptee* and is the external component that already contains all the logic that someone programmed and tested. That code could be in an external assembly or installed from a NuGet package.
- The `ExternalGreeterAdapter` class represents the *Adapter* and is where the adapter does its job. In this case, the `Greeting` method calls the `GreetByName` method of the `ExternalGreeter` class, which implements the greeting logic.

Now, we can call the `Greeting` method and get the result of the `GreetByName` call. With this in place, we can reuse the existing logic through the `ExternalGreeterAdapter` class.

> We can also test `IGreeter` consumers by mocking the `IGreeter` interface without dealing with the `ExternalGreeterAdapter` class.

In this case, the "complex logic" is pretty simple, but we are here for the Adapter pattern, not for imaginary business logic. Now, let's take a look at the consumer:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<ExternalGreeter>();
builder.Services.AddSingleton<IGreeter, ExternalGreeterAdapter>();
var app = builder.Build();
app.MapGet("/", (IGreeter greeter) => greeter.Greeting());
app.Run();
```

In the preceding code, we composed our application by registering the `ExternalGreeterAdapter` class as a singleton bound to the `IGreeter` interface. We also informed the container to provide a single instance of `ExternalGreeter` class whenever requested (in this case, we inject it into the `ExternalGreeterAdapter` class).Then, the consumer (*Client* in the class diagrams) is the highlighted endpoint where the `IGreeter` interface is injected as a parameter. Then, the delegate calls the `Greeting` method on that injected instance and outputs the greeting message to the response.The following diagram represents what's happening in this system:

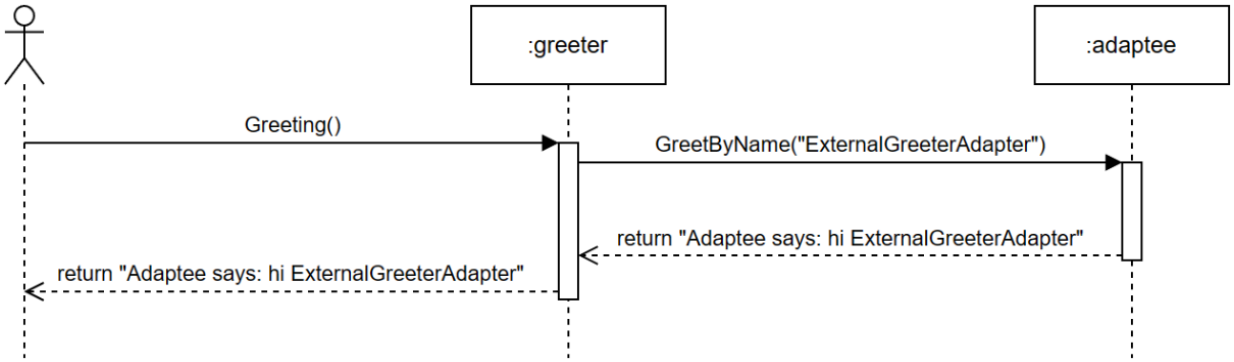


Figure 11.14: Greeter system sequence diagram

And voilà! We've adapted the `ExternalGreeterAdapter` class to the `IGreeter` interface with little effort.

Conclusion

The Adapter pattern is another simple pattern that offers flexibility. With it, we can use older or non-conforming components without rewriting them. Of course, depending on the *Target* and *Adaptee* interfaces, you may need to put more or less effort into writing the code of the *Adapter* class.Now, let's learn how the Adapter pattern can help us follow the **SOLID** principles:

- **S**: The Adapter pattern has only one responsibility: make an interface work with another interface.
- **O**: The Adapter pattern allows us to modify the *Adaptee's* interface without the need to modify its code.
- **L**: Inheritance is not much of a concern regarding the Adapter pattern, so this principle does not apply once again. If *Adapter* inherits from *Adaptee*, the goal is to change its interface, not its behavior, which should

conform to the LSP.
- **I**: We can view the *Adapter* class as a facilitator to the ISP, with the *Target* interface as the ultimate destination. The Adapter pattern relies on the design of the *Target* interface but doesn't directly influence it. Per this principle, our primary focus should be to design the *Target* interface in a manner that abides by the ISP.
- **D**: The Adapter pattern introduces only an implementation of the *Target* interface. Even if the *Adapter* depends on a concrete class, it breaks the direct dependency on that external component by adapting it to the *Target* interface.

Next, we explore the last structural pattern of the chapter that teaches foundational concepts.

Implementing the Façade design pattern

The Façade pattern is a structural pattern that simplifies the access to a complex system. It is very similar to the Adapter pattern, but it creates a wall (a façade) between one or more subsystems. The big difference between the adapter and the façade is that instead of adapting an interface to another, the façade simplifies the use of a subsystem, typically by using multiple classes of that subsystem.

> We can apply the same idea to shielding one or more programs, but in this case, we call the façade a gateway —more on that in *Chapter 19*, *Introduction to Microservices Architecture*.

The Façade pattern is extremely useful and can be adapted to multiple situations.

Goal

The Façade pattern aims to simplify the use of one or more subsystems by providing an interface that is easier to use than the subsystems themselves, shielding the consumers from that complexity.

Design

Imagine a system with a multitude of complex classes. Direct interaction between the consuming code and these classes can become problematic due to coupling, complexity, and low code readability and maintainability. The Facade design pattern offers a solution by providing a unified interface to a set of APIs in a subsystem, making it easier to use.The Facade class contains references to the objects of the complex subsystem and delegates client requests to the appropriate subsystem object. From a client's perspective, it only interacts with a single, simplified interface represented by the Facade. Behind the scenes, the Facade coordinates with the subsystem's components to fulfill the client's request.We could create multiple diagrams representing a multitude of subsystems, but let’s keep things simple. Remember that you can replace the single subsystem shown in the following diagram with as many subsystems as you need to adapt:

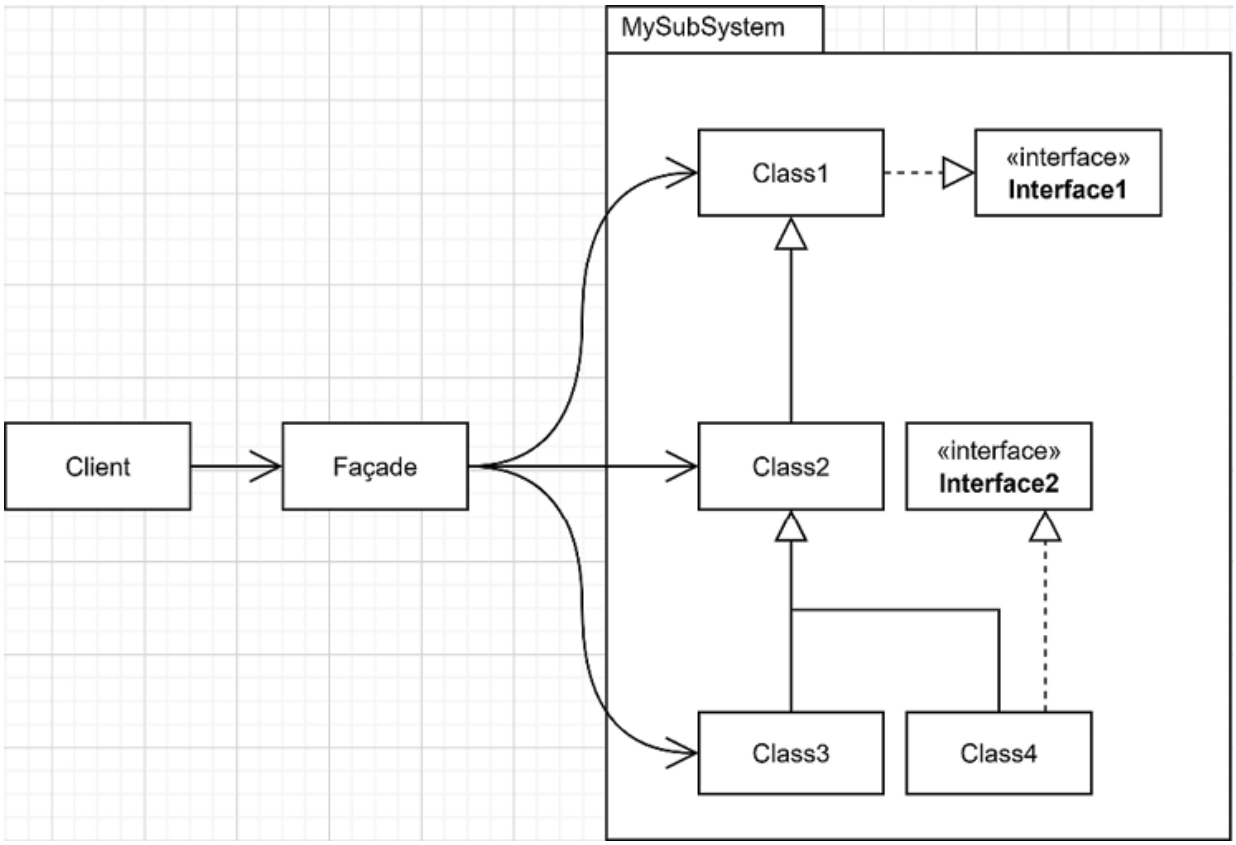


Figure 11.15: A class diagram representing a Façade object that hides a complex subsystem

The *Façade* plays the intermediary between the *Client* and the subsystem, simplifying its usage. Let's see this in action as a sequence diagram:

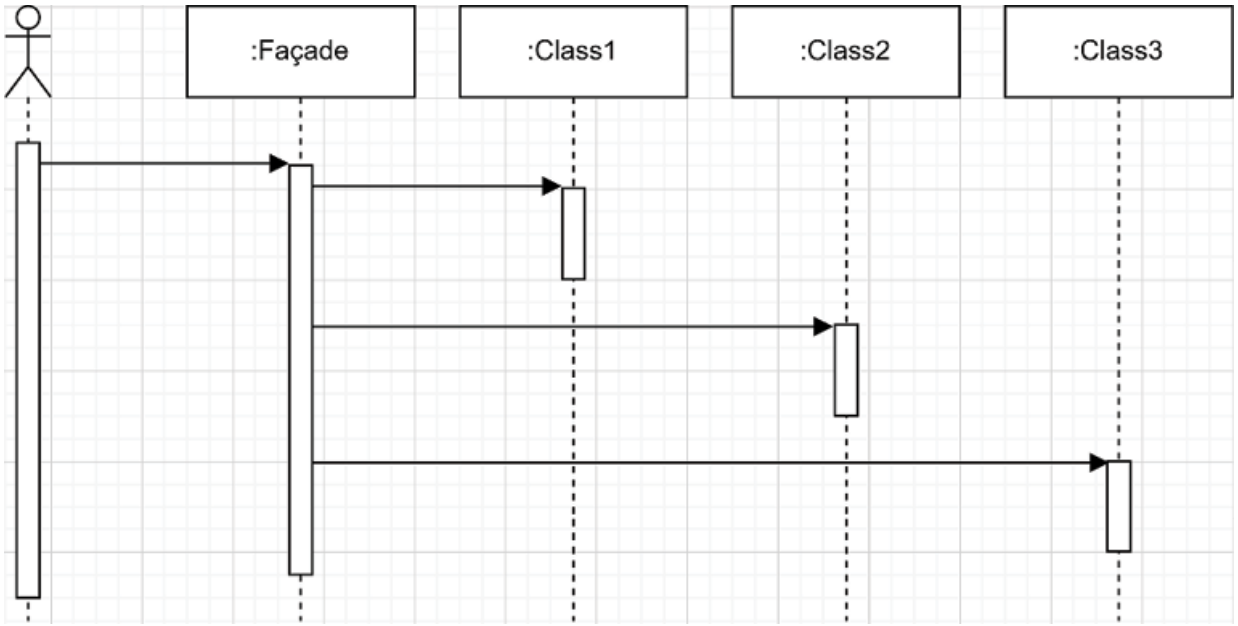


Figure 11.16: A sequence diagram representing a Façade object that interacts with a complex subsystem

In the preceding diagram, the *Client* calls the *Façade* once, while the *Façade* places multiple calls against different classes.There are multiple ways of implementing a façade:

- **Opaque façades**: In this form, the `Façade` class is inside the subsystem. All other classes of the subsystem have an `internal` visibility modifier. This way, only the classes inside the subsystem can interact with the other internal classes, forcing the consumers to use the `Façade` class.
- **Transparent façades**: In this form, the classes can have a `public` modifier, allowing the consumers to use them directly or to use the `Façade` class. This way, we can create the `Façade` class inside or outside the subsystem.
- **Static façades**: In this form, the `Façade` class is `static`. We can implement a static façade as opaque or transparent.

I recommend using static façades as a last resort because `static` elements limit flexibility and testability.

We look at some code next.

Project – The façades

In this example, we play with the following C# projects:

- The *OpaqueFacadeSubSystem* class library showcases an **opaque façade**.
- The *TransparentFacadeSubSystem* class library showcases a **transparent façade**.
- The *Facade* project is a REST API that consumes the façades. It exposes two endpoints to access the *OpaqueFacadeSubSystem* project, and two others that target the *TransparentFacadeSubSystem* project.

Let’s start with the class libraries.

To follow the SOLID principles, adding some interfaces representing the elements of the subsystem seemed appropriate. In subsequent chapters, we explore how to organize our abstractions to be more reusable, but for now, both abstractions and implementations are in the same assembly.

Opaque façade

In this assembly, only the façade is public; all the other classes are internal, which means they are hidden from the external world. In most cases, this is not ideal; hiding everything makes the subsystem less flexible and harder to extend.However, you may want to control access to your internal APIs in some scenarios. This may be because they are not mature enough and you don’t want any third party to depend on them, or for any other reasons you deem appropriate for your specific use case.Let’s start by taking a look at the following subsystem code:

```
// An added interface for flexibility
public interface IOpaqueFacade
{
    string ExecuteOperationA();
    string ExecuteOperationB();
}
// A hidden component
internal class ComponentA
{
    public string OperationA() => "Component A, Operation A";
    public string OperationB() => "Component A, Operation B";
}
// A hidden component
internal class ComponentB
{
    public string OperationC() => "Component B, Operation C";
    public string OperationD() => "Component B, Operation D";
}
// A hidden component
internal class ComponentC
{
    public string OperationE() => "Component C, Operation E";
```

```
    public string OperationF() => "Component C, Operation F";
}
// The opaque façade using the other hidden components
public class OpaqueFacade : IOpaqueFacade
{
    private readonly ComponentA _componentA;
    private readonly ComponentB _componentB;
    private readonly ComponentC _componentC;
    // Using constructor injection
    internal OpaqueFacade(ComponentA componentA, ComponentB componentB, ComponentC componentC)
    {
        _componentA = componentA ?? throw new ArgumentNullException(nameof(componentA));
        _componentB = componentB ?? throw new ArgumentNullException(nameof(componentB));
        _componentC = componentC ?? throw new ArgumentNullException(nameof(componentC));
    }
    public string ExecuteOperationA()
    {
        return new StringBuilder()
            .AppendLine(_componentA.OperationA())
            .AppendLine(_componentA.OperationB())
            .AppendLine(_componentB.OperationD())
            .AppendLine(_componentC.OperationE())
            .ToString();
    }
    public string ExecuteOperationB()
    {
        return new StringBuilder()
            .AppendLine(_componentB.OperationC())
            .AppendLine(_componentB.OperationD())
            .AppendLine(_componentC.OperationF())
            .ToString();
    }
}
```

The `OpaqueFacade` class is coupled with `ComponentA`, `ComponentB`, and `ComponentC` directly. There was no point in extracting any `internal` interfaces since the subsystem is not extensible anyway. We could have done this to offer some kind of internal flexibility, but in this case, there was no advantage.Besides this coupling, `ComponentA`, `ComponentB`, and `ComponentC` define two methods each, which return a string describing their source. With that code in place, we can observe what is happening and how the final result was composed. `OpaqueFacade` also exposes two methods, each composing a different message using the underlying subsystem's components. This is a classic use of a façade; the façade queries other objects more or less complicatedly and then does something with the results, taking away the caller's burden of knowing the subsystem.Since the members use the `internal` visibility modifier, we can't directly register the dependencies with the IoC container from the program. To solve this problem, the subsystem can register its dependencies by adding an extension method. The following extension method is accessible by the consuming application:

```
public static class StartupExtensions
{
    public static IServiceCollection AddOpaqueFacadeSubSystem(this IServiceCollection services)
    {
        services.AddSingleton<IOpaqueFacade>(serviceProvider
            => new OpaqueFacade(new ComponentA(), new ComponentB(), new ComponentC()));
        return services;
    }
}
```

The preceding code manually creates the dependencies and adds a binding to the `IOpaqueFacade` interface so the system can use it. This hides everything but the interface from the consumer.Before exploring the REST API, we look at the transparent façade implementation.

Transparent façade

The transparent façade is the most flexible type of façade and is exceptionally suitable for a system that leverages dependency injection. The implementation is similar to the opaque façade, but the `public` visibility modifier

changes how consumers can access the class library elements. For this system, it was worth adding interfaces to allow the consumers of the subsystem to extend it when needed.First, let’s take a look at the abstractions:

```
namespace TransparentFacadeSubSystem.Abstractions
{
    public interface ITransparentFacade
    {
        string ExecuteOperationA();
        string ExecuteOperationB();
    }
    public interface IComponentA
    {
        string OperationA();
        string OperationB();
    }
    public interface IComponentB
    {
        string OperationC();
        string OperationD();
    }
    public interface IComponentC
    {
        string OperationE();
        string OperationF();

}
```

The API of this subsystem is the same as the opaque façade. The only difference is how we can use and extend the subsystem (from a consumer standpoint). The implementations are mostly the same as well, but the classes implement the interfaces and are `public` ; the highlighted elements represent those changes:

```
namespace TransparentFacadeSubSystem
{
    public class ComponentA : IComponentA
    {
        public string OperationA() => "Component A, Operation A";
        public string OperationB() => "Component A, Operation B";
}
    public class ComponentB : IComponentB
    {
        public string OperationC() => "Component B, Operation C";
        public string OperationD() => "Component B, Operation D";
    }
    public class ComponentC : IComponentC
    {
        public string OperationE() => "Component C, Operation E";
        public string OperationF() => "Component C, Operation F";
    }
    public class TransparentFacade : ITransparentFacade
    {
        private readonly IComponentA _componentA;
        private readonly IComponentB _componentB;
        private readonly IComponentC _componentC;
    public TransparentFacade(IComponentA componentA, IComponentB
componentB, IComponentC componentC)
    {
        _componentA = componentA ?? throw new ArgumentNullException(nameof(componentA));
        _componentB = componentB ?? throw new ArgumentNullException(nameof(componentB));
        _componentC = componentC ?? throw new ArgumentNullException(nameof(componentC));
    }
        public string ExecuteOperationA()
        {
            return new StringBuilder()
                .AppendLine(_componentA.OperationA())
                .AppendLine(_componentA.OperationB())
                .AppendLine(_componentB.OperationD())
                .AppendLine(_componentC.OperationE())
                .ToString();
            }
        public string ExecuteOperationB()
```

```
        {
            return new StringBuilder()
                .AppendLine(_componentB.OperationC())
                .AppendLine(_componentB.OperationD())
                .AppendLine(_componentC.OperationF())
                .ToString();
        }
    }
}
```

To simplify the use of the subsystem, we create the following extension method as a good practice that makes consuming the subsystem easier. Everything that we define in that method can be overridden from the composition root (which is not the case for the opaque façade):

```
public static class StartupExtensions
{
    public static IServiceCollection AddTransparentFacadeSubSystem(this IServiceCollection servi
    {
        services.AddSingleton<ITransparentFacade, TransparentFacade>();
        services.AddSingleton<IComponentA, ComponentA>();
        services.AddSingleton<IComponentB, ComponentB>();
        services.AddSingleton<IComponentC, ComponentC>();
        return services;
    }
}
```

All the `new` elements are gone and have been replaced by simple dependency registration (singleton lifetimes, in this case). These little differences give us the tools to reimplement any part of the subsystem if we want to, as we cover soon.

> We can register bindings in the transparent façade extension method because classes and interfaces are `public`. The container needs a public constructor to do its work.
>
> In the opaque façade, we had to define the constructor of the `OpaqueFacade` class as `internal` because the type of its parameters (`ComponentA`, `ComponentB`, and `ComponentC`) are internal, making it impossible to leverage the container. Changing the visibility modifier of the opaque façade constructor from `internal` to `public` would have yielded a *CS0051 Inconsistent accessibility* error.

Besides those differences, the transparent façade plays the same role as the opaque façade, outputting the same result.We consume those two façades next.

The program

Now, let’s analyze the consumer, an ASP.NET Core application that forwards HTTP requests to the façades and return the result as their response.The first step is to register the dependencies like this:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services
    .AddOpaqueFacadeSubSystem()
    .AddTransparentFacadeSubSystem()
;
```

> With these extension methods, the application root is so clean that it is hard to know that we registered two subsystems against the IoC container. This is a good way of keeping your code organized and clean, especially when you’re building class libraries.

Now that everything has been registered, the second thing we need to do is route those HTTP requests to the façades. Let’s take a look at the code first:

```
var app = builder.Build();
app.MapGet(
    "/opaque/a",
    (IOpaqueFacade opaqueFacade)
```

```
        => opaqueFacade.ExecuteOperationA()
);
app.MapGet(
    "/opaque/b",
    (IOpaqueFacade opaqueFacade)
        => opaqueFacade.ExecuteOperationB()
);
app.MapGet(
    "/transparent/a",
    (ITransparentFacade transparentFacade)
        => transparentFacade.ExecuteOperationA()
);
app.MapGet(
    "/transparent/b",
    (ITransparentFacade transparentFacade)
        => transparentFacade.ExecuteOperationB()
);
app.Run();
```

In the preceding block, we define four routes. Each route dispatches the request to one of the façade's methods (highlighted code) using the façade that is injected in its delegate.If you run the program and navigate to the `/transparent/a` endpoint, the page should display the following:

```
Component A, Operation A
Component A, Operation B
Component B, Operation D
Component C, Operation E
```

What happened is located inside the delegates. It uses the injected `ITransparentFacade` service and calls its `ExecuteOperationA()` method, and then outputs the `result` variable to the response stream.Now, let's define how `ITransparentFacade` is composed:

- `ITransparentFacade` is an instance of `TransparentFacade`.
- We inject `IComponentA`, `IComponentB`, and `IComponentC` in the `TransparentFacade` class.
- These dependencies are instances of `ComponentA`, `ComponentB`, and `ComponentC`, respectively.

Visually, the following flow happens:

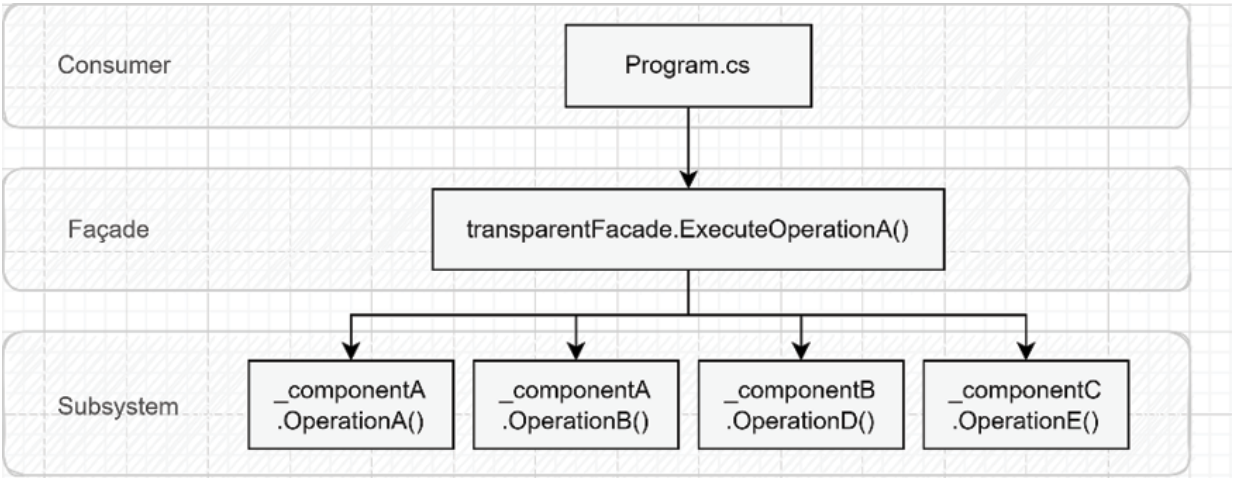


Figure 11.17: A representation of the call hierarchy that occurs when the consumer executes the ExecuteOperationA method

In the preceding diagram, we can see the shielding that's done by the façade and how it has made the consumer's life easier: one call instead of four.

One of the hardest parts of using dependency injection is its abstractness. If you are not sure how all those parts are assembled, add a breakpoint into Visual Studio (let's say, on the `var result = transparentFacade.ExecuteOperationA()` line) and run the application in debug mode. From there, **Step Into** each method call. That should help you figure out what is happening. Using the debugger to find the concrete types and their states can help find details about a system or diagnose bugs.

To use **Step Into**, you can use the following button or hit **F11**:

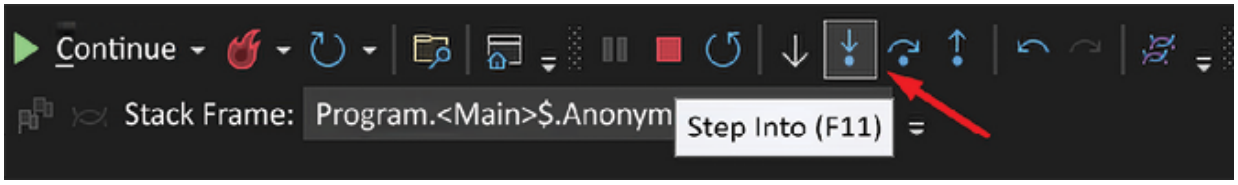


Figure 11.18: The Visual Studio Step Into (F11) button

Calling the other endpoints lead to similar results. As a reference, here's the result from the other endpoints.Here's the result for the `/transparent/b` endpoint:

```
Component B, Operation C
Component B, Operation D
Component C, Operation F
```

Here's the results for the `/opaque/a` endpoint:

```
Component A, Operation A
Component A, Operation B
Component B, Operation D
Component C, Operation E
```

Here's the result for the `/opaque/b` endpoint:

```
Component B, Operation C
Component B, Operation D
Component C, Operation F
```

Next, let's update some results without changing the component's code.

Flexibility in action

As discussed, the transparent façade adds more flexibility. Here, we explore this flexibility in action.**Context**: We want to change the behavior of the `TransparentFacade` class. At the moment, the result of the `transparent/b` endpoint looks like this:

```
Component B, Operation C
Component B, Operation D
Component C, Operation F
```

To demonstrate we can extend and change the subsystem without altering the it, let's change the output to the following:

```
Flexibility
Design Pattern
Component C, Operation F
```

Because the `ComponentB` class provides the first two lines, we must replace it with a new implementation of the `IComponentB` interface. Let's call this class `UpdatedComponentB`:

```
using TransparentFacadeSubSystem.Abstractions;
namespace Facade;
public class UpdatedComponentB : IComponentB
```

```
{
    public string OperationC() => "Flexibility";
    public string OperationD() => "Design Pattern";
}
```

The preceding code does exactly what we want. However, we have to tell the IoC container about it, like this:

```
builder.Services
    .AddOpaqueFacadeSubSystem()
    .AddTransparentFacadeSubSystem()
    .AddSingleton<IComponentB, UpdatedComponentB>()
;
```

If you run the program, you should see the desired result!

> Adding a dependency for a second time makes the container resolves that dependency, thus overriding the first one. However, both registrations remain in the services collection; for example, calling `GetServices<IComponentB>()` on `IServiceProvider` would return two dependencies. Do not confuse the `GetServices()` and `GetService()` methods (plural versus singular); one returns a collection while the other returns a single instance. That single instance is always the last that has been registered.

That's it! We updated the system without modifying it. This is what dependency injection can do for you when designing a program around it.

Alternative façade patterns

One alternative would be to create a *hybrid between a transparent façade and an opaque façade* by exposing the abstractions using the `public` visibility modifier (all of the interfaces) while keeping the implementations hidden under an `internal` visibility modifier. This hybrid design offers the right balance between **control and flexibility**.Another alternative would be to create *a façade outside of the subsystem*. In the previous examples, we created the façades inside the class libraries, but this is not mandatory; the façade is just a class that creates an accessible wall between your system and one or more subsystems. It should be located wherever you see fit. Creating external façades like this would be especially useful when you do not control the source code of the subsystem(s), such as if you only have access to the binaries. This could also be used to create project-specific façades over the same subsystem, giving you extra flexibility without cluttering your subsystems with multiple façades, shifting the maintenance cost from the subsystems to the client applications that use them.This one is more of a note than an alternative: you do not need to create an assembly per subsystem. I did it because it helped me explain different concepts in the examples, but you could create multiple subsystems in the same assembly. You could even create a single assembly that includes all your subsystems, façades, and the client code (all in a single project).

> Whether talking about subsystems or REST APIs, layering APIs is an excellent way to create low-level functionalities that are atomic but harder to use while providing a higher-level API to access them through the façade, leading to a better consumer experience.

Conclusion

The Façade pattern is handy for simplifying consumers' lives, allowing us to hide subsystems' implementation details behind a wall. There are multiple flavors to it; the two most prominent ones are:

- The **transparent façade**, which increases flexibility by exposing at least part of the subsystem(s)
- The **opaque façade**, which controls access by hiding most of the subsystem(s)

Now, let's see how the **transparent façade** pattern can help us follow the **SOLID** principles:

- **S**: A well-designed **transparent façade** serves this exact purpose by providing a cohesive set of functionalities to its consumers by hiding overly complex subsystems or internal implementation details.
- **O**: A well-designed **transparent façade** and its underlying subsystem's components can be extended without direct modification, as we saw in the *Flexibility in action* section.

- **L**: N/A
- **I**: By exposing a façade that uses different smaller objects implementing small interfaces, we could say that the segregation is done at both the façade and the component levels.
- **D**: The Façade pattern does not specify anything about interfaces, so it is up to the developers to enforce this principle by using other patterns, principles, and best practices.

Finally, let's see how the **opaque façade** pattern can help us follow the **SOLID** principles:

- **S**: A well-designed **opaque façade** serves this exact purpose by providing a cohesive set of functionalities to its clients by hiding overly complex subsystems or internal implementation details.
- **O**: By hiding the subsystem, the **opaque façade** limits our ability to extend it. However, we could implement a **hybrid façade** to help with that.
- **L**: N/A
- **I**: The **opaque façade** does not help nor diminish our ability to apply the ISP.
- **D**: The Façade pattern does not specify anything about interfaces, so it is up to the developers to enforce this principle by using other patterns, principles, and best practices.

Summary

In this chapter, we covered multiple fundamental GoF structural design patterns. They help us extend our systems from the outside without modifying the actual classes, leading to a higher degree of cohesion by composing our object graph dynamically.We started with the Decorator pattern, a powerful tool that allows us to dynamically add new functionality to an object without altering its original code. Decorators can also be chained, allowing even greater flexibility (decorating other decorators). We learned that this pattern adheres to the Open-Closed principle and promotes the separation of responsibilities into smaller, manageable pieces.We also used an open-source tool named Scrutor that simplifies the decorator pattern usage by extending the built-in ASP.NET Core dependency injection system. Then, we covered the Composite pattern, which allows us to create complex, non-linear, and self-managed data structures with minimal effort. That hierarchical data structure where groups and single components are indistinguishable makes the hierarchy's traversal and manipulation easier. We use this pattern to build graphs or trees with self-managing nodes.After that, we covered the Adapter pattern, which allows two incompatible interfaces to work together without modifying their code. This pattern is very helpful when we need to adapt the components of external systems that we have no control over, do not want to change, or can't change.Finally, we dug into the Façade pattern, similar to the Adapter pattern, but at the subsystem level. It allows us to create a wall in front of one or more subsystems, simplifying its usage. It could also be used to hide the implementation details of a subsystem from its consumers.The next chapter explores two GoF behavioral design patterns: the Template method and the Chain of Responsibility design pattern.

Questions

Here are a few revision questions:

1. What is the main advantage of the Decorator pattern?
2. Can we decorate a decorator with another decorator?
3. What is the primary goal of the Composite design pattern?
4. Can we use the Adapter pattern to migrate an old API to a new system in order to adapt its APIs before rewriting it?
5. What is the primary responsibility of the Adapter pattern?
6. What is the difference between the Adapter and the Façade patterns?
7. What is the main difference between an Opaque façade and a Transparent façade?

Further reading

- To learn more about Scrutor, please visit https://adpg.link/xvfS

Answers

1. The Decorator pattern allows us to dynamically add new functionality to an object at runtime without modifying its original code, promoting flexibility, testability, and manageability.
2. Yes, we can decorate decorators by depending only on interfaces because they are just another implementation of the interface, nothing more.
3. The Composite design pattern aims to simplify handling complex structures by treating individual and group elements indistinguishably.
4. Yes, we could use an adapter for this.
5. The Adapter pattern's primary responsibility is to adapt one interface to work with another interface that is incompatible to use directly.
6. The Adapter and Façade design patterns are almost the same but are applied to different scenarios. The Adapter pattern adapts an API to another API, while the Façade pattern exposes a unified or simplified API, hiding one or more complex subsystems.
7. An Opaque façade hides most of the subsystem (`internal` visibility), controlling access to it, while a Transparent façade exposes at least part of the subsystem (`public` visibility), increasing flexibility.

12 Behavioral Patterns

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter explores two new design patterns from the well-known **Gang of Four** (**GoF**). They are behavioral patterns, meaning they help simplify system behavior management.Often, we need to encapsulate some core algorithm while allowing other pieces of code to extend that implementation. That is where the **Template Method** pattern comes into play.Other times, we have a complex process with multiple algorithms that all apply to one or more situations, and we need to organize it in a testable and extensible fashion. This is where the **Chain of Responsibility** pattern can help. For example, the ASP.NET Core middleware pipeline is a Chain of Responsibility where all the middleware inspects and acts on the request.In this chapter, we cover the following topics:

- Implementing the Template Method pattern
- Implementing the Chain of Responsibility pattern
- Mixing the Template Method and Chain of Responsibility patterns

Implementing the Template Method pattern

The **Template Method** is a GoF behavioral pattern that uses inheritance to share code between the base class and its subclasses. It is a very powerful yet simple design pattern.

Goal

The goal of the Template Method pattern is to encapsulate the outline of an algorithm in a base class while leaving some parts of that algorithm open for modification by the subclasses, which adds flexibility at a low cost.

Design

First, we need to define a base class that contains the `TemplateMethod` method and then define one or more sub-operations that need to be implemented by its subclasses (`abstract`) or that can be overridden (`virtual`).Using UML, it looks like this:

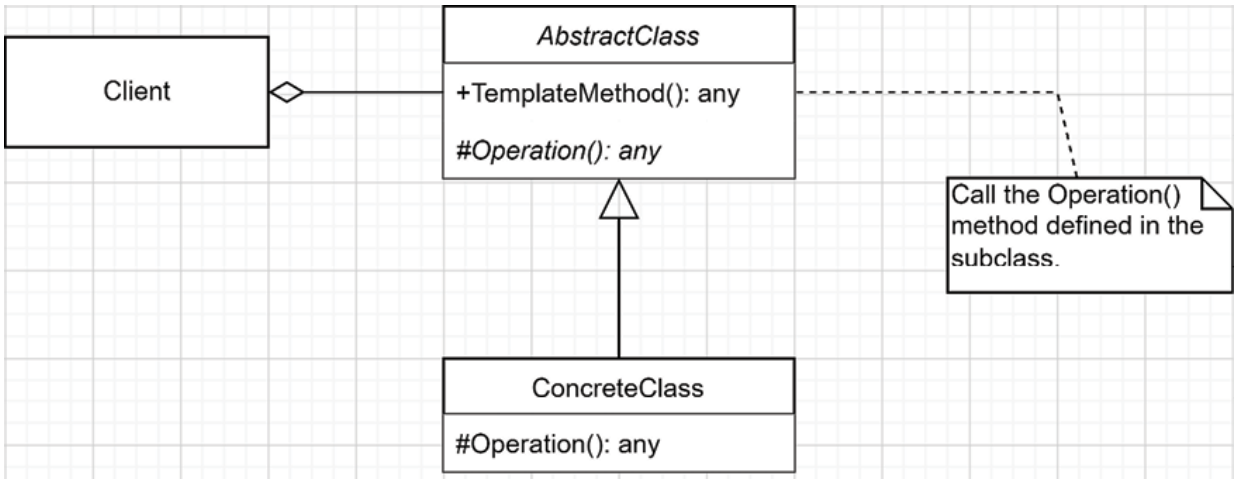


Figure 12.1: Class diagram representing the Template Method pattern

How does this work?

- `AbstractClass` implements the shared code: the algorithm in its `TemplateMethod` method.
- `ConcreteClass` implements its specific part of the algorithm in its inherited `Operation` method.
- `Client` calls `TemplateMethod()`, which calls the subclass implementation of one or more specific algorithm elements (the `Operation` method in this case).

We could also extract an interface from `AbstractClass` to allow even more flexibility, but that's beyond the scope of the Template Method pattern.

Let's now get into some code to see the Template Method pattern in action.

Project – Building a search machine

Let's start with a simple, classic example to demonstrate how the Template Method pattern works.**Context**: Depending on the collection, we want to use a different search algorithm. We want to use a *binary search* for sorted collections, but we want to use a *linear search* for unsorted collections.Let's start with the consumer, a REST endpoint in the `Program.cs` file that returns `plain/text` results:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services
    .AddSingleton<SearchMachine>(x
        => new LinearSearchMachine(1, 10, 5, 2, 123, 333, 4))
    .AddSingleton<SearchMachine>(x
        => new BinarySearchMachine(1, 2, 3, 4, 5, 6, 7, 8, 9, 10))
;
var app = builder.Build();
app.MapGet("/", (IEnumerable<SearchMachine> searchMachines) =>
{
    var sb = new StringBuilder();
    var elementsToFind = new int[] { 1, 10, 11 };
    foreach (var searchMachine in searchMachines)
    {
        var typeName = searchMachine.GetType().Name;
        var heading = $"Current search machine is {typeName}";
        sb.AppendLine("".PadRight(heading.Length, '='));
        sb.AppendLine(heading);
        foreach (var value in elementsToFind)
        {
            var index = searchMachine.IndexOf(value);
            var wasFound = index.HasValue;
            if (wasFound)
            {
```

```
                sb.AppendLine($"The element '{value}' was found at index {index!.Value}.");
            }
            else
            {
                sb.AppendLine($"The element '{value}' was not found.");
            }
        }
    }
    return sb.ToString();
});
app.Run();
```

As highlighted in the first few lines, we configure `LinearSearchMachine` and `BinarySearchMachine` as two `SearchMachine` implementations. We initialize each instance using a different sequence of numbers.Afterward, we inject all registered `SearchMachine` services into the endpoint (highlighted in the code block). That handler iterates all `SearchMachine` instances and tries to find all elements of the `elementsToFind` array before outputting the `text/plain` results.Next, let's explore the `SearchMachine` class:

```
namespace TemplateMethod;
public abstract class SearchMachine
{
    protected int[] Values { get; }
    protected SearchMachine(params int[] values)
    {
        Values = values ?? throw new ArgumentNullException(nameof(values));
    }
    public int? IndexOf(int value)
    {
        if (Values.Length == 0) { return null; }
        var result = Find(value);
        return result;
    }
    protected abstract int? Find(int value);
}
```

The `SearchMachine` class represents `AbstractClass`. It exposes the `IndexOf` template method, which uses the required hook represented by the `abstract Find` method (see highlighted code). The hook is required because each subclass must implement that method, thereby making that method a required extension point (or hook).Next, we explore our first implementation of `ConcreteClass`, the `LinearSearchMachine` class:

```
namespace TemplateMethod;
public class LinearSearchMachine : SearchMachine
{
    public LinearSearchMachine(params int[] values)
        : base(values) { }
    protected override int? Find(int value)
    {
        for (var i = 0; i < Values.Length; i++)
        {
            if (Values[i] == value) { return i; }
        }
        return null;
    }
}
```

The `LinearSearchMachine` class is a `ConcreteClass` representing the linear search implementation used by `SearchMachine`. It contributes a part of the `IndexOf` algorithm through its `Find` method.Finally, we move on to the `BinarySearchMachine` class:

```
namespace TemplateMethod;
public class BinarySearchMachine : SearchMachine
{
    public BinarySearchMachine(params int[] values)
        : base(values.OrderBy(v => v).ToArray()) { }
    protected override int? Find(int value)
    {
        var index = Array.BinarySearch(Values, value);
        return index < 0 ? null : index;
```

```
    }
}
```

The `BinarySearchMachine` class is a `ConcreteClass` representing the binary search implementation of `SearchMachine`. As you may have noticed, we skipped the binary search algorithm's implementation by delegating it to the built-in `Array.BinarySearch` method. Thanks to the .NET team!

> The binary search algorithm requires an ordered collection to work; hence the sorting done in the constructor when passing the values to the base class (`OrderBy`). That may not be the most performant way of ensuring the array is sorted (precondition/guard), but it is a very fast to write and readable way to write it. Moreover, in our case, performance is not an issue.
>
> > If you must optimize such an algorithm to work with a large data set, you can leverage parallelism (multithreading) to help out. In any case, run a proper benchmark to ensure you optimize the right thing and assess your real gains. Look at BenchmarkDotNet (https://adpg.link/C5E9) if you are looking at benchmarking .NET code.

Now that we have defined the actors and explored the code, let's see what is happening in our consumer (the `Client`):

1. The `Client` uses the registered `SearchMachine` instances and searches for values 1, 10, and 11.
2. Afterward, `Client` displays to the user whether the numbers were found or not.

The `Find` method returns `null` when it does not find a value and, by extension, the `IndexOf` method.By running the program, we get the following output:

```
==========================================
Current search machine is LinearSearchMachine
The element '1' was found at index 0.
The element '10' was found at index 1.
The element '11' was not found.
==========================================
Current search machine is BinarySearchMachine
The element '1' was found at index 0.
The element '10' was found at index 9.
The element '11' was not found.
```

The preceding output shows the two algorithms at play. Both `SearchMachine` implementations did not contain the value `11` . They both contained the values `1` and `10` placed at different positions. Here is a reminder of the values:

```
new LinearSearchMachine(1, 10, 5, 2, 123, 333, 4)
new BinarySearchMachine(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

The consumer was iterating the `SearchMachine` registered with the IoC container. The base class implements the `IndexOf` but delegates the search (`Find`) algorithm to the subclasses. The preceding output shows that each `SearchMachine` could execute the expected task by implementing only the `Find` piece of the algorithm.And voilà! We have covered the Template Method pattern, as easy as that. Of course, our algorithm was trivial, but the concept remains.To ensure the correctness of the implementation, I also created two tests, one per class. Here's a test for the `LinearSearchMachine` class:

```
namespace TemplateMethod;
public class LinearSearchMachineTest
{
    public class IndexOf
    {
        [Theory]
        [InlineData(1, 0)]
        [InlineData(2, 4)]
        [InlineData(3, 2)]
        [InlineData(7, null)]
        public void Should_return_the_expected_result(
            int value, int? expectedIndex)
        {
```

```
            // Arrange
            var sorter = new LinearSearchMachine(1, 5, 3, 4, 2);
            // Act
            var index = sorter.IndexOf(value);
            // Assert
            Assert.Equal(expectedIndex, index);
        }
    }
}
```

The preceding test ensures that the correct values are found or not by the `IndexOf` method of the `LinearSearchMachine` class.Next is a similar test for the `BinarySearchMachine` class:

```
namespace TemplateMethod;
public class BinarySearchMachineTest
{
    public class IndexOf
    {
        [Theory]
        [InlineData(1, 0)]
        [InlineData(8, 5)]
        [InlineData(3, 2)]
        [InlineData(7, null)]
        public void Should_return_the_expected_result(int value, int? expectedIndex)
        {
            // Arrange
            var sorter = new BinarySearchMachine(1, 2, 3, 4, 5, 8);
            // Act
            var index = sorter.IndexOf(value);
            // Assert
            Assert.Equal(expectedIndex, index);
        }
    }
}
```

The preceding test does a similar job of ensuring that the correct values are found or not by the `IndexOf` method of the `BinarySearchMachine` class.

> We can add `virtual` methods in the base class to create optional hooks. Those methods would become optional extension points that subclasses can implement or not. That would allow a more complex and more versatile scenario to be supported. We will not cover this here because it is not part of the pattern itself, even if very similar. There are many examples in the .NET **base class library** (**BCL**), like most methods of the `ComponentBase` class (in the `Microsoft.AspNetCore.Components` namespace). For example, when overriding the `OnInitialized` method in a Blazor component, we leverage an optional extension hook. The base method does nothing and is there only for extensibility purposes, allowing us to run code as part of the component's lifecycle. You can consult the `ComponentBase` class code in the official .NET repo on GitHub: https://adpg.link/1WYq.

This concludes our study of another simple yet powerful design pattern.

Conclusion

The Template Method is a powerful and easy-to-implement design pattern allowing subclasses to reuse the algorithm's skeleton while implementing (abstract) or overriding (virtual) subparts. It allows implementation-specific classes to extend the core algorithm. It can reduce the duplication of logic and improve maintainability while not cutting out any flexibility in the process. There are many examples in the .NET BCL, and we leverage this pattern at the end of the chapter based on a real-world scenario.Now, let's see how the Template Method pattern can help us follow the **SOLID** principles:

- **S**: The Template Method pushes algorithm-specific portions of the code to subclasses while keeping the core algorithm in the base class. Doing that helps follow the **single responsibility principle** (**SRP**) by distributing responsibilities.

- **O**: By opening extension hooks, it opens the template for extensions (allowing subclasses to extend it) and closes it from modifications (no need to modify the base class since the subclasses can extend it).
- **L**: As long as the subclasses are the implementations and do not alter the base class per se, following the **Liskov substitution principle** (**LSP**) should not be a problem. However, this principle is tricky, so it is possible to break it; by throwing a new type of exception or altering the state of a more complex base class in a way that changes its behavior, for example.
- **I**: As long as the base class implements the smallest cohesive surface possible, using the Template Method pattern should not negatively impact the program. On top of this, having a smaller interface surface in classes diminishes the chances of breaking the LSP.
- **D**: The Template Method pattern is based on an abstraction, so as long as consumers depend on that abstraction, it should help to get in line with the **dependency inversion principle** (**DIP**).

Next, we move to the Chain of Responsibility design pattern before mixing the Template Method and the Chain of Responsibility pattern to improve our code.

Implementing the Chain of Responsibility pattern

The **Chain of Responsibility** is a GoF behavioral pattern that chains classes to handle complex scenarios efficiently, with limited effort. Once again, the goal is to take a complex problem and break it into multiple smaller units.

Goal

The Chain of Responsibility pattern aims to chain multiple handlers that each solve a limited number of problems. If a handler cannot solve the specific problem, it passes the resolution to the chain's next handler.

> We often create a default handler that executes logic at the end of the chain as the terminal handler. Such a handler can throw an exception (for example, `OperationNotHandledException`) to cascade the issue up the call stack to a consumer who knows how to handle and react to it. Another strategy is creating a terminal handler that does the opposite and ensures nothing happens.

Design

The most basic Chain of Responsibility starts by defining an interface that handles a request (`IHandler`). Then we add classes that handle one or more scenarios (`Handler1` and `Handler2`):

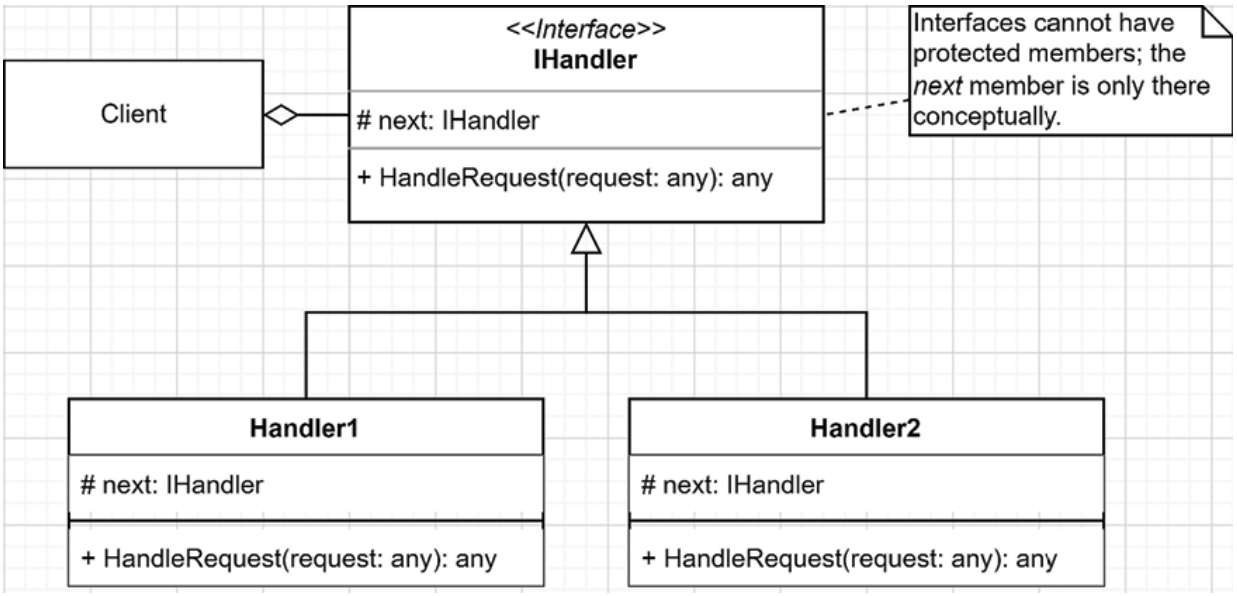


Figure 12.2: Class diagram representing the Chain of Responsibility pattern

A difference between the Chain of Responsibility pattern and many other patterns is that no central dispatcher knows the handlers; all handlers are independent. The consumer receives a handler and tells it to handle the request. Each handler determines if it can process the request. If it can, it processes it. In both cases, it also evaluates if it should forward the request to the next handler in the chain. The handler can execute these two tasks in any order, like executing some logic, sending the request down the chain, then executing more logic when the request comes back (like a pipeline).This pattern allows us to divide complex logic into multiple pieces that handle a single responsibility, improving testability, reusability, and extensibility in the process. Since no orchestrator exists, each chain element is independent, leading to a cohesive and loosely coupled design.

> When creating the chain of responsibility, you can order the handlers so that the most requested handlers are closer to the beginning of the chain and the least requested handlers are closer to the end. This helps limit the number of “chain links” that are visited for each request before reaching the right handler.

Enough theory; let’s look at some code.

Project – Message interpreter

Context: We need to create the receiving end of a messaging application where each message is unique, making it impossible to create a single algorithm to handle them all.After analyzing the problem, we decided to build a chain of responsibility where each handler can manage a single message. The pattern seems more than perfect!

> This project is based on something that I built years ago. IoT devices were sending bytes (messages) due to limited bandwidth. Then, we had to associate those bytes with real values in a web application. Each message had a fixed header size but a variable body size. The headers were handled in a base handler (template method), and each handler in the chain managed a different message type. For the current example, we keep it simpler than parsing bytes, but the concept is the same.

For our demo application, the messages are as simple as this:

```
namespace ChainOfResponsibility;
public record class Message(string Name, string? Payload);
```

The `Name` property is used as a discriminator to differentiate messages, and each handler is responsible for doing something with the `Payload` property.

We won't do anything with the payload as it is irrelevant to the pattern, but conceptually, that is the logic that should happen.

The handlers are very simple, here's the interface:

```
namespace ChainOfResponsibility;
public interface IMessageHandler
{
    void Handle(Message message);
}
```

The only thing a handler can do is handle a message. Our initial application can handle the following messages:

- The `AlarmTriggeredHandler` class handles `AlarmTriggered` messages.
- The `AlarmPausedHandler` class handles `AlarmPaused` messages.
- The `AlarmStoppedHandler` class handles `AlarmStopped` messages.

The real-world logic is that a machine can send an alarm to a REST API indicating it has reached a certain threshold. Then the REST API can push that information to a UI, send an email, a text message, or whatnot.

An alerted human can then pause the alarm while investigating the issue so other people know the alarm is getting handled.

Finally, a human can go to the physical device and stop the alarm because the person has resolved the issue.

We could extrapolate on many more sub-scenarios, but this is the gist.

The three handlers are very similar and share quite a bit of logic, but we fix that later. In the meantime, we have the following handlers:

```
namespace ChainOfResponsibility;
public class AlarmTriggeredHandler : IMessageHandler
{
    private readonly IMessageHandler? _next;
    public AlarmTriggeredHandler(IMessageHandler? next = null)
    {
        _next = next;
    }
    public void Handle(Message message)
    {
        if (message.Name == "AlarmTriggered")
        {
            // Do something clever with the Payload
        }
        else
        {
            _next?.Handle(message);
        }
    }
}
public class AlarmPausedHandler : IMessageHandler
{
    private readonly IMessageHandler? _next;
    public AlarmPausedHandler(IMessageHandler? next = null)
    {
        _next = next;
    }
    public void Handle(Message message)
    {
        if (message.Name == "AlarmPaused")
        {
            // Do something clever with the Payload
        }
        else
        {
```

```
            _next?.Handle(message);
        }
    }
}
public class AlarmStoppedHandler : IMessageHandler
{
    private readonly IMessageHandler? _next;
    public AlarmStoppedHandler(IMessageHandler? next = null)
    {
        _next = next;
    }
    public void Handle(Message message)
    {
        if (message.Name == "AlarmStopped")
        {
            // Do something clever with the Payload
        }
        else
        {
            _next?.Handle(message);
        }
    }
}
```

Each handler does two things:

- It receives an optional “next handler” from its constructor (highlighted in the code). This creates a chain similar to a singly linked list.
- It handles only the request it knows about, delegating the others to the next handler in the chain.

Let’s use `Program.cs` as the consumer of the Chain of Responsibility (the Client) and use a POST requests to interface with our REST API and build the message.Here is the first part of our REST API:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<IMessageHandler>(
    new AlarmTriggeredHandler(
        new AlarmPausedHandler(
            new AlarmStoppedHandler())));
```

In the preceding code, we manually create the Chain of Responsibility and register it as a singleton bound to the `IMessageHandler` interface. In that registration code, each handler is manually injected in the previous constructor (created with the `new` keyword).The next code represents the second half of the `Program.cs` file:

```
var app = builder.Build();
app.MapPost(
    "/handle",
    (Message message, IMessageHandler messageHandler) =>
    {
        messageHandler.Handle(message);
        return $"Message '{message.Name}' handled successfully.";
    });
app.Run();
```

The consuming endpoint is reachable through the `/handle` URL and expects a `Message` object in its body. It then uses the injected implementation of the `IMessageHandler` interface and passes it the message. If we run any of the HTTP requests in the `ChainOfResponsibility.http` file, we get a successful result similar to this:

```
Message 'AlarmTriggered' handled successfully.
```

The problem is that even if we send an invalid message, there is no way to know from the consumer, so it is always valid even when no handler picks up the message.To handle this scenario, let’s add a fourth handler (terminal handler) that notifies the consumers of invalid requests:

```
public class DefaultHandler : IMessageHandler
{
    public void Handle(Message message)
```

```
    {
        throw new NotSupportedException(
            $"Messages named '{message.Name}' are not supported.");
    }
}
```

This new terminal handler throws an exception that notifies the consumers about the error.

> We can create custom exceptions to make differentiating between system and application errors easier. In this case, throwing a system exception is good enough. In a real-world application, I recommend creating a custom exception that represents the end of the chain and contains the relevant information for the consumers to react to it according to your use case.

Next, let's register it in our chain (highlighted):

```
builder.Services.AddSingleton<IMessageHandler>(
    new AlarmTriggeredHandler(
        new AlarmPausedHandler(
            new AlarmStoppedHandler(
                new DefaultHandler()
            ))));
```

If we send a POST request with the name `SomeUnhandledMessageName`, the endpoint now yields the following exception:

```
System.NotSupportedException: Messages named 'SomeUnhandledMessageName' are not supported.
   at ChainOfResponsibility.DefaultHandler.Handle(Message message) in C12\src\ChainOfResponsibil
   at ChainOfResponsibility.AlarmStoppedHandler.Handle(Message message) in C12\src\ChainOfRespons
   at ChainOfResponsibility.AlarmPausedHandler.Handle(Message message) in C12\src\ChainOfResponsi
   at ChainOfResponsibility.AlarmTriggeredHandler.Handle(Message message) in C12\src\ChainOfRespo
   at Program.<>c.<<Main>$>b__0_0(Message message, IMessageHandler messageHandler) in C12\src\Cha
   at lambda_method1(Closure, Object, HttpContext, Object)
   at Microsoft.AspNetCore.Http.RequestDelegateFactory.<>c__DisplayClass100_2.<<HandleRequestBody
--- End of stack trace from previous location ---
   at Microsoft.AspNetCore.Routing.EndpointMiddleware.<Invoke>g__AwaitRequestTask|6_0(Endpoint en
   at Microsoft.AspNetCore.Diagnostics.DeveloperExceptionPageMiddlewareImpl.Invoke(HttpContext co
HEADERS
=======
Host: localhost:10001
Content-Type: application/json
traceparent: 00-5d737fdbb1018d5b7d060b74baf26111-2805f137fe1541af-00
Content-Length: 77
```

So far, so good, but the experience is not great, so let's add a try-catch block to handle this in the endpoint:

```
app.MapPost(
    "/handle",
    (Message message, IMessageHandler messageHandler) =>
    {
        try
        {
            messageHandler.Handle(message);
            return $"Message '{message.Name}' handled successfully.";
        }
        catch (NotSupportedException ex)
        {
            return ex.Message;
        }
    });
```

Now, when we send an invalid message, the API gently returns the following message to us:

```
Messages named 'SomeUnhandledMessageName' are not supported.
```

> Of course, when you expect machines to consume your APIs, you should produce a data structure that is easier to parse, like using JSON.

And voilà. We have built a simple Chain of Responsibility that handles messages.

Conclusion

The Chain of Responsibility pattern is another great GoF pattern. It divides a large problem into smaller, cohesive units, each doing one job: handling its specific request(s).Now, let's see how the Chain of Responsibility pattern can help us follow the **SOLID** principles:

- **S**: The Chain of Responsibility pattern aims to this exact principle: create a single unit of logic per class!
- **O**: The Chain of Responsibility pattern allows the addition, removal, and reordering of handlers without touching the code but by altering the chain's composition in the composition root.
- **L**: N/A
- **I**: The Chain of Responsibility pattern helps with the ISP if we create a small interface. The handler interface is not limited to a single method; it can expose multiple.
- **D**: By using the handler interface, no element of the chain, nor the consumers, depends on a specific handler; they only depend on the interface that represents the chain, which helps invert the dependency flow.

Next, let's use the Template Method and Chain of Responsibility patterns to encapsulate our handlers' duplicated logic.

Mixing the Template Method and Chain of Responsibility patterns

This section explores a combination of two powerful design patterns: the Template Method and the Chain of Responsibility. As we are about to explore, those two patterns fit together well. We use the Template Method pattern as the base structure, providing the handlers' blueprint. Meanwhile, the Chain of Responsibility pattern manages the handling sequence, ensuring each request is routed to the correct handler.When these two patterns work in tandem, they form a robust framework that facilitates easy management, maintains order, and increases the adaptability of our system.

Project – Improved message interpreter

Now that we know both the **Chain of Responsibility** and the **Template Method** patterns, it is time to *DRY* out our handlers by extracting the shared logic into an abstract base class using the Template Method pattern and providing extension points to the subclasses.OK, so what logic is duplicated?

- The `next` handler injection code is the same in all but the terminal handlers. Moreover, this is an important part of the pattern we should encapsulate in the base class.
- The logic testing whether the current handler can handle the message is also the same in all but the terminal handlers.

Let's create a new base class that implements the Template Method pattern and a large part of the logic of our chain of responsibility:

```
namespace ImprovedChainOfResponsibility;
public abstract class MessageHandlerBase : IMessageHandler
{
    private readonly IMessageHandler? _next;
    public MessageHandlerBase(IMessageHandler? next = null)
    {
        _next = next;
    }
    public void Handle(Message message)
    {
        if (CanHandle(message))
        {
            Process(message);
        }
        else if (HasNext())
        {
```

```
            _next.Handle(message);
        }
    }
    [MemberNotNullWhen(true, nameof(_next))]
    private bool HasNext()
    {
        return _next != null;
    }
    protected virtual bool CanHandle(Message message)
    {
        return message.Name == HandledMessageName;
    }
    protected abstract string HandledMessageName { get; }
    protected abstract void Process(Message message);
}
```

Based on those few changes, what is the template method, and what are the extension points (hooks)?The `MessageHandlerBase` class adds the `Handle` template method. Then, the `MessageHandlerBase` class exposes the following extension points:

- The `CanHandle` method tests whether `HandledMessageName` is equal to the value of the `message.Name` property. A subclass can override this method if it requires a different comparison logic. This method is an optional hook.
- All subclasses must implement the `HandledMessageName` property, which is the key driver of the `CanHandle` method. This property is a mandatory hook.
- All subclasses must implement the `Process` method, allowing them to run their logic against the message. This method is a mandatory hook.

To understand how these hooks play out, let’s take a look at the three simplified alarm handlers:

```
public class AlarmTriggeredHandler : MessageHandlerBase
{
    protected override string HandledMessageName => "AlarmTriggered";
    public AlarmTriggeredHandler(IMessageHandler? next = null)
        : base(next) { }
    protected override void Process(Message message)
    {
        // Do something clever with the Payload
    }
}
public class AlarmPausedHandler : MessageHandlerBase
{
    protected override string HandledMessageName => "AlarmPaused";
    public AlarmPausedHandler(IMessageHandler? next = null)
        : base(next) { }
    protected override void Process(Message message)
    {
        // Do something clever with the Payload
    }
}
public class AlarmStoppedHandler : MessageHandlerBase
{
    protected override string HandledMessageName => "AlarmStopped";
    public AlarmStoppedHandler(IMessageHandler? next = null)
        : base(next) { }
    protected override void Process(Message message)
    {
        // Do something clever with the Payload
    }
}
```

As we can see from the updated alarm handlers, they are now limited to a single responsibility: processing the messages they can handle. In contrast, `MessageHandlerBase` now handles the chain of responsibility’s plumbing. We left the `DefaultHandler` class unchanged since it is the end of the chain and does not support having a next handler, nor processing messages.Mixing those two patterns created a complex messaging system that divides responsibilities into handlers. There is one handler per message, and the chain logic is pushed into a base class.The

beauty of such a system is that we don't have to think about all the messages simultaneously; we can focus on just one message at a time. When dealing with a new type of message, we can focus on that precise message, implement its handler, and forget about the other *N* types. The consumers can also be super dumb, sending the request into the pipe without knowing about the Chain of Responsibility, and like magic, the right handler shall prevail!Nonetheless, have you noticed an issue with this design? Let's have a look next.

Project – A final, finer-grained design

In the last example, we used `HandledMessageName` and `CanHandle` to decide whether a handler could handle a request. There is one problem with that code: if a subclass decides to override `CanHandle`, and then decides that it no longer requires `HandledMessageName`, we would end up having a lingering, unused property in our system.

> There are worse situations, but we are talking component design here, so why not push that system a little further toward a better design?

One way to fix this is to create a finer-grained class hierarchy, as follows:

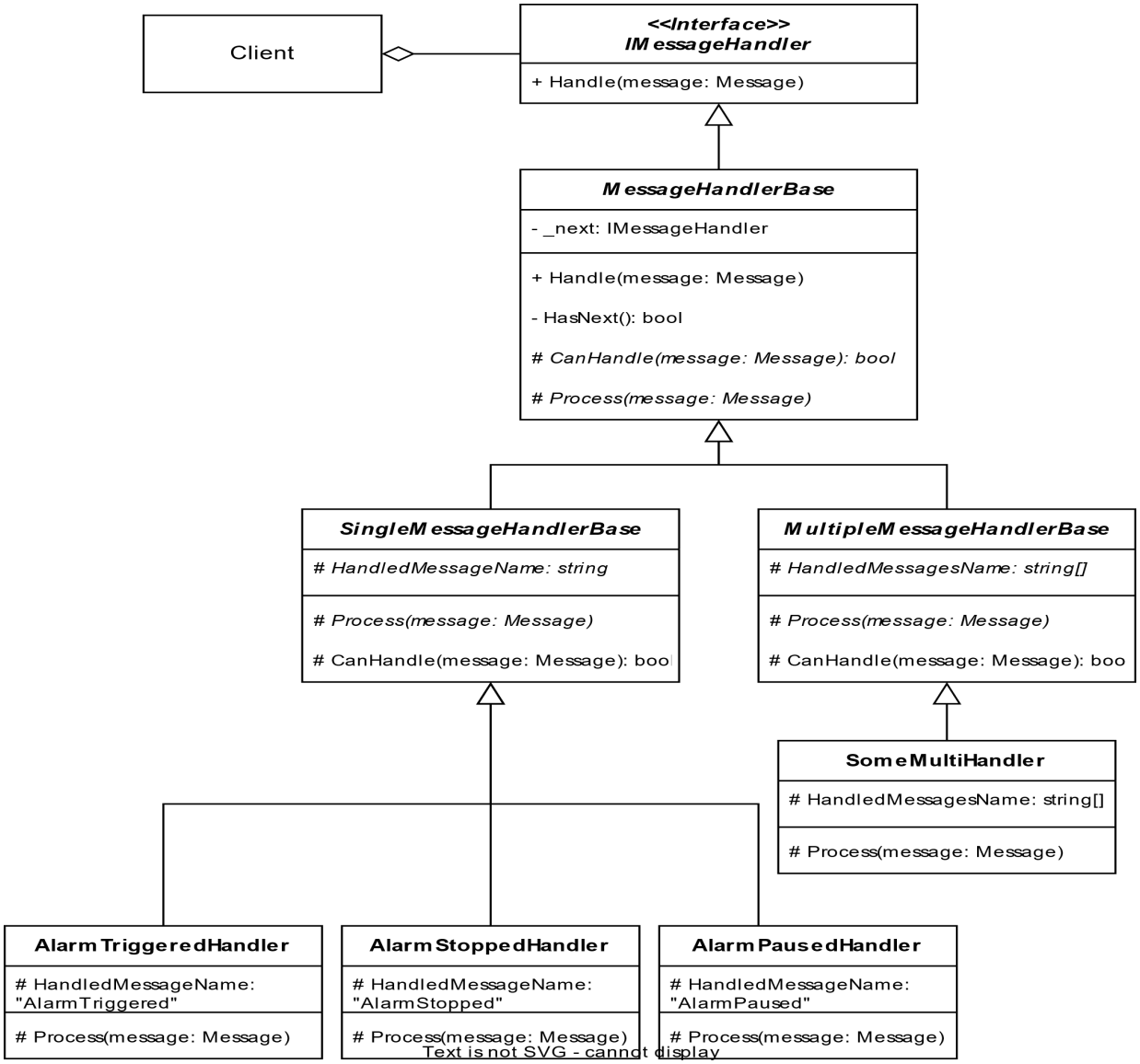


Figure 12.4: Class diagram representing the design of the finer-grained project that implements the Chain of Responsibility and Template Method patterns

The preceding diagram looks more complicated than it is. But let's look at our refactored code first, starting with the new `MessageHandlerBase` class:

```
namespace FinalChainOfResponsibility;
public interface IMessageHandler
{
    void Handle(Message message);
}
public abstract class MessageHandlerBase : IMessageHandler
{
    private readonly IMessageHandler? _next;
    public MessageHandlerBase(IMessageHandler? next = null)
    {
        _next = next;
    }
    public void Handle(Message message)
    {
        if (CanHandle(message))
        {
            Process(message);
        }
        else if (HasNext())
        {
            _next.Handle(message);
        }
    }
    [MemberNotNullWhen(true, nameof(_next))]
    private bool HasNext()
    {
        return _next != null;
    }
    protected abstract bool CanHandle(Message message);
    protected abstract void Process(Message message);
}
```

The `MessageHandlerBase` class manages the Chain of Responsibility by handling the next handler logic and by exposing two hooks (the Template Method pattern) for subclasses to extend:

- `bool CanHandle(Message message)`
- `void Process(Message message)`

This class is similar to the previous one, but the `CanHandle` method is now abstract, and we removed the `HandledMessageName` property leading to a better responsibility segregation and better hooks.Next, let's look at the `SingleMessageHandlerBase` class, which replaces the logic we removed from the `MessageHandlerBase` class:

```
public abstract class SingleMessageHandlerBase : MessageHandlerBase
{
    public SingleMessageHandlerBase(IMessageHandler? next = null)
        : base(next) { }
    protected override bool CanHandle(Message message)
    {
        return message.Name == HandledMessageName;
    }
    protected abstract string HandledMessageName { get; }
}
```

The `SingleMessageHandlerBase` class inherits from the `MessageHandlerBase` class and overrides the `CanHandle` method. It implements the logic related to it and adds the `HandledMessageName` property that subclasses must define for the `CanHandle` method to work (a required extension point).The `AlarmPausedHandler`, `AlarmStoppedHandler`, and `AlarmTriggeredHandler` classes now inherit from `SingleMessageHandlerBase` instead of `MessageHandlerBase`, but nothing else has changed. Here's the code as a reminder:

```
namespace FinalChainOfResponsibility;
public class AlarmPausedHandler : SingleMessageHandlerBase
{
    protected override string HandledMessageName => "AlarmPaused";
    public AlarmPausedHandler(IMessageHandler? next = null)
        : base(next) { }
    protected override void Process(Message message)
    {
        // Do something clever with the Payload
    }
}
public class AlarmStoppedHandler : SingleMessageHandlerBase
{
    protected override string HandledMessageName => "AlarmStopped";
    public AlarmStoppedHandler(IMessageHandler? next = null)
        : base(next) { }
    protected override void Process(Message message)
    {
        // Do something clever with the Payload
    }
}
public class AlarmTriggeredHandler : SingleMessageHandlerBase
{
    protected override string HandledMessageName => "AlarmTriggered";
    public AlarmTriggeredHandler(IMessageHandler? next = null)
        : base(next) { }
    protected override void Process(Message message)
    {
        // Do something clever with the Payload
    }
}
```

Those subclasses of `SingleMessageHandlerBase` implement the `HandledMessageName` property, which returns the message name they can handle, and they implement the handling logic by overriding the `Process` method as before.Next, we look at the `MultipleMessageHandlerBase` class, which enables its sub-types to handle more than one message type:

```
public abstract class MultipleMessageHandlerBase : MessageHandlerBase
{
    public MultipleMessageHandlerBase(IMessageHandler? next = null)
        : base(next) { }
    protected override bool CanHandle(Message message)
    {
        return HandledMessagesName.Contains(message.Name);
    }
    protected abstract string[] HandledMessagesName { get; }
}
```

The `MultipleMessageHandlerBase` class does the same as `SingleMessageHandlerBase` , but it uses a string array instead of a single string, supporting multiple handler names.The `DefaultHandler` class has not changed. For demonstration purposes, let’s add the `SomeMultiHandler` class that simulates a message handler that can handle `"Foo"` , `"Bar"` , and `"Baz"` messages:

```
namespace FinalChainOfResponsibility;
public class SomeMultiHandler : MultipleMessageHandlerBase
{
    public SomeMultiHandler(IMessageHandler? next = null)
        : base(next) { }
    protected override string[] HandledMessagesName
        => new[] { "Foo", "Bar", "Baz" };
    protected override void Process(Message message)
    {
        // Do something clever with the Payload
    }
}
```

This class hierarchy may sound complicated, but what we did was to allow extensibility without the need to keep any unnecessary code in the process, leaving each class with a single responsibility:

- The `MessageHandlerBase` class handles `_next`.
- The `SingleMessageHandlerBase` class handles the `CanHandle` method of handlers supporting a single message.
- The `MultipleMessageHandlerBase` class handles the `CanHandle` method of handlers supporting multiple messages.
- Other classes implement their version of `Process` method to handle one or more messages.

And voilà! This is another example demonstrating the strength of the Template Method and Chain of Responsibility patterns working together. That last example also emphasizes the importance of the SRP by allowing greater flexibility while keeping the code reliable and maintainable.Another strength of that design is the interface at the top. Anything that does not fit the class hierarchy can be implemented directly from the interface instead of trying to adapt logic from inappropriate structures. The `DefaultHandler` class is a good example of that.

> Tricking code into doing your bidding instead of properly designing that part of the system leads to half-baked solutions that become hard to maintain.

Conclusion

Mixing the Template Method and the Chain of Responsibility patterns leads to smaller classes with a single responsibility each.We removed the lingering property while keeping that logic out of the handlers. We even extended the logic to more use cases.

Summary

In this chapter, we covered two GoF behavioral patterns. These patterns can help us create flexible yet easy-to-maintain systems. As the name suggests, behavioral patterns aim at encapsulating application behaviors into cohesive pieces.First, we looked at the Template Method pattern, which allows us to encapsulate an algorithm's outline inside a base class, leaving some parts open for modification by subclasses. The subclasses then fill in the gaps and extend that algorithm at those predefined locations. These locations can be required (`abstract`) or optional (`virtual`).Then, you learned about the Chain of Responsibility pattern, which opens the possibility of chaining multiple small handlers into a chain of processing, inputting the message to be processed at the beginning of the chain (the interface), and waiting for one or more handlers to execute the logic related to that message against it.

> You don't have to stop the chain's execution at the first handler. The Chain of Responsibility can become a pipeline instead of associating one message to one handler, as we explored.

Lastly, leveraging the Template Method pattern to encapsulate the Chain of Responsibility's chaining logic led us to a simpler, robust, flexible, and testable implementation without any sacrifices. The two design patterns fits very well together.In the next chapter, we dig into the Operation Result design pattern to discover efficient ways of managing return values.

Questions

Let's take a look at a few practice questions:

1. What is the main goal of the Template Method pattern?
2. What is the main goal of the Chain of Responsibility pattern?
3. Is it true that we can only add one `abstract` method when implementing the Template Method design pattern?
4. Can we use the Strategy pattern in conjunction with the Template Method pattern?
5. Is it true that there is a limit of 32 handlers in a Chain of Responsibility?
6. In a Chain of Responsibility, can multiple handlers process the same message?
7. In what way can the Template Method help implement the Chain of Responsibility pattern?

Answers

1. The Template Method pattern encapsulates an algorithm's outline in a base class while leaving some parts of that algorithm open for modification by its subclasses.
2. The Chain of Responsibility pattern divides a larger problem into small pieces (handlers). Each piece is self-governed, while the chain's existence is abstracted from its consumers.
3. False; you can create as many `abstract` (required) or `virtual` (optional) extension points (hooks) as you need.
4. Yes, there is no reason not to.
5. No, there is no greater limit than with any other code.
6. Yes, you can have one handler per message or multiple handlers per message. It is up to you and your requirements.
7. It helps divide responsibilities between classes by encapsulating the shared logic into one or more base classes.

13 Understanding the Operation Result Design Pattern

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter explores the **Operation Result** pattern, starting simple and progressing to more complex cases. An operation result aims to communicate the success or failure of an operation to its caller. It also allows that operation to return both a value and one or more messages to the caller.Imagine any system where you want to display user-friendly error messages, achieve some small speed gain, or even handle failure easily and explicitly. The **Operation Result** design pattern can help you achieve these goals. One way to use it is to handle the result of a remote operation, such as after querying a remote web service.This pattern builds upon foundational object-oriented programming concepts. In this chapter, we iterate and design different possibilities incrementally. Of course, you should always base your final design on your needs, so learning multiple options will help you make the right choices.

> The Operation Result pattern is also known as the **Result Object Pattern**. I prefer Operation Result because the name specifies that it represents the result of an operation, while the Result Object has a broader meaning. Nonetheless, both are basically the same.

In this chapter, we cover the following topics:

- The Operation Result design pattern basics
- The Operation Result design pattern returning a value
- The Operation Result design pattern returning error messages
- The Operation Result design pattern returning messages with severity levels
- Using sub-classes and static factory methods for better isolation of successes and failures

The Operation Result pattern

The Operation Result design pattern can be very simple to more complex. In this section, we explore multiple ways to use this pattern. We start with its simplest form and build on that until we can return messages and values and add severity levels as the result of an operation.

Goal

The role of the **Operation Result** pattern is to give an operation (a method) the possibility to return a complex result (an object), which allows the consumer to:

- [Mandatory] Access the success indicator of the operation (that is, whether the operation succeeded or not).
- [Optional] Access the operation result if there is one (the method's return value).
- [Optional] Access the cause of the failure if the operation was unsuccessful (error messages).
- [Optional] Access other information that documents the operation’s result. This could be as simple as a list of messages or as complex as multiple properties.

This can go even further, such as returning the severity of a failure or adding any other relevant information for the specific use case. The success indicator could be binary (`true` or `false`), or there could be more than two states, such as success, partial success, and failure.

> Always focus on your needs first, then use your imagination and knowledge to find the best solution. Software engineering is not only about applying techniques that others tell you to. It's an art! The difference is that you are crafting software instead of painting or woodworking. And that most people won't see any of that art (code) or get it even if they do.

Design

It is easy to rely on throwing exceptions when an operation fails. However, the Operation Result pattern is an alternative way of communicating success or failure between components when you don't want to use exceptions. One such reason could be that the messages are not errors or that treating an erroneous result is part of the main flow, not part of a side `catch` flow.A method must return an object containing one or more elements presented in the *Goal* section to be used effectively. As a rule of thumb, a method returning an operation result should not throw exceptions. This way, consumers don't have to handle anything other than the operation result itself.

> You can throw exceptions for special cases, but at this point, it is a judgment call based on clear specifications or facing a real problem. For example, a critical event that happens, like the disk is full, would be a valid use case for an exception because it has nothing to do with the main flow, and the code must alert the rest of the program about the system failure.

Instead of walking you through all of the possible UML diagrams, let's jump into the code and explore multiple smaller examples after taking a look at the basic sequence diagram that describes the simplest form of this pattern, applicable to all examples:

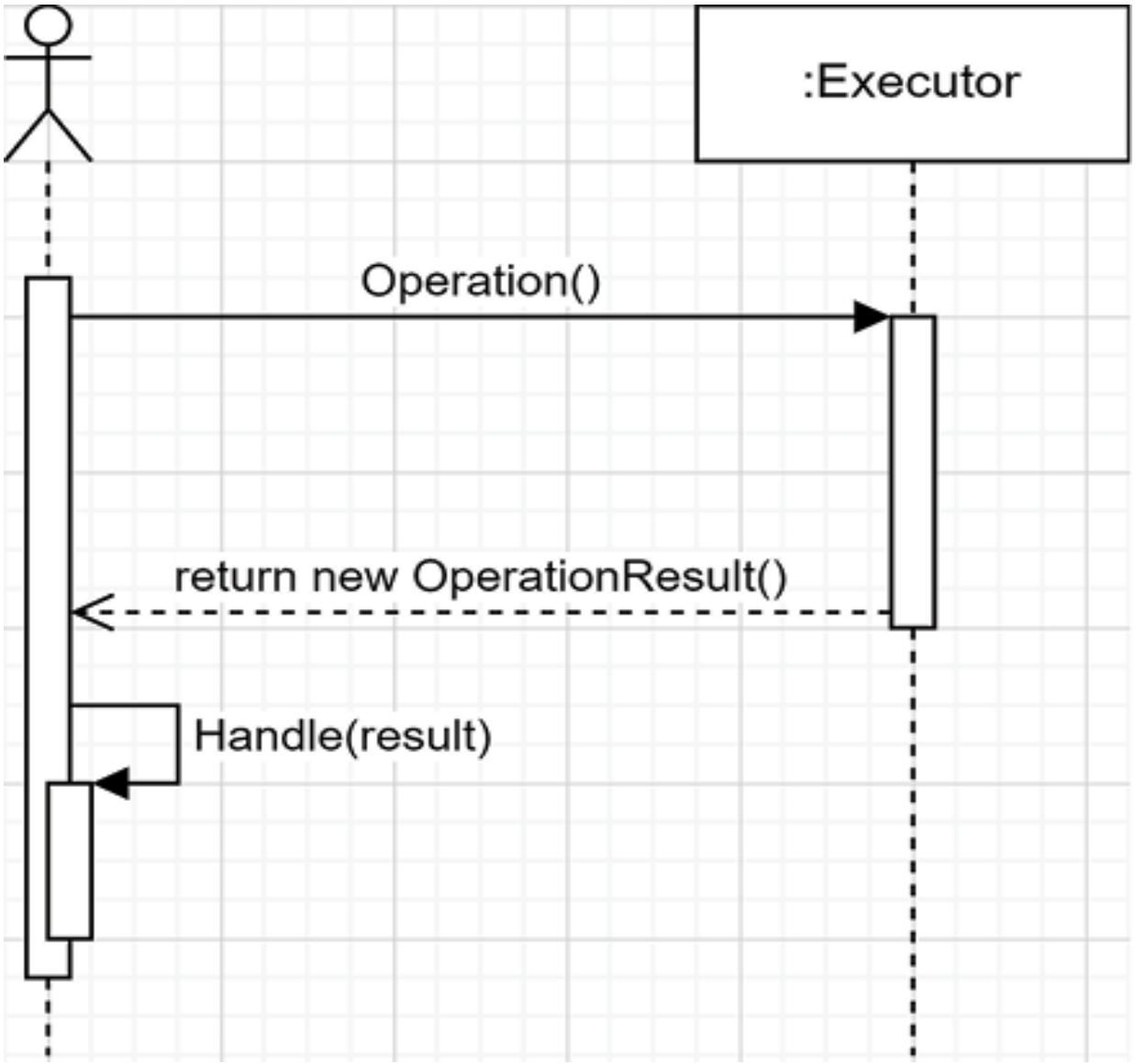


Figure 13.1: Sequence diagram of the Operation Result design pattern

The preceding diagram shows that an operation returns a result (an object), and then the caller handles that result. The following examples cover what we can include in that result object.

Project – Implementing different Operation Result patterns

In this project, a consumer (REST API) routes the HTTP requests to the correct handler. We are visiting each of those handlers one by one to create an incremental learning flow from simple to more complex operation results. This project shows you many ways to implement the Operation Result pattern to help you understand it, make it your own, and implement it as required in your projects.Let's start with the REST API.

The consumer

The consumer of all examples is the `Program.cs` file. The following code from `Program.cs` routes the HTTP requests toward a handler:

```
app.MapGet("/simplest-form", ...);
app.MapGet("/single-error", ...);
app.MapGet("/single-error-with-value", ...);
app.MapGet("/multiple-errors-with-value", ...);
app.MapGet("/multiple-errors-with-value-and-severity", ...);
app.MapGet("/static-factory-methods", ...);
```

Next, we cover each use case one by one.

The simplest form of the Operation Result pattern

The following diagram represents the simplest form of the Operation Result pattern:

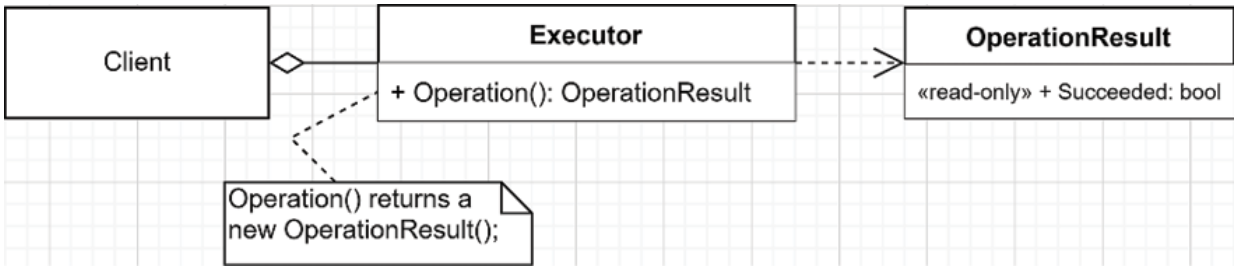


Figure 13.2: Class diagram of the Operation Result design pattern

We can translate that class diagram into the following blocks of code:

```
app.MapGet(
    "/simplest-form",
    (OperationResult.SimplestForm.Executor executor) =>
    {
        var result = executor.Operation();
        if (result.Succeeded)
        {
            // Handle the success
            return "Operation succeeded";
        }
        else
        {
            // Handle the failure
            return "Operation failed";
        }
    }
);
```

The preceding code handles the `/simplest-form` HTTP requests. The highlighted code consumes the following operation:

```
namespace OperationResult.SimplestForm;
public class Executor
{
    public OperationResult Operation()
    {
        // Randomize the success indicator
        // This should be real logic
        var randomNumber = Random.Shared.Next(100);
        var success = randomNumber % 2 == 0;
        // Return the operation result
        return new OperationResult(success);
    }
}
public record class OperationResult(bool Succeeded);
```

The `Executor` class contains the operation to execute represented by the `Operation` method. That method returns an instance of the `OperationResult` class. The implementation is based on a random number. Sometimes it

succeeds, and sometimes it fails. You would usually code real application logic in that method instead. Moreover, in an actual application, the method should have a proper name representing the operation, like `PayRegistrationFees` or `CreateConcert`.The `OperationResult` record class represents the result of the operation. In this case, a simple read-only Boolean value is stored in the `Succeeded` property.

> I chose a record class because there is no reason for the result to change. To know more about record classes, have a look at *Appendix A*.

In this form, the difference between the `Operation` method returning a `bool` and an instance of `OperationResult` is small, but it exists nonetheless. By returning an `OperationResult` object, you can extend the return value over time, adding properties and methods to it, which you cannot do with a `bool` without updating all consumers.Next, we add an error message to the result.

A single error message

Now that we know whether the operation succeeded or not, we want to know what went wrong. To do that, we add an `ErrorMessage` property to the `OperationResult` record class.With that in place, we no longer need to set whether the operation succeeded or not; we can compute that using the `ErrorMessage` property instead. The logic behind this improvement goes as follows:

- When there is no error message, the operation succeeded.
- When there is an error message, the operation failed.

The `OperationResult` record class implementing this logic looks like the following:

```
namespace OperationResult.SingleError
public record class OperationResult
{
    public bool Succeeded => string.IsNullOrWhiteSpace(ErrorMessage);
    public string? ErrorMessage { get; init; }
}
```

In the preceding code, we have the following:

- The `Succeeded` property checks for an error message.
- The `ErrorMessage` property contains an error message settable when instantiating the object.

The executor of that operation looks similar but uses the new constructor, setting an error message instead of directly setting the success indicator:

```
namespace OperationResult.SingleError
public class Executor
{
    public OperationResult Operation()
    {
        // Randomize the success indicator
        // This should be real logic
        var randomNumber = Random.Shared.Next(100);
        var success = randomNumber % 2 == 0;
        // Return the operation result
        return success
            ? new()
            : new() { ErrorMessage = $"Something went wrong with the number '{randomNumber}'." };
    }
}
```

The consuming code does the same as in the previous sample but writes the error message in the response output instead of a generic failure string:

```
app.MapGet(
    "/single-error",
    (OperationResult.SingleError.Executor executor) =>
```

```
    {
        var result = executor.Operation();
        if (result.Succeeded)
        {
            // Handle the success
            return "Operation succeeded";
        }
        else
        {
            // Handle the failure
            return result.ErrorMessage;
        }
    }
);
```

When looking at this example, we can begin to comprehend the Operation Result pattern's usefulness. It furthers us from the simple success indicator that looked like an overcomplicated Boolean.Next, we add the possibility of setting a value when the operation succeeds.

Adding a return value

Now that we have a reason for failure, we may want the operation to return a value. To achieve this, let's build over the previous example and add a `Value` property to the `OperationResult` class:

```
namespace OperationResult.SingleErrorWithValue;
public record class OperationResult
{
    public bool Succeeded => string.IsNullOrWhiteSpace(ErrorMessage);
    public string? ErrorMessage { get; init; }
    public int? Value { get; init; }
}
```

By adding a second `init`-only property, we can set the `Value` property when the operation succeeds and fails.

> In a real-world scenario, that `Value` property could be `null` in the case of an error, hence the nullable `int` property.

The operation is also very similar, but we are setting the `Value` property as well as using the object initializer in both cases (highlighted lines):

```
namespace OperationResult.SingleErrorWithValue;
public class Executor
{
    public OperationResult Operation()
    {
        // Randomize the success indicator
        // This should be real logic
        var randomNumber = Random.Shared.Next(100);
        var success = randomNumber % 2 == 0;
        // Return the operation result
        return success
            ? new() { Value = randomNumber }
            : new()
            {
                ErrorMessage = $"Something went wrong with the number '{randomNumber}'.",
                Value = randomNumber,
            };
    }
}
```

With that in place, the consumer can use the `Value` property. In our case, the program displays it when the operation succeeds:

```
app.MapGet(
    "/single-error-with-value",
    (OperationResult.SingleErrorWithValue.Executor executor) =>
```

```
    {
        var result = executor.Operation();
        if (result.Succeeded)
        {
            // Handle the success
            return $"Operation succeeded with a value of '{result.Value}'.";
        }
        else
        {
            // Handle the failure
            return result.ErrorMessage;
        }
    }
);
```

The preceding code displays the `ErrorMessage` property when the operation fails or uses the `Value` property when it succeeds. With this, the power of the Operation Result pattern continues to emerge.But we are not done yet, so let's jump into the next evolution.

Multiple error messages

Now we are at the point where we can transfer a `Value` and an `ErrorMessage` to the operation consumers; what about transferring multiple errors, such as validation errors? To achieve this, we can convert our `ErrorMessage` property from a `string` to an `IEnumerable<string>` or another type of collection that fits your needs better. Here I chose the `IReadOnlyCollection<string>` interface and the `ImmutableList<string>` class so we know that external actors can't mutate the results:

```
namespace OperationResult.MultipleErrorsWithValue;
public record class OperationResult
{
    public OperationResult()
    {
        Errors = ImmutableList<string>.Empty;
    }
    public OperationResult(params string[] errors)
    {
        Errors = errors.ToImmutableList();
    }
    public bool Succeeded => !HasErrors();
    public int? Value { get; init; }
    public IReadOnlyCollection<string> Errors { get; init; }
    public bool HasErrors()
    {
        return Errors?.Count > 0;
    }
}
```

Let's look at the new pieces in the preceding code before continuing:

- The errors are now stored in `ImmutableList<string>` object and returned as an `IReadOnlyCollection<string>`.
- The `Succeeded` property accounts for a collection instead of a single message and follows the same logic.
- The `HasErrors` method improves readability.
- The default constructor represents the successful state.
- The constructor that takes error messages as parameters represents a failed state and populates the `Errors` property.

Now that the operation result is updated, the operation itself can stay the same. The consumer stays almost the same as well (see the highlight in the code below), but we need to tell ASP.NET how to serialize the result:

```
app.MapGet(
    "/multiple-errors-with-value",
    object (OperationResult.MultipleErrorsWithValue.Executor executor)
    => {
        var result = executor.Operation();
```

```
        if (result.Succeeded)
        {
            // Handle the success
            return $"Operation succeeded with a value of '{result.Value}'.";
        }
        else
        {
            // Handle the failure
            return result.Errors;
        }
    }
);
```

We must specify the method returns an object (the highlighted code) so ASP.NET understands that the return value of our delegate can be anything. Without this, the return type could not be inferred, and the code would not compile. That makes sense since the function is returning a `string` in one path and an `IReadOnlyCollection<string>` in another.During the executing, ASP.NET serializes the `IReadOnlyCollection<string> Errors` property to JSON before outputting it to the client to help visualize the collection.

> Returning a `plain/text` string when the operation succeeds and an `application/json` array when it fails is not a good practice. I suggest avoiding this in real applications. Either return JSON or plain text. Do not mix content types in a single endpoint unless necessary per specifications. Mixing content types only creates avoidable complexity and confusion. Moreover, it is way easier for the consumers of the API to always expect the same content type.
>
> > When designing system contracts, consistency and uniformity are usually better than incoherency, ambiguity, and variance.

Our Operation Result pattern implementation is getting better and better but still lacks a few features. One of those features is the possibility to propagate messages that are not errors, such as information messages and warnings, which we implement next.

Adding message severity

Now that our operation result structure is materializing, let's update our last iteration to support message severity.First, we need a severity indicator. An `enum` is a good candidate for this kind of work, but it could also be something else. In our case, we leverage an `enum` that we name `OperationResultSeverity`.Then we need a message class to encapsulate both the message and the severity level; let's name that class `OperationResultMessage`. The new code looks like this:

```
namespace OperationResult.WithSeverity;
public record class OperationResultMessage
{
    public OperationResultMessage(string message, OperationResultSeverity severity)
    {
        Message = message ?? throw new ArgumentNullException(nameof(message));
        Severity = severity;
    }
    public string Message { get; }
    public OperationResultSeverity Severity { get; }
}
public enum OperationResultSeverity
{
    Information = 0,
    Warning = 1,
    Error = 2
}
```

As you can see, we have a simple data structure to replace our `string` messages.To ensure the enum gets serialized as string and make the output easier to read and consume, we must register the following converter:

```
builder.Services
    .Configure<JsonOptions>(o
        => o.SerializerOptions.Converters.Add(
            new JsonStringEnumConverter()))
;
```

Then we need to update the `OperationResult` class to use that new `OperationResultMessage` class instead. We then need to ensure that the operation result indicates a success only when there is no `OperationResultSeverity.Error`, allowing it to transmit the `OperationResultSeverity.Information` and `OperationResultSeverity.Warnings` messages:

```
namespace OperationResult.WithSeverity;
public record class OperationResult
{
    public OperationResult()
    {
        Messages = ImmutableList<OperationResultMessage>.Empty;
    }
    public OperationResult(params OperationResultMessage[] messages)
    {
        Messages = messages.ToImmutableList();
    }
    public bool Succeeded => !HasErrors();
    public int? Value { get; init; }
    public ImmutableList<OperationResultMessage> Messages { get; init; }
    public bool HasErrors()
    {
        return FindErrors().Any();
    }
    private IEnumerable<OperationResultMessage> FindErrors()
        => Messages.Where(x => x.Severity == OperationResultSeverity.Error);
}
```

The highlighted lines represent the updated logic that sets the success state of the operation. The operation is successful only when no error exists in the `Messages` list. The `FindErrors` method returns messages with an `Error` severity, while the `HasErrors` method bases its decision on that method's output.

The `HasErrors` method logic can be anything. In this case, this works.

With that in place, the `Executor` class is also revamped. Let's have a look at those changes:

```
namespace OperationResult.WithSeverity;
public class Executor
{
    public OperationResult Operation()
    {
        // Randomize the success indicator
        // This should be real logic
        var randomNumber = Random.Shared.Next(100);
        var success = randomNumber % 2 == 0;
        // Some information message
        var information = new OperationResultMessage(
            "This should be very informative!",
            OperationResultSeverity.Information
        );
        // Return the operation result
        if (success)
        {
            var warning = new OperationResultMessage(
                "Something went wrong, but we will try again later automatically until it works!'
                OperationResultSeverity.Warning
            );
            return new OperationResult(information, warning) { Value = randomNumber };
        }
        else
        {
            var error = new OperationResultMessage(
                $"Something went wrong with the number '{randomNumber}'.",
```

```
                OperationResultSeverity.Error
            );
            return new OperationResult(information, error) { Value = randomNumber };
        }
    }
}
```

In the preceding code, we removed the tertiary operator. The `Operation` method also uses all severity levels.

> You should always aim to write code that is easy to read. It is OK to use language features, but nesting statements over statements on a single line has limits and can quickly become a mess.

In that last code block, both successes and failures return two messages:

- When the operation succeeds, the method returns an information and a warning message.
- When the operation fails, the method returns an information and an error message.

From the consumer standpoint, we have a placeholder if-else block and return the operation result directly. Of course, we could handle this differently in a real application that needs to know about those messages, but in this case, all we want to see are those results, so this does it:

```
app.MapGet("/multiple-errors-with-value-and-severity", (OperationResult.WithSeverity.Executor exe
{
    var result = executor.Operation();
    if (result.Succeeded)
    {
        // Handle the success
    }
    else
    {
        // Handle the failure
    }
    return result;
});
```

As you can see, it is still as easy to use, but now with more flexibility added to it. We can do something with the different types of messages, such as displaying them to the user, retrying the operation, and more.For now, when running the application and calling this endpoint, successful calls return a JSON string that looks like the following:

```
{
    "succeeded": true,
    "value": 56,
    "messages": [
        {
            "message": "This should be very informative!",
            "severity": "Information"
        },
        {
            "message": "Something went wrong, but we will try again later automatically until it
            "severity": "Warning"
        }
    ]
}
```

Failures return a JSON string that looks like this:

```
{
    "succeeded": false,
    "value": 19,
    "messages": [
        {
            "message": "This should be very informative!",
            "severity": "Information"
        },
        {
            "message": "Something went wrong with the number '19'.",
```

```
            "severity": "Error"
        }
    ]
}
```

Another idea to improve this design would be adding a `Status` property that returns a complex success result based on each message's severity level. To do that, we could create another `enum`:

```
public enum OperationStatus { Success, Failure, PartialSuccess }
```

Then we could access that value through a new property named `Status`, on the `OperationResult` class. With this, a consumer could handle partial success without digging into the messages. I will leave you to play with this one on your own; for example, the `Status` property could replace the `Succeeded` property, or the `Succeeded` property could leverage the `Status` property similarly to what we did with the errors. The most important part is to define what would be a success, a partial success, and a failure. Think of a database transaction, for example; one failure could lead to the rollback of the transaction, while in another case, one failure could be acceptable.Now that we've expanded our simple example into this, what happens if we want the `Value` to be optional? To do that, we could create multiple operation result classes holding more or less information (properties); let's try that next.

Sub-classes and factories

In this iteration, we keep all the properties but instantiate the `OperationResult` objects using static factories. Moreover, we hide certain properties in the sub-classes, so each result type only contains the data it needs. The `OperationResult` class itself only exposes the `Succeeded` property in this scenario.A **static factory method** is nothing more than a static method that creates objects. It is handy and easy to use but less flexible.

> I cannot stress this enough: be careful when designing something `static`, or it could haunt you later; `static` members are not extensible and can make their consumers harder to test.

The `OperationResultMessage` class and the `OperationResultSeverity enum` remain unchanged. In the following code block, we do not consider the severity when computing the operation's success or failure state. Instead, we create an abstract `OperationResult` class with two sub-classes:

- The `SuccessfulOperationResult` class represents successful operations.
- The `FailedOperationResult` class represents failed operations.

Then the next step is to force the use of the specifically designed classes by creating two static factory methods:

- The static `Success` method returns a `SuccessfulOperationResult` object.
- The static `Failure` returns a `FailedOperationResult` object.

This technique moves the responsibility of deciding whether the operation is a success from the `OperationResult` class to the `Operation` method that explicitly creates the expected result.The following code block shows the new `OperationResult` implementation (the static factories are highlighted):

```
namespace OperationResult.StaticFactoryMethod;
public abstract record class OperationResult
{
    private OperationResult() { }
    public abstract bool Succeeded { get; }
    public static OperationResult Success(int? value = null)
    {
        return new SuccessfulOperationResult { Value = value };
    }
    public static OperationResult Failure(params OperationResultMessage[] errors)
    {
        return new FailedOperationResult(errors);
    }
    private record class SuccessfulOperationResult : OperationResult
    {
        public override bool Succeeded { get; } = true;
        public virtual int? Value { get; init; }
```

```
    }
    private record class FailedOperationResult : OperationResult
    {
        public FailedOperationResult(params OperationResultMessage[] errors)
        {
            Messages = errors.ToImmutableList();
        }
        public override bool Succeeded { get; } = false;
        public ImmutableList<OperationResultMessage> Messages { get; }
    }
}
```

After analyzing the code, there are a few closely related particularities:

- The `OperationResult` class has a private constructor.
- Both the `SuccessfulOperationResult` and `FailedOperationResult` classes are nested inside the `OperationResult` class, inherit from it, and are `private`.

Nested classes are the only way to inherit from the `OperationResult` class because, like other members of the class, nested classes have access to their private members, including the constructor. Otherwise, it is impossible to inherit from `OperationResult`. Moreover, as private classes, they can only be accessed internally from the `OperationResult` class for the same reason and become inaccessible from the outside.

> Since the beginning of the book, I have repeated **flexibility** many times; but you don't always want flexibility. Even if most of the book is about improving flexibility, sometimes you want control over what you expose and what you allow consumers to do, whether to protect internal mechanisms (encapsulation) or for maintainability reasons.
>
> For example, allowing consumers to change the internal state of an object can lead to unexpected behaviors. Another example would be when managing a library; the larger the public API, the more chances of introducing a breaking change. Nonetheless, over-hiding elements can be a detrimental experience for the consumers; if you need something somewhere, the chances are that someone else will too (eventually).
>
> In this case, we could have used a protected constructor instead or implemented a fancier way of instancing success and failure instances. Nonetheless, I decided to use this opportunity to show you how to lock a class in place without the sealed modifier, making extending by inheritance from the outside impossible. We could have built mechanisms in our classes to allow controlled extensibility (like the Template Method pattern), but for this one, let's keep it locked in tight!

From here, the only missing pieces are the operation itself and the consumer of the operation. Let's look at the operation first:

```
namespace OperationResult.StaticFactoryMethod;
public class Executor
{
    public OperationResult Operation()
    {
        // Randomize the success indicator
        // This should be real logic
        var randomNumber = Random.Shared.Next(100);
        var success = randomNumber % 2 == 0;
        // Return the operation result
        if (success)
        {
            return OperationResult.Success(randomNumber);
        }
        else
        {
            var error = new OperationResultMessage(
                $"Something went wrong with the number '{randomNumber}'.",
                OperationResultSeverity.Error
            );
            return OperationResult.Failure(error);
```

```
        }
    }
}
```

The two highlighted lines in the preceding code block show the elegance of this new improvement. I find this code very easy to read, which was the objective. We now have two methods that clearly define our intentions when using them: `Success` or `Failure`.The consumer uses the same code that we saw before in other examples, so I'll omit it here. However, the output is different for a successful or a failed operation. Here is a successful output:

```
{
    "succeeded": true,
    "value": 80
}
```

Here is a failed output:

```
{
    "succeeded": false,
    "messages": [
        {
            "message": "Something went wrong with the number '37'.",
            "severity": "Error"
        }
    ]
}
```

As the two preceding JSON outputs show, each object's properties are different. The only shared property of the two is the `Succeeded` property. Beware that this type of class hierarchy is harder to consume directly since the interface (the `OperationResult` class) has a minimal API surface, which is good in theory, and each sub-class adds different properties, which are hidden from the consumers. For example, it would be hard to use the `Value` property of a successful operation directly in the endpoint handler code. Therefore, when hiding properties, as we did here, ensure those additional properties are optional. For example, we can use this technique when sending the result to another system over HTTP (like this project does) or publish the operation result as an event (see *Chapter 19*, *Introduction to Microservices Architecture*, where we introduce event-driven architecture). Nevertheless, learning to manipulate classes using polymorphism will be helpful the day you need it.Next, let's peek at some advantages and disadvantages of the Operation Result pattern.

Advantages and disadvantages

Here are a few advantages and disadvantages of the Operation Result design pattern.

Advantages

It is more explicit than throwing an `Exception` since the operation result type is specified explicitly as the method's return type. That makes it more evident than knowing what type of exceptions the operation and its dependencies can throw.Another advantage is the execution speed; returning an object is faster than throwing an exception. Not that much faster, but faster nonetheless.Using operation results is more flexible than exceptions and gives us design flexibility; for example, we can manage different message types like warnings and information.

Disadvantages

Using operation results is more complex than throwing exceptions because we must *manually propagate it up the call stack* (i.e., the result object is returned by the callee and handled by the caller). This is especially true if the operation result must go up multiple levels, suggesting this pattern may not be the most suitable.It is easy to expose members not used by all scenarios, creating a bigger API surface than needed, where some parts are used only in some cases. But, between this and spending countless hours designing the perfect system, sometimes exposing an `int? Value { get; }` property is the best option. Nonetheless, always try to reduce that surface to a minimum and use your imagination and design skills to overcome those challenges!

Summary

In this chapter, we visited multiple forms of the Operation Result pattern, from an augmented Boolean to a complex data structure containing messages, values, and success indicators. We also explored static factories and private constructors to control external access. Furthermore, after all that exploration, let's conclude that there are almost endless possibilities around the Operation Result pattern. Each specific use case should dictate how to make it happen. From here, I am confident you have enough information about the pattern to explore the many possibilities yourself, and I highly encourage you to.The Operation Result pattern is perfect for crafting strongly typed return values that self-manage multiple states (error and success) or support complex states (like partial success). It is also ideal for transporting messages that are not necessarily errors, like information messages. Even in its simplest form, we can leverage the Operation Result pattern as a base for extensibility since we can add members to the result class over time, which would be impossible for a primitive type (or any type we don't control).

> The `HttpResponseMessage` class returned by the methods of the `HttpClient` class is an excellent example of a concrete implementation of the Operation Result pattern. It contains a single message exposed through the `ReasonPhrase` property. It exposes a complex success state through the `StatusCode` property and a simple success indicator through its `IsSuccessStatusCode` property. It also contains more information about the request and response through other properties.

At this point, we would usually explore how the **Operation Result** pattern can help us follow the SOLID principles. However, it depends too much on the implementation, so here are a few key points instead:

- The `OperationResult` class encapsulates the result, extracting that responsibility from the other system's components (SRP).
- We violated the ISP with the `Value` property in multiple examples. This infringement has a minor impact that we fixed as an example of overcoming this challenge.
- We could compare an operation result to a DTO but returned by an operation (method) instead of a REST API endpoint. From there, we could add an abstraction or stick with returning a concrete class, but sometimes using concrete types makes the system easier to understand and maintain. Depending on the implementation, this may break different principles.

When the advantages surpass the minor impacts of those kinds of violations, it is acceptable to let them slide. Principles are ideals and are not applicable in every scenario—principles are not laws.

> Most design decisions are trade-offs between two imperfect solutions, so you must choose which downsides you prefer to live with to gain the upsides.

This chapter concludes *Section 3: Components Patterns* and leads to *Section 4: Application Patterns*, where we explore higher-level design patterns.

Questions

Let's take a look at a few practice questions:

1. Is returning an operation result when doing an asynchronous call, such as an HTTP request, a good idea?
2. What is the name of the pattern that we implemented using static methods?
3. Is it faster to return an operation result than throw an exception?
4. In what scenario might the Operation Result pattern come in handy?

Further reading

Here are some links to build on what we learned in this chapter:

- An article on my blog about exceptions (title: *A beginner guide to exceptions* | *The basics*): https://adpg.link/PpEm
- An article on my blog about Operation Result (title*: Operation result* | *Design Pattern*): https://adpg.link/4o2q

Answers

1. Yes, asynchronous operations like HTTP are great candidates for the Operation Result pattern. For example, in the BCL, the `HttpResponseMessage` instance returned by the `Send` method of the `HttpClient` class is an operation result.
2. We implemented two **static factory methods**.
3. Yes, returning an object is marginally faster than throwing an exception.
4. The Operation Result pattern comes in handy when we want to return the state of the operation along with its return value as part of the main consumption flow. It is very suitable to return multiple properties describing the result of the process and is extensible.

14 Layering and Clean Architecture

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



In this chapter, we explore the inherent concepts behind layering. Layering is a popular way of organizing computer systems by encapsulating major concerns into layers. Those concerns are related to a computer vocation, such as data access, instead of a business concern, such as inventory. Understanding the concepts behind layering is essential, as other concepts were born from layers and are very common.We start this chapter by exploring the initial ideas behind layering. Then, we explore alternatives that can help us solve different problems. We use anemic and rich models and expose their pros and cons. Finally, we quickly explore **Clean Architecture**, an evolution of layering, and a way to organize layers.This chapter lays out the evolution of layering, starting with basic, restrictive, and even flawed techniques, then we gradually move toward more modern patterns. This journey should help you understand the concepts and practices behind layering, giving you a stronger understanding than just learning one way of doing things. The key is to understand.In this chapter, we cover the following topics:

- Introducing layering
- Responsibilities of the common layers
- Abstract layers
- Sharing a model
- Clean Architecture
- Implementing layering in real life

Let’s get started!

Introducing layering

Now that we’ve explored a few design patterns and played with ASP.NET Core, it is time to jump into layering. In most computer systems, there are layers. Why? Because it is an efficient way to partition and organize units of logic together. We could conceptually represent layers as horizontal software segments, each encapsulating a concern.

Classic layering model

Let’s start by examining a classic three-layer application design:

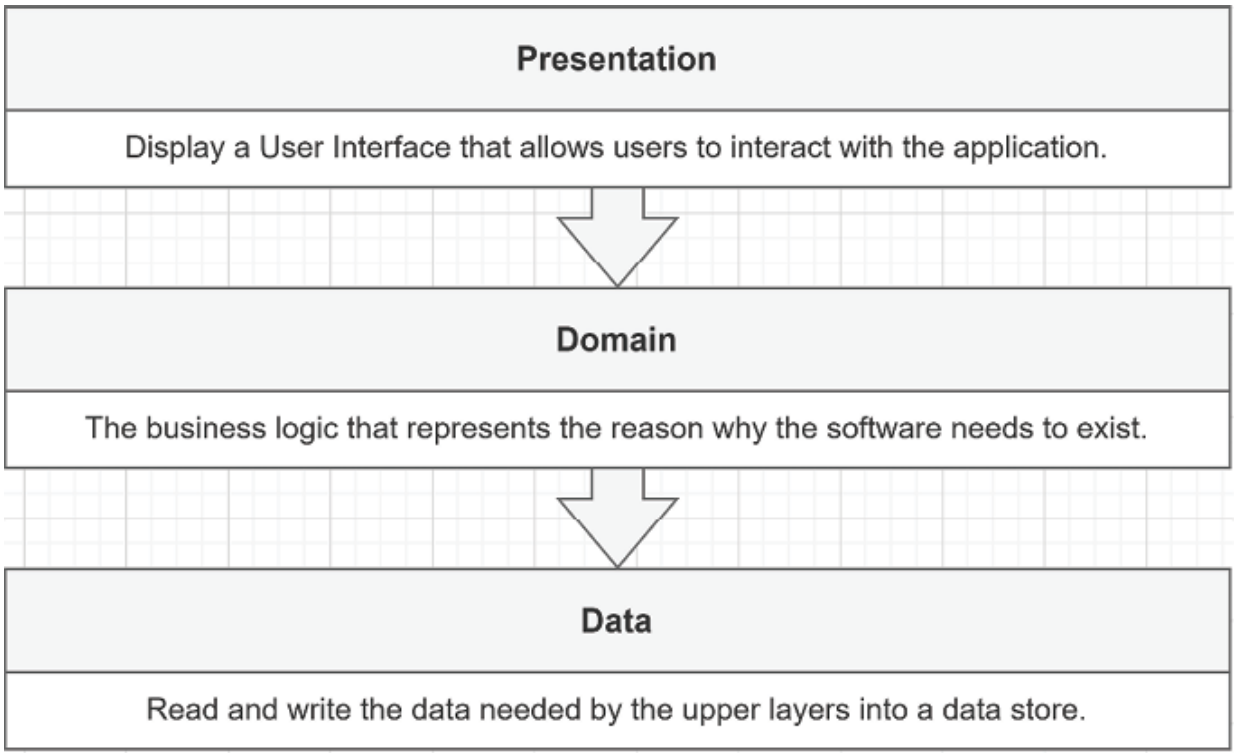


Figure 14.1: A classic three-layer application design

The **presentation layer** represents any user interface that a user can interact with to reach the **domain**. It could be an ASP.NET Core web application. Anything from WPF to WinForms to Android could be a valid non-web presentation layer alternative.The **domain layer** represents the core logic driven by the business rules; this solves the application's problem. The domain layer is also called the **business logic layer** (**BLL**).The **data layer** represents the bridge between the data and the application. The layer can store the data in a SQL Server database, a NoSQL database hosted in the cloud, a mix of many data sources, or anything else that fits the business needs. The data layer is also called the **data access layer** (**DAL**) and the **persistence layer**.Let's jump to an example. Given that a user has been authenticated and authorized, here is what happens when they want to create a book in a bookstore application built using those three layers:

1. The user requests the page by sending a `GET` request to the server.
2. The server handles that `GET` request (**presentation layer**) and then returns the page to the user.
3. The user fills out the form and sends a `POST` request to the server.
4. The server handles the `POST` request (**presentation layer**) and then sends it to the **domain layer** for processing.
5. The **domain layer** executes the logic required to create a book, then tells the **data layer** to persist that data.
6. After unrolling to the presentation layer, the server returns the appropriate response to the user, most likely a page containing a list of books and a message saying the operation was successful.

Following a classic layering architecture, a layer can only talk to the next layer in the stack—**presentation** talks to **domain**, which talks to **data**, and so on. The important part is that **each layer must be independent and isolated to limit tight coupling**.In this classic layering model, each layer should own its **model**. For example, the presentation layer should not send its **view models** to the **domain** layer; only **domain objects** should be used there. The opposite is also true: since the **domain** returns its own objects to the **presentation layer**, the **presentation layer** should not leak them to its consumers but organize the required information into **view models** or **DTO** instead.Here is a visual example:

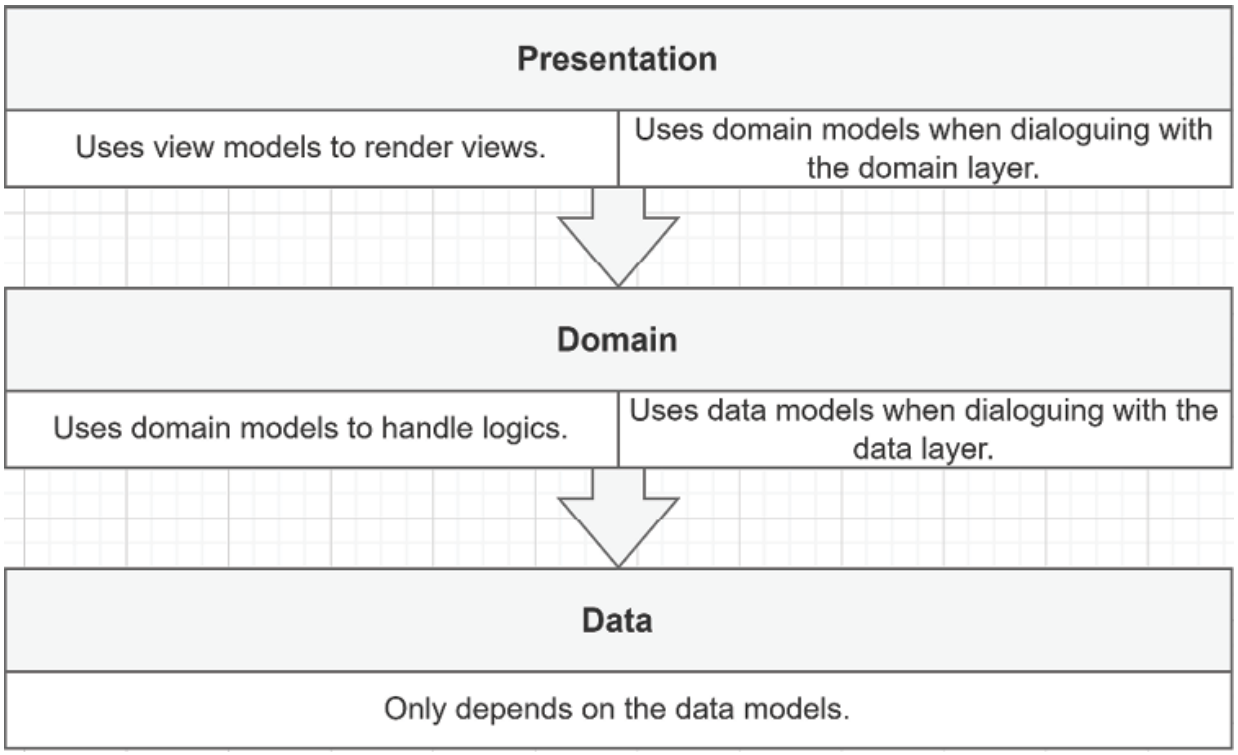


Figure 14.2: Diagram representing how the layers interact with one another

Even if three is probably the most popular number of layers, we can create as many as we need; we are not limited to three layers.Let's examine the advantages and disadvantages of classic layering, starting with the advantages:

- Knowing the purpose of a layer makes it easy to understand. For example, guessing that the data layer components read or write some data somewhere is easy.
- It creates a cohesive unit built around a single concern. For example, our **data layer** should not render any user interface but stick to accessing data.
- It allows us to decouple the layer from the rest of the system (the other layers). You can isolate and work within a layer with limited to no knowledge of the others. For example, suppose you are tasked with optimizing a query in a data access layer. In that case, you don't need to know about the user interface that eventually displays that data to a user. You only need to focus on that element, optimize it, test it in isolation, and then ship the layer or redeploy the application.
- Like any other isolated unit, it should be possible to reuse a layer. For example, we could reuse our **data access layer** in another application that needs to query the same database for a different purpose (a different **domain layer**).

TIP

Some layers are theoretically easier to reuse than others, and reusability could add more or less value, depending on the software you are building. I have never seen a layer being integrally reused in practice, and I've rarely heard or read about such a situation—each time rather ends in a not-so-reusable-after-all situation.

Based on my experience, I would strongly suggest not over-aiming at reusability when it is not a precise specification that adds value to your application. Limiting your overengineering endeavors could save you and your employers a lot of time and money. We must not forget that our job is to deliver value.

As a rule of thumb, do what needs to be done, not more, but do it well.

OK, now, let’s look at the drawbacks:

- By splitting your software horizontally into layers, each feature crosses all of the layers. This often leads to cascading changes between layers. For example, if we decide to add a field to our bookstore database, we would need to update the database, the code that accesses it (**data layer**), the business logic (**domain layer**), and the user interface (**presentation layer**). With volatile specs or low-budget projects, this can become painful!
- Implementing a full-stack feature is more challenging for newcomers because it crosses all layers.
- Using layering often leads to or is caused by a separation of responsibilities between the staff. For example, DBAs manage the data layer, backend devs manage the domain layer, and frontend devs manage the presentation layer, leading to coordination and knowledge-sharing issues.
- Since a layer directly depends on the layer under it, dependency injection is impossible without introducing an **abstraction layer** or referencing lower layers from the **presentation layer**. For example, if the **domain layer** depends on the **data layer**, changing the data layer would require rewriting all of that coupling from the **domain** to the **data**.
- Since each layer owns its entities, the more layers you add, the more copies there are of the entities, leading to minor performance loss and a higher maintenance cost. For example, the **presentation layer** copies a **DTO** to a **domain object**. Then, the **domain layer** copies it to a **data object**. Finally, the **data layer** translates it into SQL to persist it into a **database** (SQL Server, for example). The opposite is also true when reading from the database.

We explore ways to combat some of those drawbacks later.I strongly recommend that you don’t do what we just explored. It is an old, more basic way of doing layering. We are looking at multiple improvements to this layering system in this chapter, so keep reading before jumping to a conclusion. I decided to explore layering from the beginning in case you have to work with that kind of application. Furthermore, studying its chronological evolution, fixing some flaws, and adding options should help you understand the concepts instead of just knowing a single way of doing things. Understanding the patterns is the key to software architecture, not just learning how to apply them.

Splitting the layers

Now that we’ve discussed layers and seen them as big horizontal slices of responsibilities, we can organize our applications more granularly by splitting those big slices vertically, creating multiple smaller layers. This can help us organize applications by features or by bounding context, and it could also allow us to compose various user interfaces using the same building blocks, which would be easier than reusing colossal-size layers.Here is a conceptual representation of this idea:

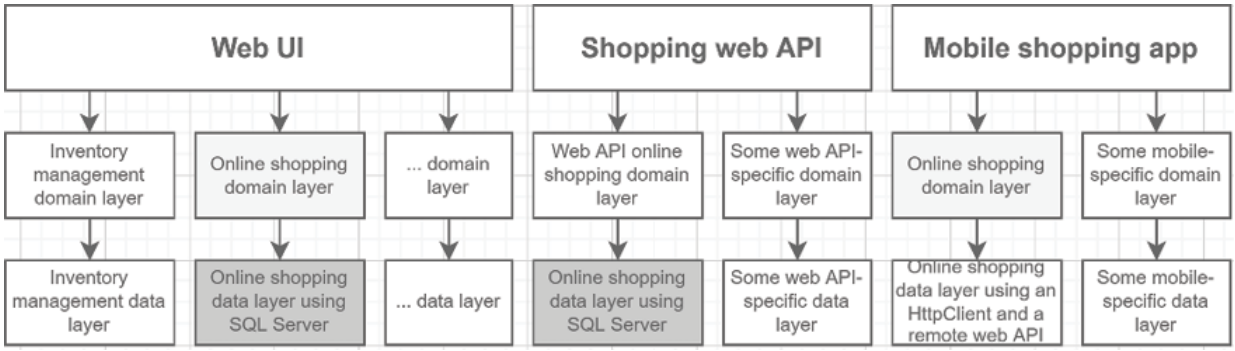


Figure 14.3: Organizing multiple applications using smaller partially shared layers

We can split an application into multiple features (vertically) and divide each into layers (horizontally). Based on the previous diagram, we named those features as follows:

- Inventory management
- Online shopping

- Others

So, we can bring in the online shopping domain and data layers to our Shopping web API without bringing everything else with it. Moreover, we can bring the online shopping domain layer to the mobile app and swap its data layer for another that talks to the web API.We could also use our web API as a plain and simple data access application with different logic attached to it while keeping the shopping data layer underneath.We could end up with the following recomposed applications (this is just one possible outcome):

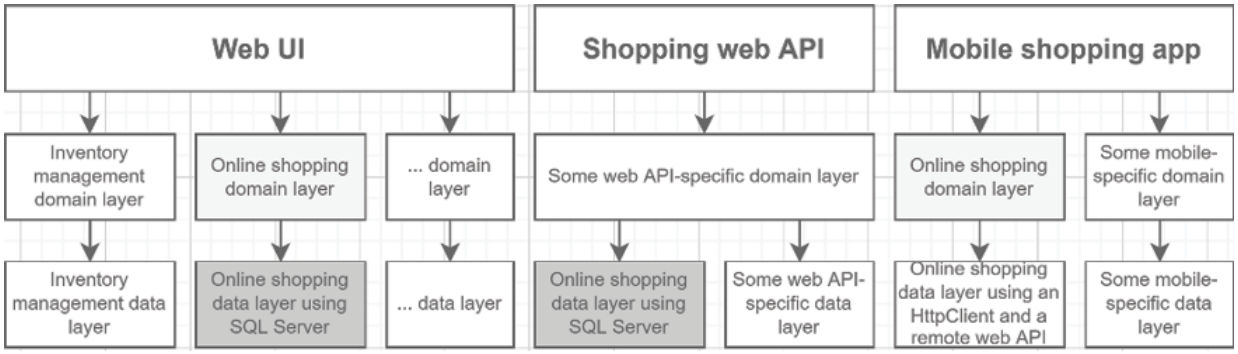


Figure 14.4: Organizing multiple applications using smaller partially shared layers

These are just examples of what we can conceptually do with layers. However, the most important thing to remember is not how the diagrams are laid out but the specifications of the applications you are building. Only those specs and good analyses can help you create the best possible design for that exact problem. I used a hypothetical shopping example here, but it could have been anything.Splitting huge horizontal slices vertically makes each piece easier to reuse and share. This improvement can yield interesting results, especially if you have multiple frontend apps or plan to migrate away from a monolith.

> A **monolithic application** (or monolith) is a program deployed as a single integrated piece with low modularity. A monolith can leverage layers or not. People often compare monolithic applications to microservices applications because they are antipodes. We explore microservices in *Chapter 19*, *Introduction to Microservices Architecture*, and monoliths in *Chapter 20*, *Modular Monoliths*.

Layers versus tiers versus assemblies

So far in this chapter, we have been talking about layers without talking about making them into code. Before jumping into that subject, I'd like to discuss **tiers**. You may have seen the term **3-tier architecture** somewhere before or heard people talking about **tiers** and **layers**, possibly interchanging them in the same context as synonyms. However, they are not the same.In a nutshell:

- **Tiers** are **physical**
- **Layers** are **logical**

What is a Tier?

We can deploy each **tier** on its own machine. For example, you could have a database server, a server hosting your web API that contains the business logic (the **domain**), and another server that serves an Angular application (**presentation**); these are three tiers (three distinct machines), and each **tier** can scale independently.We look at layers next.

What is a Layer?

On the other hand, each **layer** is only the logical organization of code, with concerns organized and divided in a layered fashion. For example, you may create one or more projects in Visual Studio and organize your code into

three layers. For example, a Razor Pages application depends on a business logic layer that depends on a data access layer. When you deploy that application, all these layers, including the database, are deployed together on the same server. This would be one tier and three layers. Of course, nowadays, chances are you have a cloud database somewhere, which adds a second tier to that architecture: the application tier (which still has three layers) and database tier.Now that we've discussed **layers** and **tiers**, let's look at a **layer** versus an **assembly**.

What is an assembly?

Assemblies are commonly compiled into `.dll` or `.exe` files; you can compile and consume them directly. In most cases, each project of a Visual Studio solution gets compiled into an assembly. You can also deploy them as NuGet packages and consume them from nuget.org or a custom NuGet repository of your choosing. But there is no one-to-one relationship between a layer and an assembly or a tier and an assembly; assemblies are only consumable units of compiled code: a library or a program.Moreover, you do not need to split your layers into different assemblies; you can have your three layers residing in the same assembly. It can be easier to create undesirable coupling this way, with all of the code being in the same project, but it is a viable option with some rigor, rules, and conventions. Moving each layer to an assembly does not necessarily improve the application; the code inside each layer or assembly can become mixed up and coupled with other system parts.Don't get me wrong: you can create an assembly per layer; I even encourage you to do so in most cases, but doing so does not mean the layers are not tightly coupled. A layer is simply a logical unit of organization, so each contributor's responsibility is to ensure the layer's code stays healthy.Furthermore, having multiple assemblies let us deploy them to one or more machines, potentially different machines, leading to multiple tiers.Let's now look at the responsibilities of the most common layers.

Responsibilities of the common layers

In this section, we explore the most commonly used layers in more depth. We do not dig too deep into each one, but the overview should help you understand the essential ideas behind layering.

Presentation

The **presentation layer** is probably the easiest layer to understand because it is the only one we can see: the user interface. However, the presentation layer can also be the data contracts in case of a REST, OData, GraphQL, or other types of web service. The presentation layer is what the user uses to access your program. As another example, a CLI program can be a presentation layer. You write commands in a terminal, and the CLI dispatches them to its domain layer, executing the required business logic.The key to a maintainable presentation layer is to keep it as focused on displaying the user interface as possible with as little business logic as possible.Next, we look at the **domain layer** to see where these calls go.

Domain

The **domain layer** is where the software's value resides and where most of the complexity lies. The **domain layer** is the home of your business logic rules.It is easier to sell a **user interface** than a **domain layer** since users connect to the domain through the presentation. However, it is important to remember that the domain is responsible for solving the problems and automating the solutions; the **presentation layer** only links users' actions to the **domain**.We usually build the domain layer around a domain model. There are two macro points of view on this:

- Using a **rich model**.
- Using an **anemic model**.

You can leverage **Domain-Driven Design** (**DDD**) to build that model and the program around it. DDD goes hand in hand with rich models, and a well-crafted model should simplify the maintenance of the program. Doing DDD is not mandatory, and you can achieve the required level of correctness without it.

Another dilemma is persisting the domain model directly into the database or using an intermediate data model. We talk about that in more detail in the *Data* section.Meanwhile, we look at the two primary ways to think about the domain model, starting with the rich domain model.

Rich domain model

A rich domain model is more object-oriented, in the “purest” sense of the term, and encapsulates the domain logic as part of the model inside methods. For example, the following class represents the rich version of a minimal `Product` class that contains only a few properties:

```
public class Product
{
    public Product(string name, int quantityInStock, int? id = null)
    {
        Name = name ?? throw new ArgumentNullException(nameof(name));
        QuantityInStock = quantityInStock;
        Id = id;
    }
    public int? Id { get; init; }
    public string Name { get; init; }
    public int QuantityInStock { get; private set; }
    public void AddStock(int amount)
    {
        if (amount == 0) { return; }
        if (amount < 0) {
            throw new NegativeValueException(amount);
        }
        QuantityInStock += amount;
    }
    public void RemoveStock(int amount)
    {
        if (amount == 0) { return; }
        if (amount < 0) {
            throw new NegativeValueException(amount);
        }
        if (amount > QuantityInStock) {
            throw new NotEnoughStockException(
                QuantityInStock, amount);
        }
        QuantityInStock -= amount;
    }
}
```

The `AddStock` and `RemoveStock` methods represent the domain logic of adding and removing stock for the product inventory. Of course, we only increment and decrement a property’s value in this case, but the concept would be the same in a more complex model.The biggest advantage of this approach is that most of the logic is built into the model, making this very domain-centric with operations programmed on model entities as methods. Moreover, it reaches the basic ideas behind object-oriented design, where behaviors should be part of the objects, making them a virtual representation of their real-life counterparts.The biggest drawback is the accumulation of responsibilities by a single class. Even if object-oriented design tells us to put logic into the objects, this does not mean it is always a good idea. If flexibility is important for your system, hardcoding logic into the domain model may hinder your ability to evolve business rules without changing the code itself (it can still be done). A rich model might be a good choice for your project if the domain is fixed and predefined.A relative drawback of this approach is that injecting dependencies into the domain model is harder than other objects, such as services. This drawback reduces flexibility and increases the complexity of creating the models.A rich domain model can be useful if you are building a stateful application where the domain model can live in memory longer than the time of an HTTP request. Other patterns can help you with that, such as **Model-View-View-Model** (**MVVM**), Model-View-Presenter (MVP), and **Model-View-Update** (**MVU**).If you believe your application would benefit from keeping the data and the logic together, then a rich domain model is most likely a good idea for your project. If you are practicing DDD, I probably don’t have to tell you that a rich model is the way to go. Without DDD notions, achieving a maintainable and flexible rich model is challenging.A rich model can be a good option if your program is built around a complex domain model and persists those classes directly to your database using an **object-relational mapper** (**ORM**). Using Cosmos DB, Firebase, MongoDB, or any other document database can make

storing complex models as a single document easier than a collection of tables (this applies to anemic models too).As you may have noticed, there are a lot of “ifs” in this section because I don’t think there is an absolute answer to whether a rich model is better or not, and it is more a question of whether it is better for your specific case than better overall. You also need to take your personal preferences and skills into account.Experience is most likely your best ally here, so I’d recommend coding, coding, and coding more applications to acquire that experience.

Anemic domain model

An anemic domain model usually does not contain methods but only getters and setters. Such models must not contain business logic rules. The `Product` class we had previously would look like this:

```
public class Product
{
    public int? Id { get; set; }
    public required string Name { get; set; }
    public int QuantityInStock { get; set; }
}
```

In the preceding code, there is no method in the class anymore, only the three properties with public setters. We can also leverage a record class to add immutability to the mix. As for the logic, we must move it elsewhere, in other classes. One such pattern would be to move the logic to a **service layer**.A **service layer** in front of such an **anemic model** would take the input, mutate the domain object, and update the database. The difference is that the service owns the logic instead of the rich model.With the anemic model, separating the operations from the data can help us add flexibility to a system. However, enforcing the model’s state at any given time can be challenging since external actors (services) are modifying the model instead of the model managing itself.Encapsulating logic into smaller units makes it easier to manage each of them, and it is easier to inject those dependencies into the service classes than injecting them into the entities themselves. Having more smaller units of code can make a system more dreadful for a newcomer as it can be more complex to understand since it has more moving parts. On the other hand, if the system is built around well-defined abstractions, it can be easier to test each unit in isolation.However, the tests can be quite different. In the case of our rich model, we test the rules and the persistence separately. We call this **persistence ignorance**, which allows us to test business rules in isolation. Then we could create integration tests to cover the persistence aspect of the service layer and more unit and integration tests on the data and domain levels. With an anemic model, we test both the business rules and the persistence simultaneously with integration tests at the service layer level or test only the business rules in unit tests that mock the persistence part away. Since the model is just a data bag without logic, there is nothing to test there.All in all, if the same rigorous domain analysis process is followed, the business rules of an anemic model backed by a service layer should be as complex as a rich domain model. The biggest difference should be in which classes the methods are located.An anemic model is a good option for stateless systems, such as RESTful APIs. Since you have to recreate the model’s state for every request, an anemic model can offer you a way to independently recreate a smaller portion of the model with smaller classes optimized for each use case. Stateless systems require a more procedural type of thinking than a purely object-oriented approach, leaving the anemic models as excellent candidates for that.

> I personally love anemic models behind a service layer, but some people would not agree with me. I recommend choosing what you think is best for the system you are building instead of doing something based on what someone else did in another system.
>
> > Another good tip is to let the refactoring flow *top-down* to the right location. For example, if you feel that a method is bound to an entity, nothing stops you from moving that piece of logic into that entity instead of a service class. If a service is more appropriate, move the logic to a service class.

Next, let’s go back to the **domain layer** and explore a pattern that emerged over the years to shield the **domain model** using a **service layer**, splitting the **domain layer** into two distinct pieces.

Service layer

The **service layer** shields the domain model and encapsulates domain logic. The service layer orchestrates the complexity of interacting with the model or external resources such as databases. Multiple components can then use the service layer while having limited knowledge of the model:

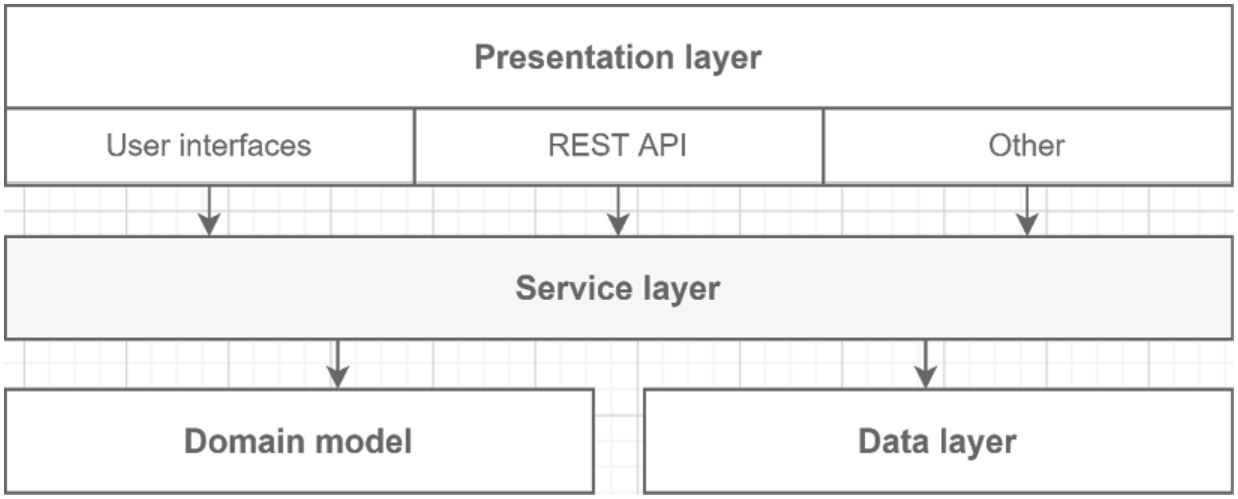


Figure 14.5: Service layer relationships with other layers

The preceding diagram shows that the presentation layer talks to the service layer, which manages the domain model and implements the business logic.The **service layer** contains services, which are classes that interact with other **domain objects**, such as the **domain model** and the **data layer**.We can further divide services into two categories, **domain services**, and **application services**:

- **Domain services** are those services we are talking about so far. They contain domain logic and allow consumers from the presentation layer to read or write data. They access and mutate the domain model.
- **Application services** like email services are unrelated to the domain and should live elsewhere, like in a shared (why rewrite an email service for every project, right?).

As with other layers, your service layer could expose its own model, shielding its consumers from domain model (internal) changes. In other words, the service layer should only expose its contracts and interfaces (keyword: shield). **A service layer is a form of façade.**

We further explore ways to keep copying anemic classes into other anemic classes to a minimum.

There are many ways to interpret this layer, and I'll try to illustrate as many as possible in a condensed manner (from simpler to more complex ones):

- The classes and interfaces of the service layer could be part of the domain layer's assembly, created in a *Services* directory, for example. This is less reusable, but it paves the way to sharing services in the future without managing multiple projects at first. It needs rigor to not depend on what you should not.
- The service layer could be an assembly containing interfaces and implementation. This is a great compromise between reusability and maintenance time. Chances are you will never need two implementations (see the next point) because the services are tied to the logic; they are the domain. You could even hide the implementation, as we did with the **opaque façade** in *Chapter 11*, *Structural Patterns*.
- The service layer could be divided into two assemblies -- one containing abstractions (referenced by consumers) and one containing implementations.
- The service layer could be an actual web service tier (such as a web API).

When writing services code, by convention, people usually suffix a service class with `Service`, such as `ProductService` and `InventoryService`; the same goes for interfaces (`IProductService` and `IInventoryService`).No matter which technique you choose, remember that the service layer contains the domain logic and shields the domain model from direct access.The service layer is an amazing addition that

shields and encapsulates the logic for manipulating an anemic domain model. It can defeat the purpose of a rich domain model if it's just a pass-through but can be very useful to handle complex, non-atomic business rules that affect multiple domain objects.The primary decider of whether or not to add a service layer is tied to the complexity of your project's domain. The more complex, the more it makes sense. The more trivial, the less it makes sense. Here are a few tips:

- Add a service layer when using an anemic model.
- Add a service layer for very complex domains.
- Do not add a service layer for low-complexity domains or *façade over database* applications.

Now, let's look at the data layer.

Data

The **data layer** is where the persistence code goes. In most programs, we need some kind of persistence to store our application data, which is often a database. Several patterns come to mind when discussing the data layer, including the **Unit of Work** and **Repository patterns**, which are very common. We cover these two patterns very briefly at the end of this subsection.We can persist our **domain model** as is or create a **data model** that is more suited to be stored. For example, a many-to-many relationship is not a thing in the object-oriented world, while it is from a relational database standpoint.You can view a **data model** like a **DTO** for the data. The **data model** is how the data is stored in your data store; that is, how you modeled your data or what you have to live with.In a classic layering project, you have no choice but to have a data model. However, we explore better solutions as we continue to explore additional options.

> An **ORM** is a piece of software that translates objects into a database language such as SQL. It allows mutating data, querying data, loading that data into objects, and more.

Modern data layers usually leverage an **ORM** such as **Entity Framework Core** (**EF Core**), which does a big part of our job, making our lives easier. In the case of **EF Core**, it allows us to choose between multiple providers, from SQL Server to Cosmos DB, passing by the in-memory provider. The great thing about EF Core is that it already implements the **Unit of Work** and the **Repository** patterns for us, among other things. In the book, we use the in-memory provider to cut down setup time and run integration tests.

> If you've used EF6 before and dread Entity Framework, know that EF Core is lighter, faster, and easier to test. Feel free to give it a second shot. EF Core's performance is very high now too. However, if you want complete control over your SQL code, look for Dapper (not to be confused with **Dapr**).

I don't want to go into too much detail about these patterns, but they are important enough to deserve an overview. As mentioned, EF Core already implements these patterns, so we don't have to deal with them. Moreover, using such patterns is not always desirable, can be hard to implement right, and can lead to bloated data access layers, but they can also be very useful when used well.

> I've written a multi-part article series about the Repository pattern. See the *Further reading* section.

In the meantime, let's at least study their goals to know what they are for, and if the situation arises where you need to write such components, you know where to look.

Repository pattern

The goal of the Repository pattern is to allow consumers to query the database in an object-oriented way. Usually, this implies caching objects and filtering data dynamically. EF Core represents this concept with a `DbSet<T>` and provides dynamic filtering using LINQ and the `IQueryable<T>` interface.People also use the term **repository** to represent the **Table Data Gateway pattern**, which is another pattern that models a class that gives us access to a single table in a database and provides access to operations such as creating, updating, deleting, and fetching entities from that database table. Both patterns are from the *Patterns of Enterprise Application Architecture* and are extensively used.Homegrown custom implementations usually follow the Table Data Gateway pattern more than

the Repository pattern. They are based on an interface that looks like the following code and contains methods to create, update, delete, and read entities. They can have a base entity or not, in this case, `IEntity<TId>`. The `Id` property can also be generic or not:

```
public interface IRepository<T, TId>
    where T : class, IEntity<TId>
{
    Task<IEnumerable<T>> AllAsync(CancellationToken cancellationToken);
    Task<T?> GetByIdAsync(TId id, CancellationToken cancellationToken);
    Task<T> CreateAsync(T entity, CancellationToken cancellationToken);
    Task UpdateAsync(T entity, CancellationToken cancellationToken);
    Task DeleteAsync(TId id, CancellationToken cancellationToken);
}
public interface IEntity<TId>
{
    TId Id { get; }
}
```

One thing that often happens with those table data gateways is that people add a save method to the interface. As long as you update a single entity, it should be fine. However, that makes transactions that cross multiple repositories harder to manage or dependent on the underlying implementation (breaking abstraction). To commit or revert such transactions, we can leverage the Unit of Work pattern, moving the save method from the table data gateway there.For example, when using EF Core, we can use `DbSet<Product>` (the `db.Products` property) to add new products to the database, like this:

```
db.Products.Add(new Data.Product
{
    Id = 1,
    Name = "Banana",
    QuantityInStock = 50
});
```

For the querying part, the easiest way to find a single product is to use it like this:

```
var product = _db.Products.Find(productId);
```

However, we could use LINQ instead:

```
_db.Products.Single(x => x.Id == productId);
```

These are some of the querying capabilities that a **repository** should provide. EF Core seamlessly translates LINQ into the configured provider expectations like SQL, adding extended filtering capabilities.Of course, with EF Core, we can query collections of items, fetching all products and projecting them as domain objects like this:

```
_db.Products.Select(p => new Domain.Product
{
    Id = p.Id,
    Name = p.Name,
    QuantityInStock = p.QuantityInStock
});
```

We can also filter further using LINQ here; for example, by querying all the products that are out of stock:

```
var outOfStockProducts = _db.Products
    .Where(p => p.QuantityInStock == 0);
```

We could also allow a margin for error, like so:

```
var mostLikelyOutOfStockProducts = _db.Products
    .Where(p => p.QuantityInStock < 3);
```

We now have briefly explored how to use the EF Core implementation of the Repository pattern, `DbSet<T>`. These few examples might seem trivial, but it would require considerable effort to implement custom repositories on par with EF Core's features.EF Core's unit of work, the `DbContext` class, contains the *save* methods to persist the

modifications done to all its `DbSet<T>` properties (the repositories). Homebrewed implementations often feature such methods on the repository itself, making cross-repository transactions harder to handle and leading to bloated repositories containing tons of operation-specific methods to handle such cases.Now that we understand the concept behind the **Repository pattern**, let's jump into an overview of the **Unit of Work pattern** before going back to layering.

Unit of Work pattern

A **unit of work** keeps track of the object representation of a transaction. In other words, it manages a registry of what objects should be created, updated, and deleted. It allows us to combine multiple changes in a single transaction (one database call), offering multiple advantages over calling the database every time we make a change.Assuming we are using a relational database, here are two advantages:

- First, it can speed up data access; calling a database is slow, so limiting the number of calls and connections can improve performance.
- Second, running a transaction instead of individual operations allows us to roll back all operations if one fails or commit the transaction as a whole if everything succeeds.

EF Core implements this pattern with the `DbContext` class and its underlying types, such as the `DatabaseFacade` and `ChangeTracker` classes.Our small applications don't need transactions, but the concept remains the same. Here is an example of what happens using EF Core:

```
var product = _db.Products.Find(productId);
product.QuantityInStock += amount;
_db.SaveChanges();
```

The preceding code does the following:

1. Queries the database for a single entity.
2. Changes the value of the `QuantityInStock` property.
3. Persists the changes back into the database.

In reality, what happened is closer to the following:

1. We ask EF Core for a single entity through the `ProductContext` (a unit of work), which exposes the `DbSet<Product>` property (the product repository). Under the hood, EF Core does the following:
   A. Queries the database.
   B. Caches the entity.
   C. Tracks changes for that entity.
   D. Returns it to us.
2. We change the value of the `QuantityInStock` property; EF Core detects the change and marks the object as *dirty*.
3. We tell the unit of work to persist the changes that it tracked, saving the *dirty product* back to the database.

In a more complex scenario, we could have written the following code:

```
_db.Products.Add(newProduct);
_db.Products.Remove(productToDelete);
product.Name = "New product name";
_db.SaveChanges();
```

Here, the `SaveChanges()` method triggers saving the three operations instead of sending them individually. You can control database transactions using the `Database` property of `DbContext` (see the *Further reading* section for more information).Now that we've explored the **unit of work** pattern, we could implement one by ourselves. Would that add value to our application? Probably not. If you want to build a custom **unit of work** or a wrapper over EF Core, there are plenty of existing resources to guide you. Unless you want to experiment or need a custom **unit of work** and **repository** (which is possible), I recommend staying away from doing that. Remember: **do only what needs to be done for your program to be correct**.

> Don’t get me wrong when I say *do only what needs to be done*; wild engineering endeavors and experimentations are a great way to explore, and I encourage you to do so. However, I recommend doing so in parallel so that you can innovate, learn, and possibly even migrate that knowledge to your application later instead of wasting time and breaking things. If you are using Git, creating an experimental branch is a good way of doing this. You can then delete it when your experimentation does not work, merge the branch if it yields positive results, or leave it there as a reference (depending on the team’s policies in place).

Now that we explored a high-level view of the Repository and Unit of Work patterns, and what those common layers are for, we can continue our journey of using layers.

Abstract layers

This section looks at abstract layers using an abstract data layer implementation. This type of abstraction can be very useful and is another step closer to **Clean Architecture**. Moreover, you can abstract almost anything this way, which is nothing more than applying the **Dependency Inversion Principle** (**DIP**).Let’s start with some context and the problem:

- The **domain layer** is where the logic lies.
- The **UI** links the user to the **domain**, exposing the features built into that **domain**.
- The **data layer** should be an implementation detail that the **domain** blindly uses.
- The **data layer** contains the code that knows where the data is stored, which should be irrelevant to the **domain**, but the **domain** directly depends on it.

The solution to **break the tight coupling** between the **domain** and the **data** persistence implementations is to create an additional abstract layer, as shown in the following diagram:

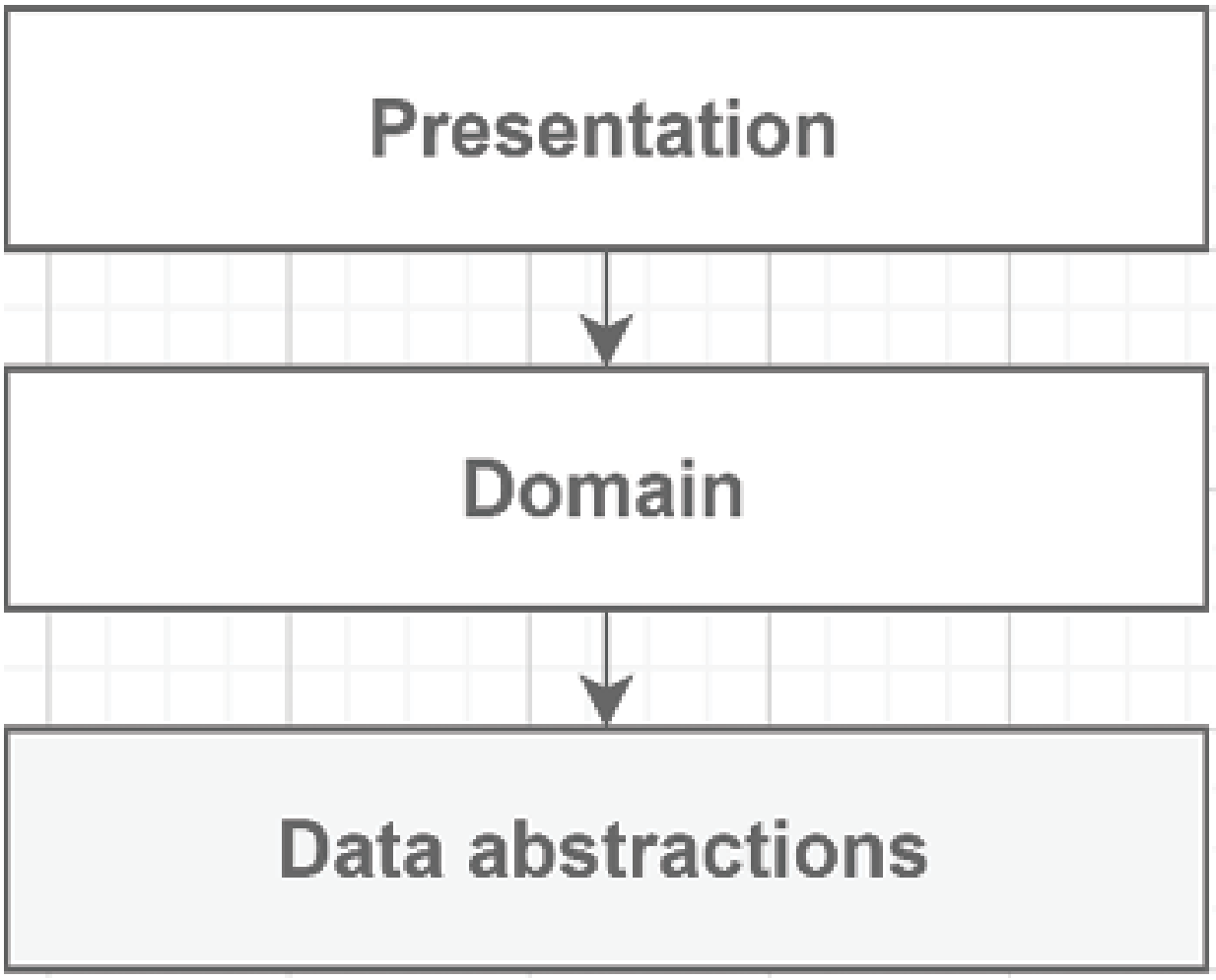


Figure 14.6: Replacing the data (persistence) layer with a data abstraction layer

New rule: **only interfaces and data model classes go into the data abstractions layer**. This new layer now defines our data access API and does nothing but expose a set of interfaces—the contract.Then, **we can create one or more data implementations** based on that abstract layer contract, like using EF Core. The link between the abstractions and implementations is done with dependency injection. The bindings defined in the **composition root** explain the indirect connection between the presentation and the data implementation.The new dependency tree looks like this:

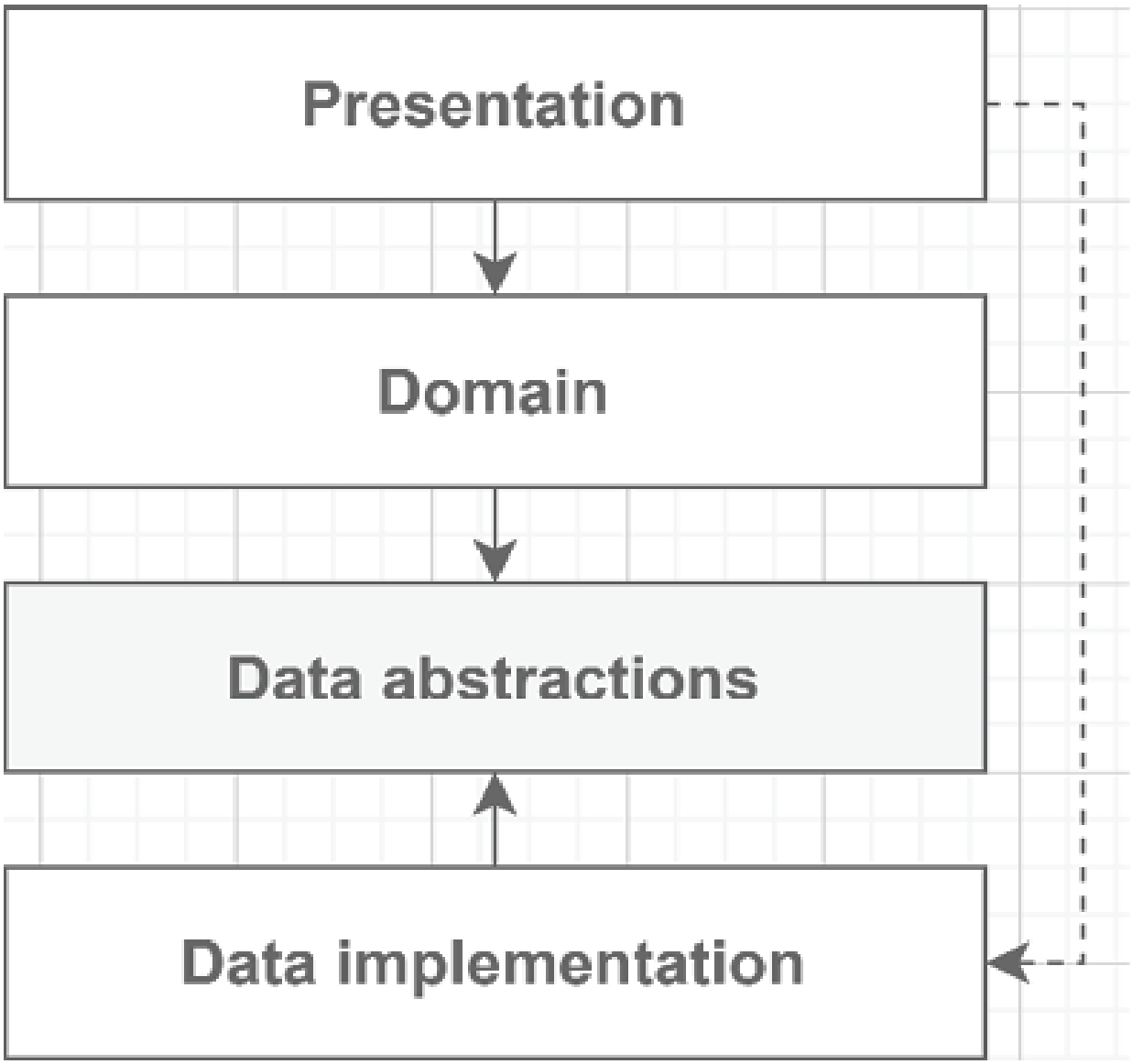


Figure 14.7: The relationships between layers

The **presentation layer** references a **data implementation layer** for the sole purpose of creating the DI bindings. We need those bindings to inject the correct implementation when creating **domain** classes. Besides, **the presentation layer must not use the data layer's abstractions or implementations**.I created a sample project that showcases the relationships between the projects and the classes. However, that project would have added pages of code, so I decided not to include it in the book. The most important thing about abstract layers is the dependency flow between the layers, not the code itself.

The project is available on GitHub (https://adpg.link/s9HX).

In that project, the program injects an instance of the `EF.ProductRepository` class when a consumer asks for an object that implements the `IProductRepository` interface. In that case, the consuming class is `ProductService` and only depends on the `IProductRepository` interface. The `ProductService` class is unaware of the implementation itself: it leverages only the interface. The same goes for the program that loads a `ProductService` class but knows only about the `IProductService` interface. Here is a visual representation of that dependency tree:

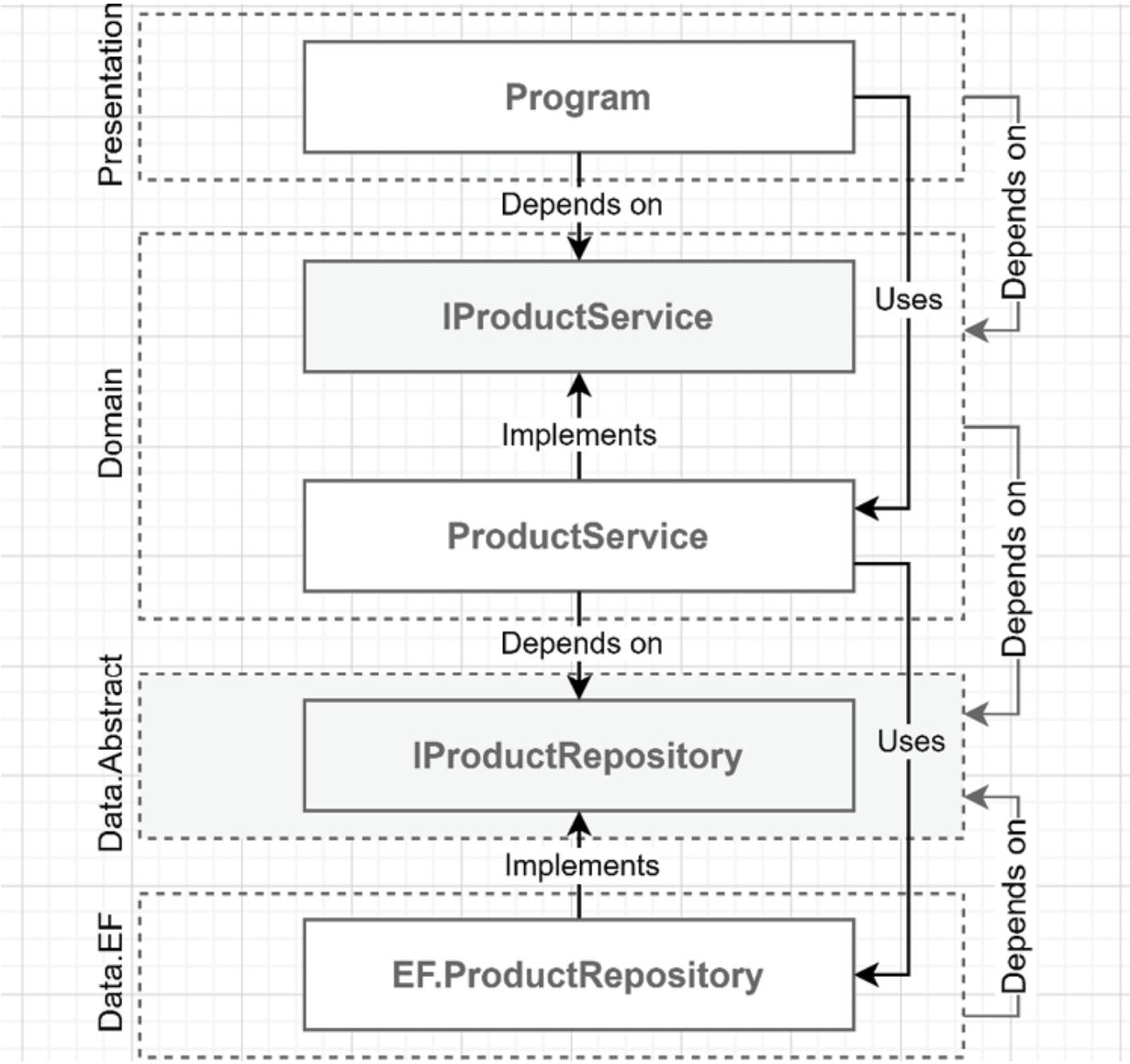


Figure 14.8: The dependency flow between layers, classes, and interfaces

In the preceding diagram, look at how dependencies converge on the `Data.Abstract` layer. The dependency tree ends up on that abstract data layer.With this applied piece of architectural theory, we are inverting the flow of dependencies on the data layer by following the **DIP**. We also cut out the direct dependency on EF Core, allowing us to implement a new data layer and swap it without impacting the rest of the application or update the implementation without affecting the domain. As I mentioned previously, swapping layers should not happen very often, if ever. Nonetheless, this is an important part of the evolution of layering, and more importantly, we can apply this technique to any layer or project, not just the data layer, so it is imperative to understand how to invert the dependency flow.

> To test the APIs, you can use the Postman collection that comes with the book; visit https://adpg.link/postman8 or GitHub (https://adpg.link/net8) for more info.

Next, let's explore sharing and persisting a rich domain model.

Sharing the model

We have explored strict layering and how to apply the DIP, but we still have multiple models. An alternative to copying models from one layer to another is to share a model between multiple layers, generally as an assembly. Visually, it looks like this:

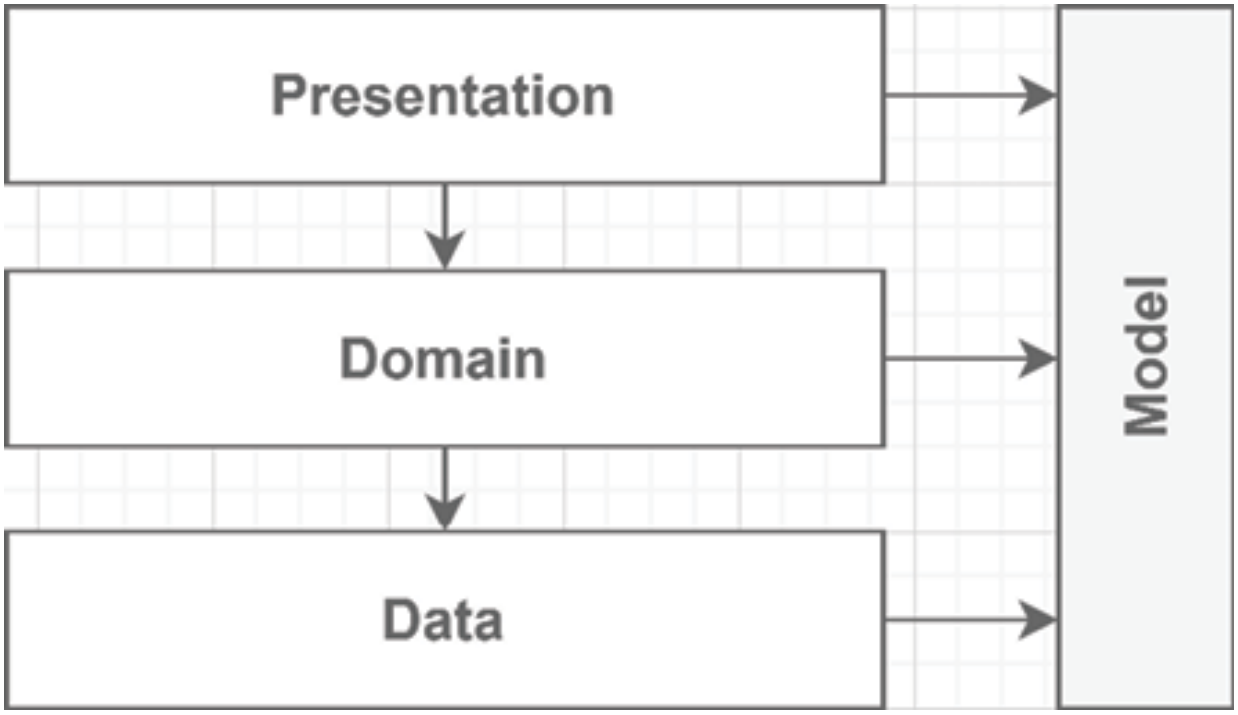


Figure 14.9: Sharing a model between all three layers

Everything has pros and cons, so no matter how much time this can save you at first, it will come back to haunt you and become a pain point later as the project advances and becomes more complex.Suppose you feel that sharing a model is worth it for your application. In that case, I recommend using **view models** or **DTOs** at the presentation level to control and keep the input and output of your application loosely coupled from your model. This way of shielding your lower layers can be represented as follows:

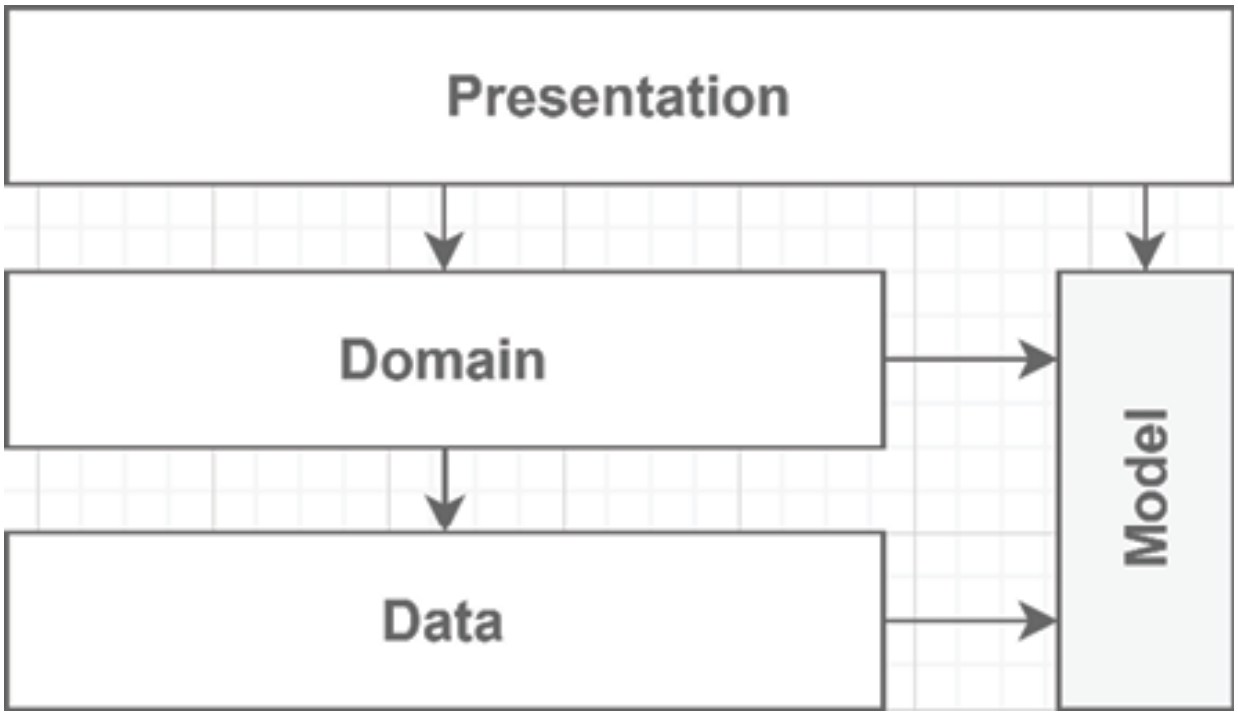


Figure 14.10: Sharing a model between the domain and data layers

By doing that, you will save some time initially by sharing your model between your domain and data layers. By hiding that shared model under the presentation layer, you should dodge many problems in the long run, making this a good compromise between quality and development time. Moreover, since your presentation layer shields your application from the outside world, you can refactor your other layers without impacting your consumers.

> This is pretty much how Clean Architecture does it but represented differently. Using that, the model is at the center of the application and is manipulated and persisted. While the layers have different names, the concept remains very similar. More on that later.

View models and **DTOs** are key elements to successful programs and developers' sanity; they should save you many headaches for long-running projects. We revisit and explore the concepts of controlling the input and output later in *Chapter 16*, *Mediator and CQRS Design Patterns*, where inputs become **commands** and **queries**.Meanwhile, let's merge that concept with an abstraction layer. In the previous project, the **data abstraction layer** owned the **data model**, and the **domain layer** owned the **domain model**.In this architectural alternative, we are sharing the model between the two layers. The presentation layer can indirectly use that shared model to dialog with the domain layer without exposing it externally. The objective is to directly persist the **domain model** and skip the copy from the **domain** to the **data layer** while having that data abstraction layer that breaks the tight coupling between the domain logic and the persistence.Here is a visual representation of that:

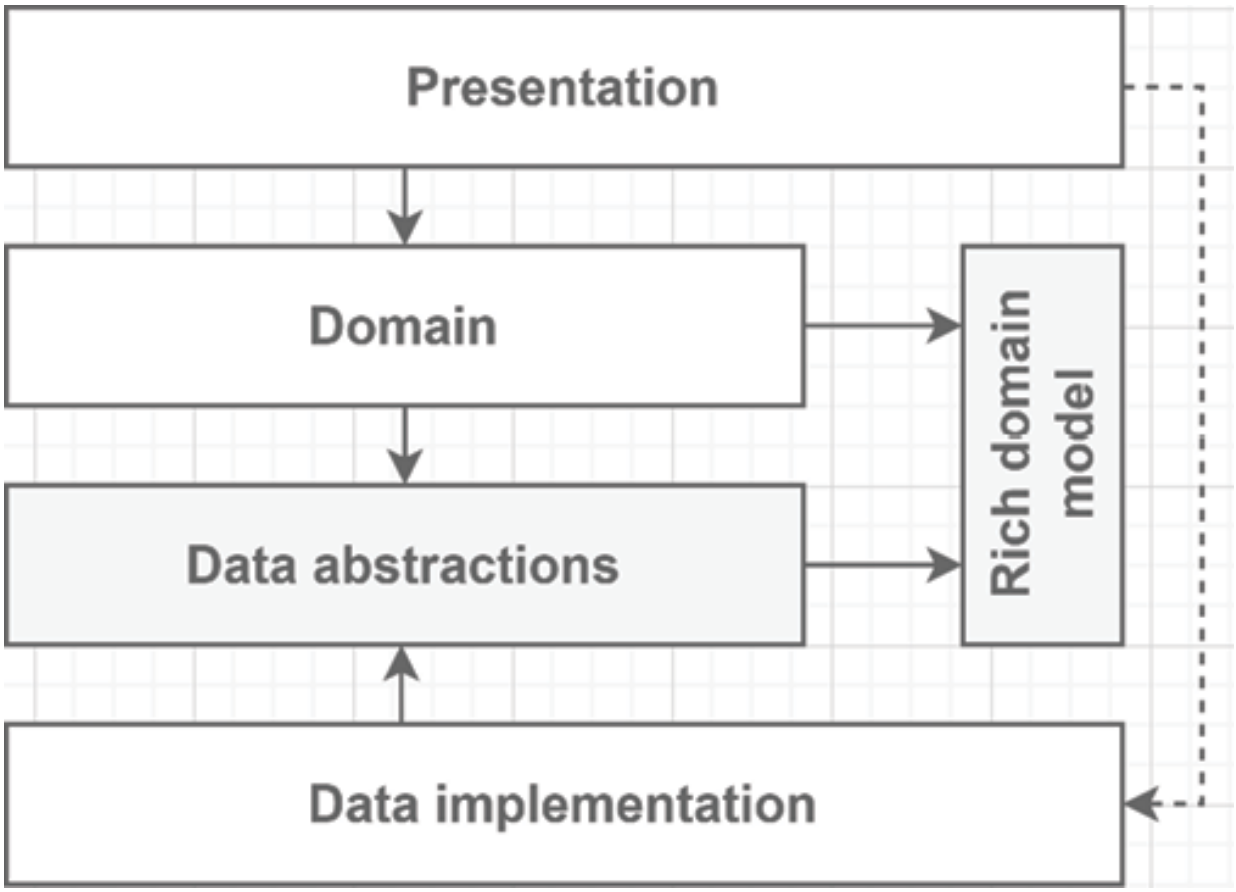


Figure 14.11: Diagram representing a shared rich model

It is well suited for **rich models,** but we can also do this for anemic models. With a **rich domain model**, you delegate the job of reconstructing the model to the ORM and immediately start calling its methods.The ORM also recreates the anemic model, but those classes just contain data, so you need to call other pieces of the software that contain the logic to manipulate those objects.In the code sample, the **data abstraction layer** now contains only the data access abstractions, such as the repositories, and it references the new `Model` project that is now the persisted model.Conceptually, it cleans up a few things:

- The data abstraction layer's only responsibility is to contain data access abstractions.
- The domain layer's only responsibility is implementing the domain services and the logic that is not part of that rich model.
- In the case of an anemic model, the domain layer's responsibility would be to encapsulate all the domain logic.
- The `Model` project contains the entities.

Once again, I skip publishing most of the code here as it is irrelevant to the overall concept. If you think reading the code would help, you can consult and explore the sample on GitHub (https://adpg.link/9F5C). Using an IDE to browse the code should help you understand the flow, and as with the abstract layer, the dependencies between the projects, classes, and interfaces are the key to this.Nevertheless, here is the `StockService` class that uses that shared model so you can peek at some code that directly relates to the explanations:

```
namespace Domain.Services;
public class StockService : IStockService
{
    private readonly IProductRepository _repository;
    public StockService(IProductRepository repository)
    {
```

```
        _repository = repository ?? throw new ArgumentNullException(nameof(repository));
    }
```

In the preceding code, we are injecting an implementation of the `IProductRepository` interface we use in the next two methods. Next, we look at the `AddStockAsync` method:

```
    public async Task<int> AddStockAsync(int productId, int amount, CancellationToken cancellati
    {
        var product = await _repository.FindByIdAsync(productId, cancellationToken);
        if (product == null)
        {
            throw new ProductNotFoundException(productId);
        }
        product.AddStock(amount);
        await _repository.UpdateAsync(product, cancellationToken);
        return product.QuantityInStock;
    }
```

The fun starts in the preceding code, which does the following:

- The repository recreates the product (model) that contains the logic.
- It validates that the product exists.
- It uses that model and calls the `AddStock` method (encapsulated domain logic).
- It tells the repository to update the product.
- It returns the updated product's `QuantityInStock` to the consumer of the service.

Next, we explore the `RemoveStockAsync` method:

```
    public async Task<int> RemoveStockAsync(int productId, int amount, CancellationToken cancell
    {
        var product = await _repository.FindByIdAsync(productId, cancellationToken);
        if (product == null)
        {
            throw new ProductNotFoundException(productId);
        }
        product.RemoveStock(amount);
        await _repository.UpdateAsync(product, cancellationToken);
        return product.QuantityInStock;
    }
}
```

We applied the same logic as the `AddStock` method to the `RemoveStock` method, but it calls the `Product.RemoveStock` method instead. From the `StockService` class, we can see the service gating the access to the domain model (the product), fetching and updating the model through the abstract data layer, manipulating the model by calling its methods, and returning domain data (an `int` in this case, but could be an object).

> This type of design can be either very helpful or undesirable. Too many projects depending on and exposing a shared model can lead to leaking part of that model to consumers, for example exposing properties that shouldn't be, exposing the whole domain model as output, or the very worst, exposing it as an input and opening exploitable holes and unexpected bugs.
>
> Be careful not to expose your shared model to the presentation layer consumers.

Pushing logic into the model is not always possible or desirable, which is why we are exploring multiple types of domain models and ways to share them. Making a good design is often about options and deciding what option to use for each scenario. There are also tradeoffs to make between flexibility and robustness.The rest of the code is similar to the abstract layer project. Feel free to explore the source code (https://adpg.link/9F5C) and compare it with the other projects. The best way to learn is to practice, so play with the samples, add features, update the current features, remove stuff, or even build your own project. Understanding these concepts will help you apply them to different scenarios, sometimes creating unexpected but efficient constructs.Now, let's look at the final evolution of layering: Clean Architecture.

Clean Architecture

Now that we've covered many layering approaches, it is time to combine them into **Clean Architecture**, also known as Hexagonal Architecture, Onion Architecture, Ports and Adapters, and more. Clean Architecture is an evolution of the layers, a way of organizing the relationships between the layers, yet very similar to what we just built. Instead of presentation, domain, and data (or persistence), Clean Architecture suggests **UI**, **Core**, and **Infrastructure**.As we saw previously, we can design a layer containing abstractions or implementations. When implementations depend only on abstractions, that inverts dependency flow. Clean Architecture emphasizes such layers but with its own guidance about organizing them.We also explored the theoretical concept of breaking layers into smaller ones (or multiple projects), thus creating "fractured layers" that are easier to port and reuse. Clean Architecture leverages that concept at the infrastructure layer level.There are probably as many points of view and variants of this as there are names for it, so I'll try to be as general as possible while keeping the essence. By doing this, if you are interested in this type of architecture, you can pick a resource and dig deeper into it, following the style you prefer.Let's take a look at a diagram that resembles what we can find online:

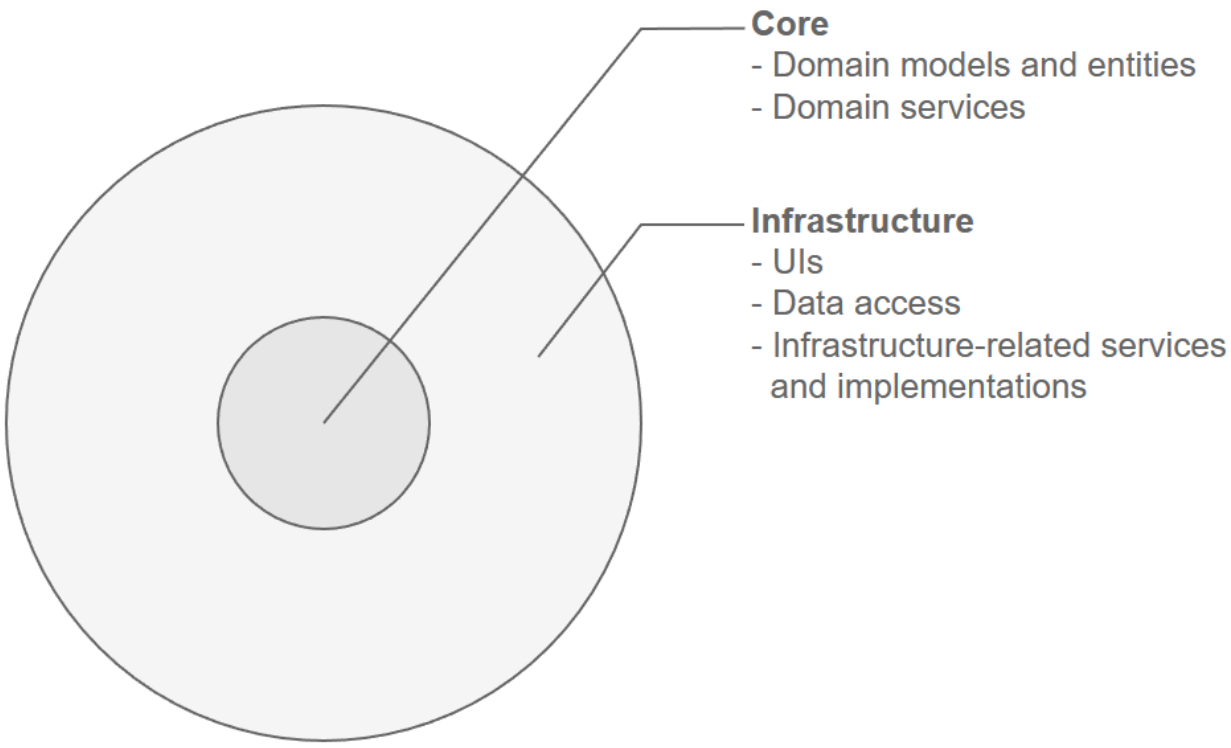


Figure 14.12: A diagram representing the most basic Clean Architecture layout

From a layering diagram-like standpoint, the preceding diagram could look like this:

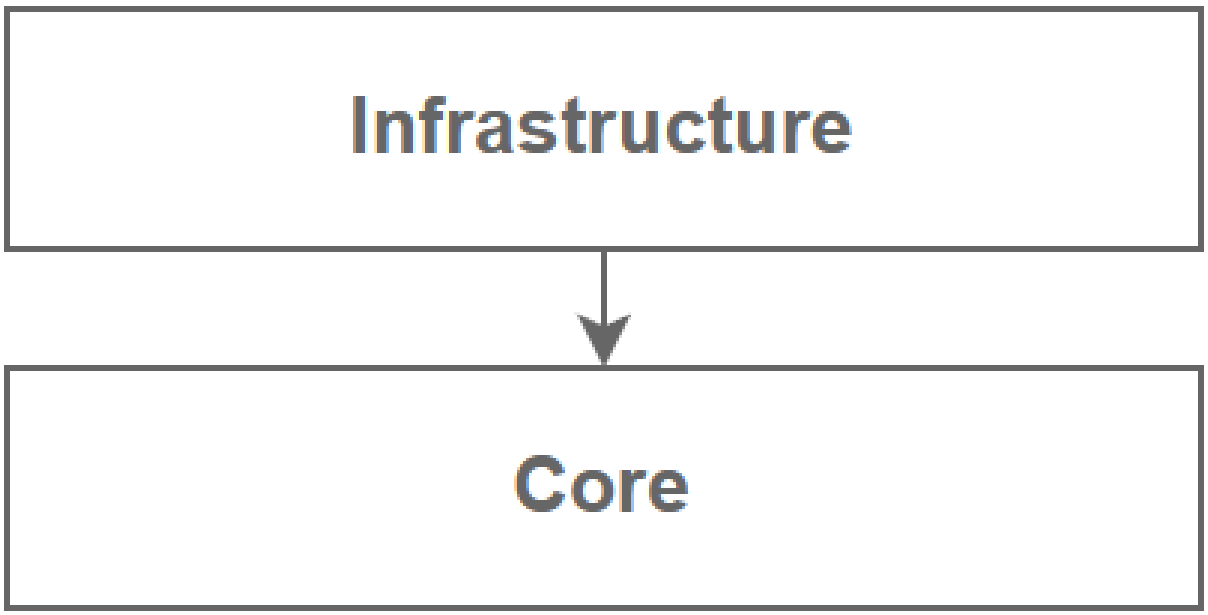


Figure 14.13: A two-layer view of the previous Clean Architecture diagram

Depending on your chosen method, you can split those layers into multiple other sublayers. One thing that we often see is dividing the **Core** layer into **Entities** and **Use cases**, like this:

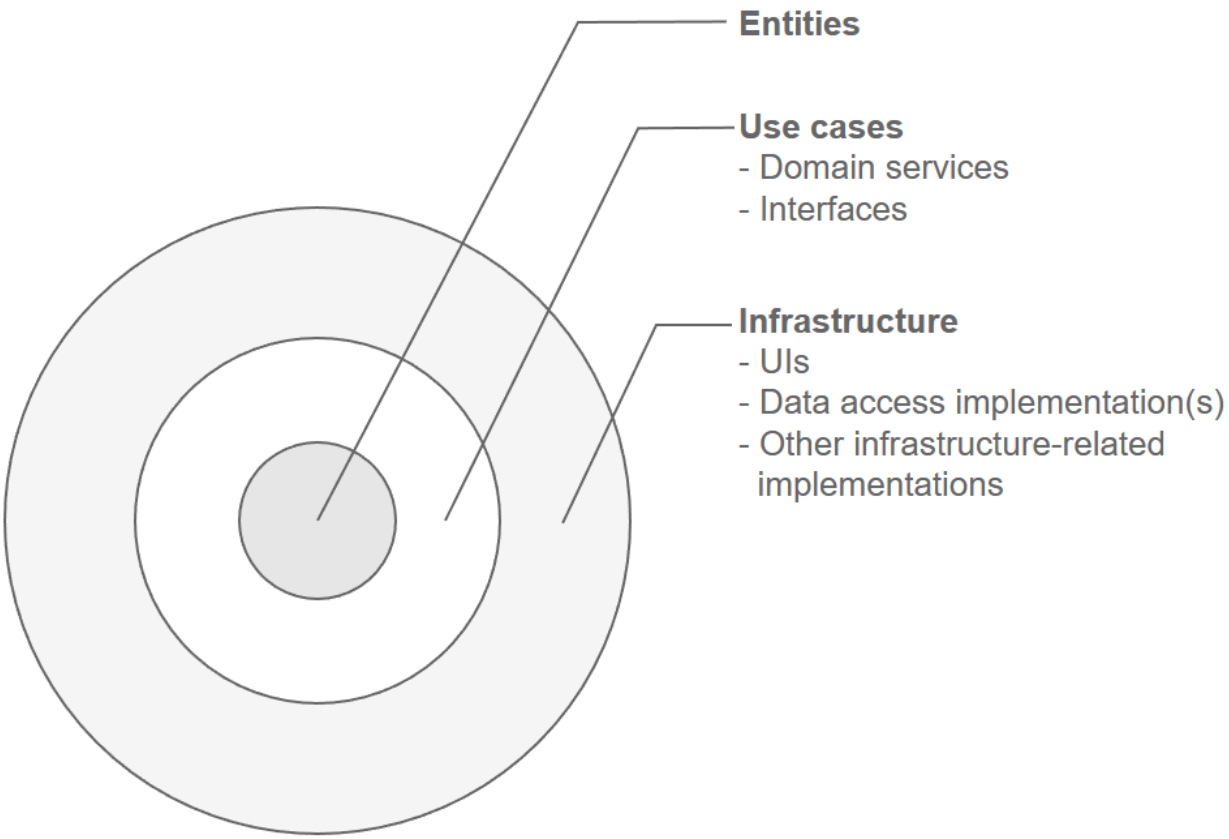


Figure 14.14: Widespread Clean Architecture layout diagram

Since people in the tech industry are creative, there are many names for many things, but the concepts remain the same. From a layering diagram-like standpoint, that diagram could look like this:

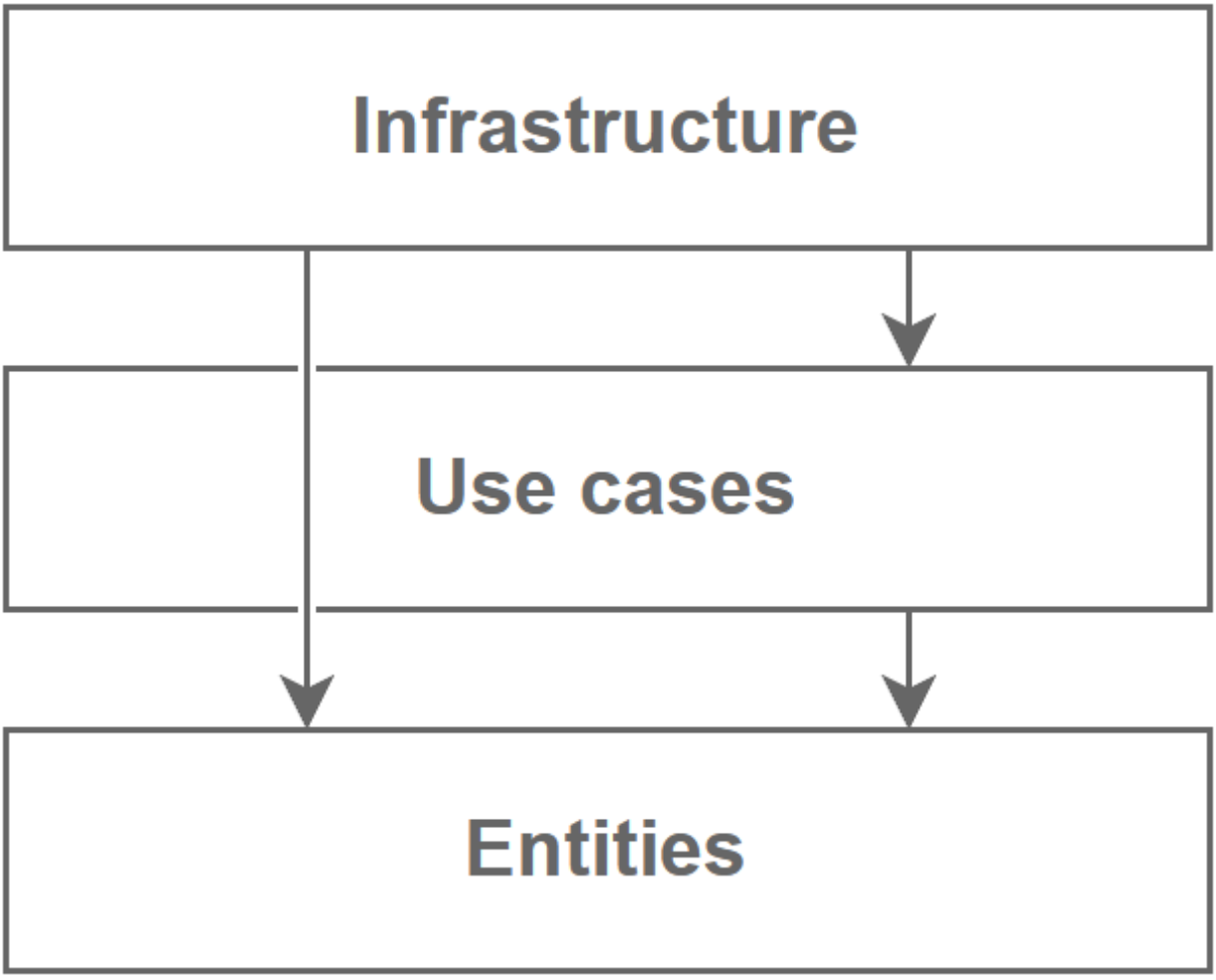


Figure 14.15: A layer-like view of the previous Clean Architecture diagram

The infrastructure layer is conceptual and can represent multiple projects, such as an infrastructure assembly containing EF Core implementations and a website project representing the web UI. We could also add more projects to the infrastructure layer.The dependency rule of Clean Architecture states that dependencies can only point inward, from the outer layers to the inner layers. This means that abstractions lie inside, and concretions lie outside. Based on the preceding layer-like diagram, inside translates to downward. That means a layer can use any direct or transitive dependencies, which means that infrastructure can depend on use cases and entities.Clean Architecture follows all the principles that we've been discussing since the beginning of this book, such as decoupling our implementations using abstractions, dependency inversion, and separation of concerns. These implementations are glued over abstractions using dependency injection (this is not mandatory, but it helps).I've always found those circle diagrams a bit confusing, so here is my take on an updated, more linear diagram:

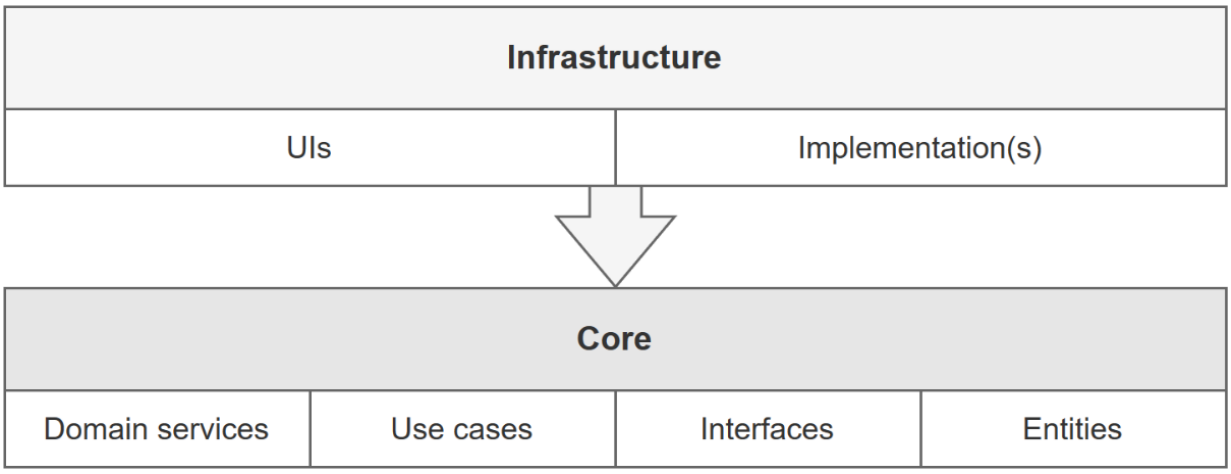


Figure 14.16: A two-layer view of Clean Architecture's common elements

Now, let's revisit our layered application using Clean Architecture, starting with the **core layer**. The core project contains the domain model, the use cases (services), and the interfaces needed to fulfill those use cases. We must not access external resources in this layer: no database calls, disk access, or HTTP requests. This layer contains the interfaces that expose such interaction, but the implementations live in the **infrastructure layer**.The presentation layer was renamed `Web` and lives in the outer layer with the EF Core implementation. The `Web` project depends only on the `Core` project. Once again, since the composition root is in this project, it must load the EF Core implementation project to configure the IoC container.Here is a diagram representing the relation between the shared model and the new Clean Architecture project structure:

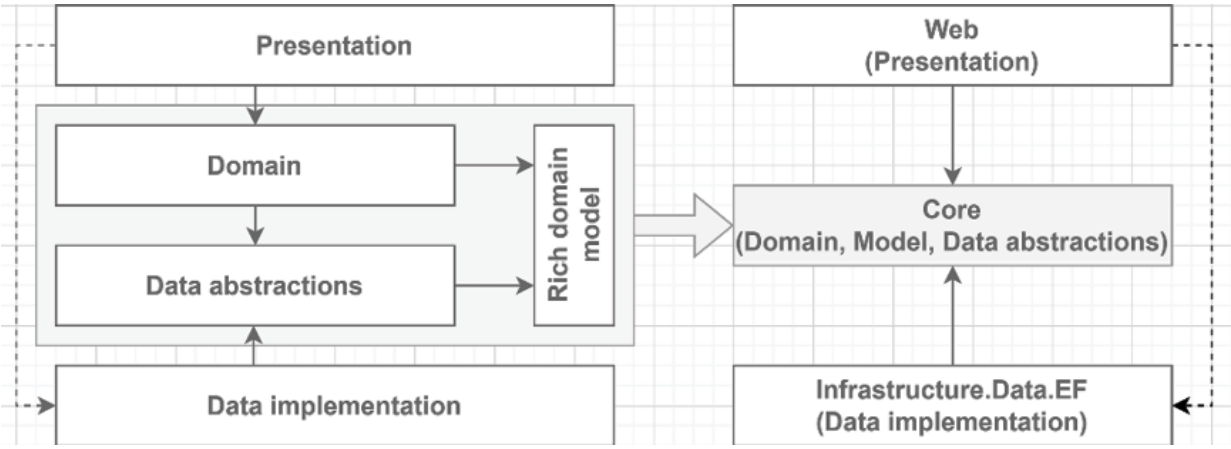


Figure 14.17: From shared project to the Clean Architecture project structure

In the preceding diagram, we took the center of the classic layered solution and merged the layers into a single `Core` project.

Here's the link to this project on GitHub: https://adpg.link/rT1P.

Most of the code is not that relevant since, once again, the most significant aspect is the dependency flow and relationships between projects. Nonetheless, here is a list of changes that I made aside from moving the pieces to different projects:

- I removed the `ProductService` class and `IProductService` interface and used the `IProductRepository` interface directly from the `StockService` class (`Core` project) and the `/products` endpoint (`Web` project: `Program.cs`).

- I removed the `IStockService` interface, and now both the add and remove stocks endpoints (`Web` project: `Program.cs`) depend directly on the `StockService` class.

Why use the `IProductRepository` interface directly, you might wonder? Since the `Web` project (**infrastructure layer**) depends on the **core layer**, we can leverage the inward dependency flow. It is acceptable to use a repository directly as long as the feature has no business logic. Programming empty shells and pass-through services adds useless complexity. However, when business logic starts to be involved, create a service or any other domain entity you deem necessary for that scenario. Don't pack business logic into your controllers or minimal API delegates.I removed the `IStockService` interface since the `StockService` class contains concrete business rules that can be consumed as is from the infrastructure layer. I know we have emphasized using interfaces since the beginning of the book, but I also often said that principles are not laws. All in all, there is nothing to abstract away: if the business rules change, the old ones won't be needed anymore. On the other hand, we could have kept the interface.To wrap this up, Clean Architecture is a proven pattern for building applications that is fundamentally an evolution of layering. Many variants can help you manage use cases, entities, and infrastructure; however, we will not cover those here. There are many open-source projects to start with Clean Architecture if you seek organizational guidance.

> I left a few links in the *Further reading* section.

If you think this is a great fit for you, your team, your project, or your organization, feel free to dig deeper and adopt this pattern. In subsequent chapters, we explore some patterns, such as CQRS, Publish-Subscribe, and feature-based design, which we can combine with Clean Architecture to add flexibility and robustness. These become particularly useful as your system grows in size and complexity.

Implementing layering in real life

Now that we covered all of this, it is important to note that on the one hand, there is the theory, and on the other, life is hitting you in the face. Suppose you are working in a big enterprise. In that case, chances are your employer can pour hundreds of thousands or even millions of dollars into a feature to run experiments, spend months designing every little piece, and ensure everything is perfect. Even then, is achieving perfection even possible? Probably not.For companies that don't have that type of capital, you must build entire products for a few thousand dollars sometimes because they are not trying to resell them but just need that tool built. That is where your architectural skills come in handy. How do you design the least-worst product in a maintainable fashion while meeting stakeholders' expectations? The most important part of the answer is to set expectations upfront. Moreover, never forget that someone needs to maintain and change the software over time; no software does not evolve; there's always something.

> If you are in a position where you must evaluate the feasibility of products and features in this context, setting expectations lower can be a good way to plan for the unplannable. It is easier to overdeliver than justify why you underdelivered.

Let's dig deeper into this and look at a few tricks to help you out. Even if you are working for a larger enterprise, you should get something out of it.

To be or not to be a purist?

In your day-to-day work, you may not always need the rigidity of a **domain layer** to create a wall in front of your data. Maybe you just don't have the time or the money, or it's just not worth doing.Taking and presenting the data can often work well enough, especially for simple data-driven applications that are only a user interface over a database, as is the case for many internal tools.The answer to the *"To be or not to be a purist?"* question is: it depends!

> This section covers layering, but we explore other patterns that are feature-oriented, so I suggest you continue reading and explore using the techniques from *Chapter 17*, *Vertical Slice Architecture*, *Chapter 18*, *Request-EndPoint-Response (REPR)*, and *Chapter 20, Modular Monolith*, to improve your design while keeping the design overhead low.

Here are a few examples of things that the answer depends on, to help you out:

- The project; for example:
  - **Domain-heavy or logic-intensive projects** will benefit from a domain layer, helping you centralize parts for an augmented level of reusability and maintainability.
  - **Data management projects** tend to have less or no logic in them. We can often build them without adding a domain layer as the **domain** is often only a tunnel from the **presentation** to the **data**; a pass-through layer. We can often simplify those systems by dividing them into two layers: **data** and **presentation**.
- Your team; for example, a highly skilled team may tend to use advanced concepts and patterns more efficiently, and the learning curve for newcomers should be easier due to the number of seasoned engineers that can support them on the team. This does not mean that less skilled teams should aim lower; on the contrary, it may just be harder or take longer to start. Analyze each project individually and find the best patterns to drive them accordingly.
- Your boss; if the company you work for puts pressure on you and your team to deliver complex applications in record time and nobody tells your boss that it is impossible, you may need to cut corners a lot and enjoy many maintenance headaches with crashing systems, painful deployments, and more. That being said, if it is inevitable for these types of projects, I'd go with a very simple design that does not aim at reusability—aim at low-to-average testability and code stuff that just works.
- Your budget; once again, this often depends on the people selling the application and the features. I saw promises that were impossible to keep but delivered anyway with a lot of effort, extra hours, and corner-cutting. The thing to remember when going down that path is that at some point, there is no return from the accumulated **technical debt**, and it will just get worse (this applies to all budgets).
- The audience; the people who use the software can make a big difference to how you build it: ask them. For example, suppose you are building a tool for your fellow developers. In that case, you can cut corners that you would not for less technically skilled users (like delivering a CLI tool instead of a full-blown user interface). On the other hand, if you're aiming your application at multiple clients (web, mobile, and so on), isolating your application's components and focusing on reusability could be a winning design.
- The expected quality; you should not tackle the problem in the same way for building a prototype and a SaaS application. It is acceptable, even encouraged, for a prototype to have no tests and not follow best practices, but I'd recommend the opposite for a production-quality application.
- Any other things that life throws at you; yes, life is unpredictable, and no one can cover every possible scenario in a book, so just keep the following in mind when building your next piece of software:
  - Do not over-engineer your applications.
  - Only implement features that you need, not more, as per the **you aren't gonna need it** (**YAGNI**) principle.
  - Use your judgment and take the less-worst options; there is no perfect solution.

I hope you found this guidance helpful and that it will serve you in your career.

Building a façade over a database

Data-driven programs are a type of software that I often see in smaller enterprises. Those companies need to support their day-to-day operations with computers, not the other way around. Every company needs internal tools, and many needed them yesterday.The reason is simple; every company is unique. Because it's unique, due to its business model, leadership, or employees, it also needs unique tools to help with its day-to-day operations. Those small tools are often simple user interfaces over a database, controlling access to that data. In these cases, you don't need over-engineered solutions, as long as everyone is informed that the tool will not evolve beyond what it is: a small tool.In real life, this one is tough to explain to non-programmers because they tend to see complex use cases as easy to implement and simple use cases as hard to implement. It's normal; they just don't know, and we all don't know something. In these scenarios, a big part of our job is also to educate people. Advising decision-makers about the differences in quality between a small tool and a large business application. Educating and working with stakeholders makes them aware of the situation and make decisions with you, leading to higher project quality that meets everyone's expectations. This can also reduce the "*it's not my fault*" syndrome from both sides.I've found that immersing customers and decision-makers in the decision process and having them follow the development cycle helps them understand the reality behind the programs and helps both sides stay happy and

grow more satisfied. Stakeholders not getting what they want is no better than you being super stressed over unreachable deadlines.That said, our educational role does not end with decision-makers. Teaching new tools and techniques to your peers is also a major way to improve your team, peers, and yourself. Explaining concepts is not always as easy as it sounds.Nevertheless, data-driven programs may be hard to avoid, especially if you are working for SMEs, so try to get the best out of it. Nowadays, with low-code and no-code solutions and all the open-source libraries, you might be able to save yourself a lot of this kind of trouble, but maybe not all.

> Remember that someday, someone must maintain those small tools. Think of that person as you, and think about how you'd like some guidelines or documentation to help you. I'm not saying to over-document projects, as documentation often gets out of sync with the code and becomes more of a problem than a solution. However, a simple `README.md` file at the project's root explaining how to build and run the program and some general guidelines could be beneficial. Always think about documentation as if you were the one reading it. Most people don't like to spend hours reading documentation to understand something simple, so keep it simple.

When building a *façade over a database*, you want to keep it simple. Also, you should make it clear that it should not evolve past that role. One way to build this would be to use EF Core as your data layer and scaffold an MVC application as your presentation layer, shielding your database. You can use the built-in ASP.NET Core authentication and authorization mechanism if you need access control. You can then choose role-based or policy-based access control or any other way that makes sense for your tool and allows you to control access to the data the way you need to.

> Keeping it simple should help you build more tools in less time, making everyone happy.

From a layering standpoint, using my previous example, you will end up having two layers sharing the data model:

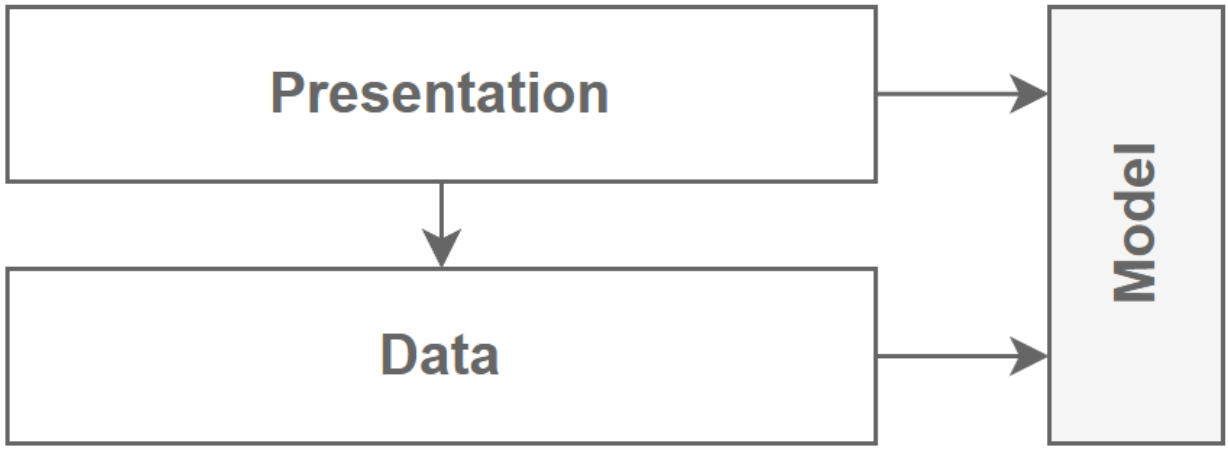


Figure 14.18: A façade-like presentation layer over a database application's design

Nothing stops you from creating a **view model** here and there for more complex views, but the key is to keep the logic's complexity to a minimum. Otherwise, you may discover the hard way that sometimes, rewriting a program from scratch takes less time than trying to fix it. Moreover, nothing stops you from using any other presentation tools and components available to you.Using this data-driven architecture as a temporary application while the main application is in development is also a good solution. It takes a fraction of the time to build, and the users have access to it immediately. You can even get feedback from it, which allows you to fix any mistakes before they are implemented in the real (future) application, working like a living prototype.

> A good database design in these sorts of applications can go a long way.

Not all projects are that simple, but still, many are; the key is to make the program good enough while ensuring you cut the right corners. The presentation layer in these types of applications could leverage a low-code solution such as Power Apps, for example.

Summary

Layering is one of the most used architectural techniques when it comes to designing applications. An application is often split into multiple different layers, each managing a single responsibility. The three most popular layers are **presentation**, **domain**, and **data**. You are not limited to three layers; you can split each into smaller layers (or smaller pieces inside the same conceptual layer), leading to composable, manageable, and maintainable applications.Moreover, you can create abstraction layers to invert the flow of dependency and separate interfaces from implementations, as we saw in the *Abstract layers* section. You can persist the domain entities directly or create an independent model for the data layer. You can also use an anemic model (no logic or method) or a rich model (packed with entity-related logic). You can share that model between multiple layers or have each layer possess its own.Out of layering was born Clean Architecture, which guides organizing your application into concentric layers, often dividing the application into use cases.Let's see how this approach can help us move toward the **SOLID** principles at app scale:

- **S**: Layering leads us toward splitting responsibilities horizontally, with each layer oriented around a single macro-concern. The main goal of layering is responsibility segregation.
- **O**: Abstract layers enable consumers to act differently (change behaviors) based on the provided implementation (concrete layer).
- **L**: N/A
- **I**: Splitting layers based on features (or cohesive groups of features) is a way of segregating a system into smaller blocks (interfaces).
- **D**: Abstraction layers lead directly to the dependency flow's inversion, while classic layering leads in the opposite direction.

In the next chapter, we learn how to centralize the logic of copying objects (models) using object mappers and an open-source tool to help us skip the implementation, also known as productive laziness.

Questions

Let's take a look at a few practice questions:

1. When creating a layered application, is it true that we must have presentation, domain, and data layers?
2. Is a rich domain model better than an anemic domain model?
3. Does EF Core implement the Repository and Unit of Work patterns?
4. Do we need to use an ORM in the data layer?
5. Can a layer in Clean Architecture access any inward layers?

Further reading

Here are a few links to help you build on what we learned in this chapter:

- **Dapper** is a simple yet powerful ORM for .NET, made by the people of Stack Overflow. If you like writing SQL, but don't like mapping data to objects, this ORM might be for you: https://adpg.link/pTYs.
- An article that I wrote in 2017, talking about the Repository pattern; that is, « Design Patterns: ASP.NET Core Web API, services, and repositories | Part 5: Repositories, the ClanRepository, and integration testing »: https://adpg.link/D53Z.
- Entity Framework Core – Using Transactions: https://adpg.link/gxwD.
- Here are resources about Clean Architecture:
  - Common web application architectures (Microsoft Learn): https://adpg.link/Pnpn
  - Microsoft eShopOnWeb ASP.NET Core Reference Application: https://adpg.link/dsw1
  - GitHub—Clean Architecture (Ardalis/Steve Smith)—Solution templates: https://adpg.link/tpPi
  - GitHub—Clean Architecture (Jason Taylor)—Solution templates: https://adpg.link/jxX2

Answers

1. No, you can have as many layers as you need and name and organize them as you want.
2. No, both have their place, their pros, and their cons.
3. Yes. A `DbContext` is an implementation of the Unit of Work pattern. `DbSet<T>` is an implementation of the Repository pattern.
4. No, you can query any system in any way you want. For example, you could use ADO.NET to query a relational database, manually create the objects using a `DataReader`, track changes using a `DataSet`, or do anything else that fits your needs. Nonetheless, ORMs can be very convenient.
5. Yes. A layer can never access outward layers, only inward ones.

15 Object Mappers, Aggregate Services, and Façade

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



In this chapter, we explore object mapping. As we saw in the previous chapter, working with layers often leads to copying models from one layer to another. Object mappers solve that problem. We first look at manually implementing an object mapper. Then, we improve our design by regrouping the mappers under a mapper service, exploring the Aggregate Services, and Mapping façade patterns along the way. Finally, we replace that manual work with two open-source tools that helps us generate business value instead of writing mapping code.In this chapter, we cover the following topics:

- Overview of object mapping and object mappers
- Implementing a simple object mapper
- Exploring the too-many-dependencies code smell
- Exploring the Aggregate Services pattern
- Implementing a Mapping Façade by leveraging the Façade pattern
- Using the Service Locator pattern to create a flexible Mapping Service in front of our mappers
- Using AutoMapper to map an object to another, replacing our homebrewed code
- Using Mapperly instead of AutoMapper

Object mapper

What is object mapping? In a nutshell, it is the action of copying the value of an object's properties into the properties of another object. But sometimes, properties' names do not match; an object hierarchy may need to be flattened and transformed. As we saw in the previous chapter, each layer can own its own model, which can be a good thing, but that comes at the price of copying objects from one layer to another. We can also share models between layers, but even then, we usually need to map one object into another. Even if it's just to map your models to **Data Transfer Objects** (**DTOs**), it is almost inevitable.

> Remember that DTOs define our API's contract. Independent contract classes help maintain the system, making us choose when to modify them. If you skip that part, each time you change your model, it automatically updates your endpoint's contract, possibly breaking some clients. Moreover, if you input your model directly, a malicious user could try to bind the values of properties that they should not, leading to potential security issues (known as **over-posting** or **over-posting attacks**). Having good data exchange contracts is one of the keys to designing robust systems.

In the previous projects, the mapping logic was hardcoded, sometimes duplicated and adding additional responsibilities to the class doing the mapping. In this chapter, we extract the mapping logic into object mappers to fix that issue.

Goal

The object mapper's goal is to copy the value of an object's properties into the properties of another object. It encapsulates the mapping logic away from where the mapping takes place. The mapper is also responsible for transforming the values from the original format to the destination format when both objects do not follow the same structure.

Design

We can design object mappers in many ways. Here is the most basic object mapper design:

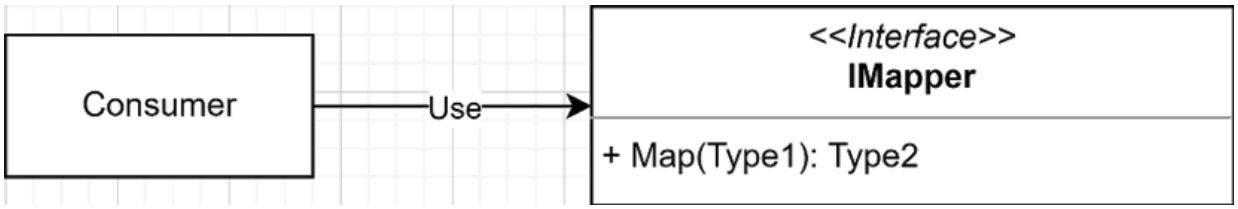


Figure 15.1: Basic design of the object mapper

In the diagram, the **Consumer** uses the `IMapper` interface to map an object of `Type1` to an object of `Type2`. That's not very reusable, but it illustrates the concept. By using the power of **generics**, we can upgrade that simple design to this more reusable version:

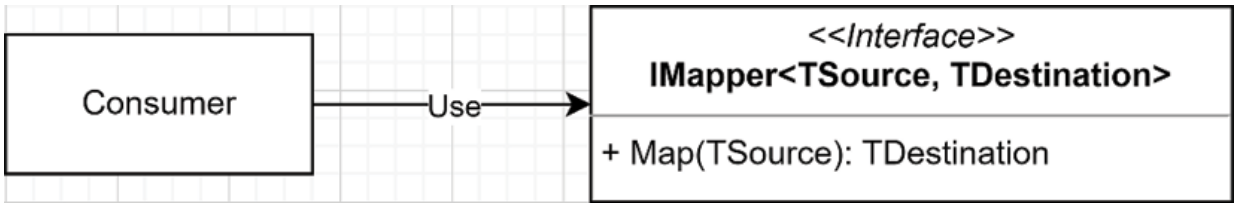


Figure 15.2: Generic object mapper design

This design lets us map any `TSource` to any `TDestination` by implementing the `IMapper<TSource, TDestination>` interface once per mapping rule. One class could also implement multiple mapping rules. For example, we could implement the mapping of `Type1` to `Type2` and `Type2` to `Type1` in the same class (a bidirectional mapper).Another way would be to use the following design and create an `IMapper` interface with a single method that handles all of the application's mapping:

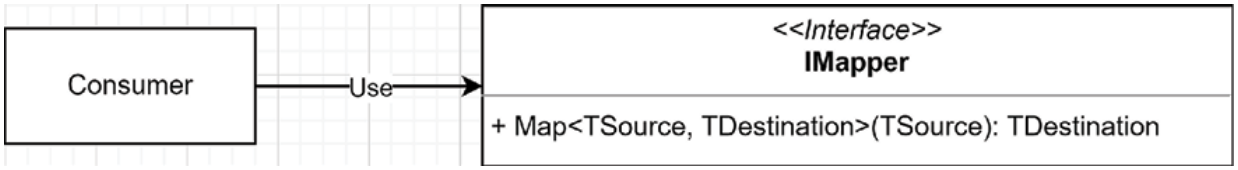


Figure 15.3: Object mapping using a single IMapper as the entry point

The biggest advantage of this last design is the ease of use. We always inject a single `IMapper` instead of one `IMapper<TSource, TDestination>` per type of mapping, which reduces the number of dependencies and the complexity of consuming such a mapper.You can implement object mapping in any way your imagination allows, but the critical part is that the mapper is responsible for mapping an object to another. A mapper should avoid complex processes, such as loading data from a database and whatnot. It should copy the values of one object into another: that's it. Think about the **Single Responsibility Principle** (**SRP**) here: the class must have a single reason to change, and since it's an object mapper, that reason should be object mapping.Let's jump into some code to explore the designs in more depth with each project.

Project – Mapper

This project is an updated version of the Clean Architecture code from the previous chapter. The project aims to demonstrate the design's versatility of encapsulating entity mapping logic into mapper classes, moving that logic away from the consumers. Of course, the project is again focused on the use case at hand, making learning the topics easier.First, we need an interface that resides in the `Core` project so the other projects can implement the mapping they need. Let's adopt the second design that we saw:

```
namespace Core.Mappers;
public interface IMapper<TSource, TDestination>
{
    TDestination Map(TSource entity);
}
```

With this interface, we can start by creating the data mappers. But first, let's start by creating record classes instead of anonymous types to name the DTOs returned by the endpoints. Here are all the DTOs (from the `Program.cs` file):

```
// Input stock DTOs
public record class AddStocksCommand(int Amount);
public record class RemoveStocksCommand(int Amount);
// Output stock DTO
public record class StockLevel(int QuantityInStock);
// Output "read all products" DTO
public record class ProductDetails(int Id, string Name, int QuantityInStock);
// Output Exceptions DTO
public record class ProductNotFound(int ProductId, string Message);
public record class NotEnoughStock(int AmountToRemove, int QuantityInStock, string Message);
```

Three of the four output DTOs need mapping:

- `Product` to `ProductDetails`
- `ProductNotFoundException` to `ProductNotFound`
- `NotEnoughStockException` to `NotEnoughStock`

Why not map the `StockLevel` DTO? In our case, the `StockService` returns an `int` when we add or remove stocks, so converting a primitive value like an `int` into a `StockLevel` object does not require an object mapper. Moreover, creating such an object mapper adds no value and makes the code more complex. If the service had returned an object, creating a mapper that maps an object to `StockLevel` would have made more sense.

Let's start with the product mapper (from the `Program.cs` file):

```
public class ProductMapper : IMapper<Product, ProductDetails>
{
    public ProductDetails Map(Product entity)
        => new(entity.Id ?? default, entity.Name, entity.QuantityInStock);
}
```

The preceding code is straightforward; the `ProductMapper` class implements the `IMapper<Product, ProductDetails>` interface. The `Map` method returns a `ProductDetails` instance. The highlighted code ensures the `Id` property is not `null`, which should not happen. We could also add a guard clause to ensure the `Id` property is not `null`.All in all, the `Map` method takes a `Product` as input and outputs a `ProductDetails` instance containing the same values.Then let's continue with the exception mappers (from the `Program.cs` file):

```
public class ExceptionsMapper : IMapper<ProductNotFoundException, ProductNotFound>, IMapper<NotEn
{
    public ProductNotFound Map(ProductNotFoundException exception)
        => new(exception.ProductId, exception.Message);
    public NotEnoughStock Map(NotEnoughStockException exception)
        => new(exception.AmountToRemove, exception.QuantityInStock, exception.Message);
}
```

Compared to the `ProductMapper` class, the `ExceptionsMapper` class implements the two remaining use cases by implementing the `IMapper` interface twice. The two `Map` methods handle mapping an exception to its DTO,

leading to one class being responsible for mapping exceptions to DTOs.Let's look at the `products` endpoint (original value from the `clean-architecture` project of *Chapter 14*, *Layering and Clean Architecture*):

```
app.MapGet("/products", async (
    IProductRepository productRepository,
    CancellationToken cancellationToken) =>
{
    var products = await productRepository.AllAsync(cancellationToken);
    return products.Select(p => new
    {
        p.Id,
        p.Name,
        p.QuantityInStock
    });
});
```

Before analyzing the code, let's look at the updated version (from the `Program.cs` file):

```
app.MapGet("/products", async (
    IProductRepository productRepository,
    IMapper<Product, ProductDetails> mapper,
    CancellationToken cancellationToken) =>
{
    var products = await productRepository.AllAsync(cancellationToken);
    return products.Select(p => mapper.Map(p));
});
```

In the preceding code, the request delegate uses the mapper to replace the copy logic (the highlighted lines of the original code). That simplifies the handler, moving the mapping responsibility into mapper objects instead (highlighted in the preceding code)—one more step toward the SRP.Let's skip the add stocks endpoint since it is very similar to the remove stocks endpoint but simpler, and let's focus on the later (original value from the `clean-architecture` project of *Chapter 14*, *Layering and Clean Architecture*):

```
app.MapPost("/products/{productId:int}/remove-stocks", async (
    int productId,
    RemoveStocksCommand command,
    StockService stockService,
    CancellationToken cancellationToken) =>
{
    try
    {
        var quantityInStock = await stockService.RemoveStockAsync(productId, command.Amount, can
        var stockLevel = new StockLevel(quantityInStock);
        return Results.Ok(stockLevel);
    }
    catch (NotEnoughStockException ex)
    {
        return Results.Conflict(new
        {
            ex.Message,
            ex.AmountToRemove,
            ex.QuantityInStock
        });
    }
    catch (ProductNotFoundException ex)
    {
        return Results.NotFound(new
        {
            ex.Message,
            productId,
        });
    }
});
```

Once again, before analyzing the code, let's look at the updated version (from the `Program.cs` file):

```
app.MapPost("/products/{productId:int}/remove-stocks", async (
    int productId,
```

```
    RemoveStocksCommand command,
    StockService stockService,
    IMapper<ProductNotFoundException, ProductNotFound> notFoundMapper,
    IMapper<NotEnoughStockException, NotEnoughStock> notEnoughStockMapper,
    CancellationToken cancellationToken) =>
{
    try
    {
        var quantityInStock = await stockService.RemoveStockAsync(productId, command.Amount, can
        var stockLevel = new StockLevel(quantityInStock);
        return Results.Ok(stockLevel);
    }
    catch (NotEnoughStockException ex)
    {
        return Results.Conflict(notEnoughStockMapper.Map(ex));
    }
    catch (ProductNotFoundException ex)
    {
        return Results.NotFound(notFoundMapper.Map(ex));
    }
});
```

The same thing happened for this request delegate, but we injected two mappers instead of just one. We moved the mapping logic from inline using an anonymous type to the mapper objects. Nevertheless, a code smell is emerging here; can you smell it? We will investigate this after we are done with this project; meanwhile, keep thinking about the number of injected dependencies.Now that the delegates depend on interfaces with object mappers encapsulating the mapping responsibility, we must configure the composition root and bind the mapper implementations to the `IMapper<TSource, TDestination>` interface. The service bindings look like this:

```
.AddSingleton<IMapper<Product, ProductDetails>, ProductMapper>()
.AddSingleton<IMapper<ProductNotFoundException, ProductNotFound>, ExceptionsMapper>()
.AddSingleton<IMapper<NotEnoughStockException, NotEnoughStock>, ExceptionsMapper>()
```

Since `ExceptionsMapper` implements two interfaces, we bind both to that class. That is one of the beauties of abstractions; the remove stocks delegate asks for two mappers but receives an instance of `ExceptionsMapper` twice without even knowing it.We could also register the classes so the same instance is injected twice, like this:

```
.AddSingleton<ExceptionsMapper>()
.AddSingleton<IMapper<ProductNotFoundException, ProductNotFound>, ExceptionsMapper>(sp => sp.GetR
.AddSingleton<IMapper<NotEnoughStockException, NotEnoughStock>, ExceptionsMapper>(sp => sp.GetRe
```

> Yes, I did that double registration of the same class on purpose. That proves we can compose an application as we want without impacting the **consumers**. That is done by depending on abstractions instead of implementations, as per the **Dependency Inversion Principle** (**DIP**—the “D” in SOLID). Moreover, the division into small interfaces, as per the **Interface Segregation Principle** (**ISP**—the “I” in SOLID), makes that kind of scenario possible. Finally, we can glue all those pieces together using the power of **Dependency Injection** (**DI**).

Conclusion

Before exploring more alternatives, let’s see how object mapping can help us follow the **SOLID** principles:

- **S**: Using mapper objects helps to separate the responsibility of mapping types from the consumers, making it easier to maintain and refactor the mapping logic.
- **O**: By injecting mappers, we can change the mapping logic without changing the code of their consumers.
- **L**: N/A
- **I**: We explored different designs that provide a small mapper interface that reduces the dependencies between the components.
- **D**: The consumers depend only on abstractions, moving the implementation’s binding to the composition root and inverting the dependency flow.

Now that we’ve explored how to extract and use mappers, let’s look at the code smell that emerged.

Code smell – Too many dependencies

Using this kind of mapping could become tedious in the long run, and we would rapidly see scenarios such as injecting three or more mappers into a single request delegate or controller. The consumer would likely already have other dependencies, leading to four or more.That should raise the following flag:

- Does the class do too much and have too many responsibilities?

In this case, the fine-grained `IMapper` interface pollutes our request delegates with tons of dependencies on mappers, which is not ideal and makes our code harder to read.

> The preferred solution would be to move the exception-handling responsibility away from the delegates or controllers, leveraging a middleware or an exception filter, for example. This tactic would move boilerplate code away from the endpoints.
>
> > Since we are talking about mappers, we will explore more object mapping concepts to help us with this problem instead, which applies to many use cases.

As a rule of thumb, you want to **limit the number of dependencies to three or less**. Over that number, ask yourself if there is a problem with that class; does it have too many responsibilities? Having more than three dependencies is not inherently wrong; it just indicates that you should investigate the design. If nothing is wrong, keep it at 4 or 5, or 10; it does not matter. If you find a way to reduce the number of dependencies or that the class actually does too many things refactor the code.If you don't like having that many dependencies, you could extract service aggregates that encapsulate two or more dependencies and inject that aggregate instead. Beware that moving your dependencies around does not fix anything; it just moves the problem elsewhere if there was a problem in the first place. Using aggregates could increase the readability of the code, though.

> Instead of blindly moving dependencies around, analyze the problem to see if you could create classes with actual logic that could do something useful to reduce the number of dependencies.

Next, let's have a quick look at aggregating services.

Pattern – Aggregate Services

Even if the Aggregate Services pattern is not a magic problem-solving pattern, it is a viable alternative to injecting too many dependencies into another class. Its goal is to aggregate many dependencies in a class to reduce the number of injected services in other classes, grouping dependencies together. The way to manage aggregates would be to group them by concern or responsibility. Putting a bunch of services in another service just for the sake of it is rarely the way to go; aim for cohesion.

> Creating one or more aggregation services that expose other services can be a way to implement service discovery in a project. Like always, analyze if the problem is not elsewhere first. Loading a service that exposes other services can be handy. However, this may create issues, so don't put everything into an aggregate firsthand either.

Here is an example of a mapping aggregate to reduce the number of dependencies of a **Create-Read-Update-Delete** (**CRUD**) controller that allows the creation, updating, deletion, and reading of one, many, or all products. Here's the aggregate service code and a usage example:

```
public interface IProductMappers
{
    IMapper<Product, ProductDetails> EntityToDto { get; }
    IMapper<InsertProduct, Product> InsertDtoToEntity { get; }
    IMapper<UpdateProduct, Product> UpdateDtoToEntity { get; }
}
public class ProductMappers : IProductMappers
{
    public ProductMappers(IMapper<Product, ProductDetails> entityToDto, IMapper<InsertProduct, P
```

```
    {
        EntityToDto = entityToDto ?? throw new ArgumentNullException(nameof(entityToDto));
        InsertDtoToEntity = insertDtoToEntity ?? throw new ArgumentNullException(nameof(insertDt
        UpdateDtoToEntity = updateDtoToEntity ?? throw new ArgumentNullException(nameof(updateDt
    }
    public IMapper<Product, ProductDetails> EntityToDto { get; }
    public IMapper<InsertProduct, Product> InsertDtoToEntity { get; }
    public IMapper<UpdateProduct, Product> UpdateDtoToEntity { get; }
}
public class ProductsController : ControllerBase
{
    private readonly IProductMappers _mapper;
    // Constructor injection, other methods, routing attributes, ...
    public ProductDetails GetProductById(int id)
    {
        Product product = ...; // Fetch a product by id
        ProductDetails dto = _mapper.EntityToDto.Map(product);
        return dto;
    }
}
```

The `IProductMappers` aggregate could be logical in this example as it groups the mappers used by the `ProductsController` class under its umbrella. It is responsible for mapping `ProductsController`-related domain objects to DTOs and vice versa while the controller gives up this responsibility.You can create aggregate services with anything, not just mappers. That's a fairly common pattern in DI-heavy applications (which can also point to some design flaws).Now that we've explored the Aggregate Services pattern, let's explore how to make a mapping façade instead.

Pattern – Mapping Façade

We studied façades already; here, we explore another way to organize our many mappers by leveraging that design pattern.Instead of what we just did, we will create a mapping façade to replace our aggregate services. The code consuming the façade will be more elegant because it uses the `Map` methods directly instead of the properties. The responsibility of the façade is the same as the aggregate, but it implements the interfaces instead of exposing them as properties.Let's look at the code:

```
public interface IProductMapperService :
    IMapper<Product, ProductDetails>,
    IMapper<InsertProduct, Product>,
    IMapper<UpdateProduct, Product>
{
}
public class ProductMapperService : IProductMapperService
{
    private readonly IMapper<Product, ProductDetails> _entityToDto;
    private readonly IMapper<InsertProduct, Product> _insertDtoToEntity;
    private readonly IMapper<UpdateProduct, Product> _updateDtoToEntity;
    // Omitted constructor injection code
    public ProductDetails Map(Product entity)
    {
        return _entityToDto.Map(entity);
    }
    public Product Map(InsertProduct dto)
    {
        return _insertDtoToEntity.Map(dto);
    }
    public Product Map(UpdateProduct dto)
    {
        return _updateDtoToEntity.Map(dto);
    }
}
```

In the preceding code, the `ProductMapperService` class implements the `IMapper` interfaces through the `IProductMapperService` interface and delegates the mapping logic to each injected mapper: a façade wrapping multiple individual mappers. Next, we look at the `ProductsController` that consumes the façade:

```
public class ProductsController : ControllerBase
{
    private readonly IProductMapperService _mapper;
    // Omitted constructor injection, other methods, routing attributes, ...
    public ProductDetails GetProductById(int id)
    {
        Product product = ...; // Fetch a product by id
        ProductDetails dto = _mapper.Map(product);
        return dto;
    }
}
```

From the consumer standpoint (the `ProductsController` class), I find it cleaner to write `_mapper.Map(...)` instead of `_mapper.SomeMapper.Map(...)`. The consumer does not want to know what mapper is doing what mapping; it only wants to map what needs mapping. If we compare the Mapping Façade with the Aggregate Services of the previous example, the façade takes the responsibility of choosing the mapper and moves it away from the consumer. This design distributes the responsibilities between the classes better.This was an excellent opportunity to review the Façade design pattern. Nonetheless, now that we've gone through multiple mapping options and examined the issue of having too many dependencies, it's time to move forward on our object mapping adventure with an enhanced version of our mapping façade.

Project – Mapping service

The goal is to simplify the implementation of the Mapper façade with a universal interface. To achieve this, we are implementing the diagram shown in *Figure 13.3*. Here's a reminder:

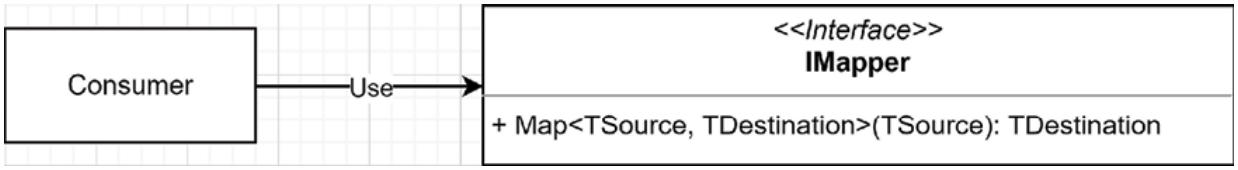


Figure 15.4: Object mapping using a single IMapper interface

Instead of naming the interface `IMapper`, we will use the name `IMappingService`. This name is more suitable because it is not mapping anything; it is a dispatcher servicing the mapping request to the right mapper. Let's take a look:

```
namespace Core.Mappers;
public interface IMappingService
{
    TDestination Map<TSource, TDestination>(TSource entity);
}
```

That interface is self-explanatory; it maps any `TSource` to any `TDestination`.On the implementation side, we are leveraging the **Service Locator** pattern, so I called the class `ServiceLocatorMappingService`:

```
namespace Core.Mappers;
public class ServiceLocatorMappingService : IMappingService
{
    private readonly IServiceProvider _serviceProvider;
    public ServiceLocatorMappingService(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider ?? throw new ArgumentNullException(nameof(serviceProv
    }
    public TDestination Map<TSource, TDestination>(TSource entity)
    {
        var mapper = _serviceProvider.GetService<IMapper<TSource, TDestination>>();
        if (mapper == null)
        {
            throw new MapperNotFoundException(typeof(TSource), typeof(TDestination));
        }
        return mapper.Map(entity);
```

```
    }
}
```

The logic is simple:

- Find the appropriate `IMapper<TSource, TDestination>` service, then map the entity with it
- If you don't find any, throw a `MapperNotFoundException`

The key to that design is to register the mappers with the IoC container instead of the service itself. Then we use the mappers without knowing every single one of them, like in the previous example. The `ServiceLocatorMappingService` class doesn't know any mappers; it just dynamically asks for one whenever needed.

> The Service Locator pattern should not be part of the application's code. However, it can be helpful at times. For example, we are not trying to cheat DI in this case. On the contrary, we are leveraging its power.
>
> > Using a service locator is wrong when acquiring dependencies in a way that removes the possibility of controlling the program's composition from the composition root, which breaks the IoC principle.
> >
> > In this case, we load mappers dynamically from the IoC container, limiting the container's ability to control what to inject which is acceptable since it has little to no negative impact on the program's maintainability, flexibility, and reliability. For example, we can replace the `ServiceLocatorMappingService` implementation with another class without affecting the `IMappingService` interface consumers.

Now, we can inject that service everywhere we need mapping and use it directly. We already registered the mappers, so we only need to bind the `IMappingService` to its `ServiceLocatorMappingService` implementation and update the consumers. Here's the DI binding:

```
.AddSingleton<IMappingService, ServiceLocatorMappingService>();
```

If we look at the new implementation of the remove stocks endpoint, we can see we reduced the number of mapper dependencies to one:

```
app.MapPost("/products/{productId:int}/remove-stocks", async (
    int productId,
    RemoveStocksCommand command,
    StockService stockService,
    IMappingService mapper,
    CancellationToken cancellationToken) => {
    try
    {
        var quantityInStock = await stockService.RemoveStockAsync(productId, command.Amount, can
        var stockLevel = new StockLevel(quantityInStock);
        return Results.Ok(stockLevel);
    }
    catch (NotEnoughStockException ex)
    {
        return Results.Conflict(mapper.Map<NotEnoughStockException, NotEnoughStock>(ex));
    }
    catch (ProductNotFoundException ex)
    {
        return Results.NotFound(mapper.Map<ProductNotFoundException, ProductNotFound>(ex));
    }
});
```

The preceding code is similar to the previous sample, but we replaced the mappers with the new service (the highlighted lines). And that's it; we now have a universal mapping service that delegates the mapping to any mapper we register with the IoC container.

> Even if you are not likely to implement object mappers manually often, exploring and revisiting those patterns and a code smell is very good and will help you craft better software.

This is not the end of our object mapping exploration. We have two tools to explore, starting with AutoMapper, which does all the object mapping work for us.

Project – AutoMapper

We just covered different ways to implement object mapping, but here we leverage an open-source tool named AutoMapper that does it for us instead of implementing our own.Why bother learning all of that if a tool already does it? There are a few reasons to do so:

- It is important to understand the concepts; you don't always need a full-fledged tool like AutoMapper.
- It allowed us to cover multiple patterns we can use in other contexts and apply them to components with different responsibilities. So, all in all, you should have learned multiple new techniques during this object mapping progression.
- Lastly, we dug deeper into applying the SOLID principles to write better programs.

The AutoMapper project is also a copy of the Clean Architecture sample. The biggest difference between this project and the others is that we don't need to define any interface because AutoMapper exposes an `IMapper` interface with all the methods we need and more.To install AutoMapper, you can install the `AutoMapper` NuGet package using the CLI (`dotnet add package AutoMapper`), Visual Studio's NuGet package manager, or by updating your `.csproj` manually.The best way to define our mappers is by using AutoMapper's profile mechanism. A profile is a simple class that inherits from `AutoMapper.Profile` and contains maps from one object to another. We use profiles to group mappers together, but in our case, with only three maps, I decided to create a single `WebProfile` class.Finally, instead of manually registering our profiles, we can scan one or more assemblies to load all of the profiles into AutoMapper by using the `AutoMapper.Extensions.Microsoft.DependencyInjection` package.

> When installing the `AutoMapper.Extensions.Microsoft.DependencyInjection` package you don't have to load the `AutoMapper` package.

There is more to AutoMapper than what we cover here, but it has enough resources online, including the official documentation, to help you dig deeper into the tool. The goal of this project is to do basic object mapping.In the *Web* project, we must create the following maps:

- Map `Product` to `ProductDetails`
- Map `NotEnoughStockException` to `NotEnoughStock`
- Map `ProductNotFoundException` to `ProductNotFound`

To do that, we create the following `WebProfile` class (in the `Program.cs` file, but could live anywhere):

```
using AutoMapper;
public class WebProfile : Profile
{
    public WebProfile()
    {
        CreateMap<Product, ProductDetails>();
        CreateMap<NotEnoughStockException, NotEnoughStock>();
        CreateMap<ProductNotFoundException, ProductNotFound>();
    }
}
```

A profile in AutoMapper is nothing more than a class where you create maps in the constructor. The `Profile` class adds the required methods for you to do that, such as the `CreateMap` method. What does that do?Invoking the method `CreateMap<Product, ProductDetails>()` tells AutoMapper to register a mapper that maps `Product` to `ProductDetails` . The other two `CreateMap` calls are doing the same for the other two maps. That's all we need for now because AutoMapper maps properties using conventions, and both our model and DTO classes have the same sets of properties with the same names.

> In the preceding examples, we defined some mappers in the `Core` layer. In this example, we rely on a library, so it is even more important to consider the dependency flow. We are mapping objects only in the `Web` layer,

so there is no need to put the dependency on AutoMapper in the `Core` layer. Remember that all layers depend directly or indirectly on `Core`, so having a dependency on AutoMapper in that layer means all layers would also depend on it.

Therefore, in this example, we created the `WebProfile` class in the `Web` layer instead, limiting the dependency on AutoMapper to only that layer. Having only the `Web` layer depend on AutoMapper allows all outer layers (if we were to add more) to control how they do object mapping, giving more independence to each layer. It is also a best practice to limit object mapping as much as possible.

I've added a link to *AutoMapper Usage Guidelines* in the *Further reading* section at the end of the chapter.

Now that we have one profile, we need to register it with the IoC container, but we don't have to do this by hand; we can scan for profiles from the composition root by using one of the `AddAutoMapper` extension methods to scan one or more assemblies:

```
builder.Services.AddAutoMapper(typeof(WebProfile).Assembly);
```

The preceding method accepts a `params Assembly[] assemblies` argument, meaning we can pass multiple `Assembly` instances to it.

That `AddAutoMapper` extension method comes from the `AutoMapper.Extensions.Microsoft.DependencyInjection` package.

Since we have only one profile in one assembly, we leverage that class to access the assembly by passing the `typeof(WebProfile).Assembly` argument to the `AddAutoMapper` method. From there, AutoMapper scans for profiles in that assembly and finds the `WebProfile` class. If there were more than one, it would register all it finds.The beauty of scanning for types like this is that once you register AutoMapper with the IoC container, you can add profiles in any registered assemblies, and they get loaded automatically; there's no need to do anything else afterward but to write useful code. Scanning assemblies also encourages composition by convention, making it easier to maintain in the long run. The downside of assembly scanning is that debugging can be hard when something is not registered because the registration process is less explicit.Now that we've created and registered the profiles with the IoC container, it is time to use AutoMapper. Let's look at the three endpoints we created initially:

```
app.MapGet("/products", async (
    IProductRepository productRepository,
    IMapper mapper,
    CancellationToken cancellationToken) =>
{
    var products = await productRepository.AllAsync(cancellationToken);
    return products.Select(p => mapper.Map<Product, ProductDetails>(p));
});
app.MapPost("/products/{productId:int}/add-stocks", async (
    int productId,
    AddStocksCommand command,
    StockService stockService,
    IMapper mapper,
    CancellationToken cancellationToken) =>
{
    try
    {
        var quantityInStock = await stockService.AddStockAsync(productId, command.Amount, cancel
        var stockLevel = new StockLevel(quantityInStock);
        return Results.Ok(stockLevel);
    }
    catch (ProductNotFoundException ex)
    {
        return Results.NotFound(mapper.Map<ProductNotFound>(ex));
    }
});
app.MapPost("/products/{productId:int}/remove-stocks", async (
    int productId,
```

```
    RemoveStocksCommand command,
    StockService stockService,
    IMapper mapper,
    CancellationToken cancellationToken) =>
{
    try
    {
        var quantityInStock = await stockService.RemoveStockAsync(productId, command.Amount, can
        var stockLevel = new StockLevel(quantityInStock);
        return Results.Ok(stockLevel);
    }
    catch (NotEnoughStockException ex)
    {
        return Results.Conflict(mapper.Map<NotEnoughStock>(ex));
    }
    catch (ProductNotFoundException ex)
    {
        return Results.NotFound(mapper.Map<ProductNotFound>(ex));
    }
});
```

The preceding code shows how similar it is to use AutoMapper to the other options. We inject an `IMapper` interface, then use it to map the entities. Instead of explicitly specifying both `TSource` and `TDestination` like in the previous example, when using AutoMapper, we must specify only the `TDestination` generic parameter, reducing the code's complexity.

Suppose you are using AutoMapper on an `IQueryable` collection returned by EF Core. In that case, you should use the `ProjectTo` method, which limits the number of fields that EF will query to those you need. In our case, that changes nothing because we need the whole entity.

Here is an example that fetches all products from EF Core and projects them to `ProductDto` instances:

```
public IEnumerable<ProductDto> GetAllProducts()
{
    return _mapper.ProjectTo<ProductDto>(_db.Products);
}
```

Performance-wise, this is the recommended way to use AutoMapper with EF Core.

One last yet significant detail is that we can assert whether our mapper configurations are valid when the application starts. This does not identify missing mappers but validates that the registered ones are configured correctly. The recommended way of doing this is in a unit test. To make this happen, I made the autogenerated `Program` class public by adding the following line at the end:

```
public partial class Program { }
```

Then I created a test project named `Web.Tests` that contain the following code:

```
namespace Web;
public class StartupTest
{
    [Fact]
    public async Task AutoMapper_configuration_is_valid()
    {
        // Arrange
        await using var application = new AutoMapperAppWebApplication();
        var mapper = application.Services.GetRequiredService<IMapper>();
        mapper.ConfigurationProvider.AssertConfigurationIsValid();
    }
}
internal class AutoMapperAppWebApplication : WebApplicationFactory<Program>{}
```

In the preceding code, we validate that all the AutoMapper maps are valid. To make the test fail, you can uncomment the following line of the `WebProfile` class:

```
CreateMap<NotEnoughStockException, Product>();
```

The `AutoMapperAppWebApplication` class is there to centralize the initialization of the test cases when there is more than one.In the test project, I created a second test case ensuring the `products` endpoint is reachable. For both tests to work together, we must change the database name to avoid seeding conflicts so each test runs on its own database. This has to do with how we seed the database in the `Program.cs` file, which is not something we usually do except for development or proofs of concept. Nonetheless, testing against multiple databases can come in handy to isolate tests.Here's that second test case and updated `AutoMapperAppWebApplication` class to give you an idea:

```
public class StartupTest
{
    [Fact]
    public async Task The_products_endpoint_should_be_reachable()
    {
        await using var application = new AutoMapperAppWebApplication();
        using var client = application.CreateClient();
        using var response = await client.GetAsync("/products");
        response.EnsureSuccessStatusCode();
    }
    // Omitted AutoMapper_configuration_is_valid method
}
internal class AutoMapperAppWebApplication : WebApplicationFactory<Program>
{
    private readonly string _databaseName;
    public AutoMapperAppWebApplication([CallerMemberName]string? databaseName = default)
    {
        _databaseName = databaseName ?? nameof(AutoMapperAppWebApplication);
    }
    protected override IHost CreateHost(IHostBuilder builder)
    {
        builder.ConfigureServices(services =>
        {
            services.AddScoped(sp =>
            {
                return new DbContextOptionsBuilder<ProductContext>()
                    .UseInMemoryDatabase(_databaseName)
                    .UseApplicationServiceProvider(sp)
                    .Options;
            });
        });
        return base.CreateHost(builder);
    }
}
```

Running the tests ensures that the mapping in our application works and that one of the endpoints is reachable. We could add more tests, but those two cover about 50% of our code.

> The `CallerMemberNameAttribute` used in the preceding code is part of the `System.Runtime.CompilerServices` namespace and allows its decorated member to access the name of the method that called it. In this case, the `databaseName` parameter receives the test method name.

And this closes the AutoMapper project. At this point, you should begin to be familiar with object mapping. I'd recommend you evaluate whether AutoMapper is the right tool for the job whenever a project needs object mapping. You can always load another tool or implement your own mapping logic if AutoMapper does not suit your needs. If too much mapping is done at too many levels, maybe another application architecture pattern would be better, or some rethinking is in order.AutoMapper is convention-based and does a lot on its own without any configuration from us. It is also configuration-based, caching the conversions to improve performance. We can also create **type converters**, **value resolvers**, **value converters**, and more. AutoMapper keeps us away from writing that boring mapping code.Yet, AutoMapper is old, feature complete, and is almost unavoidable due to the number of projects that uses it. However, it is not the fastest, which is why we are exploring Mapperly next.

Project – Mapperly

Mapperly is a newer object mapper library that leverages source generation to make it lightning-fast. To get started, we must add a dependency on the `Riok.Mapperly` NuGet package.

Source generators were introduced with .NET 5, allowing developers to generate C# code during compilation.

There are many ways to create object mappers with Mapperly and many options to adjust the mapping process. The following code sample is similar to the others but using Mapperly. We cover the following ways to use Mapperly:

- Injecting a mapper class.
- Using a static method.
- Using an extension method.

Let's start with the injected mapper. First, the class must be `partial` for the source generator to extend it (that is how source generators work). Decorate the class with the `[Mapper]` attribute (highlighted). Then, in that partial class, we must create one or more `partial` methods that have the signature of the mappers we want to create (like the `MapToProductDetails` method), like this:

```
[Mapper]
public partial class ProductMapper
{
    public partial ProductDetails MapToProductDetails(Product product);
}
```

Upon compilation, the code generator creates the following class (I formatted the code to make it easier to read):

```
public partial class ProductMapper
{
    public partial ProductDetails MapToProductDetails(Product product)
    {
        var target = new ProductDetails(
            product.Id ?? throw new ArgumentNullException(nameof(product.Id)),
            product.Name,
            product.QuantityInStock
        );
        return target;
    }
}
```

Mapperly writes the boilerplate code for us in a generated `partial` class, which is why it is so fast.To use the mapper, we must register it with the IoC Container and inject it into our endpoint. Let's make it a singleton once again:

```
builder.Services.AddSingleton<ProductMapper>();
```

Then, we can inject and use it like this:

```
app.MapGet("/products", async (
    IProductRepository productRepository,
    ProductMapper mapper,
    CancellationToken cancellationToken) =>
{
    var products = await productRepository.AllAsync(cancellationToken);
    return products.Select(p => mapper.MapToProductDetails(p));
});
```

The highlighted code in the preceding block shows we can use our mapper like any other class. The biggest drawback is that we may end up injecting many mappers into a single class or endpoint if we do not consider how we create them wisely.Moreover, we must register all of our mappers with the IoC container, which creates a lot of boilerplate code but makes the process explicit. On the other hand, we could scan the assembly for all classes decorated with the `[Mapper]` attribute.If you want an abstraction layer like an interface for your mapper, you must design that yourself because Mapperly only generates the mappers. Here is an example:

```
public interface IMapper
{
    NotEnoughStock MapToDto(NotEnoughStockException source);
    ProductNotFound MapToDto(ProductNotFoundException source);
    ProductDetails MapToProductDetails(Product product);
}
[Mapper]
public partial class Mapper : IMapper
{
    public partial NotEnoughStock MapToDto(NotEnoughStockException source);
    public partial ProductNotFound MapToDto(ProductNotFoundException source);
    public partial ProductDetails MapToProductDetails(Product product);
}
```

The preceding code centralizes all the mapper methods under the same class and interface, allowing you to inject an interface similar to AutoMapper. In subsequent chapters, we explore ways to organize mappers and app code that does not involve creating a central mapper class.

To inspect the generated code, you can add the `EmitCompilerGeneratedFiles` property in a `PropertyGroup` tag inside your project file and set its value to `true` like this:

```
<PropertyGroup>
    <EmitCompilerGeneratedFiles>true</EmitCompilerGeneratedFiles>
</PropertyGroup>
```

Then the generated C# files will be available under the `obj\Debug\net8.0\generated` directory. Change the `net8.0` subdirectory to the SDK version and `Debug` by your configuration.

Next, we explore how to make a static mapper, which follows a very similar process, but we must make both the class and the method `static` like this:

```
[Mapper]
public static partial class ExceptionMapper
{
    public static partial ProductNotFound Map(ProductNotFoundException exception);
}
```

Mapperly takes the preceding code and generates the following (formatted for improved readability):

```
public static partial class ExceptionMapper
{
    public static partial ProductNotFound Map(ProductNotFoundException exception)
    {
        var target = new ProductNotFound(
            exception.ProductId,
            exception.Message
        );
        return target;
    }
}
```

Once again, the code generator writes the boilerplate code. The difference is that we don’t have to inject any dependency since it is a static method. We can use it this way (I only included the catch block, the rest of the code is unchanged):

```
catch (ProductNotFoundException ex)
{
    return Results.NotFound(ExceptionMapper.Map(ex));
}
```

It is pretty straightforward but creates a strong bond between the generated class and its consumers. You can use those static methods if having a hard dependency on a static class is acceptable for your project.The last way to map objects we are exploring is very similar, but we create an extension method in the same class instead of just a static method. Here’s the new method:

```
public static partial NotEnoughStock ToDto(this NotEnoughStockException exception);
```

The generated code for that method looks like the following (formatted):

```
public static partial NotEnoughStock ToDto(this NotEnoughStockException exception)
{
    var target = new NotEnoughStock(
        exception.AmountToRemove,
        exception.QuantityInStock,
        exception.Message
    );
    return target;
}
```

The only difference is the addition of the `this` keyword, making a regular static method into an extension method that we can use like this:

```
catch (NotEnoughStockException ex)
{
    return Results.Conflict(ex.ToDto());
}
```

An extension method is more elegant than a static method, yet it creates a bond similar to the static method. Once again, choosing how you want to proceed with your mapping is up to you.One noteworthy thing about Mapperly is that its analyzers yield information, warnings, or errors when the mapping code is incorrect or potentially incorrect. The severity of the messages is configurable. For example, if we add the following method in the `ExceptionMapper` class, Mapperly yields the `RMG013` error:

```
public static partial Product NotEnoughStockExceptionToProduct(
    NotEnoughStockException exception
);
```

Error message:

```
RMG013 Core.Models.Product has no accessible constructor with mappable arguments
```

Moreover, the two exception mapper methods yield messages about properties that do not exist on the target class as information. Here's an example of such a message:

```
RMG020 The member TargetSite on the mapping source type Core.ProductNotFoundException is not mapp
```

With those in place, we know when something is or can be wrong, which safeguards us from misconfigurations.Let's wrap this chapter up.

Summary

Object mapping is an unavoidable reality in many cases. However, as we saw in this chapter, there are several ways of implementing object mapping, taking that responsibility away from the other components of our applications or simply coding it inline manually.At the same time, we took the opportunity to explore the Aggregate Services pattern, which gives us a way to centralize multiple dependencies into one, lowering the number of dependencies needed in other classes. That pattern can help with the too-many-dependencies code smell, which, as a rule of thumb, states that we should investigate objects with more than three dependencies for design flaws. When moving dependencies into an aggregate, ensure there is cohesion within the aggregate to avoid adding unnecessary complexity to your program and just moving the dependencies around.We also explored leveraging the Façade pattern to implement a mapping façade, which led to a more readable and elegant mapper.Afterward, we implemented a mapper service that mimicked the façade. Despite being less elegant in its usage, it was more flexible.We finally explored is AutoMapper and Mapperly, two open-source tools that do object mapping for us, offering us many options to configure the mapping of our objects. As we explored, just using the default convention of AutoMapper allowed us to eliminate all of our mapping code. On Mapperly's side, we had to define the mapper contracts using partial classes and methods to let its code generator implement the mapping code for us. You can choose from many existing object mapper libraries, AutoMapper being one of the oldest,

most famous, and hated at the same time, while Mapperly is one of the newest and fastest but yet in its infancy.Hopefully, as we are putting more and more pieces together, you are starting to see what I had in mind at the beginning of this book when stating this was an architectural journey.Now that we are done with object mapping, we explore the Mediator and CQRS patterns in the next chapter.

Questions

Let's take a look at a few practice questions:

1. Is it true that injecting an Aggregate Service instead of multiple services improves our system?
2. Is it true that using mappers helps us extract responsibilities from consumers to mapper classes?
3. Is it true that you should always use AutoMapper?
4. When using AutoMapper, should you encapsulate your mapping code into profiles?
5. How many dependencies should start to raise a flag telling you that you are injecting too many dependencies into a single class?

Further reading

Here are some links to build upon what we learned in the chapter:

- If you want more information on object mapping, I wrote an article about that in 2017, titled *Design Patterns: ASP.NET Core Web API, Services, and Repositories | Part 9: the NinjaMappingService and the Façade Pattern*: https://adpg.link/hxYf
- AutoMapper official website: https://adpg.link/5AUZ
- *AutoMapper Usage Guidelines* is an excellent do/don't list to help you do the right thing with AutoMapper, written by the library's author: https://adpg.link/tTKg
- Mapperly (GitHub): https://adpg.link/Dwcj

Answers

1. Yes, an Aggregate Service can improve a system, but not necessarily. Moving dependencies around does not fix design flaws; it just moves those flaws elsewhere.
2. Yes, mappers help us follow the SRP. However, they are not always needed.
3. No, it is not suitable for every scenario. For example, when the mapping logic becomes complex, consider not using AutoMapper. Too many mappers may also mean a flaw in the application design itself.
4. Yes, use profiles to organize your mapping rules cohesively.
5. Four or more dependencies should start to raise a flag. Once again, this is just a guideline; injecting four or more services into a class can be acceptable.

16 Mediator and CQRS Design Patterns

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter covers the building blocks of the next chapter, which is about **Vertical Slice Architecture**. We begin with a quick overview of Vertical Slice Architecture to give you an idea of the end goal. Then, we explore the **Mediator** design pattern, which plays the role of the middleman between the components of our application. That leads us to the **Command Query Responsibility Segregation** (**CQRS**) pattern, which describes how to divide our logic into commands and queries. Finally, we consolidate our learning by exploring MediatR, an open-source implementation of the Mediator design pattern, and send queries and commands through it to demonstrate how the concepts we have studied so far come to life in real-world application development.In this chapter, we cover the following topics:

- A high-level overview of Vertical Slice Architecture
- Implementing the Mediator pattern
- Implementing the CQS pattern
- Code smell – Marker Interfaces
- Using MediatR as a mediator

Let’s begin with the end goal.

A high-level overview of Vertical Slice Architecture

Before starting, let’s look at the end goal of this chapter and the next. This way, it should be easier to follow the progress toward that goal throughout the chapter.As we covered in *Chapter 14, Layering and Clean Architecture*, a layer groups classes together based on shared responsibilities. So, classes containing data access code are part of the data access layer (or infrastructure). People represent layers using horizontal slices in diagrams like this:

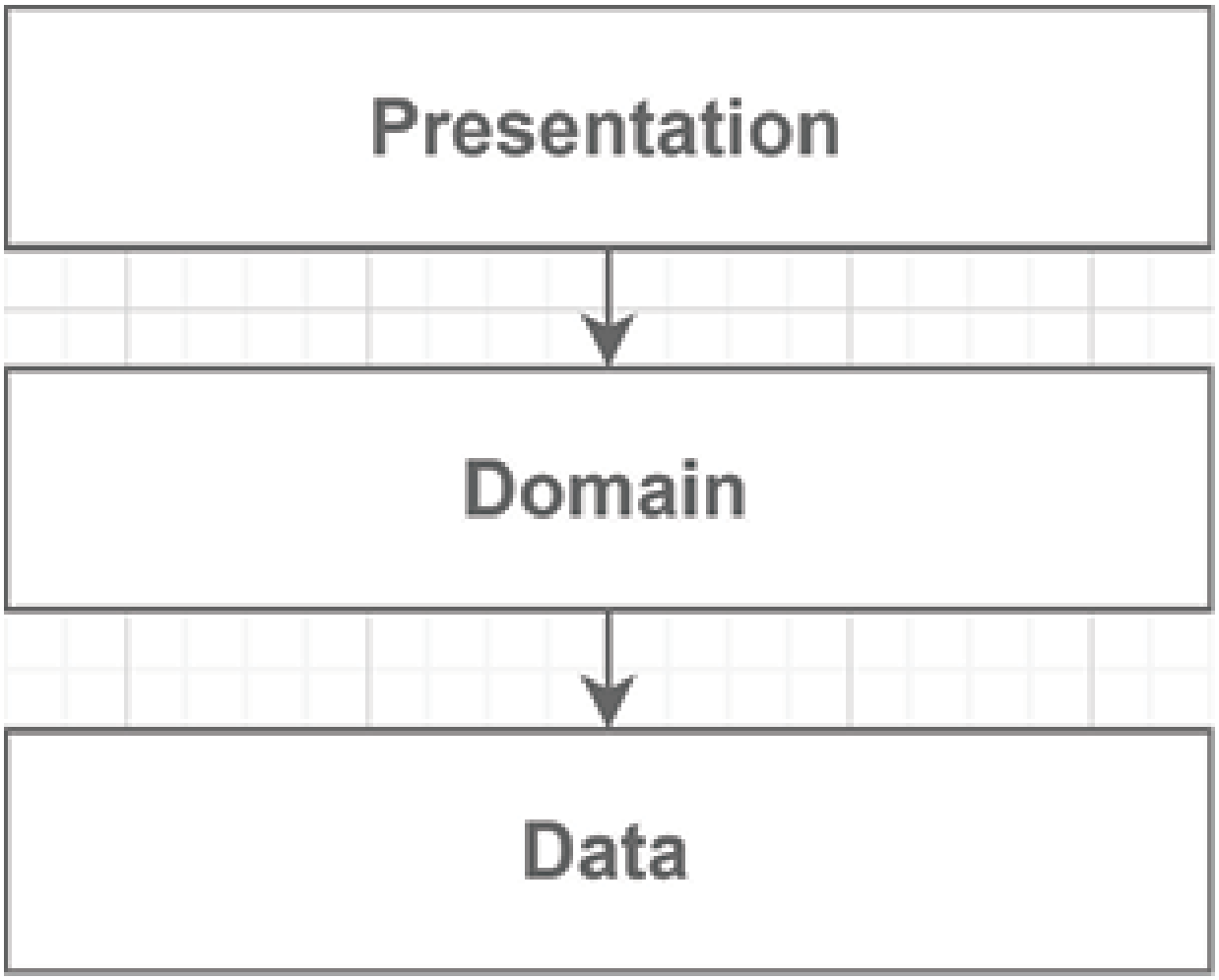


Figure 16.1: Diagram representing layers as horizontal slices

The “vertical slice” in “Vertical Slice Architecture” comes from that; a vertical slice represents the part of each layer that creates a specific feature. So, instead of dividing the application into layers, we divide it into features. A feature manages its data access code, domain logic, and possibly even presentation code. The key is to loosely couple the features from one another and keep each feature’s components close together. In a layered application, when we add, update, or remove a feature, we must change one or more layers, which too often translates to “all layers.”On the other hand, with vertical slices, we keep features isolated, allowing us to design them independently. From a layering perspective, this is like flipping your way of thinking about software to a 90° angle:

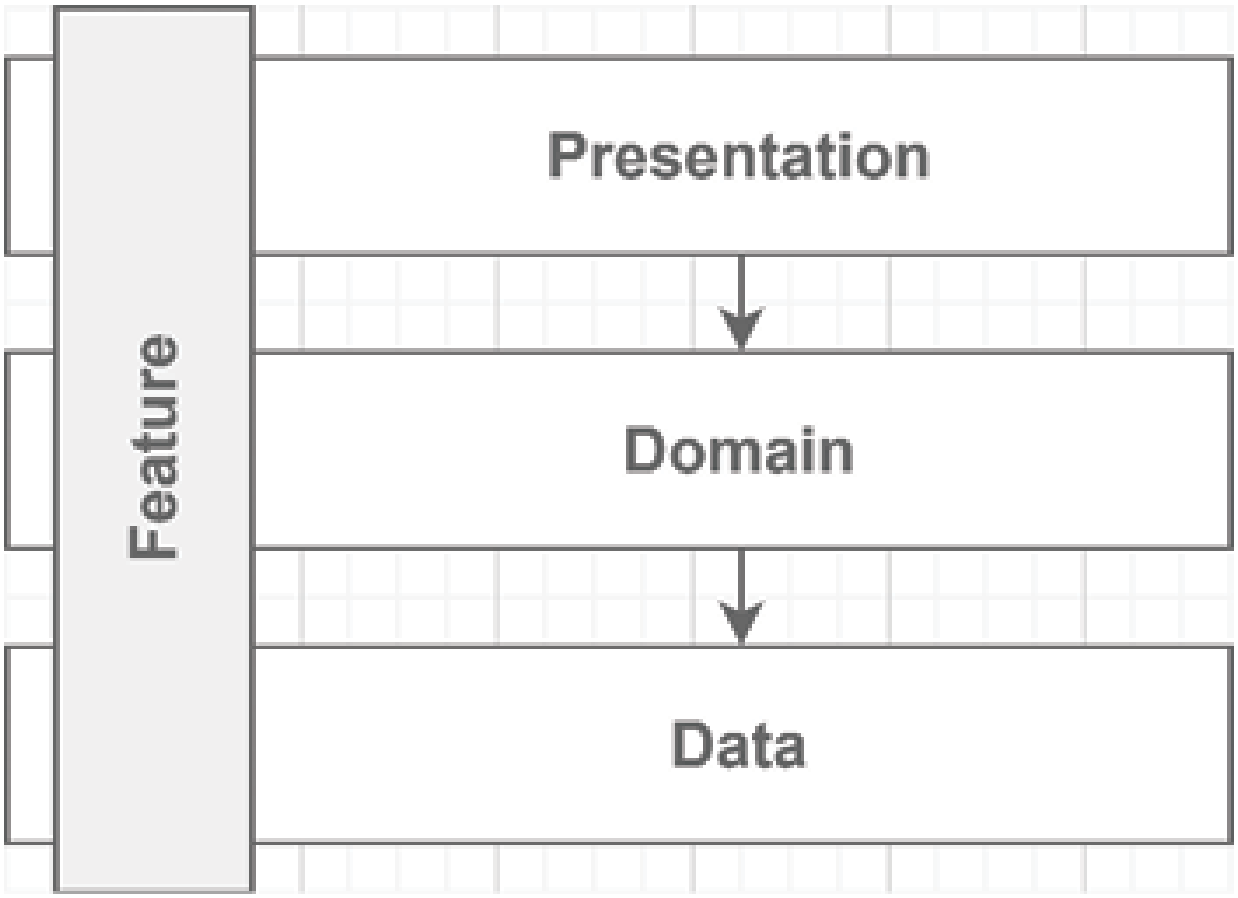


Figure 16.2: Diagram representing a vertical slice crossing all layers

Vertical Slice Architecture does not dictate the use of **CQRS**, the **Mediator** pattern, or **MediatR**, but these tools and patterns flow very well together, as we see in the next chapter. Nonetheless, these are just tools and patterns that you can use or change in your implementation using different techniques; it does not matter and does not change the concept.

> We explore additional ways of building feature-oriented applications in *Chapter 18, Request-EndPoint-Response (REPR)*, and *Chapter 20, Modular Monolith*.

The goal is to encapsulate features together, use CQRS to divide the application into requests (commands and queries), and use MediatR as the mediator of that CQRS pipeline, decoupling the pieces from one another.You now know the plan. We explore Vertical Slice Architecture later. Meanwhile, let’s begin with the Mediator design pattern.

Implementing the Mediator pattern

The **Mediator** pattern is another GoF design pattern that controls how objects interact with one another (making it a behavioral pattern).

Goal

The mediator’s role is to manage the communication between objects (colleagues). Those colleagues should not communicate together directly but use the mediator instead. The mediator helps break tight coupling between these colleagues.**A mediator is a middleman who relays messages between colleagues**.

Design

Let’s start with some UML diagrams. From a very high level, the Mediator pattern is composed of a mediator and colleagues:

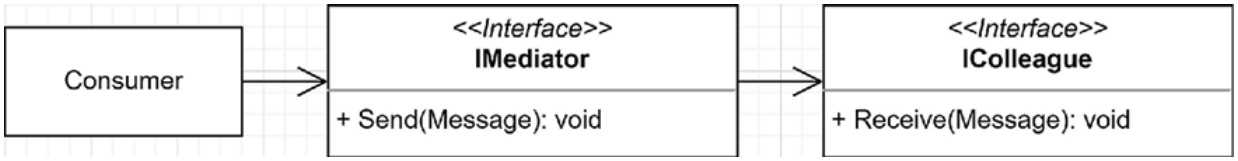


Figure 16.3: Class diagram representing the Mediator pattern

When an object in the system wants to send a message to one or more colleagues, it uses the mediator. Here is an example of how it works:

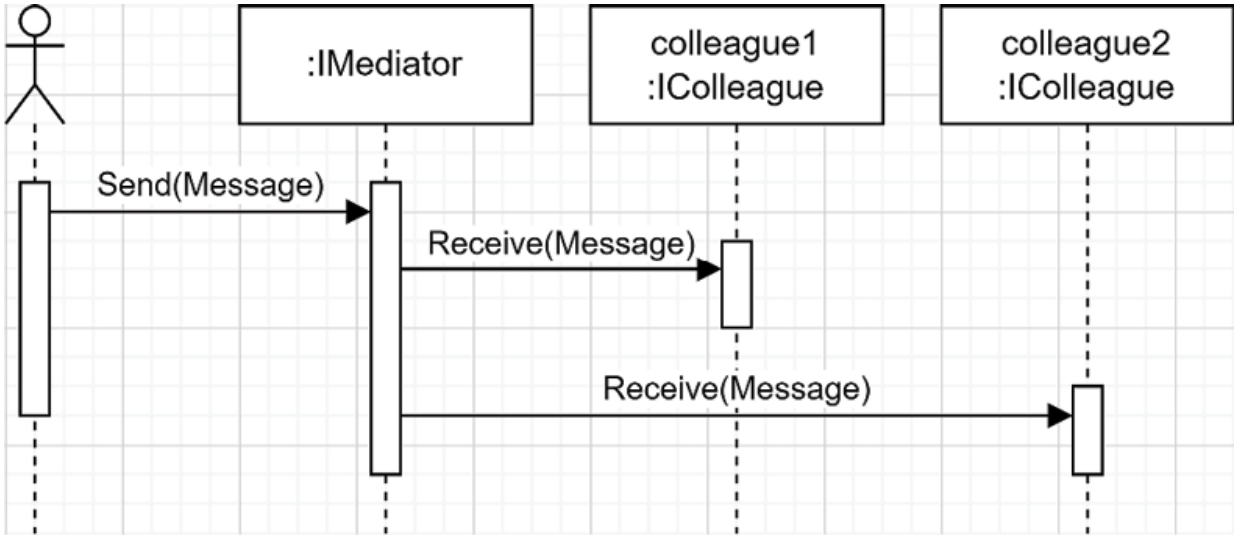


Figure 16.4: Sequence diagram of a mediator relaying messages to colleagues

That is also valid for colleagues; a colleague must also use the mediator if they need to talk to each other, as depicted in the following class diagram:

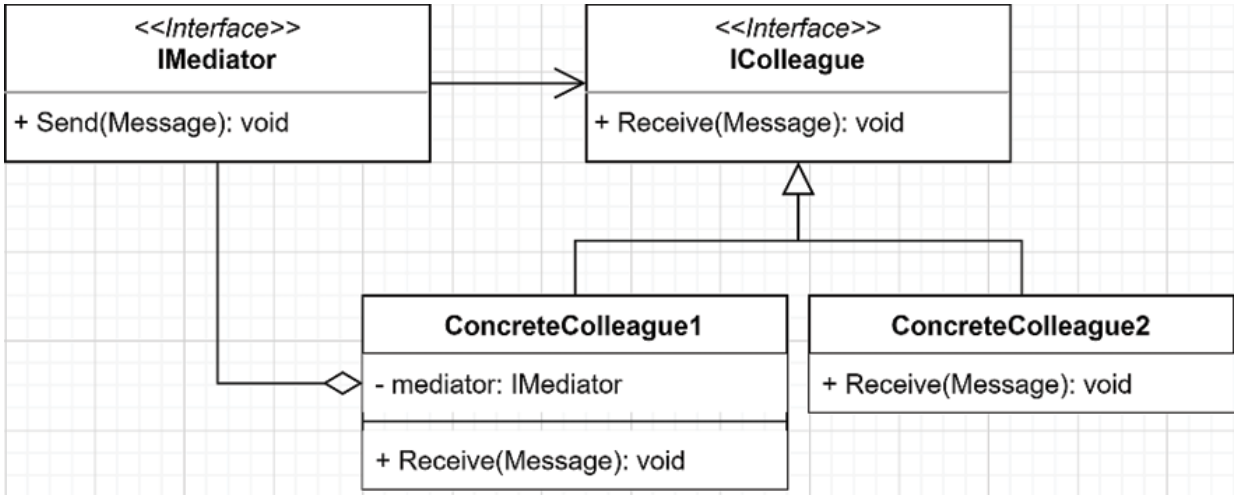


Figure 16.5: Class diagram representing the Mediator pattern including colleagues' collaboration

In this diagram, `ConcreteColleague1` is a colleague but also the consumer of the mediator. For example, that colleague could send a message to another colleague using the mediator, like this:

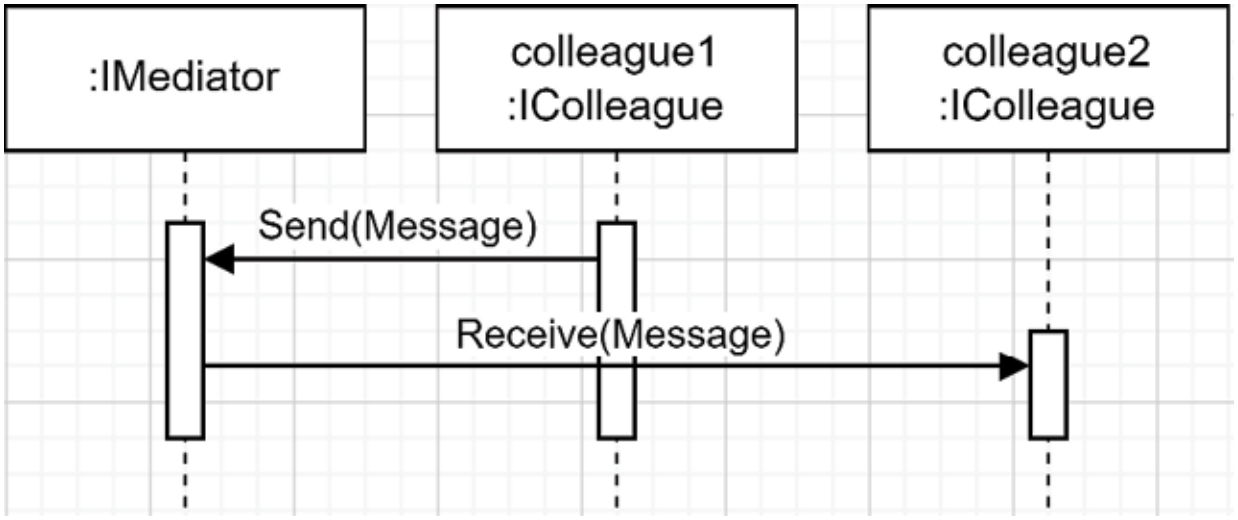


Figure 16.6: Sequence diagram representing colleague1 communicating with colleague2 through the mediator

From a mediator standpoint, its implementation most likely contains a collection of colleagues to communicate with, like this:

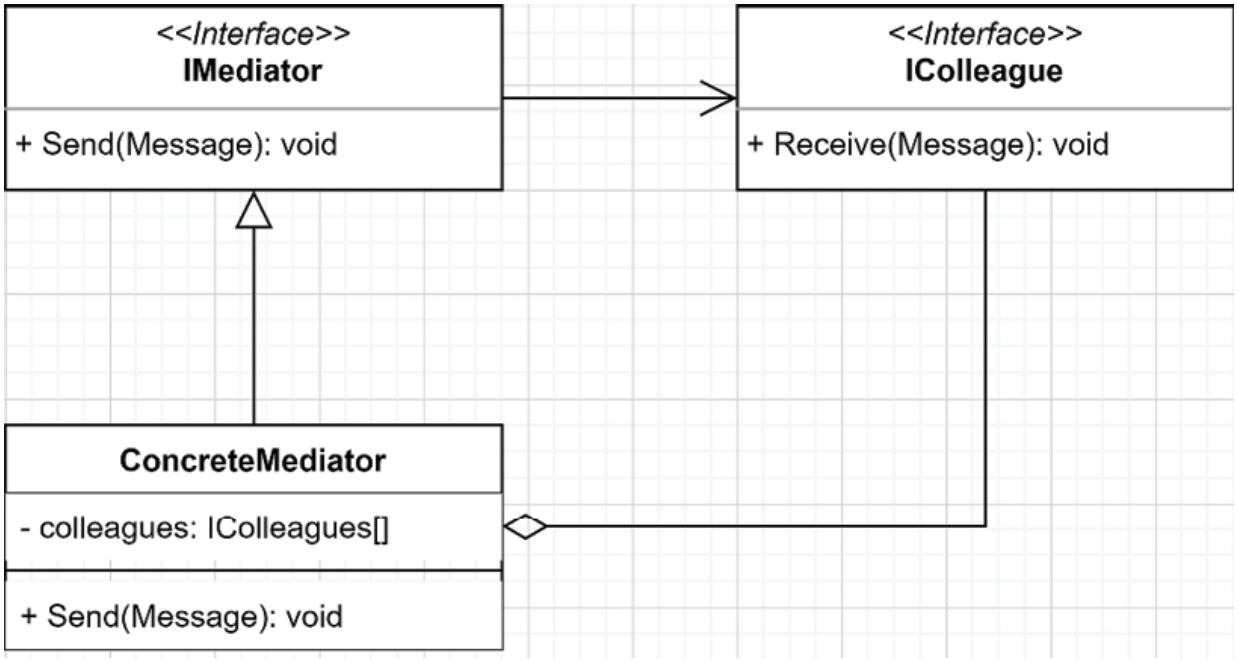


Figure 16.7: Class diagram representing a simple hypothetical concrete mediator implementation

Now that we have explored some UML diagrams, let's look at some code.

Project – Mediator (IMediator)

The Mediator project consists of a simplified chat system using the Mediator pattern. Let's start with the interfaces:

```
namespace Mediator;
public interface IMediator
{
    void Send(Message message);
}
public interface IColleague
{
    string Name { get; }
    void ReceiveMessage(Message message);
}
public record class Message(IColleague Sender, string Content);
```

The system is composed of the following:

- The `IMediator` interface represents a mediator that can send messages to colleagues.
- The `IColleague` interface represents a colleague that can receive messages. It also has a `Name` property so we can output meaningful values.
- The `Message` class represents a message sent by an `IColleague` implementation.

Next, we implement the `IMediator` interface in the `ConcreteMediator` class, which broadcasts the messages to all `IColleague` instances:

```
public class ConcreteMediator : IMediator
{
    private readonly List<IColleague> _colleagues;
    public ConcreteMediator(params IColleague[] colleagues)
    {
        ArgumentNullException.ThrowIfNull(colleagues);
        _colleagues = new List<IColleague>(colleagues);
    }
    public void Send(Message message)
    {
        foreach (var colleague in _colleagues)
        {
            colleague.ReceiveMessage(message);
        }
    }
}
```

That mediator is simple; it forwards all the messages it receives to every colleague it knows. The last part of the pattern is the `ConcreteColleague` class which lets an instance of the `IMessageWriter<TMessage>` interface output the messages (we explore that interface next):

```
public class ConcreteColleague : IColleague
{
    private readonly IMessageWriter<Message> _messageWriter;
    public ConcreteColleague(string name, IMessageWriter<Message> messageWriter)
    {
        Name = name ?? throw new ArgumentNullException(nameof(name));
        _messageWriter = messageWriter ?? throw new ArgumentNullException(nameof(messageWriter));
    }
    public string Name { get; }
    public void ReceiveMessage(Message message)
    {
        _messageWriter.Write(message);
    }
}
```

That class could hardly be simpler: it takes a name and an `IMessageWriter<TMessage>` implementation when created, then it stores a reference for future use.The `IMessageWriter<TMessage>` interface serves as a presenter and controls how the messages are displayed. The `IMessageWriter<TMessage>` interface is unrelated to the Mediator pattern. Nevertheless, it is a way to manage how a `ConcreteColleague` object outputs the messages without coupling it with a specific target. Here is the code:

```
namespace Mediator;
public interface IMessageWriter<Tmessage>
{
    void Write(Tmessage message);
}
```

The consumer of the system is an integration test defined in the `MediatorTest` class. The test uses the chat system and asserts the output using a custom implementation of the `IMessageWriter` interface. Let's start by analyzing the test:

```
namespace Mediator;
public class MediatorTest
{
    [Fact]
    public void Send_a_message_to_all_colleagues()
    {
        // Arrange
        var (millerWriter, miller) = CreateConcreteColleague("Miller");
        var (orazioWriter, orazio) = CreateConcreteColleague("Orazio");
        var (fletcherWriter, fletcher) = CreateConcreteColleague("Fletcher");
```

The test starts by defining three colleagues with their own `TestMessageWriter` implementation (names were randomly generated).

```
        var mediator = new ConcreteMediator(miller, orazio, fletcher);
        var expectedOutput = @"[Miller]: Hey everyone!
[Orazio]: What's up Miller?
[Fletcher]: Hey Miller!
";
```

In the second part of the preceding `Arrange` block, we create the subject under test (`mediator`) and register the three colleagues. At the end of that `Arrange` block, we also define the expected output of our test. It is important to note that we control the output from the `TestMessageWriter` implementation (defined at the end of the `MediatorTest` class). Next is the `Act` block:

```
        // Act
        mediator.Send(new Message(
            Sender: miller,
            Content: "Hey everyone!"
        ));
        mediator.Send(new Message(
            Sender: orazio,
            Content: "What's up Miller?"
        ));
        mediator.Send(new Message(
            Sender: fletcher,
            Content: "Hey Miller!"
        ));
```

In the preceding `Act` block, we send three messages through the `mediator` instance. Next is the `Assert` block:

```
        // Assert
        Assert.Equal(expectedOutput, millerWriter.Output.ToString());
        Assert.Equal(expectedOutput, orazioWriter.Output.ToString());
        Assert.Equal(expectedOutput, fletcherWriter.Output.ToString());
    }
```

In the `Assert` block, we ensure that all colleagues receive the messages.

```
    private static (TestMessageWriter, ConcreteColleague) CreateConcreteColleague(string name)
    {
        var messageWriter = new TestMessageWriter();
        var concreateColleague = new ConcreteColleague(name, messageWriter);
        return (messageWriter, concreateColleague);
    }
```

The `CreateConcreteColleague` method is a helper method that encapsulates the creation of the colleagues, enabling us to write the one-liner declaration used in the `Arrange` section of the test. Next, we look at the `IMessageWriter` implementation:

```
    private class TestMessageWriter : IMessageWriter<Message>
    {
        public StringBuilder Output { get; } = new StringBuilder();
        public void Write(Message message)
        {
            Output.AppendLine($"[{message.Sender.Name}]: {message.Content}");
        }
    }
} // Closing the MediatorTest class
```

Finally, the `TestMessageWriter` class writes the messages into `StringBuilder`, making it easy to assert the output. If we were to build a GUI for that, we could write an implementation of `IMessageWriter<Message>` that writes to that GUI; in the case of a web UI, it could use **SignalR** or write to the response stream directly, for example.To summarize the sample:

1. The consumer (the unit test) sends messages to colleagues through the mediator.
2. The `TestMessageWriter` class writes those messages to a `StringBuilder` instance. Each colleague has its own instance of the `TestMessageWriter` class.
3. The code asserts that all colleagues received the expected messages.

This example illustrates that the Mediator pattern allows us to break the direct coupling between colleagues. The messages reached colleagues without them knowing about each other.Colleagues should communicate through the mediator, so the Mediator pattern would not be complete without that. Let's implement a more advanced chatroom to tackle this concept.

Project – Mediator (IChatRoom)

In the previous code sample, we named the classes after the Mediator pattern actors, as shown in *Figure 14.7*. While this example is very similar, it uses domain-specific names instead and implements a few more methods to manage the system showing a more tangible implementation. Let's start with the abstractions:

```
namespace Mediator;
public interface IChatRoom
{
    void Join(IParticipant participant);
    void Send(ChatMessage message);
}
```

The `IChatRoom` interface is the mediator, and it defines two methods instead of one:

- `Join`, which allows `IParticipant` to join `IChatRoom`.
- `Send`, which sends a message to the others.

```
public interface IParticipant
{
    string Name { get; }
    void Send(string message);
    void ReceiveMessage(ChatMessage message);
    void ChatRoomJoined(IChatRoom chatRoom);
}
```

The `IParticipant` interface is the colleague and also has a few more methods:

- `Send`, to send messages.
- `ReceiveMessage`, to receive messages from the other `IParticipant` objects.
- `ChatRoomJoined`, to confirm that the `IParticipant` object has successfully joined a chatroom.

```
public record class ChatMessage(IParticipant Sender, string Content);
```

The `ChatMessage` class is the same as the previous `Message` class, but it references `IParticipant` instead of `IColleague`.Let's now look at the `IParticipant` implementation:

```
public class User : IParticipant
{
    private IChatRoom? _chatRoom;
    private readonly IMessageWriter<ChatMessage> _messageWriter;
    public User(IMessageWriter<ChatMessage> messageWriter, string name)
    {
        _messageWriter = messageWriter ?? throw new ArgumentNullException(nameof(messageWriter));
        Name = name ?? throw new ArgumentNullException(nameof(name));
    }
    public string Name { get; }
    public void ChatRoomJoined(IChatRoom chatRoom)
    {
        _chatRoom = chatRoom;
    }
    public void ReceiveMessage(ChatMessage message)
    {
        _messageWriter.Write(message);
    }
    public void Send(string message)
    {
        if (_chatRoom == null)
        {
            throw new ChatRoomNotJoinedException();
        }
        _chatRoom.Send(new ChatMessage(this, message));
    }
}
public class ChatRoomNotJoinedException : Exception
{
    public ChatRoomNotJoinedException()
        : base("You must join a chat room before sending a message.")
    { }
}
```

The `User` class represents our default `IParticipant`. A `User` instance can chat in only one `IChatRoom`. THe program can set the chat room by calling the `ChatRoomJoined` method. When it receives a message, it delegates it to its `IMessageWriter<ChatMessage>`. Finally, a `User` instance can send a message through the mediator (`IChatRoom`). The `Send` method throws a `ChatRoomNotJoinedException` to enforce that the `User` instance must join a chat room before sending messages (code-wise: the `_chatRoom` field must not be `null`).We could create a `Moderator`, `Administrator`, `SystemAlerts`, or any other `IParticipant` implementation as we see fit, but not in this sample. I am leaving that to you to experiment with the Mediator pattern.Now let's look at the `ChatRoom` class (the mediator):

```
public class ChatRoom : IChatRoom
{
    private readonly List<IParticipant> _participants = new();
    public void Join(IParticipant participant)
    {
        _participants.Add(participant);
        participant.ChatRoomJoined(this);
        Send(new ChatMessage(participant, "Has joined the channel"));
    }
    public void Send(ChatMessage message)
    {
        _participants.ForEach(participant
            => participant.ReceiveMessage(message));
    }
}
```

The `ChatRoom` class is slimmer than the `User` class. It allows participants to join and sends chat messages to registered participants. When joining a `ChatRoom`, it keeps a reference on the `IParticipant`, tells that `IParticipant` that it has successfully joined then sends a `ChatMessage` to all participants announcing the newcomer.With those few pieces, we have a Mediator implementation. Before moving to the next section, let's

look at the `Consumer` instance of `IChatRoom`, which is another integration test. Let's start with the skeleton of the class:

```
namespace Mediator;
public class ChatRoomTest
{
    [Fact]
    public void ChatRoom_participants_should_send_and_receive_messages()
    {
        // Arrange, Act, Assert blocks here
    }
    private (TestMessageWriter, User) CreateTestUser(string name)
    {
        var writer = new TestMessageWriter();
        var user = new User(writer, name);
        return (writer, user);
    }
    private class TestMessageWriter : IMessageWriter<ChatMessage>
    {
        public StringBuilder Output { get; } = new StringBuilder();
        public void Write(ChatMessage message)
        {
            Output.AppendLine($"[{message.Sender.Name}]: {message.Content}");
        }
    }
}
```

In the preceding code, we have the following pieces:

- The test case is an empty placeholder that we are about to look into.
- The `CreateTestUser` method helps simplify the `Arrange` section of the test case, similar to before.
- The `TestMessageWriter` implementation is similar to the previous example, accumulating messages in a `StringBuilder` instance.

As a reference, the `IMessageWriter` interface is the same as the previous project:

```
public interface IMessageWriter<TMessage>
{
    void Write(TMessage message);
}
```

Now, let's explore the test case, starting with the `Arrange` block, where we create four users with their respective `TestMessageWriter` instances (names were also randomly generated):

```
// Arrange
var (kingChat, king) = CreateTestUser("King");
var (kelleyChat, kelley) = CreateTestUser("Kelley");
var (daveenChat, daveen) = CreateTestUser("Daveen");
var (rutterChat, _) = CreateTestUser("Rutter");
var chatroom = new ChatRoom();
```

Then, in the `Act` block, our test users join the `chatroom` instance and send messages:

```
// Act
chatroom.Join(king);
chatroom.Join(kelley);
king.Send("Hey!");
kelley.Send("What's up King?");
chatroom.Join(daveen);
king.Send("Everything is great, I joined the CrazyChatRoom!");
daveen.Send("Hey King!");
king.Send("Hey Daveen");
```

Then in the Assert block, Rutter did not join the chatroom, so we expect no message:

```
// Assert
Assert.Empty(rutterChat.Output.ToString());
```

Since King is the first to join the channel, we expect him to receive all messages:

```
Assert.Equal(@"[King]: Has joined the channel
[Kelley]: Has joined the channel
[King]: Hey!
[Kelley]: What's up King?
[Daveen]: Has joined the channel
[King]: Everything is great, I joined the CrazyChatRoom!
[Daveen]: Hey King!
[King]: Hey Daveen
", kingChat.Output.ToString());
```

Kelley was the second user to join the chatroom, so the output contains almost all messages except the line saying `[King]: Has joined the channel:`

```
Assert.Equal(@"[Kelley]: Has joined the channel
[King]: Hey!
[Kelley]: What's up King?
[Daveen]: Has joined the channel
[King]: Everything is great, I joined the CrazyChatRoom!
[Daveen]: Hey King!
[King]: Hey Daveen
", kelleyChat.Output.ToString());
```

Daveen joined after King and Kelley exchanged a few words, so we expect the conversation to be shorter:

```
Assert.Equal(@"[Daveen]: Has joined the channel
[King]: Everything is great, I joined the CrazyChatRoom!
[Daveen]: Hey King!
[King]: Hey Daveen
", daveenChat.Output.ToString());
```

To summarize the test case, we have four users. Three of them joined the same chatroom at a different time and chatted a little. The output is different for everyone since the time you join matters. All participants are loosely coupled, thanks to the Mediator pattern, allowing us to extend the system without impacting the existing pieces. Leveraging the Mediator pattern helps us create maintainable systems; many small pieces are easier to manage and test than a large component handling all the logic.

Conclusion

As we explored in the two preceding projects, a **mediator** allows us to decouple the components of our system. **The mediator is the middleman between colleagues**, and it served us well in the small chatroom samples where each colleague can talk to the others without knowing how and without knowing them.Now let’s see how the Mediator pattern can help us follow the **SOLID** principles:

- **S**: The mediator extracts the communication responsibility from colleagues.
- **O**: With a mediator relaying the messages, we can create new colleagues and change the existing colleagues’ behaviors without impacting the others. If we need a new colleague, we can register one with the mediator.
- **L**: N/A
- **I**: The Mediator pattern divides the system into multiple small interfaces (`IMediator` and `IColleague`).
- **D**: All actors of the Mediator pattern solely depend on other interfaces. We can implement a new mediator and reuse the existing colleagues’ implementations if we need new mediation behavior because of the dependency inversion.

Next, we explore CQRS, which allows us to separate commands and queries, leading to a more maintainable application. After all, all operations are queries or commands, no matter how we call them.

Implementing the CQS pattern

Command-Query Separation (CQS) is a subset of the **Command Query Responsibility Segregation (CQRS)** pattern.Here’s the high-level difference between the two:

- With CQS, we divide operations into commands and queries.
- With CQRS, we apply the concept to the system level. We separate models for reading and for writing, potentially leading to a distributed system.

In this chapter, we stick with CQS and tackle CQRS in *Chapter 18*, *Introduction to Microservices Architecture*.

Goal

The goal is to divide all operations (or requests) into two categories: commands and queries.

- **A command mutates the state of an application.** For example, creating, updating, and deleting an entity are commands. In theory, a command should not return a value. In practice, they often do, especially for optimization purposes.
- **A query reads the state of the application but never changes it.** For example, reading an order, reading your order history, and retrieving your user profile are queries.

Dividing operations into mutator requests (write/command) and accessor requests (read/query) creates a clear separation of concerns, leading us toward the SRP.

Design

There is no definite design for this, but for us, the flow of a command should look like the following:

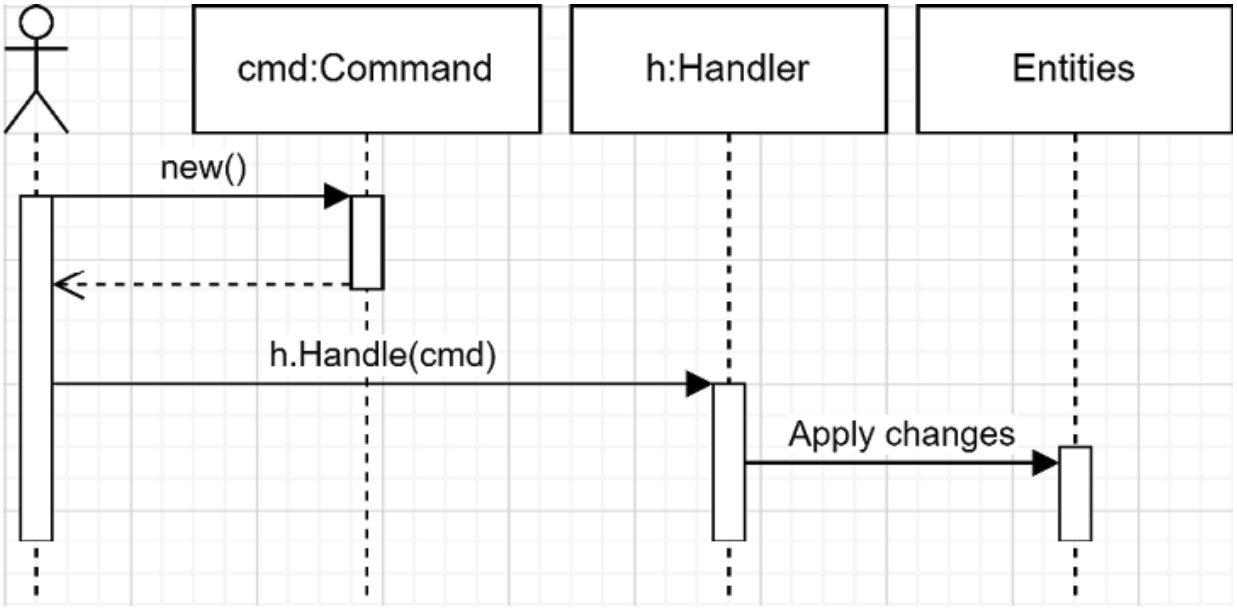


Figure 16.8: Sequence diagram representing the abstract flow of a command

The consumer creates a command object and sends it to a command handler, applying the mutation to the application. I called it `Entities` in this case, but it could have sent a SQL `UPDATE` command to a database or a web API call over HTTP; the implementation details do not matter.The concept is the same for a query, but it returns a value instead. Very importantly, the query must not change the state of the application. A query should only read data, like this:

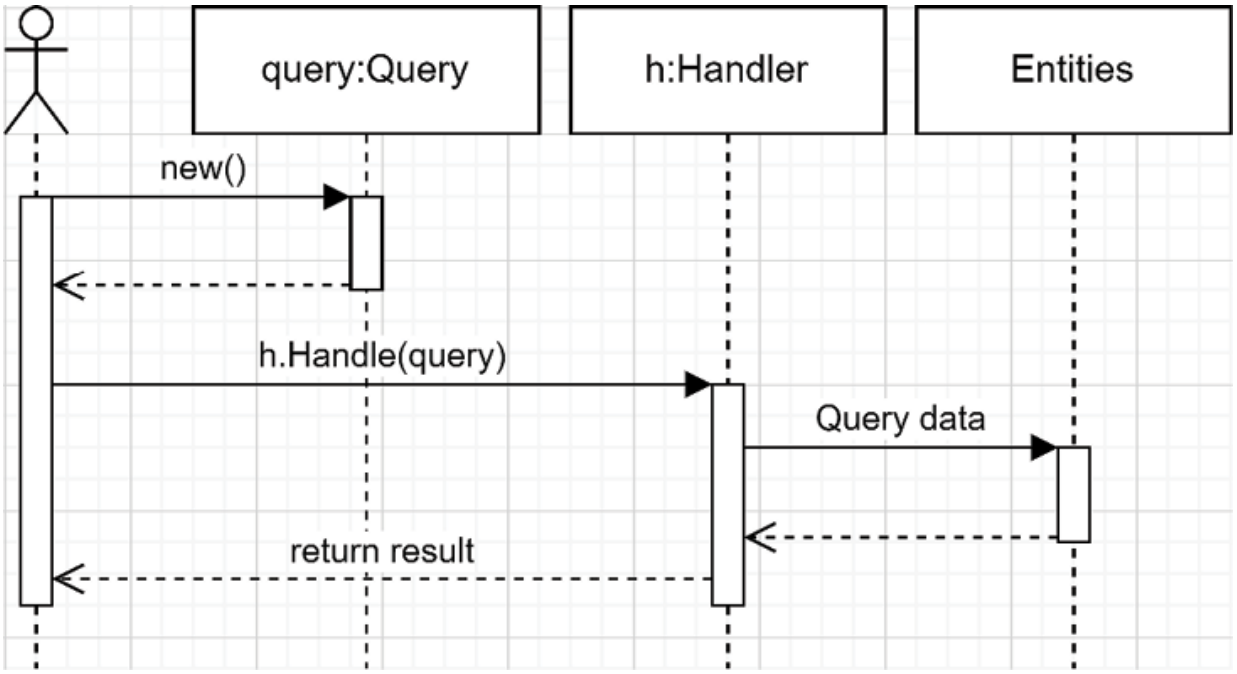


Figure 16.9: Sequence diagram representing the abstract flow of a query

Like the command, the consumer creates a query object and sends it to a handler, which then executes some logic to retrieve and return the requested data. We can replace `Entities` with anything the handler needs to query the data.Enough talk—let's look at the CQS project.

Project – CQS

Context: We need to build an improved version of our chat system. The old system worked so well that we need to scale it up. The mediator was of help to us, so we kept that part, and we picked the CQS pattern to help us with this new, improved design. A participant was limited to a single chatroom in the past, but now a participant must be able to chat in multiple rooms simultaneously.The new system is composed of three commands and two queries:

- A participant must be able to join a chatroom.
- A participant must be able to leave a chatroom.
- A participant must be able to send a message into a chatroom.
- A participant must be able to obtain the list of participants that joined a chatroom.
- A participant must be able to retrieve the existing messages from a chatroom.

The first three are commands, and the last two are queries. The system is backed by the following mediator that makes heavy use of C# generics:

```
public interface IMediator
{
    TReturn Send<TQuery, TReturn>(TQuery query)
        where TQuery : IQuery<TReturn>;
    void Send<TCommand>(TCommand command)
        where TCommand : ICommand;
    void Register<TCommand>(ICommandHandler<TCommand> commandHandler)
        where TCommand : ICommand;
    void Register<TQuery, TReturn>(IQueryHandler<TQuery, TReturn> commandHandler)
        where TQuery : IQuery<TReturn>;
}
```

If you are not familiar with generics, this might look daunting, but that code is way simpler than it looks. Next, the `ICommand` interface is empty, which we could have avoided, but it helps describe our intent. The

`ICommandHandler` interface defines the contract a class must implement to handle a command. That interface defines a `Handle` method that takes the command as a parameter. The generic parameter `TCommand` represents the type of command the class implementing the interface can handle. Here's the code:

```
public interface ICommand { }
public interface ICommandHandler<TCommand>
    where TCommand : ICommand
{
    void Handle(TCommand command);
}
```

The `IQuery<TReturn>` interface is similar to the `ICommand` interface but has a `TReturn` generic parameter indicating the query's return type. The IQueryHandler interface is also very similar, but its `Handle` method takes an object of type `TQuery` as a parameter and returns a `TReturn` type. Here's the code:

```
public interface IQuery<TReturn> { }
public interface IQueryHandler<TQuery, TReturn>
    where TQuery : IQuery<TReturn>
{
    TReturn Handle(TQuery query);
}
```

The `IMediator` interface allows registering command and query handlers using its `Register` methods. It also supports sending commands and queries through its `Send` methods. Then we have the `ChatMessage` class, which is similar to the last two samples (with an added creation date):

```
public record class ChatMessage(IParticipant Sender, string Message)
{
    public DateTime Date { get; } = DateTime.UtcNow;
}
```

Next is the updated `IParticipant` interface:

```
public interface IParticipant
{
    string Name { get; }
    void Join(IChatRoom chatRoom);
    void Leave(IChatRoom chatRoom);
    void SendMessageTo(IChatRoom chatRoom, string message);
    void NewMessageReceivedFrom(IChatRoom chatRoom, ChatMessage message);
    IEnumerable<IParticipant> ListParticipantsOf(IChatRoom chatRoom);
    IEnumerable<ChatMessage> ListMessagesOf(IChatRoom chatRoom);
}
```

All methods of the `IParticipant` interface accept an `IChatRoom` parameter to support multiple chatrooms. The updated `IChatRoom` interface has a name and a few basic operations to meet the requirement of a chatroom, like adding and removing participants:

```
public interface IChatRoom
{
    string Name { get; }
    void Add(IParticipant participant);
    void Remove(IParticipant participant);
    IEnumerable<IParticipant> ListParticipants();
    void Add(ChatMessage message);
    IEnumerable<ChatMessage> ListMessages();
}
```

Before going into commands and the chat itself, let's take a peek at the `Mediator` class:

```
public class Mediator : IMediator
{
    private readonly HandlerDictionary _handlers = new();
    public void Register<TCommand>(ICommandHandler<TCommand> commandHandler)
        where TCommand : ICommand
    {
```

```
        _handlers.AddHandler(commandHandler);
    }
    public void Register<TQuery, TReturn> (IQueryHandler<TQuery, TReturn> commandHandler)
        where TQuery : IQuery<TReturn>
    {
        _handlers.AddHandler(commandHandler);
    }
    public TReturn Send<TQuery, TReturn>(TQuery query)
        where TQuery : IQuery<TReturn>
    {
        var handler = _handlers.Find<TQuery, TReturn>();
        return handler.Handle(query);
    }
    public void Send<TCommand>(TCommand command)
        where TCommand : ICommand
    {
        var handlers = _handlers.FindAll<TCommand>();
        foreach (var handler in handlers)
        {
            handler.Handle(command);
        }
    }
}
```

The `Mediator` class supports registering commands and queries as well as sending a query to a handler or sending a command to zero or more handlers.

> I omitted the implementation of `HandlerDictionary` because it does not add value to the example, it is just an implementation detail, but it would have added unnecessary complexity. It is available on GitHub: https://adpg.link/2Lsm.

Now to the commands. I grouped the commands and the handlers together to keep it organized and readable, but you could use another way to organize yours. Moreover, since this is a small project, all the commands are in the same file, which would not be viable for something bigger. Remember, we are playing LEGO blocks, this chapter covers the CQS pieces, but you can always use them with bigger pieces like Clean Architecture or other types of architecture.

> We cover ways to organize commands and queries in subsequent chapters.

Let's start with the `JoinChatRoom` feature:

```
public class JoinChatRoom
{
    public record class Command(IChatRoom ChatRoom, IParticipant Requester) : ICommand;
    public class Handler : ICommandHandler<Command>
    {
        public void Handle(Command command)
        {
            command.ChatRoom.Add(command.Requester);
        }
    }
}
```

The `Command` class represents the command itself, a data structure that carries the command data. The `Handler` class handles that type of command. When executed, it adds the specified `IParticipant` to the specified `IChatRoom`, using the `ChatRoom` and `Requester` properties (highlighted line). Next feature:

```
public class LeaveChatRoom
{
    public record class Command(IChatRoom ChatRoom, IParticipant Requester) : ICommand;
    public class Handler : ICommandHandler<Command>
    {
        public void Handle(Command command)
        {
            command.ChatRoom.Remove(command.Requester);
        }
```

```
    }
}
```

That code represents the exact opposite of the `JoinChatRoom` command, the `LeaveChatRoom` handler removes an `IParticipant` from the specified `IChatRoom` (highlighted line).

Nesting the classes like this allows reusing the class name `Command` and `Handler` for each feature.

To the next feature:

```
public class SendChatMessage
{
    public record class Command(IChatRoom ChatRoom, ChatMessage Message) : ICommand;
    public class Handler : ICommandHandler<Command>
    {
        public void Handle(Command command)
        {
            command.ChatRoom.Add(command.Message);
            var participants = command.ChatRoom.ListParticipants();
            foreach (var participant in participants)
            {
                participant.NewMessageReceivedFrom(
                    command.ChatRoom,
                    command.Message
                );
            }
        }
    }
}
```

The `SendChatMessage` feature, on the other hand, handles two things (highlighted lines):

- It adds the specified `Message` to `IChatRoom` (now only a data structure that keeps track of users and past messages).
- It also sends the specified `Message` to all `IParticipant` instances that joined that `IChatRoom`.

We are starting to see many smaller pieces interacting with each other to create a more developed system. But we are not done; let's look at the two queries, then the chat implementation:

```
public class ListParticipants
{
    public record class Query(IChatRoom ChatRoom, IParticipant Requester) : IQuery<IEnumerable<IP
    public class Handler : IQueryHandler<Query, IEnumerable<IParticipant>>
    {
        public IEnumerable<IParticipant> Handle(Query query)
        {
            return query.ChatRoom.ListParticipants();
        }
    }
}
```

The `ListParticipants` handler uses the specified `IChatRoom` and returns its participants (highlighted line). Now, to the last query:

```
public class ListMessages
{
    public record class Query(IChatRoom ChatRoom, IParticipant Requester) : IQuery<IEnumerable<Ch
    public class Handler : IQueryHandler<Query, IEnumerable<ChatMessage>>
    {
        public IEnumerable<ChatMessage> Handle(Query query)
        {
            return query.ChatRoom.ListMessages();
        }
    }
}
```

The `ListMessages` handler uses the specified `IChatRoom` instance to return its messages.

Because all commands and queries reference `IParticipant`, we could enforce rules such as "`IParticipant` must join a channel before sending messages," for example. I decided to omit these details to keep the code simple, but feel free to add those features if you want to.

Next, let's take a look at the `ChatRoom` class, which is a simple data structure that holds the state of a chatroom:

```
public class ChatRoom : IChatRoom
{
    private readonly List<IParticipant> _participants = new List<IParticipant>();
    private readonly List<ChatMessage> _chatMessages = new List<ChatMessage>();
    public ChatRoom(string name)
    {
        Name = name ?? throw new ArgumentNullException(nameof(name));
    }
    public string Name { get; }
    public void Add(IParticipant participant)
    {
        _participants.Add(participant);
    }
    public void Add(ChatMessage message)
    {
        _chatMessages.Add(message);
    }
    public IEnumerable<ChatMessage> ListMessages()
    {
        return _chatMessages.AsReadOnly();
    }
    public IEnumerable<IParticipant> ListParticipants()
    {
        return _participants.AsReadOnly();
    }
    public void Remove(IParticipant participant)
    {
        _participants.Remove(participant);
    }
}
```

If we take a second look at the `ChatRoom` class, it has a `Name` property. It contains a list of `IParticipant` instances and a list of `ChatMessage` instances. Both `ListMessages()` and `ListParticipants()` return the list `AsReadOnly()`, so a clever programmer cannot mutate the state of `ChatRoom` from the outside. That's it; the new `ChatRoom` class is a façade over its underlying dependencies.Finally, the `Participant` class is probably the most exciting part of this system because it is the one that makes heavy use of our Mediator and CQS:

```
public class Participant : IParticipant
{
    private readonly IMediator _mediator;
    private readonly IMessageWriter _messageWriter;
    public Participant(IMediator mediator, string name, IMessageWriter messageWriter)
    {
        _mediator = mediator ?? throw new ArgumentNullException(nameof(mediator));
        Name = name ?? throw new ArgumentNullException(nameof(name));
        _messageWriter = messageWriter ?? throw new ArgumentNullException(nameof(messageWriter));
    }
    public string Name { get; }
    public void Join(IChatRoom chatRoom)
    {
        _mediator.Send(new JoinChatRoom.Command(chatRoom, this));
    }
    public void Leave(IChatRoom chatRoom)
    {
        _mediator.Send(new LeaveChatRoom.Command(chatRoom, this));
    }
    public IEnumerable<ChatMessage> ListMessagesOf(IChatRoom chatRoom)
    {
        return _mediator.Send<ListMessages.Query, IEnumerable<ChatMessage>>(new ListMessages.Que
    }
    public IEnumerable<IParticipant> ListParticipantsOf(IChatRoom chatRoom)
    {
        return _mediator.Send<ListParticipants.Query, IEnumerable<IParticipant>>(new ListPartici
```

```
    }
    public void NewMessageReceivedFrom(IChatRoom chatRoom, ChatMessage message)
    {
        _messageWriter.Write(chatRoom, message);
    }
    public void SendMessageTo(IChatRoom chatRoom, string message)
    {
        _mediator.Send(new SendChatMessage.Command (chatRoom, new ChatMessage(this, message)));
    }
}
```

Every method of the `Participant` class, apart from `NewMessageReceivedFrom`, sends a command or a query through the `IMediator` interface, breaking the tight coupling between the participants and the system's operations (that is, the commands and queries). The `Participant` class is also a simple façade over its underlying dependencies, delegating most of the work to the mediator.Now that we have covered the numerous tiny pieces let's look at how everything works together. I grouped several test cases that share the following setup code:

```
public class ChatRoomTest
{
    private readonly IMediator _mediator = new Mediator();
    private readonly TestMessageWriter _reagenMessageWriter = new();
    private readonly TestMessageWriter _garnerMessageWriter = new();
    private readonly TestMessageWriter _corneliaMessageWriter = new();
    private readonly IChatRoom _room1 = new ChatRoom("Room 1");
    private readonly IChatRoom _room2 = new ChatRoom("Room 2");
    private readonly IParticipant _reagen;
    private readonly IParticipant _garner;
    private readonly IParticipant _cornelia;
    public ChatRoomTest()
    {
        _mediator.Register(new JoinChatRoom.Handler());
        _mediator.Register(new LeaveChatRoom.Handler());
        _mediator.Register(new SendChatMessage.Handler());
        _mediator.Register(new ListParticipants.Handler());
        _mediator.Register(new ListMessages.Handler());
        _reagen = new Participant(_mediator, "Reagen", _reagenMessageWriter);
        _garner = new Participant(_mediator, "Garner", _garnerMessageWriter);
        _cornelia = new Participant(_mediator, "Cornelia", _corneliaMessageWriter);
    }
    // Omited test cases and helpers
}
```

The test program setup is composed of the following:

- One `IMediator` field initialized with a `Mediator` instance, which enables all colleagues to interact with each other.
- Two `IChatRoom` fields initialized with `ChatRoom` instances.
- Three `IParticipant` uninitialized fields, later initialized with `Participant` instances.
- Three `TestMessageWriter` instances, one per participant.
- The constructor registers all handlers with the `Mediator` instance so it knows how to handle commands and queries. It also creates the participants.

Once again, the names of the participants are randomly generated.

The `TestMessageWriter` implementation is a little different and accumulates the data in a list of tuples (`List<(IChatRoom, ChatMessage)>`) to assess what the participants send:

```
private class TestMessageWriter : IMessageWriter
{
    public List<(IChatRoom chatRoom, ChatMessage message)> Output { get; } = new();
    public void Write(IChatRoom chatRoom, ChatMessage message)
    {
        Output.Add((chatRoom, message));
    }
}
```

Here is the first test case:

```
[Fact]
public void A_participant_should_be_able_to_list_the_participants_that_joined_a_chatroom()
{
    _reagen.Join(_room1);
    _reagen.Join(_room2);
    _garner.Join(_room1);
    _cornelia.Join(_room2);
    var room1Participants = _reagen.ListParticipantsOf(_room1);
    Assert.Collection(room1Participants,
        p => Assert.Same(_reagen, p),
        p => Assert.Same(_garner, p)
    );
}
```

In the preceding test case, Reagen and Garner join Room 1, while Reagen and Cornelia join Room 2. Then Reagen requests the list of participants from Room 1, which outputs Reagen and Garner. Under the hood, it uses commands and queries through a mediator, breaking tight coupling between the colleagues. Here is a sequence diagram representing what happens when a participant joins a chatroom:

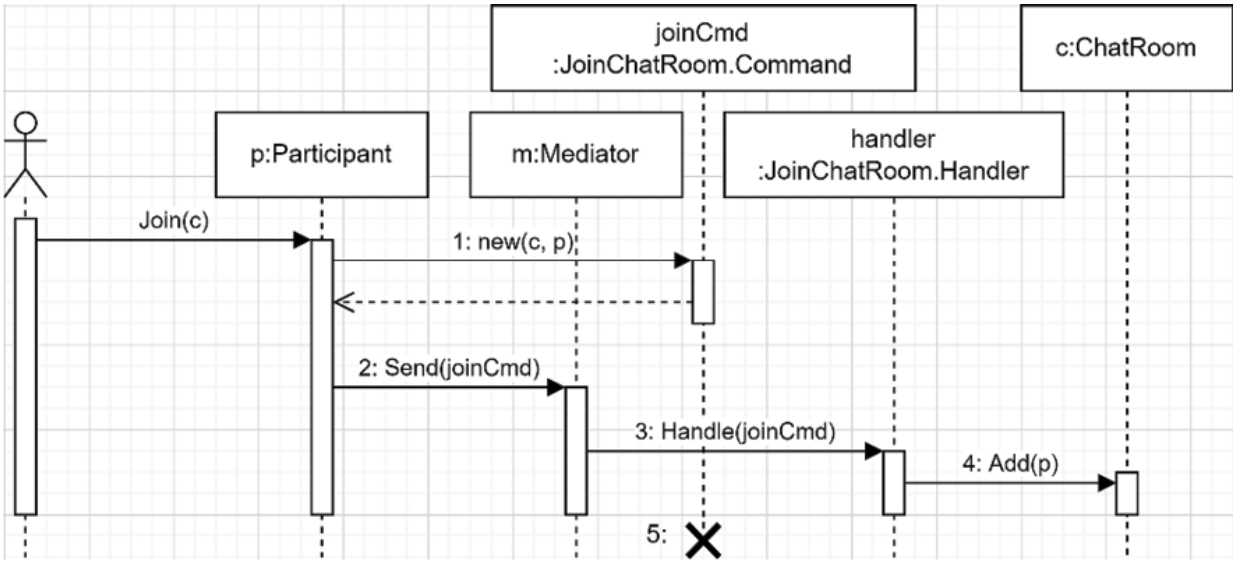


Figure 16.10: Sequence diagram representing the flow of a participant (p) joining a chatroom (c)

1. The participant (`p`) creates a `JoinChatRoom` command (`joinCmd`).
2. `p` sends `joinCmd` through the mediator (`m`).
3. `m` finds and dispatches `joinCmd` to its handler (`handler`).
4. `handler` executes the logic (adds `p` to the chatroom).
5. `joinCmd` ceases to exist afterward; commands are ephemeral.

That means `Participant` never interacts directly with `ChatRoom` or other participants.Then a similar workflow happens when a participant requests the list of participants of a chatroom:

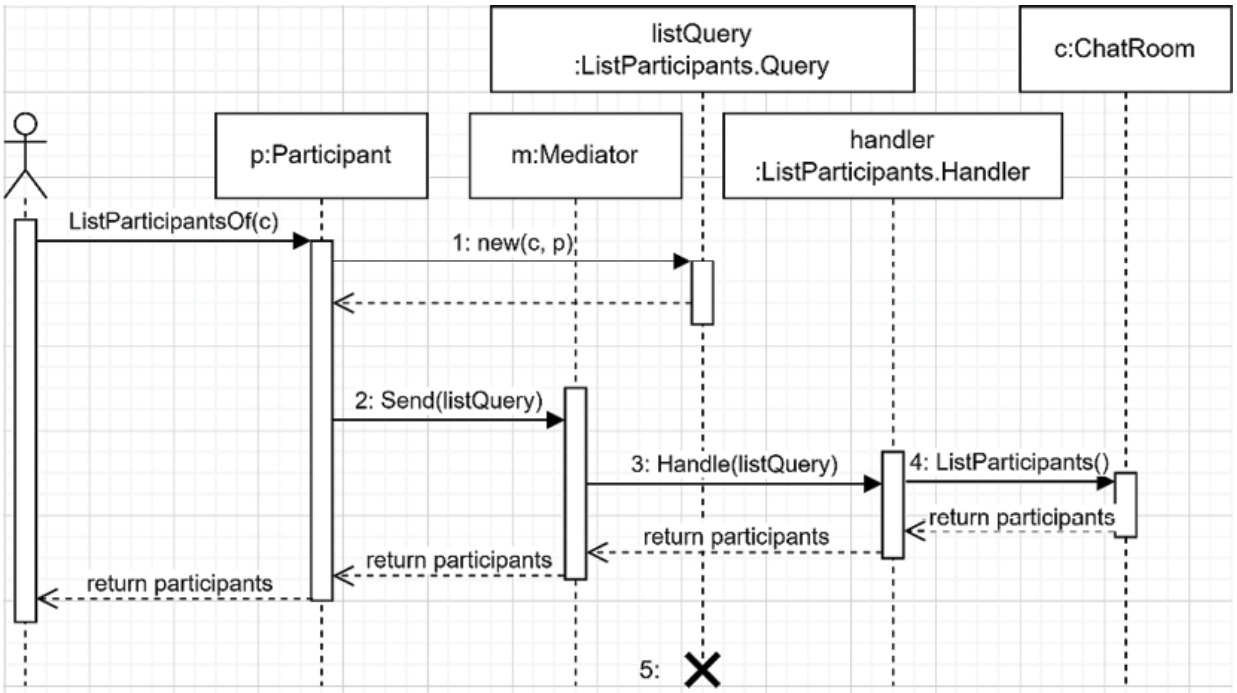


Figure 16.11: Sequence diagram representing the flow of a participant (p) requesting the list of participants of a chatroom (c)

1. `Participant` (`p`) creates a `ListParticipants` query (`listQuery`).
2. `p` sends `listQuery` through the mediator (`m`).
3. `m` finds and dispatches the query to its handler (`handler`).
4. `handler` executes the logic (lists the participants of the chatroom).
5. `listQuery` ceases to exist afterward; queries are also ephemeral.

Once again, `Participant` does not interact directly with `ChatRoom`.Here is another test case where `Participant` sends a message to a chatroom, and another `Participant` receives it:

```
[Fact]
public void A_participant_should_receive_new_messages()
{
    _reagen.Join(_room1);
    _garner.Join(_room1);
    _garner.Join(_room2);
    _reagen.SendMessageTo(_room1, "Hello!");
    Assert.Collection(_garnerMessageWriter.Output,
    line =>
    {
        Assert.Equal(_room1, line.chatRoom);
        Assert.Equal(_reagen, line.message.Sender);
        Assert.Equal("Hello!", line.message.Message);
    }
  );
}
```

In the preceding test case, Reagen joins Room 1 while Garner joins Rooms 1 and 2. Then Reagen sends a message to Room 1, and we verify that Garner received it once.The `SendMessageTo` workflow is very similar to the other one that we saw but with a more complex command handler:

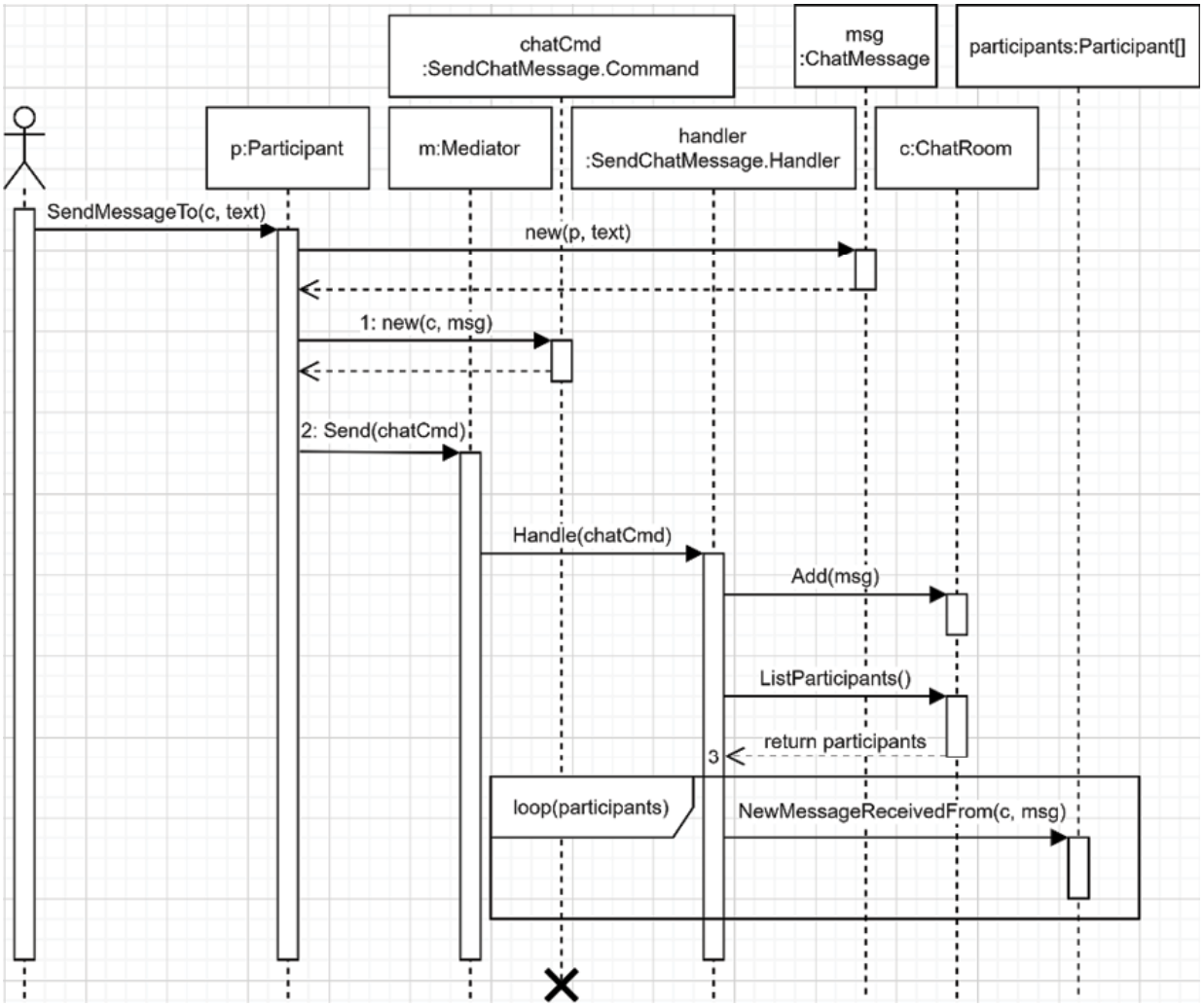


Figure 16.12: Sequence diagram representing the flow of a participant (p) sending a message (msg)to a chatroom (c)

From that diagram, we can observe that we pushed the logic to the `Handler` class of the `SendChatMessage` feature. All the other actors work together with limited to no knowledge of each other.This demonstrates how CQS works with a mediator:

1. A consumer (the participant in this case) creates a command (or a query).
2. The consumer sends that command through the mediator.
3. The mediator sends that command to one or more handlers, each executing their piece of logic for that command.

You can explore the other test cases to familiarize yourself with the program and the concepts.

> You can debug the tests in Visual Studio; use breakpoints combined with *Step Into (F11)* and *Step Over (F10)* to explore the sample.

I also created a `ChatModerator` instance that sends a message in a “moderator chatroom” when a message contains a word from the `badWords` collection. That test case executes multiple handlers for each `SendChatMessage.Command`. I’ll leave you to explore these other test cases yourself so we don’t wander astray from our goal.

Conclusion

The CQS and CQRS patterns suggest dividing the operations of a program into **commands** and **queries**. A command mutates data, and a query fetches data. We can apply the **Mediator** pattern to break the tight coupling between the pieces of a program using CQS, like sending commands and queries.Dividing the program this way helps separate the different pieces and focus on the commands and queries that travel from a consumer through the mediator to one or more handlers. The data contract of commands and queries becomes the program's backbone, trimming down the coupling between objects and tying them to those thin data structures instead, leaving the central piece (the mediator) to manage the links between them.On the other hand, you may find the codebase more intimidating when using CQS due to the multiple classes. It adds some complexity, especially for a small program like this. However, each type does less (having a single responsibility), making it easier to test than a more sizable class with many responsibilities.Now let's see how CQRS can help us follow the **SOLID** principles:

- **S**: Dividing an application into commands, queries, and handlers takes us toward encapsulating single responsibilities into different classes.
- **O**: CQS helps extend the software without modifying the existing code, such as adding handlers and creating new commands.
- **L**: N/A
- **I**: CQS makes it easier to create multiple small interfaces with a clear distinction between commands, queries, and their respective handlers.
- **D**: N/A

Now that we have explored CQRS, CQS, and the Mediator pattern, we explore the Marker Interfaces.

Code smell – Marker Interfaces

We used the empty `ICommand` and `IQuery<TReturn>` interfaces in the code samples to make the code more explicit and self-descriptive. Empty interfaces are a sign that something may be wrong: a code smell. We call those **marker interfaces**.In our case, they help identify commands and queries but are empty and add nothing. We could discard them without any impact on our system. On the other hand, we are not performing magic tricks or violating any principles, so they don't harm but help define the intent. Moreover, we could leverage them to make the code more dynamic, like leveraging dependency injection to register handlers. Furthermore, I designed those interfaces this way as a bridge to the next project.Back to the marker interfaces, here are two types of marker interfaces that are code smells in C#:

- Metadata
- Dependency identifier

Metadata

Markers can be used to define metadata. A class "implements" the empty interface, and some consumer does something with it later. It could be an assembly scanning for specific types, a choice of strategy, or something else.Instead of creating marker interfaces to add metadata, try to use custom attributes. The idea behind attributes is to add metadata to classes and their members. On the other hand, interfaces exist to create a contract, and they should define at least one member; empty contracts are like a blank sheet.In a real-world scenario, you may want to consider the cost of one versus the other. Markers are very cheap to implement but can violate architectural principles. Attributes can be as cheap to implement if the mechanism is already implemented or supported by the framework but can cost much more than a marker interface, depending on the scenario. Before deciding, I recommend you evaluate the cost of both options.

Dependency identifier

If you need markers to inject some specific dependency in a particular class, you are most likely cheating the **Inversion of Control** principle. Instead, you should find a way to achieve the same goal using dependency injection, such as by contextually injecting your dependencies.Let's start with the following interface:

```
public interface IStrategy
{
    string Execute();
}
```

In our program, we have two implementations and two markers, one for each implementation:

```
public interface IStrategyA : IStrategy { }
public interface IStrategyB : IStrategy { }
public class StrategyA : IStrategyA
{
    public string Execute() => "StrategyA";
}
public class StrategyB : IStrategyB
{
    public string Execute() => "StrategyB";
}
```

The code is barebones, but all the building blocks are there:

- `StrategyA` implements `IStrategyA`, which inherits from `IStrategy`.
- `StrategyB` implements `IStrategyB`, which inherits from `IStrategy`.
- Both `IStrategyA` and `IStrategyB` are empty marker interfaces.

Now, the consumer needs to use both strategies, so instead of controlling dependencies from the composition root, the consumer requests the markers:

```
public class Consumer
{
    public IStrategyA StrategyA { get; }
    public IStrategyB StrategyB { get; }
    public Consumer(IStrategyA strategyA, IStrategyB strategyB)
    {
        StrategyA = strategyA ?? throw new ArgumentNullException(nameof(strategyA));
        StrategyB = strategyB ?? throw new ArgumentNullException(nameof(strategyB));
    }
}
```

The `Consumer` class exposes the strategies through properties to assert its composition later. Let's test that out by building a dependency tree, simulating the composition root, and then asserting the value of the consumer properties:

```
[Fact]
public void ConsumerTest()
{
    // Arrange
    var serviceProvider = new ServiceCollection()
        .AddSingleton<IStrategyA, StrategyA>()
        .AddSingleton<IStrategyB, StrategyB>()
        .AddSingleton<Consumer>()
        .BuildServiceProvider();
    // Act
    var consumer = serviceProvider.GetRequiredService<Consumer>();
    // Assert
    Assert.IsType<StrategyA>(consumer.StrategyA);
    Assert.IsType<StrategyB>(consumer.StrategyB);
}
```

Both properties are of the expected type, but that is not the problem. The `Consumer` class controls what dependencies to use and when to use them by injecting markers A and B instead of two `IStrategy` instances. Due to that, we cannot control the dependency tree from the composition root. For example, we cannot change `IStrategyA` to `IStrategyB` and `IStrategyB` to `IStrategyA`, nor inject two `IStrategyB` instances or two `IStrategyA` instances, nor even create an `IStrategyC` interface to replace `IStrategyA` or `IStrategyB`.How do we fix this? Let's start by deleting our markers and injecting two `IStrategy` instances instead (the changes are highlighted). After doing that, we end up with the following object structure:

```
public class StrategyA : IStrategy
{
    public string Execute() => "StrategyA";
}
public class StrategyB : IStrategy
{
    public string Execute() => "StrategyB";
}
public class Consumer
{
    public IStrategy StrategyA { get; }
    public IStrategy StrategyB { get; }
    public Consumer(IStrategy strategyA, IStrategy strategyB)
    {
        StrategyA = strategyA ?? throw new ArgumentNullException(nameof(strategyA));
        StrategyB = strategyB ?? throw new ArgumentNullException(nameof(strategyB));
    }
}
```

The `Consumer` class no longer controls the narrative with the new implementation, and the composition responsibility falls back to the composition root. Unfortunately, there is no way to do contextual injections using the default dependency injection container, and I don't want to get into a third-party library for this. But all is not lost yet; we can use a factory to help ASP.NET Core build the `Consumer` instance, like this:

```
// Arrange
var serviceProvider = new ServiceCollection()
    .AddSingleton<StrategyA>()
    .AddSingleton<StrategyB>()
    .AddSingleton(serviceProvider =>
    {
        var strategyA = serviceProvider.GetRequiredService<StrategyA>();
        var strategyB = serviceProvider.GetRequiredService<StrategyB>();
        return new Consumer(strategyA, strategyB);
    })
    .BuildServiceProvider();
// Act
var consumer = serviceProvider.GetRequiredService<Consumer>();
// Assert
Assert.IsType<StrategyA>(consumer.StrategyA);
Assert.IsType<StrategyB>(consumer.StrategyB);
```

From this point forward, we control the program's composition, and we can swap A with B or do anything else that we want to, as long as the implementation respects the `IStrategy` contract.To conclude, using markers instead of doing contextual injection breaks the inversion of control principle, making the consumer control its dependencies. That's very close to using the `new` keyword to instantiate objects. Inverting the dependency control back is easy, even using the default container.

> If you need to inject dependencies contextually, I started an open source project in 2020 that does that. Multiple other third-party libraries add features or replace the default IoC container altogether if needed. See the *Further reading* section.

Next, we start the last part of this chapter. It showcases an open-source tool that can help us build CQS-oriented applications.

Using MediatR as a mediator

In this section, we are exploring MediatR, an open-source mediator implementation.What is MediatR? Let's start with its maker's description from its GitHub repository, which brands it as this:

> *"Simple, unambitious mediator implementation in .NET"*

MediatR is a simple but very powerful tool doing in-process communication through messaging. It supports a request/response flow through commands, queries, notifications, and events, synchronously and asynchronously.We can install the NuGet package using the .NET CLI: `dotnet add package MediatR` .Now that

I have quickly introduced the tool, we are going to explore the migration of our Clean Architecture sample but instead use MediatR to dispatch the `StocksController` requests to the core use cases. We use a similar pattern with MediatR than what we built in the CQS project.

> Why migrate our Clean Architecture sample? The primary reason we are building the same project using different models is for ease of comparison. It is much easier to compare the changes of the same features than if we were building completely different projects.

What are the advantages of using MediatR in this case? It allows us to organize the code around use cases (vertically) instead of services (horizontally), leading to more cohesive features. We remove the service layer (the `StockService` class) and replace it with multiple use cases (features) instead (the `AddStocks` and `RemoveStock` classes). MediatR also enables a pipeline we can extend by programming behaviors. Those extensibility points allow us to manage cross-cutting concerns, such as requests validation centrally, without impacting the consumers and use cases. We explore request validation in *Chapter 17*, *Getting Started with Vertical Slice Architecture*.Let's jump into the code now to see how it works.

Project – Clean Architecture with MediatR

Context: We want to break some more of the coupling in the Clean Architecture project we built in *Chapter 14*, *Understanding Layering*, by leveraging the **Mediator** pattern and a **CQS** approach.The clean architecture solution was already solid, but MediatR will pave the way to more good things later. The only "major" change is the replacement of the `StockService` with two feature objects, `AddStocks` and `RemoveStocks`, which we explore soon.First, we must install the `MediatR` NuGet package in the `Core` project, where the features will live. Moreover, it will transiently cascade to the `Web` project, allowing us to register MediatR with the IoC container. In the `Program.cs` file, we can register MediatR like this:

```
builder.Services
    // Core Layer
    .AddMediatR(cfg => cfg.RegisterServicesFromAssemblyContaining<NotEnoughStockException>())
;
```

That code scans the Core assembly for MediatR-compatible pieces and registers them with the IoC Container. The `NotEnoughStockException` class is part of the core project.

> I picked the `NotEnoughStockException` class but could have chosen any class from the `Core` assembly. There are more registration options.

MediatR exposes the following types of messages (as of version 12):

- *Request/response* that has one handler; perfect for commands and queries.
- *Notifications* that support multiple handlers; perfect for an event-based model applying the Publish-Subscribe pattern where a notification represents an event.
- *Request/response streams* that are similar to request/response but stream the response through the `IAsyncEnumerable<T>` interface.

> We cover the Publish-Subscribe pattern in *Chapter 19*, *Introduction to Microservices Architecture*.

Now that everything we need related to MediatR is "magically" registered, we can look at the use cases that replace the `StockService`. Let's have a look at the updated `AddStocks` code first:

```
namespace Core.UseCases;
public class AddStocks
{
    public class Command : IRequest<int>
    {
        public int ProductId { get; set; }
        public int Amount { get; set; }
    }
    public class Handler : IRequestHandler<Command, int>
    {
```

```
        private readonly IProductRepository _productRepository;
        public Handler(IProductRepository productRepository)
        {
            _productRepository = productRepository ?? throw new ArgumentNullException(nameof(pro
        }
        public async Task<int> Handle(Command request, CancellationToken cancellationToken)
        {
            var product = await _productRepository.FindByIdAsync(request.ProductId, cancellation
            if (product == null)
            {
                throw new ProductNotFoundException(request.ProductId);
            }
            product.AddStock(request.Amount);
            await _productRepository.UpdateAsync(product, cancellationToken);
            return product.QuantityInStock;
        }
    }
}
```

Since we covered both use cases in the previous chapters and the changes are very similar, we will analyze both together, after the `RemoveStocks` use case code:

```
namespace Core.UseCases;
public class RemoveStocks
{
    public class Command : IRequest<int>
    {
        public int ProductId { get; set; }
        public int Amount { get; set; }
    }
    public class Handler : IRequestHandler<Command, int>
    {
        private readonly IProductRepository _productRepository;
        public Handler(IProductRepository productRepository)
        {
            _productRepository = productRepository ?? throw new ArgumentNullException(nameof(pro
        }
        public async Task<int> Handle(Command request, CancellationToken cancellationToken)
        {
            var product = await _productRepository.FindByIdAsync(request.ProductId, cancellation
            if (product == null)
            {
                throw new ProductNotFoundException(request.ProductId);
            }
            product.RemoveStock(request.Amount);
            await _productRepository.UpdateAsync(product, cancellationToken);
            return product.QuantityInStock;
        }
    }
}
```

As you may have noticed in the code, I chose the same pattern to build the commands as I did with the CQS sample, so we have a class per use case containing two nested classes: `Command` and `Handler`. This structure makes for very clean code when you have a 1-on-1 relationship between the command class and its handler.Using the MediatR request/response model, the command (or query) becomes a request and must implement the `IRequest<TResponse>` interface. The handlers must implement the `IRequestHandler<TRequest, TResponse>` interface. Instead, we could implement the `IRequest` and `IRequestHandler<TRequest>` interfaces for a command that returns nothing (`void`).

More options are part of MediatR, and the documentation is complete enough to dig deeper yourself.

Let’s analyze the anatomy of the `AddStocks` use case. Here is the old code as a reference:

```
namespace Core.Services;
public class StockService
{
    private readonly IProductRepository _repository;
    // Omitted constructor
```

```
    public async Task<int> AddStockAsync(int productId, int amount, CancellationToken cancellati
    {
        var product = await _repository.FindByIdAsync(productId, cancellationToken);
        if (product == null)
        {
            throw new ProductNotFoundException(productId);
        }
        product.AddStock(amount);
        await _repository.UpdateAsync(product, cancellationToken);
        return product.QuantityInStock;
    }
    // Omitted RemoveStockAsync method
}
```

The first difference is that we moved the loose parameters (highlighted) into the `Command` class, which encapsulates the whole request:

```
public class Command : IRequest<int>
{
    public int ProductId { get; set; }
    public int Amount { get; set; }
}
```

Then the `Command` class specifies the handler's expected return value by implementing the `IRequest<TResponse>` interface, where `TResponse` is an `int`. That gives us a typed response when sending the request through MediatR. This is not "pure CQS" because the command handler returns an integer representing the updated `QuantityInStock`. However, we could call that optimization since executing one command and one query would be overkill for this scenario (possibly leading to two database calls instead of one).I'll skip the `RemoveStocks` use case to avoid repeating myself, as it follows the same pattern. Instead, let's look at the consumption of those use cases. I omitted the exception handling to keep the code streamlined and because `try` / `catch` blocks would only add noise to the code in this case and hinder our study of the pattern:

```
app.MapPost("/products/{productId:int}/add-stocks", async (
    int productId,
    AddStocks.Command command,
    IMediator mediator,
    CancellationToken cancellationToken) =>
{
    command.ProductId = productId;
    var quantityInStock = await mediator.Send(command, cancellationToken);
    var stockLevel = new StockLevel(quantityInStock);
    return Results.Ok(stockLevel);
});
app.MapPost("/products/{productId:int}/remove-stocks", async (
    int productId,
    RemoveStocks.Command command,
    IMediator mediator,
    CancellationToken cancellationToken) =>
{
    command.ProductId = productId;
    var quantityInStock = await mediator.Send(command, cancellationToken);
    var stockLevel = new StockLevel(quantityInStock);
    return Results.Ok(stockLevel);
});
// Omitted code
public record class StockLevel(int QuantityInStock);
```

In both delegates, we inject an `IMediator` and a command object (highlighted). We also let ASP.NET Core inject a `CancellationToken`, which we pass to MediatR. The model binder loads the data from the HTTP request into the objects that we send using the `Send` method of the `IMediator` interface (highlighted). Then we map the result into the `StockLevel` DTO before returning its value and an HTTP status code of `200 OK`. The `StockLevel` record class is the same as before.This example contains almost the same code as our CQS example, but we used MediatR instead of manually programming the pieces.

> The default model binder cannot load data from multiple sources. Because of that, we must inject `productId` and assign its value to the `command.ProductId` property manually. Even if both values could be taken from

the body, the resource identifier of that endpoint would become less exhaustive (no `productId` in the URI).

With MVC, we could create a custom model binder.

With minimal APIs, we could create a static `BindAsync` method to manually do the model binding, which is not very extensible and would tightly couple the `Core` assembly with the `HttpContext`. I suppose we will need to wait for .NET 9+ to get improvements into that field.

I've left a few links in the *further reading* section relating to this.

Conclusion

With MediatR, we packed the power of a CQS-inspired pipeline with the Mediator pattern into a Clean Architecture application. We broke the coupling between the request delegates and the use case handler (previously a service). A simple DTO, such as a command object, makes endpoints and controllers unaware of the handlers, leaving MediatR as the middleman between the commands and their handlers. Due to that, the handlers could change along the way without impacting the endpoint.Moreover, we could configure more interaction between the command and the handler with `IRequestPreProcessor`, `IRequestPostProcessor`, and `IRequestExceptionHandler`. These allow us to extend the MediatR request pipeline with cross-cutting concerns like validation and error handling.MediatR helps us follow the SOLID principles the same way as the Mediator and CQS patterns combined. The only drawback of the overall design, which has nothing to do with MediatR, is that we used the commands as the DTOs. We could create custom DTOs and map them to command objects. However, you will understand in the next chapter where I was heading with this transitory design.

Summary

In this chapter, we looked at the Mediator pattern, which allows us to cut the ties between collaborators, mediating the communication between them. Then we studied the CQS pattern, which advises the division of software behaviors into commands and queries. Those two patterns are tools that cut tight coupling between components.Afterward, we updated a Clean Architecture project to use MediatR, an open-source generic mediator that is CQS-oriented. There are many more possible uses than we explored, but this is still a great start. This concludes another chapter exploring techniques to break tight coupling and divide systems into smaller parts.All those building blocks lead us to the next chapter, where we piece those patterns and tools together to explore the Vertical Slice Architecture.

Questions

Let's take a look at a few practice questions:

1. What does the CQS stand for, and what is the purpose of this design pattern?
2. Can we use a mediator inside a colleague to call another colleague?
3. In CQS, can a command return a value?
4. How much does MediatR cost?
5. Imagine a design with a marker interface to add metadata to some classes. Do you think you should review that design?

Further reading

Here are a few links to build on what we have learned in the chapter:

- MediatR: https://adpg.link/ZQap
- To get rid of setting `ProductId` manually in the Clean Architecture with MediatR project, you can use the open-source project `HybridModelBinding` or read the official documentation about custom model binding and implement your own:

1. Custom Model Binding in ASP.NET Core: https://adpg.link/65pb
2. Damian Edward's MinimalApis.Extensions project on GitHub: https://adpg.link/M6zS

- `ForEvolve.DependencyInjection` is an open-source project that adds support for contextual dependency injection and more: https://adpg.link/myW8

Answers

1. CQS stands for Command-Query Separation. It's a software design principle that separates operations that change the state of an object (commands) from those that return data (queries). This helps in minimizing side effects and preventing unexpected changes in program behavior.
2. Yes, you can. The goal of the Mediator pattern is to mediate communication between colleagues.
3. In the original sense of CQS: no, a command can't return a value. The idea is that a query reads data while commands mutate data. A command can return a value in a looser sense of CQS. For example, nothing stops a create command from returning the created entity partially or totally. You can always trade a bit of modularity for a bit of performance.
4. MediatR is a free, open-source project licensed under Apache License 2.0.
5. Yes, you should. Using Marker Interfaces to add metadata is generally wrong. Nevertheless, you should analyze each use case individually, considering the pros and cons before jumping to a conclusion.

17 Getting Started with Vertical Slice Architecture

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter introduces Vertical Slice Architecture, an effective way to organize our ASP.NET Core applications. Vertical Slice Architecture moves elements from multiple layers to a feature-oriented design, helping us maintain a clean, simple, cohesive, loosely-coupled, and manageable codebase.Vertical Slice Architecture flips our architectural perspective toward simplified architecture. Historically, we divided the logic of a feature across various layers like UI, business logic, and data access. However, we create independent slices of functionality with Vertical Slice Architecture instead. Think of your application as a cake; instead of cutting it horizontally (layers), we're cutting vertically (features), with each slice being fully functional on its own.This style changes how we design and organize our project, testing strategies, and coding approach. We don't have to worry about bloated controllers or overly complicated "God objects"; instead, making changes becomes more manageable because of the loose coupling between features.This chapter guides you through applying Vertical Slice Architecture to your ASP.NET Core applications, detailing how to handle commands, queries, validation, and entity mapping using CQS, MVC, MediatR, AutoMapper, and FluentValidation, which we explored in the previous chapters.

> We don’t have to use those tools to apply the architectural style and can replace those libraries with others or even code the whole stack ourselves.

By the end of this chapter, you will understand Vertical Slice Architecture and its benefits, and should have the confidence to apply this style to your next project. In this chapter, we cover the following topics:

- Anti-pattern – Big Ball of Mud
- Vertical Slice Architecture
- Continuing your journey: A few tips and tricks

Let's journey through the vertical slices and piece the architecture together, one slice at a time.

Anti-pattern – Big Ball of Mud

Let’s start with an anti-pattern. The **Big Ball of Mud** anti-pattern describes a system that ended badly or was never properly designed. Sometimes a system starts great but evolves into a Big Ball of Mud due to pressure, volatile requirements, impossible deadlines, bad practices, or other reasons. We often refer to the Big Ball of Mud as **spaghetti code**, which means the same thing.This anti-pattern means a very hard-to-maintain codebase, poorly written code that is difficult to read, lots of unwanted tight coupling, low cohesion, or worse: all that in the same codebase.Applying the techniques covered in this book should help you avoid this anti-pattern. Aim at small, well-designed components that are testable. Enforce that using automated testing. Refactor and improve your codebase whenever you can, iteratively (continuous improvement). Apply the SOLID principles. Define your application pattern before starting. Think of the best way to implement each component and feature; do research, and make one or more proof of concept or experiments if unsure of the best approach. Ensure you understand the business requirements of the program you are building (this is probably the best advice). Those tips should help you avoid

creating a Big Ball of Mud.Building feature-oriented applications is one of the best ways to avoid creating a Big Ball of Mud. Let’s get started!

Vertical Slice Architecture

Instead of separating an application horizontally (layers), a vertical slice groups all horizontal concerns together to encapsulate a feature. Here is a diagram that illustrates that:

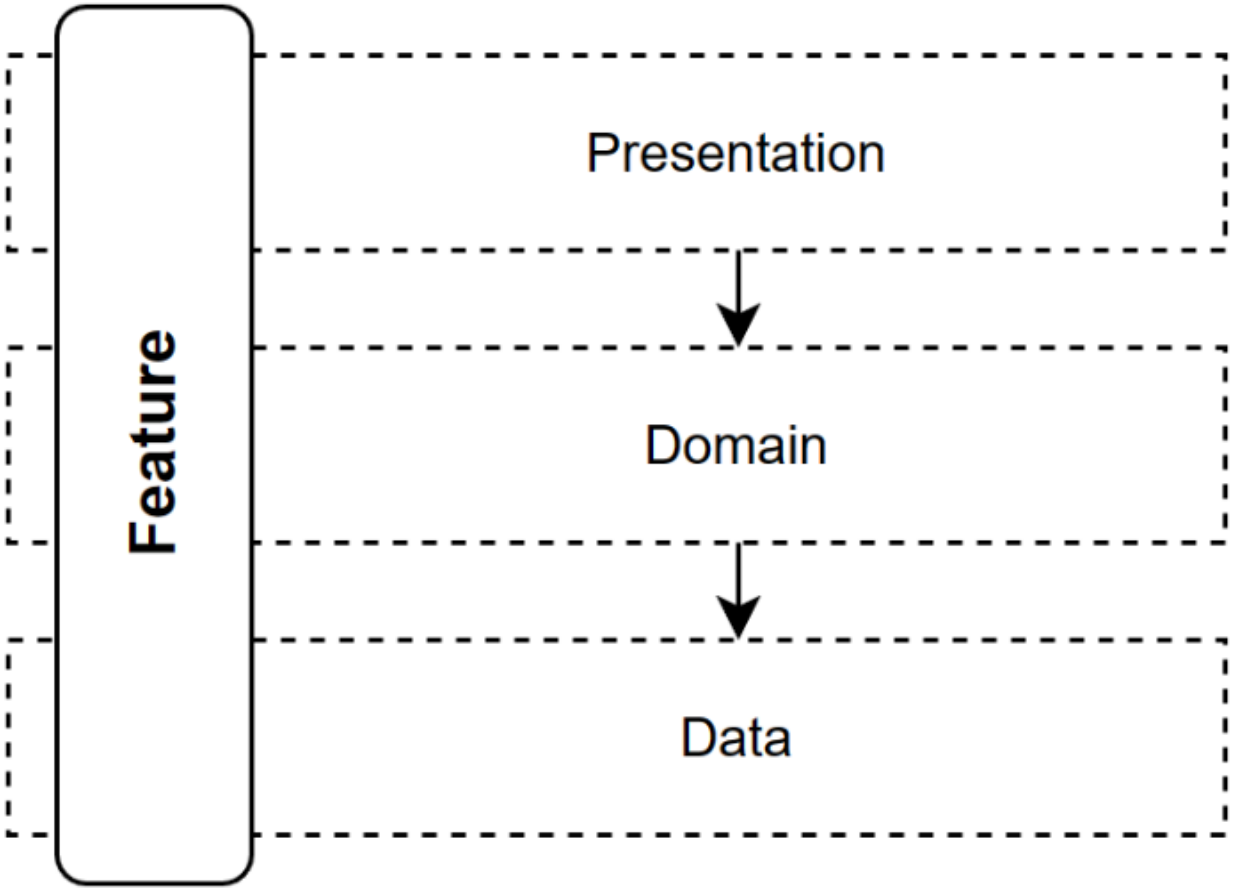


Figure 17.1: Diagram representing a vertical slice crossing all layers

Jimmy Bogard, who is a pioneer of this type of architecture, wrote the following:

> *[The goal is to] minimize coupling between slices and maximize coupling within a slice.*

What does that mean? Let’s split that sentence into two distinct points:

- “minimize coupling between slices” (improved maintainability, loose coupling)
- “maximize coupling within a slice” (cohesion)

We could see the former as one vertical slice should not depend on another, so when you modify a vertical slice, you don’t have to worry about the impact on the other slices because the coupling is minimal.We could see the latter as: instead of spreading code around multiple layers, with potentially superfluous abstractions along the way, let’s regroup and simplify that code. That helps keep the tight coupling inside a vertical slice to create a cohesive unit of code that serves a single purpose: handling the feature end to end.Then we could wrap that to create software around the business problem we are trying to solve instead of the developer’s concerns, which your customers have no interest in (such as data access).Now, what is a slice in more generic terms? I see slices as

composite hierarchies. For example, a shipping manager program has a multistep creation workflow, a list, and a details page. Each step of the creation flow would be a slice responsible for handling its respective logic. When put together, they compose the “create slice”, which is responsible for creating a shipment (a bigger slice). The list and details pages are two other slices. Then, all of those slices become another bigger slice, leading to something like this:

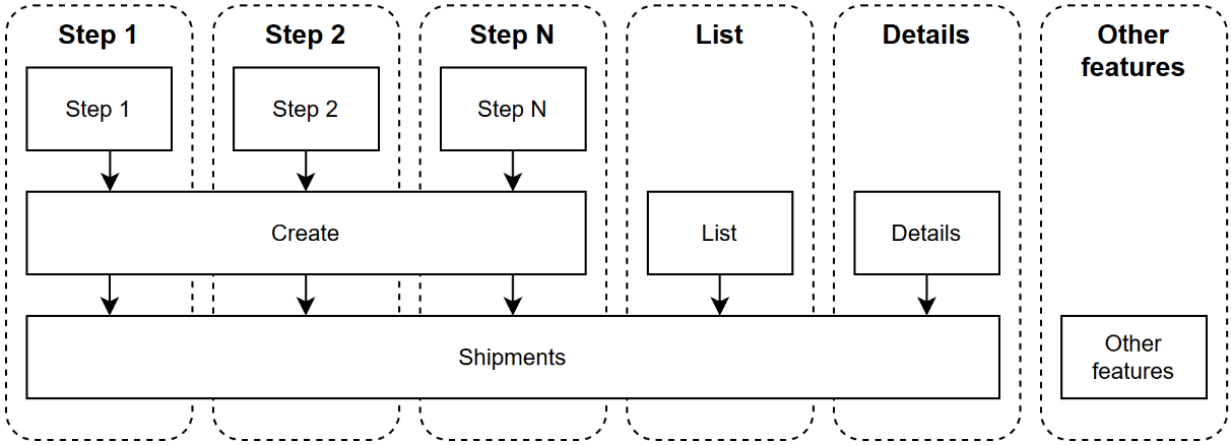


Figure 17.2: A diagram displaying a top-down coupling structure where smaller parts (top) depend on bigger parts (middle) of complex features (bottom) based on their cohesion with one another (vertically)

There is strong coupling inside **Step 1**, with limited coupling between the other steps; they share some creation code as part of the **Create** slice. **Create**, **List**, and **Details** also share some code, but in a limited way; they are all part of the **Shipments** slice and access or manipulate the same entity: one or more shipments. Finally, the **Shipments** slice shares no code (or very little) with **Other features**.

> Following the pattern I just described, we have limited coupling and maximum cohesion. The downside is that you must continuously design and refactor the application, which requires stronger design skills than a layered approach. Moreover, you must know how to build the feature end to end, limiting the division of tasks between people and centralizing them on each team member instead; each member becomes a full-stack developer. We revisit this example in the *Continuing your journey* section near the end of the chapter.

We explore the advantages and disadvantages next.

What are the advantages and disadvantages?

Let’s explore some advantages and disadvantages of Vertical Slice Architecture.

Advantages

On the upside, we have the following:

- We reduce coupling between features, making working on such a project more manageable. We only need to think about a single vertical slice, not *N* layers, improving **maintainability** by centralizing the code around a shared concern.
- We can choose how each vertical slice interacts with the external resources they require without considering the other slices. That adds **flexibility** since one slice can use T-SQL while another uses EF Core, for example.
- We can start small with a few lines of code (described as **Transaction Scripts** in *Patterns of Enterprise Application Architecture*, by Martin Fowler) without extravagant design or over-engineering. Then we can refactor our way to a better design when the need arises and patterns emerge, leading to a **faster time to market**.
- Each vertical slice should contain precisely the right amount of code needed to be correct—not more, not less. That leads to a **more robust** codebase (less code means less extraneous code and less code to maintain).

- It is easier for newcomers to find their way around an existing system since each feature is near-independent, leading to a **faster onboarding time**.
- All patterns and techniques you learned in previous chapters still apply.

From my experience, features tend to start small and grow over time. The users often find out what they need while using the software, changing the requirements over time, which leads to changes in the software. After the fact, I wish many projects I worked on were built using Vertical Slice Architecture instead of layering.

Disadvantages

Of course, nothing is perfect, so here are some downsides:

- It may take time to wrap your head around Vertical Slice Architecture if you're used to layering, leading to an adaptation period to learn a new way to think about your software.
- It is a "newer" type of architecture, and people don't like change.

Another thing that I learned the hard way is to embrace change. I don't think I've seen one project end as it was supposed to. Everyone identifies the missing pieces of the business processes while using the software. That leads to the following advice: release the software as fast as possible and have your customers use the software as soon as possible. That advice can be easier to achieve with Vertical Slice Architecture because you build value for your customers instead of more or less valuable abstractions and layers. Having a customer try staged software is very hard; no customer has time to do such a thing; they are busy running their business. However, releasing production-ready slices may lead to faster adoption and feedback.

> At the beginning of my career, I was frustrated when specifications changed, and I thought that better planning would have fixed that. Sometimes better planning would have helped, but sometimes, the customer just did not know how to express their business processes or needs and had to try the application to figure it out. My advice here is don't be frustrated when the specs change, even if that means rewriting a part of the software that took you days or more to code in the first place; that will happen all the time. Learn to accept that instead, and find ways to make this process easier and faster. If you are in contact with the customers, find ways to help them figure out their needs and reduce the number of changes.

Downside or Upsides?

The following points are downsides that we can tame as upsides:

- Suppose you are used to working in silos (such as the DBAs doing the data stuff). In that case, assigning tasks that touch the whole feature may be more challenging, but this can become an advantage since everyone in your team works more closely together, leading to more learning and collaboration and possibly a new cross-functional team—which is excellent. Having a data expert on the team is great; no one is an expert in all areas.
- Refactoring: strong refactoring skills will go a long way. Over time, most systems need some refactoring, which is even more true for Vertical Slice Architecture. That can be caused by changes in the requirements or due to technical debt. No matter the reason, you may end up with a **Big Ball of Mud** if you don't. First, writing isolated code and then refactoring to patterns is a crucial part of Vertical Slice Architecture. That's one of the best ways to keep cohesion high inside a slice and coupling as low as possible between slices. This tip applies to all types of architecture and is made easier with a robust test suite that validates your changes automatically.

A way to start refactoring that business logic would be to push the logic into the **domain model**, creating a **rich domain model**. You can also use other design patterns and techniques to fine-tune the code and make it more maintainable, such as creating services or layers. A layer does not have to cross all vertical slices; it can cross only a subset of them.

> Compared to other application-level patterns, such as layering, fewer rules lead to more choices (Vertical Slice Architecture). *You can use all design patterns, principles, and best practices inside a vertical slice without exporting those choices application-wide.*

How do you organize a project into Vertical Slice Architecture? Unfortunately, there is no definitive answer to that, and it depends on the engineers working on the project. We explore one way in the next project, but you can organize your project as you see fit. Then we dig deeper into refactoring and organization.

Project – Vertical Slice Architecture

Context: We are getting tired of layering, and we got asked to rebuild our small demo shop using Vertical Slice Architecture.Here is an updated diagram that shows how we conceptually organized the project:

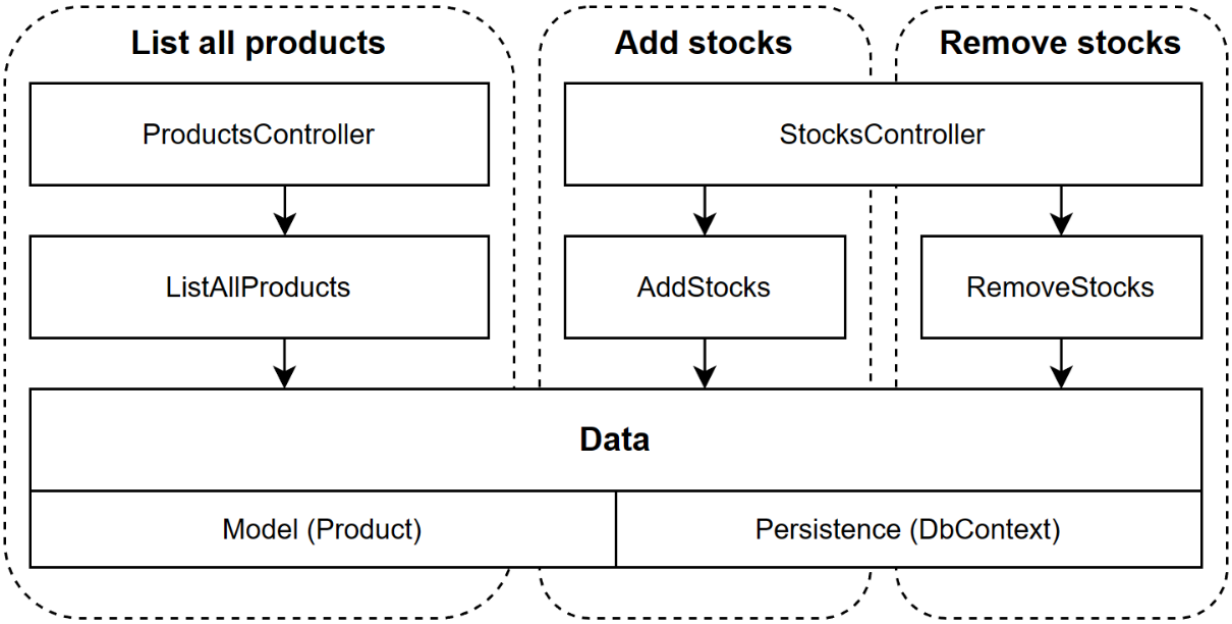


Figure 17.3: Diagram representing the organization of the demo shop project

Each vertical box is a use case (or slice), while each horizontal box is a crosscutting concern or a shared component. This is a small project, so we share the data access code (`DbContext`) and the `Product` model between the three use cases. This sharing is unrelated to Vertical Slice Architecture, but splitting it more in a small project like this is hard and pointless.In this project, I decided to go with web API controllers instead of minimal APIs and an anemic product model instead of a rich one. We could have used minimal APIs, a rich model, or any combination. I chose this so you have a glimpse of using controllers, as this is something you might very well end up using. We go back to minimal APIs in the next chapter.Here are the actors:

- `ProductsController` is the REST API to manage products.
- `StocksController` is the REST API to manage the inventory.
- `AddStocks` , `RemoveStocks` , and `ListAllProducts` are the same use cases we have copied in our project since *Chapter 14*, *Layering and Clean Architecture*.
- The persistence “layer” consists of an EF Core `DbContext` that persists the `Product` model to an in-memory database.

We could add other crosscutting concerns on top of our vertical slices, such as authorization, error management, and logging, to name a few.Next, let’s look at how we organized the project.

Project organization

Here is how we organized the project:

- The `Data` directory contains EF Core-related classes.
- The `Features` directory contains the features. Each subfolder contains its underlying features (vertical slices), including controllers, exceptions, and other support classes required to implement the feature.
- Each use case is self-contained and exposes the following classes:

1. `Command` or `Query` representing the MediatR request.
2. `Result` is the return value of that request.
3. `MapperProfile` instructs AutoMapper on how to map the use case-related objects (if any).
4. `Validator` contains the validation rules to validate the `Command` or `Query` objects (if any).
5. `Handler` contains the use case logic: how to handle the request.

- The `Models` directory contains the domain model.

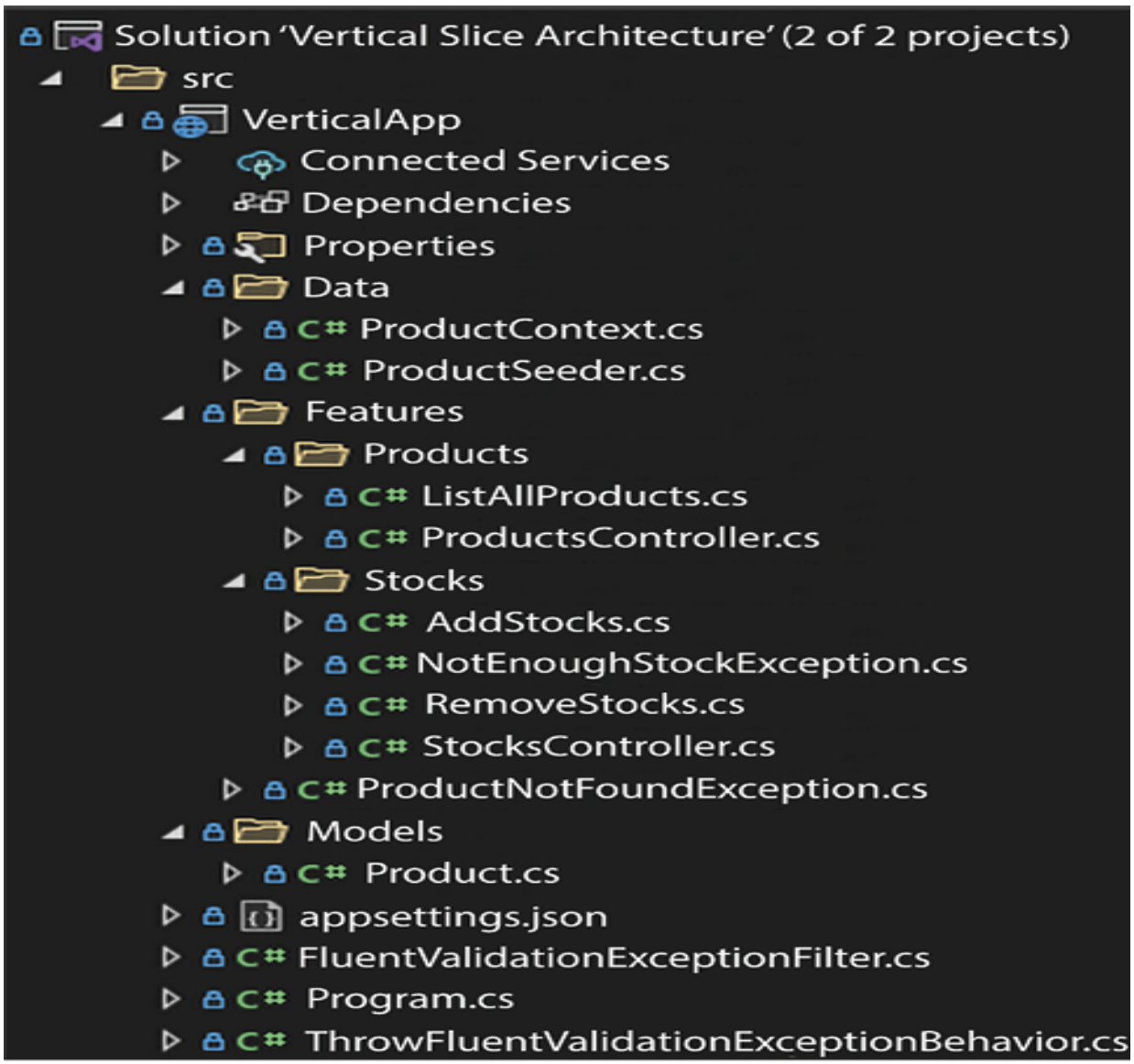


Figure 17.4: Solution Explorer view of the file organization

In this project, we support request validation using **FluentValidation**, a third-party NuGet package. We can also use `System.ComponentModel.DataAnnotations` or any other validation library that we want.

> With FluentValidation, I find it easy to keep the validation within our vertical slice but outside the class we want to validate. The out-of-the-box .NET validation framework, `DataAnnotations`, does the opposite, forcing us to include the validation as metadata on the entities themselves. Both have pros and cons, but FluentValidation is easier to test and extend.

The following code is the `Program.cs` file. The highlighted lines represent registering FluentValidation and scanning the assembly to find validators:

```
var currentAssembly = typeof(Program).Assembly;
var builder = WebApplication.CreateBuilder(args);
builder.Services
    // Plumbing/Dependencies
    .AddAutoMapper(currentAssembly)
    .AddMediatR(o => o.RegisterServicesFromAssembly(currentAssembly))
    .AddSingleton(typeof(IPipelineBehavior<,>), typeof(ThrowFluentValidationExceptionBehavior<,>)
    // Data
    .AddDbContext<ProductContext>(options => options
        .UseInMemoryDatabase("ProductContextMemoryDB")
        .ConfigureWarnings(builder => builder.Ignore(InMemoryEventId.TransactionIgnoredWarning))
    )
    // Web/MVC
    .AddFluentValidationAutoValidation()
    .AddValidatorsFromAssembly(currentAssembly)
    .AddControllers()
;
var app = builder.Build();
app.MapControllers();
using (var seedScope = app.Services.CreateScope())
{
    var db = seedScope.ServiceProvider.GetRequiredService<ProductContext>();
    await ProductSeeder.SeedAsync(db);
}
app.Run();
```

The preceding code adds the bindings we explored in previous chapters, FluentValidation, and the other pieces required to run the application. The highlighted lines register FluentValidation and scan the `currentAssembly` for validator classes. The validators themselves are part of each vertical slice. Now that we covered the organization of the project, let's look at features.

Exploring the RemoveStock feature

In this subsection, we explore the `RemoveStocks` feature with the same logic as in previous samples but organized differently (a.k.a. the difference between architectural styles). Since we use an anemic product model, we moved the add and remove stocks logic from the `Product` class to the `Handler` class. Let's look at the code next. I describe each nested class along the way.The sample starts with the `RemoveStocks` class that contains the feature's nested classes. That helps organize the feature and saves us some headaches about naming collision.

> We could use namespaces instead, but tools like Visual Studio recommend adding a `using` statement and removing the inline namespace. Nowadays, it often automatically adds the using statement, like when pasting code, which is great for many scenarios but inconvenient for this specific one. So using nested classes fixes this.

Here is the `RemoveStocks` class skeleton:

```
using AutoMapper;
using FluentValidation;
using MediatR;
using VerticalApp.Data;
using VerticalApp.Models;
namespace VerticalApp.Features.Stocks;
```

```
public class RemoveStocks
{
    public class Command : IRequest<Result> {/*...*/}
    public class Result {/*...*/}
    public class MapperProfile : Profile {/*...*/}
    public class Validator : AbstractValidator<Command> {/*...*/}
    public class Handler : IRequestHandler<Command, Result> {/*...*/}
}
```

The preceding code showcases that the `RemoveStocks` class contains all the required elements it needs for its specific use case:

- `Command` is the input DTO.
- `Result` is the output DTO.
- `MapperProfile` is the AutoMapper profile that maps feature-specific classes to non-feature-specific classes and vice versa.
- `Validator` validates the input before an instance hits the `Handler` class (the `Command` class).
- `Handler` encapsulates the use case logic.

Next, we explore those nested classes, starting with the `Command` class, which is the **input of the use case** (the request):

```
public class Command : IRequest<Result>
{
    public int ProductId { get; set; }
    public int Amount { get; set; }
}
```

The preceding request contains everything it needs to remove stocks from the inventory and fulfill the operation. The `IRequest<TResult>` interface tells MediatR that the `Command` class is a request and should be routed to its handler. The `Result` class is the return value of that handler and represents the **output of the use case**:

```
public record class Result(int QuantityInStock);
```

The mapper profile is optional and allows encapsulating AutoMapper *maps* related to the use case. The following `MapperProfile` class registers the mapping from a `Product` instance to a `Result` instance:

```
public class MapperProfile : Profile
{
    public MapperProfile()
    {
        CreateMap<Product, Result>();
    }
}
```

The `validator` class is also optional and allows validating the input (`Command`) before it hits the handler; in this case, it ensures the `Amount` value is greater than zero:

```
public class Validator : AbstractValidator<Command>
{
    public Validator()
    {
        RuleFor(x => x.Amount).GreaterThan(0);
    }
}
```

Finally, the most important piece is the `Handler` class, which implements the use case logic:

```
public class Handler : IRequestHandler<Command, Result>
{
    private readonly ProductContext _db;
    private readonly IMapper _mapper;
    public Handler(ProductContext db, IMapper mapper)
    {
        _db = db ?? throw new ArgumentNullException(nameof(db));
```

```
        _mapper = mapper ?? throw new ArgumentNullException(nameof(mapper));
    }
    public async Task<Result> Handle(Command request, CancellationToken cancellationToken)
    {
        var product = await _db.Products.FindAsync(new object[] { request.ProductId }, cancellat
        if (product == null)
        {
            throw new ProductNotFoundException(request.ProductId);
        }
        if (request.Amount > product.QuantityInStock)
        {
            throw new NotEnoughStockException(product.QuantityInStock, request.Amount);
        }
        product.QuantityInStock -= request.Amount;
        await _db.SaveChangesAsync(cancellationToken);
        return _mapper.Map<Result>(product);
    }
}
```

The `Handler` class implements the `IRequestHandler<Command, Result>` interface, which links the `Command`, the `Handler`, and the `Result` classes. The `Handle` method implements the same logic as the previous implementations from *Chapter 14*, *Layering and Clean Architecture*, onward.Now that we have a fully functional use case, let's look at the skeleton of the `StocksController` class that translates the HTTP requests to the MediatR pipeline so our use case gets executed:

```
using MediatR;
using Microsoft.AspNetCore.Mvc;
namespace VerticalApp.Features.Stocks;
[ApiController]
[Route("products/{productId}/")]
public class StocksController : ControllerBase
{
    private readonly IMediator _mediator;
    public StocksController(IMediator mediator)
    {
        _mediator = mediator ?? throw new ArgumentNullException(nameof(mediator));
    }
    [HttpPost("add-stocks")]
    public async Task<ActionResult<AddStocks.Result>> AddAsync(
        int productId,
        [FromBody] AddStocks.Command command
    ) {/*...*/}
    [HttpPost("remove-stocks")]
    public async Task<ActionResult<RemoveStocks.Result>> RemoveAsync(
        int productId,
        [FromBody] RemoveStocks.Command command
    ) {/*...*/}
}
```

In the controller, we inject an `IMediator` into the constructor. We used constructor injection because all actions of this controller use the `IMediator` interface. We have two actions, add and remove stocks.The following code represents the remove stocks action method:

```
[HttpPost("remove-stocks")]
public async Task<ActionResult<RemoveStocks.Result>> RemoveAsync(
    int productId,
    [FromBody] RemoveStocks.Command command
)
{
    try
    {
        command.ProductId = productId;
        var result = await _mediator.Send(command);
        return Ok(result);
    }
    catch (NotEnoughStockException ex)
    {
        return Conflict(new
        {
```

```
            ex.Message,
            ex.AmountToRemove,
            ex.QuantityInStock
        });
    }
    catch (ProductNotFoundException ex)
    {
        return NotFound(new
        {
            ex.Message,
            productId,
        });
    }
}
```

In the preceding code, we read the content of the `RemoveStocks.Command` instance from the body, the action sets the `ProductId` property from the route value, and it sends the `command` object into the MediatR pipeline. From there, MediatR routes the request to its handler before returning the result of that operation with an HTTP `200 OK` status code. One of the differences between the preceding code and previous implementations is that we moved the DTOs to the vertical slice itself. Each vertical slice defines the input, the logic, and the output of its feature, as follows:

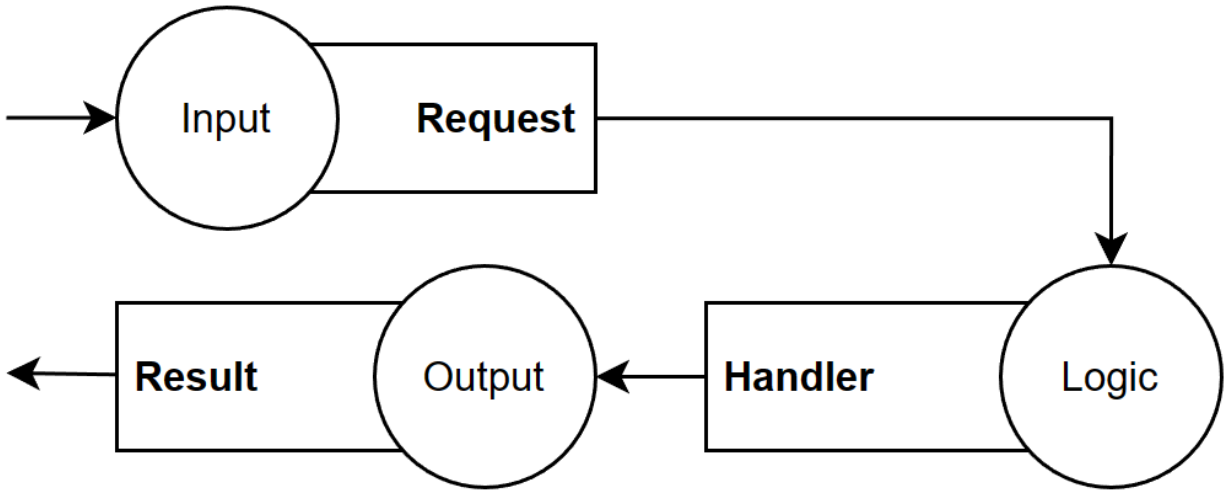


Figure 17.5: Diagram representing the three primary pieces of a vertical slice

When we add input validation, we have the following:

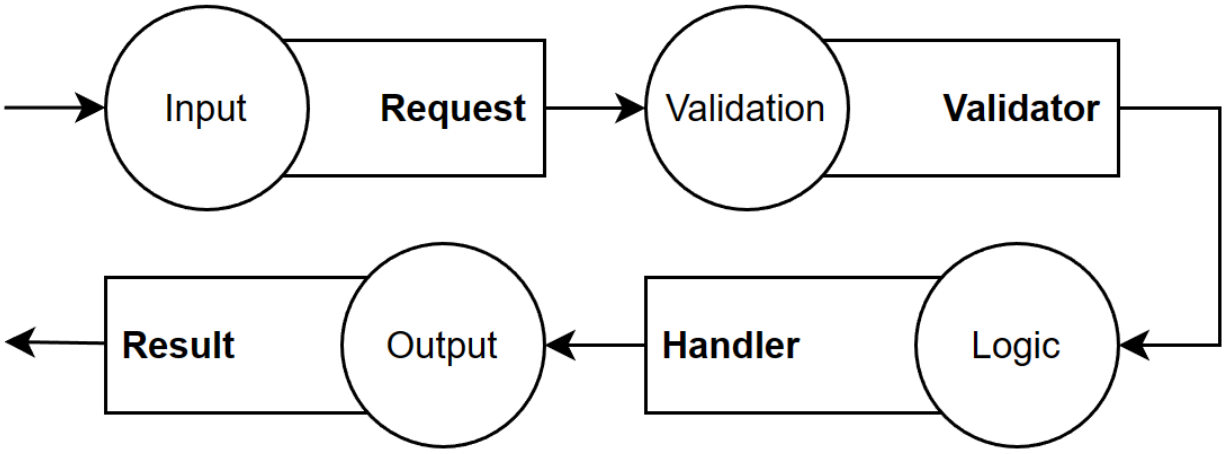


Figure 17.6: Diagram representing the three primary pieces of a vertical slice, with added validation

The controller is a tiny layer between HTTP and our domain, guiding the HTTP requests to the MediatR pipeline and the responses back to HTTP. That thin piece represents the presentation of the API and allows access to the domain logic; the features. When controllers grow, it is often a sign that part of the feature logic is in the wrong place, most likely leading to code that is harder to test because the HTTP and other logic become intertwined.

> We still have the extra line for the `productId` and `try/catch` blocks in the controller's code, but we could eliminate these using custom model binders and exception filters. I left additional resources at the end of the chapter, and we dig deeper into this in the next chapter.

With that in place, it is now straightforward to add new features to the project. Visually, we end up with the following vertical slices (bold), possible vertical expansions (normal), and shared classes (italics):

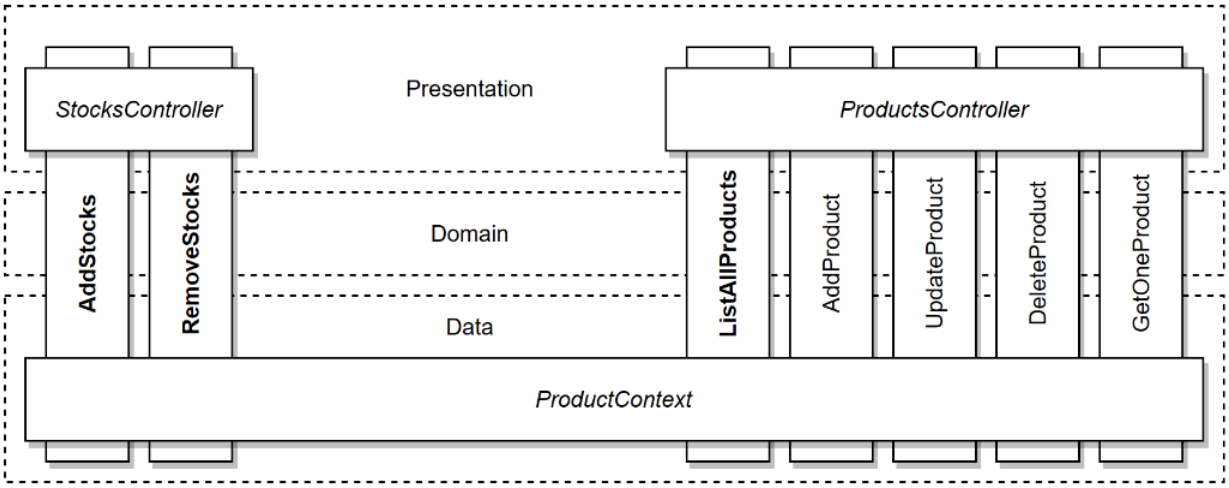


Figure 17.7: Diagram representing the project and possible extensions related to product management

The diagram shows the grouping of the two main areas, products, and stocks. On the products side, I included an expansion that depicts a CRUD-like feature group.In our tiny application, it is tough to divide the data access part into more than one `DbContext` , so `ProductContext` is used by all slices, creating a shared data access layer.

> In other cases, create multiple `DbContext` when possible. This has nothing to do with Vertical Slice Archvaliitecture but is a good practice to divide your domain into smaller bounded contexts.

Think about grouping features when they are cohesive and fit under the same part of the domain.Next, let's test our application.

Testing

For this project, I wrote one integration test per use case outcome, which lowers the number of unit tests required while increasing the level of confidence in the system at the same time. Why? Because we are testing the features themselves instead of many abstracted parts independently. This is grey-box testing.We can also add as many unit tests as we need. This approach helps us write fewer but better feature-oriented tests, diminishing the need for mock-heavy unit tests. Unit tests are practical for validating complex use cases and algorithms faster than integration tests.Let's look at the `StocksTest` class skeleton first:

```
namespace VerticalApp.Features.Stocks;
public class StocksTest
{
    private static async Task SeederDelegate(ProductContext db)
    {
        db.Products.RemoveRange(db.Products.ToArray());
```

```
        await db.Products.AddAsync(new Product(
            id: 4,
            name: "Ghost Pepper",
            quantityInStock: 10
        ));
        await db.Products.AddAsync(new Product(
            id: 5,
            name: "Carolina Reaper",
            quantityInStock: 10
        ));
        await db.SaveChangesAsync();
    }
    public class AddStocksTest : StocksTest
    {
         // omitted test methods
    }
    public class RemoveStocksTest : StocksTest
    {
         // omitted test methods
    }
    public class StocksControllerTest : StocksTest
    {
         // omitted test methods
    }
}
```

The `SeedAsync` method removes all products and inserts two new ones in the in-memory test database so the test methods can run using a predictable data set. The `AddStocksTest` and `RemoveStocksTest` classes contain the test methods for their respective use case. `StocksControllerTest` tests the MVC part. Let's explore the happy path of the `AddStocksTest` class:

```
[Fact]
public async Task Should_increment_QuantityInStock_by_the_specified_amount()
{
    // Arrange
    await using var application = new VerticalAppApplication();
    await application.SeedAsync(SeederDelegate);
    using var requestScope = application.Services.CreateScope();
    var mediator = requestScope.ServiceProvider
        .GetRequiredService<IMediator>();
    // Act
    var result = await mediator.Send(new AddStocks.Command
    {
        ProductId = 4,
        Amount = 10
    });
    // Assert
    using var assertScope = application.Services.CreateScope();
    var db = assertScope.ServiceProvider
        .GetRequiredService<ProductContext>();
    var peppers = await db.Products.FindAsync(4);
    Assert.NotNull(peppers);
    Assert.Equal(20, peppers!.QuantityInStock);
}
```

In the *Arrange* section of the preceding test case, we create an instance of the application, create a scope to simulate an HTTP request, access the EF Core `DbContext`, and then get an `IMediator` instance to act on.In the *Act* block, we send a valid `AddStocks.Command` through the MediatR pipeline.We create a new scope in the *Assert* block then and get a `ProductContext` out of the container. With that `DbContext`, we find the product, ensure it's not null, and validate that the quantity in stock is what we expect. Using a new `ProductContext` ensures we are not dealing with any cached items from the previous operations and the transaction has been saved as expected.With that test case, we know that if a valid command is issued to the mediator, that handler gets executed and successfully increments the stock property by the specified amount.

The `VerticalAppApplication` class inherits from `WebApplicationFactory<TEntryPoint>`, creates a new `DbContextOptionsBuilder<ProductContext>` instance that has a configurable database name, implements a `SeedAsync` method that allows seeding the database, and allows altering the application services. I omitted

the code for brevity reasons, but you can consult the complete source code in the GitHub repository (https://adpg.link/mWep).

Now, we can test the MVC part to ensure the controller is configured correctly. In the `StocksControllerTest` class, the `AddAsync` class contains the following test method:

```
public class AddAsync : StocksControllerTest
{
    [Fact]
    public async Task Should_send_a_valid_AddStocks_Command_to_the_mediator()
    {
        // Arrange
        var mediatorMock = new Mock<IMediator>();
        AddStocks.Command? addStocksCommand = default;
        mediatorMock
            .Setup(x => x.Send(It.IsAny<AddStocks.Command>(), It.IsAny<CancellationToken>()))
            .Callback((IRequest<AddStocks.Result> request, CancellationToken cancellationToken) =
        ;
        await using var application = new VerticalAppApplication(
            afterConfigureServices: services => services
                .AddSingleton(mediatorMock.Object)
        );
        var client = application.CreateClient();
        var httpContent = JsonContent.Create(
            new { amount = 1 },
            options: new JsonSerializerOptions(JsonSerializerDefaults.Web)
        );
        // Act
        var response = await client.PostAsync("/products/5/add-stocks", httpContent);
        // Assert
        Assert.NotNull(response);
        Assert.NotNull(addStocksCommand);
        response.EnsureSuccessStatusCode();
        mediatorMock.Verify(
            x => x.Send(It.IsAny<AddStocks.Command>(), It.IsAny<CancellationToken>()),
            Times.Once()
        );
        Assert.Equal(5, addStocksCommand!.ProductId);
        Assert.Equal(1, addStocksCommand!.Amount);
    }
}
```

The highlighted code of the preceding test case *Arrange* block mocks the `IMediator` and saves what is passed to the `Send` method in the `addStocksCommand` variable. We are using that value in the highlighted code of the *Assert* block. When creating the `VerticalAppApplication` instance, we register the mock with the container to use it instead of the MediatR one, which bypasses the default behavior. We then create an `HttpClient` connected to our in-process application and craft a valid HTTP request to add the stocks we POST in the *Act* section.The *Assert* block code ensures the request was successful, verifies the mock method was hit once, and ensures `AddStocks.Command` was configured correctly.From the first test, we know the MediatR piece works. With this second test in place, we know the HTTP piece works. We are now almost certain that a valid add stocks request will hit the database with those two tests.

> I say “almost certain” because our tests run against an in-memory database, which is different from a real database engine (for example, it has no relational integrity and the like). In case of more complex database operations that affect more than one table or to ensure the correctness of the feature, you can run the tests against a database closer to the production database. For example, we can run the tests against a SQL Server container to spawn and tear down the databases in our CI/CD pipeline easily.

In the test project, I added more tests covering the remove stocks and listing all products’ features, and ensuring AutoMapper configuration correctness. Feel free to browse the code. I omitted them here as they become redundant. The objective is to explore testing a feature almost end to end with very few tests (two for the happy path in this case), and I think we covered that.

Conclusion

The vertical slice project shows how we can remove abstractions while keeping the objects loosely coupled. We also organized the project into features (verticals) instead of layers (horizontals). We leveraged CQS, Mediator, and MVC patterns. Conceptually, the layers are still there; for example, the controllers are part of the presentation layer, but they are not organized that way, making them part of the feature. The sole dependency that crosses all our features is the `ProductContext` class, which makes sense since our model comprises a single class (`Product`). We could, for example, add a new feature that leverages minimal APIs instead of a controller, which would be okay because each slice is independent.We can significantly reduce the number of mocks required by testing each vertical slice with integration tests. That can also significantly lower the number of unit tests, testing features instead of mocked units of code. We should focus on producing features and business value, not the details behind querying the infrastructure or the code itself. We should not neglect the technical aspects either; performance and maintainability are also important characteristics, but reducing the number of abstractions can also make the application easier to maintain and for sure easier to understand.Overall, we explored a modern way to design an application that aligns well with Agile and helps generate value for our customers.Before moving to the summary, let's see how Vertical Slice Architecture can help us follow the **SOLID** principles:

- **S**: Each vertical slice (feature) becomes a cohesive unit that changes as a whole, leading to the segregation of responsibilities per feature. Based on a CQS-inspired approach, each feature splits the application's complexity into commands and queries, leading to multiple small pieces. Each piece handles a part of the process. For example, we can define an input, a validator, a mapper profile, a handler, a result, an HTTP bridge (controller or endpoint), and as many more pieces as we need to craft the slice.
- **O**: We can enhance the system globally by extending the ASP.NET Core, MVC, or MediatR pipelines. We can design the features as we see fit, including respecting the OCP.
- **L**: N/A
- **I**: By organizing features by units of domain-centric use cases, we create many client-specific components instead of general-purpose elements, like layers.
- **D**: The slice pieces depend on interfaces and are tied together using dependency injection. Furthermore, by cutting the less useful abstractions out of the system, we simplify it, making it more maintainable and concise. Having that many pieces of a feature living close to each other makes the system easier to maintain and improves its discoverability.

Next, we look at a few tricks and processes to get started with a bigger application. These are ways that I found work for me and will hopefully work for you too. Take what works for you and leave the rest; we are all different and work differently.

Continuing your journey: A few tips and tricks

The previous project was tiny. It had a shared model that served as the data layer because it was composed of a single class. When building real-world applications, you have more than one class, so I'll give you a good starting point to tackle bigger apps. The idea is to create slices as small as possible, limit interactions with other slices as much as possible, and refactor that code into better code. We cannot remove coupling, so we need to organize it instead, and the key is to centralize that coupling inside a feature.Here is a workflow inspired by TDD, yet less rigid:

1. Write the contracts that cover your feature (input and output).
2. Write one or more integration tests covering your feature, using those contracts; the `Query` or `Command` class (`IRequest`) as input and the `Result` class as output.
3. Implement your `Handler`, `Validator`, `MapperProfile`, and any other bit that needs to be coded. At this point, the code could be a giant `Handler`; it does not matter.
4. Once your integration tests pass, refactor that code by breaking down your giant `Handle` method as needed.
5. Make sure your tests still pass.

During *step 2*, you may also test the validation rules with unit tests. It is way easier and faster to test multiple combinations and scenarios from unit tests, and you don't need to access a database for that. The same also applies to any other parts of your system that are not tied to an external resource.During *step 4*, you may find duplicated logic between features. If that's the case, it is time to encapsulate that logic elsewhere, in a shared place. That could be creating a method in the model, a service class, or any other pattern and technique you know might solve

your duplication of logic problem. Working from isolated features and extracting shared logic will help you design the application. You want to push that shared logic outside of a handler, not the other way around (well, once you have that shared logic, you can use it as needed). Here, I want to emphasize *shared logic*, which means a business rule. When a business rule changes, all consumers of that business rule must also change their behavior. Avoid sharing *similar code* but do share business rules. Remember the DRY principle.What is very important when designing software is to focus on the functional needs, not the technical ones. Your customers and users don't care about the technical stuff; they want results, new features, bug fixes, and improvements. Simultaneously, beware of the technical debt, so don't skip the refactoring step, or your project may get in trouble. This advice applies to all types of architecture.Another advice is to keep all the code that makes a vertical slice as close as possible. You don't have to keep all use case classes in a single file, but I find this helps. Partial classes are a way to split classes into multiple files. If named correctly, Visual Studio will nest them under the primary file. For example, Visual Studio will nest the `MyFeature.Hander.cs` file under the `MyFeature.cs` file, and so on.You can also create a folder hierarchy where the deeper levels share the previous levels. For example, the creation process of a workflow I implemented in an MVC application related to shipments had multiple steps. So I ended up with a hierarchy that looked like the following:

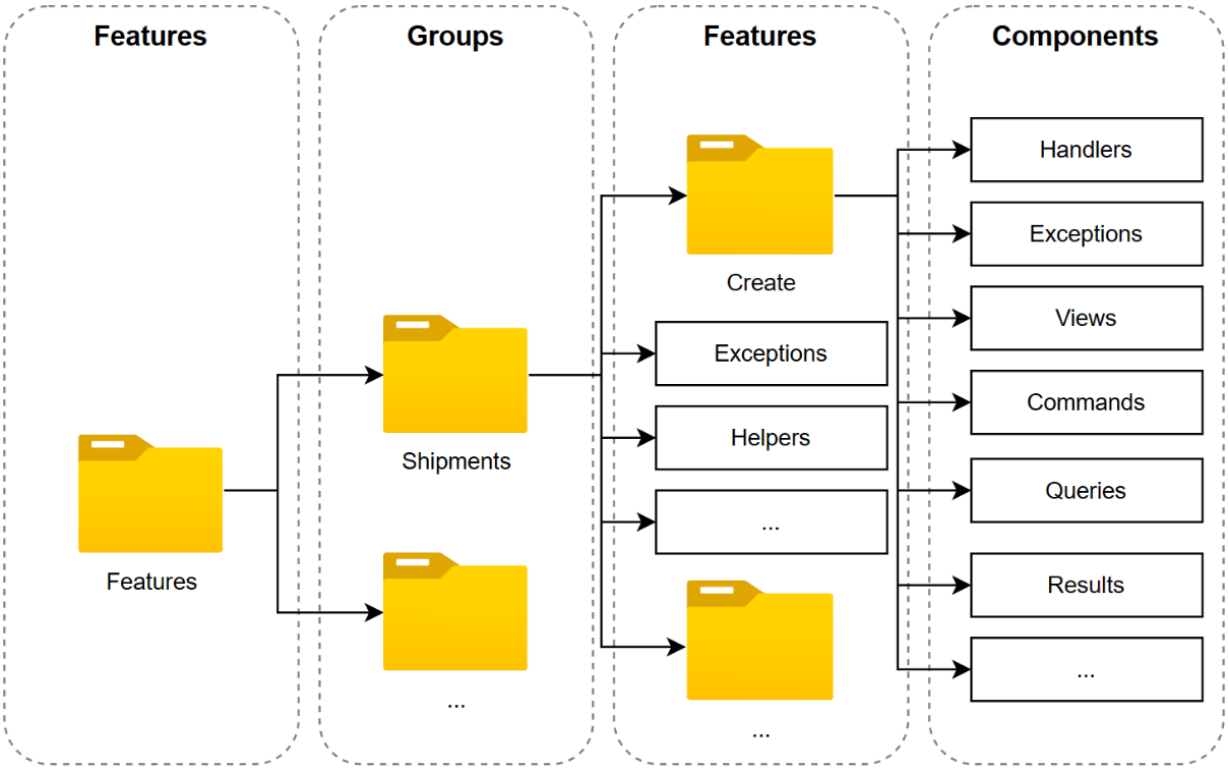


Figure 17.12: The organizational hierarchy of directories and elements

Initially, I coded all the handlers individually. Then I saw patterns emerge, so I encapsulated that shared logic into shared classes. Then I reused some upper-level exceptions, so I moved those up from the `Features/Shipments/Create` folder to the `Features/Shipments` folder. I also extracted a service class to manage shared logic between multiple use cases. Ultimately, I have only the code I need, no duplicated logic, and the collaborators (classes, interfaces) are as close as possible. The coupling between features was minimal, while parts of the system work in synergy (cohesion). Moreover, there is very little to no coupling with other parts of the system. If we compare that result to another type of architecture, such as layering, I would most likely have needed more abstractions, such as repositories, services, and whatnot; the result with Vertical Slice Architecture was cleaner and simpler.The key point here is to code your handlers independently, organize them the best you can, keep an eye open for shared logic and emerging patterns, extract and encapsulate that logic, and try to limit interactions between use cases and slices.

Summary

This chapter overviewed Vertical Slice Architecture, which flips layers by 90°. Vertical Slice Architecture is about writing minimal code to generate maximum value by getting superfluous abstractions and rules out of the equation by relying on the developers' skills and judgment instead.Refactoring is critical in a Vertical Slice Architecture project; success or failure will most likely depend on it. We can also use any patterns with Vertical Slice Architecture. It has lots of advantages over layering with only a few disadvantages. Teams who work in silos (horizontal teams) may need to rethink switching to Vertical Slice Architecture and first create or aim at creating multi-functional teams instead (vertical teams).We replaced the low-value abstraction with commands and queries (CQS-inspired). Those are then routed to their respective `Handler` using the Mediator pattern (helped by MediatR). That allows encapsulating the business logic and decoupling it from its callers. Those commands and queries ensure that each bit of domain logic is centralized in a single location.Of course, if you start with a strong analysis of your problem, you will most likely have a head start, like with any project. Nothing stops you from building and using a robust domain model in your slices. The more requirements you have, the easier the initial project organization will be. To reiterate, all engineering practices that you know still apply.The next chapter simplifies the concept of Vertical Slice Architecture even more by exploring the Request-EndPoint-Response (REPR) pattern using Minimal APIs.

Questions

Let's take a look at a few practice questions:

1. What design patterns can we use in a vertical slice?
2. When using Vertical Slice Architecture, is it true that you must pick a single ORM and stick with it, such as a data layer?
3. What will likely happen if you don't refactor your code and pay the technical debt in the long run?
4. What does cohesion mean?
5. What does tight coupling mean?

Further reading

Here are a few links to build upon what we learned in the chapter:

- For UI implementations, you can look at how Jimmy Bogard upgraded ContosoUniversity:

1. ContosoUniversity on ASP.NET Core with .NET Core: https://adpg.link/UXnr
2. ContosoUniversity on ASP.NET Core with .NET Core and Razor Pages: https://adpg.link/6Lbo

- FluentValidation: https://adpg.link/xXgp
- AutoMapper: https://adpg.link/5AUZ
- MediatR: https://adpg.link/ZQap

Answers

1. Any pattern and technique you know that can help you implement your feature. That's the beauty of Vertical Slice Architecture; you are limited only by yourself.
2. No, you can pick the best tool for the job inside each vertical slice; you don't even need layers.
3. The application will most likely become a Big Ball of Mud and be very hard to maintain, which is not good for your stress level, the product quality, time to market of changes, and so on.
4. Cohesion means elements that should work together as a united whole.
5. Tight coupling describes elements that cannot change independently; that directly depend on one another.

18 Request-EndPoint-Response (REPR) and Minimal APIs

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



This chapter introduces the **Request-EndPoint-Response (REPR)** pattern, which builds on top of Vertical Slice Architecture and CQS. We continue to simplify our codebase to make it even more readable, maintainable, and less abstract, yet still testable.

> We pronounce REPR like “reaper”, which sounds way better than “rer” or “reper”. I must credit Steve "ardalis" Smith for this outstanding pattern name. I left a link to his article in the *Further reading* section.

We leveraged this pattern already, possibly without you knowing about its name. Now it is time to formally introduce it, then assemble a technology stack to make it scalable for a real-world application.We build that solution, then improve it during the chapter to make it better by exploring manual techniques, existing tools, and open-source libraries. The result is not perfect, but we are not done improving this new e-commerce-inspired solution.

> The key to this approach is learning to think about architecture and improve your design skills, so you have the tools to overcome the unique challenges the real world will throw at you!

In this chapter, we cover the following topics:

- Request-EndPoint-Response (REPR) pattern
- Project – REPR—A slice of the real-world

Let's explore the pattern before jumping into a more hands-on example.

Request-EndPoint-Response (REPR) pattern

The Request-EndPoint-Response (REPR) pattern offers a simple approach similar to what we explored in Vertical Slice Architecture which deviates from the traditional Model-View-Controller (MVC) pattern. As we explored in the MVC Chapter, REST APIs don't have views, so we have to distort the concept to make it work. REPR is more appropriate than MVC for building REST APIs in the context of HTTP since each URL is a way to describe how to reach an endpoint (execute an operation), not a controller.

Goal

REPR aims to align our REST APIs to HTTP and treat the inherent request-response concept being the web as a first-class citizen in our application design.On top of this, the REPR pattern using Minimal APIs is well-aligned with Vertical Slice Architecture and facilitates building feature-oriented software instead of layer-heavy applications.

Design

REPR has three components:

- A request that contains the required information for the endpoint to do its work and plays the role of an input DTO.
- An endpoint handler containing the business logic to execute, which is the central piece of this pattern.
- A response that the endpoint returns to the client and plays the role of an output DTO.

You can treat each request as a *Query* or a *Command* as we explore in the CQS and Vertical Slice Architecture chapters.Here's a diagram that represents this concept:

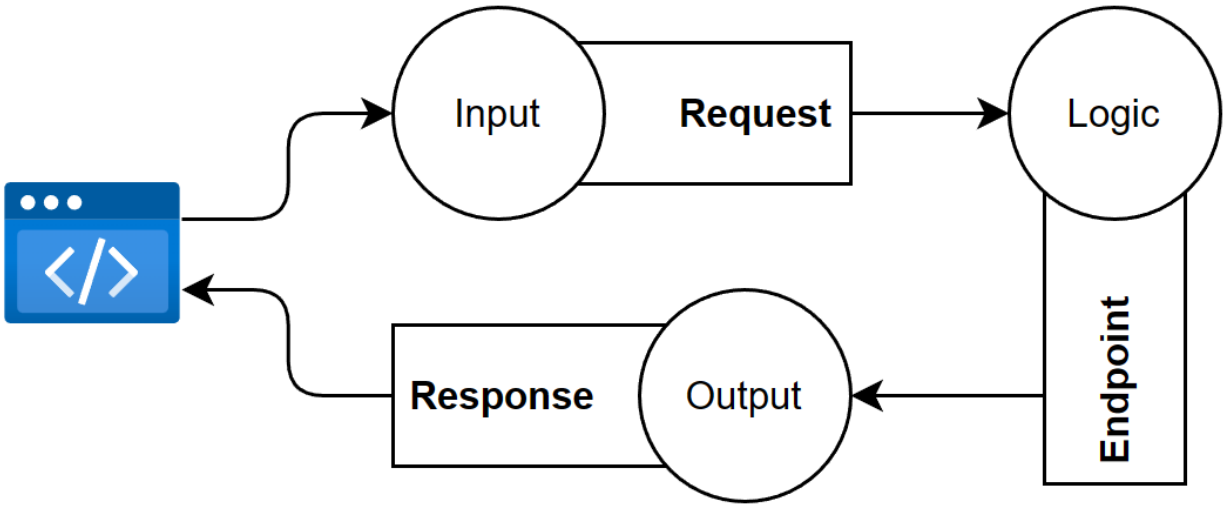


Figure 18.1: a diagram representing the logical flow and the REPR pattern.

The preceding diagram should sound familiar since it resembles what we explored in the Vertical Slice Architecture. However, instead of Request-Handler-Result, we use Request-Endpoint-Response (a.k.a. REPR).Bottom line, a request can be a Query or a Command, then it hits the endpoint that executes the logic and finally returns a response.

> The server returns an HTTP response even when the response's body is empty.

Let's explore an example using Minimal API.

Project – SimpleEndpoint

The SimpleEndpoint project showcases a few simple features and patterns to organize our REPR features without dependencies on external libraries.

Feature: ShuffleText

The first feature gets a string as input, shuffles its content, and returns it:

```
namespace SimpleEndpoint;
public class ShuffleText
{
    public record class Request(string Text);
    public record class Response(string Text);
    public class Endpoint
    {
        public Response Handle(Request request)
        {
```

```
            var chars = request.Text.ToArray();
            Random.Shared.Shuffle(chars);
            return new Response(new string(chars));
        }
    }
}
```

The preceding code leverages the `Random` API and shuffles the `request.Text` property then returns the results wrapped in a `Response` object.Before executing our feature, we must create a minimal API map and register our handler with the container. Here's the Program.cs class that achieves this:

```
using SimpleEndpoint;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<ShuffleText.Endpoint>();
var app = builder.Build();
app.MapGet(
    "/shuffle-text/{text}",
    ([AsParameters] ShuffleText.Request query, ShuffleText.Endpoint endpoint)
        => endpoint.Handle(query)
);
app.Run();
```

The preceding code registers the `ShuffleText.Endpoint` as a singleton so we can inject it in the delegate. The delegate leverages the `[AsParameters]` attribute to bind the route parameter to the `ShuffleText.Request` property. Finally, the logic is straightforward; the endpoint delegate sends the request to the injected endpoint handler and returns the result, which ASP.NET Core serializes to JSON.When we send the following HTTP request:

```
GET https://localhost:7289/shuffle-text/I%20love%20ASP.NET%20Core
```

We receive some gibberish results similar to the following:

```
{
  "text": "eo .e vNrCAT PSElIo"
}
```

This pattern is close to the simplest we can get out of the box. Next, we encapsulate the endpoint itself.

Feature: RandomNumber

This feature generates a number of random numbers between a minimum and a maximum.The first pattern divided the code between the `Program.cs` file and the feature itself. In this pattern, we encapsulate the endpoint delegate into the feature (same file in this case):

```
namespace SimpleEndpoint;
public class RandomNumber
{
    public record class Request(int Amount, int Min, int Max);
    public record class Response(IEnumerable<int> Numbers);
    public class Handler
    {
        public Response Handle(Request request)
        {
            var result = new int[request.Amount];
            for (var i = 0; i < request.Amount; i++)
            {
                result[i] = Random.Shared.Next(request.Min, request.Max);
            }
            return new Response(result);
        }
    }
    public static Response Endpoint([AsParameters] Request query, Handler handler)
        => handler.Handle(query);
}
```

The preceding code is very similar to the first feature. However, the delegate we named `Endpoint` is now part of the feature class (highlighted code). The class that contains the logic is now called `Handler` instead of `Endpoint`. This change makes the full feature live closer together.Nonetheless, we still need to register the dependency with the container and map the endpoint to our delegate like this in the `Program.cs` file:

```
builder.Services.AddSingleton<RandomNumber.Handler>();
// ...
app.MapGet(
    "/random-number/{Amount}/{Min}/{Max}",
    RandomNumber.Endpoint
);
```

The preceding code is routing the request to the `RandomNumber.Endpoint` method.When we send the following HTTP request:

```
https://localhost:7289/random-number/5/0/100
```

We receive a result similar to the following:

```
{
  "numbers": [
    60,
    27,
    78,
    63,
    87
  ]
}
```

We moved more of our feature's code together; however, our code is still divided into two files. Let's explore a way to fix this next.

Feature: UpperCase

This feature transforms the input text to uppercase and returns the result.Our objective is to centralize as much of the code as possible in the `UpperCase` feature class so we control it from a single place. To achieve this, we create the following extension methods (highlighted):

```
namespace SimpleEndpoint;
public static class UpperCase
{
    public record class Request(string Text);
    public record class Response(string Text);
    public class Handler
    {
        public Response Handle(Request request)
        {
            return new Response(request.Text.ToUpper());
        }
    }
    public static IServiceCollection AddUpperCase(this IServiceCollection services)
    {
        return services.AddSingleton<Handler>();
    }
    public static IEndpointRouteBuilder MapUpperCase(this IEndpointRouteBuilder endpoints)
    {
        endpoints.MapGet(
            "/upper-case/{Text}",
            ([AsParameters] Request query, Handler handler)
                => handler.Handle(query)
        );
        return endpoints;
    }
}
```

In the preceding code, we changed the following:

- The `UpperCase` class is static to allow us to create extension methods. Turning the `UpperCase` class into a static class does not hinder our maintainability because we use it as an organizer and do not instantiate it.
- We added the `AddUpperCase` method that registers the dependencies with the container.
- We added the `MapUpperCase` method that creates the endpoint itself.

In the Program.cs file, we can now register our feature like this:

```
builder.Services.AddUpperCase();
// ...
app.MapUpperCase();
```

The preceding code calls our extension methods, which move all the related code into the `UpperCase` class but its connection with ASP.NET Core.

> I find this approach elegant and very clean for a no-dependency project. Of course, we could design this in a million different ways, use an existing library to help us, scan the assembly and auto-register our features, and more.
>
> > You can use this pattern to build a real-world application. I'd suggest creating an `AddFeatures` and a `MapFeatures` extension method that register all the features instead of cluttering the `Program.cs` file, but besides a few final organization touch, this is a robust enough pattern. We explore more of this in the next project.

When we send the following HTTP request:

```
GET https://localhost:7289/upper-case/I%20love%20ASP.NET%20Core
```

We receive the following response:

```
{
  "text": "I LOVE ASP.NET CORE"
}
```

Now that we explored REPR and how to encapsulate our REPR features in several ways, we are almost ready to explore a larger project.

Conclusion

Creating a feature-based design using Minimal APIs, the REPR pattern, and no external dependencies is possible and simple. We organized our project in different ways. Each feature comprises a request, a response, and a handler attached to an endpoint.

> We can combine the handler and the endpoint to make it a three-component pattern. What I like about having a distinct handler is that we can reuse the handler in a non-HTTP context; say, we could create a CLI tool in front of the application and reuse the same logic. It all depends on what we are building.

Let's see how the REPR pattern can help us follow the **SOLID** principles:

- **S**: Each piece has a single responsibility, and all pieces are centralized under a feature for ease of navigation, making this pattern a perfect SRP ally.
- **O**: Using an approach similar to what we did with the `UpperCase` feature, we can change the feature's behavior without affecting the rest of the codebase.
- **L**: N/A
- **I**: The REPR pattern divides a feature into three smaller interfaces: the request, the endpoint, and the response.
- **D**: N/A

Now that we familiarized ourselves with the REPR pattern, it is time to explore a larger project, including exception handling and grey-box testing.

Project – REPR—A slice of the real-world

Context: this project slightly differs from the previous ones about products and stocks. We remove the inventory from the product, add a unit price, and create a barebone shopping basket as a foundation for an e-commerce application. The inventory management became so complex that we had to extract and handle it separately (not included here).By using the REPR pattern, Minimal APIs, and what we learned with Vertical Slice Architecture, we analyzed that the application contains two major areas:

- A product catalog
- A shopping cart

For this first iteration, we kept the management of the products away from the application, supporting only the following features:

- Listing all products
- Fetching the details of a product

As for the shopping cart, we kept it to a minimum. The basket only persists the `Id` of the products in the cart and its quantity. The basket does not support any more advanced use cases. Here are the operations it supports:

- Add an item to the cart
- Fetch the items that are in the cart
- Remove an item from the cart
- Update the quantity of an item in the cart

For now, the shopping cart is not aware of the product catalog.

> We improve the application in *Chapter 19*, *Introduction to Microservices Architecture*, and *Chapter 20*, *Modular Monolith*.

Let's assemble the stack we'll build upon next.

Assembling our stack

I want to keep the project as barebone as possible, using Minimal APIs, yet, we don't have to struggle with manually implementing every single concern ourselves. Here are the tools we will use to build this project:

- *ASP.NET Core Minimal API* as our backbone.
- *FluentValidation* as our validation framework.
- *FluentValidation.AspNetCore.Http* connects FluentValidation into Minimal API.
- *Mapperly* is our mapping framework.
- *ExceptionMapper* helps us handle exceptions globally, shifting our pattern to error management.
- *EF Core* (InMemory) as our ORM.

From a terminal window, we can use the CLI to install the packages:

```
dotnet add package FluentValidation.AspNetCore
dotnet add package ForEvolve.ExceptionMapper
dotnet add package ForEvolve.FluentValidation.AspNetCore.Http
dotnet add package Microsoft.EntityFrameworkCore.InMemory
dotnet add package Riok.Mapperly
```

We know most of those pieces and will dig deeper into the new ones in time. Meanwhile, let's explore the structure of the project.

Dissecting the code structure

The directory structure is very similar to what we explored in Vertical Slice Architecture. The root of the project contains the `Program.cs` file and a `Features` directory that holds the features or slices. The following diagram represents the features:

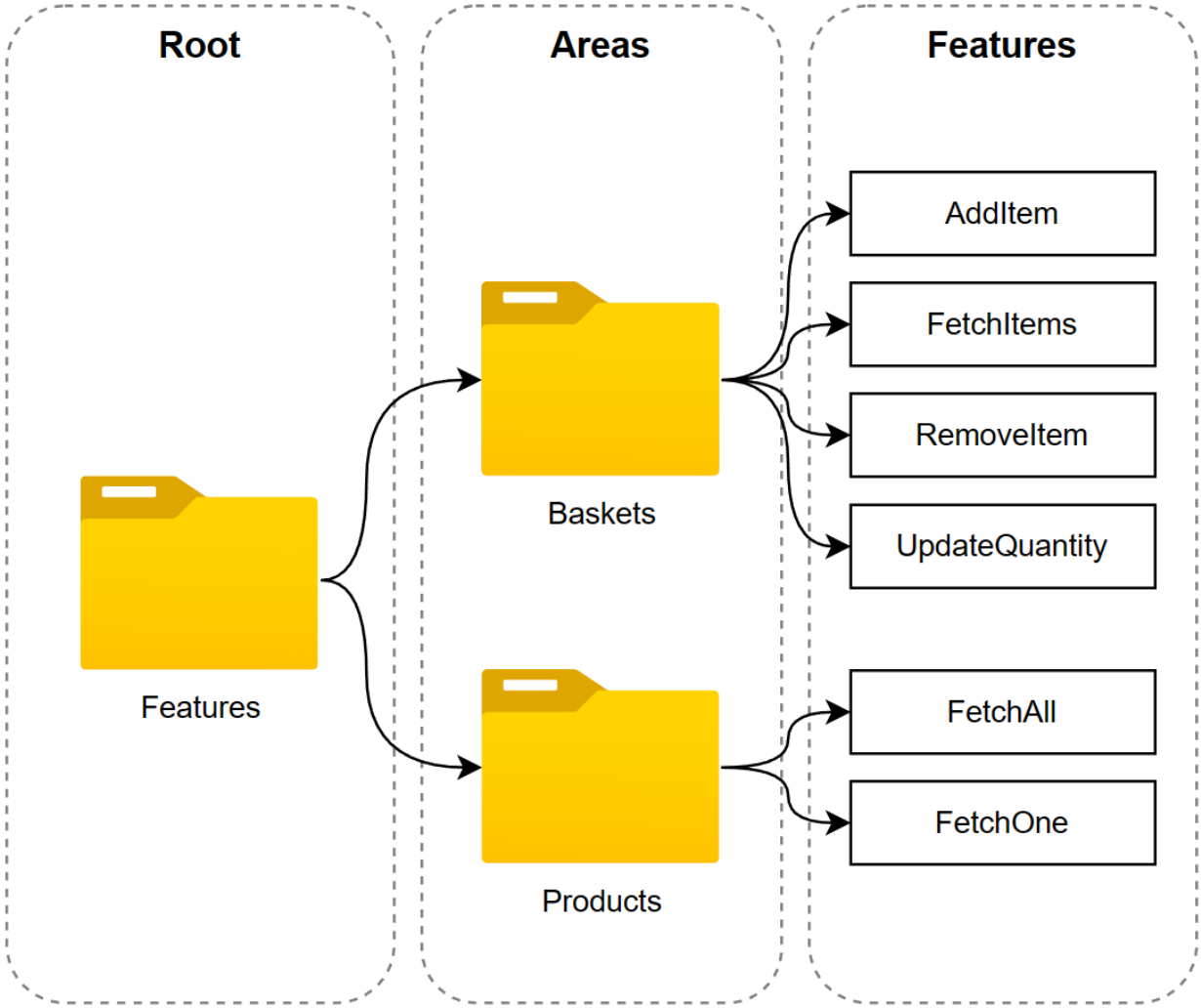


Figure 18.2: The project's directory structure that represents the features' hierarchical relationships.

The features inside each area share a cohesive bond and some pieces of code (coupling), while the two areas are entirely disconnected (loosely coupled).The `Program.cs` file is very light and only bootstraps the application:

```
using Web.Features;
var builder = WebApplication.CreateBuilder(args);
builder.AddFeatures();
var app = builder.Build();
app.MapFeatures();
await app.SeedFeaturesAsync();
app.Run();
```

The highlighted lines are extension methods defined in the `Features` class (located under the `Features` folder), which cascades the responsibility of registering the dependencies with the container, mapping the endpoints, and seeding the database to each area. Here's the skeleton of the class:

```
using FluentValidation;
using FluentValidation.AspNetCore;
using System.Reflection;
```

```
namespace Web.Features;
public static class Features
{
    public static IServiceCollection AddFeatures(
        this WebApplicationBuilder builder){}
    public static IEndpointRouteBuilder MapFeatures(
        this IEndpointRouteBuilder endpoints){}
    public static async Task SeedFeaturesAsync(
        this WebApplication app){}
}
```

Let's now explore the `AddFeatures` method:

```
public static IServiceCollection AddFeatures(this WebApplicationBuilder builder)
{
    // Register fluent validation
    builder.AddFluentValidationEndpointFilter();
    return builder.Services
        .AddFluentValidationAutoValidation()
        .AddValidatorsFromAssembly(Assembly.GetExecutingAssembly())
        // Add features
        .AddProductsFeature()
        .AddBasketsFeature()
    ;
}
```

The `AddFeatures` method registers FluentValidation and the Minimal API filters to validate our endpoints (the highlighted line). Each slice defines its own configuration methods, like the `AddProductsFeature` and `AddBasketsFeature` methods. We will come back to those. Meanwhile, let's explore the `MapFeatures` method:

```
public static IEndpointRouteBuilder MapFeatures(this IEndpointRouteBuilder endpoints)
{
    var group = endpoints
        .MapGroup("/")
        .AddFluentValidationFilter();
    ;
    group
        .MapProductsFeature()
        .MapBasketsFeature()
    ;
    return endpoints;
}
```

The `MapFeatures` method creates a root route group, adds the *FluentValidation* filter to it so the validation applies to all endpoints in this group, then it calls the `MapProductsFeature` and `MapBasketsFeature` methods that map their features into the group. Finally, the `SeedFeaturesAsync` method seeds the database using the feature extension methods:

```
public static async Task SeedFeaturesAsync(this WebApplication app)
{
    using var scope = app.Services.CreateScope();
    await scope.SeedProductsAsync();
    await scope.SeedBasketAsync();
}
```

With those building blocks in place, the program starts, adds features, and registers endpoints. Afterward, each category of features—products and baskets—cascades the calls, letting each feature registers its pieces. Next are a few diagrams representing the call hierarchy from the `Program.cs` file. Let's start with the `AddFeatures` method:

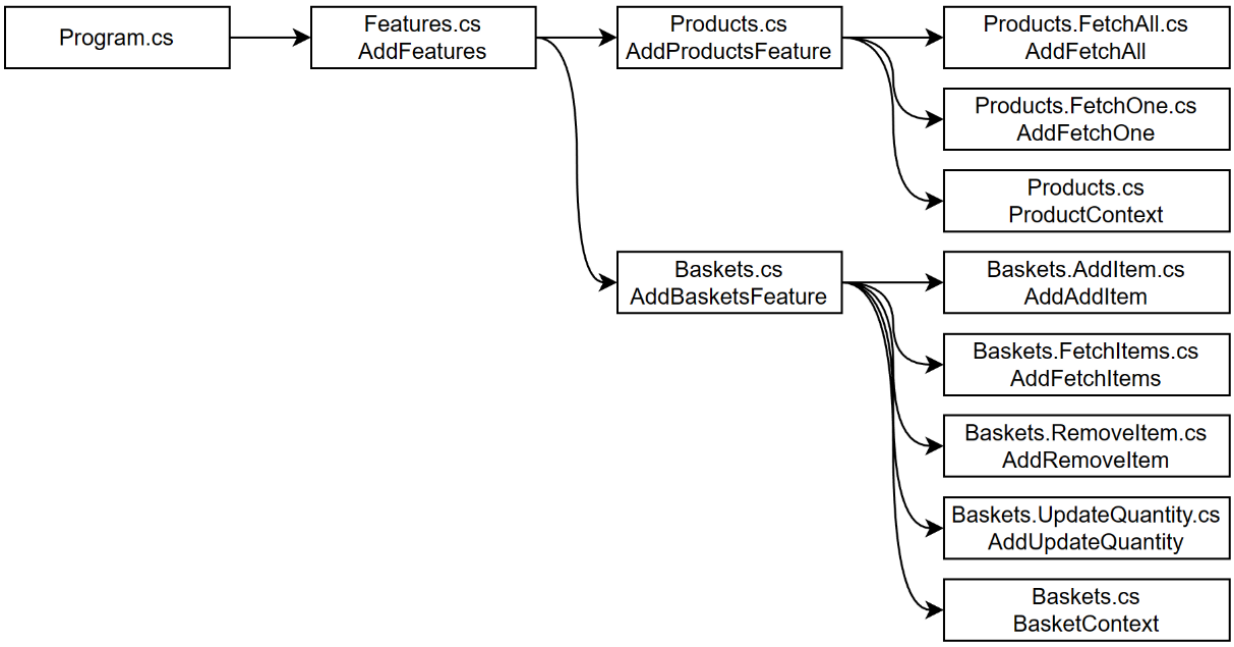


Figure 18.3: the call hierarchy of the AddFeatures method.

The preceding diagram showcases the division of responsibilities where each piece aggregates its sub-parts or registers its dependencies.A similar flow happens from the `MapFeatures` method:

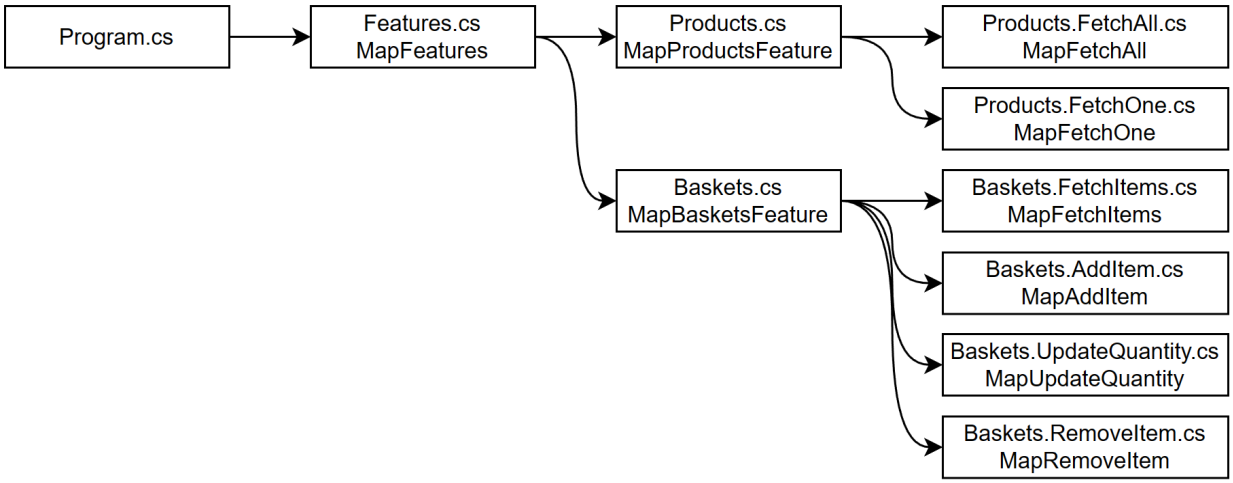


Figure 18.4: the call hierarchy of the MapFeatures method.

Finally, the `SeedFeaturesAsync` method utilizes a similar approach to seed the in-memory database:

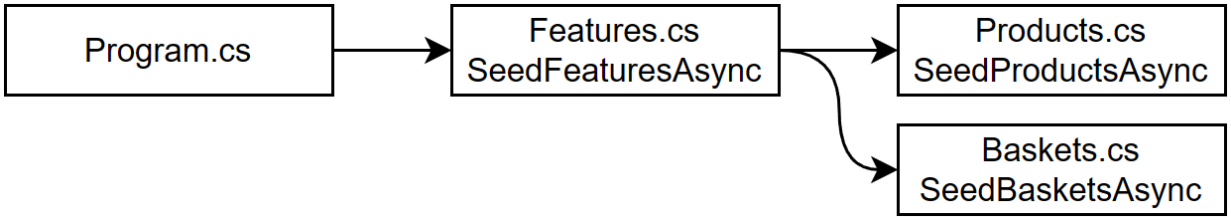


Figure 18.5: the call hierarchy of the SeedFeaturesAsync method.

These diagrams showcase the entry point (`Program.cs`) sending a request to each feature so that every piece handles itself.

> In a real project using an actual database, you do not want to seed the database this way. In this case, it works because each time we start the project, the database is empty because it only lives for the time the program runs—it lives in memory. There are numerous strategies to seed your data sources in real life, from executing a SQL script to deploying a Docker container that runs only once.

Now that we explored the high-level view of the program, it is time to dig into a feature and explore how it works.

Exploring the shopping basket

This section explores the `AddItem` and `FetchItems` features of the shopping basket slice. The slice is completely decoupled from the `Products` slice, and does not know the products themselves. All it knows is how to accumulate product identifiers and quantities and associate those with a customer. We address this problem later.

> There are no customer features and no authentication to keep the project simple.

The code of the `Features/Baskets/Baskets.cs` file powers the shopping basket features. Here's the skeleton:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Diagnostics;
namespace Web.Features;
public static partial class Baskets
{
    // Baskets.cs
    public record class BasketItem(int CustomerId, int ProductId, int Quantity);
    public class BasketContext : DbContext {}
    public static IServiceCollection AddBasketsFeature(this IServiceCollection services) {}
    public static IEndpointRouteBuilder MapBasketsFeature(this IEndpointRouteBuilder endpoints) {
    public static Task SeedBasketsAsync(this IServiceScope scope) {}
    // Baskets.AddItem.cs
    public partial class AddItem {}
    public static IServiceCollection AddAddItem(this IServiceCollection services) {}
    public static IEndpointRouteBuilder MapAddItem(this IEndpointRouteBuilder endpoints) {}
    // Baskets.FetchItems.cs
    public partial class FetchItems {}
    public static IServiceCollection AddFetchItems(this IServiceCollection services) {}
    public static IEndpointRouteBuilder MapFetchItems(this IEndpointRouteBuilder endpoints) {}
    // Baskets.RemoveItem.cs
    public partial class RemoveItem {}
    public static IServiceCollection AddRemoveItem(this IServiceCollection services) {}
    public static IEndpointRouteBuilder MapRemoveItem(this IEndpointRouteBuilder endpoints) {}
    // Baskets.UpdateQuantity.cs
    public partial class UpdateQuantity {}
    public static IServiceCollection AddUpdateQuantity(this IServiceCollection services) {}
    public static IEndpointRouteBuilder MapUpdateQuantity(this IEndpointRouteBuilder endpoints) {
}
```

The highlighted code of the preceding block contains the `BasketItem` data model and the `BasketContext` EF Core `DbContext` , which all basket features share. It also includes the three methods that register and make the features work (`AddBasketsFeature` , `MapBasketsFeature` , and `SeedBasketsAsync`). The other methods and classes are divided into several files. We explore a few of them in the chapter.

> We used the `partial` modifier to split the nested classes into multiple files. We made the class `static` to create extension methods in it.

The `BasketItem` class allows us to persist a simple shopping cart to the database:

```
public record class BasketItem(
    int CustomerId,
    int ProductId,
    int Quantity
);
```

The `BasketContext` class configures the primary key of the `BasketItem` class and exposes the `Items` property (highlighted):

```
public class BasketContext : DbContext
{
    public BasketContext(DbContextOptions<BasketContext> options)
        : base(options) { }
    public DbSet<BasketItem> Items => Set<BasketItem>();
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder
            .Entity<BasketItem>()
            .HasKey(x => new { x.CustomerId, x.ProductId })
        ;
    }
}
```

The `AddBasketsFeature` method registers each feature and the `BasketContext` with the IoC container:

```
public static IServiceCollection AddBasketsFeature(this IServiceCollection services)
{
    return services
        .AddAddItem()
        .AddFetchItems()
        .AddRemoveItem()
        .AddUpdateQuantity()
        .AddDbContext<BasketContext>(options => options
            .UseInMemoryDatabase("BasketContextMemoryDB")
            .ConfigureWarnings(builder => builder.Ignore(InMemoryEventId.TransactionIgnoredWarni
        )
    ;
}
```

Besides the `AddDbContext` method, the `AddBasketsFeature` delegates the registration of dependencies to each feature. We explore the highlighted ones shortly. The EF Core code registers the in-memory provider that serves the `BasketContext` .Next, the `MapBasketsFeature` method maps the endpoints:

```
public static IEndpointRouteBuilder MapBasketsFeature(this IEndpointRouteBuilder endpoints)
{
    var group = endpoints
        .MapGroup(nameof(Baskets).ToLower())
        .WithTags(nameof(Baskets))
    ;
    group
        .MapFetchItems()
        .MapAddItem()
        .MapUpdateQuantity()
        .MapRemoveItem()
    ;
    return endpoints;
}
```

The preceding code creates a group, naming it `baskets` , making its endpoints accessible using the `/baskets` URL prefix. We also tag the group “Baskets” to leverage an OpenAPI generator in the future. Then the method uses a similar pattern to the `AddBasketsFeature` method and delegates the endpoint mapping to the features.

> Have you noticed that the method returns the `endpoints` object directly? This allows us to chain the feature mapping. In another scenario, we could return the `group` object (`RouteGroupBuilder` instance) to let the caller further configure the group. What we build is always about the needs and objectives.

Finally, the `SeedBasketsAsync` method does nothing; we do not create any shopping cart when starting the program, unlike the `Products` slice:

```
public static Task SeedBasketsAsync(this IServiceScope scope)
{
```

```
    return Task.CompletedTask;
}
```

> We could have omitted the preceding method. I left it so we follow a linear pattern between the features. Such a linear pattern makes it easier to understand and learn. It also allows identifying the recurring pieces we could work on to automate the registration process.

Now that we have covered the shared pieces, let's add data to our shopping basket.

AddItem feature

The role of the `AddItem` feature is to create a `BasketItem` object and persist it in the database. To achieve this, we leverage the REPR pattern. Inspired by the preceding few chapters, we name the request `Command` (CQS pattern), add a mapper object using Mapperly, and leverage FluentValidation to ensure the request is valid. Here's the skeleton of the `AddItem` class:

```
using FluentValidation;
using Microsoft.EntityFrameworkCore;
using Riok.Mapperly.Abstractions;
namespace Web.Features;
public partial class Baskets
{
    public partial class AddItem
    {
        public record class Command(
            int CustomerId,
            int ProductId,
            int Quantity
        );
        public record class Response(
            int ProductId,
            int Quantity
        );
        [Mapper]
        public partial class Mapper {}
        public class Validator : AbstractValidator<Command> {}
        public class Handler {}
    }
    public static IServiceCollection AddAddItem(this IServiceCollection services) {}
    public static IEndpointRouteBuilder MapAddItem(this IEndpointRouteBuilder endpoints) {}
}
```

The preceding code contains all the necessary pieces of the feature:

- The request (the `Command` class).
- The response (the `Response` class).
- The endpoint (the `MapAddItem` method pointing the requests to the `Handler` class).
- A mapper object that has Mapperly generate the mapping code for us.
- A validator class that ensures the input we receive is valid.
- The `AddAddItem` method registers its services with the IoC container.

Let's start with the `AddAddItem` method that registers the feature's services:

```
public static IServiceCollection AddAddItem(this IServiceCollection services)
{
    return services
        .AddScoped<AddItem.Handler>()
        .AddSingleton<AddItem.Mapper>()
    ;
}
```

Then the `MapAddItem` method routes the appropriate POST requests with a valid `Command` object in its body to the `Handler` class:

```
public static IEndpointRouteBuilder MapAddItem(
    this IEndpointRouteBuilder endpoints)
{
    endpoints.MapPost(
        "/",
        async (AddItem.Command command, AddItem.Handler handler, CancellationToken cancellationT
        {
            var result = await handler.HandleAsync(
                command,
                cancellationToken
            );
            return TypedResults.Created(
                $"/products/{result.ProductId}",
                result
            );
        }
    );
    return endpoints;
}
```

The `Command` instance is a copy of the `BasketItem` class, while the response only returns the `ProductId` and `Quantity` properties. The highlighted lines represent the endpoint handing off the `Command` object to the use case `Handler` class.

We could write the `Handler` code in the delegate, which would make unit testing the delegate very hard.

The `Handler` class is the glue of the feature:

```
public class Handler
{
    private readonly BasketContext _db;
    private readonly Mapper _mapper;
    public Handler(BasketContext db, Mapper mapper)
    {
        _db = db ?? throw new ArgumentNullException(nameof(db));
        _mapper = mapper ?? throw new ArgumentNullException(nameof(mapper));
    }
    public async Task<Response> HandleAsync(Command command, CancellationToken cancellationToken)
    {
        var itemExists = await _db.Items.AnyAsync(
            x => x.CustomerId == command.CustomerId && x.ProductId == command.ProductId,
            cancellationToken: cancellationToken
        );
        if (itemExists)
        {
            throw new DuplicateBasketItemException(command.ProductId);
        }
        var item = _mapper.Map(command);
        _db.Add(item);
        await _db.SaveChangesAsync(cancellationToken);
        var result = _mapper.Map(item);
        return result;
    }
}
```

The preceding code contains the business logic of the feature by ensuring the item is not already in the basket. If it is, it throws a `DuplicateBasketItemException`. Otherwise, it saves the item to the database. It then returns a `Response` object.

Each customer (`CustomerId`) can have each product (`ProductId`) once in its cart (composite primary key), which is why we test for this condition.

The handler leveraged the `Mapper` class:

```
[Mapper]
public partial class Mapper
{
    public partial BasketItem Map(Command item);
```

```
    public partial Response Map(BasketItem item);
}
```

Implicitly, the `Command` object was validated using the following `Validator` class:

```
public class Validator : AbstractValidator<Command>
{
    public Validator()
    {
        RuleFor(x => x.CustomerId).GreaterThan(0);
        RuleFor(x => x.ProductId).GreaterThan(0);
        RuleFor(x => x.Quantity).GreaterThan(0);
    }
}
```

> As a reminder, in the `Features.cs` file, we called the `AddFluentValidationFilter` method on the root route group, letting the `FluentValidationEndpointFilter` class validate the inputs for us using the `Validator` class.

With this in place, we can send the following HTTP request:

```
POST https://localhost:7252/baskets
Content-Type: application/json
{
    "customerId": 1,
    "productId": 3,
    "quantity": 10
}
```

The endpoint responds with the following:

```
{
  "productId": 3,
  "quantity": 10
}
```

And has the following HTTP header:

```
Location: /products/3
```

To recap, here's what happens:

1. ASP.NET Core routes the request to the delegate we registered in the `MapAddItem` method.
2. The validation middle runs an `AddItem.Validator` object against the `AddItem.Command` sent to the endpoint. The request is valid.
3. The `HandleAsync` method of the `AddItem.Handler` class is executed.
4. Assuming the item is not already in the customer's basket, it is added to the database.
5. The `HandleAsync` method returns a `Response` object to the delegate.
6. The delegate returns a `201 Create` status code with the `Location` header set to the URL of the product that got added.

As the preceding list depicts, the process is quite simple; a request gets in, the business logic is executed (endpoint), then a response goes out: REPR.

> There are a few more pieces, but they save us the trouble of object mapping and validation. Those pieces are optional; you can conceive your own stack with more or less pieces in it.

On top of the feature code, we also have a few tests to assess that the business logic remains correct over time. We cover those under the *Grey-box testing* section. Meanwhile, let's look at the `FetchItems` feature.

FetchItems feature

Now that we know the pattern, this feature should be faster to cover. It allows a client to retrieve the shopping basket of the specified customer using the following request:

```
public record class Query(int CustomerId);
```

The client expects a collection of items in the response:

```
public record class Response(IEnumerable<Item> Items) : IEnumerable<Item>
{
    public IEnumerator<Item> GetEnumerator()
        => Items.GetEnumerator();
    IEnumerator IEnumerable.GetEnumerator()
        => ((IEnumerable)Items).GetEnumerator();
}
public record class Item(int ProductId, int Quantity);
```

Since the client knows about the customer, it does not need the endpoint to return the `CustomerId` property, which is why the `Item` class only has two of the `BasketItem` properties.Here are the `Mapper` and the `Validator` classes, which should be self-explanatory at this point:

```
[Mapper]
public partial class Mapper
{
    public partial Response Map(IQueryable<BasketItem> items);
}
public class Validator : AbstractValidator<Query>
{
    public Validator()
    {
        RuleFor(x => x.CustomerId).GreaterThan(0);
    }
}
```

Then, a last piece of plumbing is the `AddFetchItems` method which registers the feature's services with the containers:

```
public static IServiceCollection AddFetchItems(this IServiceCollection services)
{
    return services
        .AddScoped<FetchItems.Handler>()
        .AddSingleton<FetchItems.Mapper>()
    ;
}
```

Now to the endpoint itself, forwarding the `FetchItems.Query` object to a `FetchItems.Handler` instance:

```
public static IEndpointRouteBuilder MapFetchItems(this IEndpointRouteBuilder endpoints)
{
    endpoints.MapGet(
        "/{CustomerId}",
        ([AsParameters] FetchItems.Query query, FetchItems.Handler handler, CancellationToken ca
            => handler.HandleAsync(query, cancellationToken)
    );
    return endpoints;
}
```

The preceding code is simpler than the `AddItem` feature because it serializes the handler's response directly as a 200 OK status code without transforming it.Finally, the `Handler` class itself:

```
public class Handler
{
    private readonly BasketContext _db;
    private readonly Mapper _mapper;
    public Handler(BasketContext db, Mapper mapper)
    {
        _db = db ?? throw new ArgumentNullException(nameof(db));
        _mapper = mapper ?? throw new ArgumentNullException(nameof(mapper));
```

```
    }
    public async Task<Response> HandleAsync(Query query, CancellationToken cancellationToken)
    {
        var items = _db.Items.Where(x => x.CustomerId == query.CustomerId);
        await items.LoadAsync(cancellationToken);
        var result = _mapper.Map(items);
        return result;
    }
}
```

The preceding code loads all the items associated with the specified customer from the database and returns them. If there are no items, the client receives an empty array.That's it; we can now send the following HTTP request and hit the endpoint:

```
GET https://localhost:7252/baskets/1
```

Assuming we added an item to the basket, we should receive a response similar to the following:

```
[
  {
    "productId": 3,
    "quantity": 10
  }
]
```

We now have a working shopping basket!

> You can explore the other features in the codebase available on GitHub (https://adpg.link/ikAn). All features have tests and are functional.

Next, we look at exception handling.

Managing exception handling

The `AddItem` feature throws a `DuplicateBasketItemException` when the product is already in the basket. However, when that happens, the server returns an error that resembles the following (partial output):

```
Web.Features.DuplicateBasketItemException: The product '3' is already in your shopping cart.
   at Web.Features.Baskets.AddItem.Handler.HandleAsync(Command command, CancellationToken cancel
   at Web.Features.Baskets.<>c.<<MapAddItem>b__2_0>d.MoveNext() in C18\REPR\Web\Features\Baskets\
--- End of stack trace from previous location ---
```

That error is ugly and impractical for a client calling the API. To circumvent this, we can add a try-catch somewhere and treat each exception individually, or we can use a middleware to catch the exceptions and normalize their output.Managing exceptions one by one is tedious and error-prone. On the other hand, centralizing exception management and treating them as a cross-cutting concern transforms the tedious mechanism into a new tool to leverage. Moreover, it ensures that the API always returns the errors in the same format with no additional effort.Let's program a basic middleware.

Creating an exception handler middleware

A middleware in ASP.NET Core is executed as part of the pipeline and can run before and after the execution of an endpoint.When an exception occurs, the request is re-executed in a parallel pipeline, allowing different middleware to manage the error flow.To create a middleware, we must implement an `InvokeAsync` method. The easiest way to do this is by implementing the `IMiddleware` interface. You can add middleware types to the default or exception-handling alternate pipelines.The following code represents a basic exception-handling middleware:

```
using Microsoft.AspNetCore.Diagnostics;
namespace Web;
public class MyExceptionMiddleware : IMiddleware
{
    public async Task InvokeAsync(HttpContext context, RequestDelegate next)
```

```
    {
        var exceptionHandlerPathFeature = context.Features
            .Get<IExceptionHandlerFeature>() ?? throw new NotSupportedException();
        var exception = exceptionHandlerPathFeature.Error;
        await context.Response.WriteAsJsonAsync(new
        {
            Error = exception.Message
        });
        await next(context);
    }
}
```

The middleware fetches the `IExceptionHandlerFeature` to access the error and outputs an object containing the error message (ASP.NET Core manages this feature). If the feature is unavailable, the middleware throws a `NotSupportedException`, which rethrows the original exception.

Any type of exception a middleware of the alternate pipeline throws will rethrow the original exception.

If any, the highlighted code executes the next middleware in the pipeline. These pipelines are like a chain of responsibilities but with a different objective.To register the middleware, we must first add it to the container:

```
builder.Services.AddSingleton<MyExceptionMiddleware>();
```

Then, we must register it as part of the exception-handling alternate pipeline:

```
app.UseExceptionHandler(errorApp =>
{
    errorApp.UseMiddleware<MyExceptionMiddleware>();
});
```

We could also register more middleware or create them inline, like this:

```
app.UseExceptionHandler(errorApp =>
{
    errorApp.Use(async (context, next) =>
    {
        var exceptionHandlerPathFeature = context.Features
            .Get<IExceptionHandlerFeature>() ?? throw new NotSupportedException();
        var logger = context.RequestServices
            .GetRequiredService<ILoggerFactory>()
            .CreateLogger("ExceptionHandler");
        var exception = exceptionHandlerPathFeature.Error;
        logger.LogWarning(
            "An exception occurred: {message}",
            exception.Message
        );
        await next(context);
    });
    errorApp.UseMiddleware<MyExceptionMiddleware>();
});
```

The possibilities are vast.

Now, if we try to add a duplicated item to the basket, we get a *500 Internal Server Error* with the following body:

```
{
  "error": "The product \u00273\u0027 is already in your shopping cart."
}
```

This response is more elegant than before and easier to handle for the clients. We could also alter the status code in the middleware. However, customizing this middleware would take many pages, so we leverage an existing library instead.

Exception handling using ExceptionMapper

The `ForEvolve.ExceptionMapper` package is an ASP.NET Core middleware that allows us to map exceptions to different status codes. Out-of-the-box, it offers many exception types to get started, handles them, and allows easy mapping between a custom exception and a status code. By default, the library serializes the exceptions to a `ProblemDetails` object (based on RFC 7807) by leveraging as many ASP.NET Core components as possible, so we can customize parts of the library by customizing ASP.NET Core.To get started, in the `Program.cs` file, we must add the following lines:

```
// Add the dependencies to the container
builder.AddExceptionMapper();
// Register the middleware
app.UseExceptionMapper();
```

Now, if we try to add a duplicated product to the basket, we receive a response with a *409 Conflict* status code with the following body:

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.10",
  "title": "The product \u00273\u0027 is already in your shopping cart.",
  "status": 409,
  "traceId": "00-74bdbaa08064fd97ba1de31802ec6f8f-31ffd9ea8215b706-00",
  "debug": {
    "type": {
      "name": "DuplicateBasketItemException",
      "fullName": "Web.Features.DuplicateBasketItemException"
    },
    "stackTrace": "..."
  }
}
```

This output is starting to look like something!

The `debug` object (highlighted) only appears in development or as an opt-in option.

How can the middleware know it's a 409 Conflict and not a 500 Internal Server Error? Simple! The `DuplicateBasketItemException` inherits from the `ConflictException` that comes from the `ForEvolve.ExceptionMapper` namespace (highlighted):

```
using ForEvolve.ExceptionMapper;
namespace Web.Features;
public class DuplicateBasketItemException : ConflictException
{
    public DuplicateBasketItemException(int productId)
        : base($"The product '{productId}' is already in your shopping cart.")
    {
    }
}
```

With this setup, we can leverage exceptions to return errors with different status codes.

I have used this methodology for many years, and it simplifies the program structure and developers' lives. The idea is to harness the power and simplicity of exceptions.

For example, we may want to map EF Core errors, `DbUpdateException` and `DbUpdateConcurrencyException`, to a *409 Conflict* as well, so in case we forget to catch a database error, the middleware will do it for us. To achieve this, we can customize the middleware this way:

```
builder.AddExceptionMapper(builder =>
{
    builder
        .Map<DbUpdateException>()
        .ToStatusCode(StatusCodes.Status409Conflict)
    ;
    builder
        .Map<DbUpdateConcurrencyException>()
        .ToStatusCode(StatusCodes.Status409Conflict)
```

```
    ;
});
```

With that in place, if a client hits an unhandled EF Core exception, the server will respond with something like the following (I omitted the stack trace for brevity reasons):

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.10",
  "title": "Exception of type \u0027Microsoft.EntityFrameworkCore.DbUpdateException\u0027 was th
  "status": 409,
  "traceId": "00-74bdbaa08064fd97ba1de31802ec6f8f-a5ac17f17da8d2db-00",
  "debug": {
    "type": {
      "name": "DbUpdateException",
      "fullName": "Microsoft.EntityFrameworkCore.DbUpdateException"
    },
    "stackTrace": "..."
  },
  "entries": []
}
```

> In an actual project, for security reasons, I recommend customizing the error handling further to hide the fact that we are using EF Core. We must give as little information as possible about our systems to malicious actors to keep them as secure and safe as possible. We won't cover creating custom exception handlers here because it is out of the scope of the chapter.

As we can see, it is easy to register custom exceptions and associate them with a status code. We can do this with any custom exception or inherit from an existing one to make it work with customization.As of version 3.0.29, *ExceptionMapper* offers the following custom exception associations:

Exception Type	Status Code
`BadRequestException`	`StatusCodes.Status400BadRequest`
`ConflictException`	`StatusCodes.Status409Conflict`
`ForbiddenException`	`StatusCodes.Status403Forbidden`
`GoneException`	`StatusCodes.Status410Gone`
`NotFoundException`	`StatusCodes.Status404NotFound`
`ResourceNotFoundException`	`StatusCodes.Status404NotFound`
`UnauthorizedException`	`StatusCodes.Status401Unauthorized`
`GatewayTimeoutException`	`StatusCodes.Status504GatewayTimeout`
`InternalServerErrorException`	`StatusCodes.Status500InternalServerError`
`ServiceUnavailableException`	`StatusCodes.Status503ServiceUnavailable`

Table 18.1: ExceptionMapper custom exception associations.

You can inherit from those standard exceptions, and the middleware will associate them with the correct status code as we did with the `DuplicateBasketItemException` class.*ExceptionMapper* also maps the following .NET exceptions automatically:

- `BadHttpRequestException` to `StatusCodes.Status400BadRequest`
- `NotImplementedException` to `StatusCodes.Status501NotImplemented`

In the project, there are three custom exceptions that you can find on GitHub:

- `BasketItemNotFoundException` that inherits from `NotFoundException`
- `DuplicateBasketItemException` that inherits from `ConflictException`
- `ProductNotFoundException` that inherits from `NotFoundException`

Next, we explore this way of thinking about error propagation a little more.

Leveraging exceptions to propagate errors

With the middleware of *ExceptionMapper* in place, we can treat exceptions as a simple tool to propagate errors to the clients. We can throw an existing exception, like a `NotFoundException`, or create a custom reusable one with a more precise preconfigured error message.When we want the server to return a specific error, all we must do is:

1. Create a new exception type.
2. Inherit from an existing type from *ExceptionMapper* or register our custom exception with the middleware.
3. Throw our custom exception anywhere in the REPR flow.
4. Let the middleware do its job.

Here's a simplified representation of this flow, using the `AddItem` endpoint as an example:

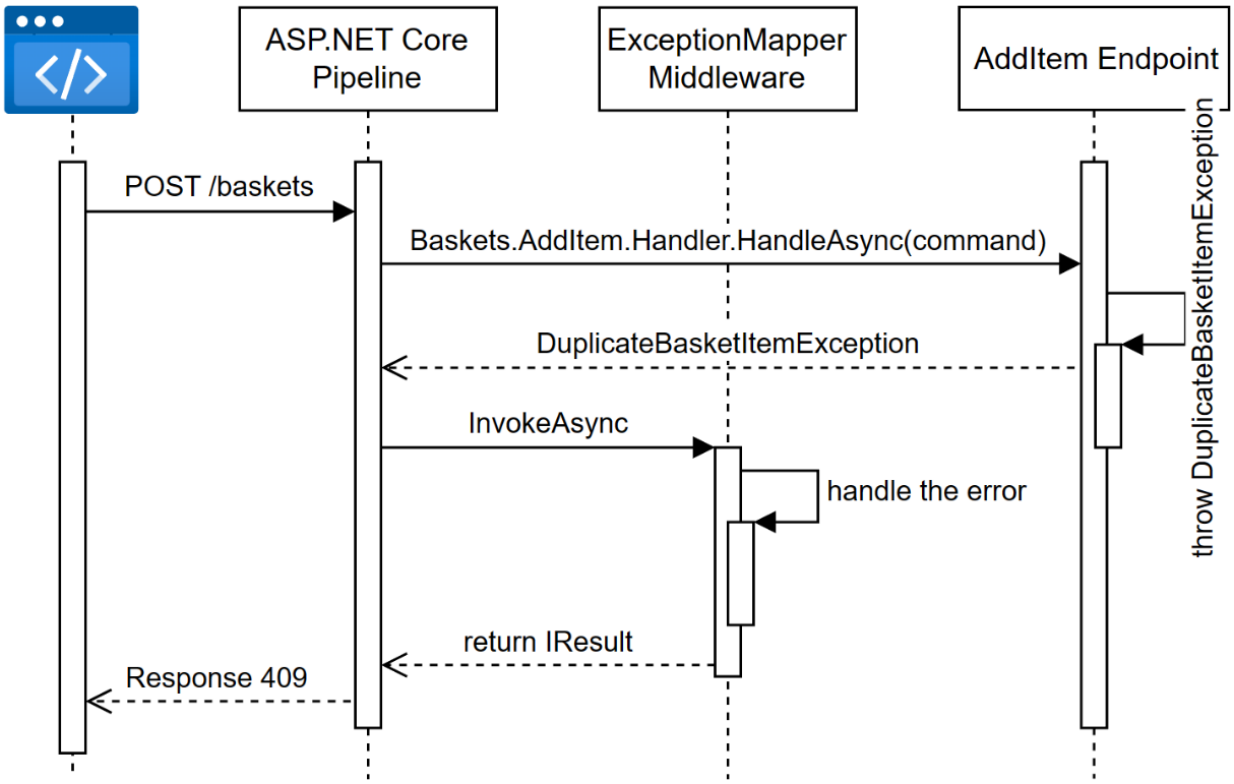


Figure 18.6: a simplified view of an exception flow using ExceptionMapper.

With this in place, we have a simple way to return errors to the clients from anywhere in the REPR flow. Moreover, our errors are consistently formatted the same way.

> The exception handling pattern and the *ExceptionMapper* library also work with MVC and allow customizing the error formatting process.

Next, let's explore a few test cases.

Grey-box testing

Using Vertical Slice Architecture or REPR makes writing grey-box tests very convenient. The test project mainly comprises integration tests that use the grey-box philosophy. Since we know the application under test's inner workings, we can manipulate the data from the EF Core `DbContext` objects, which allows us to write almost end-to-end tests very quickly. The confidence level we get from those tests is very high because they test the whole stack, including HTTP, not just some scattered pieces, leading to a very high level of code coverage per test case.

Of course, integration tests are slower, yet not that slow. It is up to you to create the right balance of unit and integration tests. In this case, I focused on grey-box integration testing, which led to 13 tests covering 97.2% of the lines and 63.1% of the branches. The guard clauses represent most of the branches that we do not test. We could write a few unit tests to boost the numbers if we'd like.

> We explored white-, grey- and black-box testing in *Chapter 2, Automated Testing*.

Let's start by exploring the AddItem tests.

AddItemTest

The `AddItem` feature is the first use case we explored. We need three tests to cover all scenarios but the `Handler` class guard clauses.

First test method

The following grey-box integration test ensures an HTTP POST request adds the item to the database:

```
[Fact]
public async Task Should_add_the_new_item_to_the_basket()
{
    // Arrange
    await using var application = new C18WebApplication();
    var client = application.CreateClient();
    // Act
    var response = await client.PostAsJsonAsync(
        "/baskets",
        new AddItem.Command(4, 1, 22)
    );
    // Assert the response
    Assert.NotNull(response);
    Assert.True(response.IsSuccessStatusCode);
    var result = await response.Content
        .ReadFromJsonAsync<AddItem.Response>();
    Assert.NotNull(result);
    Assert.Equal(1, result.ProductId);
    Assert.Equal(22, result.Quantity);
    // Assert the database state
    using var seedScope = application.Services.CreateScope();
    var db = seedScope.ServiceProvider
        .GetRequiredService<BasketContext>();
    var dbItem = db.Items.FirstOrDefault(x => x.CustomerId == 4 && x.ProductId == 1);
    Assert.NotNull(dbItem);
    Assert.Equal(22, dbItem.Quantity);
}
```

The *Arrange* block of the preceding test case creates a test application and an `HttpClient`. It then sends an `AddItem.Command` to the endpoint in its *Act* block.Afterward, it splits the *Assert* block in two: the HTTP response and the database itself. The first part ensures that the endpoint returns the expected data. The second part ensures that the database is in the correct state.

> It is a good habit to ensure the database is in the correct state, especially with EF Core or most Unit of Work implementations, because one could add an item and forget to save the changes leading to an incorrect database state. Yet, the data returned by the endpoint would have been correct.

We could test more or less elements here. We could refactor the *Assert* block so it becomes more elegant. We can and should continuously improve all types of code, including tests. However, in this case, I wanted to keep as much of the logic in the test method to make it easier to understand.

> It is also a good practice to keep test methods as independent as possible. This does not mean that improving readability and encapsulating code into helper classes or methods is wrong; on the contrary.

The only opaque piece of the test method is the `C18WebApplication` class, which inherits from the `WebApplicationFactory<Program>` class and implements a few helper methods to simplify the configuration of the test application. You can treat it as an instance of the `WebApplicationFactory<Program>` class. Feel free to browse the code on GitHub and explore its inner workings.

> Creating an `Application` class is a good reusability pattern. However, creating an application per test method is not the most performant because you are booting the entire program for every test.
>
> > You can use test fixtures to reuse and share an instance of the program between multiple tests. However, remember that the application's state and potentially the database are also shared between tests.

To the second test, next.

Second test method

This test ensures that the `Location` header contains a valid URL. This test is important since the `Baskets` and the `Products` features are loosely coupled and can change independently. Here's the code:

```
[Fact]
public async Task Should_return_a_valid_product_url()
{
    // Arrange
    await using var application = new C18WebApplication();
    await application.SeedAsync<Products.ProductContext>(async db =>
    {
        db.Products.RemoveRange(db.Products);
        db.Products.Add(new("A test product", 15.22m, 1));
        await db.SaveChangesAsync();
    });
    var client = application.CreateClient();
    // Act
    var response = await client.PostAsJsonAsync(
        "/baskets",
        new AddItem.Command(4, 1, 22)
    );
    // Assert
    Assert.NotNull(response);
    Assert.Equal(HttpStatusCode.Created, response.StatusCode);
    Assert.NotNull(response.Headers.Location);
    var productResponse = await client
        .GetAsync(response.Headers.Location);
    Assert.NotNull(productResponse);
    Assert.True(productResponse.IsSuccessStatusCode);
}
```

The preceding test method is similar to the first one. The *Arrange* block creates an application, seeds the database, and creates an `HttpClient`. The `SeedAsync` method is one of the helper methods of the `C18WebApplication` class.The *Act* block sends a request to create a basket item.The *Assert* block is divided in two. The first ensures that the HTTP response contains a `Location` header and that the status code is 201. The second part (highlighted) takes the `Location` header and sends an HTTP request to validate the URL's validity. This test ensures that if we change the URL of the `Products.FetchOne` endpoint, say we prefer `/catalog` over `/products`, this test will alert us.We explore the third test case next.

Third test method

The last test method ensures that the endpoint responds with a 409 Conflict status when a consumer tries to add an existing item:

```
[Fact]
public async Task Should_return_a_ProblemDetails_with_a_Conflict_status_code()
{
    // Arrange
    await using var application = new C18WebApplication();
    await application.SeedAsync<BasketContext>(async db =>
```

```
    {
        db.Items.RemoveRange(db.Items);
        db.Items.Add(new(
            CustomerId: 1,
            ProductId: 1,
            Quantity: 10
        ));
        await db.SaveChangesAsync();
    });
    var client = application.CreateClient();
    // Act
    var response = await client.PostAsJsonAsync(
        "/baskets",
        new AddItem.Command(
            CustomerId: 1,
            ProductId: 1,
            Quantity: 20
        )
    );
    // Assert the response
    Assert.NotNull(response);
    Assert.False(response.IsSuccessStatusCode);
    Assert.Equal(HttpStatusCode.Conflict, response.StatusCode);
    var problem = await response.Content
        .ReadFromJsonAsync<ProblemDetails>();
    Assert.NotNull(problem);
    Assert.Equal("The product \u00271\u0027 is already in your shopping cart.", problem.Title);
    // Assert the database state
    using var seedScope = application.Services.CreateScope();
    var db = seedScope.ServiceProvider
        .GetRequiredService<BasketContext>();
    var dbItem = db.Items.FirstOrDefault(x => x.CustomerId == 1 && x.ProductId == 1);
    Assert.NotNull(dbItem);
    Assert.Equal(10, dbItem.Quantity);
}
```

The preceding test method is very similar to the other two.The *Arrange* block creates a test application, seeds the database, and creates an `HttpClient` .The *Act* block sends a request using the only item in the database, which we expect to result in a conflict.The first part of the *Assert* block ensures that the endpoint returns the expected `ProblemDetails` object. The second part validates that the endpoint has not changed the quantity in the database.With those three tests, we are covering the relevant code of the `AddItem` feature.The other test cases are similar, sending HTTP requests and validating the database content. Each feature has between one and three tests. We explore a test related to the `UpdateQuantity` feature next.

UpdateQuantityTest

We did not cover the `UpdateQuantity` feature, but one of its branches is that if the current quantity and the new quantities are the same, the endpoint will not update the data. Here's the snippet:

```
if (item.Quantity != command.Quantity)
{
    _db.Items.Update(itemToUpdate);
    await _db.SaveChangesAsync(cancellationToken);
}
```

To test this use case, we subscribe to the `SavedChanges` event on the EF Core `DbContext` , then ensure the code never calls it. This uses no mocks or stubs and tests the real code.This test stands out of the lot, so I considered it worth exploring before moving on. Here's the code:

```
[Fact]
public async Task Should_not_touch_the_database_when_the_quantity_is_the_same()
{
    // Arrange
    await using var application = new C18WebApplication();
    await application.SeedAsync<BasketContext>(async db =>
    {
```

```
        db.Items.RemoveRange(db.Items.ToArray());
        db.Items.Add(new BasketItem(2, 1, 5));
        await db.SaveChangesAsync();
    });
    using var seedScope = application.Services.CreateScope();
    var db = seedScope.ServiceProvider
        .GetRequiredService<BasketContext>();
    var mapper = seedScope.ServiceProvider
        .GetRequiredService<UpdateQuantity.Mapper>();
    db.SavedChanges += Db_SavedChanges;
    var saved = false;
    var sut = new UpdateQuantity.Handler(db, mapper);
    // Act
    var response = await sut.HandleAsync(
        new UpdateQuantity.Command(2, 1, 5),
        CancellationToken.None
    );
    // Assert
    Assert.NotNull(response);
    Assert.False(saved);
    void Db_SavedChanges(object? sender, SavedChangesEventArgs e)
    {
        saved = true;
    }
}
```

The preceding test method should sound very familiar by now. However, we use a different pattern here.In the *Arrange* block, we create a test application and seed the database, but we do not create an `HttpClient`. We use the `ServiceProvider` to create the dependencies instead. Then manually instantiate the `UpdateQuantity.Handler` class. This allows us to customize the `BasketContext` instance to assess whether the endpoint called its `SaveChange` method (highlighted code).The *Act* block invokes the `HandleAsync` method directly with a command that should not trigger the update because the item has the same quantity as the one we seeded. Unlike the other tests, we are not sending an HTTP request.The *Assert* block is simpler than the other tests we explored because we test the method, not the HTTP response or the database. In this case, we only care whether the `saved` variable is `true` or `false`.

> This test is much faster than the other because no HTTP is involved. When calling the `CreateClient` method of a `WebApplicationFactory<T>` object (the `C18WebApplication` class in this case), it starts the webserver and then creates the `HttpClient`, which has a significant performance overhead.
>
> Remember this tip when you have to optimize your test suites.

And we are done; the test knows whether or not the `DbContext`'s `SavedChanges` method was called. Let's summarize what we learned before moving to the next chapter.

Summary

We delved into the Request-EndPoint-Response (REPR) design pattern and learned that REPR follows the most foundational pattern of the web. The client sends a request to an endpoint, which processes it and returns a response. The pattern focuses on designing the backend code around the endpoint, making it faster to develop, easier to find your way around the project, and more focused on features than MVC and layers.We also took a CQS approach around the requests, making them queries or commands, depicting all that can happen in a program: read or write states.We explored ways to organize the code around such a pattern, from implementing trivial to more complex features. We built a technology stack to create an e-commerce web application that leverages the REPR pattern and a feature-oriented design. We learned how to leverage middleware to handle exceptions globally and how the *ExceptionMapper* library provides us with this capability. We also used grey-box testing to cover almost all of the project's logic with just a few tests.Next, we explore microservices architecture.

Questions

Let's take a look at a few practice questions:

1. Must we use the *FluentValidation* and *ExceptionMapper* libraries when implementing the REPR pattern?
2. What are the three components of the REPR pattern?
3. Does the REPR pattern dictate that we use nested classes?
4. Why are grey-box integration tests provide much confidence?
5. Name an advantage of handling exceptions using middleware.

Further reading

Here are a few links to build upon what we learned in the chapter:

- FluentValidation: https://adpg.link/xXgp
- FluentValidation.AspNetCore.Http: https://adpg.link/qsao
- ExceptionMapper: https://adpg.link/ESDb
- Mapperly: https://adpg.link/Dwcj
- MVC Controllers are Dinosaurs - Embrace API Endpoints: https://adpg.link/NGjm

Answers

1. No. REPR does not dictate how to implement it. You can create your own stack or go barebone ASP.NET Core minimal API and implement everything by hand in the project.
2. REPR consists of a request, an endpoint, and a response.
3. No. REPR does not prescribe any implementation details.
4. Grey-box integration tests provide much confidence in their outcome because they test the feature almost end-to-end, ensuring all the pieces are there, from the services in the IoC container to the database.
5. Handling exceptions using middleware allows for centralizing the management of exceptions, encapsulating that responsibility in a single place. It also provides for uniformizing the output, sending the clients a response in the same format for all errors. It removes the burden of handling each exception individually, eliminating `try-catch` boilerplate code.

19 Introduction to Microservices Architecture

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



The chapter covers some essential microservices architecture concepts. It is designed to get you started with those principles and an overview of the concepts surrounding microservices, which should help you make informed decisions about whether to go for a microservices architecture or not.Since microservices architecture is larger in scale than the previous application-scale patterns we visited and often involves complex components or setup, there is very limited C# code in the chapter. Instead, I explain the concepts and list open-source or commercial offerings that you can leverage to apply these patterns to your applications. Moreover, you should not aim to implement many of the pieces discussed in the chapter because it can be a lot of work to get them right, and they don't add business value, so you are better off just using an existing implementation instead. There is more context about this throughout the chapter.Monolithic architecture patterns, such as Vertical Slice and Clean Architecture, are still good to know, as you can apply those to individual microservices. Don't worry—all of the knowledge you have acquired since the beginning of this book is not forfeit and is still worthwhile.In this chapter, we cover the following topics:

- What are microservices?
- An introduction to event-driven architecture
- Getting started with message queues
- Implementing the Publish-Subscribe pattern
- Introducing Gateway patterns
- Project – REPR.BFF—that transforms the REPR project into microservices
- Revisiting the CQRS pattern
- The Microservices Adapter pattern

Let's get started!

What are microservices?

Microservices represent an application that is divided into multiple smaller applications. Each application, or microservice, interacts with the others to create a scalable system. Usually, microservices are deployed to the cloud as containerized or serverless applications.Before getting into too many details, here are the principles to keep in mind when building microservices:

- Each microservice should be a cohesive unit of business.
- Each microservice should own its data.
- Each microservice should be independent of the others.

Furthermore, everything we have studied so far—the other principles of designing software—applies to microservices but on another scale. For example, you don't want tight coupling between microservices (solved by microservices independence), but the coupling is inevitable (as with any code). There are numerous ways to solve

this problem, such as the Publish-Subscribe pattern.There are no hard rules about how to design microservices, how to divide them, how big they should be, and what to put where. Nevertheless, I'll lay down a few foundations to help you get started and orient your journey into microservices.

Cohesive unit of business

A microservice should have a single business responsibility. Always design the system with the domain in mind, which should help you divide the application into multiple pieces. If you know **Domain-Driven Design** (**DDD**), a microservice will most likely represent a **Bounded Context**, which in turn is what I call a *cohesive unit of business*. Basically, a cohesive unit of business (or bounded context) is a self-contained part of the domain with limited interactions with other parts.Even if a **microservice** has *micro* in its name, it is more important to group logical operations under it than to aim at a micro-size. Don't get me wrong here; if your unit is tiny, that's even better. However, suppose you split a unit of business into multiple smaller parts instead of keeping it together (breaking cohesion); you are likely to introduce useless chattiness within your system (coupling between microservices). This could lead to performance degradation and to a system that is harder to debug, test, maintain, monitor, and deploy. Moreover, it is easier to split a big microservice into smaller pieces than to assemble multiple microservices back together. Try to apply the SRP to your microservices: a microservice should have only one reason to change unless you have a good reason to do otherwise.

Ownership of data

Each microservice should be the source of truth of its cohesive unit of business. A microservice should share its data through an API (a web API/HTTP, for example) or another mechanism (integration events, for example). It should own that data and not share it with other microservices directly at the database level.For instance, two different microservices should never access the same relational database table. If a second microservice needs some of the same data, it can create its own cache, duplicate the data, or query the owner of that data but not access the database directly; **never**.This data-ownership concept is probably the most critical part of the microservices architecture and leads to microservices independence. Failing at this will most likely lead to a tremendous number of problems. For example, if multiple microservices can read or write data in the same database table, each time something changes in that table, all of them must be updated to reflect the changes. If different teams manage the microservices, that means cross-team coordination. If that happens, each microservice is not independent anymore, which opens the floor to our next topic.

Microservice independence

At this point, we have microservices that are cohesive units of business and own their data. That defines **independence**.This independence allows the systems to scale while having minimal to no impact on the other microservices. Each microservice can also scale independently without needing the whole system to be scaled. Additionally, when the business requirements grow, each part of that domain can evolve independently.Furthermore, you could update one microservice without impacting the others or even have a microservice go offline without the whole system stopping.Of course, microservices have to interact with one another, but the way they do should define how well your system runs. A little like Vertical Slice architecture, you are not limited to using one set of architectural patterns; you can independently make specific decisions for each microservice. For example, you could choose a different way for how two microservices communicate with each other versus two others. You could even use different programming languages for each microservice.

> I recommend sticking to one or a few programming languages for smaller businesses and organizations, as you most likely have fewer developers, and each has more to do. Based on my experience, you want to ensure business continuity when people leave and make sure you can replace them and not sink the ship due to some obscure technologies used here and there (or too many technologies).

Now that we've covered the basics, let's jump into the different ways microservices can communicate using event-driven architecture.

An introduction to event-driven architecture

Event-driven architecture (**EDA**) is a paradigm that revolves around consuming streams of events, or data in motion, instead of consuming static states.What I define by a static state is the data stored in a relational database table or other types of data stores, like a NoSQL documents store. That data is dormant in a central location and waiting for actors to consume and mutate it. It is stale between every mutation; the data (a record, for example) represents a finite state.On the other hand, data in motion is the opposite: you consume the ordered events and determine the change in state that each event brings or what process the program should trigger in reaction to an event.What is an event? People often interchange the words event, message, and command. Let's try to clarify this:

- A message is a piece of data that represents something.
- A message can be an object, a JSON string, bytes, or anything else your system can interpret.
- An event is a message that represents something that happened in the past.
- A command is a message sent to tell one or more recipients to do something.
- A command is sent (past tense), so we can also consider it an event.

A message usually has a payload (or body), headers (metadata), and a way to identify it (this can be through the body or headers).We can use events to divide a complex system into smaller pieces or have multiple systems talk to each other without creating tight coupling. Those systems could be subsystems or external applications, such as microservices.Like a REST API's **Data Transfer Objects (DTOs)**, events become the data contracts that tie the multiple systems together (coupling). It is essential to think about that carefully when designing events. Of course, we cannot foresee the future, so we can only do so much to get it perfect the first time. We can version the events to improve maintainability.EDA is a fantastic way of breaking tight coupling between microservices but requires rewiring your brain to learn this newer paradigm. Tooling is becoming more mature, and expertise is less scarce than more linear ways of thinking (like using point-to-point communication and relational databases). However, this is slowly changing and well worth learning.Before moving further, we can categorize events into the following overlapping buckets:

- Domain events
- Integration events
- Application events
- Enterprise events

As we explore next, all types of events play a similar role with different intents and scopes.

Domain events

A domain event is a term based on DDD representing an event in the domain. This event could then trigger other pieces of logic to be executed subsequently. It allows us to divide a complex process into multiple smaller processes. Domain events work well with domain-centric designs, like Clean Architecture, as we can use them to split complex domain objects into multiple smaller pieces. Domain events are usually application events. For example, we can use MediatR to publish domain events inside an application.To summarize, **domain events integrate pieces of domain logic together while keeping the domain logic segregated**, leading to loosely coupled components that hold one domain responsibility each (single responsibility principle).

Integration events

Integration events are like domain events but propagate messages to external systems, integrating multiple systems together while keeping them independent. For example, a microservice could send the `new user registered` event message that other microservices react to, like saving the `user id` to enable additional capabilities or sending a greeting email to that new user.We use a message broker or message queue to publish such events. We explore those after covering application and enterprise events.To summarize, **integration events integrate multiple systems together while keeping them independent**.

Application events

An application event is an event that is internal to an application; it is just a matter of scope. If the event is internal to a single process, that event is also a domain event (most likely). If the event crosses microservices boundaries

that your team owns (the same application), it is also an integration event. The event itself won't be different; it is the reason why it exists and its scope that describes it as an application event or not.To summarize, **application events are related to a single application**.

Enterprise events

An enterprise event describes an event that crosses internal enterprise boundaries. These are tightly coupled with your organizational structure. For example, a microservice sends an event that other teams, part of other divisions or departments, consume.The governance model around those events should differ from application events only your team consumes and be more strict with strong oversight.Someone must consider who can consume that data, under what circumstances, the impact of changing the event schema (data contract), schema ownership, naming conventions, data-structure conventions, and more, or risk building an unstable data highway.

> I like to see EDA as a central **data highway** in the middle of applications, systems, integrations, and organizational boundaries, where the events (data) flow between systems in a loosely coupled manner.
>
> > It's like a highway where cars flow between cities (without traffic jams). The cities are not controlling what car goes where but are open to visitors.

To summarize, **enterprise events are integration events that cross organizational boundaries**.

Conclusion

In this overview of event-driven architecture, we defined events, messages, and commands. An event is a snapshot of the past, a message is data, and a command is an event that suggests other systems take action. Since all messages are from the past, calling them events is accurate. We then organized events into a few overlapping buckets to help identify the intents. We can send events for different objectives, but whether it is about designing independent components or reaching out to different parts of the business, an event remains a payload that respects a certain format (schema). That schema is the data contract (coupling) between the consumers of those events. That data contract is probably the most important piece of it all: break the contract, break the system.Now, let's see how event-driven architecture can help us follow the **SOLID** principles at cloud-scale:

- **S**: Systems are independent of each other by raising and responding to events. The events themselves are the glue that ties those systems together. Each piece has a single responsibility.
- **O**: We can modify the system's behaviors by adding new consumers to a particular event without impacting the other applications. We can also raise new events to build a new process without affecting existing applications.
- **L**: N/A
- **I**: Instead of building a single system, EDA allows us to create multiple smaller systems that integrate through data contracts (events), and those contracts are the messaging interfaces of the system.
- **D**: EDA enables systems to break tight coupling by depending on the events (interfaces/abstractions) instead of communicating directly with one another, inverting the dependency flow.

EDA does not only come with advantages; it also has a few drawbacks that we explore in subsequent sections of the chapter.Next, we explore message queues followed by the Publish-Subscribe pattern, two ways of interacting with events.

Getting started with message queues

A **message queue** is nothing more than a queue we leverage to send ordered messages. A queue works on a **First In, First Out** (**FIFO**) basis. If our application runs in a single process, we could use one or more `Queue<T>` instances to send messages between our components or a `ConcurrentQueue<T>` instance to send messages between threads. Moreover, queues can be managed by an independent program to send messages in a distributed fashion (between applications or microservices).A distributed message queue can add more or less features to the mix, especially for cloud programs that handle failures at more levels than a single server. One of those features is the **dead letter queue**, which stores messages that failed some criteria in another queue. For example, if the target

queue is full, a message could be sent to the **dead letter queue** instead. One could requeue such messages by putting the message back at the end of the queue.

> Beware that requeing messages change the order of the messages. If the order is important in your app, consider this.

Many messaging queue protocols exist; some are proprietary, while others are open source. Some messaging queues are cloud-based and used *as a service*, such as Azure Service Bus and Amazon Simple Queue Service. Others are open source and can be deployed to the cloud or on-premises, such as Apache ActiveMQ.If you need to process messages in order and want each message to be delivered to a single recipient at a time, a **message queue** seems like the right choice. Otherwise, the **Publish-Subscribe** pattern could be a better fit for you.Here is a basic example that illustrates what we just discussed:



Figure 19.1: A publisher that enqueues a message with a subscriber that dequeues it

For a more concrete example, in a distributed user registration process, when a user registers, we could want to do the following:

- Send a confirmation email.
- Process their picture and save one or more thumbnails.
- Send an onboarding message to their in-app mailbox.

To sequentially achieve this, one operation after the other, we could do the following:

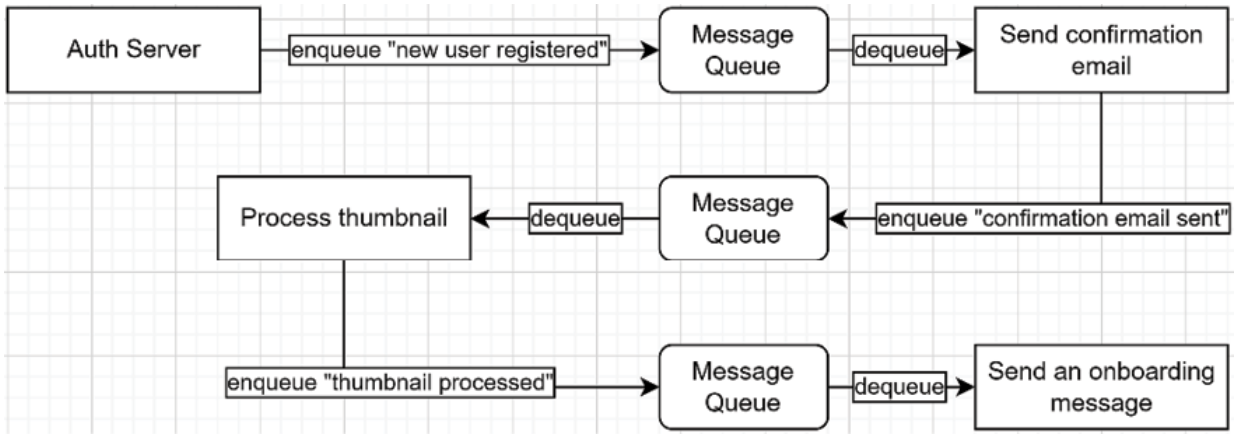


Figure 19.2: A process flow that sequentially executes three operations that happen after a user creates an account

In this case, the user would not receive the *Onboarding Message* if the process crashes during the *Process Thumbnail* operation. Another drawback would be that to insert a new operation between the *Process Thumbnail* and *Send an onboarding message* steps, we'd have to modify the *Send an onboarding message* operation (tight coupling).If the order does not matter, we could queue all the messages from the *Auth Server* instead, right after the user's creation, like this:

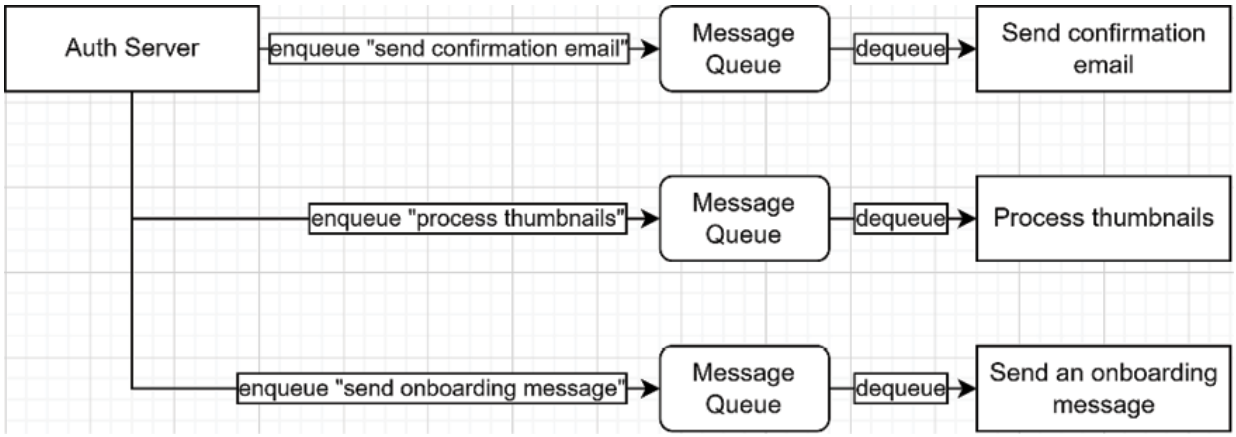


Figure 19.3: The Auth Server is queuing the operations sequentially while different processes execute them in parallel

This process is better, but the *Auth Server* now controls what should happen once a new user has been created. The *Auth Server* was queuing an event in the previous workflow that told the system that a new user registered. However, now, it has to be aware of the post-processing workflow to queue each operation sequentially to enqueue the correct commands. Doing this is not wrong in itself and is easier to follow when you dig into the code, but it creates tighter coupling between the services where the *Auth Server* is aware of the external processes. Moreover, it packs too many responsibilities into the *Auth Server*.

> SRP-wise, why would an authentication/authorization server be responsible for anything other than authentication, authorization, and managing that data?

If we continue from there and want to add a new operation between two existing steps, we would only have to modify the *Auth Server*, which is less error-prone than the preceding workflow.If we want the best of both worlds, we could use the **Publish-Subscribe** pattern instead, which we cover next, and continue building on top of this example.

Conclusion

If you need messages to be delivered sequentially, a queue might be the right tool. The example we explored was destined to fail from the beginning, but it allowed us to explore the thinking process behind designing the system. Sometimes, the first idea is not the best and can be improved by exploring new ways of doing things or learning new skills. Being open-minded to the ideas of others can also lead to better solutions.

> Sometimes, just speaking out loud makes our own brain solve the issue by itself. So explain the problem to someone and see what happens.

Message queues are amazing at buffering messages for high-demand scenarios where an application may not be able to handle spikes of traffic. In that case, the messages are enqueued so the application can catch up at its own speed, reading them sequentially.Implementing distributed message queues requires a lot of knowledge and effort and is not worth it for almost all scenarios. The big cloud providers like AWS and Azure offer fully managed message queue systems as a service. You can also look at **ActiveMQ**, **RabbitMQ**, or any **Advanced Message Queuing Protocol** (**AMQP**) broker.One essential aspect of choosing the right queue system is whether you are ready and have the skills to manage your own distributed message queue. Suppose you want to speed up development, cut infrastructure management costs, and have enough money. In that case, you could use a fully managed offering for at least your production environment, especially if you expect a large volume of messages. On the other hand, using a local or on-premise instance for development or smaller-scale usage may save you a considerable sum of money. Choosing an open source system with fully managed cloud offerings is a good way to achieve both: low local development cost with an always available high-performance cloud production offering that the service provider maintains for you.Another aspect is to base your choice on needs. Have clear

requirements and ensure the system you choose does what you need. Some offerings cover multiple use cases like queues and pub-sub, leading to a simplified tech stack requiring fewer skills.Before moving to the pub-sub pattern, let's see how message queues can help us follow the **SOLID** principles at the app scale:

- **S**: Helps centralize and divide responsibilities between applications or components without them directly knowing each other, breaking tight coupling.
- **O**: Allows us to change the message producer's or subscriber's behaviors without the other knowing about it.
- **L**: N/A
- **I**: Each message and handler can be as small as needed, while each microservice indirectly interacts with the others to solve the bigger problem.
- **D**: By not knowing the other dependencies (breaking tight coupling between microservices), each microservice depends only on the messages (abstractions) instead of concretions (the other microservices API).

One drawback is the delay between enqueuing a message and processing a message. We talk about delay and latency in more detail in subsequent sections.

Implementing the Publish-Subscribe pattern

The **Publish-Subscribe** pattern (Pub-Sub) is similar to what we did using **MediatR** and what we explored in the *Getting started with message queues* section. However, instead of sending one message to one handler (or enqueuing a message), we publish (send) a message (event) to zero or more subscribers (handlers). Moreover, the publisher is unaware of the subscribers; it only sends messages out, hoping for the best (also known as **fire and forget**).

> Using a message queue does not mean you are limited to only one recipient.

We can use **Publish-Subscribe** in-process or in a distributed system through a **message broker**. The message broker is responsible for delivering the messages to the subscribers. Using a message broker is the way to go for microservices and other distributed systems since they are not running in a single process.This pattern has many advantages over other ways of communication. For example, we could recreate the state of a database by replaying the events that happened in the system, leading to the **event sourcing** pattern. More on that later.The design depends on the technology used to deliver the messages and the system's configuration. For example, you could use **MQTT** to deliver messages to **Internet of Things** (**IoT**) devices and configure them to retain the last message sent on each topic. That way, when a device connects to a topic, it receives the latest message. You could also configure a **Kafka** broker that keeps a long history of messages and asks for all of them when a new system connects to it. All of that depends on your needs and requirements.

> **MQTT and Apache Kafka**
>
> If you were wondering what MQTT is, here is a quote from their website https://adpg.link/mqtt:
>
> *"MQTT is an OASIS standard messaging protocol for the Internet of Things (IoT). It is designed as an extremely lightweight publish/subscribe messaging transport […]"*
>
> Here is a quote from Apache Kafka's website https://adpg.link/kafka:
>
> *"Apache Kafka is an open-source distributed event streaming platform […]"*

We cannot cover every single scenario of every system that follows every protocol. Therefore, I'll highlight some shared concepts behind the Pub-Sub design pattern so you know how to get started. Then, you can dig into the specific technology you want (or need) to use.A topic is a way to organize events, a channel, a place to read or write specific events so consumers know where to find them. As you can probably imagine, sending all events to the same place is like creating a relational database with a single table: it would be suboptimal and hard to manage, use, and evolve.To receive messages, subscribers must subscribe to topics (or the equivalent of a topic):



Figure 19.4: A subscriber subscribes to a pub-sub topic

The second part of the Pub-Sub pattern is to publish messages, like this:

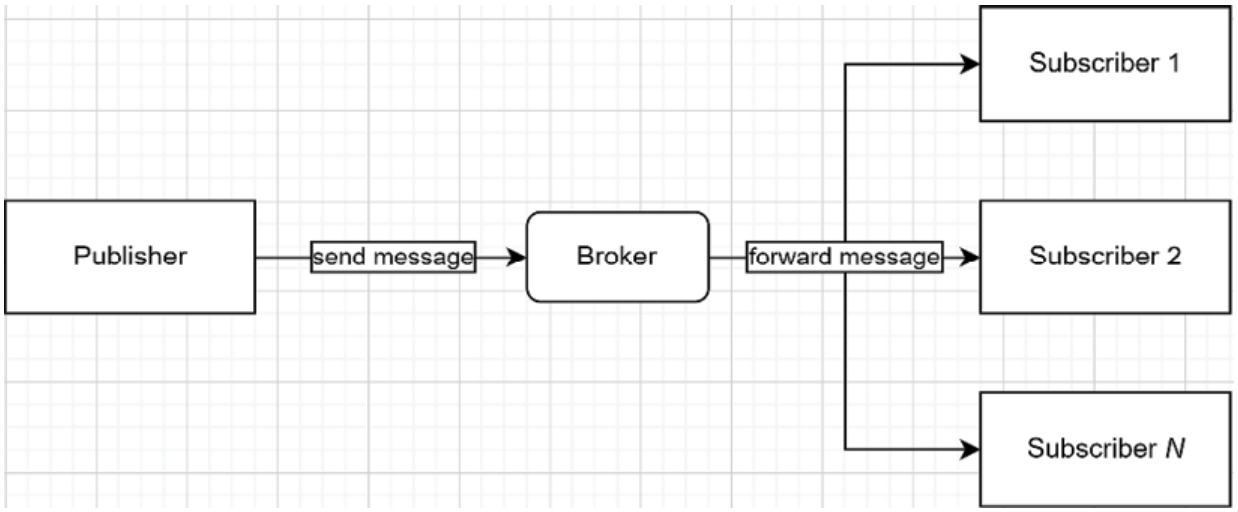


Figure 19.5: A publisher is sending a message to the message broker. The broker then forwards that message to N subscribers, where N can be zero or more

Many abstracted details here depend on the broker and the protocol. However, the following are the two primary concepts behind the Publish-Subscribe pattern:

- Publishers publish messages to topics.
- Subscribers subscribe to topics to receive messages when they are published.

Security is a crucial implementation detail not illustrated here. Security is mandatory in most systems; not every subsystem or device should have access to all topics.

Publishers and subscribers could be any part of any system. For example, many Microsoft Azure services are publishers (for example, Blob storage). You can then have other Azure services (for example, Azure Functions) subscribe to those events and react to them.You can also use the **Publish-Subscribe** pattern inside your applications—there's no need to use cloud resources for that; this can even be done inside the same process (we explore this in the next chapter).The most significant advantage of the Publish-Subscribe pattern is breaking tight coupling between systems. One system publishes events while others consume them without the systems knowing each other.That loose coupling leads to scalability, where each system can scale independently, and messages can be processed in parallel using the required resources. It is also easier to add new processes to a workflow since the systems are unaware of the others. To add a new process that reacts to an event, you only have to create a new microservice, deploy it, start to listen to one or more events, and process them.On the downside, the message broker can become the application's single point of failure and must be configured appropriately. It is also essential to consider the best delivery policies for each message type. An example of a policy could be to ensure the delivery of crucial messages while delaying less time-sensitive messages and dropping unimportant messages during load surges.If we revisit our previous example using Publish-Subscribe, we end up with the following simplified workflow:

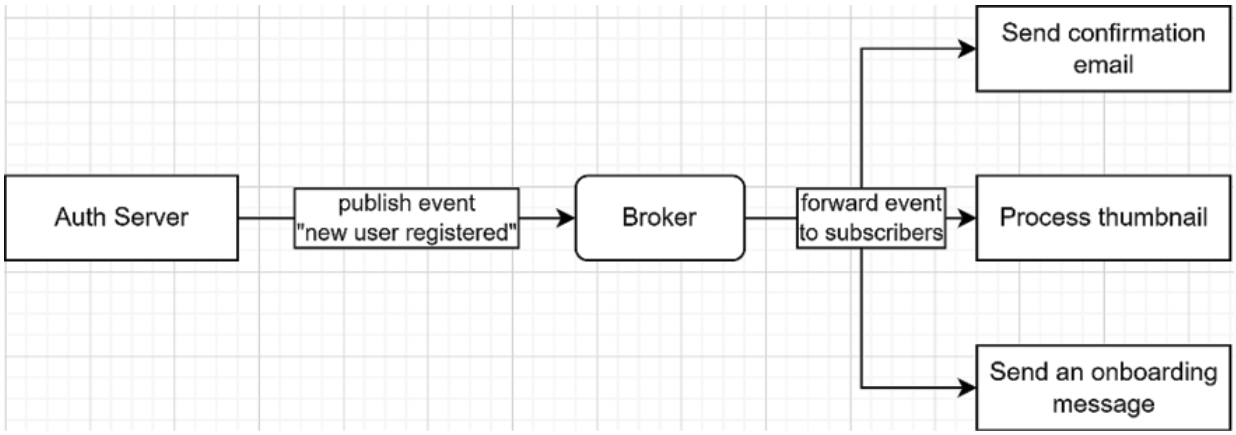


Figure 19.6: The Auth Server is publishing an event representing the creation of a new user. The broker then forwards that message to the three subscribers, who then execute their tasks in parallel

Based on this workflow, we broke the tight coupling between the *Auth Server* and the post-registration process. The *Auth Server* is unaware of the workflow, and the individual services are unaware of each other. Moreover, if we want to add a new task, we only have to create or update a microservice that subscribes to the right topic (in this case, the “new user registered” topic).The current system does not support synchronization and does not handle process failures or retries, but it is a good start since we combine the pros of the message queue examples and leave the cons behind.Using an event broker inverts the dependency flow. The diagrams we explored show the message flow, but here’s what happens on the dependency sides of things:

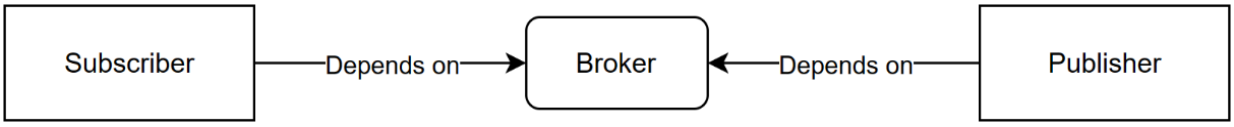


Figure 19.7: A diagram representing the inverted dependency flow of using the pub-sub pattern

Now that we have explored the Publish-Subscribe pattern, we look at message brokers, then dig deeper into EDA and leverage the Publish-Subscribe pattern to create a persistent database of events that can be replayed: the Event Sourcing pattern.

Message brokers

A message broker is a program that allows us to send (**publish**) and receive (**subscribe**) messages. It plays the mediator role at scale, allowing multiple applications to talk to each other without knowing each other (**loose coupling**). The message broker is usually the central piece of any event-based distributed system that implements the publish-subscribe pattern.An application (**publisher**) publishes messages to topics, while other applications (**subscribers**) receive messages from those topics. The notion of **topics** may differ from one protocol or system to another, but all systems I know have a topic-like concept to route messages to the right place. For example, you can publish to the `Devices` topic using Kafka, but to `devices/abc-123/do-something` using MQTT.How you name your topics depends significantly on the system you are using and the scale of your installation. For example, MQTT is a lightweight event broker recommending a path-like naming convention. On the other hand, Apache Kafka is a full-featured event broker and event streaming platform that is not opinionated about topic names, leaving you in charge of that. Depending on the scale of your implementation, you can use the entity name as the topic name or may need prefixes to identify who in the enterprise can interact with what part of the system. Due to the small scale of the examples in the chapter, we stick with simple topic names, making the examples easier to understand.The message broker is responsible for forwarding the messages to the registered recipients. The lifetime of those messages can vary by broker or even per individual message or topic.There are multiple message brokers out there using different protocols. Some brokers are cloud-based, such as Azure Event Grid. Other brokers are lightweight and more suited for IoT, such as Eclipse Mosquitto/MQTT. In contrast to MQTT, others

are more robust and allow for high-velocity data streaming, such as Apache Kafka.What message broker to use should be based on the requirements of the software you are building. Moreover, you are not limited to one broker. Nothing stops you from picking a message broker that handles the dialogs between your microservices and using another to handle the dialogs with external IoT devices. If you are building a system in Azure, want to go serverless, or prefer paying for SaaS components that scale without investing maintenance time, you can leverage Azure services such as Event Grid, Service Bus, and Queue Storage. If you prefer open-source software, you can choose Apache Kafka and even run a fully managed cloud instance as a service using Confluent Cloud if you don't want to manage your own cluster.

The event sourcing pattern

Now that we have explored the Publish-Subscribe pattern, learned what an event is, and talked about event brokers, it is time to explore **how to replay the state of an application**. To achieve this, we can follow the **event sourcing pattern**.The idea behind event sourcing is to **store a chronological list of events** instead of a single entity, where that collection of events becomes the source of truth. That way, every single operation is saved in the right order, helping with concurrency. Moreover, we could replay all of these events to generate an object's current state in a new application, allowing us to deploy new microservices more easily.Instead of just storing the data, if the system propagates it using an event broker, other systems can cache some of it as one or more **materialized views**.

> A **materialized view** is a model created and stored for a specific purpose. The data can come from one or more sources, improving performance when querying that data. For example, the application returns the materialized view instead of querying multiple other systems to acquire the data. You can view the materialized view as a cached entity that a microservice stores in its own database.

One of the drawbacks of event sourcing is data consistency. There is an unavoidable delay between when a service adds an event to the store and when all the other services update their materialized views. We call this phenomenon **eventual consistency**.

> **Eventual consistency** means that the data will be consistent at some point in the future, but not outright. The delay can be from a few milliseconds to much longer, but the goal is to keep that delay as small as possible.

Another drawback is the complexity of creating such a system compared to a single application that queries a single database. Like the microservices architecture, event sourcing is not just rainbows and unicorns. It comes at a price: **operational complexity**.

> In a microservices architecture, each piece is smaller, but gluing them together has a cost. For example, the infrastructure to support microservices is more complex than a monolith (one app and one database). The same goes for event sourcing; all applications must subscribe to one or more events, cache data (materialized view), publish events, and more. This **operational complexity** represents the shift of complexity from the application code to the operational infrastructure. In other words, it requires more work to deploy and maintain multiple microservices and databases and to fight the possible instability of network communication between those external systems than it does for a single application containing all of the code. Monoliths are simpler: they work or don't; they rarely partially work.

A crucial aspect of event sourcing is appending new events to the store and never changing existing events (append-only). In a nutshell, microservices communicating using the Pub-Sub pattern publish events, subscribe to topics, and generate materialized views to serve their clients.

Example

Let's explore an example of what could happen if we combine what we just studied. **Context**: We need to build a program that manages IoT devices. We begin by creating two microservices:

- The `DeviceTwin` microservice handles an IoT device's twin's data (digital representation of the device).
- The `Networking` microservice manages the networking-related information of IoT devices (how to reach a device).

As a visual reference, the final system could look as follows (we cover the `DeviceLocation` microservice later):

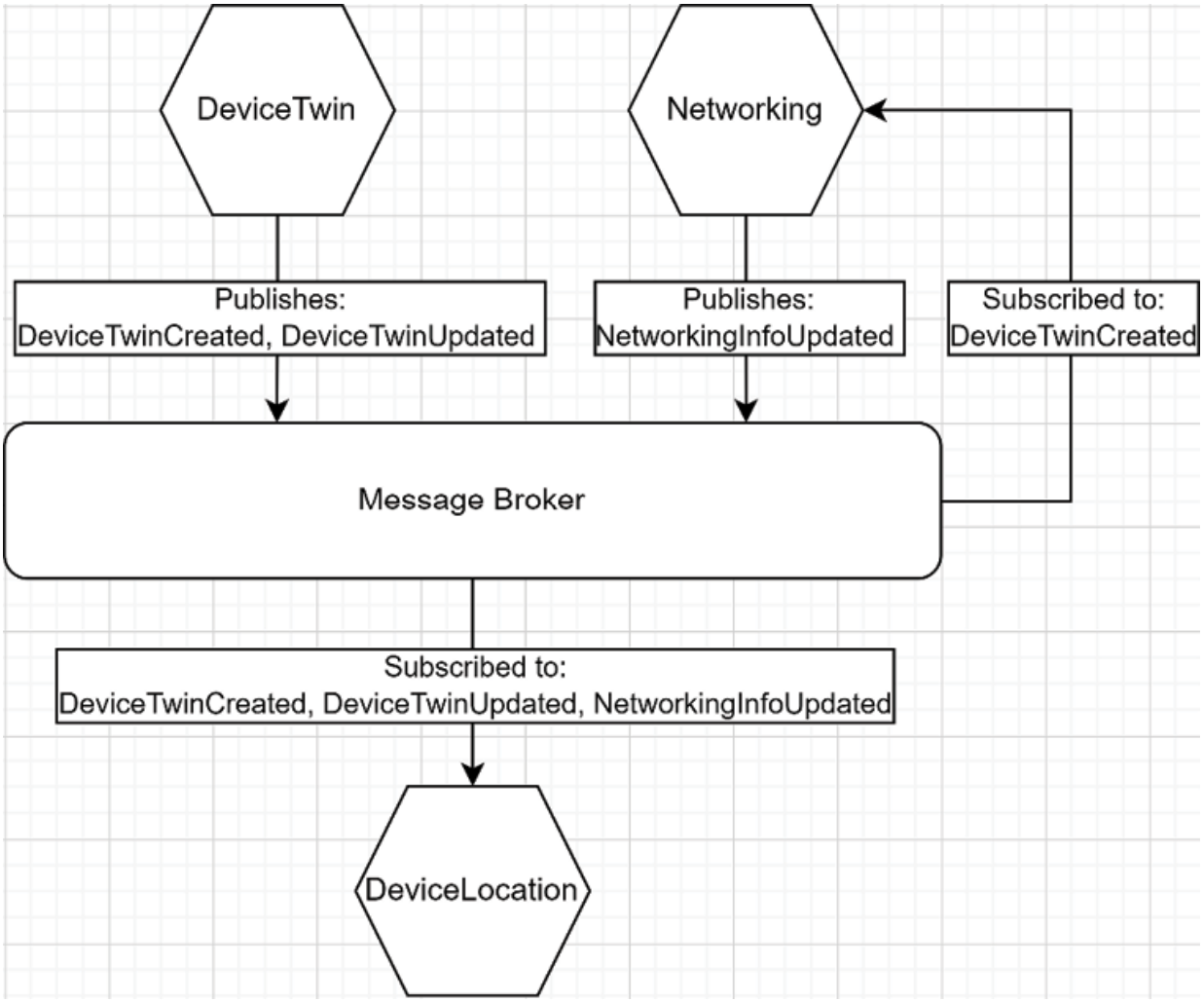


Figure 19.8: Three microservices communicating using the Publish-Subscribe pattern

Here are the user interactions and the published events:

1. A user creates a twin in the system named Device 1. The `DeviceTwin` microservice saves the data and publishes the `DeviceTwinCreated` event with the following payload:

```
{
    "id": "some id",
    "name": "Device 1",
    "other": "properties go here..."
}
```

In parallel, the `Networking` microservice must know when a device is created, so it subscribed to the `DeviceTwinCreated` event. When a new device is created, the `Networking` microservice creates default networking information for that device in its database; the default is `unknown`. This way, the `Networking` microservice knows what devices exist or not:

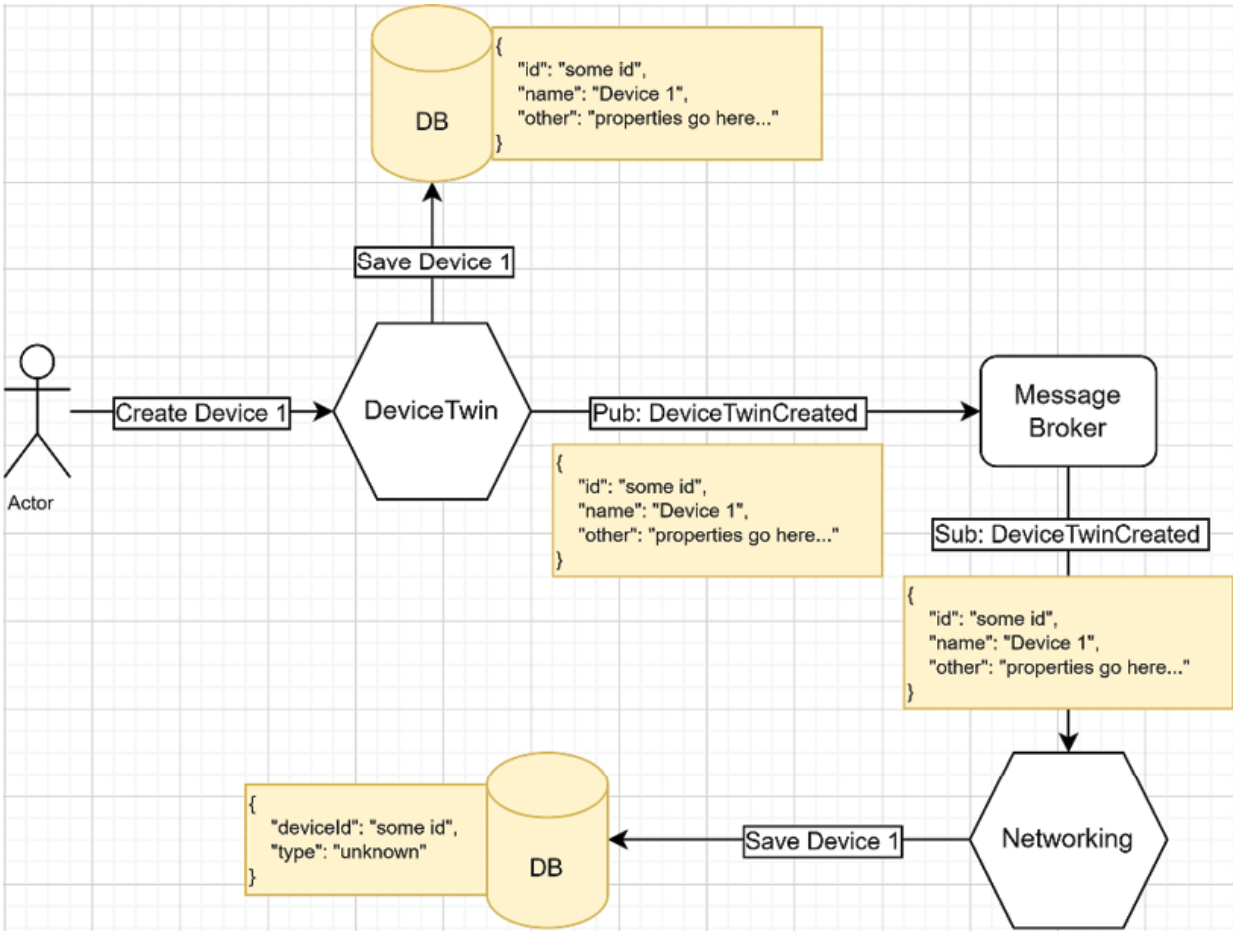


Figure 19.9: A workflow representing the creation of a device twin and its default networking information

1. A user then updates the networking information of that device and sets it to `MQTT`. The `Networking` microservice saves the data and publishes the `NetworkingInfoUpdated` event with the following payload:

```
{
    "deviceId": "some id",
    "type": "MQTT",
    "other": "networking properties..."
}
```

This is demonstrated by the following diagram:

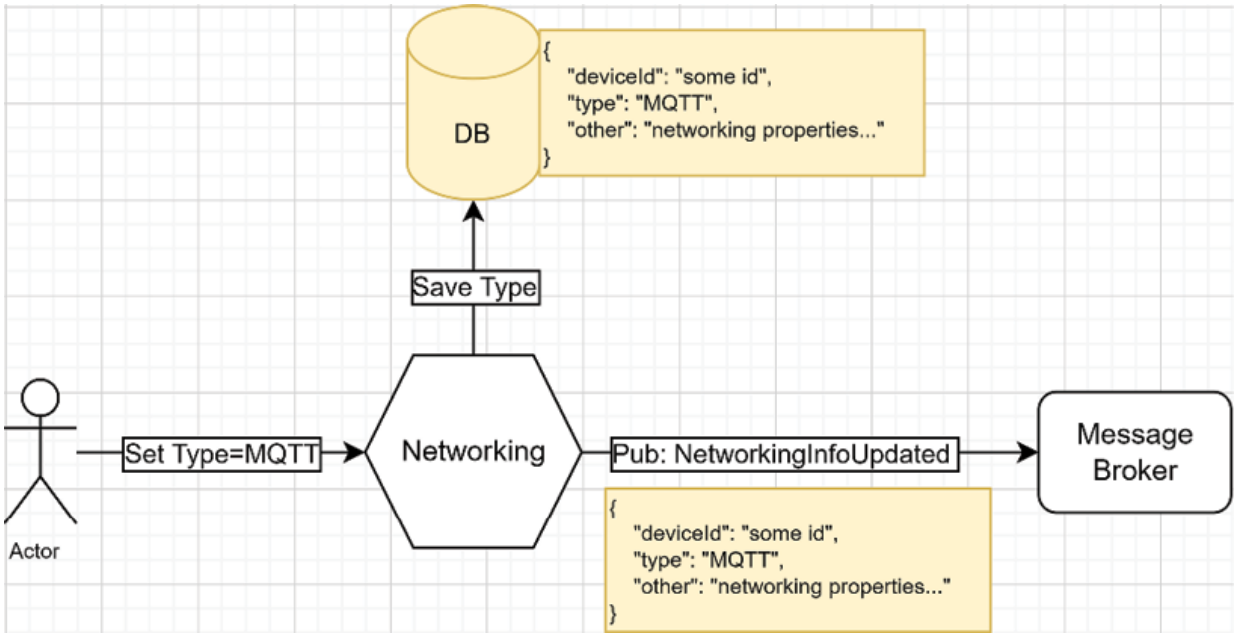


Figure 19.10: A workflow representing updating the networking type of a device

1. A user changes the device's display name to `Kitchen Thermostat`, which is more relevant. The `DeviceTwin` microservice saves the data and publishes the `DeviceTwinUpdated` event with the following payload. The payload uses **JSON patch** to publish only the differences instead of the whole object (see the *Further reading* section for more information):

```
{
    "id": "some id",
    "patches": [
        { "op": "replace", "path": "/name", "value": "Kitchen Thermostat" },
    ]
}
```

The following diagram demonstrates this:

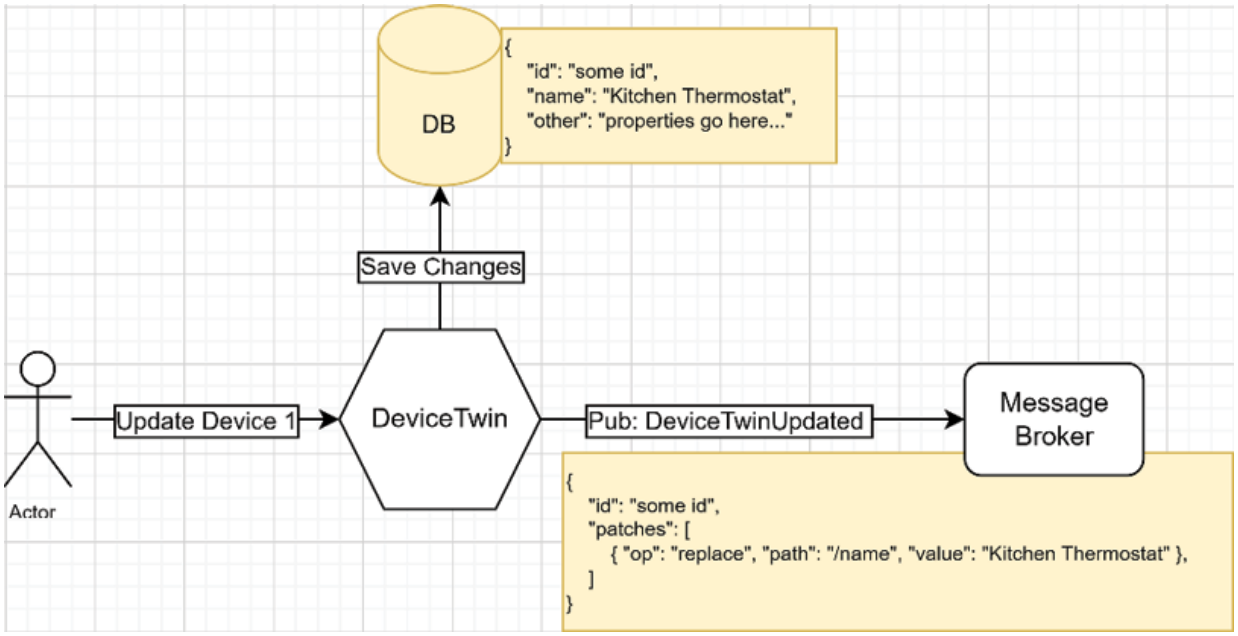


Figure 19.11: A workflow representing a user updating the name of the device to Kitchen Thermostat

From there, let's say another team designed and built a new microservice that organizes the devices at physical locations. This new `DeviceLocation` microservice allows users to visualize their devices' location on a map, such as a map of their house.The `DeviceLocation` microservice subscribes to all three events to manage its materialized view, like this:

- When receiving a `DeviceTwinCreated` event, it saves its unique identifier and display name.
- When receiving a `NetworkingInfoUpdated` event, it saves the communication type.
- When receiving a `DeviceTwinUpdated` event, it updates the device's display name.

When the service is deployed for the first time, it replays all events from the beginning (**event sourcing**); here is what happens:

1. `DeviceLocation` receives the `DeviceTwinCreated` event and creates the following model for that object:

```
{
    "device": {
        "id": "some id",
        "name": "Device 1"
    },
    "networking": {},
    "location": {...}
}
```

The following diagram demonstrates this:

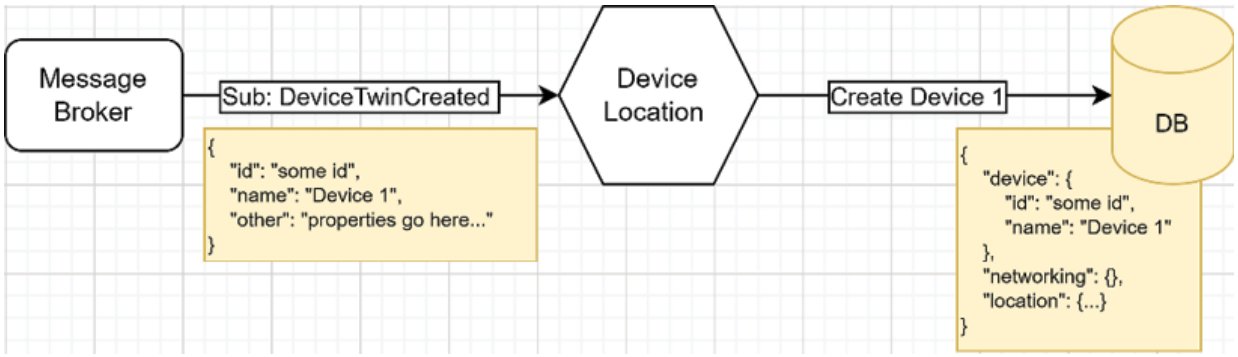


Figure 19.12: The DeviceLocation microservice replaying the DeviceTwinCreated event to create its materialized view of the device twin

1. The `DeviceLocation` microservice receives the `NetworkingInfoUpdated` event, which updates the networking type to `MQTT`, leading to the following:

```
{
    "device": {
        "id": "some id",
        "name": "Device 1"
    },
    "networking": {
        "type": "MQTT"
    },
    "location": {...}
}
```

The following diagram demonstrates this:

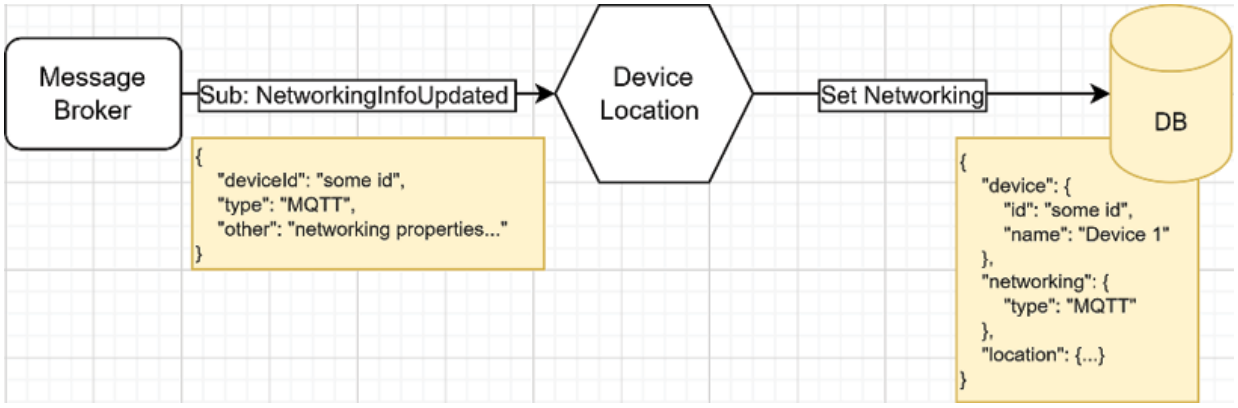


Figure 19.13: The DeviceLocation microservice replaying the NetworkingInfoUpdated event to update its materialized view of the device twin

1. The `DeviceLocation` microservice receives the `DeviceTwinUpdated` event, updating the device's name. The final model looks like this:

```
{
    "device": {
        "id": "some id",
        "name": "Kitchen Thermostat"
    },
    "networking": {
        "type": "MQTT"
    },
```

```
    "location": {...}
}
```

The following diagram demonstrates this:

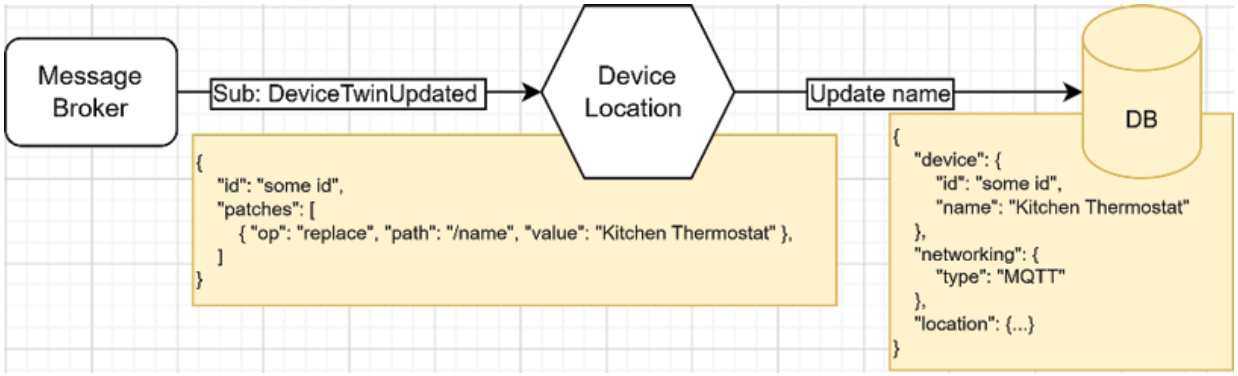


Figure 19.14: The DeviceLocation microservice replaying the DeviceTwinUpdated event to update its materialized view of the device twin

From there, the `DeviceLocation` microservice is initialized and ready. Users could set the kitchen thermostat's location on the map or continue using the other microservices. When a user queries the `DeviceLocation` microservice for information about `Kitchen Thermostat`, it displays the **materialized view**, which contains all the required information without sending external requests.With that in mind, we could spawn new instances of the `DeviceLocation` microservice or other microservices, and they could generate their materialized views from past events—all of that with very limited to no knowledge of other microservices. In this type of architecture, a microservice can only know about events, not the other microservices. How a microservice handles events should be relevant only to that microservice, never to the others. The same applies to both publishers and subscribers.This example illustrates the event sourcing pattern, integration events, the materialized view, the use of a message broker, and the Publish-Subscribe pattern.In contrast, using direct communication (HTTP, gRPC, and so on) would look like this:

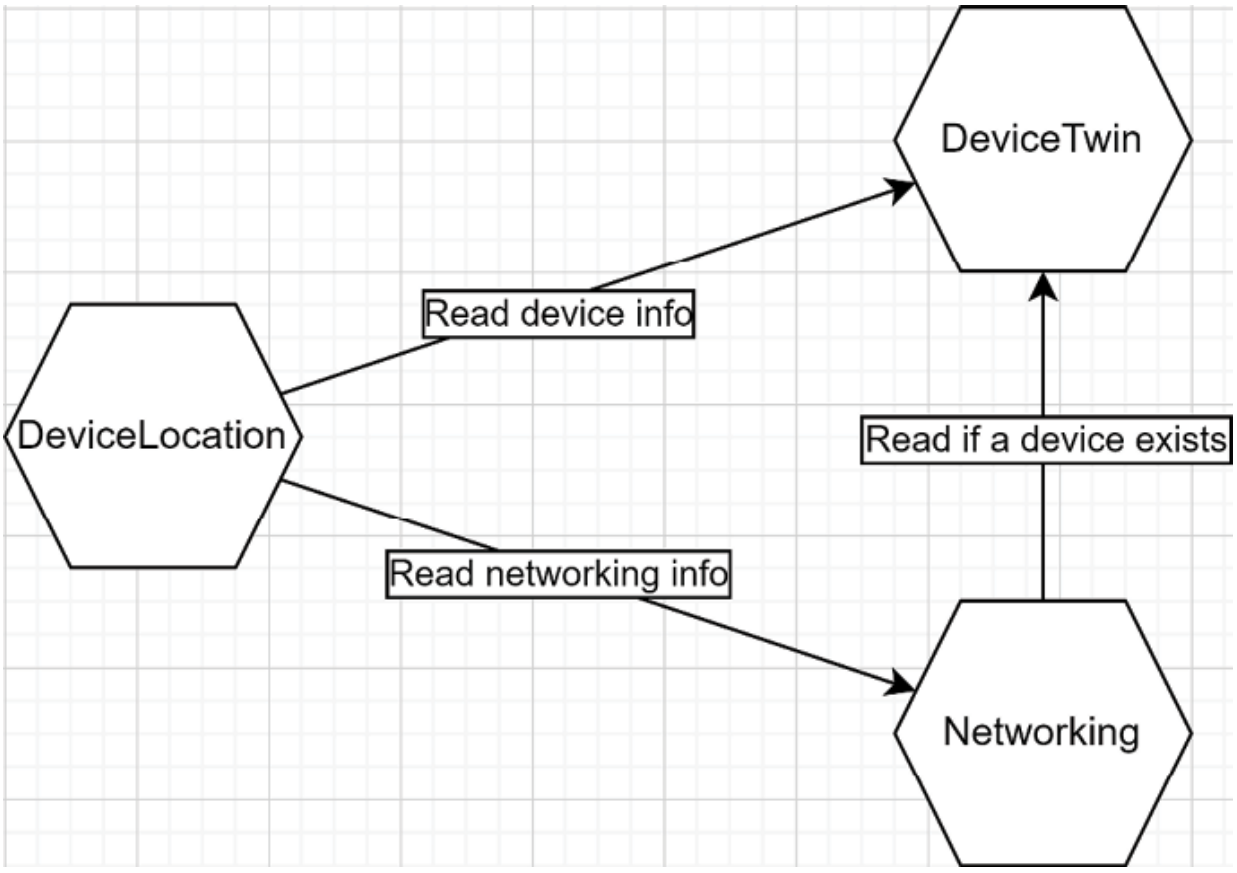


Figure 19.15: Three microservices communicating directly with one another

If we compare both approaches by looking at the first diagram (*Figure 16.7*), we can see that the message broker plays the role of a **mediator** and breaks the direct coupling between the microservices. By looking at the preceding diagram (*Figure 16.14*), we can see the tight coupling between the microservices, where the `DeviceLocation` microservice would need to interact with the `DeviceTwin` and `Networking` microservices directly to build the equivalent of its materialized view. Furthermore, the `DeviceLocation` microservice translates one interaction into three since the `Networking` microservice also talks to the `DeviceTwin` microservice, leading to indirect tight coupling between microservices, which can negatively impact performance.Suppose eventual consistency is not an option, or the Publish-Subscribe pattern cannot be applied or could be too hard to apply to your scenario. In this case, microservices can directly call each other. They can achieve this using HTTP, gRPC, or any other means that best suits that particular system's needs.I won't be covering this topic in this book, but one thing to be careful of when calling microservices directly is the indirect call chain that could bubble up fast. You don't want your microservices to create a super deep call chain, or your system will likely become very slow. Here is an abstract example of what could happen to illustrate what I mean:

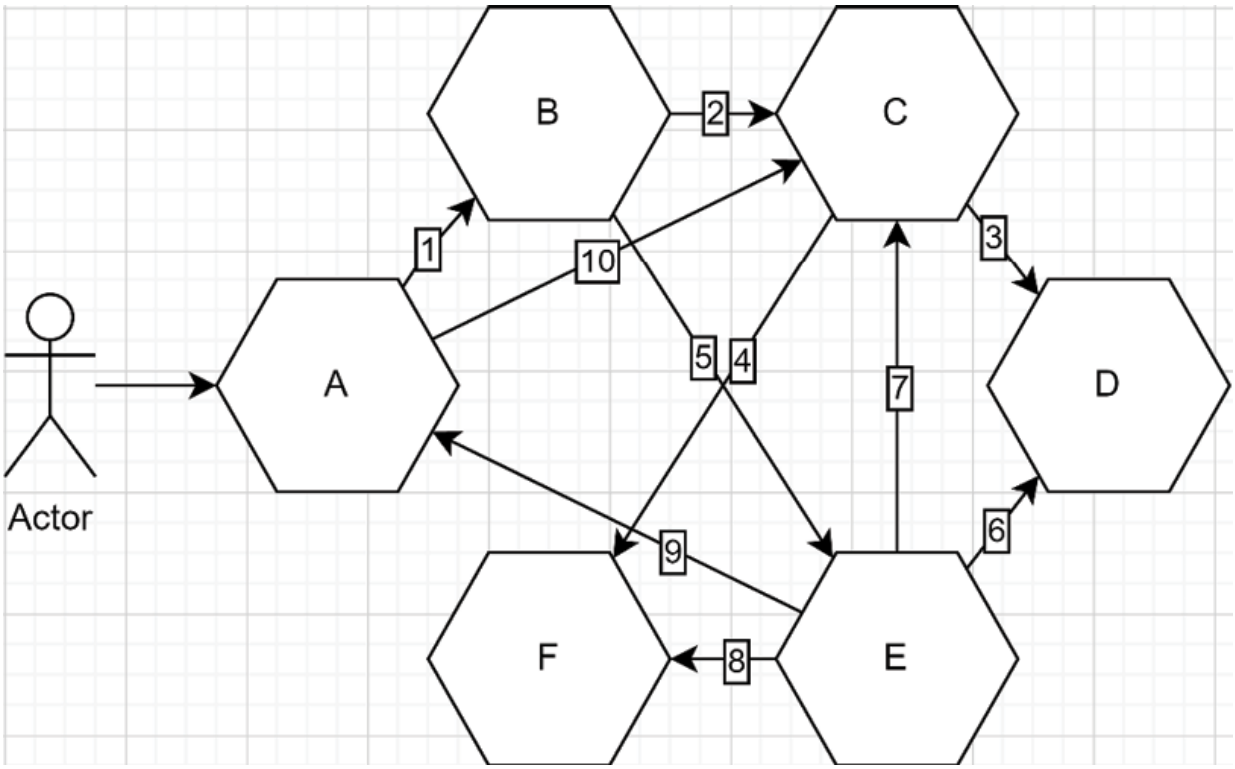


Figure 19.16: A user calling microservice A, which then triggers a chain reaction of subsequent calls, leading to disastrous performance

In terms of the preceding diagram, let's think about failures (for one). If microservice C goes offline, the whole request ends with an error. No matter the measures we put in place to mitigate the risks, if microservice C cannot recover, the system will remain down; goodbye to microservices' promise of independence. Another issue is latency: ten calls are made for a single operation; that takes time.Such chatty systems have most likely emerged from an incorrect domain modeling phase, leading to multiple microservices working together to handle trivial tasks. Now think of *Figure 16.15* but with 500 microservices instead of 6. That could be catastrophic!This type of interdependent microservices system is known as the **Death Star anti-pattern**. We can see the Death Star anti-pattern as a *distributed big ball of mud*. One way to avoid such pitfalls is to ensure that the bounded contexts are well segregated and that responsibilities are well distributed. A good domain model should allow you to avoid building a Death Star and create the "most correct" system possible instead. No matter the type of architecture you choose, if you are not building the right thing, you may end up with a big ball of mud or a Death Star. Of course, the Pub-Sub pattern and EDA can help us break the tight coupling between microservices to avoid such issues.

Conclusion

The Publish-Subscribe pattern uses events to break tight coupling between parts of an application. In a microservices architecture, we can use a message broker and integration events to allow microservices to talk to each other indirectly. The different pieces are now coupled with the data contract representing the event (its schema) instead of each other, leading to a potential gain in flexibility. One risk of this type of architecture is breaking events' consumers by publishing breaking changes in the event's format without letting them know or without having events versioning in place so they can adapt to the changes. Therefore, it is critical to think about event schema evolutions thoroughly. Most systems evolve, as will events, but since schemas are the glue between systems in a Publish-Subscribe model, it is essential to treat them as such. Some brokers, like Apache Kafka, offer a schema store and other mechanisms to help with these; some don't.Then, we can leverage the event sourcing pattern to persist those events, allowing new microservices to populate their databases by replaying past events. The event store then becomes the source of truth of those systems. Event sourcing can also become very handy for tracing and auditing purposes since the whole history is persisted. We can also replay messages to recreate the

system's state at any given point in time, making it very powerful for debugging purposes. The storage size requirement for the event store is something to consider before going down the event sourcing path. The event store could grow quite large because we have been keeping all messages since the beginning of time and could grow fast based on the number of events sent. You could compact the history to reduce the data size but lose part of the history. Once again, you must decide based on the requirements and ask yourself the appropriate questions. For example, is it acceptable to lose part of the history? How long should we keep the data? Do we want to keep the original data in cheaper storage if we need it later? Do we even need replaying capabilities? Can we afford to keep all the data forever? What are the data retention policies or regulations the system must follow? Craft your list of questions based on the specific business problem you want to solve. This advice applies to all aspects of software engineering: clearly define the business problem first, then find how to fix it. Such patterns can be compelling but take time to learn and implement. Like message queues, cloud providers offer fully managed brokers as a service. Those can be faster to get started with than building and maintaining your own infrastructure. If building servers is your thing, you can use open-source software to "economically" build your stack or pay for managed instances of such software to save yourself the trouble. The same tips as with message queues apply here; for example, you can leverage a managed service for your production environment and a local version on the developer's machine.Apache Kafka is one of the most popular event brokers that enables advanced functionalities like event streaming. Kafka has partially and fully managed cloud offerings like Confluent Cloud. Redis Pub/Sub is another open-source project with fully managed cloud offerings. Redis is also a key-value store trendy for distributed caching scenarios. Other offerings are (but are not limited to) Solace PubSub+, RabbitMQ, and ActiveMQ. Once again, I suggest comparing the offerings with your requirements to make the best choice for your scenarios.Now, let's see how the Publish-Subscribe pattern can help us follow the **SOLID** principles at cloud-scale:

- **S**: Helps centralize and divide responsibilities between applications or components without them directly knowing each other, breaking tight coupling.
- **O**: Allows us to change how publishers and subscribers behave without directly impacting the other microservices (breaking tight coupling between them).
- **L**: N/A
- **I**: Each event can be as small as needed, leading to multiple smaller communication interfaces (data contracts).
- **D**: The microservices depend on events (abstractions) instead of concretions (the other microservices), breaking tight coupling between them and inverting the dependency flow.

As you may have noticed, pub-sub is very similar to message queues. The main difference is the way messages are read and dispatched:

- Queues: Messages are pulled one at a time, consumed by one service, and then disappear.
- Pub-Sub: Messages are read in order and sent to all consumers instead of to only one, like with queues.

I intentionally kept the **Observer design pattern** out of this book since we rarely need it in .NET. C# offers multicast events, which are well-versed in replacing the Observer pattern (in most cases). If you don't know the Observer pattern, don't worry–chances are, you will never need it. Nevertheless, if you already know the Observer pattern, here are the differences between it and the Pub-Sub pattern.

> In the Observer pattern, the subject keeps a list of its observers, creating direct knowledge of their existence. Concrete observers also often know about the subject, leading to even more knowledge of other entities and more coupling.
>
> In the Pub-Sub pattern, the publisher is not aware of the subscribers; it is only aware of the message broker. The subscribers are not aware of the publishers either, only of the message broker. The publishers and subscribers are linked only through the data contract of the messages they publish or receive.
>
> We could view the Pub-Sub pattern as the distributed evolution of the Observer pattern or, more precisely, like adding a mediator to the Observer pattern.

Next, we explore some patterns that directly call other microservices by visiting a new kind of **Façade**: the **Gateway**.

Introducing Gateway patterns

When building a microservices-oriented system, the number of services grows with the number of features; the bigger the system, the more microservices you have.When you think about a user interface that has to interact with such a system, this can become tedious, complex, and inefficient (dev-wise and speed-wise). Gateways can help us achieve the following:

- Hide complexity by routing requests to the appropriate services.
- Hide complexity by aggregating responses and translating one external request into many internal ones.
- Hide complexity by exposing only the subset of features that a client needs.
- Translate a request into another protocol.

A gateway can also centralize different processes, such as logging and caching requests, authenticating and authorizing users and clients, enforcing request rate limits, and other similar policies.You can see gateways as façades, but instead of being a class in a program, it is a program of its own, shielding other programs. There are multiple variants of the Gateway pattern, and we explore many of them here.Regardless of the type of gateway you need, you can code it yourself or leverage existing tools to speed up the development process.

> Beware that there is a strong chance that your homemade gateway version 1.0 has more flaws than a proven solution. This tip is not only applicable to gateways but to most complex systems. That being said, sometimes, no proven solution does exactly what we want, and we have to code it ourselves, which is where the real fun begins!

An open-source project that could help you out is Ocelot (https://adpg.link/UwiY). It is an API gateway written in C# that supports many things that we expect from a gateway. You can route requests using configuration or write custom code to create advanced routing rules. Since it is open source, you can contribute to it, fork it, and explore the source code if necessary.If you want a managed offering with a long list of features, you can explore Azure API Management (https://adpg.link/8CEX). It supports security, load-balancing, routing, and more. It also offers a service catalog where teams can consult and manage the APIs with internal teams, partners, and customers.We can see a gateway as a **reverse proxy** that offers advanced functionalities. A Gateway fetches the information clients request, which can come from one or more resources, possibly from one or more servers. A reverse proxy usually routes a request to only one server. A reverse proxy often serves as a load balancer. Microsoft released a reverse proxy named YARP, written in C# and open-source (https://adpg.link/YARP). Microsoft built it for their internal teams. YARP is now part of Azure App Service (https://adpg.link/7eu4). If YARP does what you need, it seems like a stable enough product to invest in that will evolve and be maintained over time. A significant advantage of such a service is the ability to deploy it with your application, optionally as a container, allowing us to use it locally during development.Now, let’s explore a few types of gateways.

Gateway Routing pattern

We can use this pattern to hide the complexity of our system by having the gateway route requests to the appropriate services.For example, let’s say we have two microservices: one that holds our device data and another that manages device locations. We want to show the latest known location of a specific device (`id=102`) and display its name and model.To achieve that, a user requests the web page, and then the web page calls two services (see the following diagram). The `DeviceTwin` microservice is accessible from `service1.domain.com` , and the `Location` microservice is accessible from `service2.domain.com` . From there, the web application must track the two services, their domain name, and their operations. The UI has to handle more complexity as we add more microservices. Moreover, if we decide to change `service1` to `device-twins` and `service2` to `location` , we’d also need to update the web application. If there is only a UI, it is still not so bad, but if we have multiple user interfaces, each has to handle that complexity.Furthermore, if we want to hide the microservices inside a private network, it would be impossible unless all the user interfaces are also part of that private network (which exposes it). Here’s the diagram representing the interactions mentioned previously:

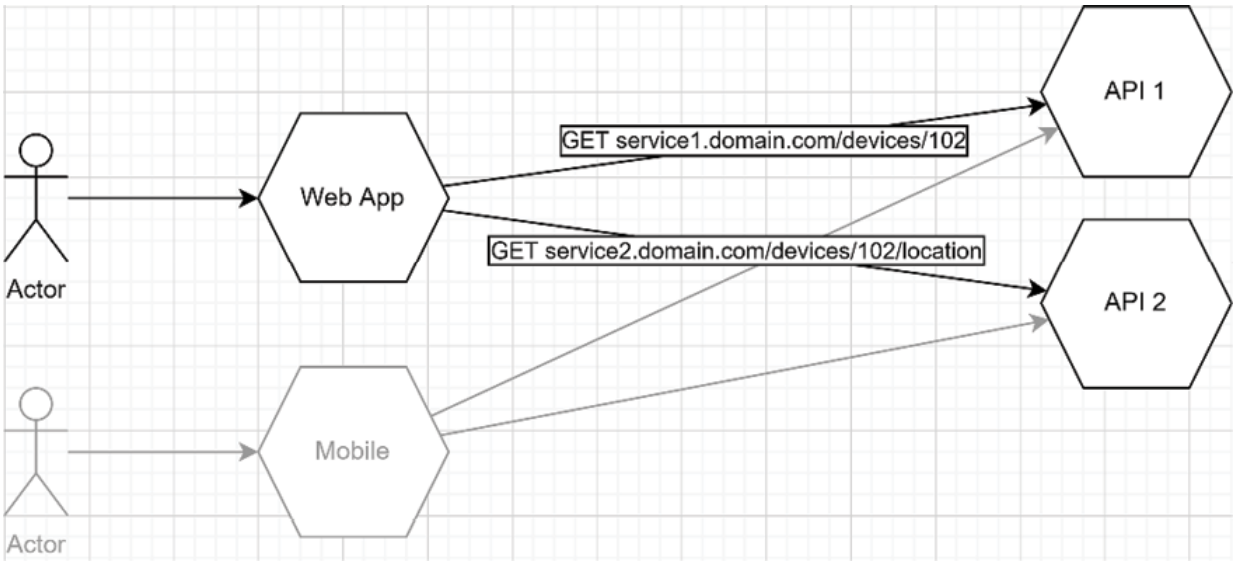


Figure 19.17: A web application and a mobile app that are calling two microservices directly

We can implement a gateway that does the routing for us to fix some of these issues. That way, instead of knowing what services are accessible through what sub-domain, the UI only has to know the gateway:

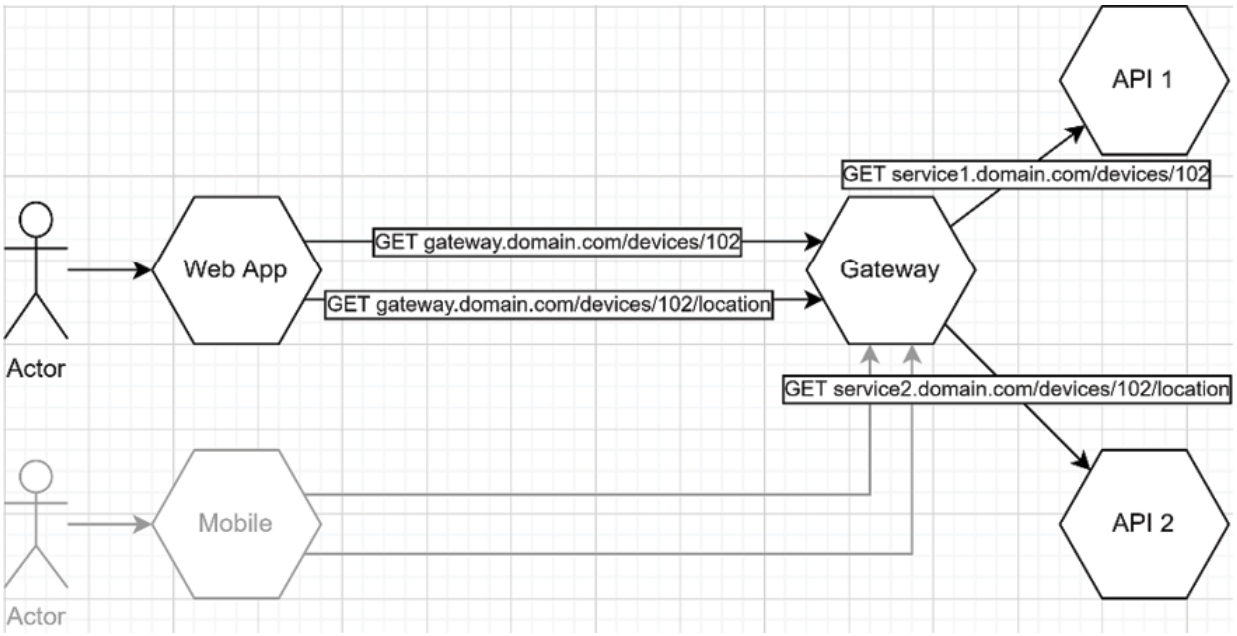


Figure 19.18: A web application and a mobile app that are calling two microservices through a gateway application

Of course, this brings some possible issues to the table as the gateway becomes a single point of failure. We could consider using a load balancer to ensure we have strong enough availability and fast enough performance. Since all requests pass through the gateway, we may also need to scale it up at some point.We should also ensure the gateway supports failure by implementing different resiliency patterns, such as **Retry** and **Circuit Breaker**. The chances that an error will occur on the other side of the gateway increase with the number of microservices you deploy and the number of requests sent to those microservices.You can also use a routing gateway to reroute the URI to create easier-to-use URI patterns. You can also reroute ports; add, update, or remove HTTP headers; and more. Let’s explore the same example but using different URIs. Let’s assume the following:

Microservice	URI
API 1 (get a device)	`internal.domain.com:8001/{id}`
API 2 (get a device location)	`internal.domain.com:8002/{id}`

Table 19.1: Internal microservice URI patterns.

UI developers would have a harder time remembering what port is leading to what microservice and what is doing what (and who could blame them?). Moreover, we could not transfer the requests as we did earlier (only routing the domain). We could use the gateway as a way to create memorable URI patterns for developers to consume, like these:

Gateway URI	Microservice URI
`gateway.domain.com/devices/{id}`	`internal.domain.com:8001/{id}`
`gateway.domain.com/devices/{id}/location`	`internal.domain.com:8002/{id}`

Table 19.1: Memorable URI patterns that are easier to use and semantically meaningful.

As we can see, we took the ports out of the equation to create usable, meaningful, and easy-to-remember URIs.However, we are still making two requests to the gateway to display one piece of information (the location of a device and its name/model), which leads us to our next Gateway pattern.

Gateway Aggregation pattern

Another role we can give to a gateway is aggregating requests to hide complexity from its consumers. Aggregating multiple requests into one makes it easier for consumers of a microservices system to interact with it; clients need to know about one endpoint instead of multiple. Moreover, it moves the chattiness from the client to the gateway, which is closer to the microservices, lowering the many calls' latency, and thus making the request-response cycle faster.Continuing with our previous example, we have two UI applications that contain a feature to show a device's location on a map before identifying it using its name/model. To achieve this, they must call the device twin endpoint to obtain the device's name and model and the location endpoint to get its last known location. So, two requests to display a small box times two UIs means four requests to maintain a simple feature. If we extrapolate, we could end up managing a huge number of HTTP requests for a handful of features.Here is a diagram showing our feature in its current state:

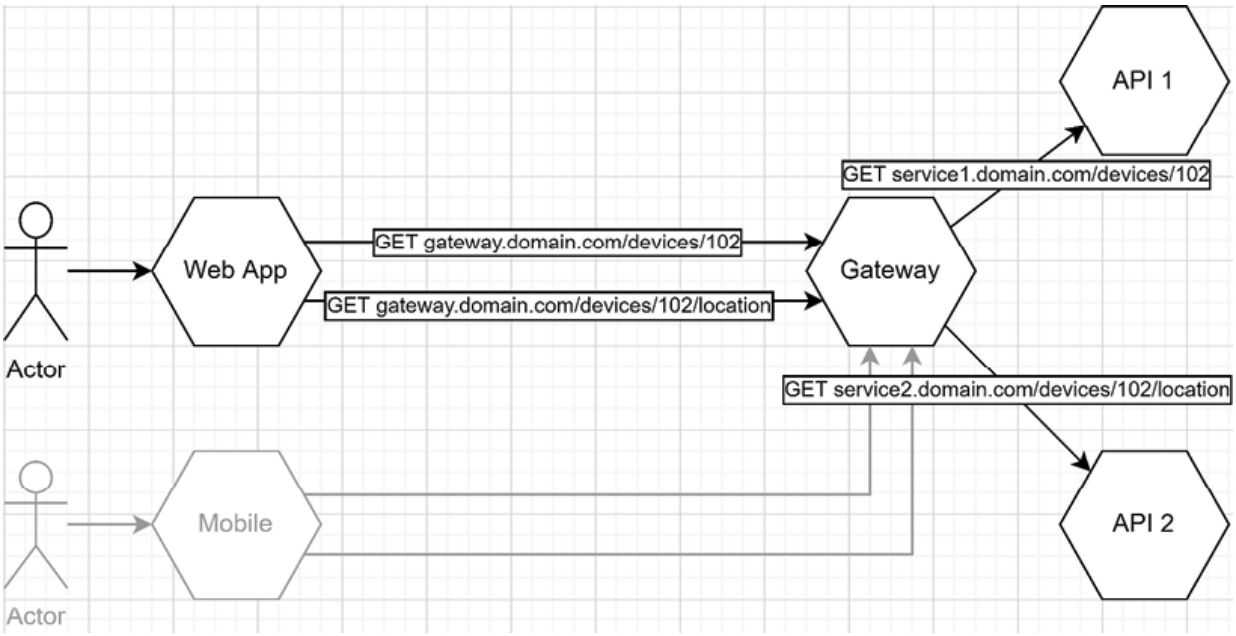


Figure 19.19: A web application and a mobile app that are calling two microservices through a gateway application

To remedy this problem, we can apply the Gateway Aggregation pattern to simplify our UIs and offload the responsibility of managing those details to the gateway.By applying the Gateway Aggregation pattern, we end up with the following simplified flow:

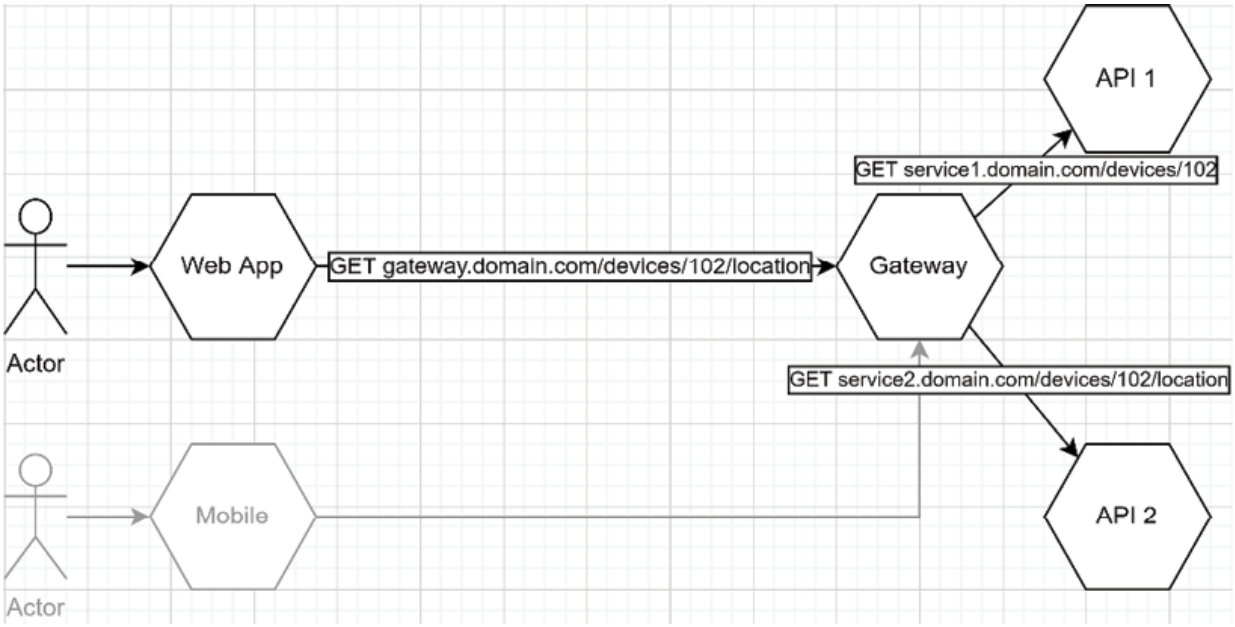


Figure 19.20: A gateway that aggregates the response of two requests to serve a single request from both a web application and a mobile app

In the previous flow, the Web App calls the Gateway that calls the two APIs, then crafts a response combining the two responses it got from the APIs. The Gateway then returns that response to the Web App. With that in place, the Web App is loosely coupled with the two APIs while the Gateway plays the intermediary. With only one HTTP request, the Web App has all the information it needs, aggregated by the Gateway.Next, let's explore the steps that occurred. The following diagram shows that the Web App makes a single request (1) while the gateway makes two calls (2 and 4). In the diagram, the requests are sent in series, but we could have sent them in parallel to speed things up:

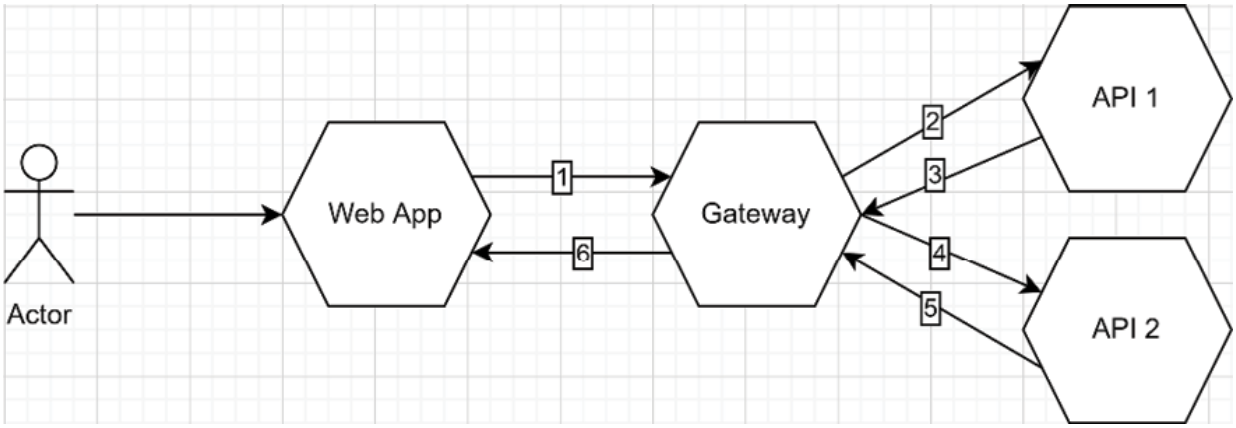


Figure 19.21: The order in which the requests take place

Like the routing gateway, an aggregation gateway can become the bottleneck of your application and a single point of failure, so beware of that.Another important point is the latency between the gateway and the internal APIs. The clients will wait for every response if the latency is too high. So, deploying the gateway close to the microservices it interacts with could become crucial for system performance. The gateway can also implement caching to improve performance further and make subsequent requests faster.Next, we explore another type of gateway that creates specialized gateways instead of generic ones.

Backend for Frontend pattern

The Backend for Frontend (BFF) pattern is yet another variation of the Gateway pattern. With Backend for Frontend, instead of building a general-purpose gateway, we build a gateway per user interface (for each application that interacts with the system), lowering the complexity. Moreover, it allows for fine-grained control of what endpoints are exposed. It removes the chances of app B breaking when changes are made to app A. Many optimizations can come out of this pattern, such as sending only the data that's required for each call instead of sending data that only a few applications are using, saving some bandwidth along the way.Let's say that our Web App needs to display more data about a device. To achieve that, we would need to change the endpoint and send that extra information to the mobile app as well. However, the mobile app doesn't need that information since it doesn't have room on its screen to display it. Next is an updated diagram that replaces the single gateway with two gateways, one per frontend:

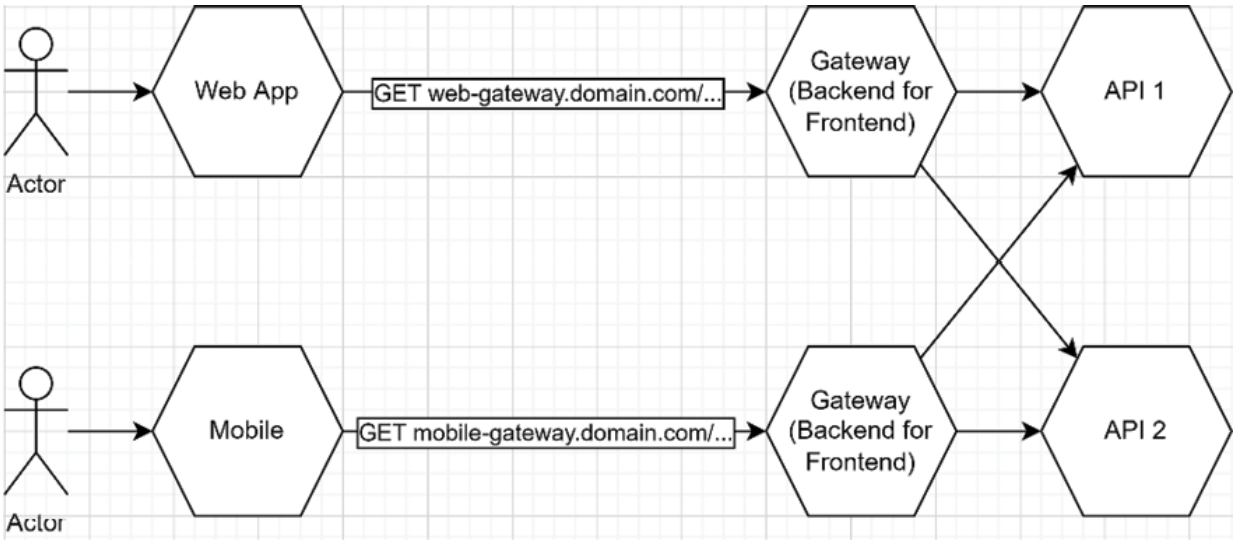


Figure 19.22: Two Backend for Frontend gateways; one for the Web App and one for the Mobile App

Doing this allows us to develop specific features for each frontend without impacting the other. Each gateway now shields its particular frontend from the rest of the system and the other frontend. This is the most important benefit this pattern brings: client independence.Once again, the Backend for Frontend pattern is a gateway. Like other variations of the Gateway pattern, it can become the bottleneck of its frontend and its single point of failure. The good news is that the outage of one BFF gateway limits the impact to a single frontend, shielding the other frontends from that downtime.

Mixing and matching gateways

Now that we've explored three variations of the Gateway pattern, it is important to note that we can mix and match them, either at the codebase level or as multiple microservices.For example, a gateway can be built for a single client (backend for frontend), perform simple routing, and aggregate results.We can also mix them as different applications, for example, by putting multiple backend for frontend gateways in front of a more generic gateway to simplify the development and maintenance of those backend for frontend gateways.Beware that each hop has a cost. The more pieces you add between your clients and your microservices, the more time it will take for those

clients to receive the response (latency). Of course, you can put mechanisms in place to lower that overhead, such as caching or non-HTTP protocols such as gRPC, but you still must consider it. That goes for everything, not just gateways.Here is an example illustrating this:

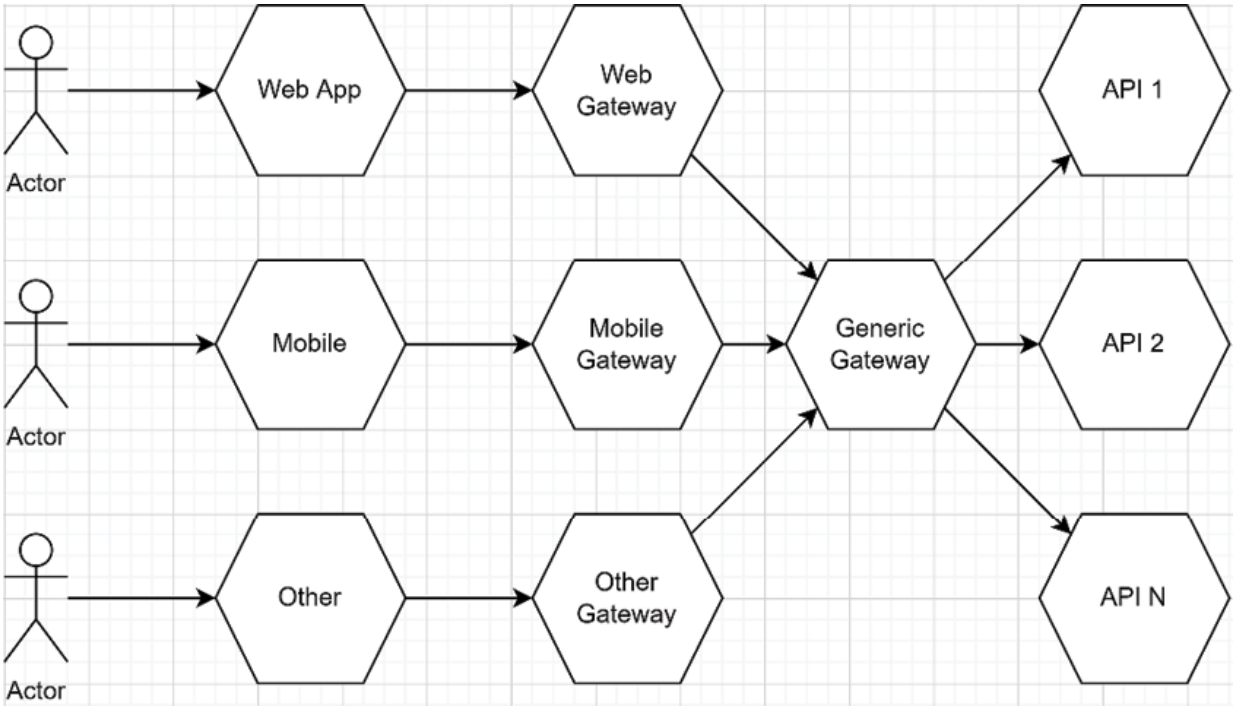


Figure 19.23: A mix of the Gateway patterns

As you've possibly guessed, the Generic Gateway is the single point of failure of all applications, while at the same time, each backend for frontend gateway is a point of failure for its specific client.

> A **service mesh** is an alternative to help microservices communicate with one another. It is a layer, outside of the application, that proxies communications between services. Those proxies are injected on top of each service and are referred to as **sidecars**. The service mesh can also help with distributed tracing, instrumentation, and system resiliency. If your system needs service-to-service communication, a service mesh would be an excellent place to look.

Conclusion

A gateway is a façade that shields or simplifies access to one or more other services. In this section, we explored the following:

- **Routing**: This forwards a request from point A to point B (a reverse proxy).
- **Aggregation**: This combines the result of multiple sub-requests into a single response.
- **Backend for Frontend**: This is used in a one-to-one relationship with a frontend.

We can use any microservices pattern, including gateways, and like any other pattern, we can mix and match them. Just consider the advantages, but also the drawbacks, that they bring to the table. If you can live with them, you've got your solution.Gateways often become the single point of failure of the system, so that is a point to consider. On the other hand, a gateway can have multiple instances running simultaneously behind a load balancer. Moreover, we must also consider the delay added by calling a service that calls another service since that slows down the response time.All in all, a gateway is a great tool to simplify consuming microservices. They also allow hiding the microservices topology behind them, possibly even isolated in a private network. They can also handle cross-cutting concerns such as security.

> It is imperative to use gateways as a requests passthrough and avoid coding business logic into them; gateways are just reverse proxies. Think single responsibility principle: a gateway is a façade in front of your microservices cluster. Of course, you can unload specific tasks into your gateways like authorization, resiliency (retry policies, for example), and similar cross-cutting concerns, but the business logic must remain in the backend microservices.
>
> The BFF's role is to simplify the UI, so moving logic from the UI to the BFF is encouraged.

In most cases, I recommend against rolling out your hand-crafted gateway and suggest leveraging existing offerings instead. There are many open-source and cloud gateways that you can use in your application. Using existing components leaves you more time to implement the business rules that solve the issues your program is trying to tackle.Of course, cloud-based offerings exist, like Azure Application Gateway and Amazon API Gateway. Both are extendable with cloud offerings like load balancers and **web application firewalls** (**WAF**). For example, Azure Application Gateway also supports autoscaling, zone redundancy, and can serve as **Azure Kubernetes Service** (**AKS**) Ingress Controller (in a nutshell, it controls the traffic to your microservices cluster).If you want more control over your gateways or to deploy them with your application, you can leverage one existing options, like Ocelot, YARP, or Envoy.Ocelot is an open source production-ready API Gateway programmed in .NET. Ocelot supports routing, request aggregation, load-balancing, authentication, authorization, rate limiting, and more. It also integrates well with Identity Server. In my eyes, the biggest advantage of Ocelot is that you create the .NET project yourself, install a NuGet package, configure your gateway, and then deploy it like any other ASP.NET Core application. Since Ocelot is written in .NET, extending it if needed or contributing to the project or its ecosystem is easier.To quote their GitHub `README.md` file: « *YARP is a reverse proxy toolkit for building fast proxy servers in .NET using the infrastructure from ASP.NET and .NET. The key differentiator for YARP is that it's been designed to be easily customized and tweaked to match the specific needs of each deployment scenario.* »Envoy is an « *open source edge and service proxy, designed for cloud-native applications* », to quote their website. Envoy is a **Cloud Native Computing Foundation** (**CNCF**) graduated project originally created by Lyft. Envoy was designed to run as a separate process from your application, allowing it to work with any programming language. Envoy can serve as a gateway and has an extendable design through TCP/UDP and HTTP filters, supports HTTP/2 and HTTP/3, gRPC, and more.Which offering to choose? If you are looking for a fully managed service, look at the cloud provider's offering of your choice. Consider YARP or Ocelot if you are looking for a configurable reverse proxy or gateway that supports the patterns covered in this chapter. If you have complex use cases that Ocelot does not support, you can look into Envoy, a proven offering with many advanced capabilities. Please remember that these are just a few possibilities that can play the role of a gateway in a microservices architecture system and are not intended to be a complete list.Now, let's see how gateways can help us follow the **SOLID** principles at cloud-scale:

- **S**: A gateway can handle routing, aggregation, and other similar logic that would otherwise be implemented in different components or applications.
- **O**: I see many ways to tackle this one, but here are two takes on this:

1. Externally, a gateway could reroute its sub-requests to new URIs without its consumers knowing about it, as long as its contract does not change.
2. Internally, a gateway could load its rules from configurations, allowing it to change without updating its code.

- **L**: N/A
- **I**: Since a backend for frontend gateway serves a single frontend system, one contract (interface) per frontend system leads to multiple smaller interfaces instead of one big general-purpose gateway.
- **D**: We could see a gateway as an abstraction, hiding the real microservices (implementations) and inverting the dependency flow.

Next, we build a BFF and evolve e-commerce application from *Chapter 18*.

Project – REPR.BFF

This project leverages the Backend for Frontend (BFF) design pattern to reduce the complexity of using the low-level API of the *REPR project* we created in *Chapter 18*. The BFF endpoints act as several types of gateway we

explore.This design makes two layers of API, so let’s start here.

Layering APIs

From a high-level architecture perspective, we can leverage multiple layers of APIs to group different levels of operation granularity. For example, in this case, we have two layers:

- Low-level APIs that offer atomic foundational operations.
- High-level APIs that offer domain-specific functionalities.

Here’s a diagram that represents this concept (high-level APIs are BFFs in this case, but the design could be nuanced):

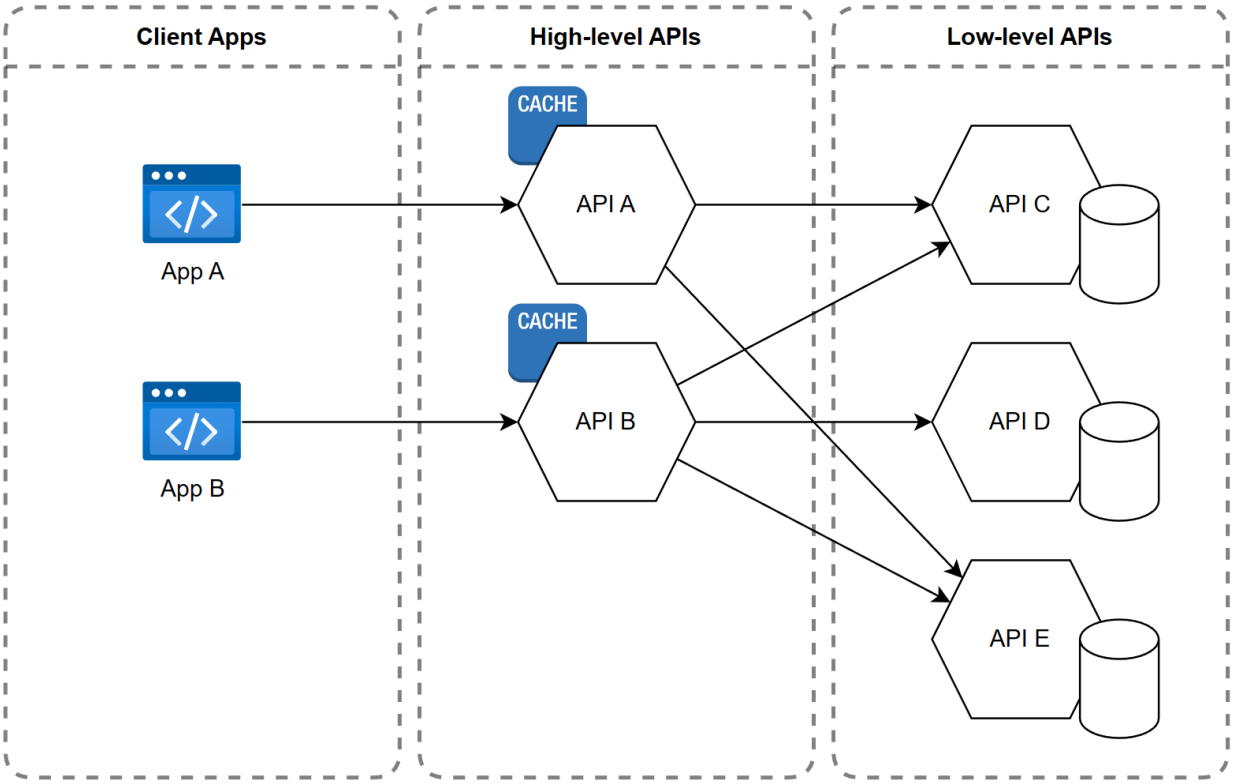


Figure 19.24: diagram showcasing a two-layer architecture.

The low-level layer showcases atomic foundational operations, like adding an item to the shopping basket and removing an item from the shopping basket. Those operations are simple, so they are more complicated to use. For example, loading the products in the user’s shopping cart requires multiple API calls, one to get the items and quantity and one per item to get the product details like its name and price. The high-level layer offers domain-specific functionalities, which are easier to use but can become more complex. For example, a single endpoint could handle adding, updating, and deleting items from the shopping basket, making its usage trivial for its consumer but its logic more complex to implement. Moreover, the product team could prefer a shopping cart to a shopping basket, so the endpoint’s URL could reflect this.Let’s have a look at the advantages and disadvantages.

Advantages of a two-layer design

- **Separation of Concerns:** This architecture separates the generic functionalities from the domain-specific ones, promoting cleaner code and modularization.
- **Scalability:** Each layer can be scaled independently based on the demand.

- **Flexibility and Reusability:** The low-level APIs can be reused across multiple high-level functionalities or applications, promoting code reusability.
- **Optimized Data Fetching:** BFFs can call multiple low-level APIs, aggregate responses, and send only the necessary data to the frontend, reducing payload sizes and making frontend development more straightforward.
- **Easier Maintenance:** We can address issues in a specific domain without touching the low-level generic APIs. On the other hand, we can fix an issue in a lower-level API, which will propagate to all the domains.
- **Tailored User Experience:** High-level APIs can be crafted specifically for individual client types (web, mobile, etc.), ensuring an optimal user experience.
- **Security:** Domain-specific functionalities can implement additional security measures relevant to their context without burdening the low-level APIs with unnecessary complexity.

Disadvantages of a two-layer design

- **Increased Complexity:** Maintaining two layers introduces additional deployment, monitoring, and management complexity.
- **Potential Performance Overhead:** An additional layer introduces latency, especially if not properly optimized.
- **Duplication:** There's potential for code duplication when similar logic gets implemented in multiple high-level functionalities.
- **Tight Coupling Concerns:** Changes in the low-level APIs can impact multiple domain-specific functionalities. A poor design could lead to a tightly coupled distributed system.
- **Coordination Required:** As the system evolves, ensuring that the low-level APIs meet the needs of all high-level functionalities requires more coordination among development teams.
- **Overhead in Development:** Developers need to consider two layers, which can slow down the development process, especially if there's a need to modify both layers to achieve a specific feature or fix.
- **Potential for Stale Data:** If high-level functionalities cache data from low-level APIs, there's potential for serving stale data to users.
- **Increased Risk of Failures:** Introducing additional APIs increases the odds of one of them experiencing issues or outages.

While a two-layer design can offer flexibility and optimization, it also introduces additional complexities. The decision to use such an architecture should be based on the specific needs of the project, the anticipated scale, and the capabilities of the development and operations teams.We look at booting up these APIs next.

Running the microservices

Let’s start by exploring the deployment topology. First, we split the *Chapter 18* REPR project into two services: *Baskets* and *Products*. Then, we add a *BFF* API that fronts the two services to simplify using the system. We do not have a UI per se, but one `http` file per project exists to simulate HTTP requests. Here’s a diagram that represents the relationship between the different services:

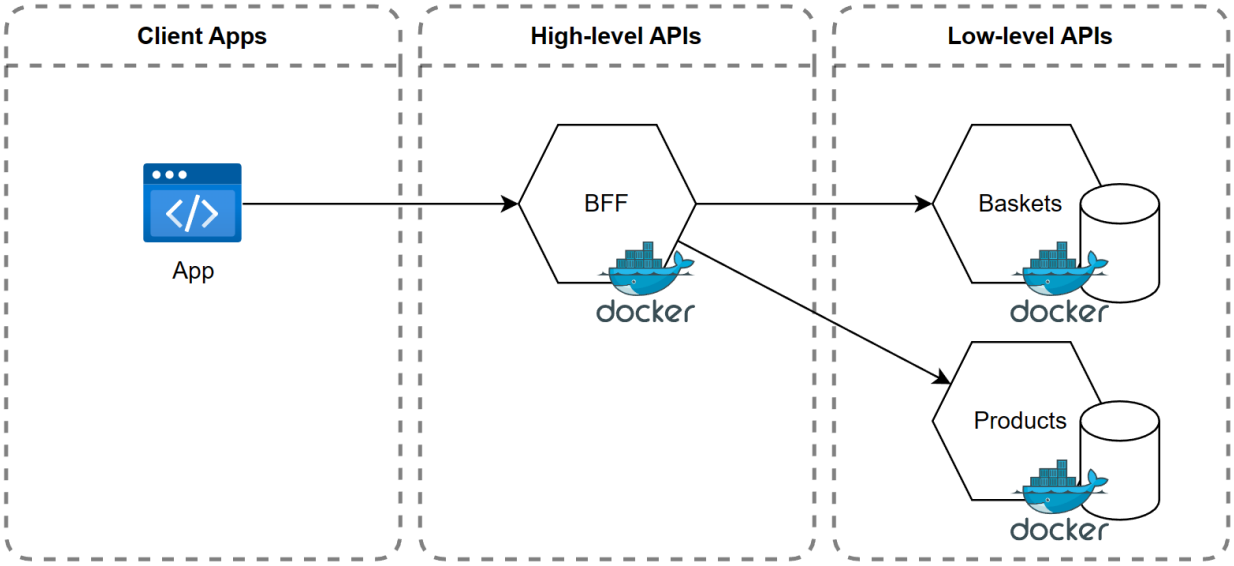


Figure 19.25: a diagram that represents the deployment topology and relationship between the different services

The easiest and most extendable way to start the projects is to use Docker, but it is optional; we can also start the three projects manually. Using Docker opens many possibilities, like using a real SQL Server to persist the data between runs and add more pieces to our puzzle, like a Redis cache or an event broker, to name a few.Let's start by manually starting the apps.

Manually starting the projects

We have three projects and need three terminals to start them all. From the chapter directory, you can execute the following commands, one set per terminal window, and all projects should start:# In one terminal

```
cd REPR.Baskets
dotnet run
# In a second terminal
cd REPR.Products
dotnet run
# In a third terminal
cd REPR.BFF
dotnet run
```

Doing this should work. You can use the `PROJECT_NAME.http` files to test the APIs.Next, let's explore the second option about using Docker.

Using Docker Compose to run the projects

At the same level as the solution file, the `docker-compose.yml`, `docker-compose.override.yml`, and various `Dockerfile` files are preconfigured to make the projects start in the correct order.

Here's a link to get started with Docker: https://adpg.link/1zfM

Since ASP.NET Core uses HTTPS by default, we must register a development certificate with the container, so let's start here.

Configuring HTTPS

This section quickly explores using PowerShell to set up HTTPS on Windows. If you are using a different operating system or if the instructions are not working, please consult the official documentation:

https://adpg.link/o1tuFirst, we must generate a development certificate. In a PowerShell terminal, run the following commands:

```
dotnet dev-certs https -ep "$env:APPDATA\ASP.NET\Https\adpg-net8-chapter-19.pfx" -p devpassword
dotnet dev-certs https --trust
```

The preceding commands create a `pfx` file with the password `devpassword` (you must provide a password, or it won't work), then tell .NET to trust the dev certificates.From there, the `ASPNETCORE_Kestrel__Certificates__Default__Path` and `ASPNETCORE_Kestrel__Certificates__Default__Password` environment variables are configured in the `docker-compose.override.yml` file and should be taken into account.

> If you change the certificate location or the password, you must update the `docker-compose.override.yml` file.

Composing the application

Now that we set up HTTPS, we can build the container using the following commands:

```
docker compose build
```

We can execute the following command to start the containers:

```
docker compose up
```

This should start the containers and feed you an aggregated log with a color per service. The beginning of the log trail should look like this:

```
[+] Running 3/0
 ✔ Container c19-repr.products-1  Created    0.0s
 ✔ Container c19-repr.baskets-1   Created    0.0s
 ✔ Container c19-repr.bff-1       Created    0.0s
Attaching to c19-repr.baskets-1, c19-repr.bff-1, c19-repr.products-1
c19-repr.baskets-1   | info: Microsoft.Hosting.Lifetime[14]
c19-repr.baskets-1   |       Now listening on: http://[::]:80
c19-repr.baskets-1   | info: Microsoft.Hosting.Lifetime[14]
c19-repr.baskets-1   |       Now listening on: https://[::]:443
...
```

To stop the services, press `Ctrl+C`. When you want to destroy the running application, enter the following command:

```
docker compose down
```

Now, with `docker compose up`, our services should be running. To make sure, let's try them out.

Briefly testing the services

The project contains the following services, each containing an `http` file you can leverage to query the services using Visual Studio or in VS Code using an extension:

Service	HTTP file	Host
`REPR.Baskets`	`REPR.Baskets.http`	https://localhost:60280
`REPR.BFF`	`REPR.BFF.http`	https://localhost:7254
`REPR.Products`	`REPR.Products.http`	https://localhost:57362

Table 19.3: each service, HTTP file, and HTTPS hostname and port.

We can leverage the HTTP requests from each directory to test the API. I suggest starting by trying the low-level APIs, then the BFF, so you know if something is wrong with them directly instead of wondering what is wrong with the BFF (which calls the low-level APIs).

I use the *REST Client* extension in VS Code (https://adpg.link/UCGv) and the built-in support in Visual Studio 2022 version 17.6 or later.

Here's a part of the `REPR.Baskets.http` file:

```
@Web_HostAddress = https://localhost:60280
@ProductId = 3
@CustomerId = 1
GET {{Web_HostAddress}}/baskets/{{CustomerId}}
###
POST {{Web_HostAddress}}/baskets
Content-Type: application/json
{
    "customerId": {{CustomerId}},
    "productId": {{ProductId}},
    "quantity": 10
}
...
```

The highlighted lines are variables that the requests reuse. The `###` characters act as a separator between requests. In VS or VS Code, you should see a `Send request` button on top of each request. Executing the `POST` request, then the `GET` should output the following:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
[
  {
    "productId": 3,
    "quantity": 10
  }
]
```

If you can reach one endpoint, this means the service is running. Nonetheless, feel free to play with the requests, modify them, and add more.

I did not move the tests over from *Chapter 18*. Automating the validation of our deployment could be a good exercise for you to test your testing skills.

After you validate that the three services are running, we can continue and look at how the BFF communicates with the Baskets and Products services.

Creating typed HTTP clients using Refit

The BFF service must communicate to the Baskets and Products services. The services are REST APIs, so we must leverage HTTP. We could leverage the out-of-the-box ASP.NET Core `HttpClient` class and `IHttpClientFactory` interface, then send raw HTTP requests to the downstream APIs. On the other hand, we could also create a typed client, which translates the HTTP calls to simple method calls with evocative names. We are exploring the second option, encapsulating the HTTP calls inside the typed clients.The concept is simple: we create one interface per service and translate its operation into methods. Each interface revolves around a service. Optionally, we can aggregate the services under a master interface to inject the aggregate service and have access to all child services. Moreover, this central access point allows us to reduce the number of injected services to one and improve discoverability with IntelliSense. Here's a diagram representing this concept:

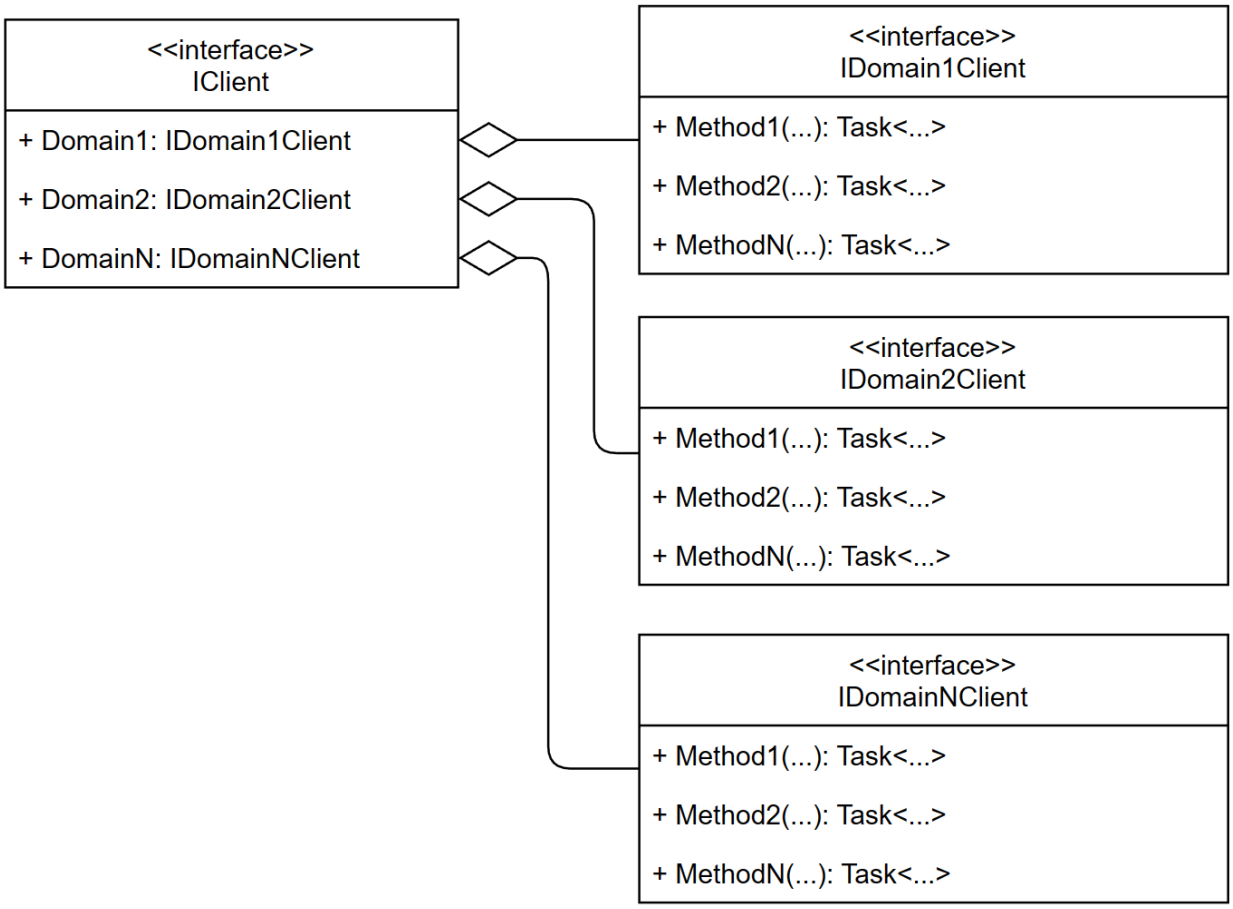


Figure 19.25: UML class diagram representing a generic typed client class hierarchy.

In the preceding diagram, the `IClient` interface is composed and exposes the other typed clients, each of which queries a specific downstream API.In our case, we have two downstream services, so our interface hierarchy looks like the following:

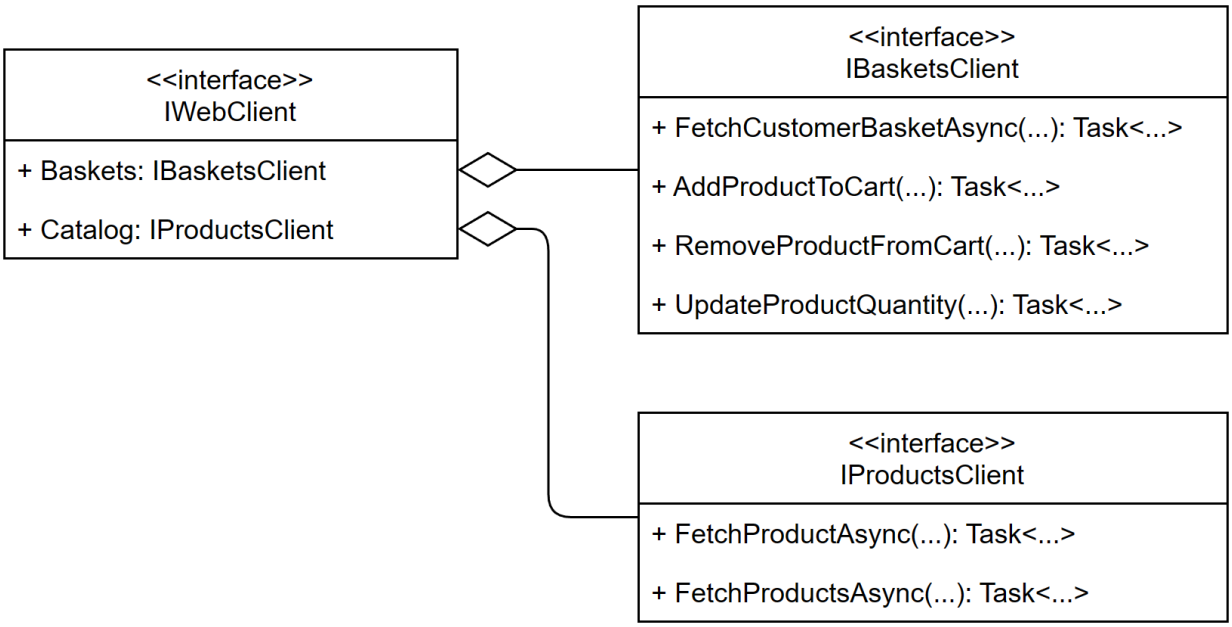


Figure 19.26: UML class diagram representing the BFF downstream typed client class hierarchy.

After implementing this, we can query the downstream APIs from our code without worrying about their data contract because our client is strongly typed.We leverage *Refit*, an open-source library, to implement the interfaces automatically.

> We could use any other library or barebone ASP.NET Core `HttpClient`; it does not matter. I picked *Refit* to leverage its code generator, save myself the trouble of writing the boilerplate code, and save you the time of reading through such code. Refit on GitHub: https://adpg.link/hneJ.
>
> > I used the out-of-the-box `IHttpClientFactory` functionalities in the past, so if you want to reduce the number of dependencies in your project, you can also use that instead. Here's a link to help you get started: https://adpg.link/HCj7.

Refit acts like Mapperly and generates code based on attributes, so all we have to do is define our methods, and Refit writes the code.

> The *BFF* project references the *Products* and *Baskets* projects to reuse their DTOs. I could have architected this in many different ways, including hosting the typed client in a library of its own so we could share it between many projects. We could also extract the DTOs from the web applications to one or more shared projects so we don't depend on the web applications themselves. For this demo, there is no need to overengineer the solution.

Let's look at the typed client interfaces, starting with the `IBasketsClient` interface:

```
using Refit;
using Web.Features;
namespace REPR.BFF;
public interface IBasketsClient
{
    [Get("/baskets/{query.CustomerId}")]
    Task<IEnumerable<Baskets.FetchItems.Item>> FetchCustomerBasketAsync(
        Baskets.FetchItems.Query query,
        CancellationToken cancellationToken);
    [Post("/baskets")]
    Task<Baskets.AddItem.Response> AddProductToCart(
        Baskets.AddItem.Command command,
        CancellationToken cancellationToken);
```

```
    [Delete("/baskets/{command.CustomerId}/{command.ProductId}")]
    Task<Baskets.RemoveItem.Response> RemoveProductFromCart(
        Baskets.RemoveItem.Command command,
        CancellationToken cancellationToken);
    [Put("/baskets")]
    Task<Baskets.UpdateQuantity.Response> UpdateProductQuantity(
        Baskets.UpdateQuantity.Command command,
        CancellationToken cancellationToken);
}
```

The preceding interface leverages Refit's attributes (highlighted) to explain to its code generator what to write. The operations themselves are self-explanatory and carry the features' DTOs over HTTP.Next, we look at the `IProductsClient` interface:

```
using Refit;
using Web.Features;
namespace REPR.BFF;
public interface IProductsClient
{
    [Get("/products/{query.ProductId}")]
    Task<Products.FetchOne.Response> FetchProductAsync(
        Products.FetchOne.Query query,
        CancellationToken cancellationToken);
    [Get("/products")]
    Task<Products.FetchAll.Response> FetchProductsAsync(
        CancellationToken cancellationToken);
}
```

The preceding interface is similar to `IBasketsClient` but creates a typed bridge on the *Products* API.

> The generated code contains much gibberish code and would be very hard to clean enough to make it relevant to study, so let's assume those interfaces have working implementations instead.

Next, let's look at our aggregate:

```
public interface IWebClient
{
    IBasketsClient Baskets { get; }
    IProductsClient Catalog { get; }
}
```

The preceding interface exposes the two clients we had Refit generate for us. Its implementation is fairly straightforward as well:

```
public class DefaultWebClient : IWebClient
{
    public DefaultWebClient(IBasketsClient baskets, IProductsClient catalog)
    {
        Baskets = baskets ?? throw new ArgumentNullException(nameof(baskets));
        Catalog = catalog ?? throw new ArgumentNullException(nameof(catalog));
    }
    public IBasketsClient Baskets { get; }
    public IProductsClient Catalog { get; }
}
```

The preceding default implementation composes itself through constructor injection, exposing the two typed clients.Of course, dependency injection means we must register services with the container. Let's start with some configuration. To make the setup code parametrizable and allow the Docker container to override those values, we extract the services base addresses to the settings file like this (`appsettings.Development.json`):

```
{
  "Downstream": {
    "Baskets": {
      "BaseAddress": "https://localhost:60280"
    },
    "Products": {
      "BaseAddress": "https://localhost:57362"
```

```
    }
  }
}
```

The preceding code defines two keys, one per service, which we then load individually in the `Program.cs` file, like this:

```
using Refit;
using REPR.BFF;
using System.Collections.Concurrent;
using System.Net;
var builder = WebApplication.CreateBuilder(args);
var basketsBaseAddress = builder.Configuration
    .GetValue<string>("Downstream:Baskets:BaseAddress") ?? throw new NotSupportedException("Canno
var productsBaseAddress = builder.Configuration
    .GetValue<string>("Downstream:Products:BaseAddress") ?? throw new NotSupportedException("Cann
```

The preceding code loads the two configurations into variables.

> We can leverage all the techniques we learned in *Chapter 9*, *Options, Settings, and Configuration*, to create a more elaborate system.

Next, we register our Refit clients like this:

```
builder.Services
    .AddRefitClient<IBasketsClient>()
    .ConfigureHttpClient(c => c.BaseAddress = new Uri(basketsBaseAddress))
;
builder.Services
    .AddRefitClient<IProductsClient>()
    .ConfigureHttpClient(c => c.BaseAddress = new Uri(productsBaseAddress))
;
```

In the preceding code, calling the `AddRefitClient` method replaces the .NET `AddHttpClient` method and registers our auto-generated client with the container. Because Refit registration returns an `IHttpClientBuilder` interface, we can use the `ConfigureHttpClient` method to configure the `HttpClient` as we would any other typed HTTP client. In this case, we set the `BaseAddress` property to the values of the previously loaded settings.Next, we must also register our aggregate:

```
builder.Services.AddTransient<IWebClient, DefaultWebClient>();
```

I picked a transient state because the service only fronts other services, so it will serve the other services as they are registered, regardless of whether it is the same instance every time. Moreover, it needs a transient or scoped lifetime because the BFF must manage who is the current customer, not the client. It would be quite a security vulnerability to allow users to decide who they want to impersonate for every request.

> The project does not authenticate the users, but the service we explore next is designed to make this evolve, abstracting and managing this responsibility so we could add authentication without impacting the code we are writing.

Let’s explore how we manage the current user.

Creating a service that serves the current customer

To keep the project simple, we are not using any authentication or authorization middleware, yet we want our BFF to be realistic and to handle who’s querying the downstream APIs. To achieve this, let’s create the `ICurrentCustomerService` interface that abstracts this away from the consuming code:

```
public interface ICurrentCustomerService
{
    int Id { get; }
}
```

The only thing that interface does is provide us with the identifier representing the current customer. Since we do not have authentication in the project, let's implement a development version that always returns the same value:

```
public class FakeCurrentCustomerService : ICurrentCustomerService
{
    public int Id => 1;
}
```

Finally, we must register it in the `Program.cs` class like this:

```
builder.Services.AddScoped<ICurrentCustomerService, FakeCurrentCustomerService>();
```

With this last piece, we are ready to write some features in our BFF service.

> In a project that uses authentication, you can inject the `IHttpContextAccessor` interface into a class to access the current `HttpContext` object that contains a `User` property that enables access to the current user's `ClaimsPrincipal` object, which should include the current user's `CustomerId`. Of course, you must ensure the authentication server returns such a claim. You must register the accessor using the following method before using it: `builder.Services.AddHttpContextAccessor()`.

Features

The BFF service serves an unexisting user interface, yet we can imagine what it needs to do; it must:

- Serve the product catalog so customers can browse the shop.
- Serve a specific product to render a product details page.
- Serve the list of items in a user's shopping cart.
- Enable users to manage their shopping cart by adding, updating, and removing items.

Of course, the list of features could go on, like allowing the users to purchase the items, which is the ultimate goal of an e-commerce website. However, we are not going that far. Let's start with the catalog.

Fetching the catalog

The catalog acts as a routing gateway and forwards the requests to the `Products` downstream service.The first endpoint serves the whole catalog by using our typed client (highlighted):

```
app.MapGet(
    "api/catalog",
    (IWebClient client, CancellationToken cancellationToken)
        => client.Catalog.FetchProductsAsync(cancellationToken)
);
```

Sending the following requests should hit the endpoint:

```
GET https://localhost:7254/api/catalog
```

The endpoint should respond with something like the following:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
{
  "products": [
    {
      "id": 2,
      "name": "Apple",
      "unitPrice": 0.79
    },
    {
      "id": 1,
      "name": "Banana",
      "unitPrice": 0.30
```

```
    },
    {
      "id": 3,
      "name": "Habanero Pepper",
      "unitPrice": 0.99
    }
  ]
}
```

Here's a visual representation of what happens:

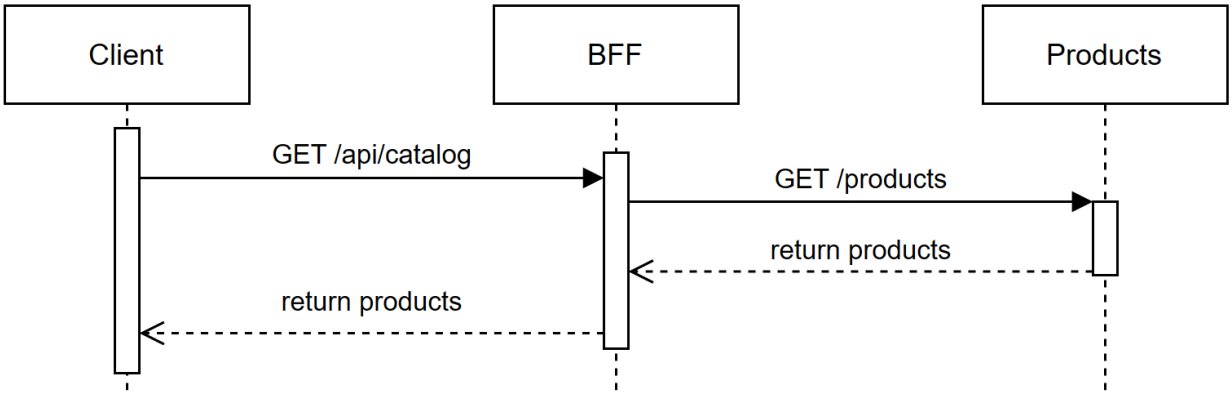


Figure 19.27: a sequence diagram representing the BFF routing the request to the Products service

The other catalog endpoint is very similar and also simply routes the request to the correct downstream service:

```
app.MapGet(
    "api/catalog/{productId}",
    (int productId, IWebClient client, CancellationToken cancellationToken)
        => client.Catalog.FetchProductAsync(new(productId), cancellationToken)
);
```

Sending an HTTP call will result in the same as calling it directly because the BFF only acts as a router.We explore more exciting features next.

Fetching the shopping cart

The *Baskets* service only stores the `customerId`, `productId`, and `quantity` properties. However, a shopping cart page displays the product name and price, but the *Products* service manages those two properties.To overcome this problem, the endpoint acts as an aggregation gateway. It queries the shopping cart and loads all the products from the *Products* service before returning an aggregated result, removing the burden of managing this complexity from the client/UI.Here's the code main feature code:

```
app.MapGet(
    "api/cart",
    async (IWebClient client, ICurrentCustomerService currentCustomer, CancellationToken cancella
    {
        var basket = await client.Baskets.FetchCustomerBasketAsync(
            new(currentCustomer.Id),
            cancellationToken
        );
        var result = new ConcurrentBag<BasketProduct>();
        await Parallel.ForEachAsync(basket, cancellationToken, async (item, cancellationToken) =>
        {
            var product = await client.Catalog.FetchProductAsync(
                new(item.ProductId),
                cancellationToken
            );
            result.Add(new BasketProduct(
                product.Id,
```

```
                product.Name,
                product.UnitPrice,
                item.Quantity
            ));
        });
        return result;
    }
);
```

The preceding code starts by fetching the items from the Baskets service and then loads the products using the `Parallel.ForEachAsync` method before returning the aggregated result.The `Parallel` class allows us to execute multiple operations in parallel, in this case, multiple HTTP calls. There are many ways of achieving a similar result using .NET, and this is one of those. When an HTTP call succeeds, it adds a `BasketProduct` item to the `result` collection. Once all operations are completed, the endpoint returns the collection of `BasketProduct` objects, which contains all the combined information required by the user interface to display the shopping cart. Here's the `BasketProduct` class:

```
public record class BasketProduct(int Id, string Name, decimal UnitPrice, int Quantity)
{
    public decimal TotalPrice => UnitPrice * Quantity;
}
```

The sequence of this endpoint is like this (the `loop` represents the `Parallel.ForEachAsync` method):

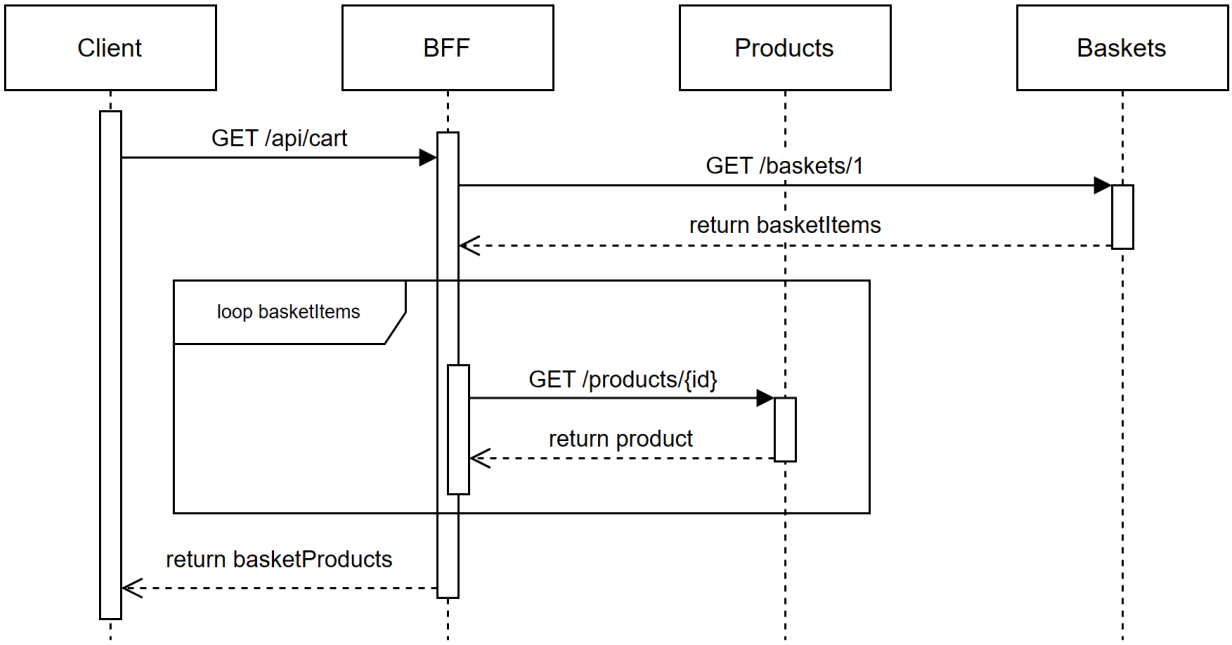


Figure 19.28: A sequence diagram representing the shopping cart endpoint interacting with the Products and the Baskets downstream services.

Since the requests to the *Products* service are sent in parallel, we cannot predict the order they will complete. Here is an excerpt from the application log depicting what can happen (I omitted the logging code in the book, but it is available on GitHub):

```
trce: GetCart[0]
      Fetching product '3'.
trce: GetCart[0]
      Fetching product '2'.
trce: GetCart[0]
      Found product '2'(Apple).
trce: GetCart[0]
      Found product '3'(Habanero Pepper).
```

The preceding trace shows that we requested products `3` and `2` but received inverted responses (`2` and `3`). This is a possibility when running code in parallel.When we send the following request to the BFF:

```
GET https://localhost:7254/api/cart
```

The BFF returns a response similar to the following:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
[
  {
    "id": 3,
    "name": "Habanero Pepper",
    "unitPrice": 0.99,
    "quantity": 10,
    "totalPrice": 9.90
  },
  {
    "id": 2,
    "name": "Apple",
    "unitPrice": 0.79,
    "quantity": 5,
    "totalPrice": 3.95
  }
]
```

The preceding example showcases the aggregated result, simplifying the logic the client (UI) must implement to display the shopping cart.

> Since we are not ordering the results, the items will not always be in the same order. As an exercise, you could sort the results using one of the existing properties or add a property that saves when a customer adds the item to the cart and sort the items using this new property; the first item added is displayed first, and so on.

Let’s move to the last endpoint and explore how the BFF manages the shopping cart items.

Managing the shopping cart

One of the primary goals of our BFF is to reduce the frontend’s complexity. When examining the *Baskets* service, we realized it would add a bit of avoidable complexity if we were only to serve the raw operation, so instead, we decided to encapsulate all of the shopping cart logic behind a single endpoint. When a client POST to the `api/cart` endpoint, it:

- Adds a non-existent item.
- Update an existing item’s quantity.
- Remove an item that has a quantity equal to 0 or less.

With this endpoint, the clients don’t have to worry about adding or updating. Here’s a simplified sequence diagram that represents this logic:

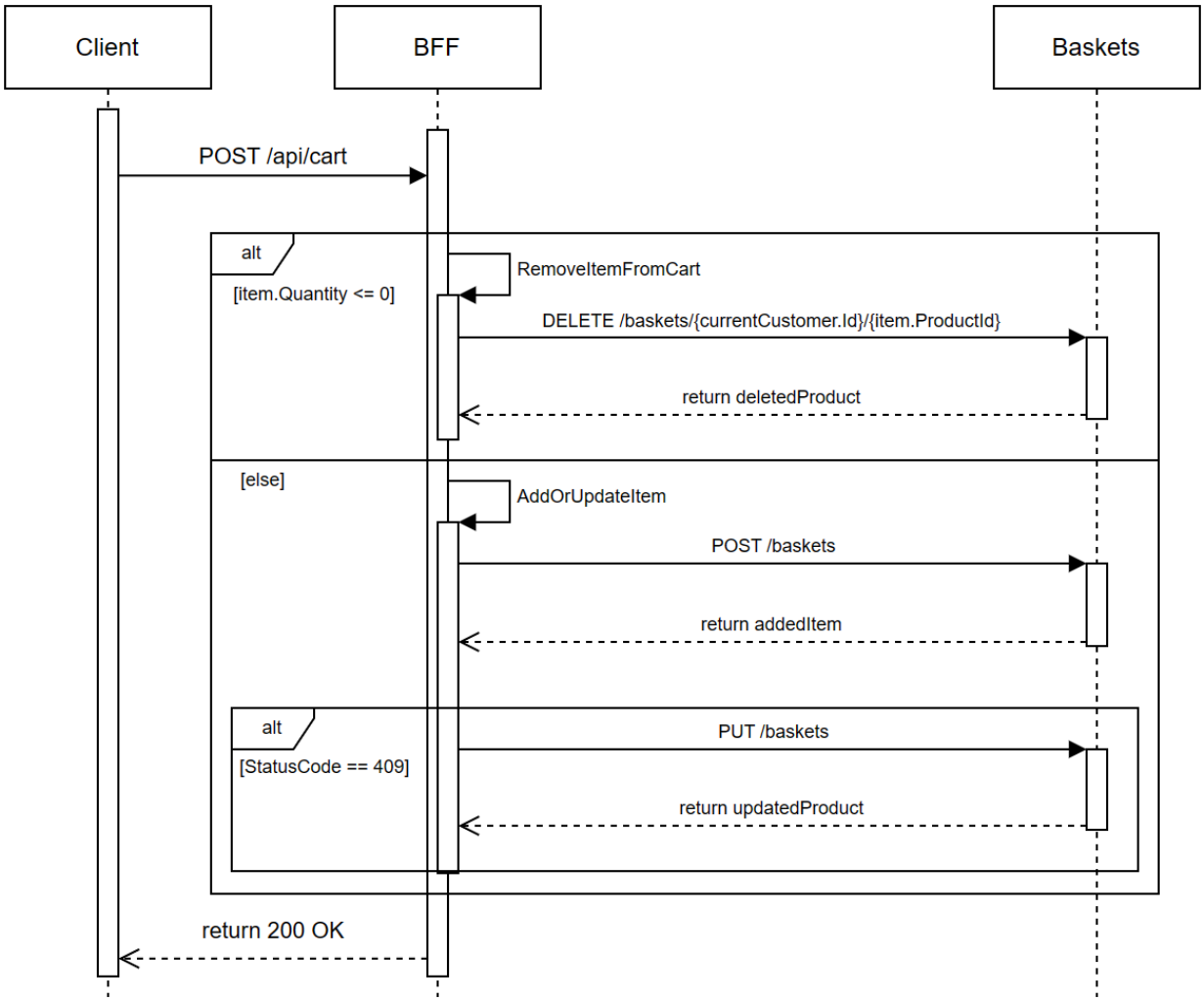


Figure 19.29: A sequence diagram that displays the high-level algorithm of the cart endpoint.

As the diagram depicts, we call the remove endpoint if the quantity is inferior or equal to zero. Otherwise, we try to add the item to the basket. If the endpoint returns a `409 Conflict`, we try to update the quantity. Here's the code:

```
app.MapPost(
    "api/cart",
    async (UpdateCartItem item, IWebClient client, ICurrentCustomerService currentCustomer, Cance
    {
        if (item.Quantity <= 0)
        {
            await RemoveItemFromCart(
                item,
                client,
                currentCustomer,
                cancellationToken
            );
        }
        else
        {
            await AddOrUpdateItem(
                item,
                client,
                currentCustomer,
                cancellationToken
```

```
            );
        }
        return Results.Ok();
    }
);
```

The preceding code follows the same pattern but contains the previously explained logic. We explore the two highlighted methods next, starting with the `RemoveItemFromCart` method:

```
static async Task RemoveItemFromCart(UpdateCartItem item, IWebClient client, ICurrentCustomerSer
{
    try
    {
        var result = await client.Baskets.RemoveProductFromCart(
            new Web.Features.Baskets.RemoveItem.Command(
                currentCustomer.Id,
                item.ProductId
            ),
            cancellationToken
        );
    }
    catch (ValidationApiException ex)
    {
        if (ex.StatusCode != HttpStatusCode.NotFound)
        {
            throw;
        }
    }
}
```

The highlighted code of the preceding block leverages the typed HTTP client and sends a remove item command to the *Baskets* service. If the item is not in the cart, the code ignores the error and continues. Why? Because it does not affect the business logic or the end-user experience. Maybe the customer clicked the remove or update button twice. However, the code propagates to the client any other error.Let's explore the `AddOrUpdateItem` method's code:

```
static async Task AddOrUpdateItem(UpdateCartItem item, IWebClient client, ICurrentCustomerService
{
    try
    {
        // Add the product to the cart
        var result = await client.Baskets.AddProductToCart(
            new Web.Features.Baskets.AddItem.Command(
                currentCustomer.Id,
                item.ProductId,
                item.Quantity
            ),
            cancellationToken
        );
    }
    catch (ValidationApiException ex)
    {
        if (ex.StatusCode != HttpStatusCode.Conflict)
        {
            throw;
        }
        // Update the cart
        var result = await client.Baskets.UpdateProductQuantity(
            new Web.Features.Baskets.UpdateQuantity.Command(
                currentCustomer.Id,
                item.ProductId,
                item.Quantity
            ),
            cancellationToken
        );
    }
}
```

The preceding logic is very similar to the other method. It starts by adding the item to the cart. If it receives a `409 Conflict`, it tries to update its quantity. Otherwise, it lets the exception bubble up the stack to let an exception middleware catch it later to uniformize the error messages.With that code in place, we can send `POST` requests to the `api/cart` endpoint for adding, updating, and removing an item from the cart. The three operations return an empty `200 OK` response.Assuming we have an empty shopping cart, the following request adds *10 Habanero Peppers* (`id=3`) to the shopping cart:

```
POST https://localhost:7254/api/cart
Content-Type: application/json
{
    "productId": 3,
    "quantity": 10
}
```

The following request adds *5 Apples* (`id=2`) to the cart:

```
POST https://localhost:7254/api/cart
Content-Type: application/json
{
    "productId": 2,
    "quantity": 5
}
```

The following request updates the quantity to *20 Habanero Peppers* (`id=3`) :

```
POST https://localhost:7254/api/cart
Content-Type: application/json
{
    "productId": 3,
    "quantity": 20
}
```

The following request removes the *Apples* (`id=2`) from the cart:

```
POST https://localhost:7254/api/cart
Content-Type: application/json
{
    "productId": 2,
    "quantity": 0
}
```

Leaving us with *20 Habanero Peppers* in our shopping cart (`GET https://localhost:7254/api/cart`):

```
[
  {
    "id": 3,
    "name": "Habanero Pepper",
    "unitPrice": 0.99,
    "quantity": 20,
    "totalPrice": 19.80
  }
]
```

The requests of the previous sequence are all in the same format, reaching the same endpoint but doing different things, which makes it very easy for the frontend client to manage.

> If you prefer having the UI to manage the operations individually or want to implement a batch update feature, you can; this is only an example of what you can leverage a BFF for.

We are now done with the BFF service.

Conclusion

In this section, we learned about using the Backend for Frontend (BFF) design pattern to front a micro e-commerce web application. We discussed layering APIs and the advantages and disadvantages of a two-layer design. We autogenerated strongly typed HTTP clients using Refit, managed a shopping cart, and fetched the catalog from the BFF. We learned how to use a BFF to reduce complexity by moving domain logic from the frontend to the backend by implementing multiple Gateway patterns.Here are a few benefits that we explored:

- The BFF pattern can significantly simplify the interaction between frontend and backend systems. It provides a layer of abstraction that can reduce the complexity of using low-level atomic APIs. It separates generic and domain-specific functionalities and promotes cleaner, more modular code.
- A BFF can act as a gateway that routes specific requests to relevant services, reducing the work the frontend has to perform. It can also serve as an aggregation gateway, gathering data from various services into a unified response. This process can simplify frontend development by reducing the complexity of the frontend and the number of separate calls the frontend must make. It can also reduce the payload size transported between the frontend and backend.
- Each BFF is tailored to a specific client, optimizing the frontend interaction.
- A BFF can handle issues in one domain without affecting the low-level APIs or the other applications, thus providing easier maintenance.
- A BFF can implement security logic, such as specific domain-oriented authentication and authorization rules.

Despite these benefits, using a BFF may also increase complexity and introduce potential performance overhead. Using a BFF is no different than any other pattern and must be counter-balanced and adapted to the specific needs of a project.Next, we revisit CQRS on a distributed scale.

Revisiting the CQRS pattern

Command Query Responsibility Segregation (**CQRS**) applies the **Command Query Separation** (**CQS**) principle. Compared to what we saw in *Chapter 14*, *Mediator and CQRS Design Patterns*, we can push CQRS further using microservices or serverless computing. Instead of simply creating a clear separation between commands and queries, we can divide them even more using multiple microservices and data sources.**CQS** is a principle stating that a method should either return data or mutate data, but not both. On the other hand, **CQRS** suggests using one model to read the data and one model to mutate the data.**Serverless computing** is a cloud execution model where the cloud provider manages the servers and allocates the resources on-demand, based on usage and configuration. Serverless resources fall into the platform as a service (PaaS) offering.Let’s come back to our IoT example again. We queried the last known location of a device in the previous examples, but what about the device updating that location? This can mean pushing many updates every minute. To solve this issue, we are going to use CQRS and focus on two operations:

- Updating the device location.
- Reading the last known location of a device.

Simply put, we have a `Read Location` microservice, a `Write Location` microservice, and two databases. Remember that each microservice should own its data. This way, a user can access the last known device location through the read microservice (query model), while a device can punctually send its current position to the write microservice (command model). By doing this, we split the load from reading and writing the data as both occur at different frequencies:

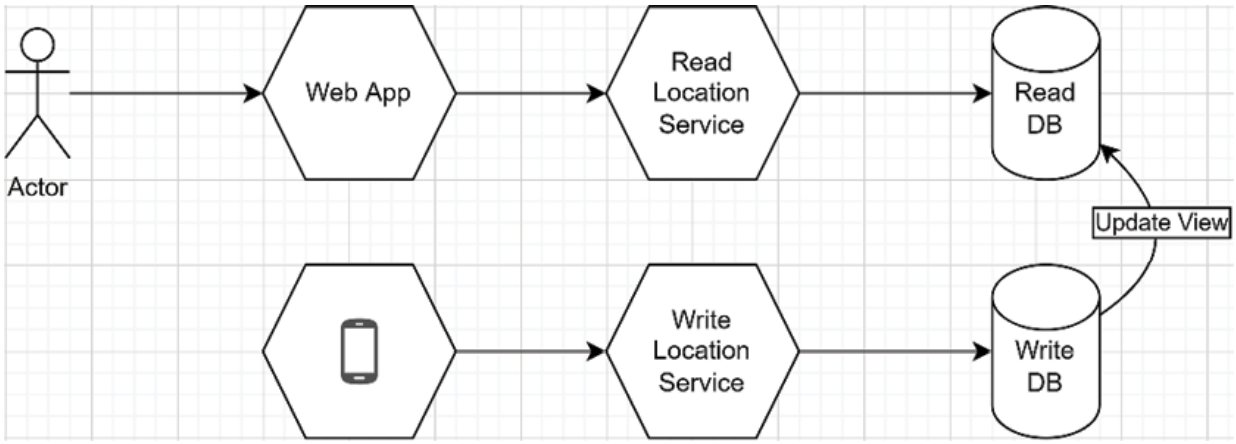


Figure 19.30: Microservices that apply CQRS to divide the reads and writes of a device's location

In the preceding schema that illustrates the concept, the reads are queries, and the writes are commands. How to update the Read DB once a new value is added to the Write DB depends on the technology at play. One essential thing in this type of architecture is that, per the CQRS pattern, a command should not return a value, enabling a "fire and forget" scenario. With that rule in place, consumers don't have to wait for the command to complete before doing something else.

> Fire and forget does not apply to every scenario; sometimes, we need synchronization. Implementing the Saga pattern is one way to solve coordination issues.

Conceptually, we can implement this example by leveraging serverless cloud infrastructures, such as Azure Functions. Let's revisit this example using a high-level conceptual serverless design:

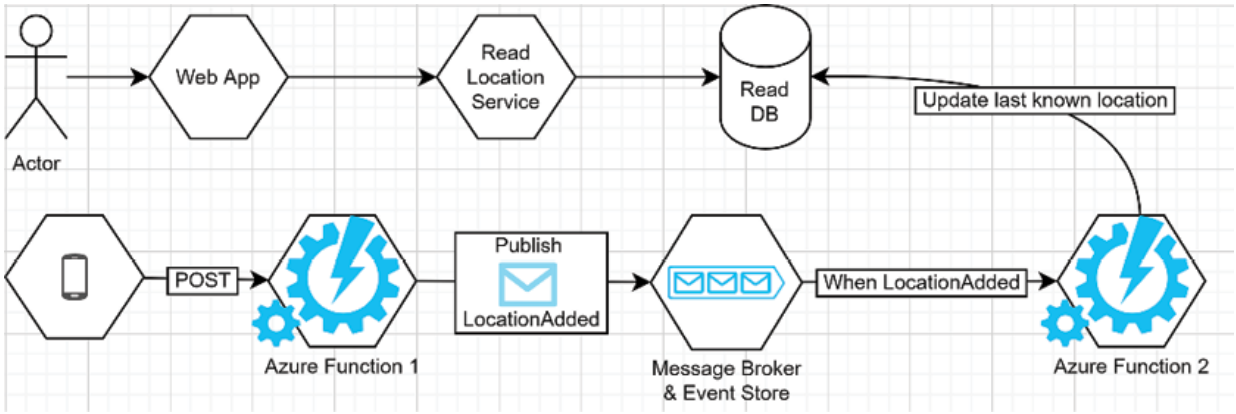


Figure 19.31: Using Azure services to manage a CQRS implementation

The previous diagram illustrates the following:

1. The device sends its location regularly by posting it to *Azure Function 1*.
2. *Azure Function 1* then publishes the `LocationAdded` event to the event broker, which is also an event store (the Write DB).
3. All subscribers to the `LocationAdded` event can now handle the event appropriately, in this case, *Azure Function 2*.
4. *Azure Function 2* updates the device's last known location in the *Read DB*.
5. Any subsequent queries should result in reading the new location.

The message broker is also the event store in the preceding diagram, but we could store events elsewhere, such as in an Azure Storage Table, in a time-series database, or in an Apache Kafka cluster. Azure-wise, the datastore could also be CosmosDB. Moreover, I abstracted this component for multiple reasons, including the fact that there are multiple "as-a-service" offerings to publish events in Azure and multiple ways of using third-party components (both open-source and proprietary).Furthermore, the example demonstrates **eventual consistency** well. All the last known location reads between *steps 1* and *4* get the old value while the system processes the new location updates (commands). If the command processing slows down for some reason, a longer delay could occur before the next read database updates. The commands could also be processed in batches, leading to another kind of delay. No matter what happens with the command processing, the read database is available all that time, whether it serves the latest data or not and whether the write system is overloaded or not. This is the beauty of this type of design, but it is more complex to implement and maintain.

> **Time-series databases** are optimized for temporally querying and storing data, where you always append new records without updating old ones. This kind of NoSQL database can be useful for temporal-intensive usage, like metrics.

Once again, we used the Publish-Subscribe pattern to get another scenario going. Assuming that events are persisted forever, the previous example could also support event sourcing. Furthermore, new services could subscribe to the `LocationAdded` event without impacting the code that has already been deployed. For example, we could create a SignalR microservice that pushes the updates to its clients. It is not CQRS-related, but it flows well with everything that we've explored so far, so here is an updated conceptual diagram:

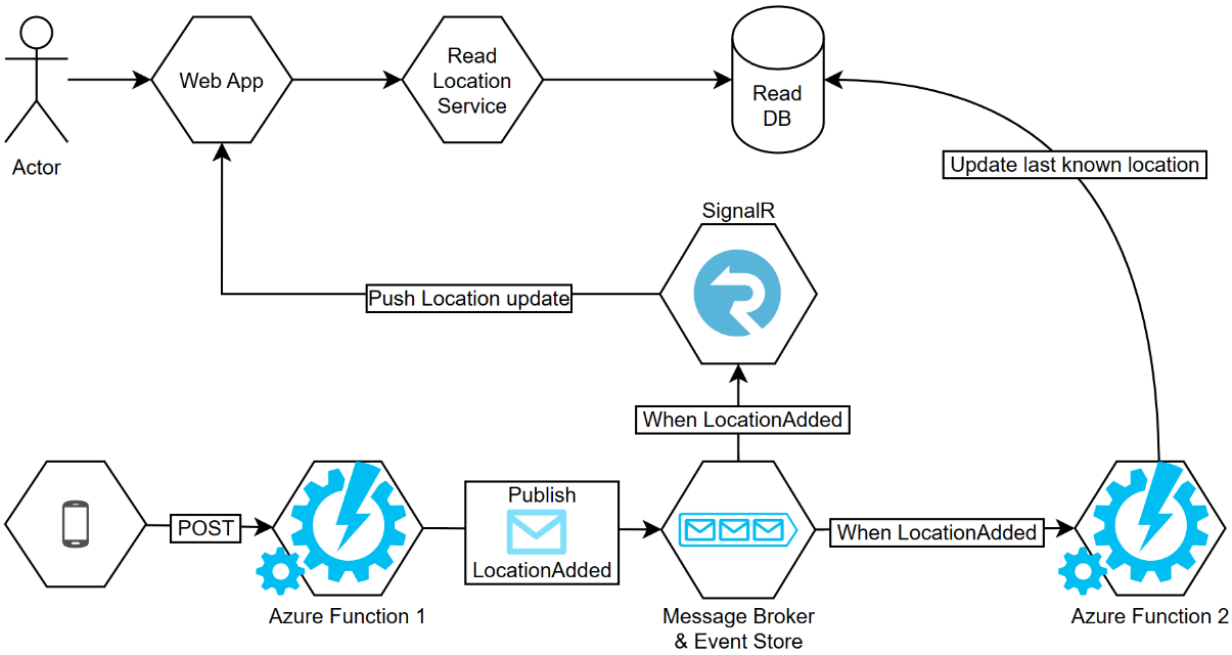


Figure 19.32: Adding a SignalR service as a new subscriber without impacting the other part of the system

The SignalR microservice could be custom code or an Azure SignalR Service (backed by another Azure Function); it doesn't matter. With this design, the Web App could know that a change occurred before the Read DB gets updated.

> With this design, I wanted to illustrate that dropping new services into the mix is easier when using a Pub-Sub model than with point-to-point communication.

As you can see, a microservices system adds more and more small pieces that indirectly interconnect with each other over one or more message brokers. Maintaining, diagnosing, and debugging such systems is harder than with a single application; that's the **operational complexity** we discussed earlier. However, containers can help deploy

and maintain such systems.Starting in ASP.NET Core 3.0, the ASP.NET Core team invested much effort into **distributed tracing**. Distributed tracing is necessary to find failures and bottlenecks related to an event that flows from one program to another (such as microservices). If something bugs out, it is important to trace what the user did to isolate the error, reproduce it, and then fix it. The more independent pieces there are, the harder it can become to make that trace possible. This is outside the scope of this book, but it is something to consider if you plan to leverage microservices.

Advantages and potential risks

This section explores some advantages and risks of separating a data store's read and write operations using the CQRS pattern.

Benefits of the CQRS pattern

- **Scalability:** Given that read and write workloads can be scaled independently, CQRS can lead to much higher scalability in a distributed cloud- or microservices-based applications.
- **Simplified and Optimized Models:** It separates the read model (query responsibility) and write model (command responsibility), which simplifies application development and can optimize performance.
- **Flexibility:** Different models increase the number of choices one can make, increasing flexibility.
- **Enhanced Performance:** CQRS can prevent unnecessary data fetching and allows choosing an optimized database for each job, improving the performance of both read and write operations.
- **Increased Efficiency:** It enables parallel development on complex applications, as teams can work independently on the separate read and write sides of the application.

Potential Risks of using the CQRS pattern

- **Complexity:** CQRS adds complexity to the system. It may not be necessary for simple CRUD apps and could over-complicate the application unnecessarily. Therefore, using CQRS only in complex systems and when the advantages outweigh the cons is advisable.
- **Data Consistency:** It can introduce eventual consistency issues between the read and write sides because the read model's updates are asynchronous, which might not fit every business requirement.
- **Increased Development Effort:** CQRS could mean increased development, testing, and maintenance efforts due to handling two separate models and more pieces.
- **Learning Curve:** The pattern has its own learning curve. Team members unfamiliar with the CQRS pattern will require training and to gain some experience.
- **Synchronization Challenges:** Maintaining synchronization between the read and write models can be challenging, especially in high data volume cases.

Conclusion

CQRS helps divide queries and commands and helps encapsulate and isolate each block of logic independently. Mixing that concept with serverless computing or microservices architecture allows us to scale reads and writes independently. We can also use different databases, empowering us with the tools we need for the transfer rate required by each part of that system (for example, frequent writes and occasional reads or vice versa).Major cloud providers like Azure and AWS provide serverless offerings to help support such scenarios. Each cloud provider’s documentation should help you get started. Meanwhile, for Azure, we have Azure Functions, Event Grid, Event Hubs, Service Bus, Cosmos DB, and more. Azure also offers bindings between the different services that are triggered or react to events for you, removing a part of the complexity yet locking you down with that vendor.Now, let’s see how CQRS can help us follow the **SOLID** principles at the cloud scale:

- **S**: Dividing an application into smaller reads and writes applications (or functions) leans toward encapsulating single responsibilities into different programs.
- **O**: CQRS, mixed with serverless computing or microservices, helps extend the software without needing us to modify the existing code by adding, removing, or replacing applications.
- **L**: N/A

- **I**: CQRS set us up to create multiple small interfaces (or programs) with a clear distinction between commands and queries.
- **D**: N/A

Exploring the Microservice Adapter pattern

The Microservice Adapter pattern allows adding missing features, adapting one system to another, or migrating an existing application to an event-driven architecture model, to name a few possibilities. The Microservice Adapter pattern is similar to the Adapter pattern we cover in *Chapter 9, Structural Patterns*, but applied to a microservices system that uses event-driven architecture instead of creating a class to adapt an object to another signature.In the scenarios we cover in this section, the microservices system represented by the following diagram can be replaced by a standalone application as well; this pattern applies to all sorts of programs, not just microservices, which is why I abstracted away the details:

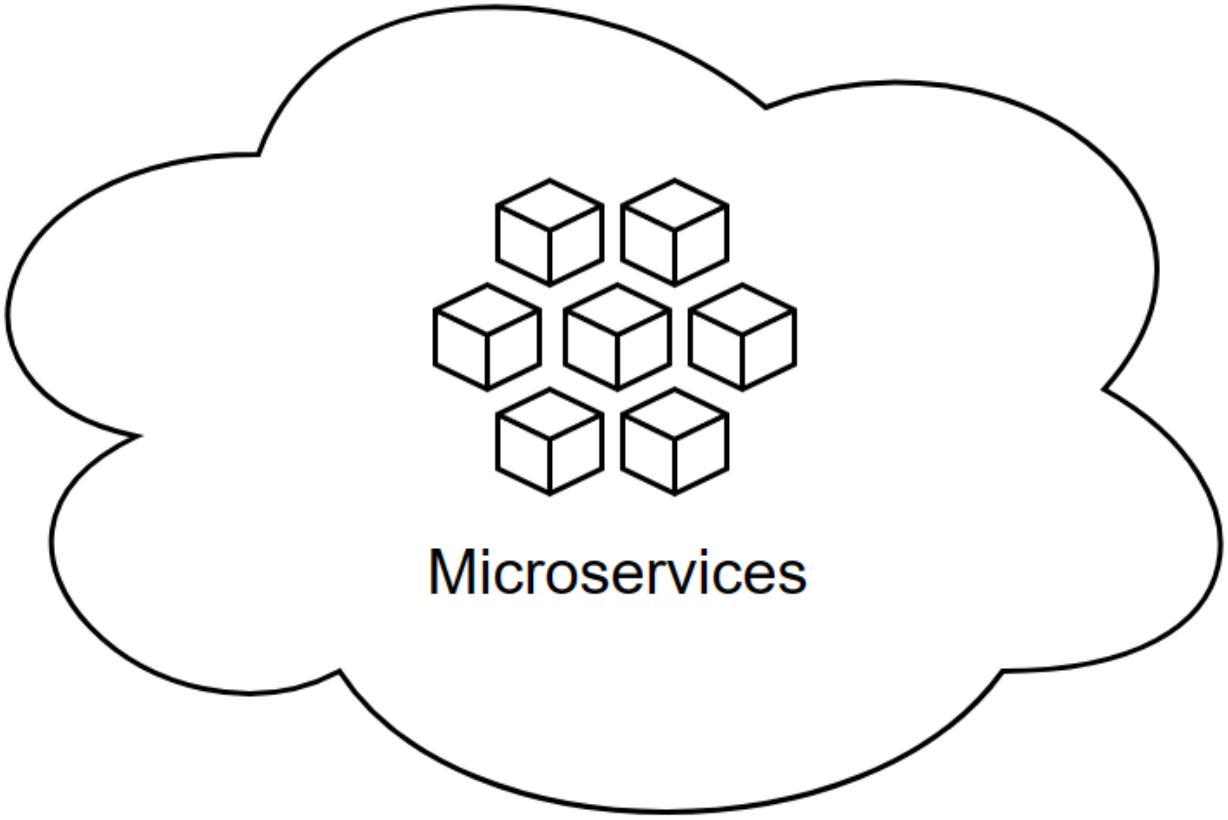


Figure 19.33: Microservice system representation used in the subsequent examples

Here are the examples we are covering next and possible usages of this pattern:

- Adapting an existing system to another.
- Decommissioning a legacy application.
- Adapting an event broker to another.

Let’s start by connecting a standalone system to an event-driven one.

Adapting an existing system to another

In this scenario, we have an existing system of which we don’t control the source code or don’t want to change, and we have a microservices system built around an event-driven architecture model. We don’t have to control the

source code of the microservices system either as long as we have access to the event broker.Here is a diagram that represents this scenario:

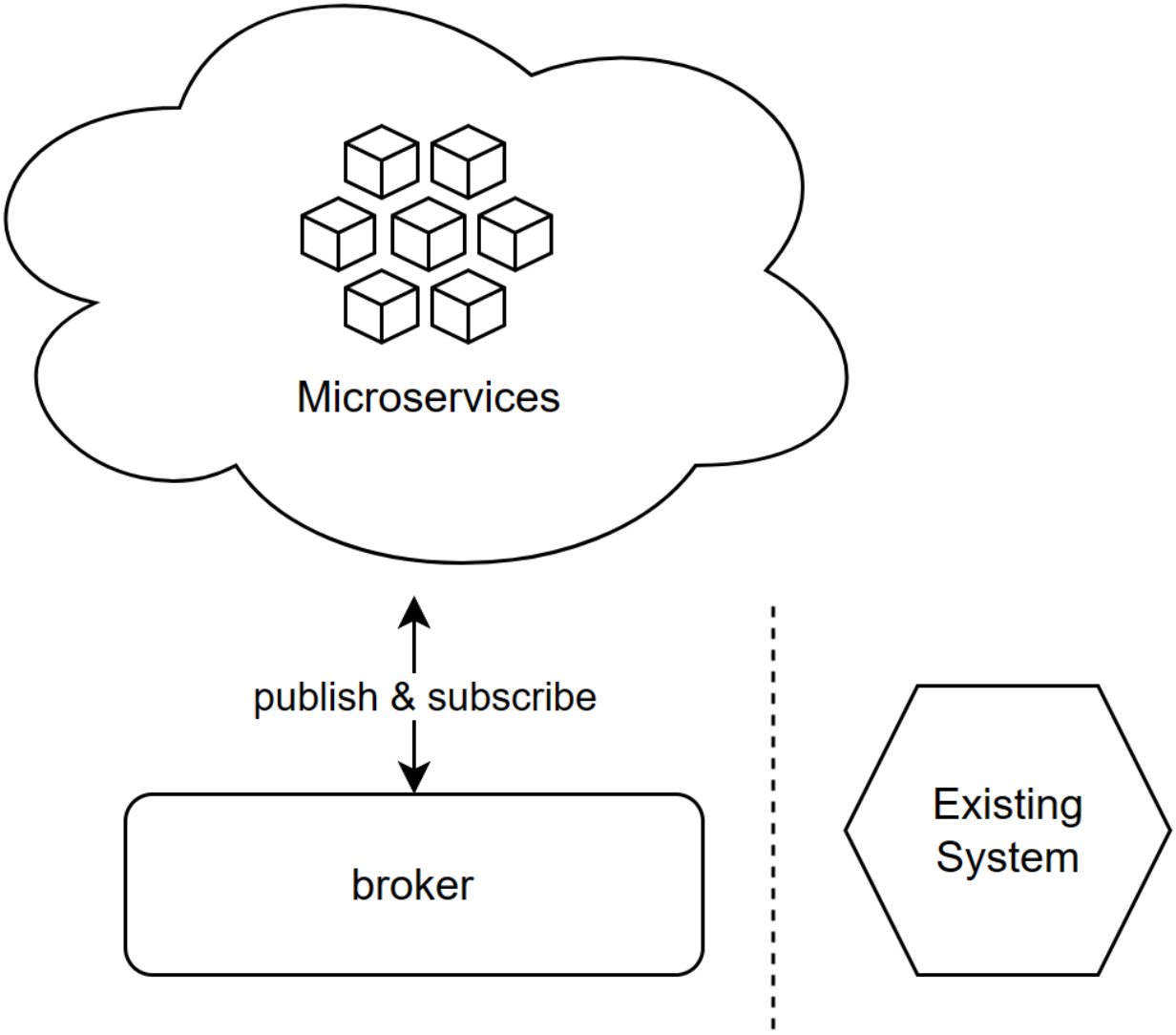


Figure 19.34: A microservices system that interacts with an event broker and an existing system that is disconnected from the microservices

As we can see from the preceding diagram, the existing system is disconnected from the microservices and the broker. To adapt the existing system to the microservices system, we must subscribe or publish certain events. Let's see how to read data from the microservices (subscribe to the broker) and then update that data into the existing system.When we control the existing system's code, we can open the source code, subscribe to one or more topics, and change the behaviors from there. In our case, we don't want to do that or can't, so we can't directly subscribe to topics, as demonstrated by the following diagram:

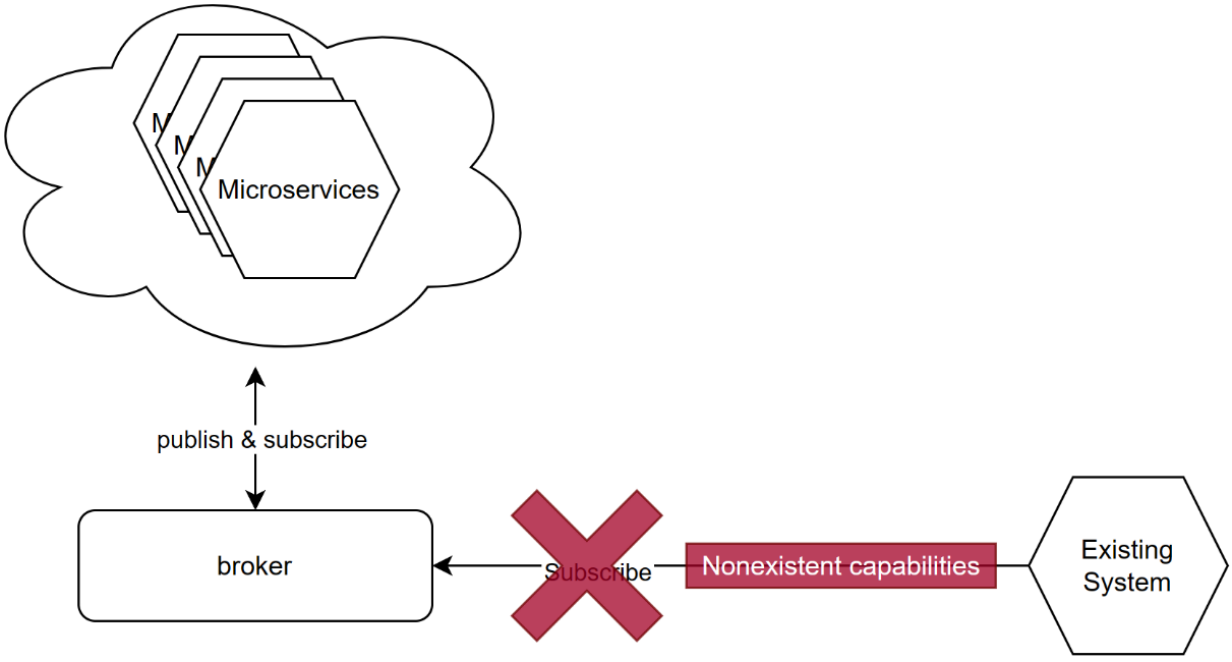


Figure 19.35: Missing capabilities to connect an existing system to an event-driven one

This is where the microservice adapter comes into play and allows us to fill the capability gap of our existing system. To add the missing link, we create a microservice that subscribes to the appropriate events, then apply the changes in the existing system, like this:

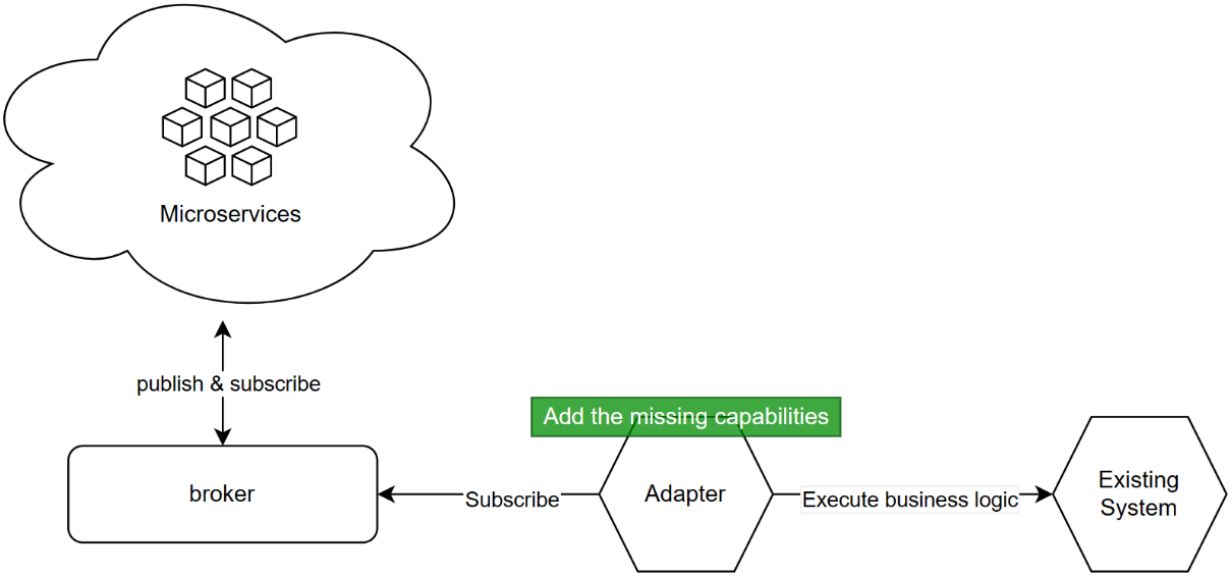


Figure 19.36: An adapter microservice adding missing capabilities to an existing system

As we can see in the preceding diagram, the `Adapter` microservice gets the events (subscribes to one or more topics) and then uses that data from the microservices system to execute some business logic on the existing system.In this design, the new `Adapter` microservice allowed us to add missing capabilities to a system we had no control over with little to no disruption to users' day-to-day activities.The example assumes the existing system had some form of extensibility mechanism like an API. If the system does not, we would have to be more creative to interface with it.For example, the microservices system could be an e-commerce website, and the existing

system could be a legacy inventory management system. The adapter could update the legacy system with new order data.The existing system could also be an old **customer relationship management** (**CRM**) system that you want to update when users of the microservices application execute some actions, like changing their phone number or address.The possibilities are almost endless; you create a link between an event-driven system and an existing system you don't control or don't want to change. In this case, the microservice adapter allows us to follow the **Open-Closed principle** by extending the system without changing the existing pieces. The primary drawback is that we are deploying another microservice that has direct coupling with the existing system, which may be best for temporary solutions. On that same line of thought, next, we replace a legacy application with a new one with limited to no downtime.

Decommissioning a legacy application

In this scenario, we have a legacy application to decommission and a microservices system to which we want to connect some existing capabilities. To achieve this, we can create one or more adapters to migrate all features and dependencies to the new model.Here is a representation of the current state of our system:

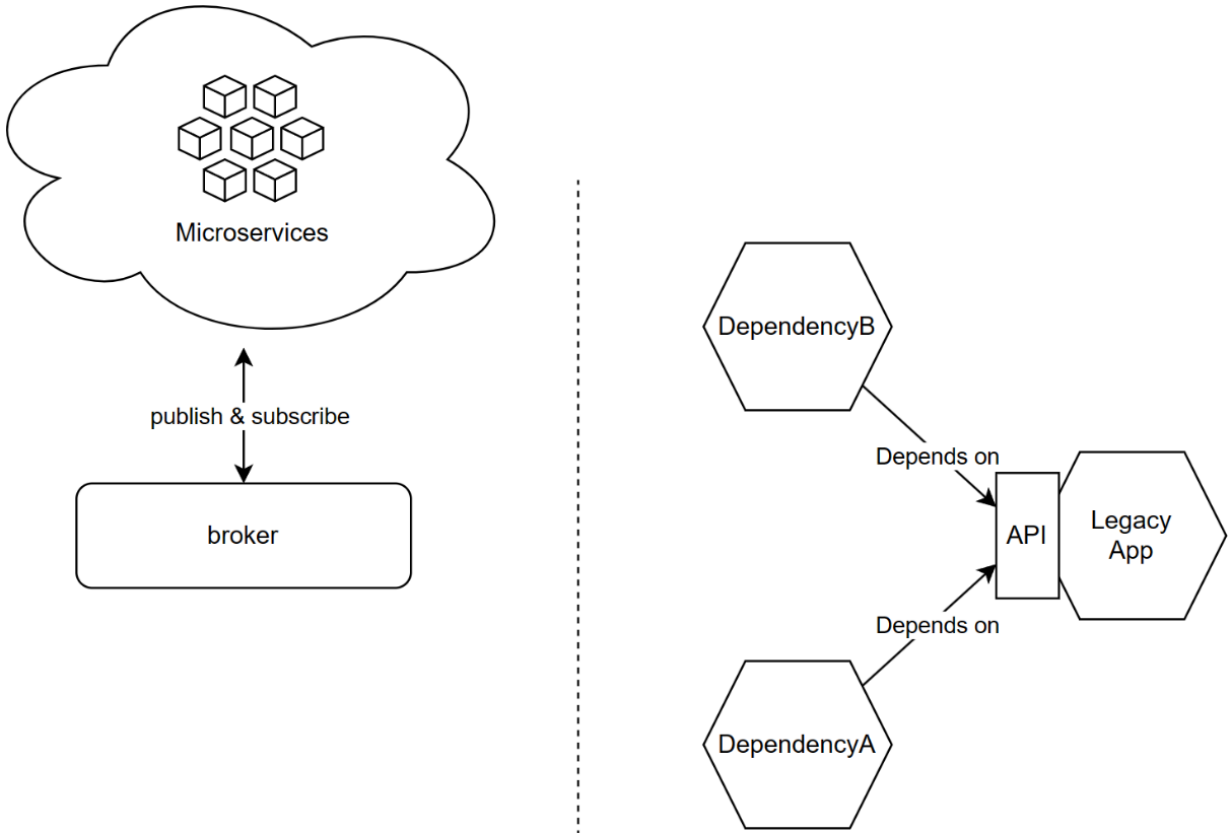


Figure 19.37: The original legacy application and its dependencies

The preceding diagram shows the two distinct systems, including the legacy application we want to decommission. Two other applications, dependency A and B, directly depend on the legacy application. The exact migration flow is strongly dependent on your use case. If you want to keep the dependencies, we want to migrate them first. To do that, we can create an event-driven `Adapter` microservice that breaks the tight coupling between the dependencies and the legacy application like this:

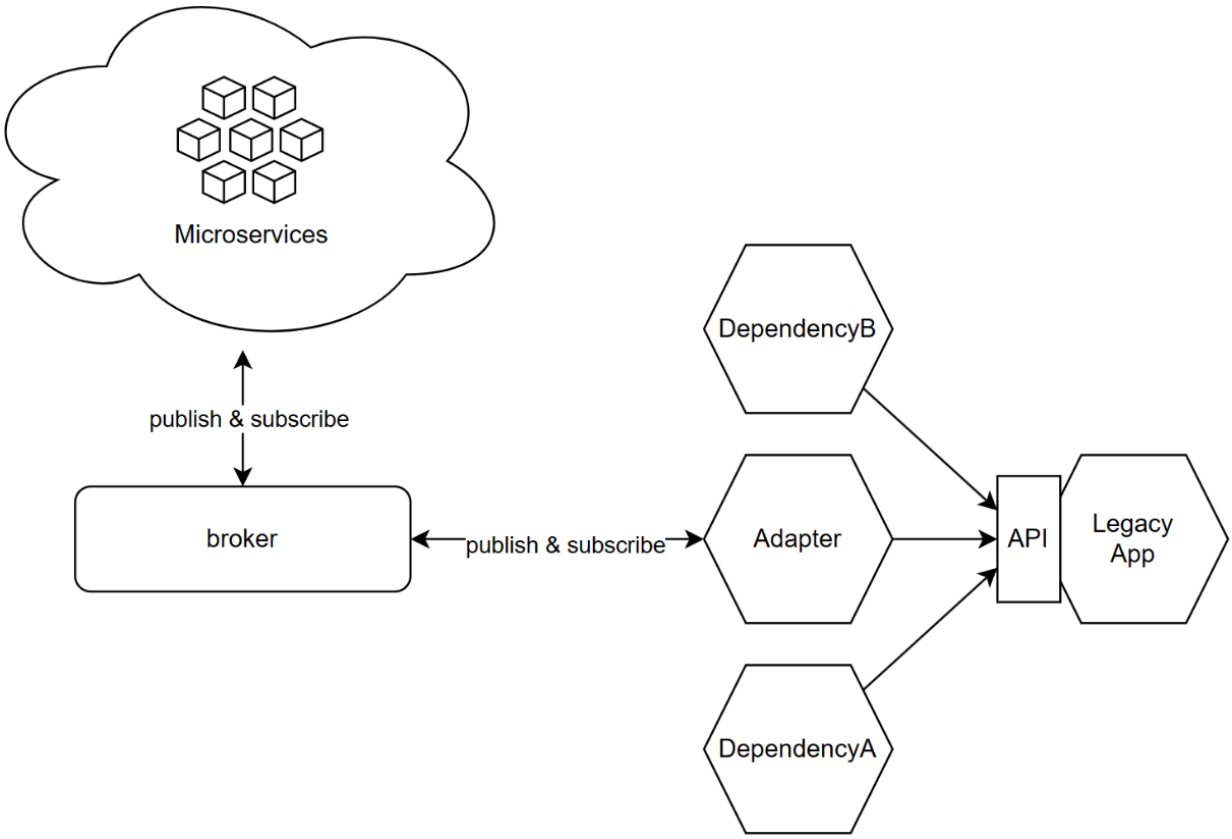


Figure 19.38: Adding a microservice adapter that implements the event-driven flow required to break tight coupling between the dependencies and the legacy application

The preceding diagram shows an `Adapter` microservice and the rest of a microservices system that communicates using an event broker. As we explored in the previous example, the adapter was placed there to connect the legacy application to the microservices. Our scenario focuses on removing the legacy application and migrating its two dependencies. Here, we carved out the required capabilities using the adapter, allowing us to migrate the dependencies to an event-driven model and break tight coupling with the legacy application. Such migration could be done in multiple steps, migrating each dependency one by one, and we could even create one adapter per dependency. For the sake of simplicity, I chose to draw only one adapter. You may want to revisit this choice if your dependencies are large or complex.Once we are done migrating the dependencies, our systems look like the following:

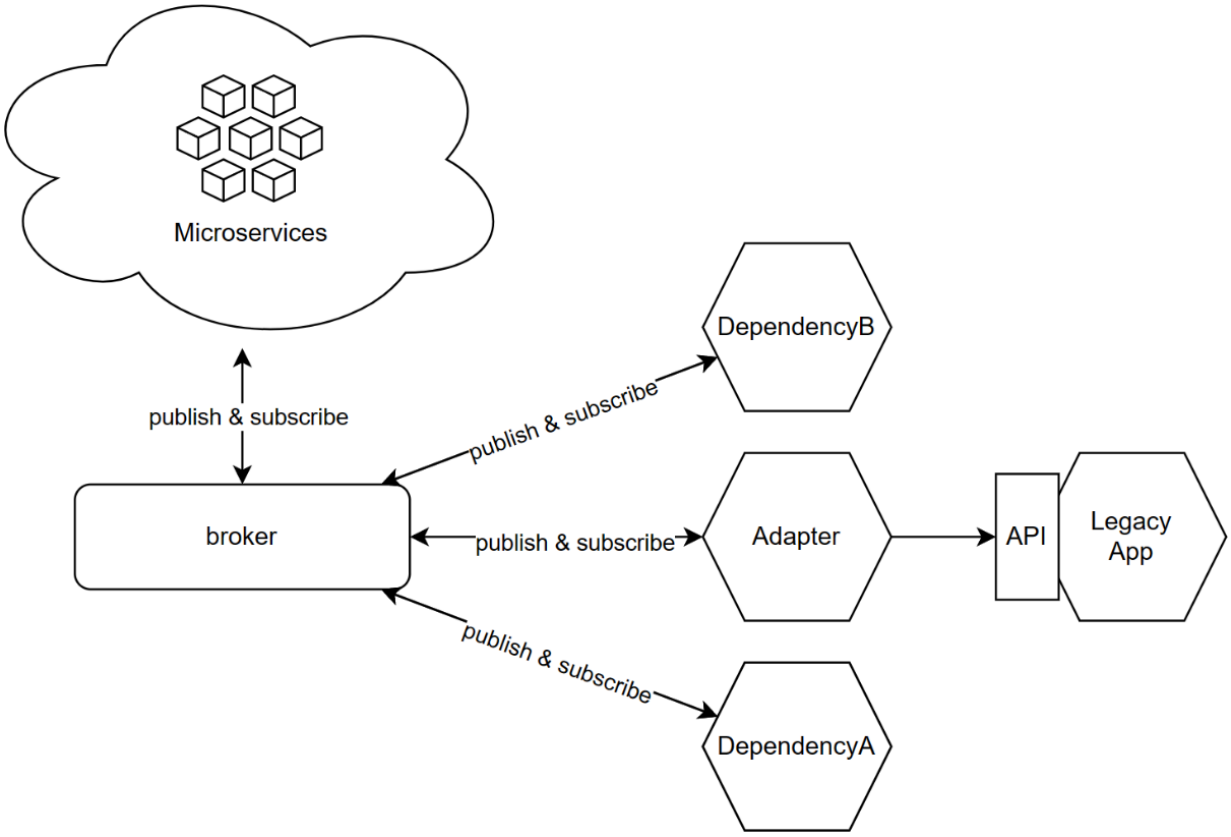


Figure 19.39: The dependencies are now using an event-driven architecture, and the adapter microservice is bridging the gap between the events and the legacy system

In the preceding diagram, the Adapter microservice executes the operations against the legacy application API that the two dependencies were doing before. The dependencies are now publishing events instead of using the API. For example, when an operation happens in `DependencyB`, it publishes an event to the broker. The Adapter microservice picks up that event and executes the original operation against the API. Doing this creates more complexity and is a temporary state.With this new architecture in place, we can start migrating existing features away from the legacy application into the new application without impacting the dependencies; we broke tight coupling.

> From this point forward, we are applying the **Strangler Fig** pattern to migrate the legacy system piece by piece to our new architecture. For the sake of simplicity, think of the Strangler Fig pattern as migrating features from one application to another, one by one. In this case, we replaced one application with another, but we could also use the same patterns to split an application into multiple smaller applications (like microservices).
>
> > I left a few links in the further reading section in case migrating legacy systems is something you do or simply if you want to know more about that pattern.

The following diagram is a visual representation that adds the modern application we are building to replace the legacy application. That new modern application could also be a purchased product you are putting in place instead; the concepts we are exploring apply to both use cases, but the exact steps are directly related to the technology at play.

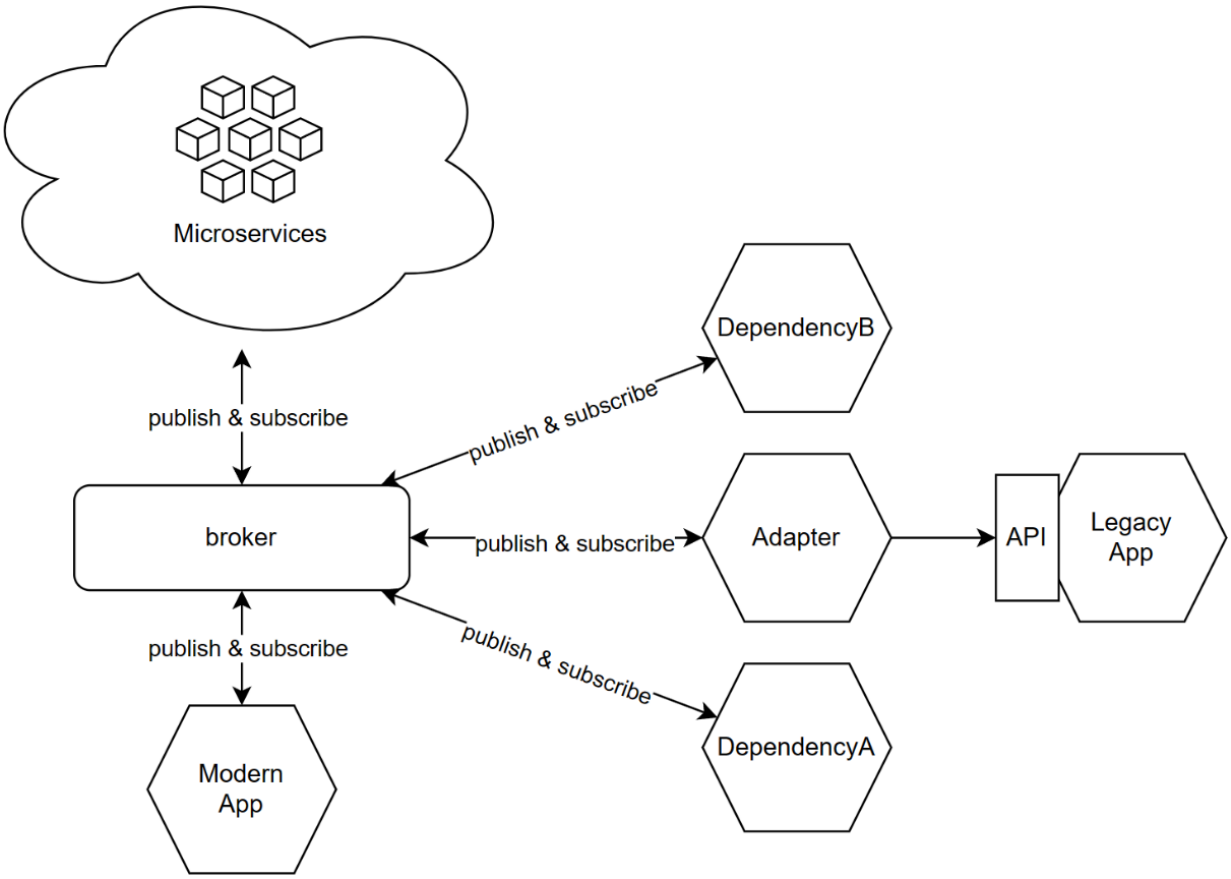


Figure 19.40: The modern application to replace the legacy application is starting to emerge by migrating capabilities to that new application

In the preceding diagram, we see the new modern application has appeared. Each time we deploy a new feature to the new application, we can remove it from the adapter, leading to a graceful transition between the two models. At the same time, we are keeping the legacy application in place to continue to provide the capabilities that are not yet migrated.Once all the features we want to keep are migrated, we can remove the adapter and decommission the legacy application, leading to the following system:

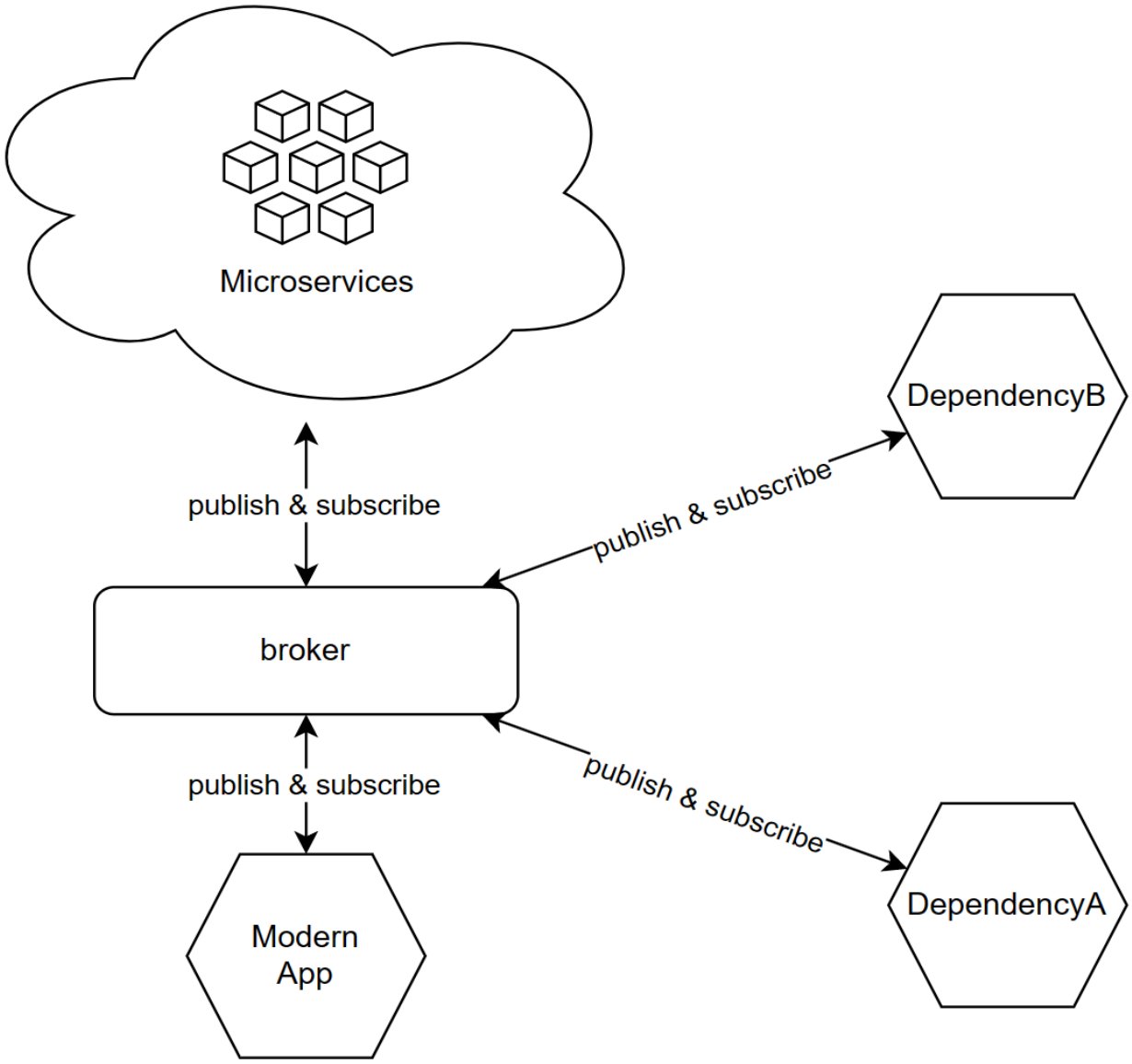


Figure 19.41: The new system topology after the retirement of the legacy application, showing the new modern application and its two loosely coupled dependencies

The preceding diagram shows the new system topology encompassing a new modern application and the two original dependencies that are now loosely coupled through event-driven architecture. Of course, the bigger the migration, the more complex it will be and the longer it will take, but the Adapter Microservice pattern is one way to help do a partial or complete migration from one system to another.Like the preceding example, the main advantage is adding or removing capabilities without impacting the other systems, which allows us to migrate and break the tight coupling between the different dependencies. The downside is the added complexity of this temporary solution. Moreover, during the migration step, you will most likely need to deploy both the modern application and the adapter in the correct sequence to ensure both systems are not handling the same events twice, leading to duplicate changes. For example, updating the phone number to the same value twice should be all right because it leads to the same final data set. However, creating two records instead of one should be more important to mitigate as it may lead to integrity errors in the data set. For example, creating an online order twice instead of once could create customer dissatisfaction or internal issues.And voilà, we decommissioned a system using the Microservice Adapter pattern without breaking its dependencies. Next, we look at an **Internet of Things** (**IoT**) example.

Adapting an event broker to another

In this scenario, we are adapting an event broker to another. In the following diagram, we look at two use cases: one that translates events from broker B to broker A (left) and the other that translates events from broker A to broker B (right). Afterwards, we explore a more concrete example:

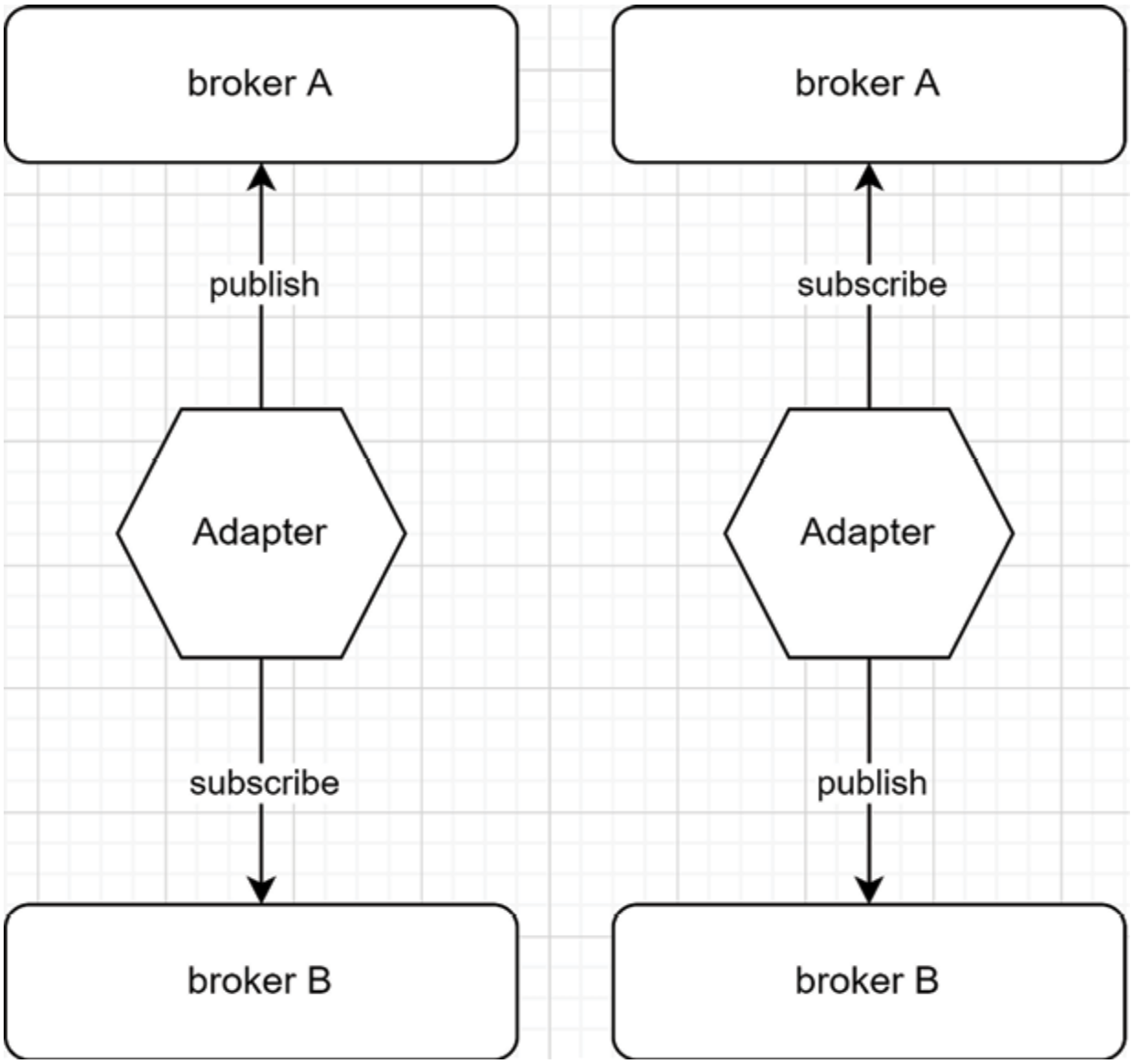


Figure 19.42: An adapter microservice that translates events from broker B to broker A (left) and from broker A to broker B (right)

We can see the two possible flows in the preceding diagram. The first flow, on the left, allows the adapter to read events from broker B and publish them to broker A. The second flow, on the right, enables the adapter to read events from broker A and publish them to broker B. Those flows allow us to translate or copy events from one broker to another by leveraging the Microservice Adapter pattern.

> In *Figure 16.35*, there is one adapter per flow. I did that to make the two flows as independent as possible, but the adapters could be a single microservice.

This pattern can be very useful for an IoT system where your microservices leverage Apache Kafka internally for its full-featured suite of event-streaming capabilities but use MQTT to communicate with the low-powered IoT devices that connect to the system. An adapter can solve this problem by translating the messages from one protocol to the other. Here is a diagram that represents the complete flows, including a device and the microservices:

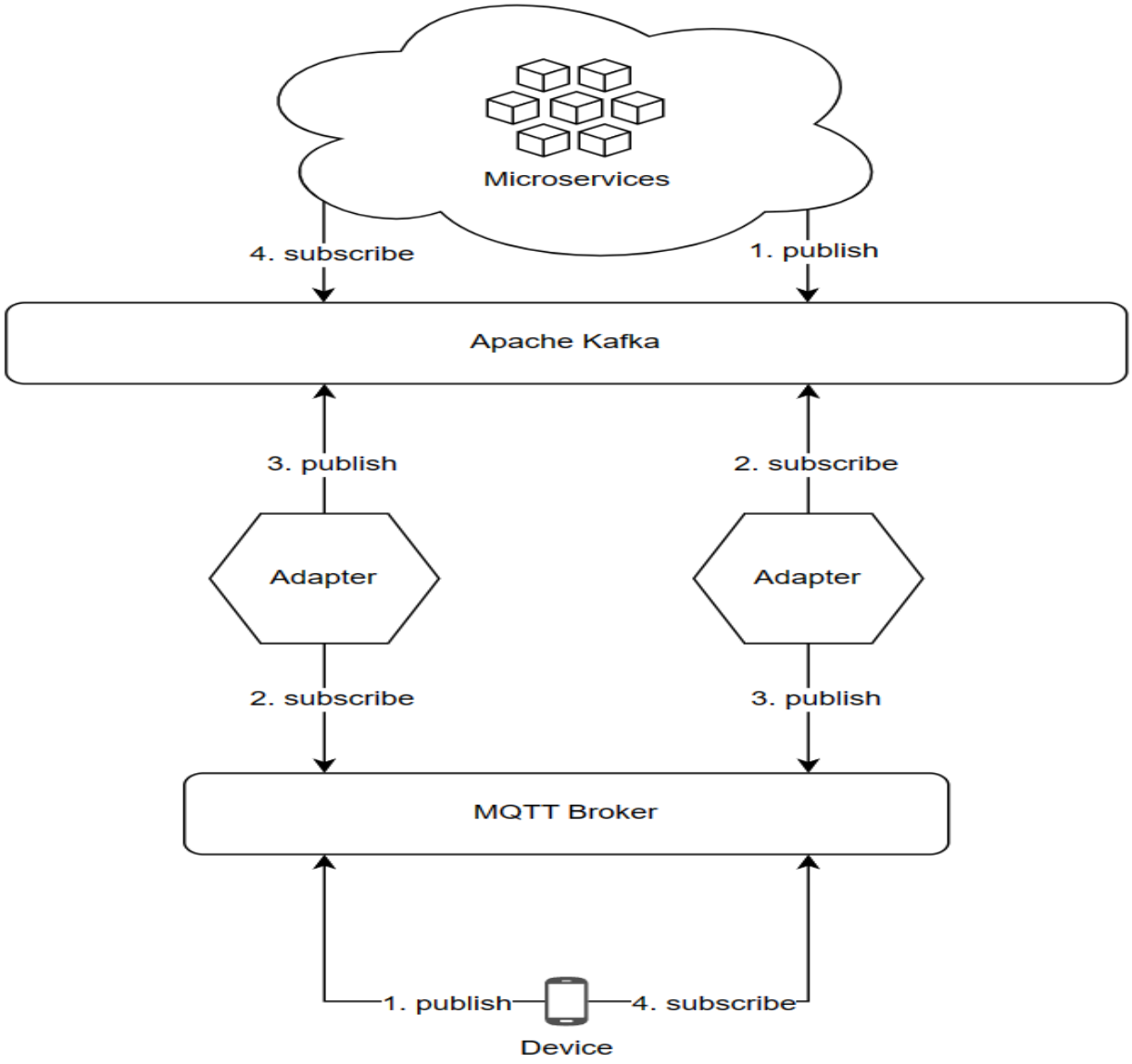


Figure 19.43: Complete protocol adapter flows, including a device and microservices

Before we explore what the events could be, let's explore both flows step by step. The left flow allows getting events inside the system from the devices through the following sequence:

1. A device publishes an event to the MQTT broker.
2. The adapter reads that event.
3. The adapter publishes a similar or different event to the Kafka broker.
4. Zero or more microservices subscribed to the event act on it.

On the other hand, the right flow allows getting events out of the system to the devices through the following sequence:

1. A microservice publishes an event to the Kafka broker.
2. The adapter reads the event.
3. The adapter publishes a similar or different event to the MQTT broker.
4. Zero or more devices subscribed to the event act on it.

You don’t have to implement both flows; the adapter could be bidirectional (supporting both flows), we could have two unidirectional adapters that support one of the flows, or we could allow the communication to flow only one way (in or out but not both). The choice relates to your specific use cases.Concrete examples of sending a message from a device to a microservice (left flow) could be sending its GPS position, a status update (the light is now on), or a message indicating a sensor failure.Concrete examples of sending a message to a device (right flow) could be to remotely control a speaker’s volume, flip a light on, or send a confirmation that a message has been acknowledged.In this case, the adapter is not a temporary solution but a permanent capability. We could leverage such adapters to create additional capabilities with minimal impact on the rest of the system. The primary downside is deploying one or more other microservices, but your system and processes are probably robust enough to handle that added complexity when leveraging such capabilities.This third scenario that leverages the Microservice Adapter is our last. Hopefully, I sparked your imagination enough to leverage this simple yet powerful design pattern.

Conclusion

We explored the Microservice Adapter pattern that allows us to connect two elements of a system by adapting one to the other. We explored how to push information from an event broker into an existing system that does not support such capabilities. We also explored how to leverage an adapter to break tight coupling, migrate features into a newer system, and decommission a legacy application seamlessly. We finally connected two event brokers through an adapter microservice, allowing a low-powered IoT device to communicate with a microservices system without draining their battery and without the complexity it would incur to use a more complex communication protocol.This pattern is very powerful and can be implemented in many ways, but it all depends on the exact use cases. You can write an adapter using a serverless offering like an Azure function, no-code/low-code offerings like Power Automate, or C#. Of course, these are just a few examples. The key to designing the correct system is to nail down the problem statement because once you know what you are trying to fix, the solution becomes clearer.Now, let’s see how the Microservice Adapter pattern can help us follow the **SOLID** principles at cloud-scale:

- **S**: The microservice adapter helps manage long- or short-term responsibilities. For example, adding an adapter that translates between two protocols or creating a temporary adapter to decommission a legacy system.
- **O**: You can leverage microservice adapters to dynamically add or remove features without impacting or with limited impact on the rest of the system. For example, in the IoT scenario, we could add support for a new protocol like AMQP without changing the rest of the system.
- **L**: N/A
- **I**: Adding smaller adapters can make changes easier and less risky than updating large legacy applications. As we saw in the legacy system decommissioning scenario, we could also leverage temporary adapters to split large applications into smaller pieces.
- **D**: A microservice adapter inverts the dependency flow between the system it adapts. For example, in the legacy system decommissioning scenario, the adapter reversed the flow from the two dependencies to the legacy system by leveraging an event broker.

Summary

The microservices architecture is different from everything we’ve covered in this book and how we build monoliths. Instead of one big application, we split it into multiple smaller ones called microservices. Microservices must be independent of one another; otherwise, we will face the same problems associated with tightly coupled classes, but at the cloud scale.We can leverage the Publish-Subscribe design pattern to loosely couple microservices while keeping them connected through events. Message brokers are programs that dispatch those messages. We can use event sourcing to recreate the application’s state at any point in time, including when

spawning new containers. We can use application gateways to shield clients from the microservices cluster's complexity and publicly expose only a subset of services.We also looked at how we can build upon the CQRS design pattern to decouple reads and writes of the same entities, allowing us to scale queries and commands independently. We also looked at using serverless resources to create that kind of system.Finally, we explored the Microservice Adapter pattern that allowed us to adapt two systems together, decommission a legacy application, and connect two event brokers. This pattern is simple but powerful at inverting the dependency flow between two dependencies in a loosely coupled manner. The use of the pattern can be temporary, as we saw in the legacy application decommissioning scenario, or permanent, as we saw in the IoT scenario.On the other hand, microservices come at a cost and are not intended to replace all that exists. Building a monolith is still a good idea for many projects. Starting with a monolith and migrating it to microservices when scaling is another solution. This allows us to develop the application faster (monolith). It is also easier to add new features to a monolith than it can be to add them to a microservice application. Most of the time, mistakes cost less in a monolith than in a microservices application. You can also plan your future migration toward microservices, which leads to the best of both worlds while keeping operational complexity low. For example, we could leverage the Publish-Subscribe pattern through MediatR notifications in your monolith and migrate the events dispatching responsibility to a message broker later when migrating your system to microservices architecture (if the need ever arises). We are exploring ways to organize our monolith in *Chapter 20*, *Modular Monolith*.I don't want you to discard the microservices architecture, but I want to ensure you weigh up the pros and cons of such a system before blindly jumping in. Your team's skill level and ability to learn new technologies may also impact the cost of jumping into the microservices boat.**DevOps** (development [Dev] and IT operations [Ops]) or **DevSecOps** (adding security [Sec] to the DevOps mix), which we do not cover in the book, is essential when building microservices. It brings deployment automation, automated quality checks, auto-composition, and more. Your microservices cluster will be very hard to deploy and maintain without that.Microservices are great when you need scaling, want to go serverless, or split responsibilities between multiple teams, but keep the operational costs in mind.In the next chapter, we combine the microservices and monolith worlds.

Questions

Let's take a look at a few practice questions:

1. What is the most significant difference between a **message queue** and a **pub-sub** model?
2. What is **event sourcing**?
3. Can an **application gateway** be both a **routing gateway** and an **aggregation gateway**?
4. Is it true that real CQRS requires a serverless cloud infrastructure?
5. What is a significant advantage of using the BFF design pattern?

Further reading

Here are a few links that will help you build on what you learned in this chapter:

- Event Sourcing pattern by Martin Fowler: https://adpg.link/oY5H
- Event Sourcing pattern by Microsoft: https://adpg.link/ofG2
- Publisher-Subscriber pattern by Microsoft: https://adpg.link/amcZ
- Event-driven architecture by Microsoft: https://adpg.link/rnck
- Microservices architecture and patterns on microservices.io: https://adpg.link/41vP
- Microservices architecture and patterns by Martin Fowler: https://adpg.link/Mw97
- Microservices architecture and patterns by Microsoft: https://adpg.link/s2Uq
- RFC 6902 (JSON Patch): https://adpg.link/bGGn
- JSON Patch in ASP.NET Core web API: https://adpg.link/u6dw

Strangler Fig Application pattern:

- Martin Fowler: https://adpg.link/Zi9G
- Microsoft: https://adpg.link/erg2

Answers

1. The message queue gets a message and has a single subscriber dequeue it. If nothing dequeues a message, it stays in the queue indefinitely (FIFO model). The Pub-Sub model gets a message and sends it to zero or more subscribers.
2. Event sourcing is the process of chronologically accumulating events that happened in a system instead of persisting in the current state of an entity. It allows you to recreate the entity's state by replaying those events.
3. Yes, you can mix Gateway patterns (or sub-patterns).
4. No, you can deploy micro-applications (microservices) on-premises if you want to.
5. It separates generic functionalities from app-specific ones, promoting cleaner code and modularization. It also helps simplify the frontend.

20 Modular Monolith

Before you begin: Join our book community on Discord

Give your feedback straight to the author himself and chat to other early readers on our Discord server (find the "architecting-aspnet-core-apps-3e" channel under EARLY ACCESS SUBSCRIPTION).

https://packt.link/EarlyAccess



In the ever-evolving software development landscape, choosing the right architecture is like laying the foundation for a building. The architecture dictates how the software is structured, impacting its scalability, maintainability, and overall success. Traditional monolithic architecture and microservices have long been the dominant paradigms, each with advantages and challenges.However, a new architectural style has been gaining traction—Modular Monoliths. This approach aims to offer the best of both worlds by combining the simplicity of monoliths with the flexibility of microservices. It serves as a middle ground that addresses some of the complexities associated with microservices, making it particularly appealing for small to medium-sized projects or teams transitioning from a traditional monolithic architecture.

> I wrote an article about this in 2017 entitled *Microservices Aggregation*. I recently read the name *Modular Monolith* and loved it better. This architectural style is gaining traction, yet it is not entirely new. The Modular Monolith style is a way to modularize and organize an application, lowering our chances of creating a big ball of mud.

This chapter aims to provide a comprehensive understanding of Modular Monoliths. We delve into its core principles, advantages, and key components and explore when and how to implement this architecture. We build upon our nano e-commerce application to get hands-on insights into Modular Monoliths' practical applications. Additionally, we discuss how they compare with other architectural styles to help you make informed decisions for your next projects.By the end of this chapter, you should have a solid grasp of Modular Monoliths, why they might be the right choice for your project, and how to implement them.In this chapter, we cover the following topics:

- What is a Modular Monolith?
- Advantages of Modular Monoliths
- Key Components of a Modular Monolith
- Implementing a Modular Monolith
- Project—Modular Monolith
- Transitioning to Microservices
- Challenges and Pitfalls

Let’s start by exploring what is a Modular Monolith.

What is a Modular Monolith?

A Modular Monolith is an architectural style that aims to combine the best aspects of traditional monolithic architectures and microservices. It organizes the software application into well-defined, loosely coupled modules. Each is responsible for a specific business capability. However, unlike microservices, all these modules are deployed as a single unit like a monolith.The core principles of a Modular Monolith are:

- Treat each module as a microservice.
- Deploy the application as a single unit.

Here are the fundamental principles of a successful microservice as studied in *Chapter 19, Introduction to Microservices Architecture*:

- Each microservice should be a cohesive unit of business.
- Each microservice should own its data.
- Each microservice should be independent of the others.

In a nutshell, we get the best of both worlds. Yet, understanding how a Modular Monolith compares with other architectural styles is crucial for making informed decisions.

What are traditional Monoliths?

In a traditional monolithic architecture, we build the application as a single, indivisible unit. This leads to the functionalities being tightly coupled together, making it difficult to make changes or scale specific features. This approach makes creating a big ball of mud easier, particularly when the team invests little effort in domain modeling and analysis before and during development.On top of that, while this approach is simple and straightforward, it lacks the flexibility and scalability of more modern architectures.

> A monolith does not have to be indivisible, yet most end up this way because it is easy to create tight coupling in a single application.

Let’s look at microservices next.

What are microservices?

Microservices architecture, on the other hand, takes modularity to the extreme. Each service is a completely independent unit, running in its own environment. Microservices architecture allows for high scalability and flexibility but comes at the cost of increased operational complexity.

> *Chapter 19, Introduction to Microservices Architecture*, covers this topic more in-depth.

The following sections delve deeper into this emerging architectural style, starting with its advantages.

Advantages of Modular Monoliths

One of the best things about Modular Monoliths is that they are easy to manage. You don't have to worry about many moving parts like with microservices. Everything is in one place but still separated into modules. This makes it easier for us to keep track of things and to work with them.With Modular Monoliths, each module is like its own small project. We can test, fix, or improve a module without affecting the others. This is great because it lets us focus on one thing at a time, which improves productivity by lowering the cognitive load required to work on that feature.When it's time to release the software, we only have one application to deploy. Even though it has many modules, we treat them like one deployable unit. This makes managing deployments much more straightforward as we don’t have to juggle multiple services like we would with microservices.Modular Monoliths can save us money because we don't need as many resources since we deploy a single monolith. Because of that, we don't need a team to manage and run complicated infrastructure. We don't need to worry about distributed tracing between our services, which reduces the upfront monitoring cost. This deployment style is highly beneficial when starting a project with a small team or if the team is not proficient with microservices architecture.

> Modular Monoliths can still be valuable even if you are part of a large team, part of a larger organization, or have experience with microservices architecture. It is not one or the other kind of scenario.

As we just explored, Modular Monoliths bring many advantages:

- The reduced operational complexity makes deploying a complex application as a single unit easier than microservices.
- They improve our development and testing experience because of their simplicity, improving our efficiency.
- They are easier to manage than most traditional monoliths because modules are well segregated.
- They are more cost-effective than microservices.

The following section looks at what makes up a Modular Monolith.

Key Components of a Modular Monolith

The first thing to know about Modular Monoliths is that they are composed of different parts called **modules**. Each module is like a mini-app that performs a specific job. This job is related to a particular business capability. A business capability comes from the business domain and aims at a cohesive group of scenarios. Such a group is called a bounded context in DDD. Think of each module as a microservice or a well-defined chunk of the domain.That separation means everything we need to complete a specific task is in one module, which makes it easier for us to understand and work on the software because we don't have to jump from one place to another to get things done. So, if one module needs an update or has a problem, we can fix it without touching the others. This segregation is also perfect for testing since we can test each module in isolation to ensure it works well. For example, one module might handle a shopping cart while another takes care of the shipping, but we piece all modules together in a final aggregated application.

> We can create the modular monolith as a single project. However, in a .NET-specific environment, when each module comprises one or more assemblies, it is harder to create unwanted coupling between modules. We explore this in the project we are about to build.

The **module aggregator**—the monolith—is responsible for loading and serving the modules as if they were just one application.Even though each module is independent, they still need to talk to each other sometimes. For example, the product catalog module might need to tell the shopping cart module that an employee has added a new product to the catalog. There are many ways to make this communication happen. One of the best ways to keep a low coupling level between the modules is to leverage event-driven architecture. On top of loose coupling, this opens doors like scaling the Modular Monolith by migrating one or more modules to a microservices architecture; more on that later.Here's a diagram that represents these relationships:

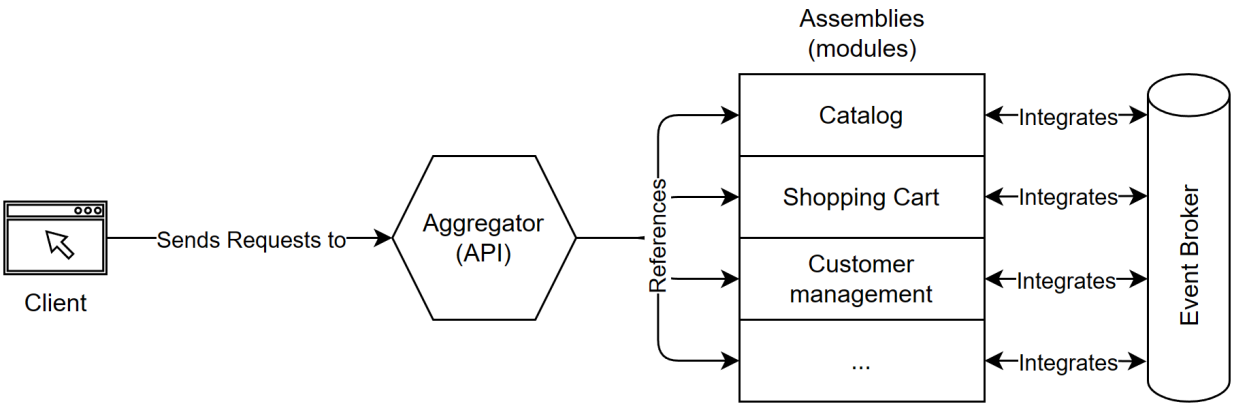


Figure 20.1: Diagram representing the relationships between the module aggregator, the modules, and an event broker.

Now that we have explored the key components of a Modular Monolith, the next section discusses planning and implementing one.

Implementing a Modular Monolith

Planning is essential before building a Modular Monolith. We must consider what each module does and how modules work together. A good plan helps us avoid problems later on.Choosing the right tools to create a lean stack is also essential. The good news is that we don't need to define a large shared stack since each module is independent. Like a slice in Vertical Slice architecture, each module can determine its patterns and data sources. Yet, we must define a few common elements to assemble a Modular Monolith successfully. Here are a few items to consider to improve the chances of success of a Modular Monolith:

- The modules share a URL space.
- The modules share the configuration infrastructure.
- The modules share a single dependency injection object graph (one container).
- The modules share the inter-module communication infrastructure (event broker).

We can mitigate the first two elements using the module name as a discriminator. For example, using the `/{module name}/{module space}` URI space would yield the following results (`products` , `baskets` , and `customers` are modules):

- `/products`
- `/products/123`
- `/baskets`
- `/customers`

Using the module name as the top-level key of the configuration also makes it easy to avoid conflicts, like `{module name}:{key}` , or like the following JSON (say from the `appsettings.json` file):

```
"{module name}": {
    "{key}": "Module configs"
}
```

We can mitigate the last two elements by managing the shared code in a certain way. For example, limiting the amount of shared code reduces the chances of conflict between the modules. Yet, multiple modules globally configuring ASP.NET Core or a third-party library can result in conflicts; centralizing those configurations into the aggregator instead and treating them as a convention will help mitigate most issues. Cross-cutting concerns like exception handling, JSON serialization, logging, and security are great candidates for this.Lastly, sharing a single way to communicate between modules and configuring it in the aggregator will help mitigate the communication issues.Let's start planning the project next.

Planning the project

Planning is a crucial part of any software. Without a plan, you increase your chances of building the wrong thing. A plan does not guarantee success, but it improves your chances. Overplanning is the other side of this coin. It can lead to **analysis paralysis**, which means you may never even deliver your project or that it will take you so long to deliver it that it will be the wrong product because the needs changed along the way and you did not adapt.Here's a high-level way to accomplish planning a Modular Monolith. You don't have to execute all the steps in order. You can perform many of them iteratively, or multiple people or teams can even work on them in parallel:

1. Analyse and model the domain.
2. Identify and design the modules.
3. Identify the interactions between modules and design the integration events that cover those interactions.

Once we are done planning, we can develop and operate the application. Here are a few high-level steps:

1. Build and test the modules in isolation.
2. Build and test the module aggregator application that integrates one or more modules.
3. Deploy, operate, and monitor the monolith.

Implementing a Modular Monolith, like any program, is a step-by-step process. We plan, build, test, and then deploy it. Each part is simple enough on its own, and when we piece all of them together, we get an easy-to-maintain system. Even if continuously improving the application and refining the analysis and the model over time

should yield the best results, having a good idea of the high-level domain—the modules—and at least a vague view of their interactions before starting will help avoid mistakes and potentially significant refactoring further down the road. Here's a general representation of this process:

∞

Figure 20.2: A partial Agile and DevOps view of the Modular Monolith phases

Next, we plan our nano e-commerce application.

Analyzing the domain

As we continue to iterate over the nano e-commerce application we built in *Chapter 18* and *Chapter 19*, the domain analysis will be very short. Moreover, we are not expanding the application further than products and baskets because the application is already too large to fit in a single chapter. Of course, this time, we are making it a Modular Monolith. Here are the high-level entities and their relationships:

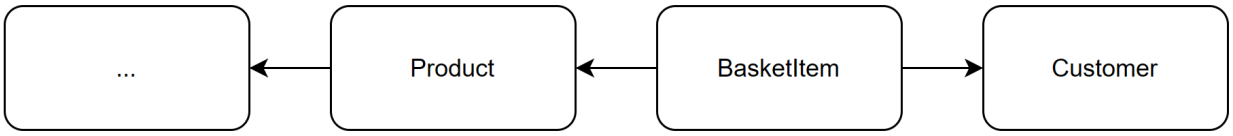


Figure 20.3: The high-level entities of our nono e-commerce app and their relationships

As the diagram shows, we have a `Product` entity that could benefit from more details like categories, hence the `...` box. We also have the `BasketItem` entity we use for people to save their shopping baskets to the database. Finally, a `Customer` entity represents the person to whom a shopping basket belongs.

> We did not implement a `Customer` class, yet the customer is conceptually present through the `CustomerId` property.

Next, we split this subset of the domain into modules.

Identifying the modules

Now that we have identified the entities, it is time to map them into modules. Let's start with the following diagram:

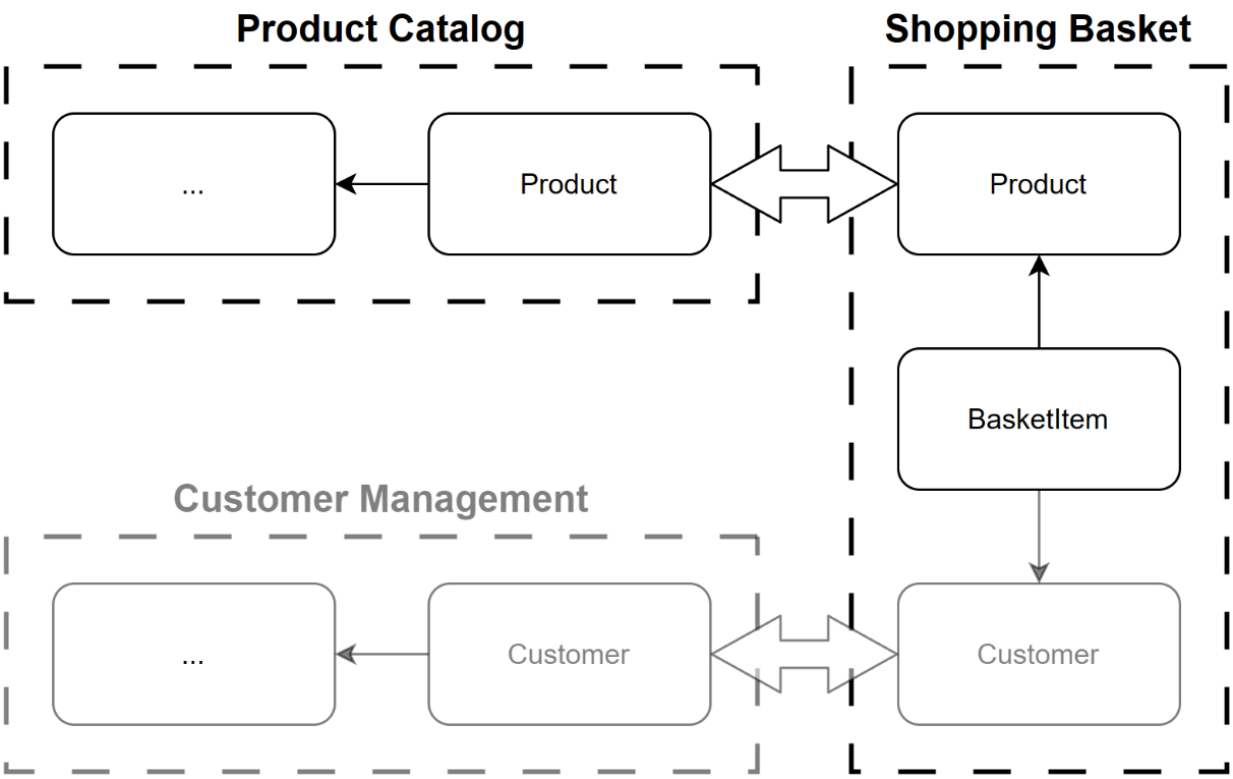


Figure 20.4: Modules (bounded context) separation and entities relationships.

As expected, we have a product catalog, a shopping basket, and a customer management modules. What's new here is the relationships between the entities. The catalog mainly manages the `Product` entity, yet the shopping basket needs to know about it to operate. The same logic applies to the `Customer` entity. In this case, the shopping basket only needs to know the unique identifier of each entity, but another module could need more information.Based on that high-level view, we need to create three modules. In our case, we continue with only two modules. In an actual application, we'd have more than three modules since we'd have to manage the purchases, the shipping, the inventory, and more.Let's look at the interactions between the modules.

Identifying the interactions between modules

Based on our analysis and limited to the two modules we are building, the shopping basket module needs to know about the products. Here's our `BasketItem` class:

```
public record class BasketItem(int CustomerId, int ProductId, int Quantity);
```

The preceding class shows that we only need to know about the unique product identifier. So, with an event-driven mindset, the shopping basket module wants to be notified when:

- A product is created.
- A product is deleted.

With those two events, the basket module can manage its cache of products and only allow customers to add existing items to their shopping basket. It can also remove items from customers' shopping baskets when unavailable. Here's a diagram representing these flows:

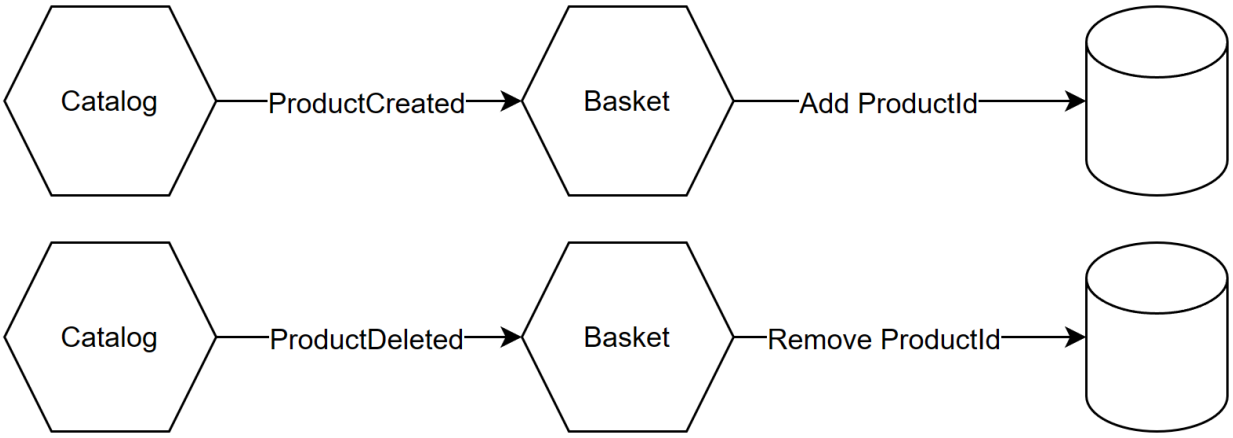


Figure 20.5: The integration event flows between the catalog and the basket modules.

Now that we have analyzed the domain and the modules, before building anything, let's define our stack.

Defining our stack

We know we are using ASP.NET Core and C#. We continue leveraging minimal APIs, yet MVC could achieve the same. We also continue to leverage *EF Core*, *ExceptionMapper*, *FluentValidation*, and *Mapperly*. But what about the modules and the other shared aspects of the project? Let's have a look.

The module structure

We are using a flexible yet straightforward module structure. You can organize your projects however you like; this is not a prescriptive approach. For example, you can get inspired by other architectural styles, like Clean Architecture, or invent your own based on your own experience, context, and work environment.In our case, I opted for the following directory structure:

- The `applications` directory contains the deployable applications, like the aggregator. We could add user interfaces in this directory or other deployable applications, like the BFF we built in Chapter 19. Each application is contained within its own subdirectory.
- The `modules` directory contains the modules, each within its own subdirectory.
- The `shared` directory contains the shared projects.

In real-world software, we could extend this setup and add `infrastructure`, `docs`, and `pipelines` directories to store our Infrastructure as Code (IaC), documentation, and CI/CD pipelines next to our code.

What I like about this mono-repo-inspired structure is that each module and application is self-contained. For example, the aggregator's API, contracts, and tests are next to each other:

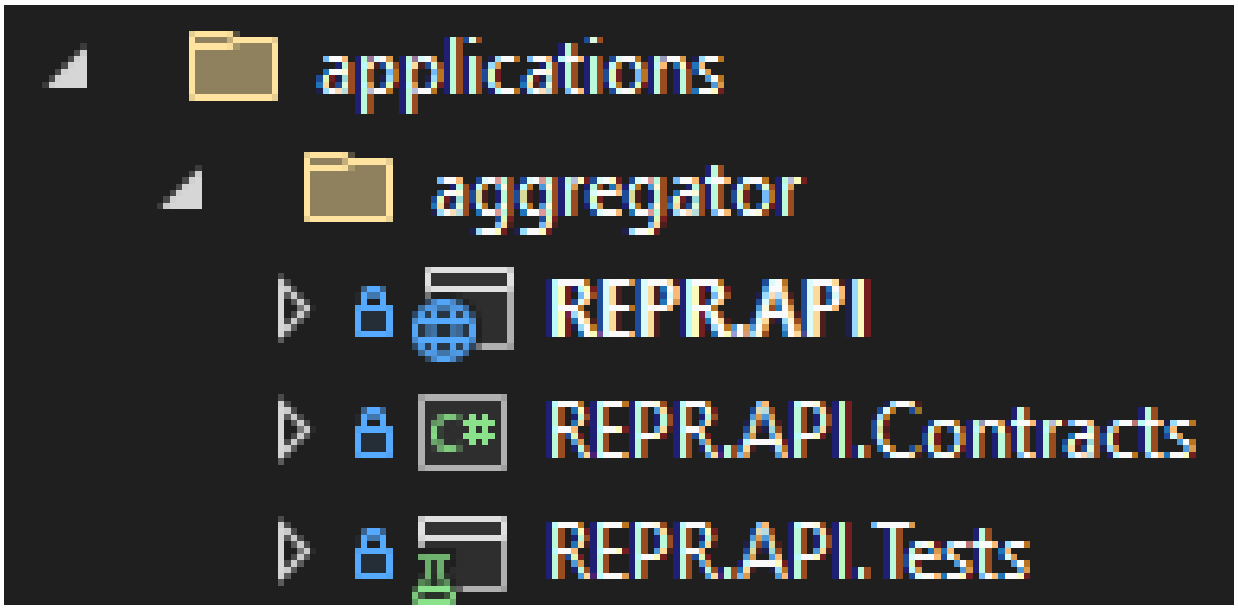


Figure 20.6: The aggregator's directory and project hierarchy.

The modules are organized similarly:

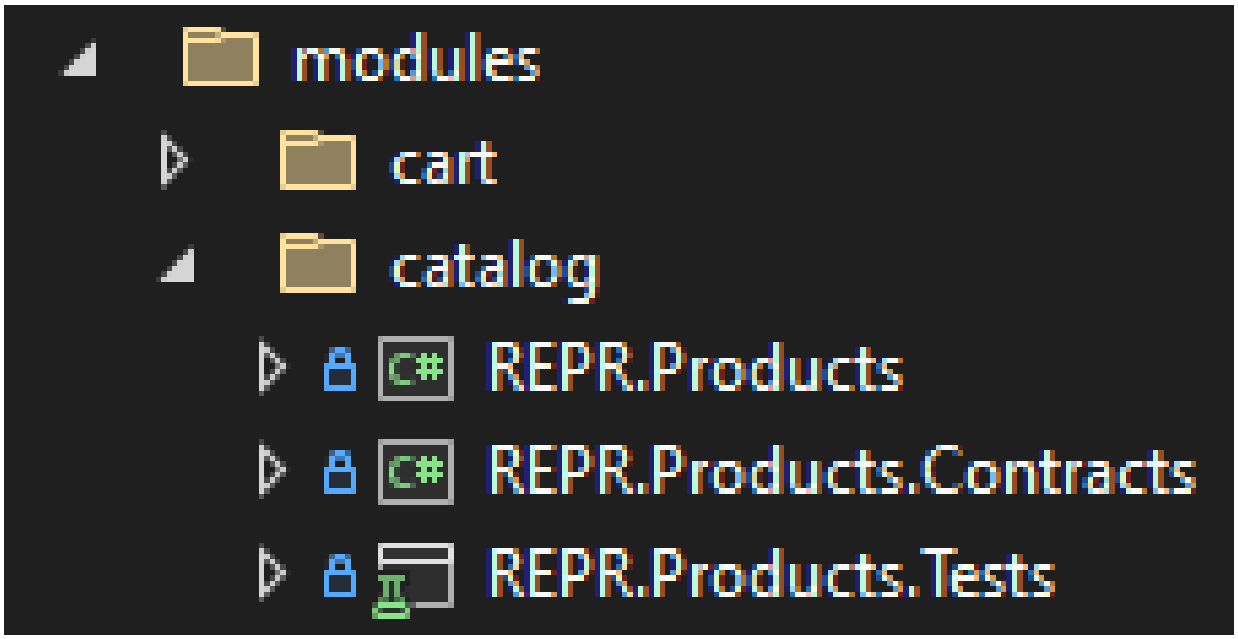


Figure 20.7: The modules' subdirectories and catalog's project hierarchy.

I kept the REPR prefix since it's based on Chapter 18's code, but I changed the code structure a bit. In this version, I got rid of the nested classes and created one class per file. This follows a more classic .NET convention and allows us to extract the API contracts to another assembly. If you remember, in Chapter 19, the BFF project referenced the two APIs to reuse their `Query`, `Command`, and `Response` contracts. We fix this problem through the `Contracts` class library projects in this solution.

> Why is it a problem? The BFF depends on all the microservices, including their logic and dependencies. This is a recipe to introduce unwanted coupling. Moreover, since it inherits all the dependencies transitively, it

increases its deployment size and vulnerability surface; more dependencies and more code means more possibilities for a malicious actor to find an exploitable hole.

On top of the API contracts, the `Contracts` projects also contain the integration events. We could have separated the API contracts and the integration events if the application was larger; in this case, we only have two integration events. Design choices must be taken relative to the current project and context.Let's explore the URI space next.

The URI space

The modules of this application follow the previously discussed URI space: `/{module name}/{module space}`. Each module has a `Constants` file at its root that looks like this:

```
namespace REPR.Baskets;
public sealed class Constants
{
    public const string ModuleName = nameof(Baskets);
}
```

We use the `ModuleName` constant in the `{module name}ModuleExtensions` files to set the URI prefix and tag the endpoints like this:

```
namespace REPR.Baskets;
public static class BasketModuleExtensions
{
    public static IEndpointRouteBuilder MapBasketModule(this IEndpointRouteBuilder endpoints)
    {
        _ = endpoints
            .MapGroup(Constants.ModuleName.ToLower())
            .WithTags(Constants.ModuleName)
            .AddFluentValidationFilter()
            // Map endpoints
            .MapFetchItems()
            .MapAddItem()
            .MapUpdateQuantity()
            .MapRemoveItem()
        ;
        return endpoints;
    }
}
```

With this in place, both modules self-register themselves in the correct URI space.

We can apply these types of conventions in many different ways. In this case, we opted for simplicity, which is the most error-prone, leaving the responsibility to the mercy of each module. With a more framework-oriented mindset, we could create a strongly typed module contract that gets loaded automatically, like an `IModule` interface. The aggregator could also create the root groups and enforce the URI space.

Next, we explore the data space.

The data space

Since we are following the microservices architecture tenets and each module should own its data, we must find a way to ensure our data contexts do not conflict.The project uses the EF Core in-memory provider to develop locally. For production, we plan on using SQL Server. One excellent way to ensure our `DbContext` classes do not conflict with each other is to create one database schema per context. Each module has one context, so one schema per module. We don't have to overthink this; we can reuse the same idea as the URI and leverage the module name. So, each module will group its tables under the `{module name}` schema instead of `dbo` (the default SQL Server schema).

We can apply different security rules and permissions to each schema in SQL Server, so we could craft a very secure database model by expanding this. For instance, we could employ multiple users possessing minimal

privileges, utilize different connection strings within the modules, etc.

In code, doing this is reflected by setting the default schema name in the `OnModelCreating` method of each `DbContext`. Here's the `ProductContext` class:

```
namespace REPR.Products.Data;
public class ProductContext : DbContext
{
    public ProductContext(DbContextOptions<ProductContext> options)
        : base(options) { }
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.HasDefaultSchema(Constants.ModuleName.ToLower());
    }
    public DbSet<Product> Products => Set<Product>();
}
```

The preceding code makes all `ProductContext`'s tables part of the `products` schema. We then apply the same for the basket module:

```
namespace REPR.Baskets.Data;
public class BasketContext : DbContext
{
    public BasketContext(DbContextOptions<BasketContext> options)
        : base(options) { }
    public DbSet<BasketItem> Items => Set<BasketItem>();
    public DbSet<Product> Products => Set<Product>();
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.HasDefaultSchema(Constants.ModuleName.ToLower());
        modelBuilder
            .Entity<BasketItem>()
            .HasKey(x => new { x.CustomerId, x.ProductId })
        ;
    }
}
```

The preceding code makes all `BasketContext`'s tables part of the `baskets` schema.Due to the schema, both contexts are safe from hindering the other. But wait! Both contexts have a `Products` table; what happens then? The catalog module uses the `products.products` table, while the basket module uses the `baskets.products` table. Different schema, different tables, case closed!

> We can apply these notions to more than Modular Monolith as it is general SQL Server and EF Core knowledge.

If you are using another relational database engine that does not offer schema or a NoSQL database, you must also think about this. Each NoSQL database has different ways to think about the data, and it would be impossible to cover them all here. The important piece is to find a discriminator that segregates the data of your modules. At the limit, it can even be one different database per module; however, this increases the operational complexity of the application.Next, we explore the message broker.

The message broker

To handle the integration events between the catalog and the basket modules, I decided to pick MassTransit. To quote their GitHub project:

> *MassTransit is a free, open-source distributed application framework for .NET. MassTransit makes it easy to create applications and services that leverage message-based, loosely-coupled asynchronous communication for higher availability, reliability, and scalability.*

I picked MassTransit because it is a popular project with 5,800 GitHub stars as of 2023 and supports many providers, including in-memory. Moreover, it offers many features that are way above our needs. Once again, we could have used anything. For example, MediatR could have also done the job.The `REPR.API` project—the aggregator—and the modules depend on the following NuGet package to use MassTransit:

```
<PackageReference Include="MassTransit" Version="8.1.0" />
```

Our usage is very simple; the aggregator registers and configures MassTransit like this:

```
builder.Services.AddMassTransit(x =>
{
    x.SetKebabCaseEndpointNameFormatter();
    x.UsingInMemory((context, cfg) =>
    {
        cfg.ConfigureEndpoints(context);
    });
    x.AddBasketModuleConsumers();
});
```

The highlighted line delegates the registration of event consumers to the basket module. The `AddBasketModuleConsumers` method is part of the `BasketModuleExtensions` class and contains the following code:

```
public static void AddBasketModuleConsumers(this IRegistrationConfigurator configurator)
{
    configurator.AddConsumers(typeof(ProductEventsConsumers));
}
```

The `ProductEventsConsumers` class manages the two events. The `AddConsumers` method is part of MassTransit. We explore the `ProductEventsConsumers` class in the project section.

> We would register the event consumers of other modules here if we had more. Yet the delegation of that registration to each module makes it modular.

Next, we write some C# code to transform our microservices application into a Modular Monolith.

Project—Modular Monolith

This project has the same building block as *Chapter 18* and *Chapter 19*, but we use the Modular Monolith approach. On top of the previous versions, we leverage events to enable the shopping basket to validate the existence of a product before allowing customers to add it to their shopping basket.

> The complete source code is available on GitHub: https://adpg.link/gyds
>
> > The test projects in the solution are empty. They only exist for the organizational aspect of the solution. As an exercise, you can migrate the tests from *Chapter 18* and adapt them to this new architectural style.

Let’s start with the communication piece.

Sending events from the catalog module

For the catalog to communicate the events that the basket module needs, it must define the following new operations:

- Create products
- Delete products

Here are the API contracts we must create in the `REPR.Products.Contracts` project to support those two operations:

```
namespace REPR.Products.Contracts;
public record class CreateProductCommand(string Name, decimal UnitPrice);
public record class CreateProductResponse(int Id, string Name, decimal UnitPrice);
public record class DeleteProductCommand(int ProductId);
public record class DeleteProductResponse(int Id, string Name, decimal UnitPrice);
```

The API contracts should look very familiar by now and are similar to those from previous chapters. We then need the following two event contracts:

```
namespace REPR.Products.Contracts;
public record class ProductCreated(int Id, string Name, decimal UnitPrice);
public record class ProductDeleted(int Id);
```

The two event classes are also very straightforward, but their name is in the past tense because an event happened in the past. So, the module creates the product and then notifies its subscribers that a product was created (in the past). Exactly like we studied in *Chapter 19*, *Introduction to Microservices Architecture*. Moreover, the events are simple data containers, like an API contract—a DTO—is.How do we send those events? Let's have a look at the `CreateProductHandler` class:

```
namespace REPR.Products.Features;
public class CreateProductHandler
{
    private readonly ProductContext _db;
    private readonly CreateProductMapper _mapper;
    private readonly IBus _bus;
    public CreateProductHandler(ProductContext db, CreateProductMapper mapper, IBus bus)
    {
        _db = db ?? throw new ArgumentNullException(nameof(db));
        _mapper = mapper ?? throw new ArgumentNullException(nameof(mapper));
        _bus = bus ?? throw new ArgumentNullException(nameof(bus));
    }
    public async Task<CreateProductResponse> HandleAsync(CreateProductCommand command, Cancellat
    {
        var product = _mapper.Map(command);
        var entry = _db.Products.Add(product);
        await _db.SaveChangesAsync(cancellationToken);
        var productCreated = _mapper.MapToIntegrationEvent(entry.Entity);
        await _bus.Publish(productCreated, CancellationToken.None);
        var response = _mapper.MapToResponse(entry.Entity);
        return response;
    }
}
```

In the preceding code, we inject an `IBus` interface from MassTransit in the `CreateProductHandler` class. We also inject an object mapper and an EF Core `ProductContext`. The `HandleAsync` method does the following:

1. Creates the product and saves it to the database.
2. Publishes a `ProductCreated` event (highlighted code).
3. Returns a `CreateProductResponse` instance based on the new product.

The `Publish` method sends the event to the configured pipe, which is in memory in our case. The code passes a `CancellationToken.None` argument here because we don't want this operation to be canceled by any external force because the change is already saved in the database.Because of the mapping code, the publishing code may be hard to understand in a book. The `MapToIntegrationEvent` method converts a `Product` object to a `ProductCreated` instance, so the `productCreated` variable is of type `ProductCreated`. Here's the *Mapperly* mapper class with that method highlighted:

```
namespace REPR.Products.Features;
[Mapper]
public partial class CreateProductMapper
{
    public partial Product Map(CreateProductCommand product);
    public partial ProductCreated MapToIntegrationEvent(Product product);
    public partial CreateProductResponse MapToResponse(Product product);
}
```

The `DeleteProductHandler` class follows a similar pattern but publishes the `ProductDeleted` event instead.Now, let's explore how the basket module consumes those events.

Consuming the events from the basket module

The basket module wants to cache existing products. We can achieve this in different ways. In this case, we create the following `Product` class that we persist in the database:

```
namespace REPR.Baskets.Data;
public record class Product(int Id);
```

To make use of it, we must expose the following property from the `BasketContext` class:

```
public DbSet<Product> Products => Set<Product>();
```

Then, we can start to leverage this cache. Firstly, we must populate it when the catalog module creates a product and remove that product when deleted. The `ProductEventsConsumers` class handles both events. Here's the skeleton of this class:

```
using REPR.Products.Contracts;
namespace REPR.Baskets.Features;
public class ProductEventsConsumers : IConsumer<ProductCreated>, IConsumer<ProductDeleted>
{
    private readonly BasketContext _db;
    private readonly ILogger _logger;
    public ProductEventsConsumers(BasketContext db, ILogger<ProductEventsConsumers> logger)
    {
        _db = db ?? throw new ArgumentNullException(nameof(db));
        _logger = logger ?? throw new ArgumentNullException(nameof(logger));
    }
    public async Task Consume(ConsumeContext<ProductCreated> context)
    {...}
    public async Task Consume(ConsumeContext<ProductDeleted> context)
    {...}
}
```

The highlighted code represents the two event handlers. The `IConsumer<TMessage>` interface contains the `Consume` method. Implementing the interface twice with a different `TMessage` generic parameter prescribes implementing the two `Consume` methods in the `ProductEventsConsumers` class. Each method handles its own event.When a product is created in the product module, the following method is executed in the basket module (I removed the logging code for brevity):

```
public async Task Consume(ConsumeContext<ProductCreated> context)
{
    var product = await _db.Products.FirstOrDefaultAsync(
        x => x.Id == context.Message.Id,
        cancellationToken: context.CancellationToken
    );
    if (product is not null)
    {
        return;
    }
    _db.Products.Add(new(context.Message.Id));
    await _db.SaveChangesAsync();
}
```

The preceding code adds a new product to the database if it does not exist. If it does exist, it does nothing.

> We could add telemetry here to log and flag the conflict and explore how to solve it. If the product already exists, the event was received twice, which may indicate an issue with our system.

Here's the flow of what happens when an employee creates a new product:

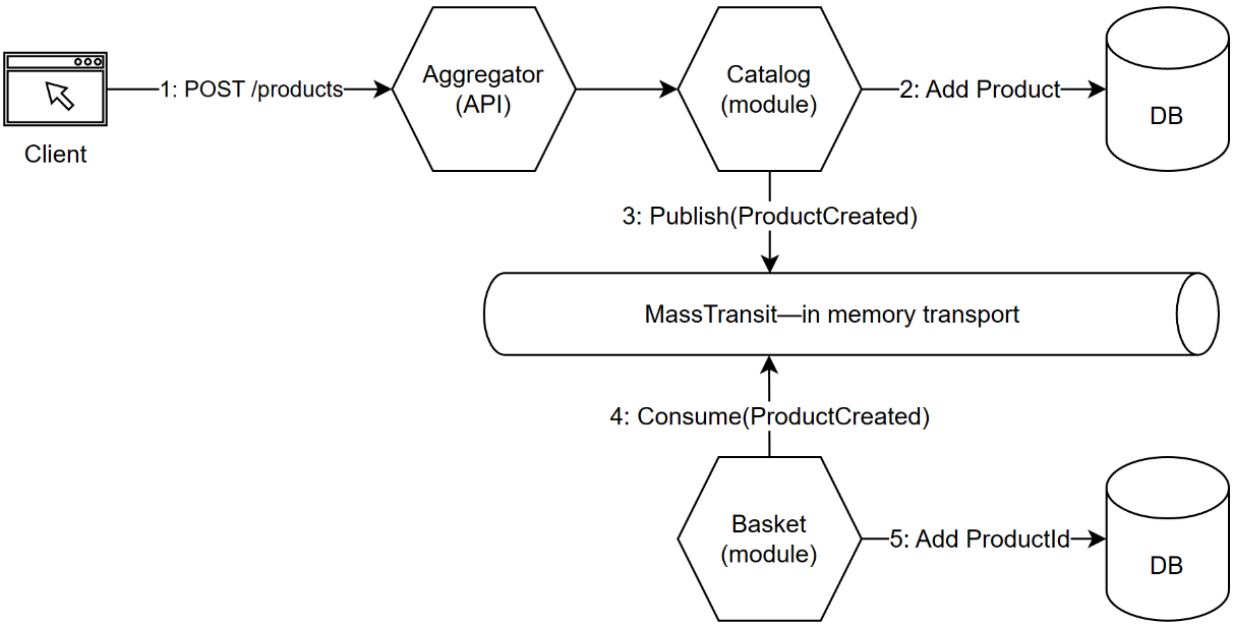


Figure 20.8: A diagram showcasing the sequence of operations when someone adds a product to the system.

Let's review the operations:

1. A client sends a POST request to the API (the aggregator application). The catalog module receives the request.
2. The catalog creates the product and adds it to the database.
3. The catalog then publishes the `ProductCreated` event using MassTransit.
4. In the basket module, the `Consume` method of the `ProductEventsConsumers` class gets called by MassTransit in reaction to the `ProductCreated` event.
5. The basket module adds the product's identifier to the database to its materialized view.

Now that we know how the `ProductEventsConsumers` class handles the `ProductCreated` events, let's explore the `ProductDeleted` events (logging code omitted for brevity):

```
public async Task Consume(ConsumeContext<ProductDeleted> context)
{
    var item = await _db.Products.FirstOrDefaultAsync(
        x => x.Id == context.Message.Id,
        cancellationToken: context.CancellationToken
    );
    if (item is null)
    {
        return;
    }
    // Remove the products from existing baskets
    var existingItemInCarts = _db.Items
        .Where(x => x.ProductId == context.Message.Id);
    var count = existingItemInCarts.Count();
    _db.Items.RemoveRange(existingItemInCarts);
    // Remove the product from the internal cache
    _db.Products.Remove(item);
    // Save the changes
    await _db.SaveChangesAsync();
}
```

The preceding `Consume` method removes the products from people's shopping baskets and the materialized view. If the product does not exist, the method does nothing because there's nothing to handle.A similar flow happens when an employee deletes a product from the catalog and publishes a `ProductDeleted` event. The basket module

then reacts to the event and updates its cache (materialized view).Now that we have explored this exciting part of the project, let’s look at the aggregator.

Inside the aggregator

The aggregator application is like an empty shell that loads the other assemblies and configures the cross-cutting concerns. It references the modules, then assembles and boots the application. Here’s the first part of the `Program.cs` file:

```
using FluentValidation;
using FluentValidation.AspNetCore;
using MassTransit;
using REPR.API.HttpClient;
using REPR.Baskets;
using REPR.Products;
using System.Reflection;
var builder = WebApplication.CreateBuilder(args);
// Register fluent validation
builder.AddFluentValidationEndpointFilter();
builder.Services
    .AddFluentValidationAutoValidation()
    .AddValidatorsFromAssemblies(new[] {
        Assembly.GetExecutingAssembly(),
        Assembly.GetAssembly(typeof(BasketModuleExtensions)),
        Assembly.GetAssembly(typeof(ProductsModuleExtensions)),
    })
;
builder.AddApiHttpClient();
builder.AddExceptionMapper();
builder
    .AddBasketModule()
    .AddProductsModule()
;
builder.Services.AddMassTransit(x =>
{
    x.SetKebabCaseEndpointNameFormatter();
    x.UsingInMemory((context, cfg) =>
    {
        cfg.ConfigureEndpoints(context);
    });
    x.AddBasketModuleConsumers();
});
```

The preceding code registers the following:

- *FluentValidation*; it also scans the assemblies for validators.
- The `IWebClient` interface that we use to seed the database (the `AddApiHttpClient` method).
- *ExceptionMapper*.
- The modules' dependencies (highlighted)
- *MassTransit*; it also registers the consumers from the basket module (highlighted).

The container we register those dependencies into is also used for and by the modules because the aggregator is the host. Those dependencies are shared across the modules.The second part of the `Program.cs` file is the following:

```
var app = builder.Build();
app.UseExceptionMapper();
app
    .MapBasketModule()
    .MapProductsModule()
;
// Convenience endpoint, seeding the catalog
app.MapGet("/", async (IWebClient client, CancellationToken cancellationToken) =>
{
    await client.Catalog.CreateProductAsync(new("Banana", 0.30m), cancellationToken);
    await client.Catalog.CreateProductAsync(new("Apple", 0.79m), cancellationToken);
    await client.Catalog.CreateProductAsync(new("Habanero Pepper", 0.99m), cancellationToken);
```

```
    return new
    {
        Message = "Application started and catalog seeded. Do not refresh this page, or it will r
    };
});
app.Run();
```

The preceding code registers the `ExceptionMapper` middleware, then the modules. It also adds a seeding endpoint (highlighted). If you remember, we were seeding the database using the `DbContext` in the previous versions. However, since the basket module needs to receive the events from the catalog to build a materialized view, it is more convenient to seed the database through the catalog module. In this version, the program seeds the database when a client hits the `/` endpoint. By default, when starting the application, Visual Studio should open a browser at that URL, which will seed the database.

> Seeding the database by sending a GET request to the `/` endpoint is very convenient for an academic scenario where we use in-memory databases. However, this could be disastrous in a production environment because it would reseed the database whenever someone hits that endpoint.

Let's explore the `IWebClient`, next.

Exploring the REST API HttpClient

In the `shared` directory, the `REPR.API.HttpClient` project contains the REST API client code. The code is very similar to the previous project, but the `IProductsClient` now exposes a create and delete method (highlighted):

```
using Refit;
using REPR.Products.Contracts;
namespace REPR.API.HttpClient;
public interface IProductsClient
{
    [Get("/products/{query.ProductId}")]
    Task<FetchOneProductResponse> FetchProductAsync(
        FetchOneProductQuery query,
        CancellationToken cancellationToken);
    [Get("/products")]
    Task<FetchAllProductsResponse> FetchProductsAsync(
        CancellationToken cancellationToken);
    [Post("/products")]
    Task<CreateProductResponse> CreateProductAsync(
        CreateProductCommand command,
        CancellationToken cancellationToken);
    [Delete("/products/{command.ProductId}")]
    Task<DeleteProductResponse> DeleteProductAsync(
        DeleteProductCommand command,
        CancellationToken cancellationToken);
}
```

On top of this, the project only references the `Contracts` projects, limiting its dependencies to the classes it needs. It does not reference the complete modules anymore. This makes this project easy to reuse. For example, we could build another project, like a user interface (UI), then reference and use this typed client to query the API (modular monolith) from that .NET UI.Since we created this client for a microservices application, we have two base downstream service URLs—one for the product microservice and one for the basket microservice. This nuance suits us well since we may want to migrate our monolith to microservices later. In the meantime, all we have to do is set the two keys to the same host, like the following `appsettings.json` file from the aggregator:

```
{
  "Downstream": {
    "Baskets": {
      "BaseAddress": "https://localhost:7164/"
    },
    "Products": {
      "BaseAddress": "https://localhost:7164/"
    }
  }
}
```

With these configurations, the `AddApiHttpClient` method configures two `HttpClient` with the same `BaseAddress` value, like this:

```
using Microsoft.AspNetCore.Builder;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Refit;
namespace REPR.API.HttpClient;
public static class ApiHttpClientExtensions
{
    public static WebApplicationBuilder AddApiHttpClient(this WebApplicationBuilder builder)
    {
        const string basketsBaseAddressKey = "Downstream:Baskets:BaseAddress";
        const string productsBaseAddressKey = "Downstream:Products:BaseAddress";
        var basketsBaseAddress = builder.Configuration
            .GetValue<string>(basketsBaseAddressKey) ?? throw new BaseAddressNotFoundException(ba
        var productsBaseAddress = builder.Configuration
            .GetValue<string>(productsBaseAddressKey) ?? throw new BaseAddressNotFoundException(p
        builder.Services
            .AddRefitClient<IBasketsClient>()
            .ConfigureHttpClient(c => c.BaseAddress = new Uri(basketsBaseAddress))
        ;
        builder.Services
            .AddRefitClient<IProductsClient>()
            .ConfigureHttpClient(c => c.BaseAddress = new Uri(productsBaseAddress))
        ;
        builder.Services.AddTransient<IWebClient, DefaultWebClient>();
        return builder;
    }
}
```

And voilà, we have a functional typed client we can reuse and a working modular monolith. Let's test this out next.

Sending HTTP requests to the API

Now that we have a working modular monolith, we can reuse similar HTTP requests than the previous versions.At the root of the REPR.API project, we can use the following:

- The `API-Products.http` file contains requests to the product module.
- The `API-Baskets.http` file contains requests to the basket module.

A new outcome in this API compared to the previous versions is when we try to add a product that does not exist in the catalog to our basket, like the following request:

```
POST https://localhost:7164/baskets
Content-Type: application/json
{
    "customerId": 1,
    "productId": 5,
    "quantity": 99
}
```

Since the product `5` does not exist, the API returns the following:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "productId": [
      "The Product does not exist."
    ]
  }
}
```

That error confirms that the validation works as expected. But how are we validating this?

Validating the existence of a product

In the *add item* feature, the `AddItemValidator` class ensures the product exists while validating the `AddItemCommand` object. To achieve this, we leverage the `MustAsync` and `WithMessage` methods from FluentValidation. Here's the code:

```
namespace REPR.Baskets.Features;
public class AddItemValidator : AbstractValidator<AddItemCommand>
{
    private readonly BasketContext _db;
    public AddItemValidator(BasketContext db)
    {
        _db = db ?? throw new ArgumentNullException(nameof(db));
        RuleFor(x => x.CustomerId).GreaterThan(0);
        RuleFor(x => x.ProductId)
            .GreaterThan(0)
            .MustAsync(ProductExistsAsync)
            .WithMessage("The Product does not exist.")
        ;
        RuleFor(x => x.Quantity).GreaterThan(0);
    }
    private async Task<bool> ProductExistsAsync(int productId, CancellationToken cancellationTok
    {
        var product = await _db.Products
            .FirstOrDefaultAsync(x => x.Id == productId, cancellationToken);
        return product is not null;
    }
}
```

The preceding code implements the same rules as the previous versions but also calls the `ProductExistsAsync` method that fetches the requested product from the cache. If the result is `false` , the validation fails with the message "*The Product does not exist.*".Here's the resulting flow of this change:

1. The client calls the API.
2. The validation middleware calls the validator.
3. The validator fetches the record from the database and validates its existence.
4. If the product does not exist, the request is short-circuited here and returned to the client. Otherwise, it continues to the `AddItemHandler` class.

With this in place, we covered all new scenarios from this project. We also explored how the aggregator connects the modules together and how easy it is to keep our modules independent.Next, we get the BFF back into the project and explore how to transition our modular monolith to a microservices architecture.

Transitioning to Microservices

You don't have to transition your monolith to microservices; deploying a monolith is fine if it fits your needs. However, if ever you need to, you could shield your aggregator with a gateway or a reverse proxy so you can extract modules into their own microservices and reroute the requests without impacting the clients. This would also allow you to gradually transfer the traffic to the new service while keeping the monolith intact in case of an unexpected failure.In a modular monolith, the aggregator registers most dependencies, so when migrating, you must also migrate this shared setup. One way to not duplicate code would be to create and reference a shared assembly containing those registrations. This makes it easier to manage the dependencies and shared configurations, but it is also coupling between the microservices and the aggregator. Leveraging the code of this shared assembly is even easier when the microservices are part of the same mono-repo; you can reference the project directly without managing a NuGet package. Yet, when you update the library, it updates all microservices and the monolith, voiding the deployment independence of each application.Once again, consider keeping the monolith intact versus the operational complexity that deploying microservices would bring.In the solution, the `REPR.BFF` project is a migration of *Chapter 19* code. The code and the logic are almost the same. It leverages the `REPR.API.HttpClient` project and configures the downstream base addresses in its

`appsettings.Development.json` file. The `REPR.BFF.http` file at the root of the project contains a few requests to test the application.

> The code is available on GitHub: https://adpg.link/bn1F. I suggest playing with the live application, which will yield a better result than copying the code in the book a second time. The relationships between the projects are also more evident in Visual Studio than written in a book. Feel free to refactor the projects, add tests, or add features. The best way to learn is to practice!

Next, let's have a look at challenges and pitfalls.

Challenges and Pitfalls

It is a misconception that monoliths are only suitable for small projects or small teams. A well-built monolith can go a long way, which is what a modular structure and a good analysis bring. Modular Monoliths can be very powerful and work well for different sizes of projects. It is essential to consider the pros and cons of each approach when selecting the application pattern for a project. Maybe a cloud-native, serverless, microservices application is what your next project needs, but perhaps a well-designed Modular Monolith would achieve the same performance at a tremendously lower cost.To help you make those decisions, here are some potential challenges and how to avoid or mitigate them:

- **Too much complexity within a module:** One risk is making the modules too complex. Like any piece of code, it becomes harder to manage a module if it does too many things. Moreover, the bigger the piece of software, the more chance the shiny initial design starts to bleed into a big ball of mud. We can avoid this by keeping each module focused on a specific part of the domain and by applying the SRP.
- **Poorly defined module boundaries:** If the boundaries between modules are unclear, it can create problems. For example, if two modules do similar things, it can confuse developers and lead to one part of the system depending on the wrong module. Another example is when two modules that should be part of the same bounded context are separated. In that case, the chances are that the two modules will interact a lot with each other, creating chattiness and significantly increasing the coupling between them. We avoid this with good planning, domain analysis, and ensuring each module has a specific job (SRP).
- **Scaling:** Even though Modular Monoliths are easier to manage, they carry the monolith issues when they must scale up. We must deploy the whole application, so we can't scale modules independently. Moreover, even if modules own their data, they probably share one database, which must also be scaled as a whole. If scaling the whole system is impossible, we can migrate specific modules to microservices. We can also extract in-memory services to distributed ones, like using a distributed cache instead of a memory cache, leveraging a cloud-based event broker instead of an in-memory one, or even having compute or data-intensive modules start using their own database so they partially scale independently. However, these solutions make the deployment and the infrastructure more complex, gradually moving towards complex infrastructure, which defeats the monolith's benefits.
- **Eventual consistency:** Keeping data in sync between modules can be challenging. We can handle this using event-driven architecture. Using an in-memory message broker is low-latency and high-fidelity (no network involved), which is a good first step towards learning and dealing with eventual consistency and breaking the monolith into microservices (if the transition is ever needed). However, a more resilient transport is recommended for production, so the system resists to failures, increasing the synchronization latency. You do not have to use events if you prefer to remove this complexity; a well-designed relational database can do the trick.
- **Transition to microservices:** Moving an application to microservices architecture after the fact can be a significant undertaking. However, starting with an event-powered Modular Monolith should make the journey less painful.

While Modular Monoliths offer many benefits, they can also pose challenges. The key is to plan well, keep things simple, and be ready to adapt as the program evolves.

Conclusion

In this chapter, we learned about the Modular Monolith architectural style, which blends the simplicity of monolithic architectures with the flexibility of microservices. This architectural style organizes software applications into distinct, loosely coupled modules, each responsible for a specific business capability. Unlike microservices, we deploy these modules as a single unit, like a monolith. We discussed the benefits of Modular Monoliths, including easier overall management, sound development and testing experiences, cost-effectiveness, and a simplified deployment model.We saw that a Modular Monolith comprises modules, a module aggregator, and an inter-module communication infrastructure—event-driven in this case.We learned that analyzing the domain, designing the modules, and identifying the interactions between modules before starting the development improves the chances of success of the product.We touched on transitioning from a Modular Monolith to a microservices architecture, which involves extracting modules into separate microservices and rerouting requests.We also highlighted the importance of knowing potential challenges and pitfalls. These include module complexity, poorly defined module boundaries, scaling limitations, and eventual consistency caused by an asynchronous communication model.

Questions

Let’s take a look at a few practice questions:

1. What are the core principles of a Modular Monolith?
2. What are the advantages of Modular Monoliths?
3. What are traditional Monoliths?
4. Are poorly defined module boundaries indeed beneficial to a Modular Monolith?
5. Is it true that transitioning an application to microservices architecture can be a significant undertaking?

Further reading

Here is a link to build upon what we learned in the chapter:

- Microservices Aggregation (Modular Monolith): https://adpg.link/zznM
- When (modular) monolith is the better way to build software: https://adpg.link/KBGB
- Source code: https://adpg.link/gyds

An end is simply a new beginning

This may be the end of the book, but it is also the continuation of your journey into software architecture and design. I hope you found this to be a refreshing view of design patterns and how to design SOLID apps.Depending on your goals and current situation, you may want to explore one or more application-scale design patterns in more depth, start your next personal project, start a business, apply for a new job, or all of those. No matter your goals, keep in mind that designing software is technical but also an art. There is rarely one way to implement a feature but multiple acceptable ways. Every decision has trade-offs, and experience is your best friend, so keep programming, learn from your mistakes, become better, and continue. The path to mastery is a never-ending continuous learning loop. Remember that we are all born knowing next to nothing, so not knowing something is expected; we need to learn. Please ask questions, read, experiment, learn, and share your knowledge with others. Explaining a concept to someone is extremely rewarding and reinforces your own learning and knowledge.Now that this book is complete, you may find interesting articles on my blog (https://adpg.link/blog). Feel free to reach out on Discord, Twitter `@CarlHugoM` (https://adpg.link/twit), or LinkedIn (https://adpg.link/edin). I hope you found the book educational and approachable and that you learned many things. I wish you success in your career.

Answers

1. We must treat each module as a microservice and deploy the application as a single unit—a monolith.
2. A few moving parts make the application simpler. Each module is independent, making modules loosely coupled. Its simple deployment model leads to cost-effective hosting and ease of deployment.

3. Traditional monolithic architectures build the application as a single, indivisible unit, often resulting in tightly coupled functionalities.
4. False. Poorly defined module boundaries hinder the health of the application.
5. True. Even if a well-conceived Modular Monolith can help, the transition will be a journey.